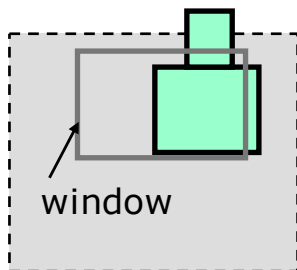
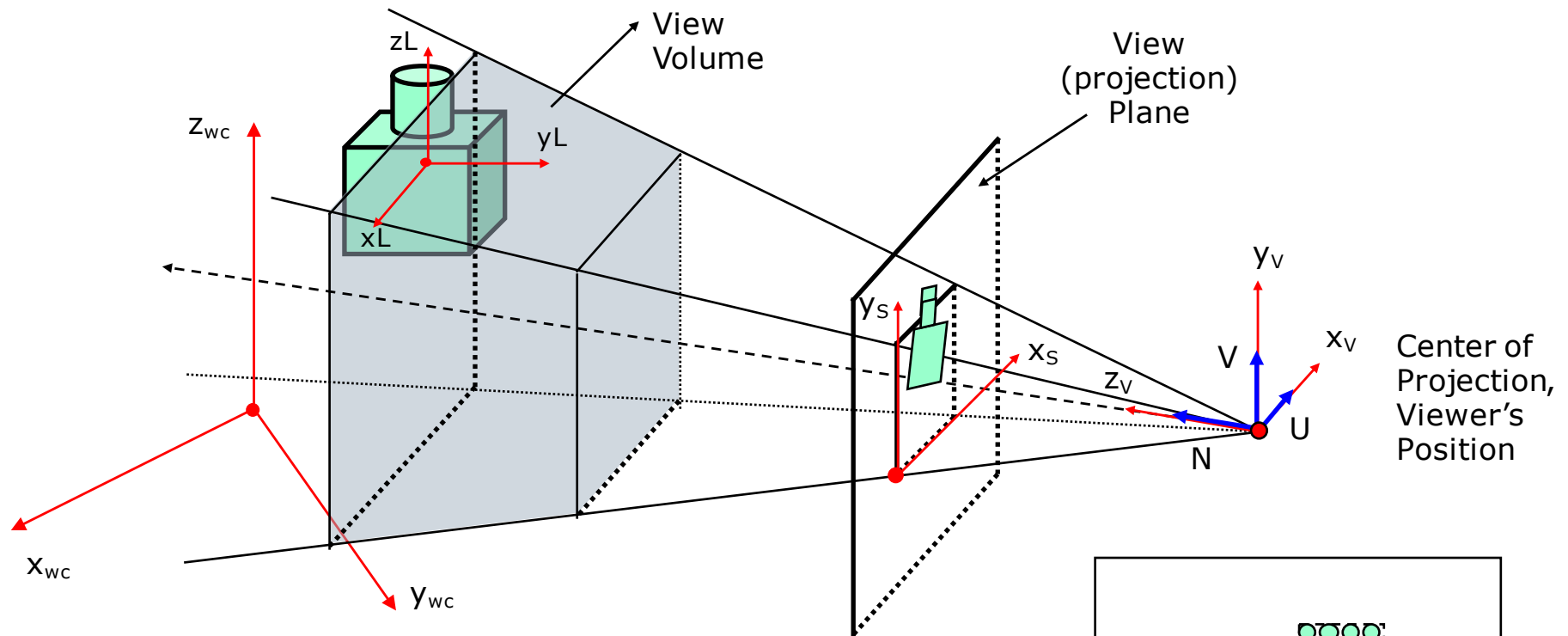


OpenGL – Open Graphics Library

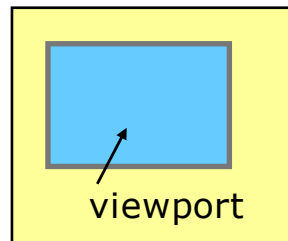
References

- ❑ Shreiner D., Sellers G., Kessenich J., Licea-Kane B.,
“*OpenGL Programming Guide*”, Addison-Wesley, 2013.
- ❑ OpenGL Web site, <http://www.opengl.org/>
- ❑ Sample code of OpenGL examples, <http://www.opengl-redbook.com/>

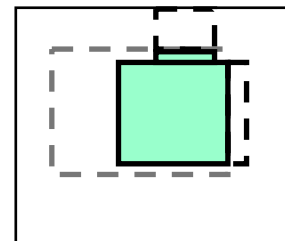
Graphics rendering pipeline



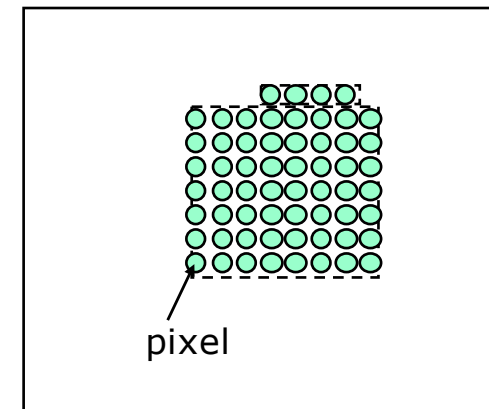
Window definition
in projection plane



Viewport definition
in projection plane

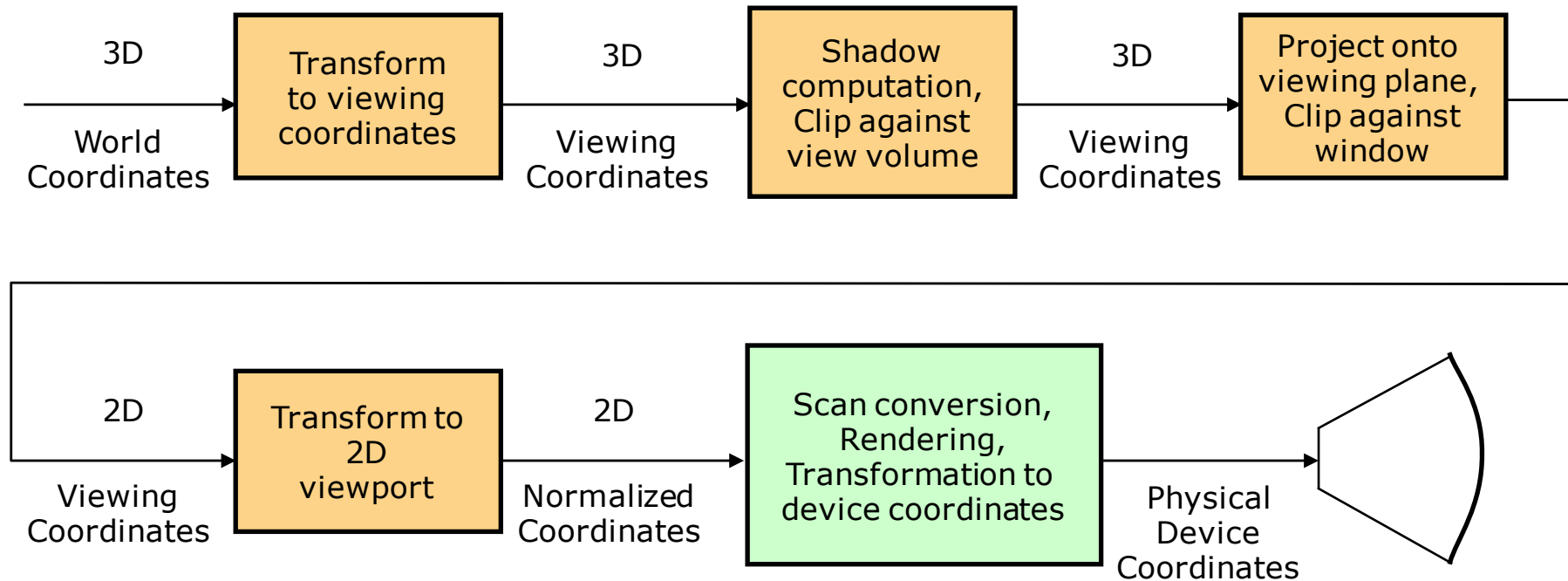


Window's content
inside the viewport



Scan conversion onto the
raster screen

Graphics rendering pipeline



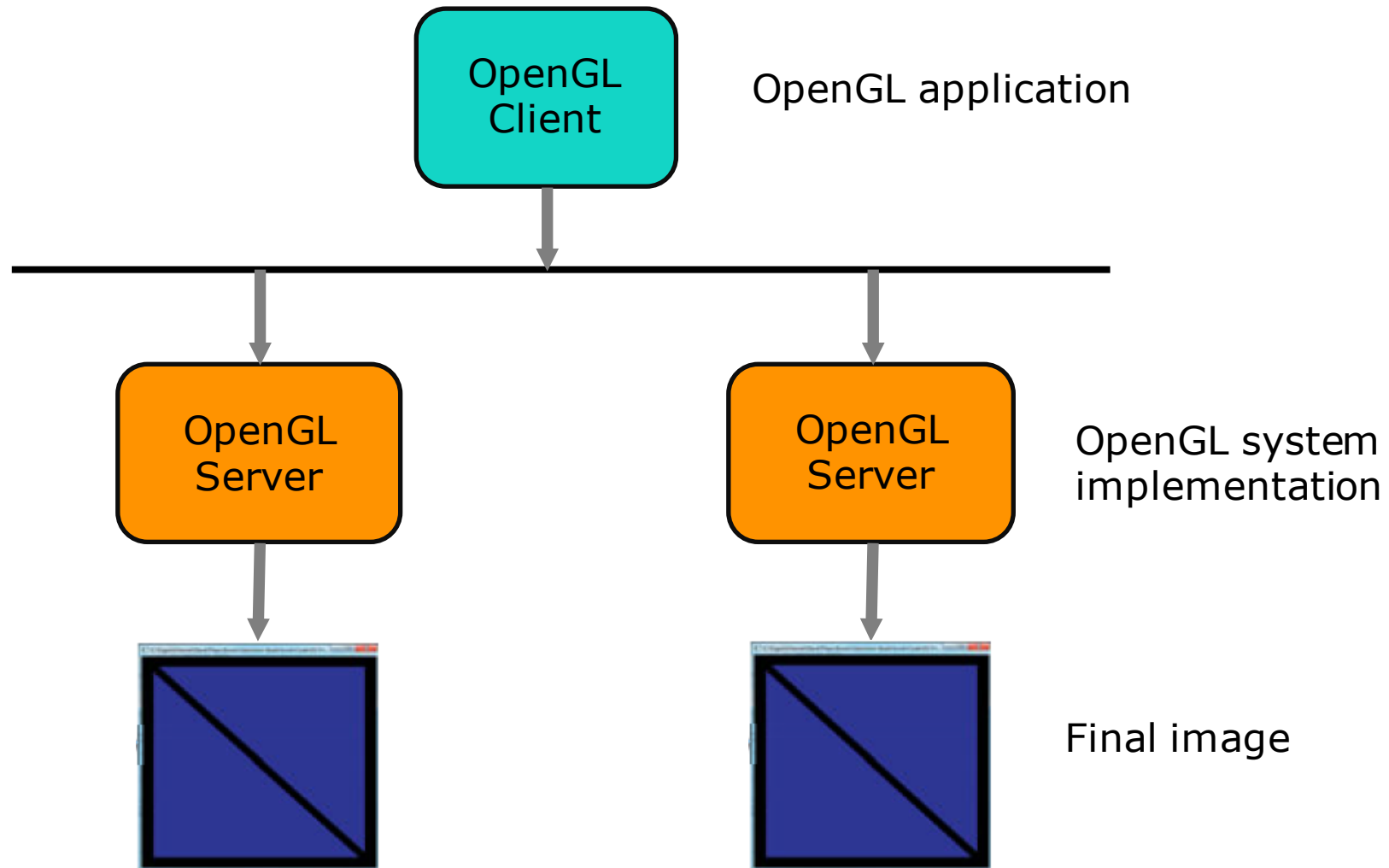
Overview

- ❑ Version 1.0 released in January 1992 at Silicon Graphics Computer Systems
- ❑ API - application programming interface - software library (~500 distinct commands) for accessing features in graphics hardware
- ❑ Designed as a streamlined, hardware-independent interface or entirely in software (if no graphics hardware is present in the system) independent of a computer's operating or windowing system
- ❑ It does not include
 - functions for performing windowing tasks
 - functions for processing user input
 - functionality for describing models of three-dimensional objects, or operations for reading image files (e.g. jpeg)

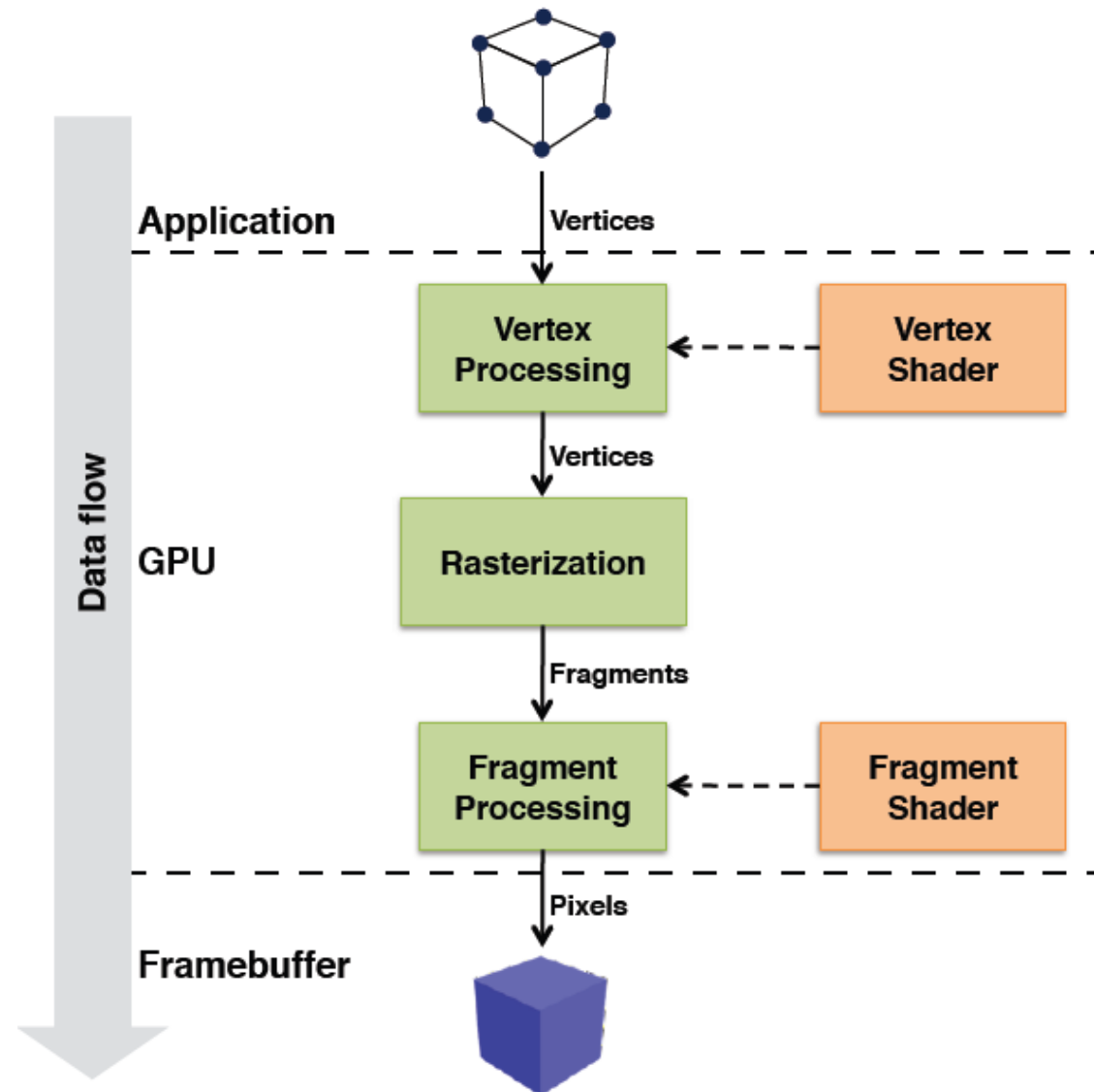
OpenGL system architecture

- ❑ Client-server system
- ❑ *Client* – OpenGL application developed by programmer
- ❑ *Server* - OpenGL implementation provided by the manufacturer of your computer graphics hardware
- ❑ In some implementations of OpenGL
 - - Client and server run on the same machine (e.g. Mac OS, Windows). The command generated by the client are executed by the server (e.g. GPU modules), which generates the final image within windows.
 - - Client and server run on different machines (e.g. X Window System), which are connected by a network. The OpenGL commands are converted into a graphics window system specific protocol that is transmitted to the server via their shared network, where they are executed to produce the final image.

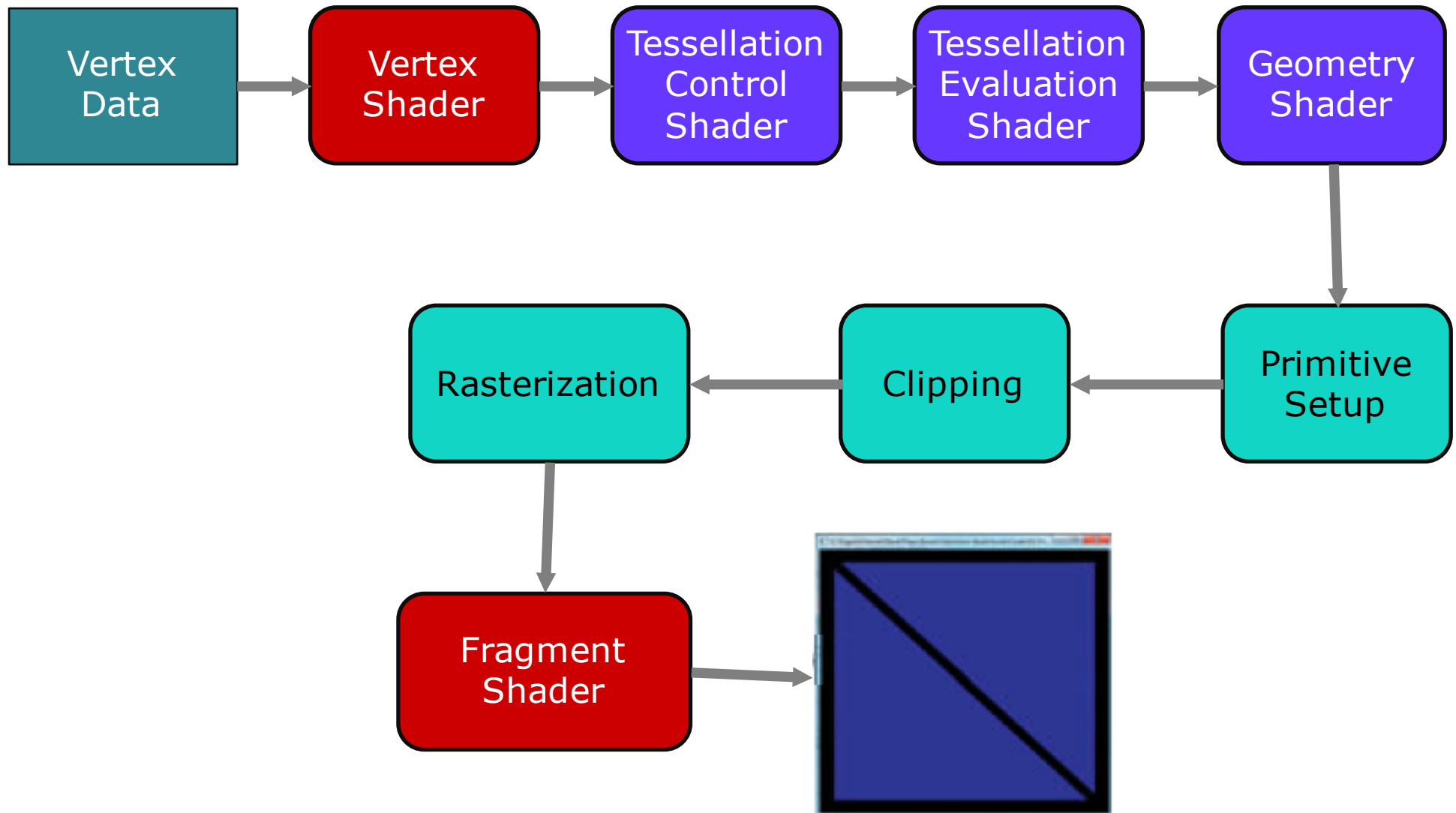
Client – server architecture



Basic graphics pipeline



OpenGL pipeline



OpenGL pipeline description

Main stages:

1. OpenGL pipeline begins with geometric data (vertices and geometric primitives)
2. Processes data through a sequence of shader stages: vertex shading, tessellation shading (control and evaluation), and finally geometry shading
3. The rasterizer generates fragments for any primitive that is inside of the clipping region, and executes a fragment shader for each of the generated fragments

Observation:

- Vertex shaders and fragment shaders must be included
- Tessellation and geometry shaders are optional

Prepare and send data to OpenGL

□ Preparing data

- OpenGL requires that all data be stored in *buffer objects*
- Buffer object are just chunks of memory managed by the OpenGL server
- Populating these buffers with data can occur in numerous ways. One of the most common is using the `glBufferData()` command

□ Sending data

- Drawing in OpenGL means to transfer vertex data to the OpenGL Server
- Vertex is a bundle of data values that are processed together
- Data in the vertex bundle can be any data that makes up a vertex (e.g. always includes position data, other data values need to determine the final color of the pixel).

Vertex shading

- ❑ Each vertex issued by a drawing command is processed by a vertex shader
- ❑ Vertex shaders could have different complexity
 - Very simple - just copying data to pass it through this shading stage
 - Very complex - performing many computations
 - ❑ Compute the screen position of the vertex (usually using transformation matrices)
 - ❑ Determining the color of the vertex (using lighting computations)
 - ❑ Any multitude of other techniques
- ❑ An application of any complexity has multiple vertex shaders, but only one can be active at any one time

Tessellation shading

- ❑ Tessellation shader stage will continue processing data associated to the vertex, if it has been activated
- ❑ Uses *patches* to describe the shape of an object (e.g. triangles, polygons), and allows relatively simple collections of patch geometry to be *tessellated* to increase the number of geometric primitives providing better-looking models (e.g. triangle mesh, polygon mesh)
- ❑ The tessellation shading stage can potentially use two shaders to manipulate the patch data and generate the final shape (control and evaluation)

Geometry shading

- ❑ Allows additional processing of individual geometric primitives (e.g. create new primitives before rasterization)
- ❑ This shading stage is optional

Primitive assembly

- ❑ The previous shading stages all operate on vertices, with the information about how those vertices are organized into geometric primitives being carried along internal to OpenGL
- ❑ Primitive assembly stage organizes the vertices into their associated geometric primitives in preparation for clipping and rasterization

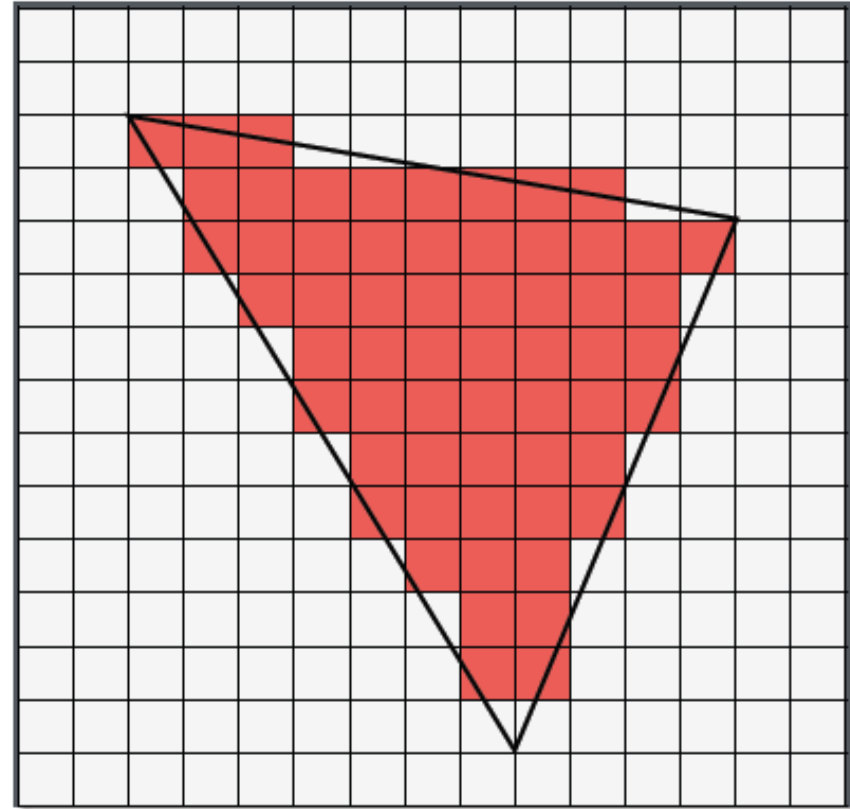
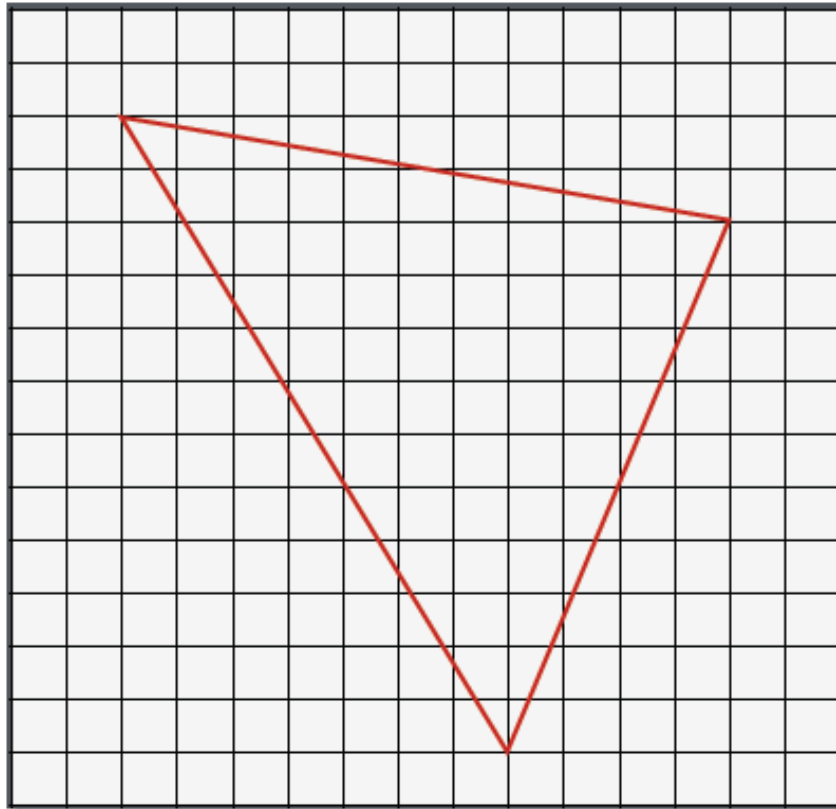
Clipping

- ❑ Primitive associated with the vertex are modified so none of its pixels are outside of the *viewport* (region of the window where you are permitted to draw)
- ❑ This operation is called *clipping* and is handled automatically by OpenGL

Rasterization

- ❑ After clipping, the updated primitives are sent to the rasterizer for fragment generation
- ❑ The fragment is considered a “candidate pixel”. The pixels have positions in the framebuffer, while a fragment still can be rejected and never update its associated pixel location.
- ❑ The fragments are processed in the next two stages, fragment shading and per-fragment operations.

Example of rasterization



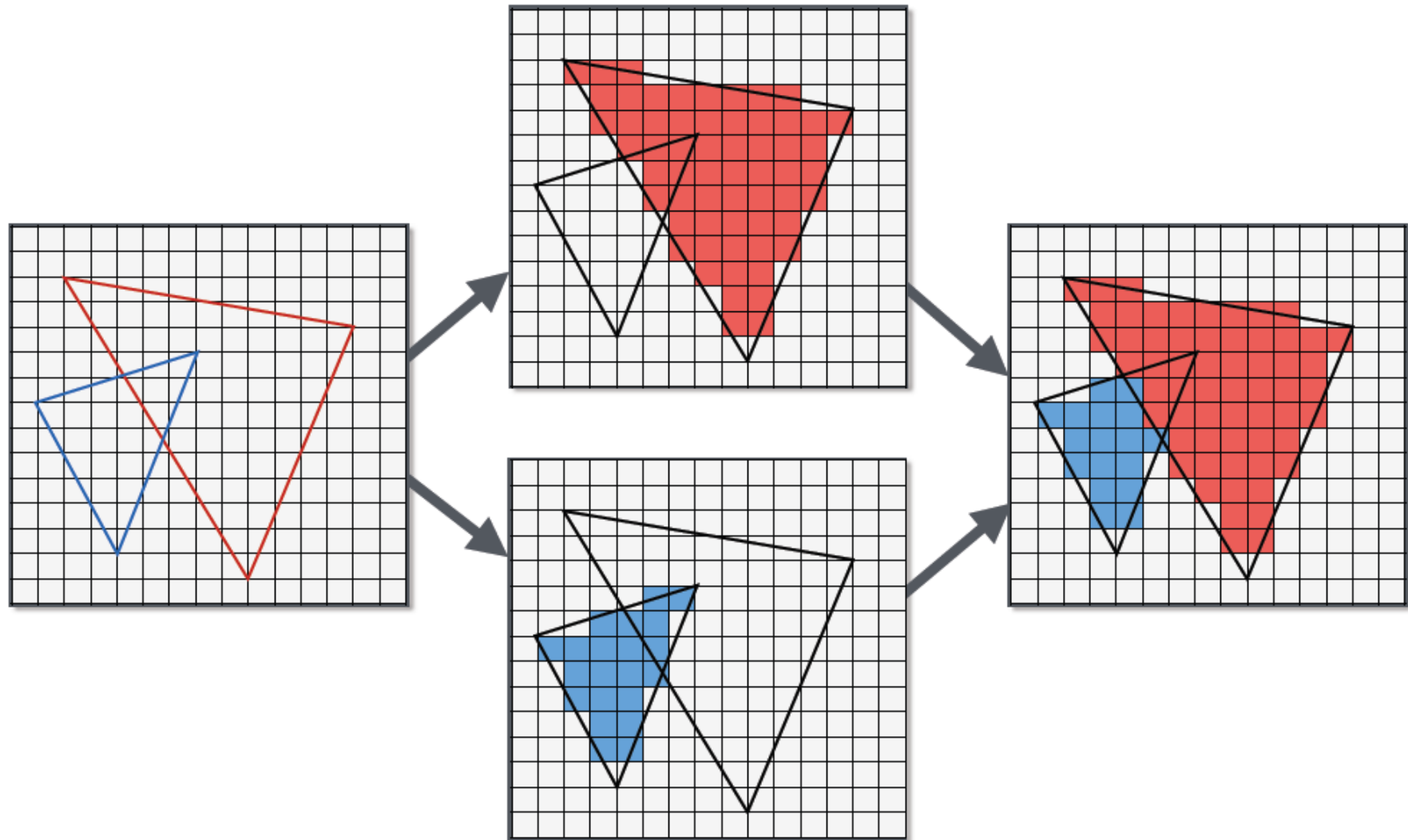
Fragment shading

- ❑ It is the final stage where the programmer has control over the color of a screen location (i.e. *fragment color* and *depth value*)
- ❑ Fragment shaders are very powerful as they often employ texture mapping to augment the colors provided by the vertex processing stages
- ❑ A fragment shader may also terminate processing a fragment if it determines the fragment should not be drawn (fragment *discard process*)
- ❑ The final generated image consists of pixels drawn on the screen. A pixel is the smallest visible element on your display. The pixels in your system are stored in a framebuffer, which is a chunk of memory that the graphics hardware manages, and feeds to your display device.

Fragment operations

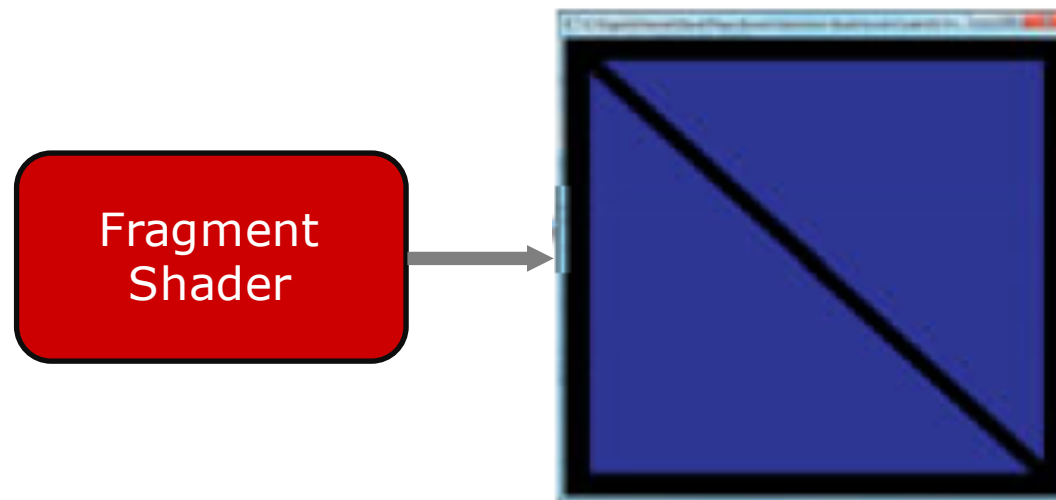
- ❑ Final processing of individual fragments
- ❑ Visibility of the fragment is determined using *depth testing* (i.e. *z-buffering*) and *stencil testing*
- ❑ If a fragment successfully makes it through all of the enabled tests, then:
 - it may be written directly to the framebuffer, updating the color (and possibly depth value) of its pixel
 - if *blending* is enabled, the color of the fragment is combined with the current color to generate a new color into the framebuffer
- ❑ Generally, pixel data comes from an image file, although it may also be created by rendering using OpenGL
- ❑ Pixel data is usually stored in texture map for use with texture mapping, which allows any texture stage to look up data values from one or more texture maps

Fragment processing

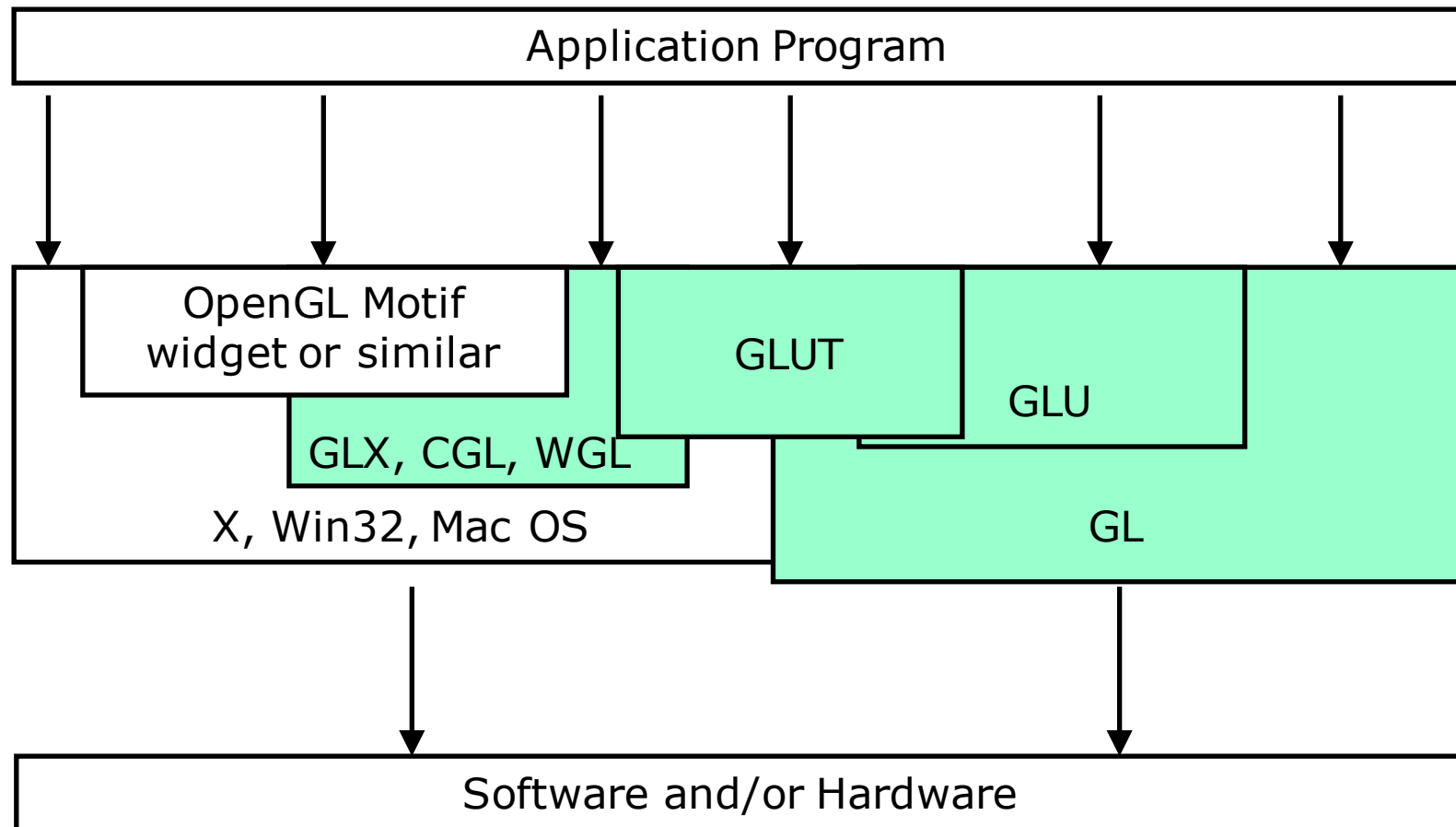


Final output image

- The final generated image consists of pixels drawn on the screen. A *pixel* is the smallest visible element on your display. The pixels in your system are stored in a *framebuffer*, which is a chunk of memory that the graphics hardware manages, and feeds to your display device.



OpenGL programming levels



OpenGL- related libraries

- Window tasks
- User input
- Complex shapes



For portability, there are no commands for these

- OpenGL Utility Library (GLU)
- Window system support libraries: GLX / WGL / CGL
- OpenGL Utility Toolkit (GLUT)
- OpenGL User Interface (GLUI)
- OpenInventor

Context and window toolkits

- ❑ Creating an OpenGL context is a complex process that varies between operating systems. The automatic OpenGL context creation has become a common feature of several game-development and user-interface libraries, including SDL, Allegro, SFML, FLTK, and Qt
- ❑ A few libraries have been designed solely to produce an OpenGL-capable window. The first such library was OpenGL Utility Toolkit (GLUT), later superseded by freeglut

Context and window toolkits

- ❑ OpenGL Utility Toolkit (GLUT) – is an old windowing handler, but no longer maintained
- ❑ freeglut – is a cross-platform windowing and keyboard-mouse handler; its API is a superset of the GLUT API, and it is more stable and up to date than GLUT
- ❑ GLFW is a cross-platform lightweight utility library for use with OpenGL. It provides programmers with the ability to create and manage windows and OpenGL contexts, as well as handle joystick, keyboard and mouse input

OpenGL multimedia libraries

- Several "multimedia libraries" can create OpenGL windows. In addition to input, sound and other tasks useful for game-like applications:
 - Allegro 5 – cross-platform multimedia library with a C API focused on game development
 - Simple DirectMedia Layer (SDL) – cross-platform multimedia library with a C API
 - SFML – cross-platform multimedia library with a C++ API and multiple other bindings to languages such as C#, Java, Haskell, and GOlangWidget toolkits
 - FLTK – small cross-platform C++ widget library
 - Qt – cross-platform C++ widget toolkit. It provides many OpenGL helper objects, which even abstract away the difference between desktop GL and OpenGL

OpenGL supporting languages

- ❑ More than ~500 functions/commands similar to those of the programming language C
- ❑ Language independent functions
- ❑ Language bindings:
 - JavaScript: WebGL (based on OpenGL ES 2.0 for 3D rendering within the web browsers). It uses HTML5 canvas element and is accessed through DOM (Document Object Model) interface.
 - C bindings:
 - WGL (Microsoft Windows interface to OpenGL),
 - GLX (X11 interface to OpenGL)
 - CGL (Mac OS X interface to OpenGL)
 - C binding provided by iOS
 - Java and C bindings provided by Android

Shader languages for GPU

- ❑ Assembler
- ❑ Cg (NVIDIA)
C for Graphics
- ❑ GLSL (OpenGL)
OpenGL Shading Language
- ❑ HLSL (Microsoft/Direct3D)
High Level Shading Language

GLSL – OpenGL Shading Language

- ❑ GLSL (OpenGL Shading Language), also known as GLSLang, is a high level shading language based on the C programming language. It was created by the OpenGL ARB to give developers more direct control of the graphics pipeline without having to use assembly language or hardware-specific languages.
- ❑ GLSL is actually two closely related languages. These languages are used to create shaders for the programmable processors contained in the OpenGL processing pipeline.
- ❑ The specific languages are referred by the name of the processor they target: *vertex* or *fragment*.

GLSL – OpenGL Shading Language

- ❑ The *vertex processor* is a programmable unit that operates on incoming vertices and their associated data. Compilation units written in the OGSL to run on this processor are called *vertex shaders*.
- ❑ The *fragment processor* is a programmable unit that operates on fragment values and their associated data. Compilation units written in the OGSL to run on this processor are called *fragment shaders*.

Reference:

GLSL specification, <http://www.opengl.org/documentation/glsl/>

Software technologies

- ❑ OS: Windows, Linux, Mac
- ❑ API: OpenGL, DirectX, CUDA C/C++, OpenCL , DirectCompute, CUDA Fortran, and others
- ❑ Language: Cg, GLSL, HLSL
- ❑ Compiler: Cg, OpenGL, DirectX, ASHLI
- ❑ Runtime: CgFX, OSG, ASHLI, other codes

OpenGL application program

- Basic structure of an OpenGL application:
 1. *Initialize the state* associated with how objects should be rendered
 2. Specify those objects *to be rendered*

Questions and problems

1. Explain the reason OpenGL is considered an API, library, and graphics system.
2. Where is located the data files (textures, graphics objects) in the client-server architecture of OpenGL? Explain the reasons for such a solution.
3. Explain the difference between fragment and pixel.
4. Define and exemplify the notion of shader.
5. Why only the Vertex and Fragment shaders must be included within the OpenGL pipeline?

Questions and problems

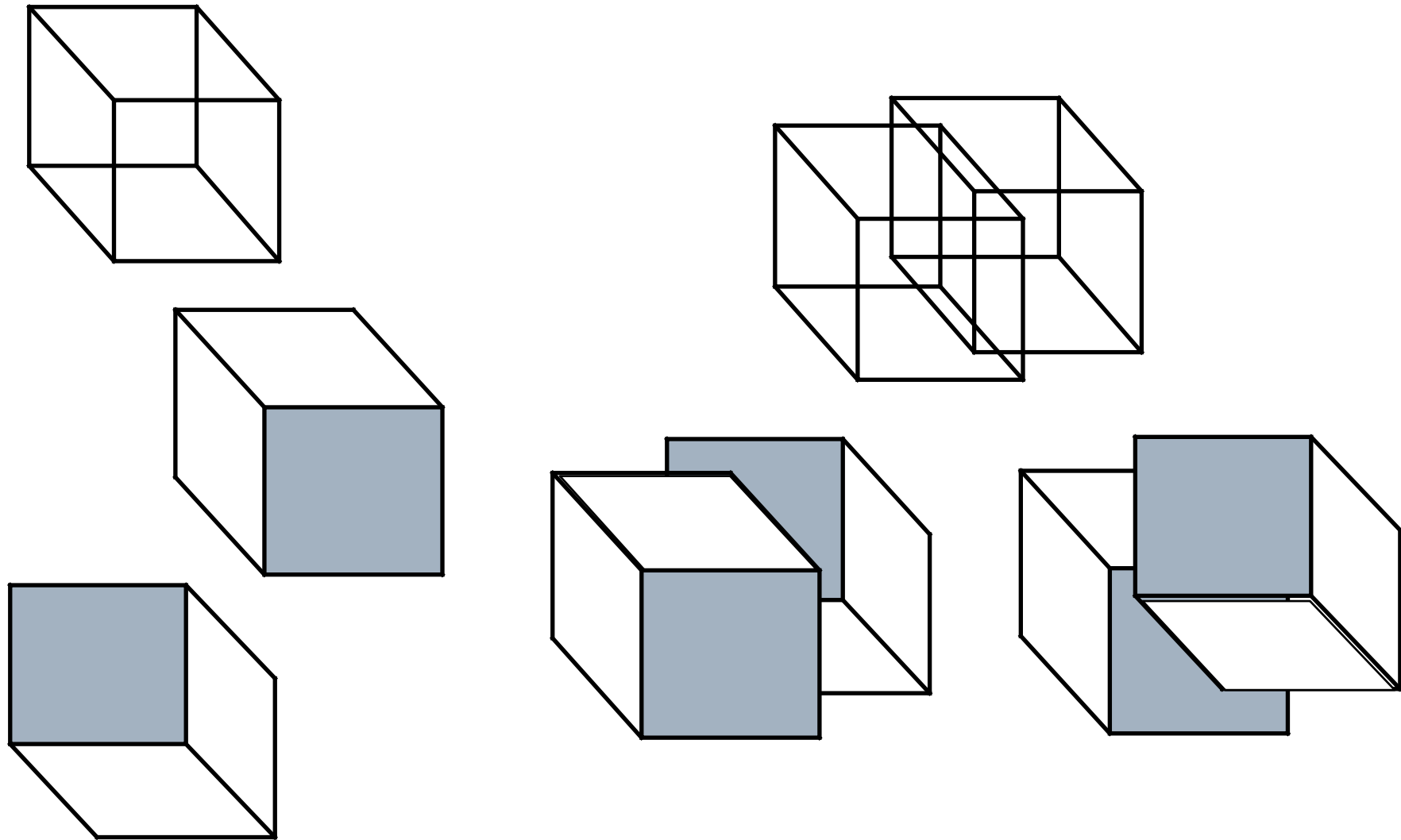
6. Explain the mechanism of transferring data through buffer objects to the OpenGL Server.
7. Explain the differences between the Vertex and Geometry shaders.
8. Exemplify the graphics processing of a cube as 3D object, along the OpenGL pipeline.

Hidden line and surface removal algorithms (1)

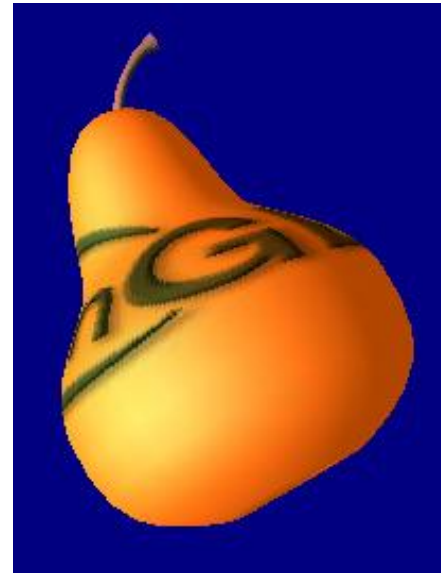
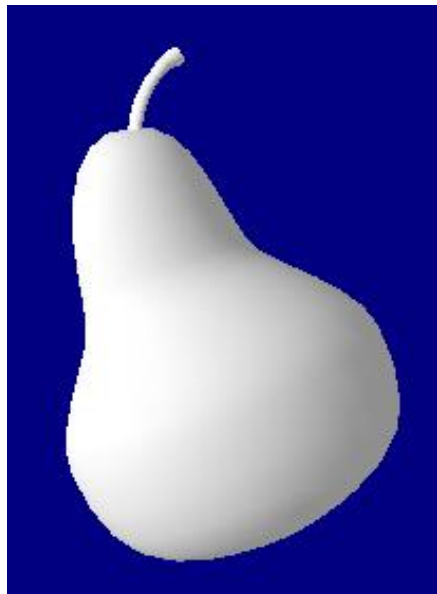
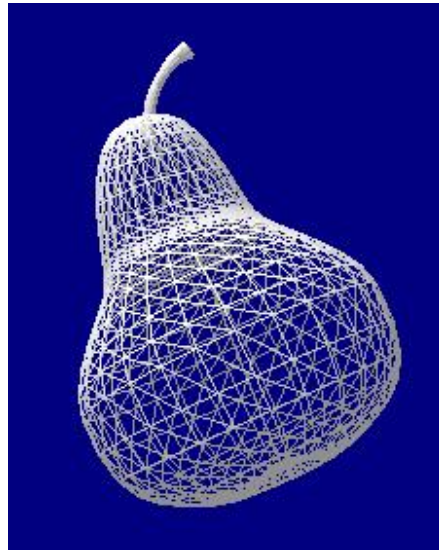
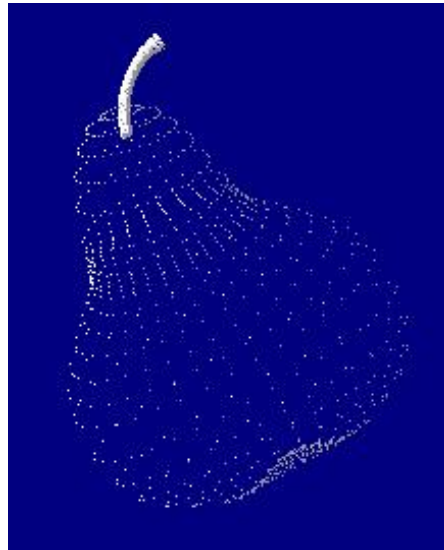
Contents

- Introduction
- Hidden line and surface removal algorithms
 - 1. Floating horizon algorithm**
 - 2. Back-face culling**
 - 3. Area subdivision algorithm**
 - 4. Depth buffer algorithms**
 5. Depth sorting algorithms
 6. Scan-line algorithms
 7. Octree algorithm
 8. Ray casting algorithms
 9. BSP trees

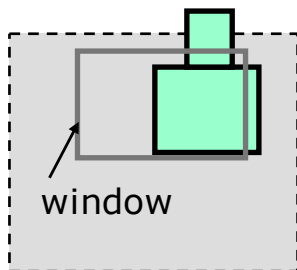
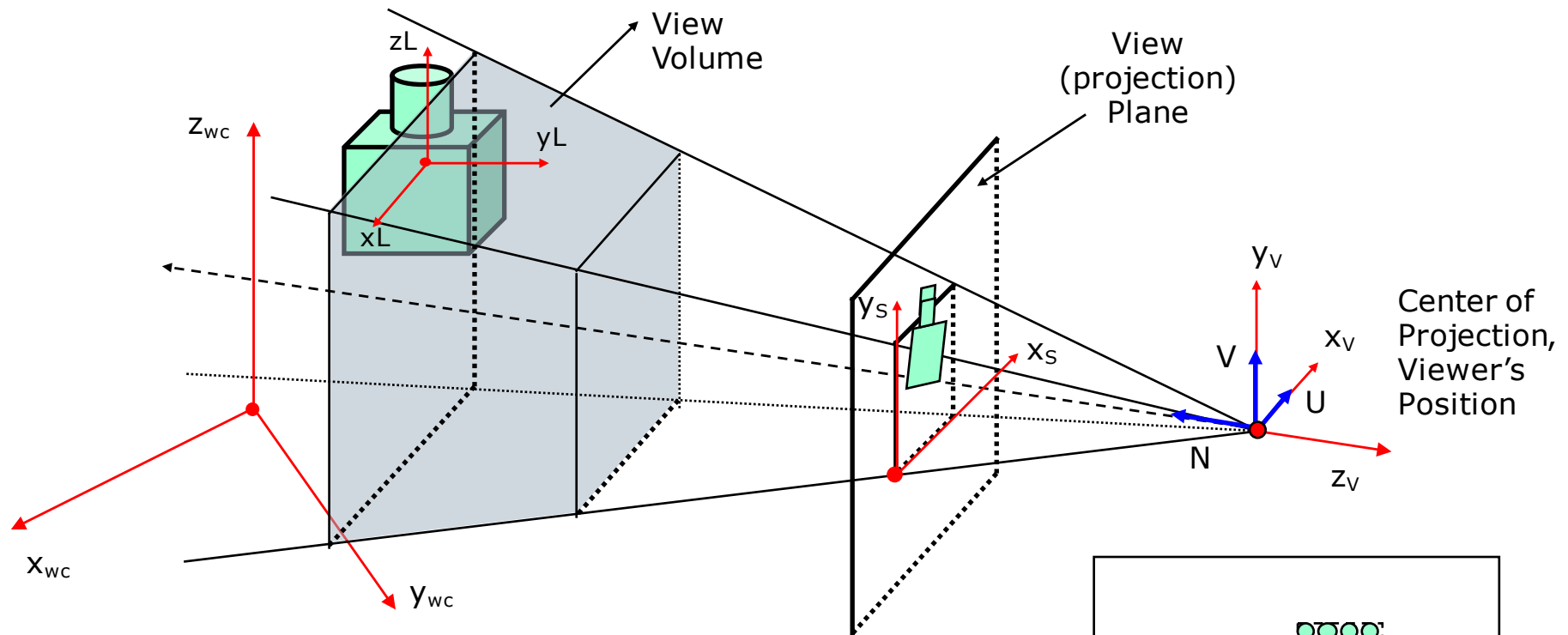
Introduction



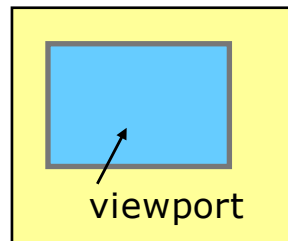
Introduction



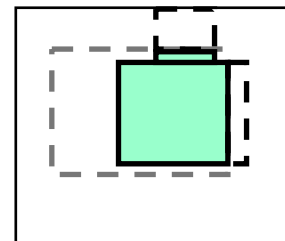
Conceptual model of 3D viewing process



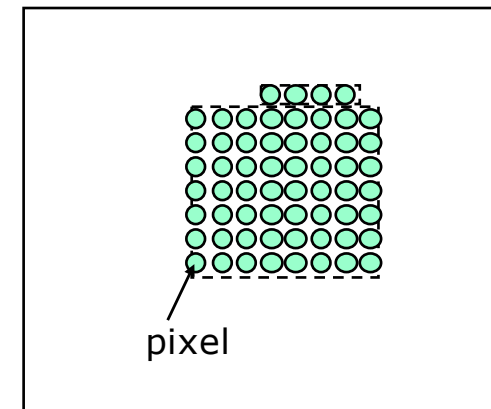
Window definition
in projection plane



Viewport definition
in projection plane

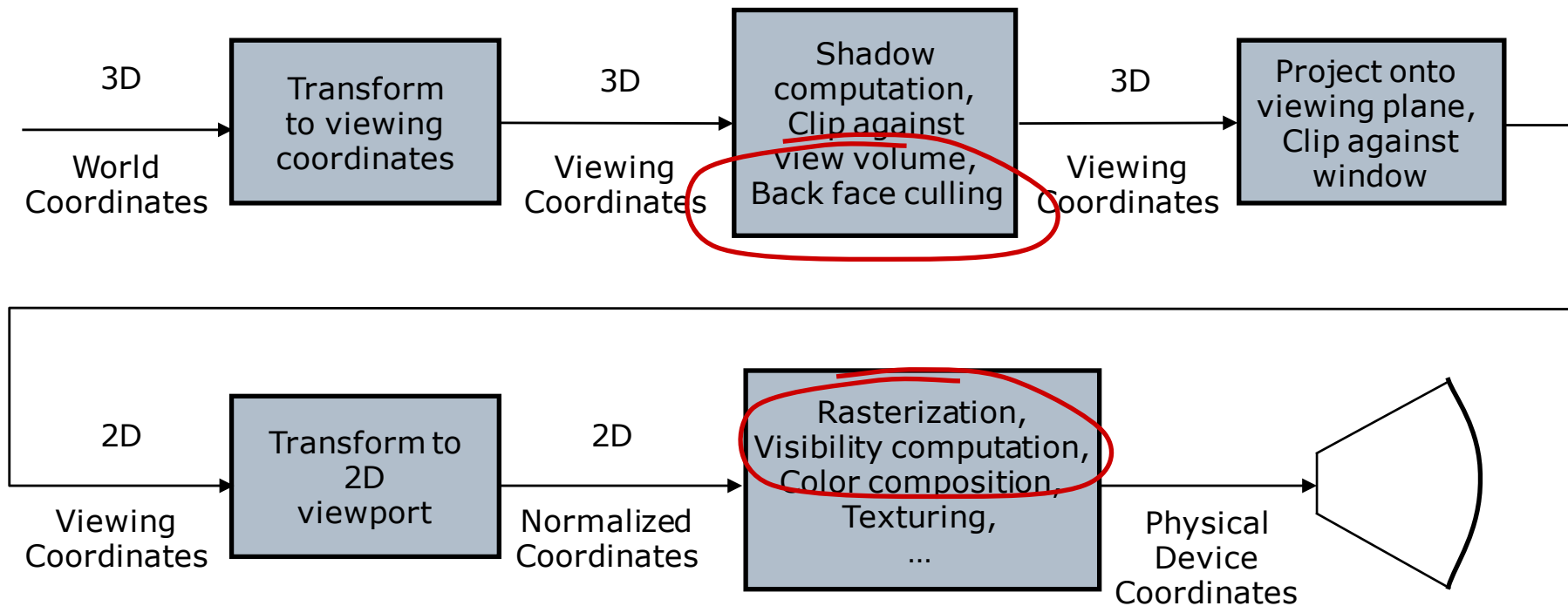


Window's content
inside the viewport

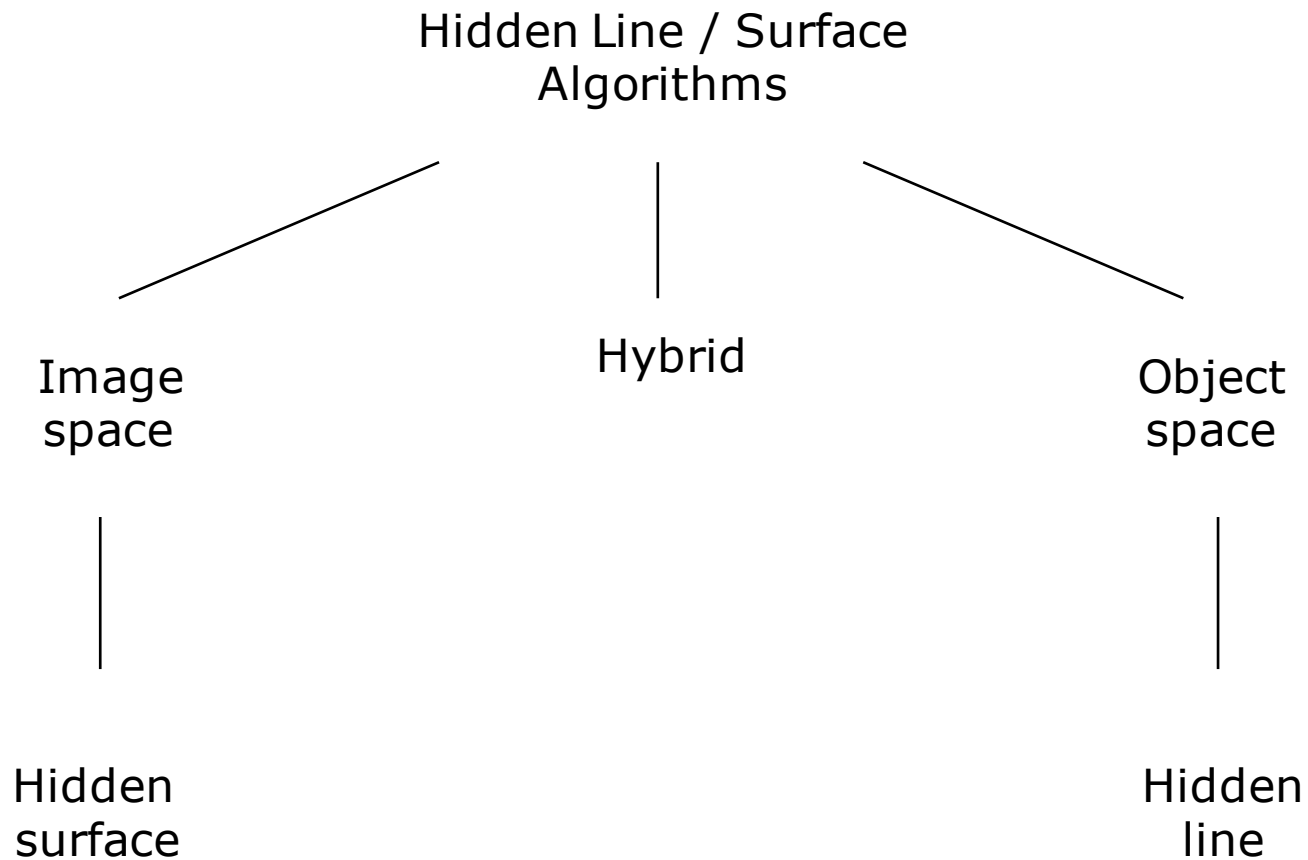


Scan conversion onto the
raster screen

Viewing transformations pipeline



Taxonomy of algorithms



Hidden line and surface removal

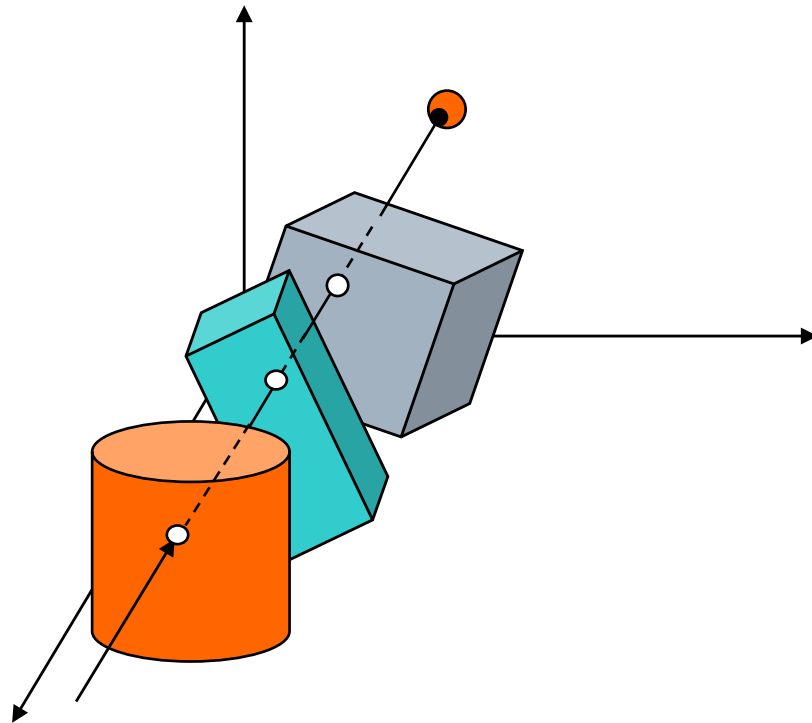
□ Algorithms in

1. Object space

- Determine if an element (point, polygon, surface, etc) is visible

2. Image space

- Determine what is visible on every pixel



for *each object in the world* {

*determines those parts of the object
whose view is unobstructed by other
parts of it or any other objects;*

*draw those parts in the appropriate
color;*

}

Computational effort: n^2

n : number of the objects

Hidden line and surface removal

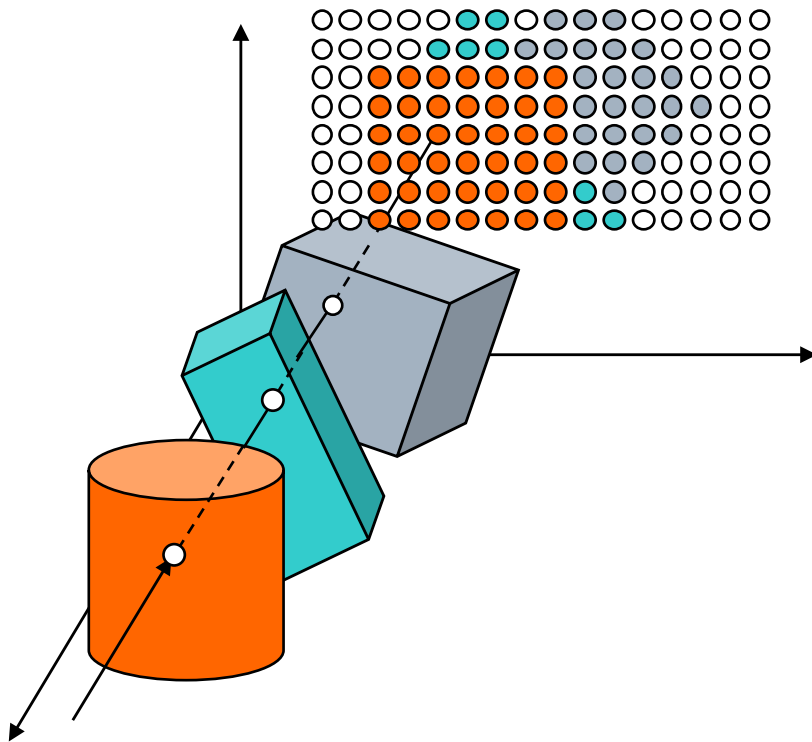
□ Algorithms in

1. Object space

- Determine if an element (point, polygon, surface, etc) is visible

2. Image space

- Determine what is visible on every pixel



for *each pixel in the image* {
 *determines the object closest to
 the viewer that is projected onto
 the pixel;*

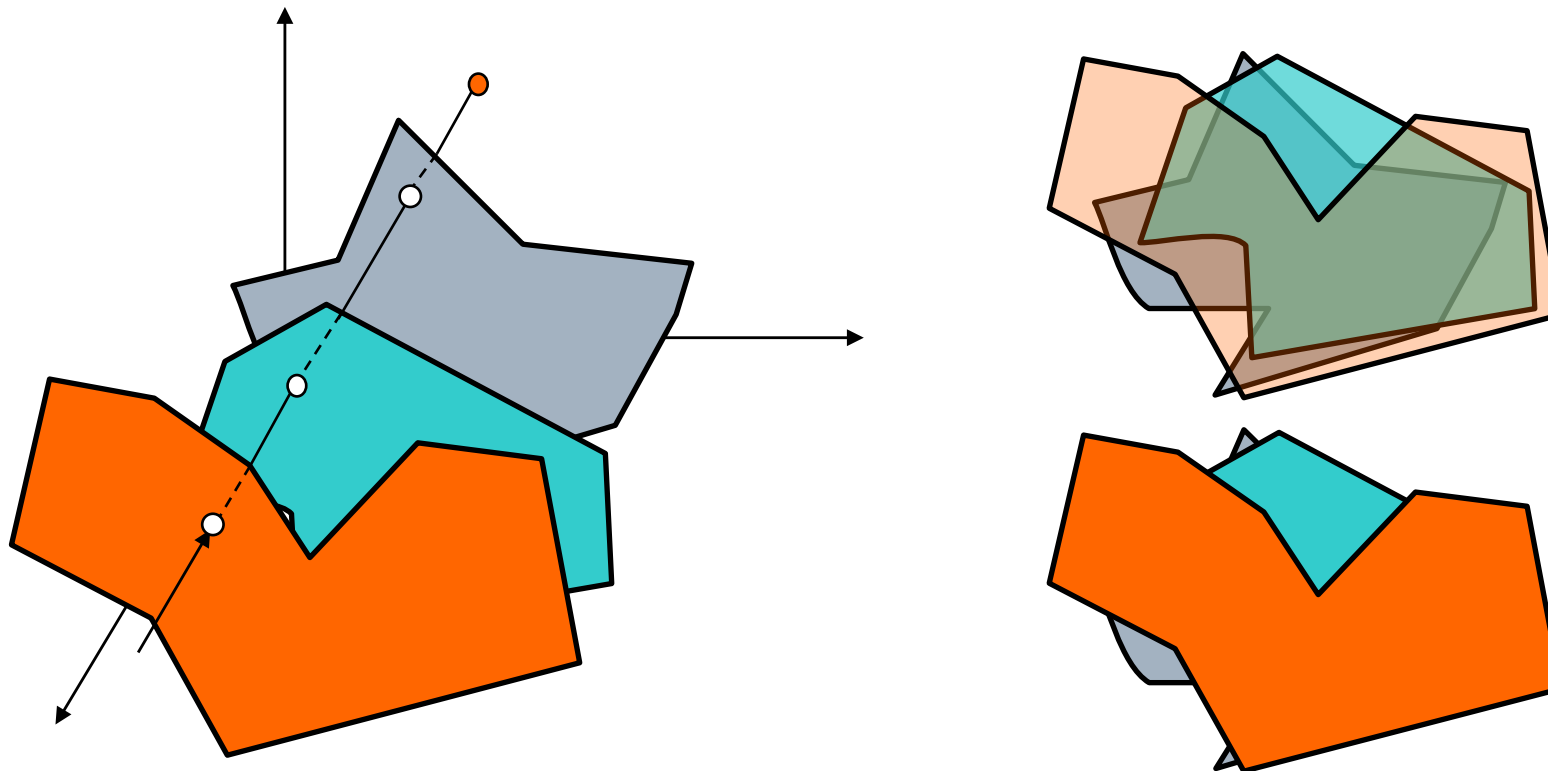
*draw the pixel in the appropriate
color;*

}

Computational effort: np
 n : number of the objects
 P : number of pixels

Basic tools

- Sorting
 - Points, polygons, edges, vertices etc.
- Ordered drawing
 - Over drawing into the image buffer.



Sorting efficiency

1. Coherence

- On scan line, on edge, on surface, on polygon, area, depth, etc.

2. Depth sort simplification

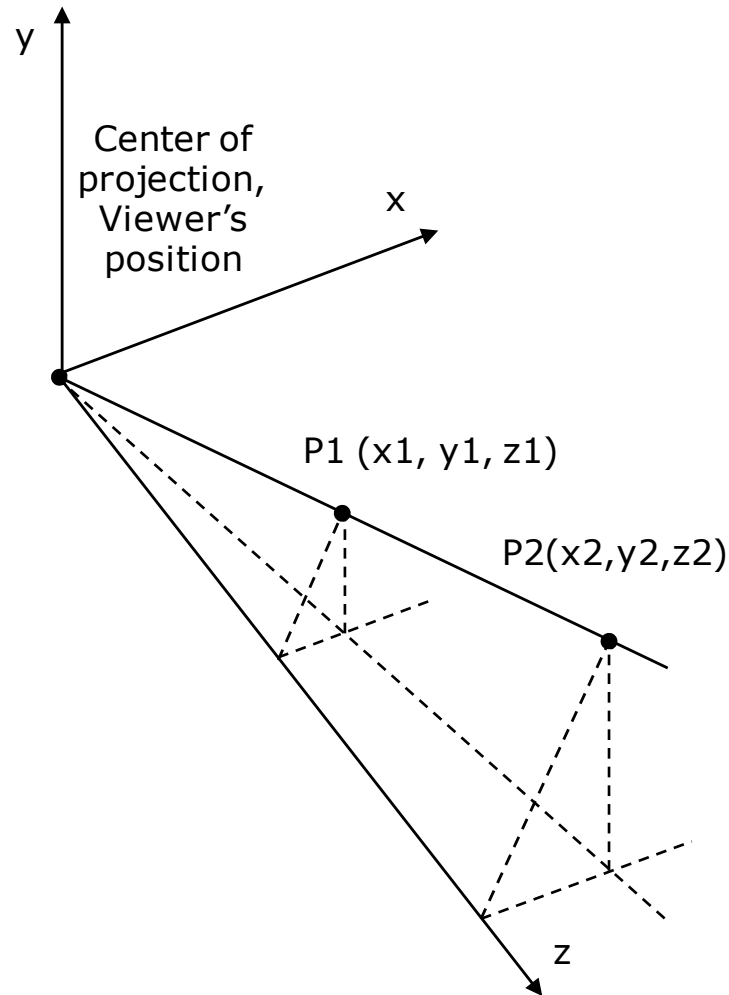
- Perspective transformation
- Keep depth information
- Perspective vs orthogonal projection

3. 2D and 3D extents

Coherence

- Object coherence
 - If one object is entirely separate from another, comparisons may need to be done only between the two objects, and not between their component faces or edges.
- Face coherence
 - Surface properties typically vary smoothly across a face, allowing computations for one part of a face to be modified incrementally to apply to adjacent parts.
- Edge coherence
 - An edge may change visibility only where it crosses behind a visible edge or penetrates a visible face.
- Scan-line coherence
 - The set of visible object spans determined for one scan line of an image typically differs little from the set on the previous line.
- Depth coherence
 - Adjacent parts of the same surface are typically close in depth. Once the depth at one point of the surface is calculated, the depth of points on the rest of the surface can often be determined by a simple difference equation.
- Frame coherence
 - Pictures at two successive points in time are likely to be quite similar, despite small changes in objects and viewpoint.

Visibility in perspective projection



P1 and P2 are collinear, if:

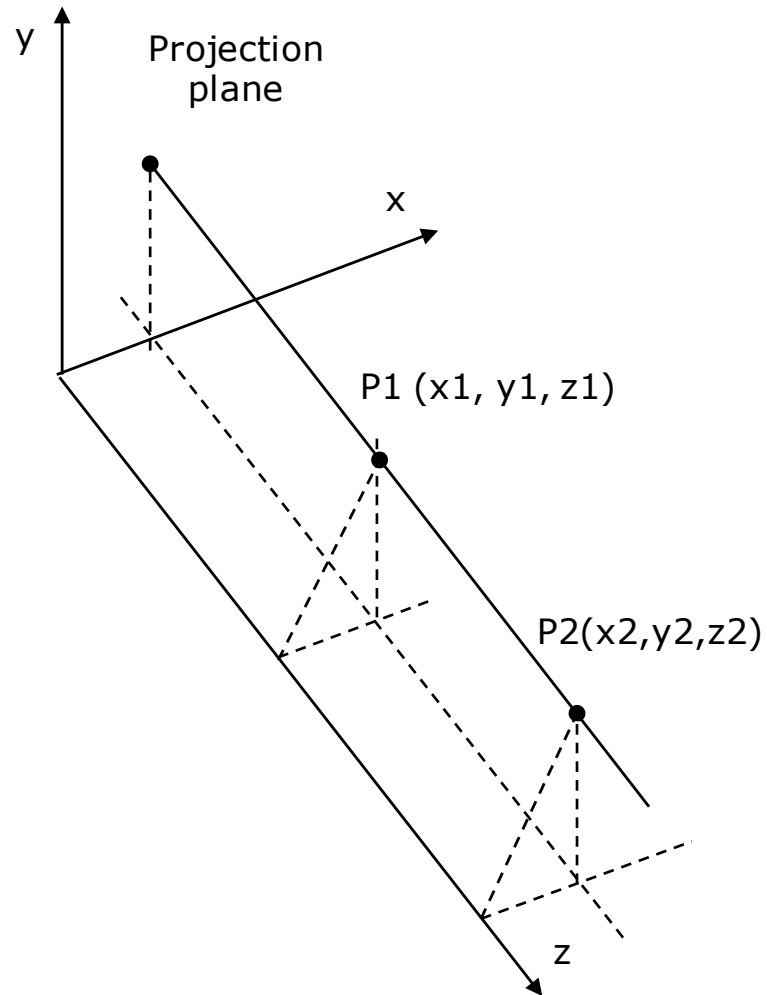
$$x1/x2 = y1/y2 = z1/z2 = k \text{ (const)}$$

Checking a point P requires:

$$x/x2 = y/y2 = z/z2 = k$$

It needs to avoid expensive computation
(multiplication, division, ...)

Visibility in orthogonal projection



P1 and P2 collinear, if:

$$x_1 = x_2 \text{ and } y_1 = y_2$$

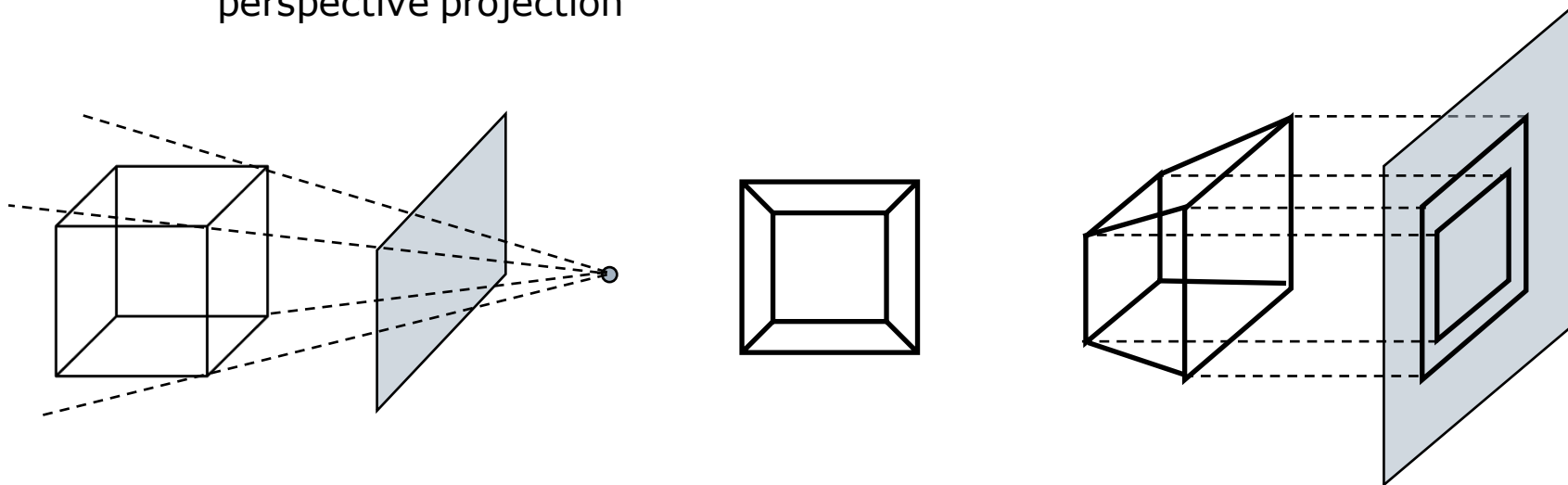
Checking a point P requires:

$$x = x_2 \text{ and } y = y_2$$

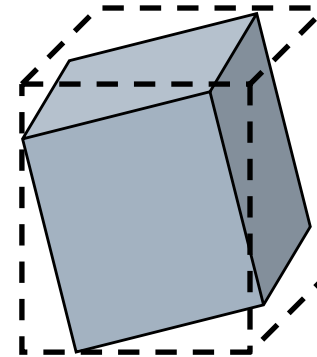
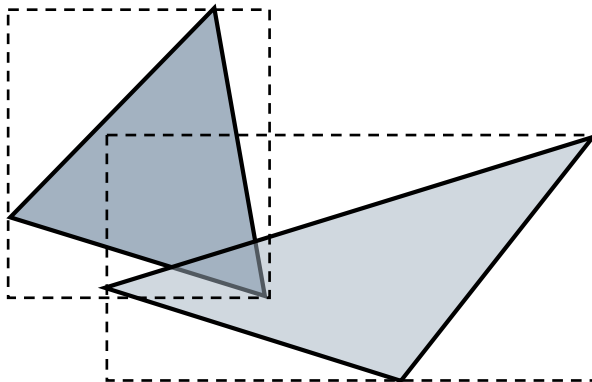
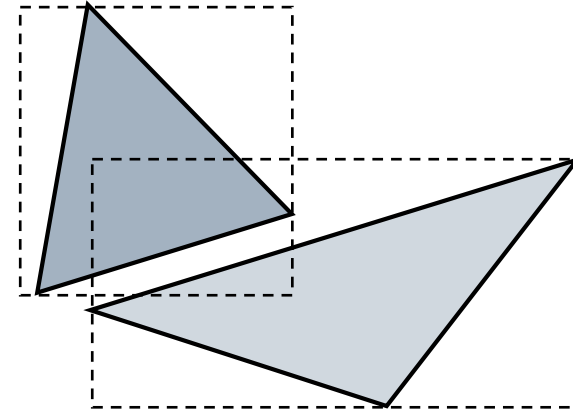
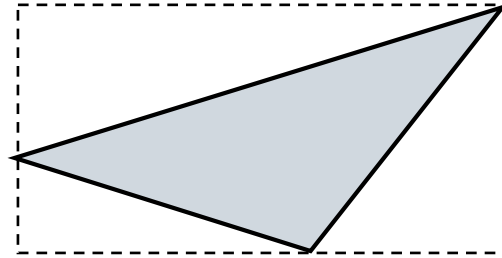
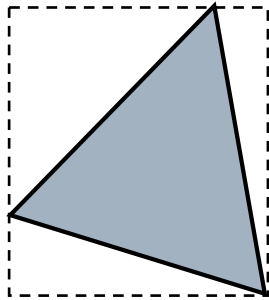
No multiplication, division, ...
just comparisons.

Visibility and projections

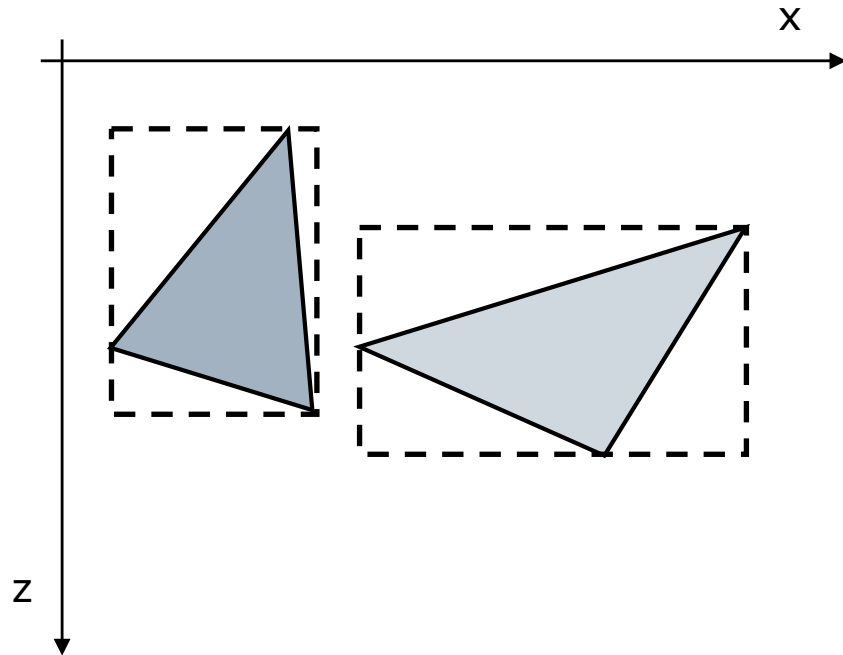
- The computation of the visibility requires:
 - Orthogonal projection
 - The visibility related computation needs depth information (z coordinates)
 - Keep depth information
- Example: frustum perspective transformation
 - Projection along the frustum (viewing volume) transforms the perspective projection into orthogonal projection
 - Keeps the z coordinate
 - Keeps depth order
 - Deform the object shape (e.g. cube into pyramidal frustum) in order to have through the orthogonal projection the same 2D image as through the perspective projection



Object extents

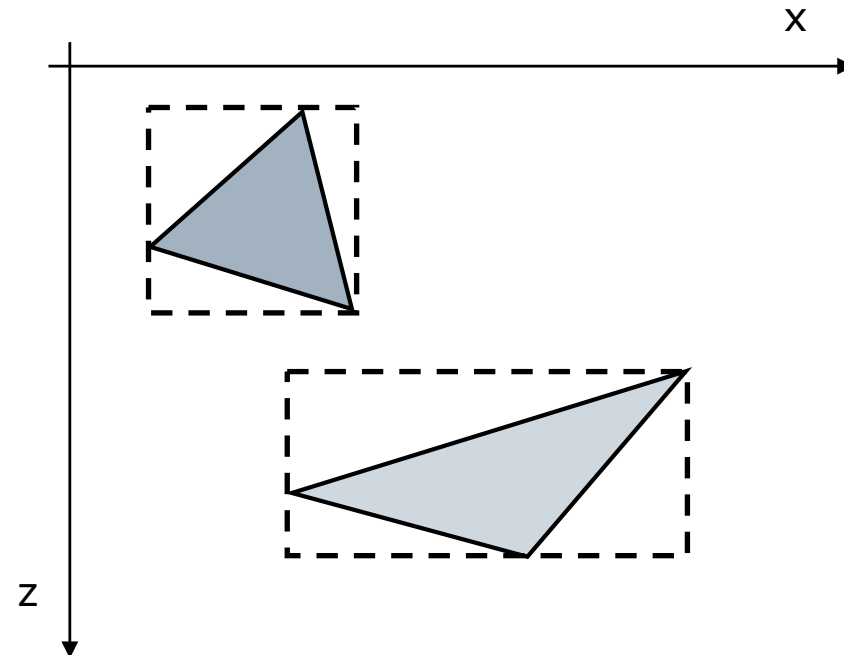


Object extents



$$x_{\min 2} \geq x_{\max 1}$$

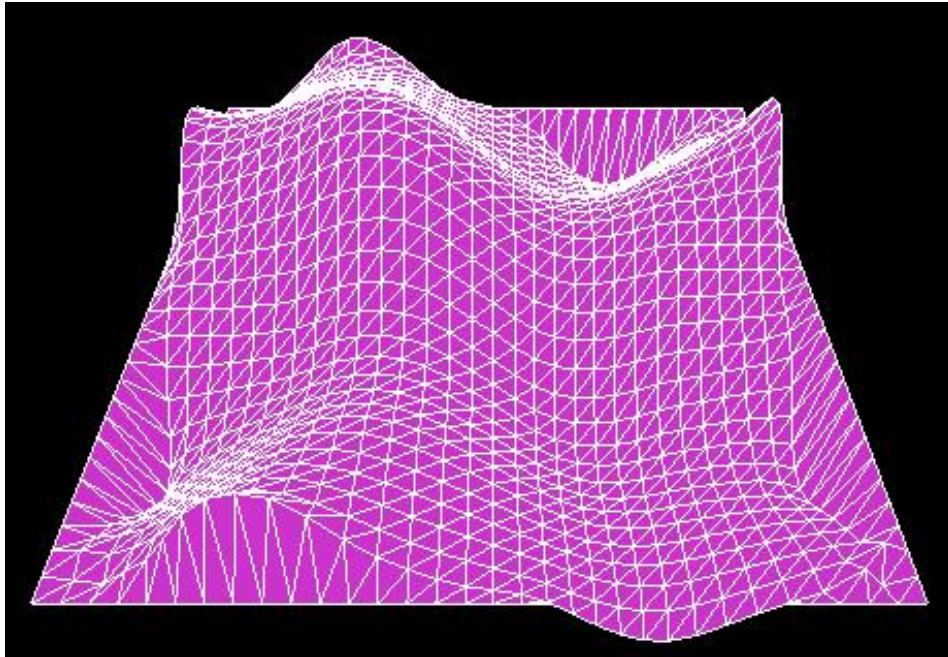
$$x_{\min 1} \geq x_{\max 2}$$



$$z_{\min 2} \geq z_{\max 1}$$

$$z_{\min 1} \geq z_{\max 2}$$

Hidden surface algorithms - example



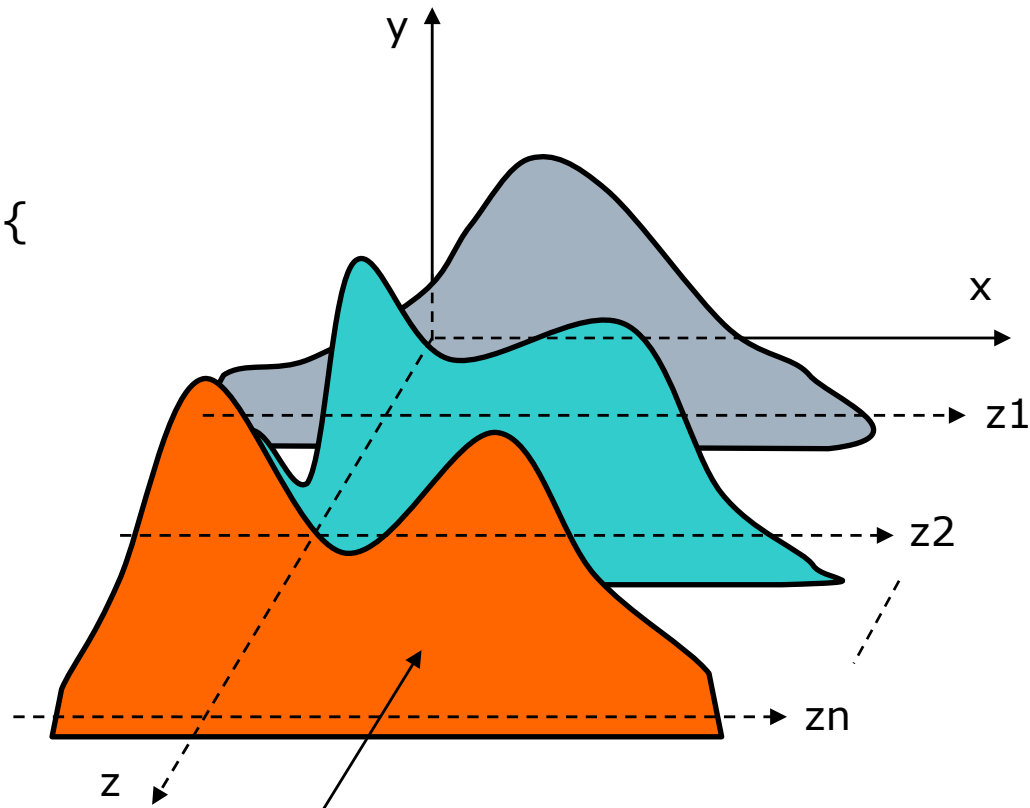
1. Floating Horizon Algorithm

□ $F(x,y,z)=0$

E.g. $x^2+y^2+z^2-r^2=0$

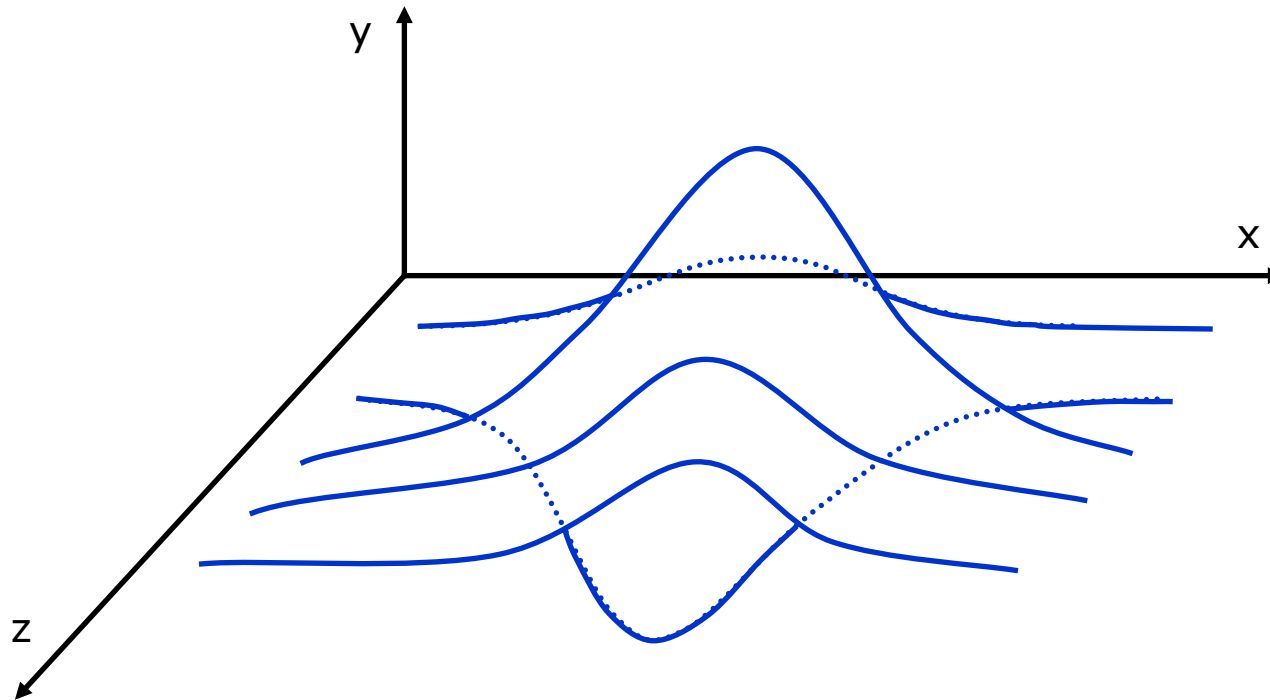
$z=z_i, i=1,2,\dots,n$

```
for z=z1 to zn {  
  for x=xmin to xmax {  
    render y=f(x,z)  
  }  
}
```

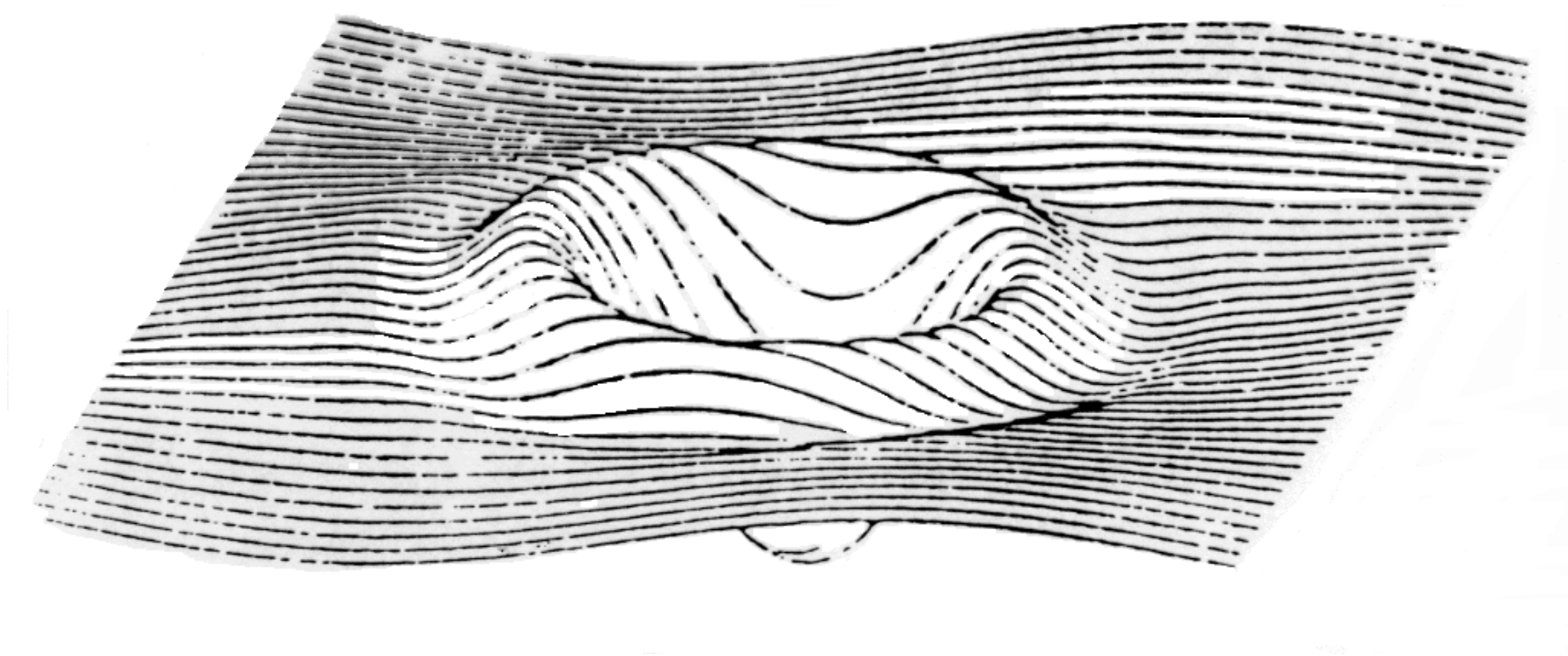


Functions in cutting planes

- Curves in cutting planes constant on z

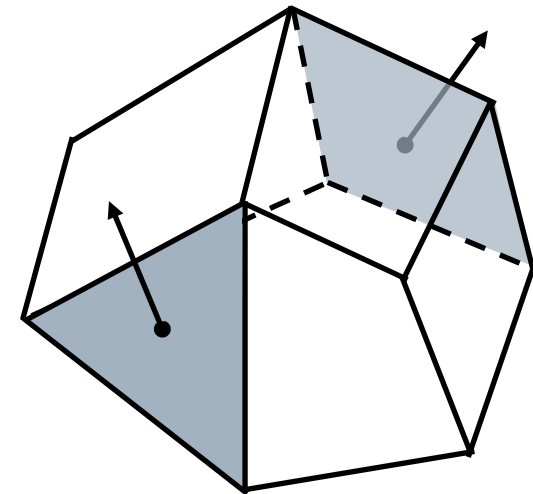
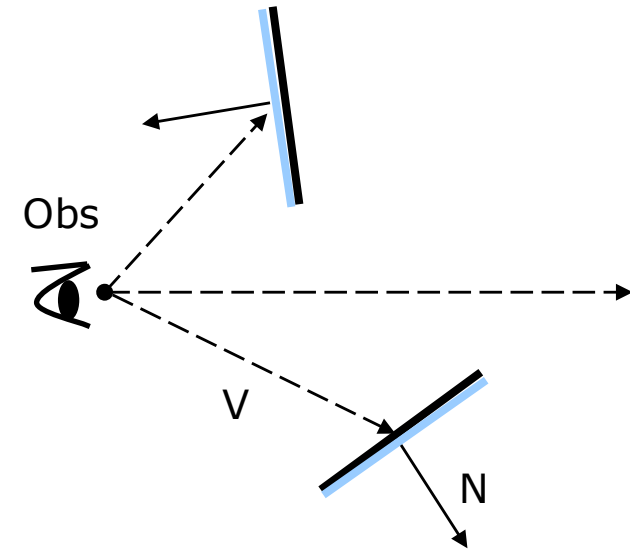


Floating Horizon Algorithm - example



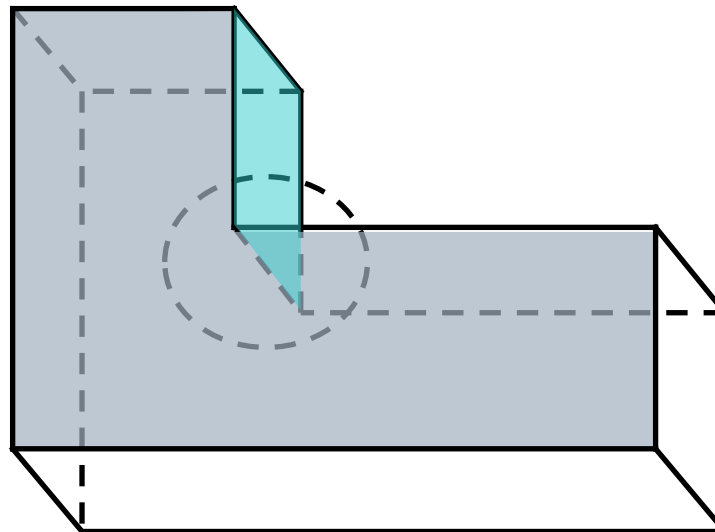
2. Back-face culling algorithm

- A simple way to perform hidden surface removal by cutting off all the “backfacing” polygons
- “backfacing” polygon is the polygon for which the normal is facing away from the viewer
- For convex solid objects, the backfacing polygons are not visible by the viewer
- To compute the visibility of a polygon use the dot product (see “point to plane and point to 3D convex object relationships” in a previous course “Mathematics in Computer Graphics”) between normal vector N and vector V from viewer to polygon: $N \cdot V$
- if $N \cdot V > 0$, cull the polygon.



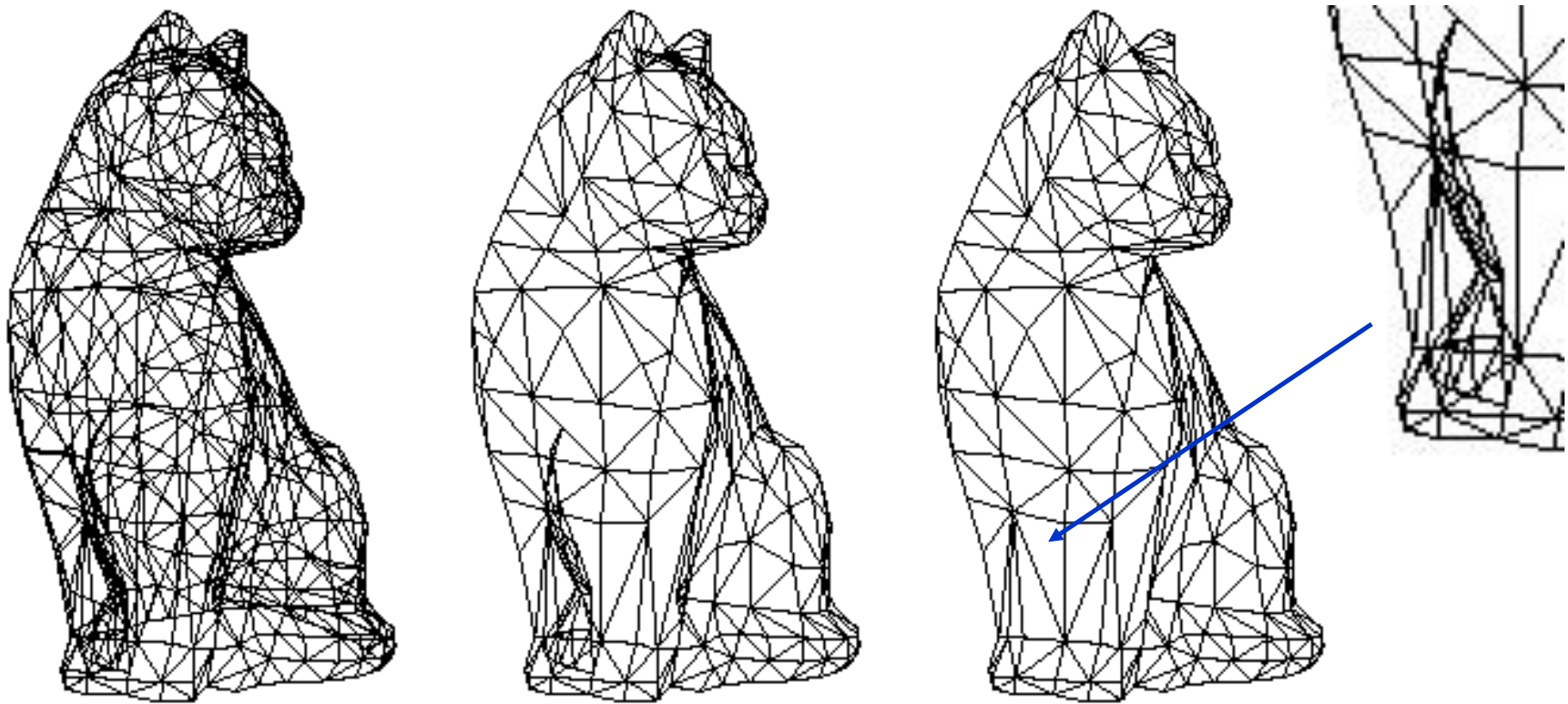
Back-face culling algorithm

- ❑ The back-face algorithm works well just for convex objects
- ❑ Usually works in conjunction with a more complete hidden surface algorithm
- ❑ Easy to integrate into hardware
- ❑ Usually improves performance by a factor of 2



Back-face and hidden algorithms

- ❑ Rendering by wireframe, back face culling, and hidden surface removal

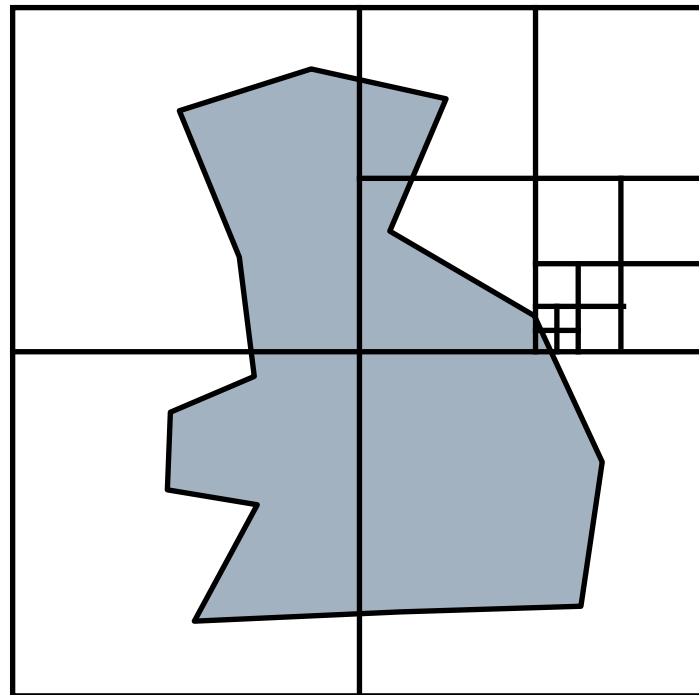


3. Area Subdivision Algorithms

- ❑ Warnock, 1969
- ❑ Catmull, 1974
- ❑ Weiler-Atherton, 1977

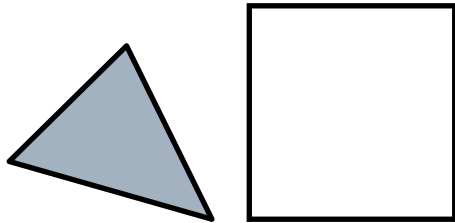
3.1 Warnock Algorithm (1969)

- ❑ Divide a window if any feature of interest (until pixel size)
- ❑ Consider the area coherence
- ❑ Work within image space
- ❑ For more polygons requires depth information
- ❑ Published in 1969, by Warnock, J., "A Hidden-Surface Algorithm for Computer Generated Half-Tone Pictures", Technical Report, Computer Science Department, Univ. of Utah.

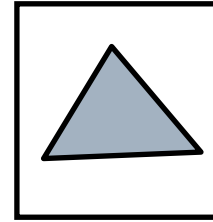


Polygon and elementary area

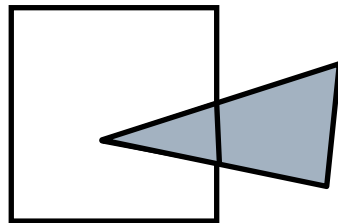
- There are 4 relationship types:



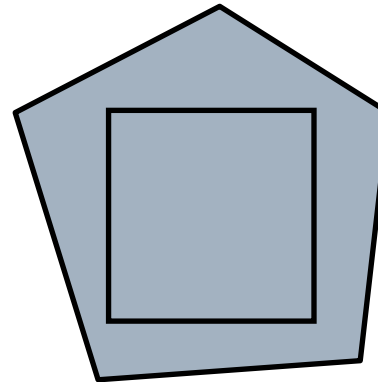
disjoint



contained



intersecting

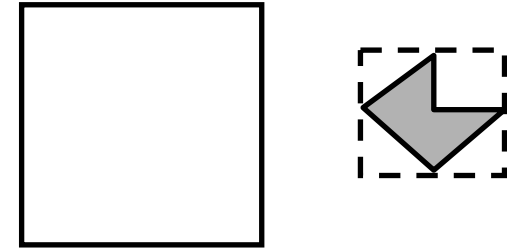


surrounding

Testing hierarchy

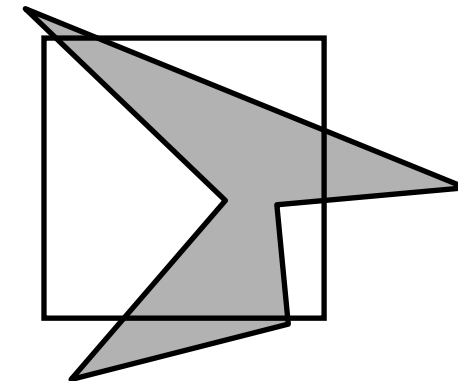
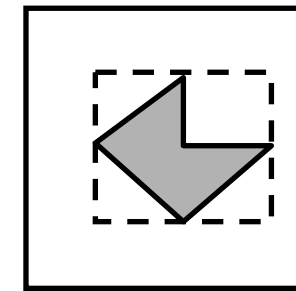
1. Box test

- All the polygons are disjoint from the area. The background color can be displayed in the area



2. Intersection test

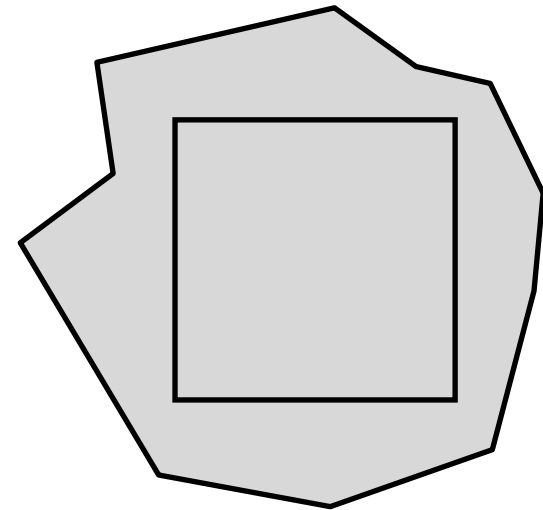
- There is only one intersecting or only one contained polygon. The area is first filled with the background color, and then the part of the polygon contained in the area is scan-converted



Testing hierarchy

3. Surrounding test

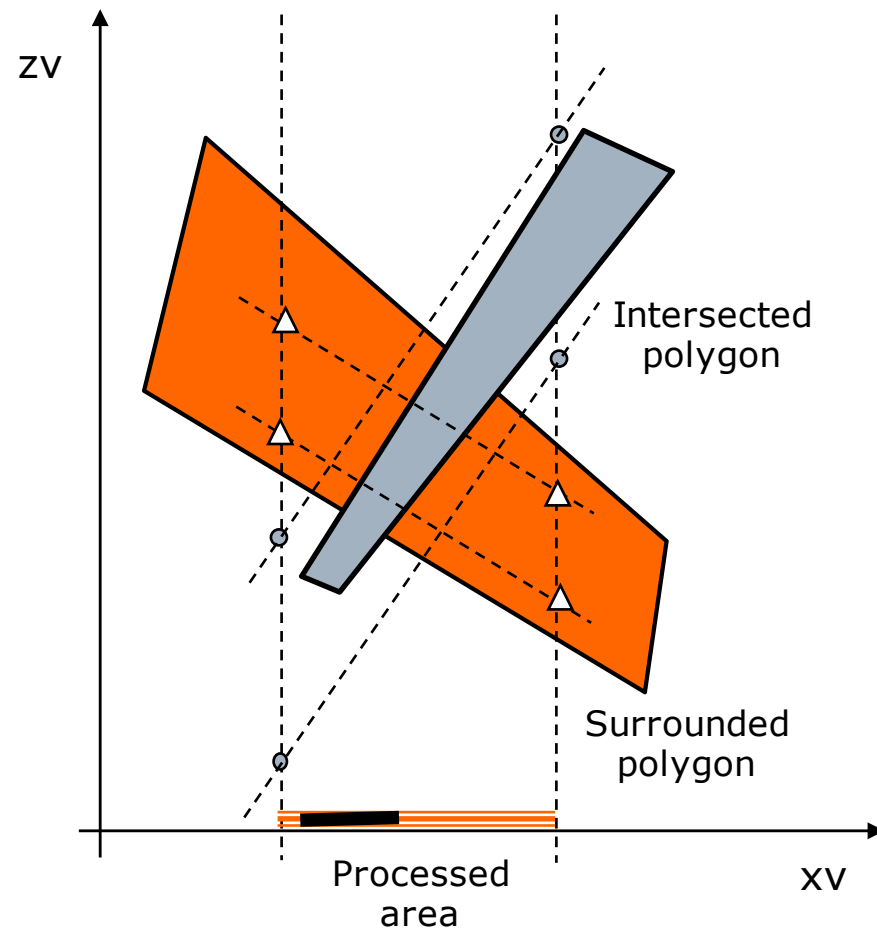
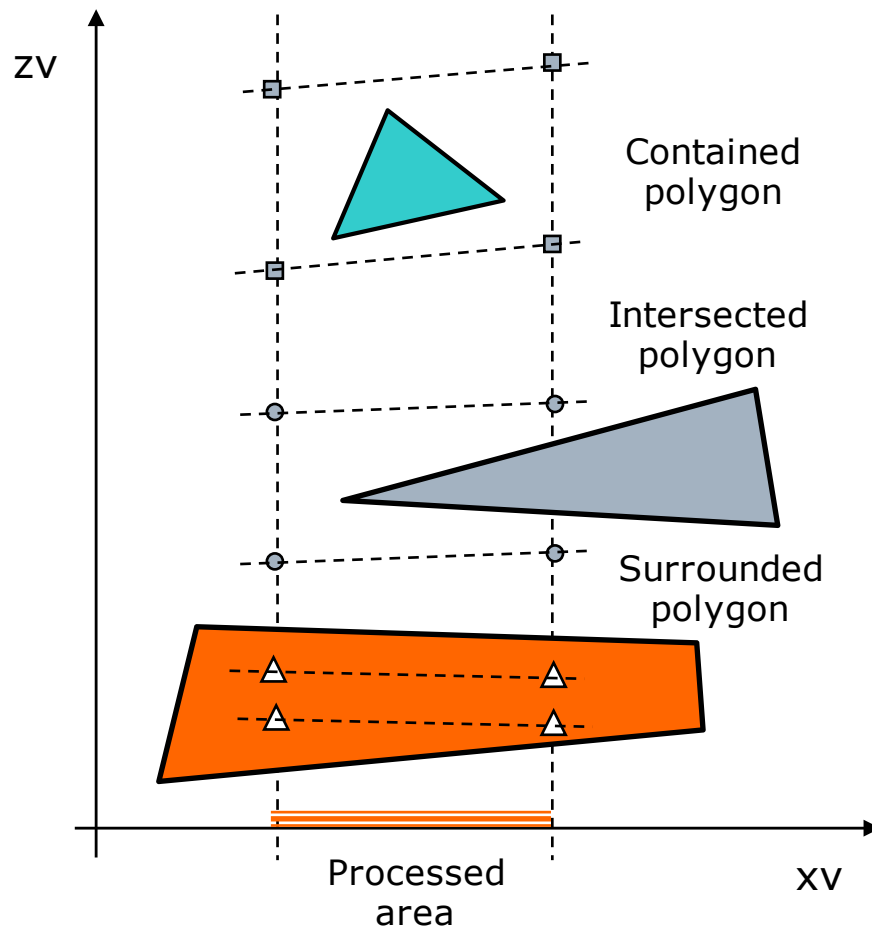
- There is a single surrounding polygon, but no intersecting or contained polygons. The area is filled with the color of the surrounding polygon



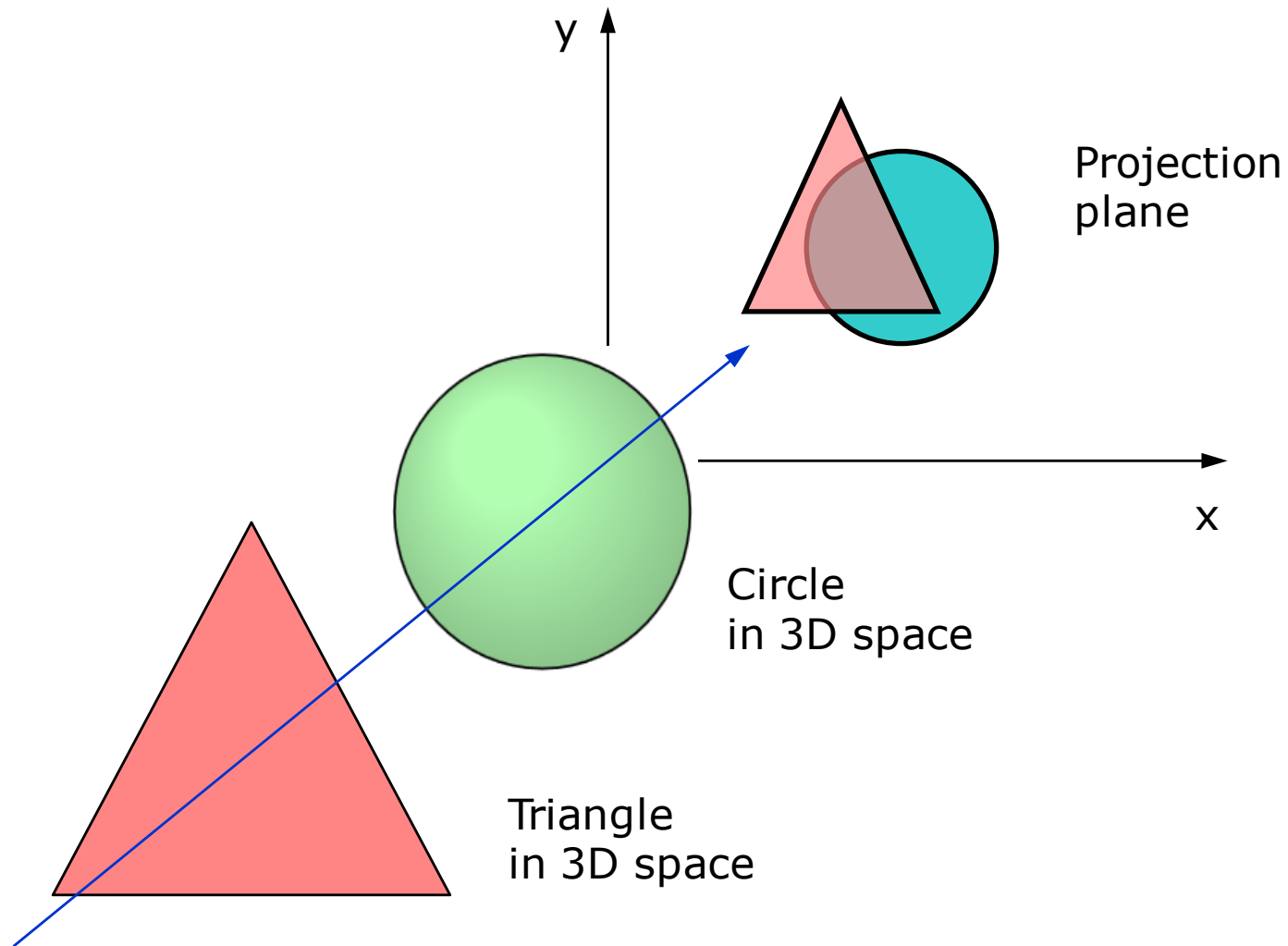
angle test

Testing hierarchy

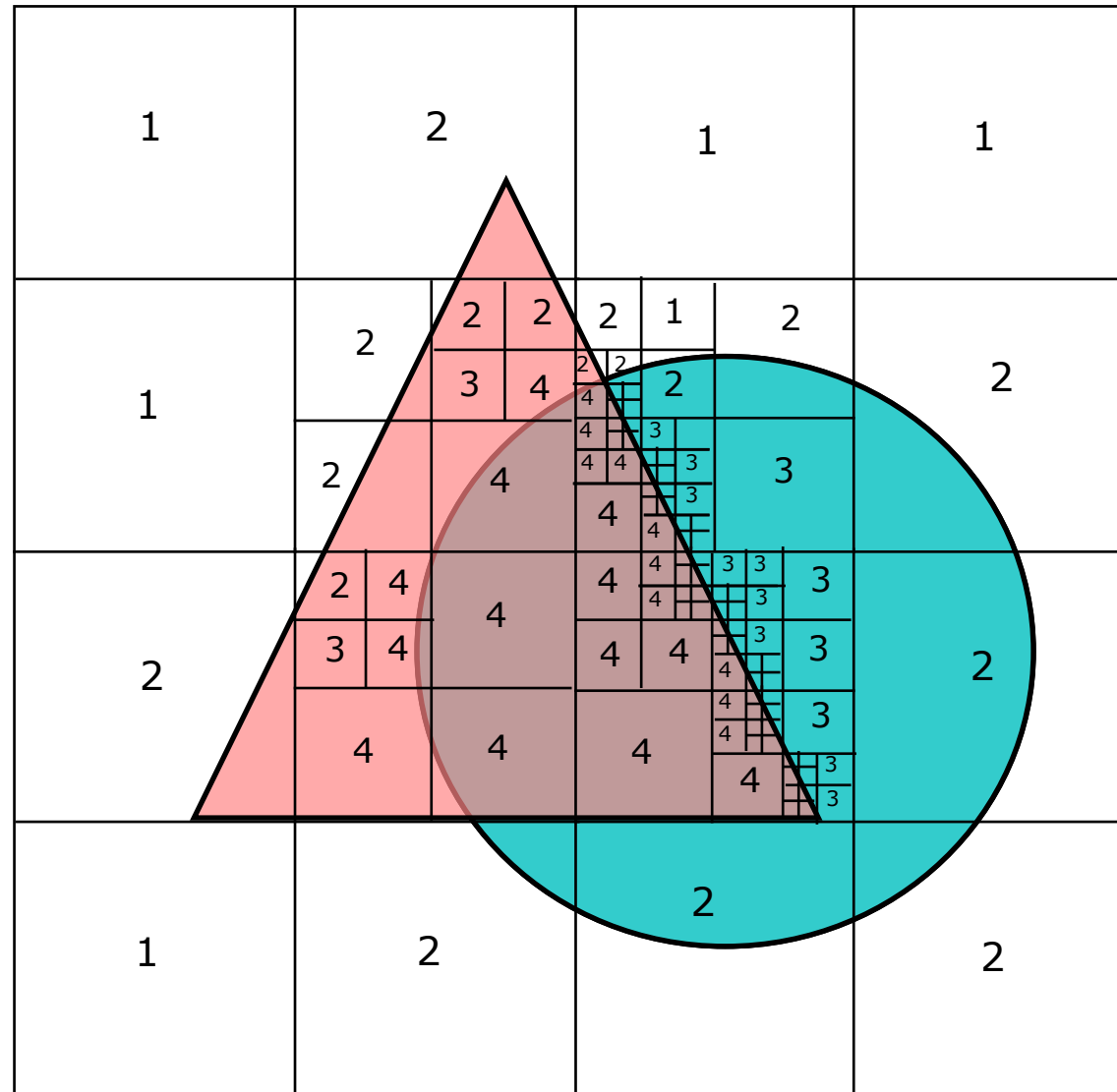
4. More intersected polygons. One of them is a surrounding polygon and it is in front of all of the other polygons.



Warnock algorithm - example

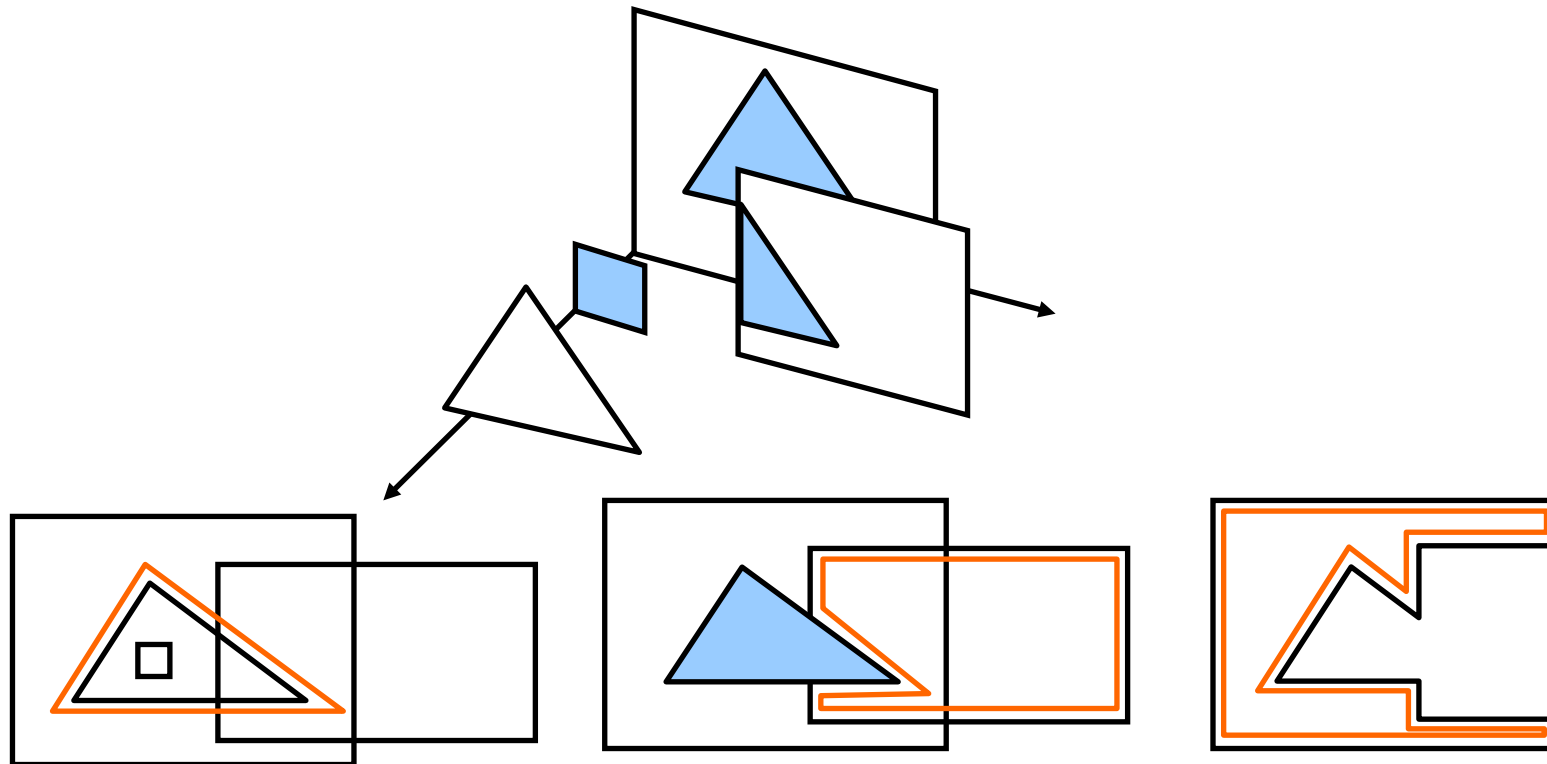


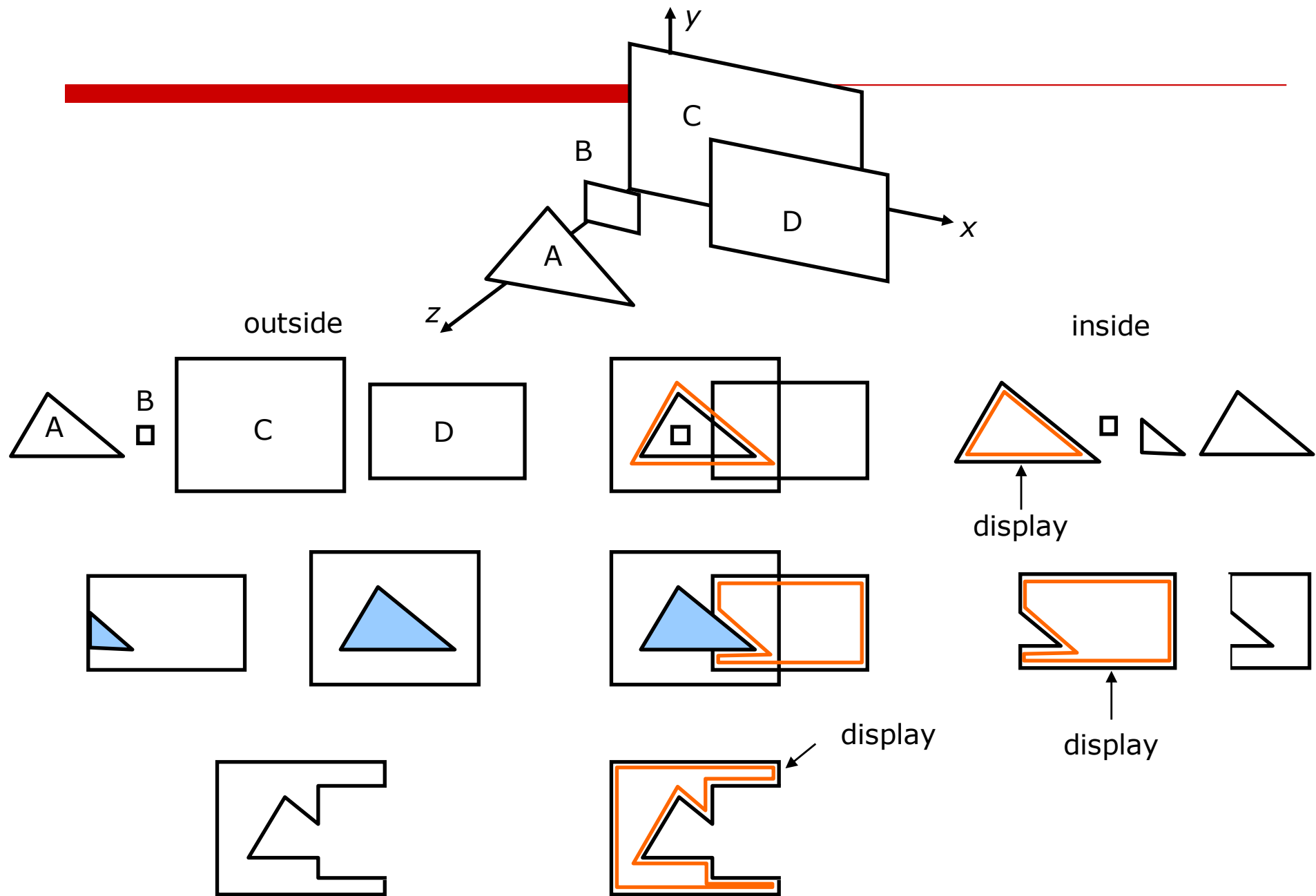
Warnock algorithm - example



3.2 Weiler-Atherton Algorithm

- Minimize number of subdivisions in a Warnock-style algorithm.
- Published in 1977, by Weiler, K., and Atherton, P. "Hidden Surface Removal Using Polygon Area Sorting", SIGGRAPH 77.

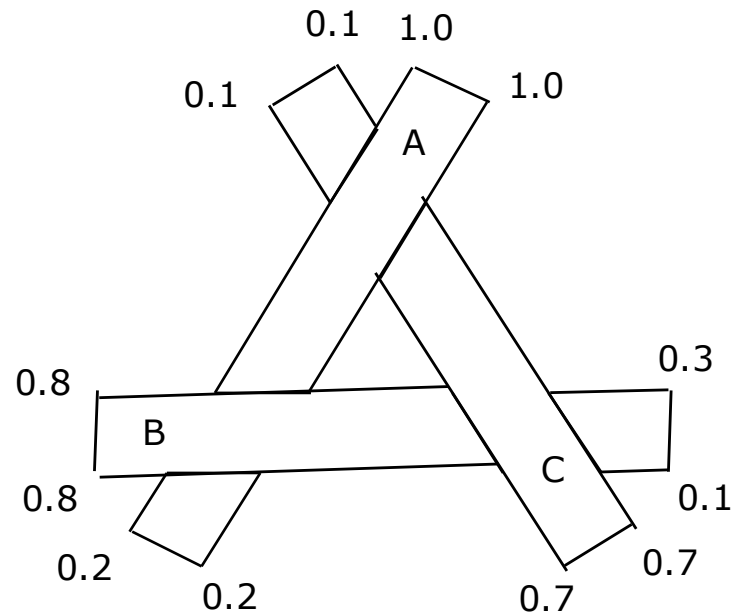




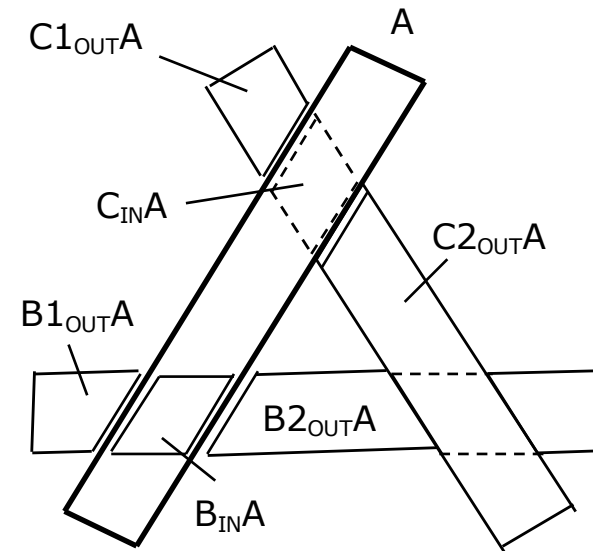
Weiler-Atherton Algorithm

- ❑ Subdivides the screen area along polygon boundaries rather than along rectangle boundaries
- ❑ Requires the Weiler polygon clipping algorithm
- ❑ Basic approach:
 1. Sort polygons on some value of z , such as on their nearest z coordinate
 2. The closest polygon is used to clip all the polygons, including the clip polygon
 3. Result two lists of polygons: the list of the polygons inside and the list of the polygons outside the clip polygon
 4. All polygons on the inside list that are behind the clip polygon are then deleted, since they are invisible
 5. If any polygon on the inside list is closer than it is the clip polygon. It means the initial sort did not give a correct priority order
 6. Each such polygon is processed recursively to clip the pieces on the inside list against it
 7. When the recursive subdivision is over, the inside list is displayed
 8. The algorithm then continues to process the polygons on the outside list.

Weiler-Atherton Algorithm - example



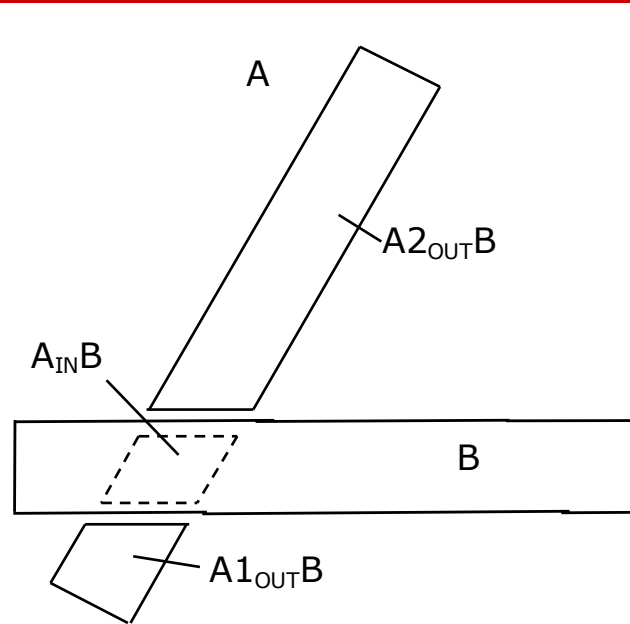
(a) Initially the scene contains the polygons A, B and C.



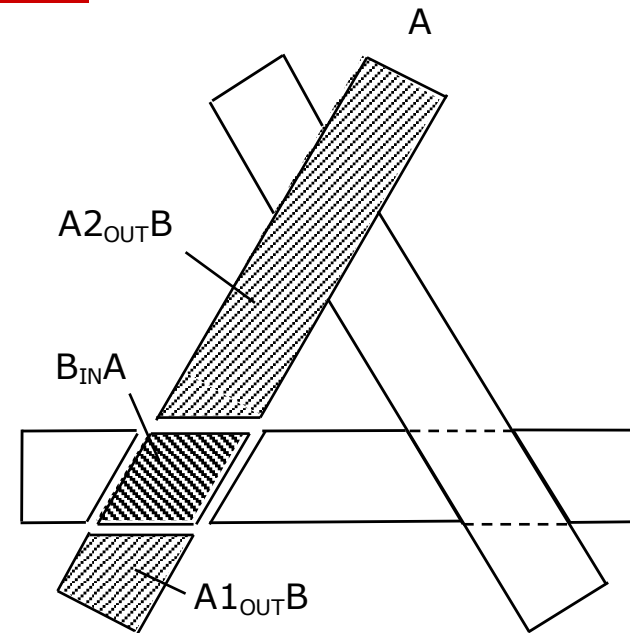
(b) Clipping against the polygon A.

- A is the clip polygon.
- A's inside list: A, $C_{IN}A$ and $B_{IN}A$
 - $C_{IN}A$ behind the A, then deleted. $B_{IN}A$ in front of A, then B the new clip polygon.
- A's outside list: $C1_{OUT}A$, $C2_{OUT}A$, $B1_{OUT}A$ and $B2_{OUT}A$.

Weiler-Atherton Algorithm - example



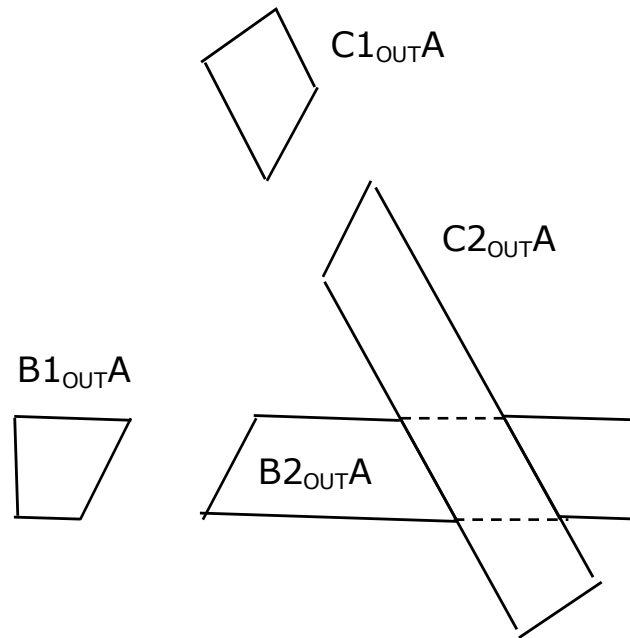
(c) Clipping the A's inside list against B.



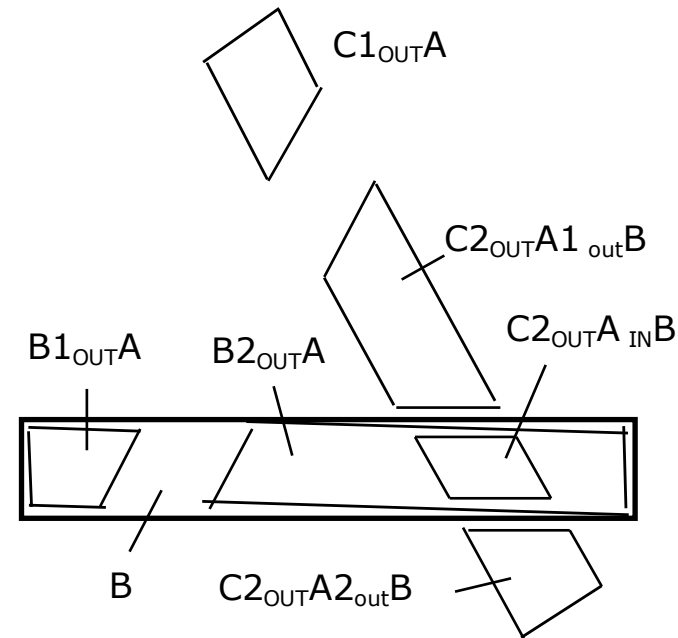
(d) Visible parts of the A's inside list.

- B is the new clip polygon for the A's inside list: A and $B_{IN}A$
- B's inside list: $A_{IN}B$ and $B_{IN}A$
 - $A_{IN}B$ behind B, then deleted. $B_{IN}A$ displayed.
- B's outside list: $A1_{OUT}B$ and $A2_{OUT}B$
 - Visible parts of the B's outside list: $A1_{OUT}B$ și $A2_{OUT}B$

Weiler-Atherton Algorithm - example



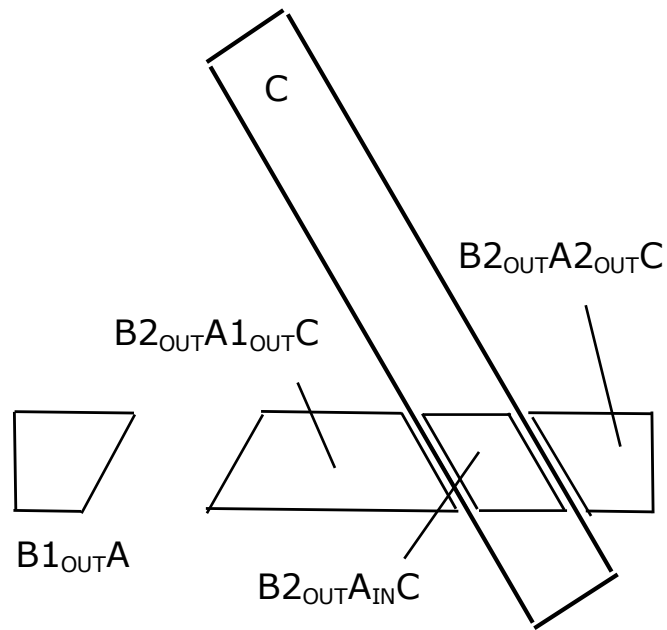
(e) A's outside list.



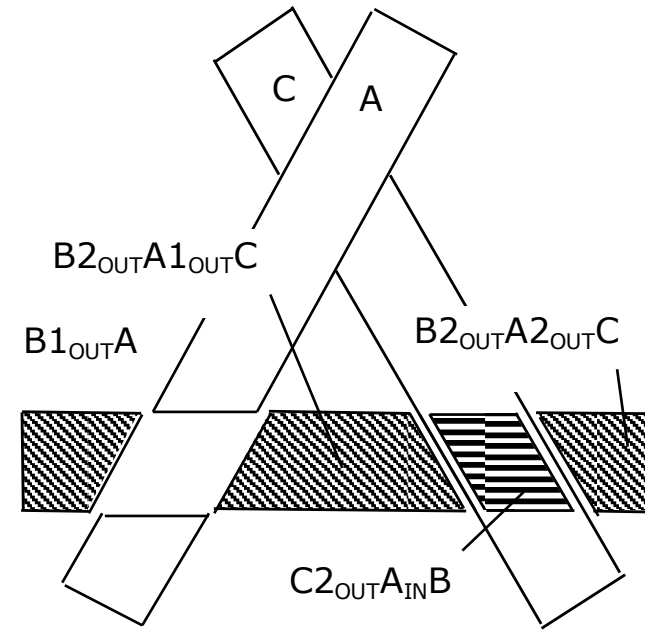
(f) Clipping the A's outside list against polygon B.

- B is the clip polygon for the A's outside list: $C1_{OUTA}$, $C2_{OUTA}$, $B1_{OUTA}$ și $B2_{OUTA}$
- B's inside list: $B1_{OUTA}$, $C2_{OUTA_{INB}}$ and $B2_{OUTA}$
 - $C2_{OUTA_{INB}}$ in front of B, then the new clip polygon is C.
- B's outside list: $C1_{OUTA}$, $C2_{OUTA1_{OUTB}}$ și $C2_{OUTA2_{OUTB}}$

Weiler-Atherton Algorithm - example



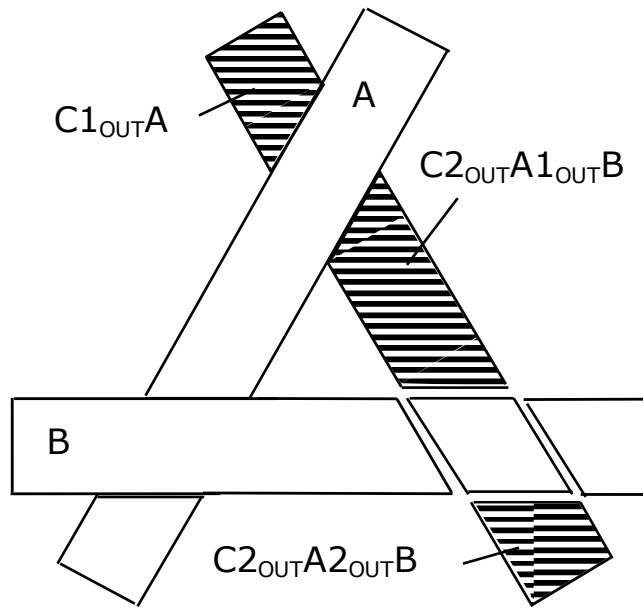
(g) Clipping the B's inside list against C.



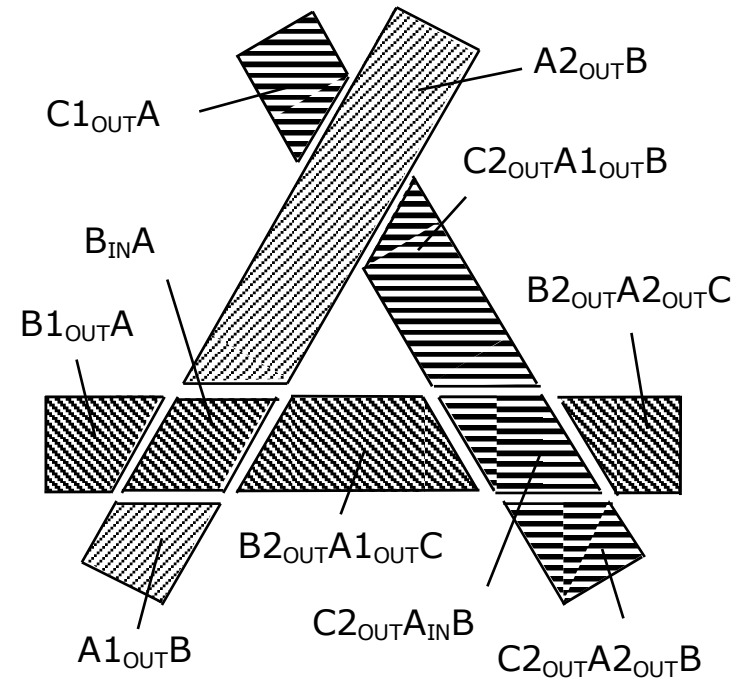
(h) Visible parts of the B's inside list.

- C is the clip polygon for the B's inside list: $B1_{OUT}A$, $C2_{OUT}A_{IN}B$ and $B2_{OUT}A$
- C's inside list: $C2_{OUT}A_{IN}B$ și $B2_{OUT}A_{IN}C$
 - $B2_{OUT}A_{IN}C$ behind C, then it is deleted from the C's inside list. Displays $C2_{OUT}A_{IN}B$.
- C's outside list: $B1_{OUT}A$, $B2_{OUT}A1_{OUT}C$, $B2_{OUT}A2_{OUT}C$
 - Displays the whole C's outside list.

Weiler-Atherton Algorithm - example



(i) Visible parts of the B's outside list.

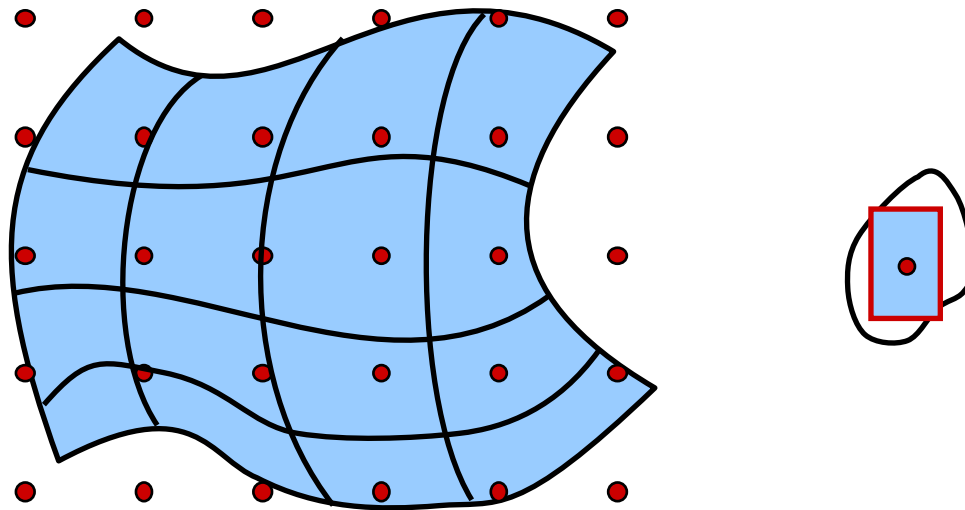


(j) All visible parts displayed at completion.

- Return from recursion and display the B's outside list:
 $C1_{OUT}A$, $C2_{OUT}A1_{OUT}B$ și $C2_{OUT}A2_{OUT}B$.
- Finally display all visible pieces (j).

3.3 Catmull Algorithm (1974)

- ❑ Warnock-style
- ❑ Curved surfaces
- ❑ Subdivide surface (computations in object space rather than image space)
- ❑ Recursively subdivide surface into subpatches until a subpatch covers at most one pixel. A z-buffer algorithm determines whether the patch is visible at this pixel
- ❑ Compute the attribute of the surface at the pixel and display if needed.



Catmull algorithm - pseudocode

- ❑ Published in 1974, by Catmull, E., "A Subdivision Algorithm for Computing Display of Curved Surfaces", PhD Thesis, Univ. of Utah.
- ❑ Catmull recursive-subdivision algorithm:

```
for each patch {  
    push patch onto stack;  
    while stack not empty {  
        pop patch from stack;  
        if patch covers  $\leq 1$  pixel {  
            if patch's pixel closer in z then  
                determine shade and draw  
        } else {  
            subdivide patch into 4 subpatches;  
            push subpatches onto stack  
        }  
    }  
}
```

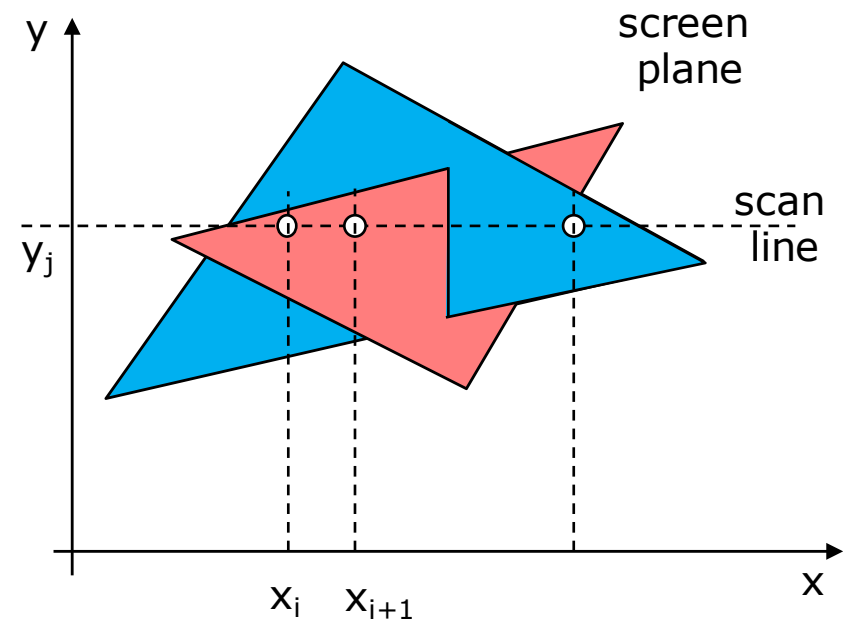
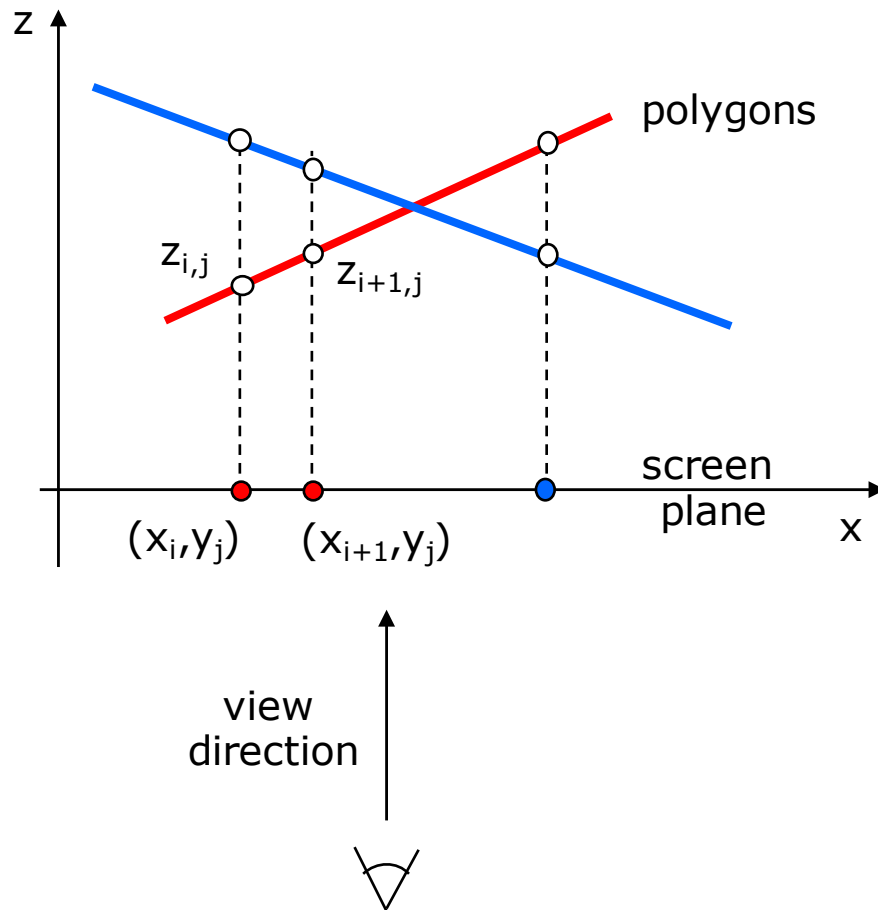
4. Depth-Buffer Algorithms

- ❑ Called **Z-buffer** or depth buffer, developed by Catmull, in 1974.
- ❑ Published in 1974, by Catmull, E., "A Subdivision Algorithm for Computing Display of Curved Surfaces", PhD Thesis, Univ. of Utah.
- ❑ Works with two buffers:
 1. frame buffer F – stores the color values, builds up the image
 2. Z-buffer Z – with the same number of entries, in which a z-value is stored for each pixel
- ❑ Z-buffer is initialized to the largest value that can be stored and represents the z of the far clipping plane (see the frustum definition)
- ❑ Frame buffer is initialized to the background color
- ❑ Polygons are scan-converted into the frame buffer in arbitrary order
- ❑ During the scan-conversion process, if the polygon point being scan-converted at (x,y) is closer from the viewer than is the point whose color and depth are currently in the buffers, then the new point's color and depth replace the old values
- ❑ Complexity
 - Time : $n \times m$ buffer, k polygons $\rightarrow O(nmk)$
 - Space : b depth precision $\rightarrow O(nmb)$

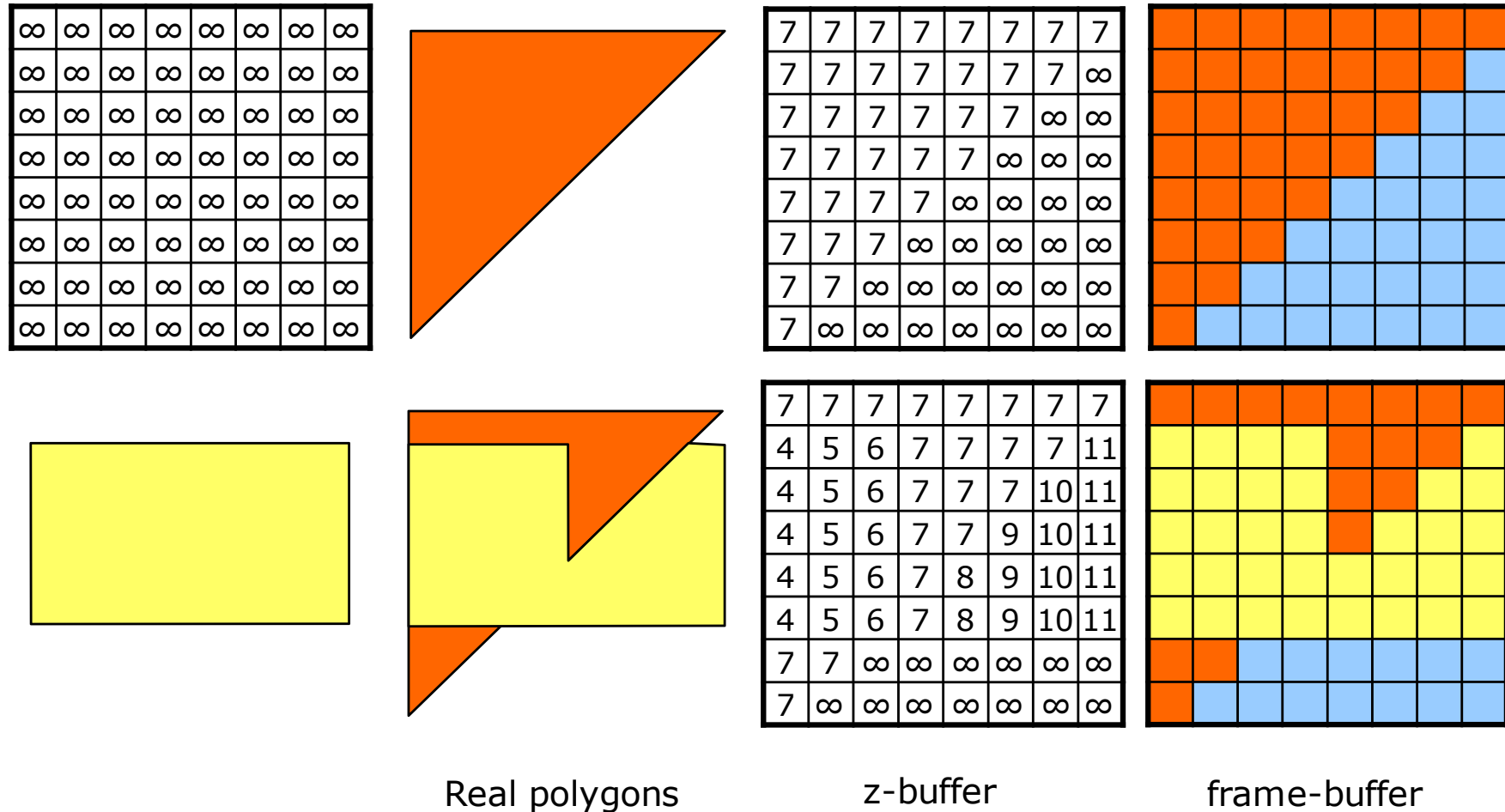
z-buffer algorithm - pseudocode

```
function zBuffer {  
    int pz;  
    for  $y=0$  to YMAX {  
        for  $x=0$  to XMAX {  
            WritePixel(x, y, background_value);  
            WriteZ(x, y, ZMAX)  
        }  
    }  
  
    for each polygon {  
        for each pixel in polygon's projection {  
             $pz = \text{polygon's } z\text{-value at pixel coords } (x,y);$   
            if  $pz \leq \text{ReadZ}(x, y)$  {  
                WriteZ(x, y, pz)  
                WritePixel(x, y, polygon's color at pixel coords (x, y))  
            }  
        }  
    }  
}
```

z-buffer algorithm



z-buffer algorithm - example



z-buffer algorithm – depth coherence

Considering three vertices of the polygonal face:

$V1(x_1, y_1, z_1)$, $V2(x_2, y_2, z_2)$, and $V3(x_3, y_3, z_3)$

The equation of the polygonal face can be computed (see the course “Mathematics in Computer Graphics”), and looks like:

$$Ax + By + Cz + D = 0$$

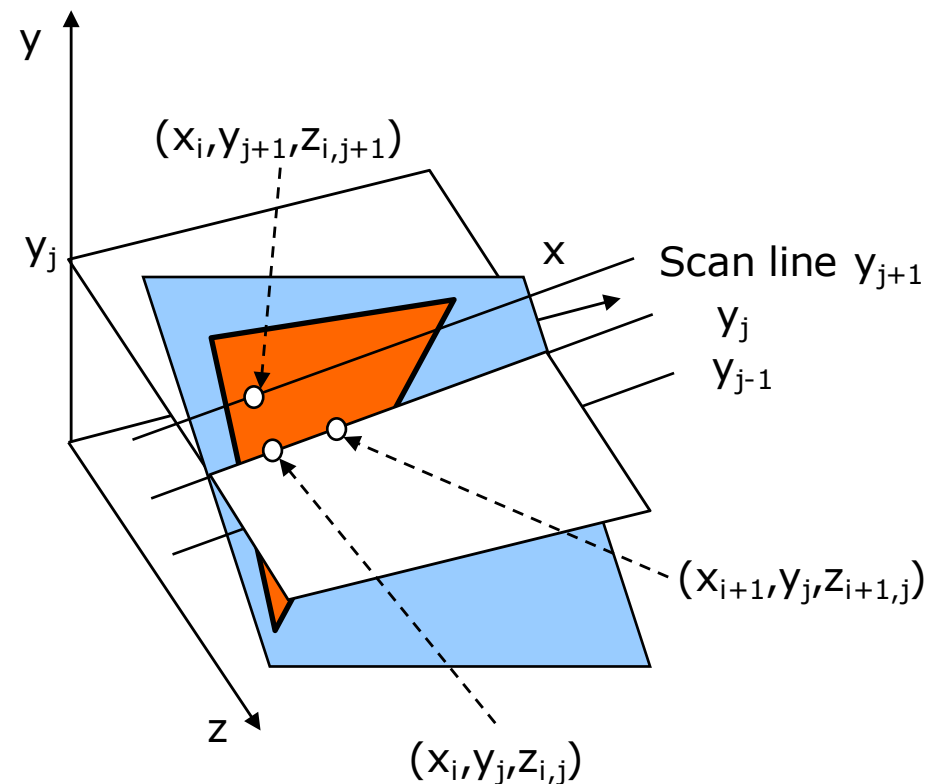
$$z_{i,j} = \frac{-D - Ax_i - By_j}{C}$$

$$z_{i+1,j} = \frac{-D - Ax_{i+1} - By_j}{C}$$

$$x_{i+1} = x_i + 1, y_{j+1} = y_j + \Delta y$$

$$z_{i+1,j} = z_{i,j} - \frac{A}{C}$$

$$z_{i,j+1} = z_{i,j} - \frac{B \Delta y}{C}$$



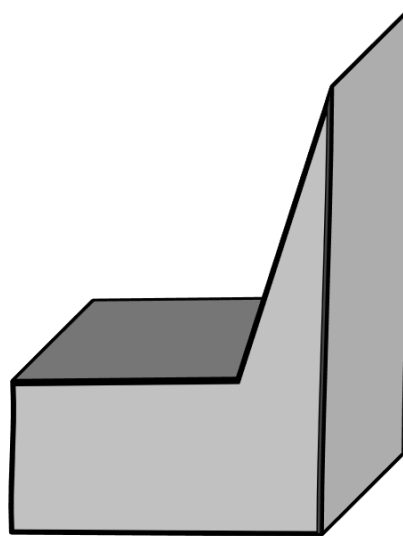
Questions and problems

1. Enunciate and explain two reasons for using the hidden line and surface removal algorithms mainly in one space instead of another (e.g. in the view space rather than in the screen space).
2. Exemplify and explain the concept of depth coherence.
3. Why the hidden line and surface removal algorithms work faster in the case of orthogonal projection than the case of perspective projection?
4. Why the back face culling algorithm works better for convex 3D objects?
5. Why throughout the Warnock algorithm is using the classification of the relationship between the projected polygon and the elementary area?
6. Exemplify and explain the Warnock algorithm in the visualization of: (a) just one triangle; (b) two triangles; (c) one cube.

Questions

7. Specify, relative to the right hand coordinate system, one example of the object's position and orientation, and viewer's position, to get on the the XY plane the image like in the figure 1, by orthogonal projection.
8. Exemplify and explain the Weiler-Artherton algorithm for the visualization of a cube.
9. Exemplify and explain the Weiler-Artherton algorithm, for the hidden line and surface removal, for the 3D object in the figure 1. The resulted image by orthogonal projection is like in the figure.

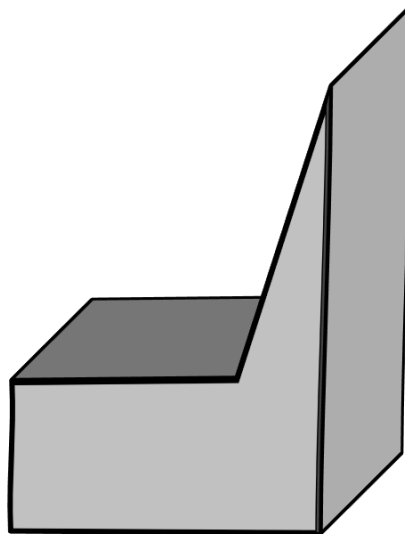
Figure 1.



Questions

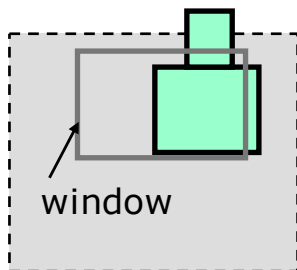
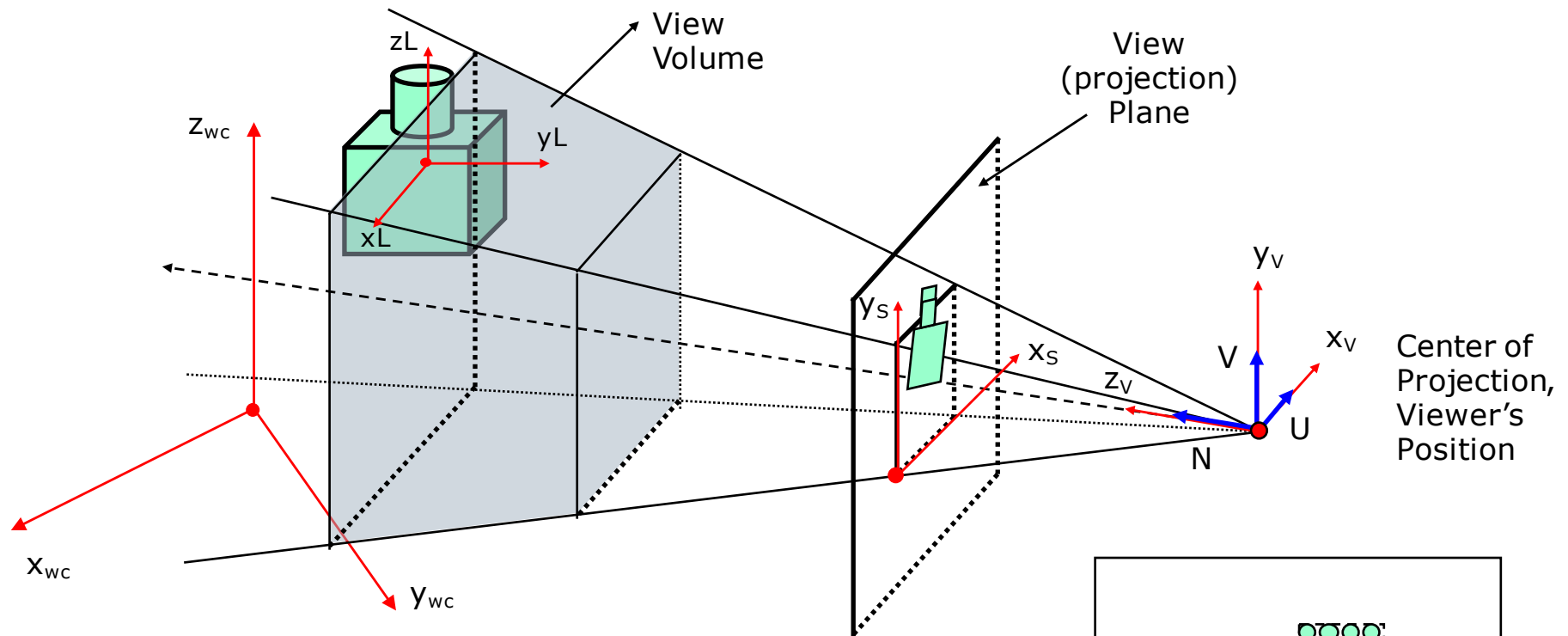
10. Exemplify and explain the z-Buffer algorithm for the visualization of a cube.
11. Exemplify and explain the z-Buffer algorithm, for the hidden line and surface removal, for the 3D object in the figure 1. The resulted image by orthogonal projection is like in the figure.

Figure 1.

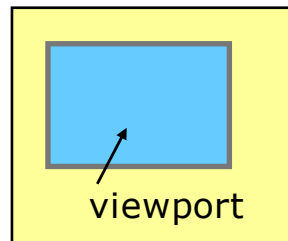


2D Clipping – Weiler Algorithm

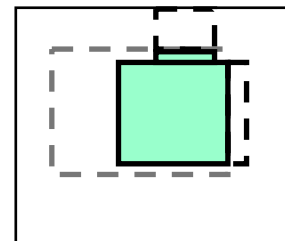
Real objects to image on the screen



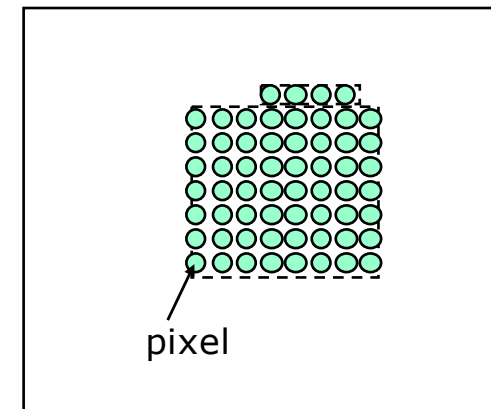
Window definition
in projection plane



Viewport definition
in projection plane

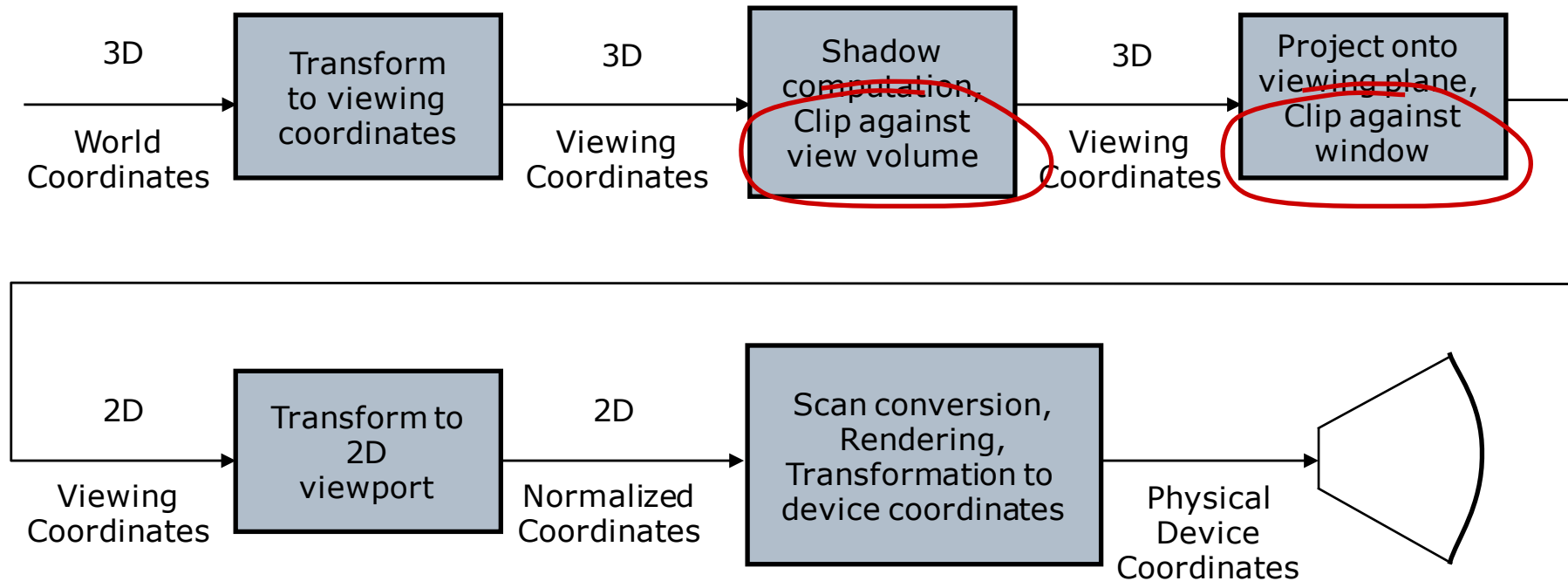


Window's content
inside the viewport

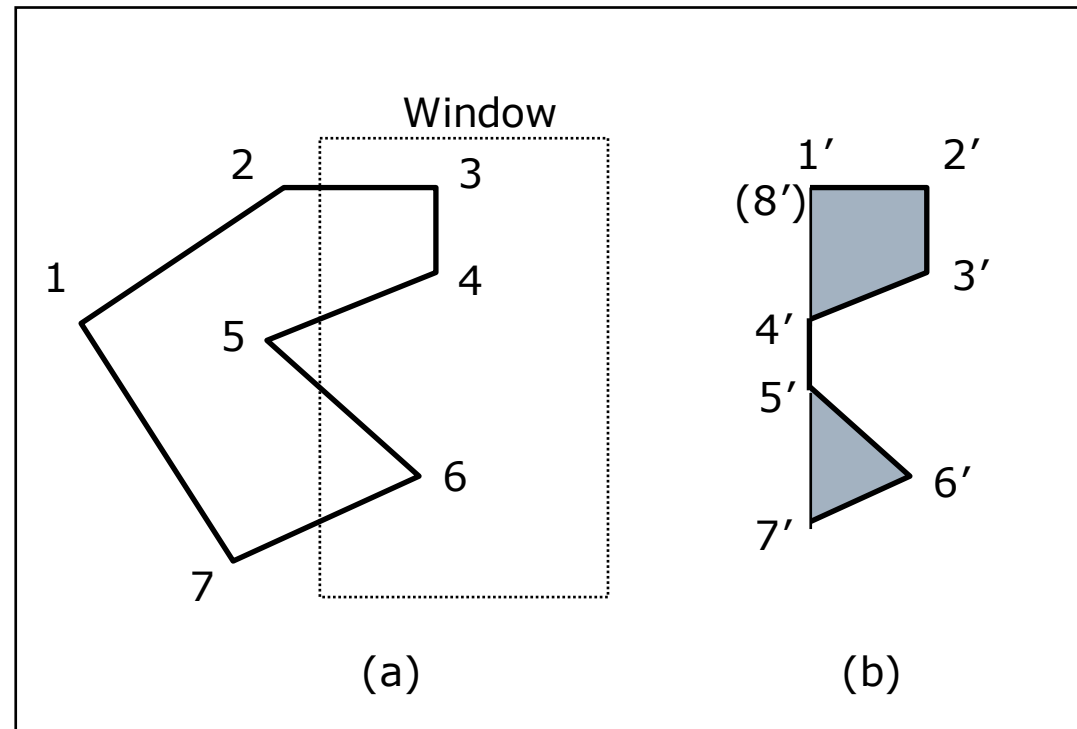
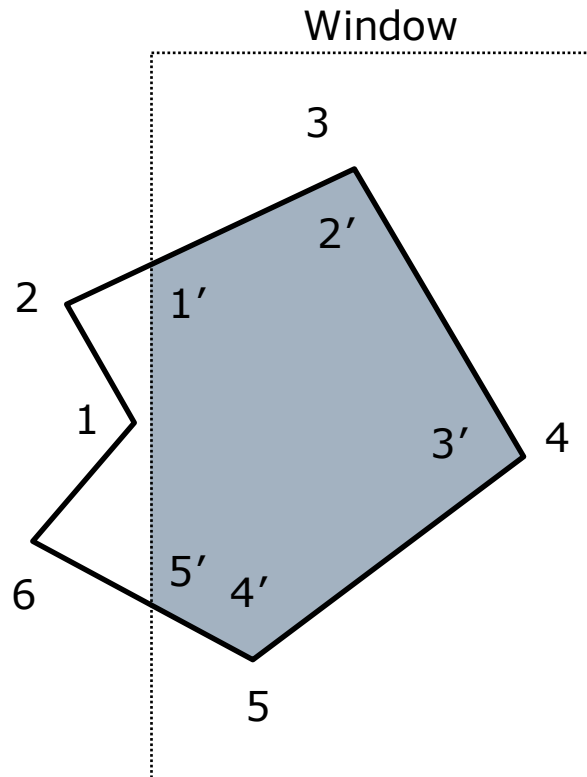


Scan conversion onto the
raster screen

Viewing transformations pipeline



Sutherland Hodgman algorithm

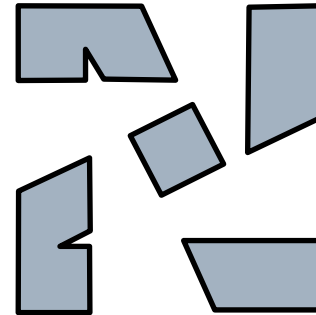


Clipping polygons against polygons

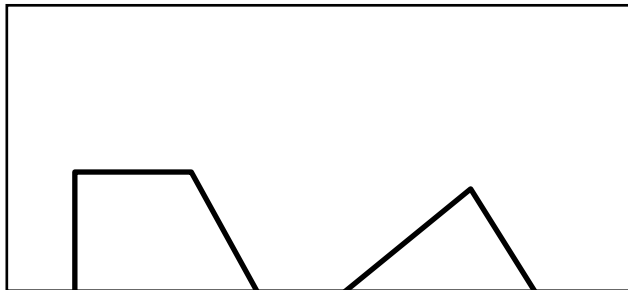
clipped polygons



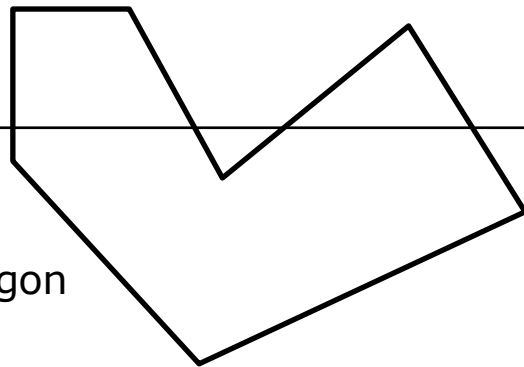
clipped polygons



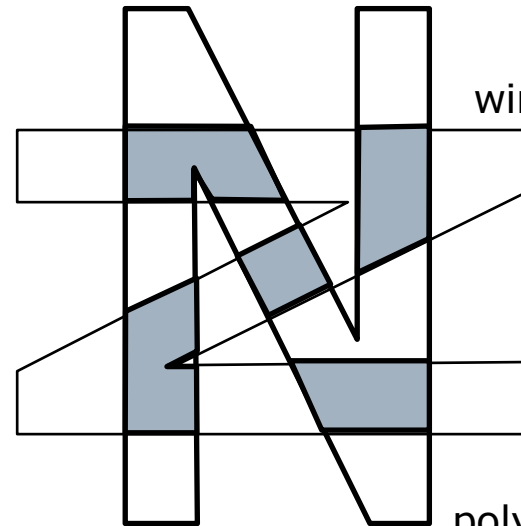
window



polygon



window

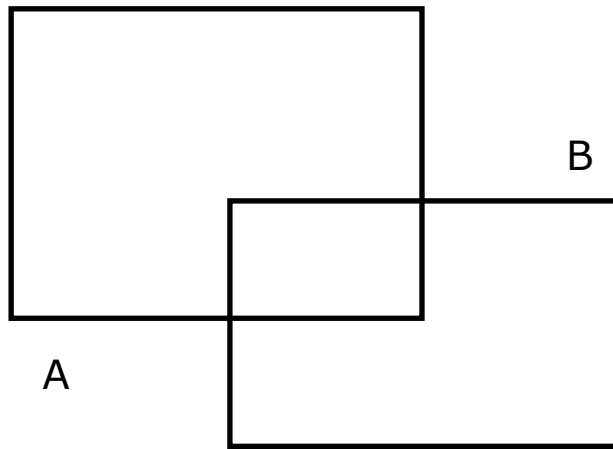


polygon

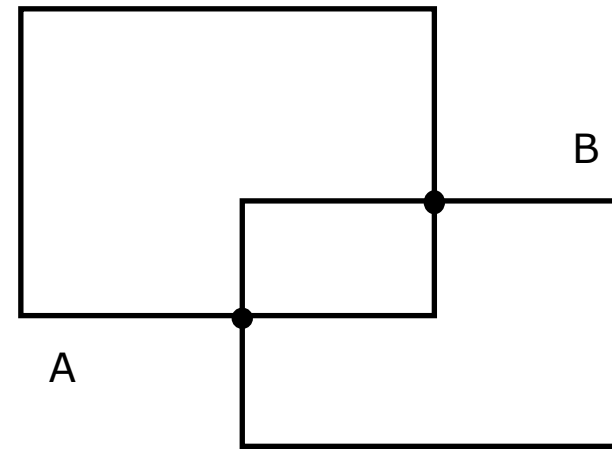
Weiler polygon algorithm (1980)

- ❑ Published in 1980 in SIGGRAPH'80, "Polygon Comparison Using a Graph Representation", by Weiler, K.
- ❑ Clipping one polygon to another arbitrary polygon
- ❑ The clipped polygon may be disconnected and may be nonconvex even the original polygon was convex
- ❑ Works for any number of polygons
- ❑ Basic principle:
 - The edge of the polygon A and the polygon to be clipped B divide the drawing surface into disjoint regions
 - Each of these regions is entirely in A, entirely in B, entirely contained in both, or contained in neither
 - The clipped input polygon consists of the regions contained in both A and B

Weiler polygon algorithm – step 1 and 2

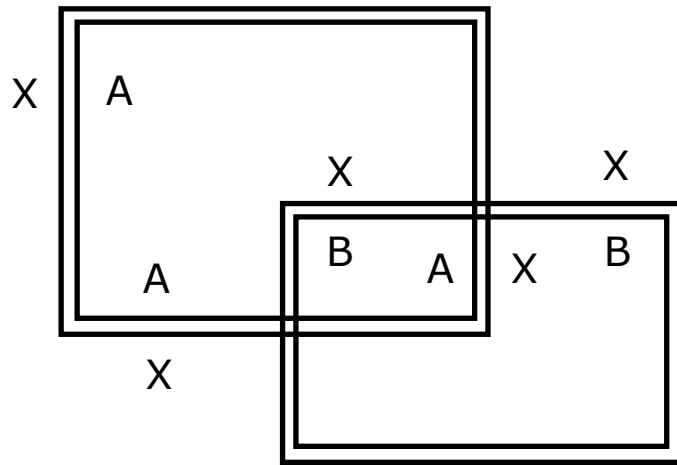


1. Initially two polygons
A and B.

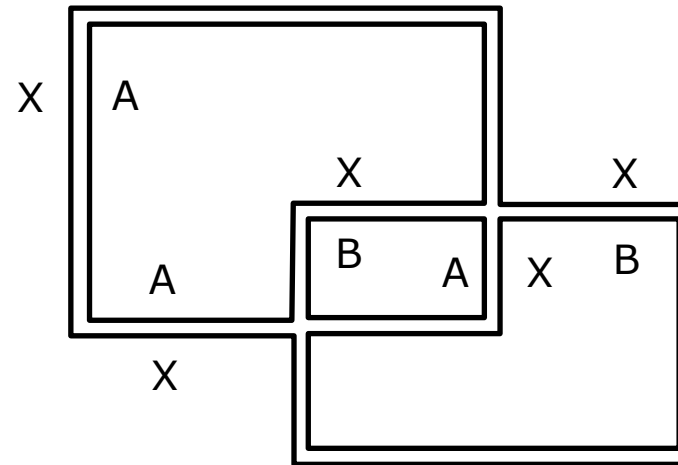


2. Intersections are
added as vertices.

Weiler polygon algorithm – step 3 and 4



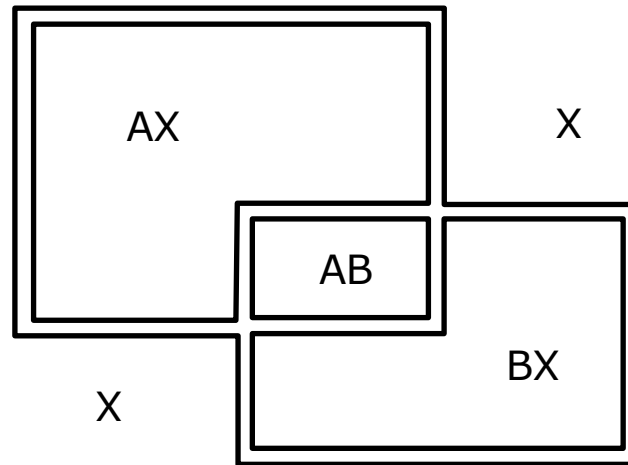
3. The polygons are drawn as doubled contours with labels.



4. The contours are reconnected so they do not cross.

[illegible]

Weiler polygon algorithm – step 5



5. The labels on the contours are collected into labels for the regions they bounded. There are four regions: AX, AB, BX, X.

Weiler polygon algorithm – main phases

1. Set up (steps 1,2)

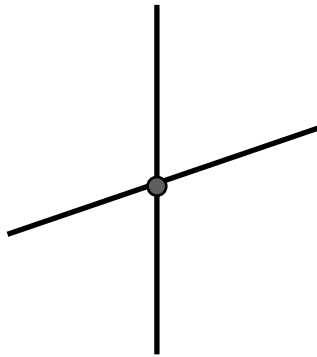
- Determine all intersections of edges of A with edges of B
- All such intersections will be vertices of the output polygon

2. Determine the regions (steps 3,4)

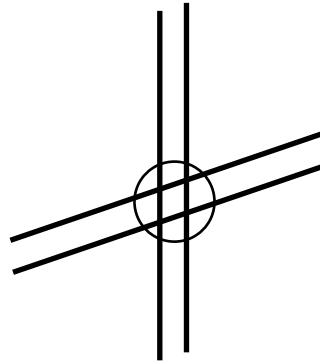
- At each vertex where the two polygons intersect, an adjustment must be made
- The polygons are drawn with double edges. Each edge contributes to two contours
- The contour is a closed polyline that starts and ends at the same edge. The segment is the part of the contour that an edge contributes.
- Rearrange the contours such as no two contours intersect
- The contours define the disjoint regions

3. Select the regions that are in both A and B (step 5)

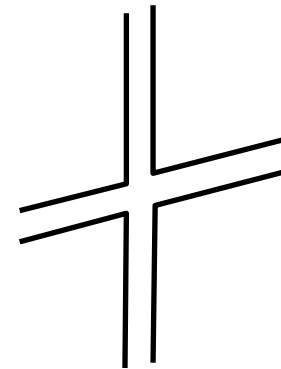
Merging contours for crossing edges



1. The two polygons cross at a vertex.

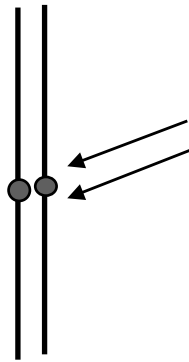


2. The contours for the edges.

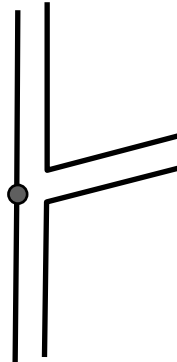


3. The final arrangement of contours.

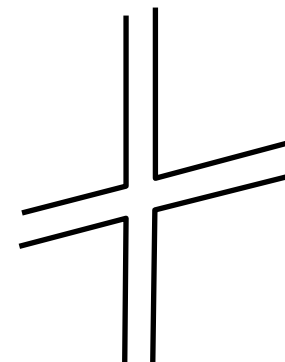
Merging contours for crossing edges



4. Add the contour segments introduced by the edge R to the vertical contour segments.

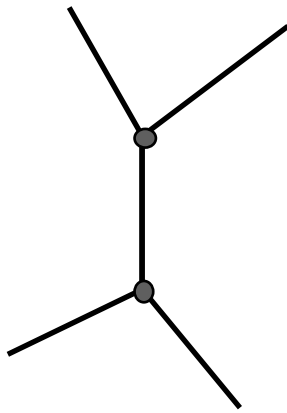


5. The result of R edge addition.

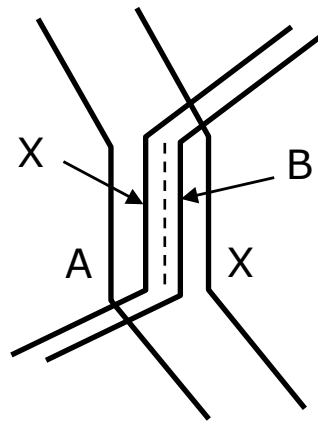


6. The result after merging the R and L edges.

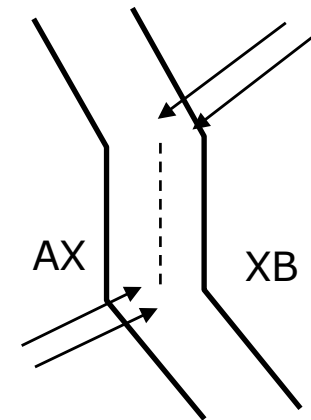
Merging contours for no traverse edges



1. The zigzag contours have vertical coincident edges.

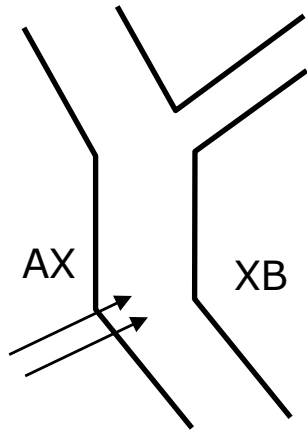


2. The contours associated with this arrangement of edges.

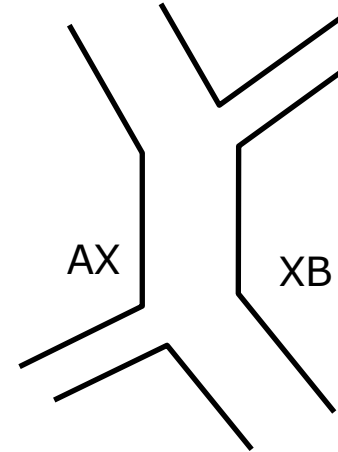


3. The vertical segments from the B polygon have been deleted, and their labels merged with the labels of the other segments.

Merging contours for no traverse edges

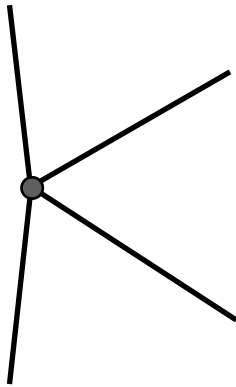


4. One adjoining edge's contour segments have been merged.

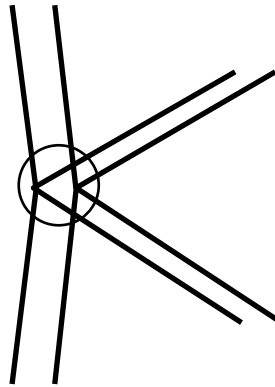


5. Merge the other adjoining edge's contour segments.

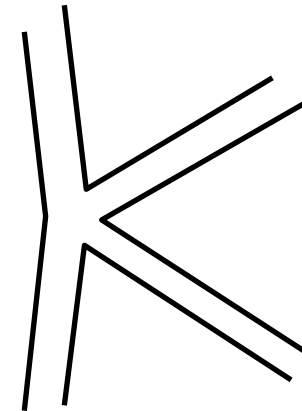
Merging contours at a tangential edges



1. The edge configuration.



2. The initial contour configuration.



3. The final contour configuration.

Data structures in Weiler algorithm

type

```
side = 1..2;           {each polygon edge has two sides}
ends = 1..2;           {each edge has two ends}
direction = (CW, CCW); {Clock wise, counterclockwise directions for contours}
owner = (X, A, B);      {exterior region, A, and B}
owners = set of owner;

cptr = ^contour;        {pointer to contour}
eptr = ^edge;           {pointer to edge}
vptr = ^vertex;         {pointer to vertex}

contour = record
    belongsTo: owners;           {filled in during traversal stage}
    startingPoint = record      {where traversal of contour begins}
        entryEdge: eptr;
        entrySide: side;
    end
end;
```

Data structures in Weiler algorithm

```
edge = record
    vertices: array [ends] of vptr;
    edgeLinks: array [side, direction] of eptr;
    edgeOwners: array [side] of owners;
    contourPtr: array [side] of cptr;

end;

vertex = record
    x, y: real
end;
```

{the ends of the edge}
{double link structure}
{owners of inside and outside}
{points to any contour whose entry point is this edge side}

- ❑ An edge has two vertices and two sides
- ❑ These sides are the segments that initially form the inner and outer contours for the polygons, and contribute to the output contours at the end
- ❑ An edge side is what is called a segment of a contour
- ❑ Each edge side points to its clockwise and counterclockwise neighbors
- ❑ Each edge side also has a list of owners (i.e. labels). Excepting the case of NIL pointer.

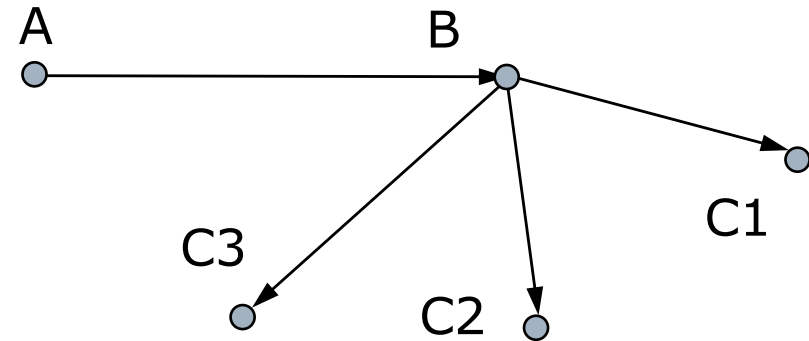
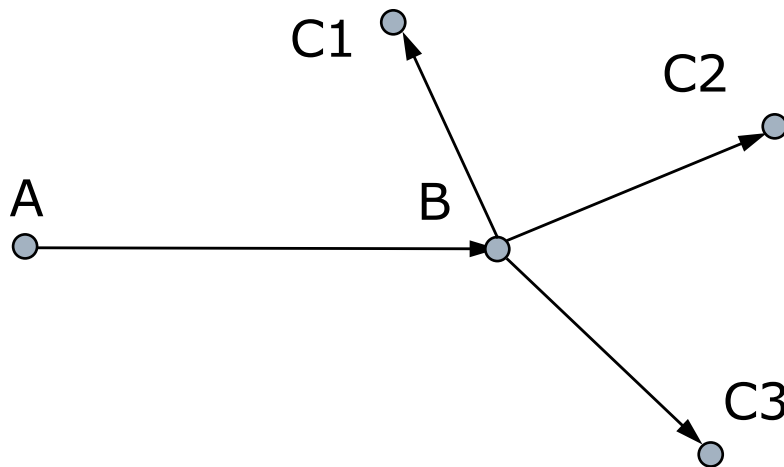
Merging contours

```
for each contour c {  
    c.belongsTo = empty set;  
    for each edge-side e in the contour {  
        if e.contourPtr does not point to c then  
            delete the contour it does point to;  
        c.belongsTo = c.belongsTo + e.edgeOwners  
    }  
}
```

- Now there is a complete collection of contours, each corresponding to a single entry point
- If the input polygons are simple closed curves in the plane, the task is done
 - The output polygons are closed curves
 - No output contour is contained in any other
 - The contour whose owner sets contain both A and B can be simply selected

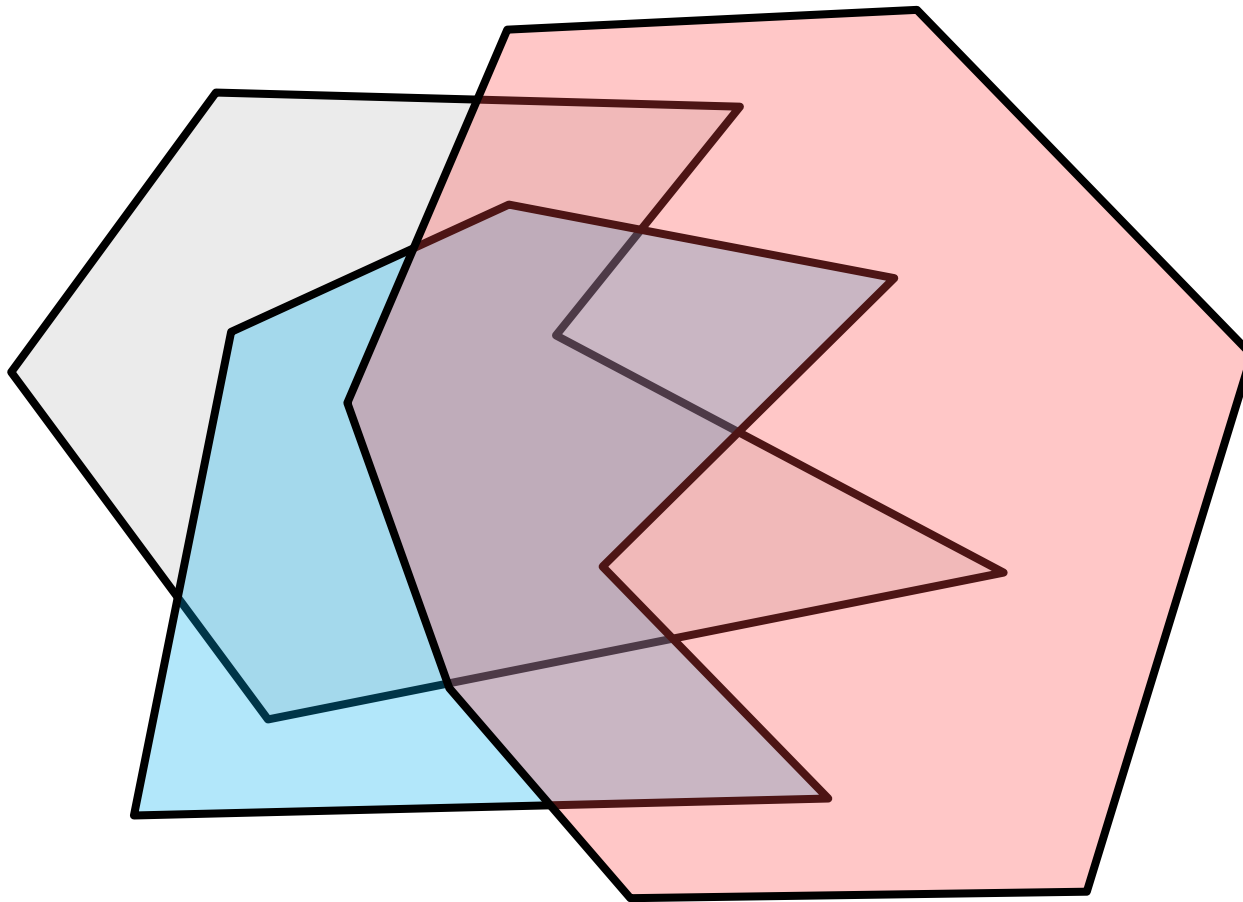
Questions and problems

1. Specify two main differences between the Weiler and Sutherland Hodgman polygon clipping algorithms.
2. Explain an approach to determine the most left edge BC1 from a given set of BC2, BC3 leaving an intersection point B like in the following figure.



Questions and problems

3. Explain the Weiler clipping polygon algorithm on the following set of polygons:

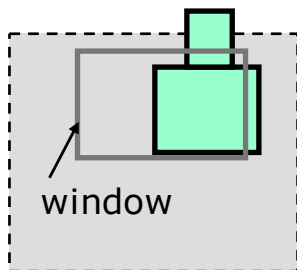
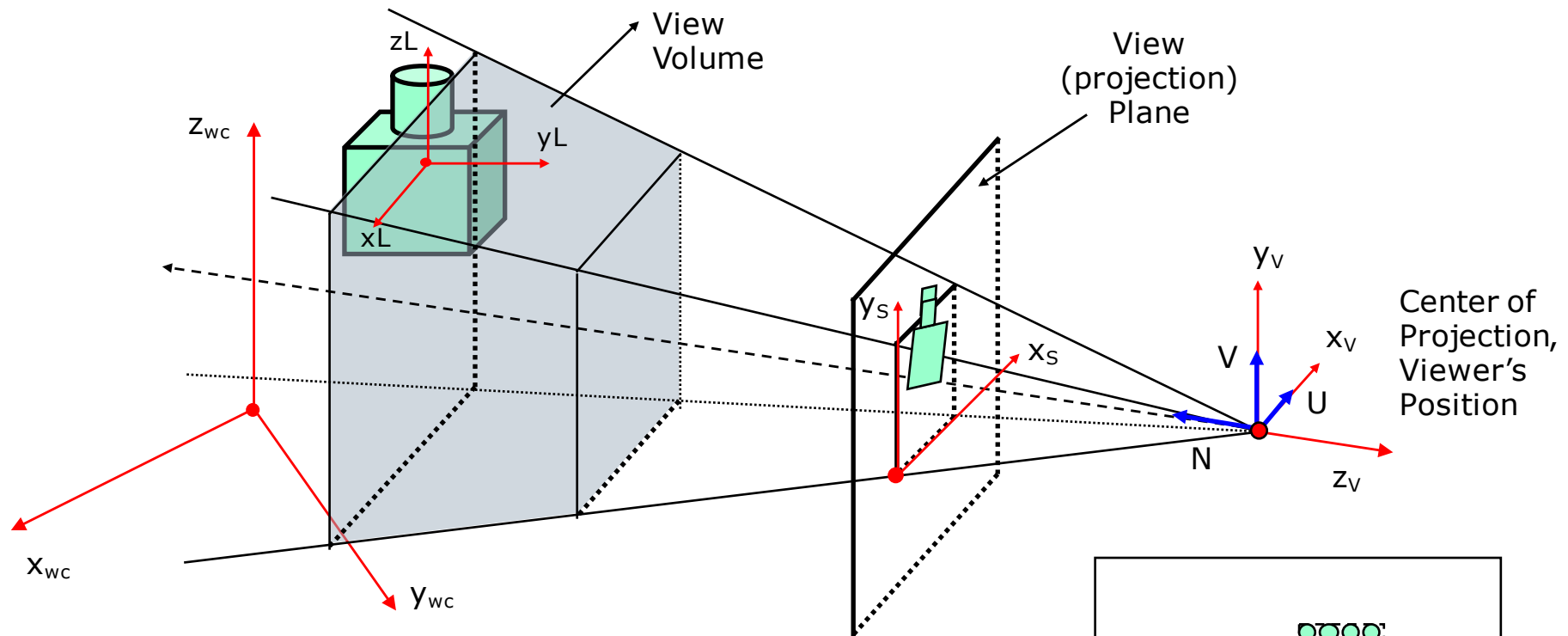


Hidden line and surface removal algorithms (2)

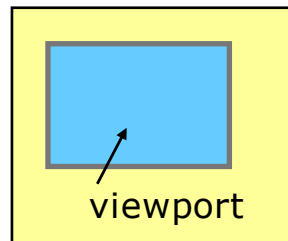
Contents

- Introduction
- Hidden line and surface removal algorithms
 1. Floating horizon algorithm
 2. Back-face culling
 3. Area subdivision algorithm
 4. Depth buffer algorithms
 - 5. Depth sorting algorithms**
 - 6. Scan-line algorithms**
 - 7. Octree algorithm**
 - 8. Ray casting algorithms**
 - 9. BSP trees**

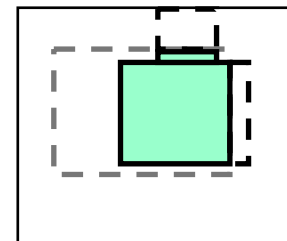
Conceptual model of 3D viewing process



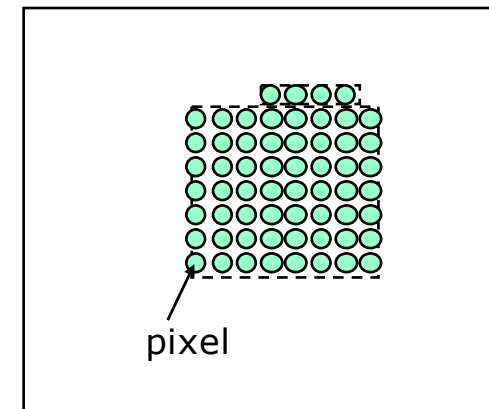
Window definition
in projection plane



Viewport definition
in projection plane



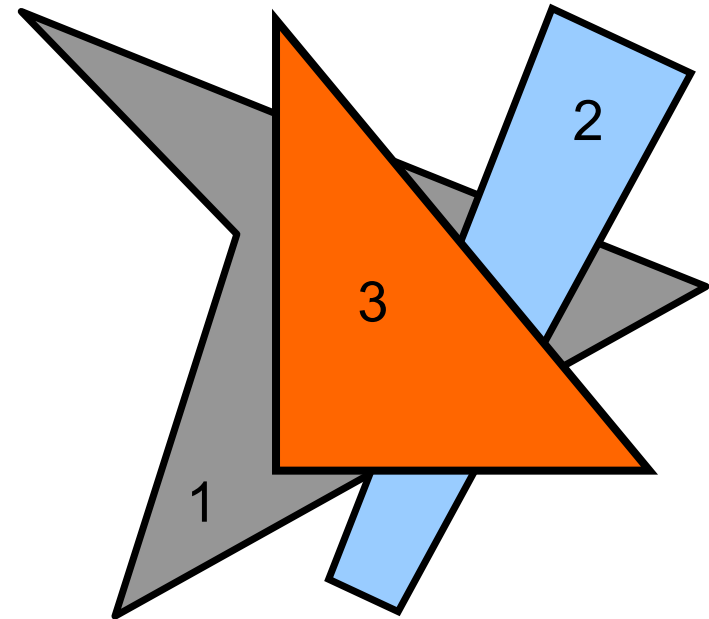
Window's content
inside the viewport



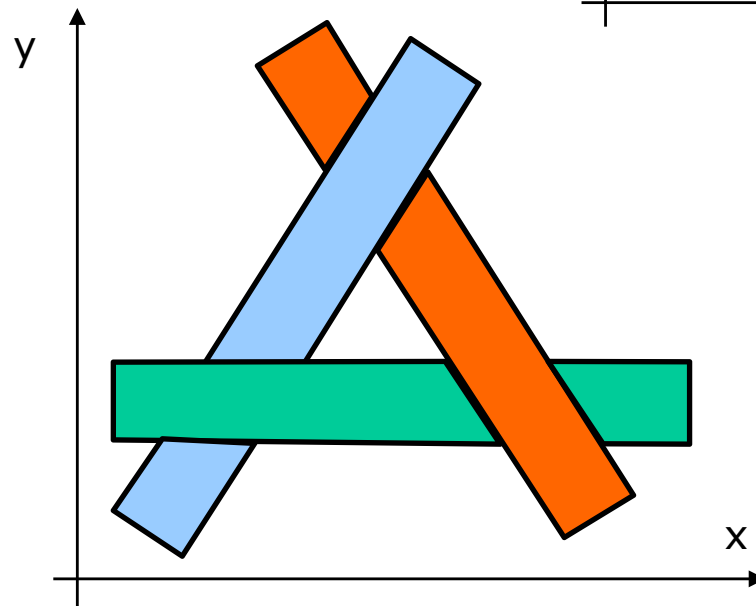
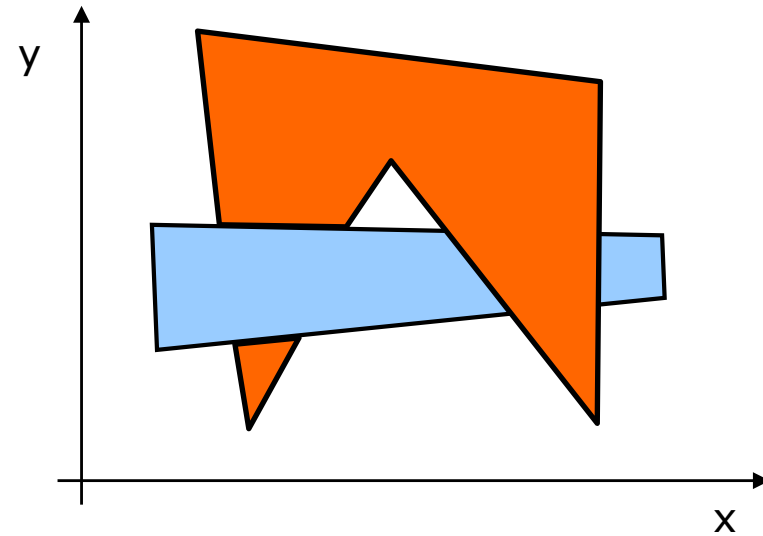
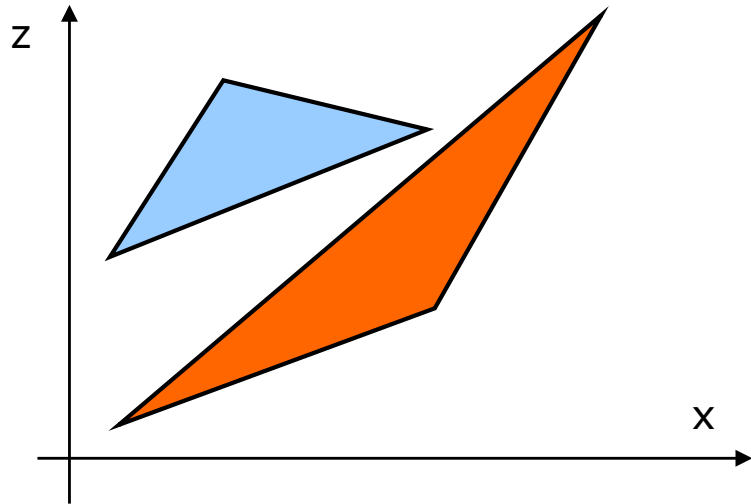
Scan conversion onto the
raster screen

5. Depth Sorting Algorithm

- ❑ Published in 1972, by Newell, M.E., Newell, R.G., Sancha, T.L., "A Solution to the Hidden Surface Problem", Proceedings of the ACM National Conference.
- ❑ Known as painter's algorithm
 - paint the polygons into the frame buffer in order of decreasing distance from the viewpoint.
- ❑ The algorithm consists of three steps:
 1. Sort all polygons according to the greatest (furthest) z coordinate of each
 2. Resolve any ambiguities that may cause when the polygons' z extents overlap, splitting polygons if necessary
 3. Scan convert each polygon in descending order of greatest z coordinate (i.e. back to front).



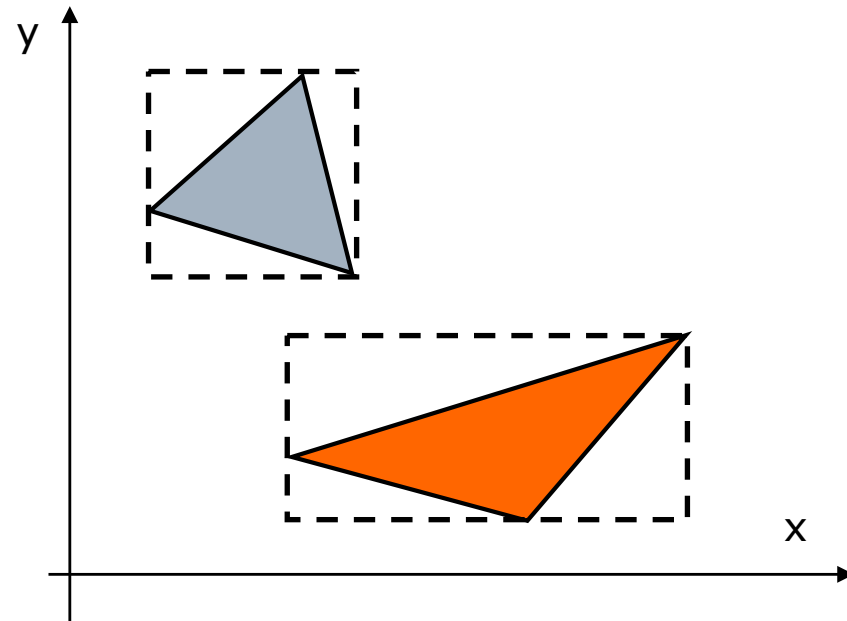
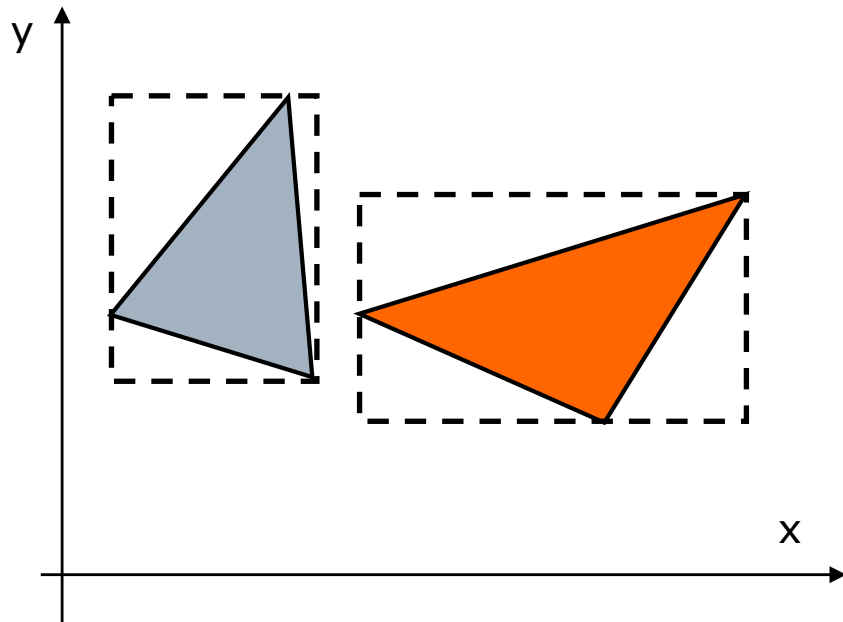
Cases of ambiguities



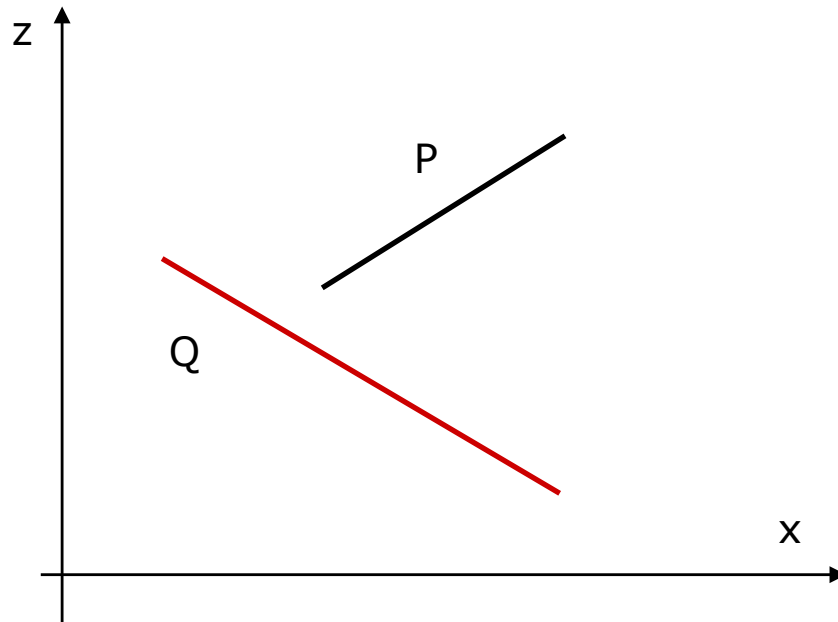
Depth sorting algorithm – 5 tests

- ❑ Let us consider the polygon P at the far end of the sorted list
- ❑ Before scan converting P into the frame buffer, it is tested against each polygon Q whose z extent overlaps the z extent of P. The test proves P cannot obscure Q and that P can be written in the list before Q.
- ❑ There are 5 tests in order of increasing complexity:
 1. Polygons' x extents not overlap.
 2. Polygons' y extents not overlap.
 3. Entire P on the opposite side of Q's plane from the viewpoint.
 4. Entire Q on the same side of P's plane as the viewpoint.
 5. Projections of the polygons onto the (x,y) plane not overlap.
- ❑ If all 5 tests fail, it can be assumed for the moment that P actually obscures Q, and therefore test whether Q can be scan-converted before P.
- ❑ The tests 3 and 4 are performed, with the polygons reversed:
 - 1) Entire Q on the opposite side of P's plane from the viewpoint
 - 2) Entire P on the same side of Q's plane as the viewpoint
- ❑ If the tests 3) and 4) succeed the polygon Q is moved to the end of the list and it becomes the new P. The Q polygon is marked to avoid the infinitely swapping.
- ❑ If the tests 3) and 4) fail again it means that either P or Q must be split by the plane of the other. The unsplit polygon is discharged, and its pieces are inserted into the list in proper z order, and the algorithm proceeds as before.

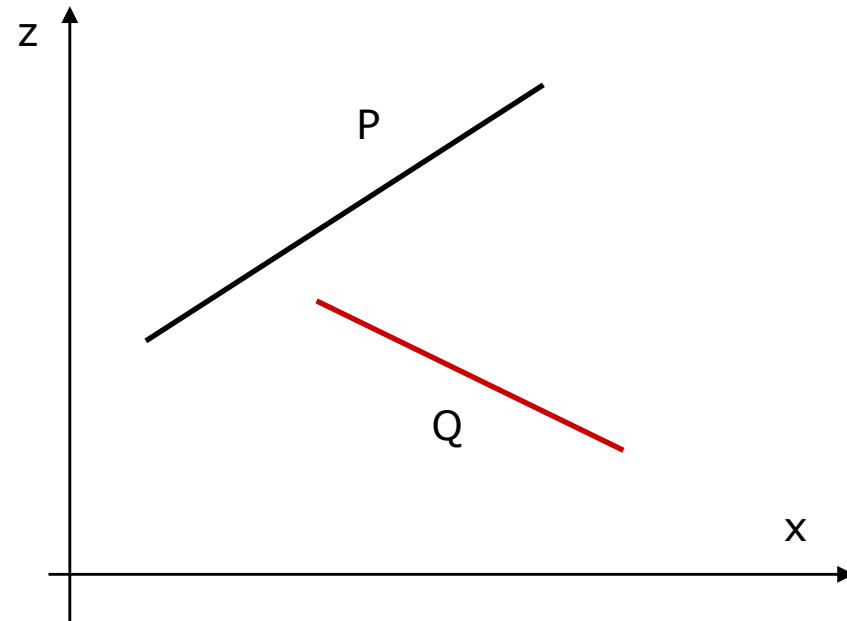
Test 1 and 2



Test 3 and 4



Test 3: draw P, Q

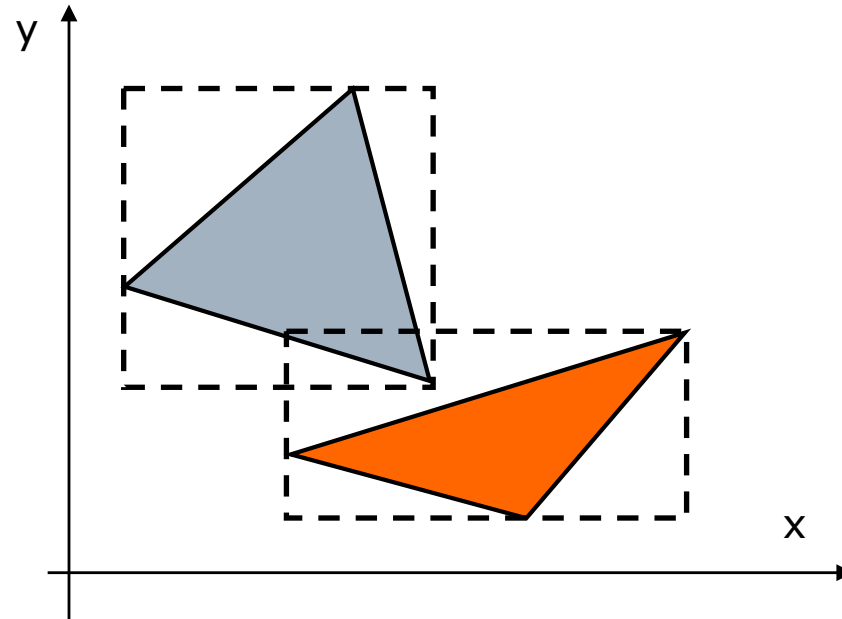


Test 4: draw P, Q

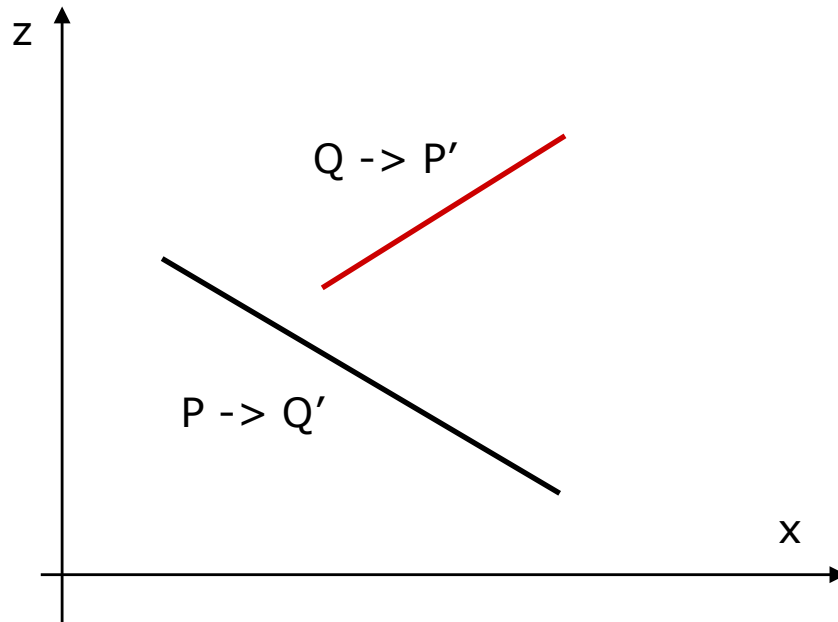


Viewing direction

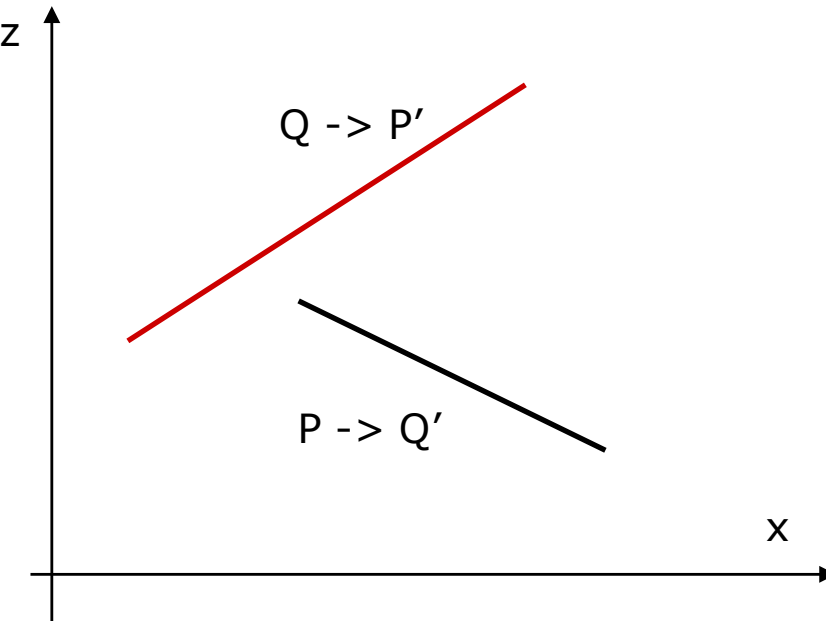
Test 5



Test 3) and 4) – swap P and Q



Test 3): test 3 for P and Q fails,
than test 3 for P' and Q', draw P', Q'



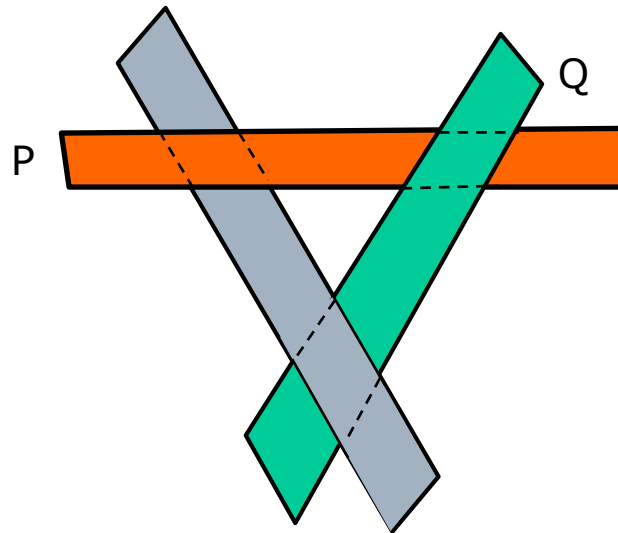
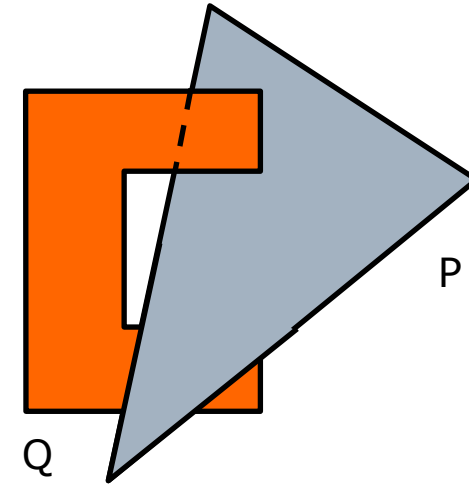
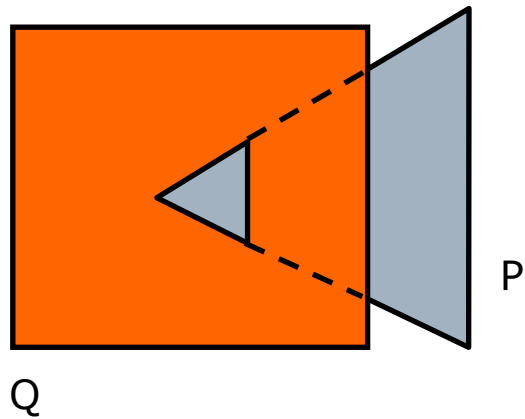
Test 4): test 4 for P and Q fails,
Than test 4 for P' and Q', draw P', Q'

Viewing direction



Cycle testing and splitting

If not successful in box testing and swapping, then split P

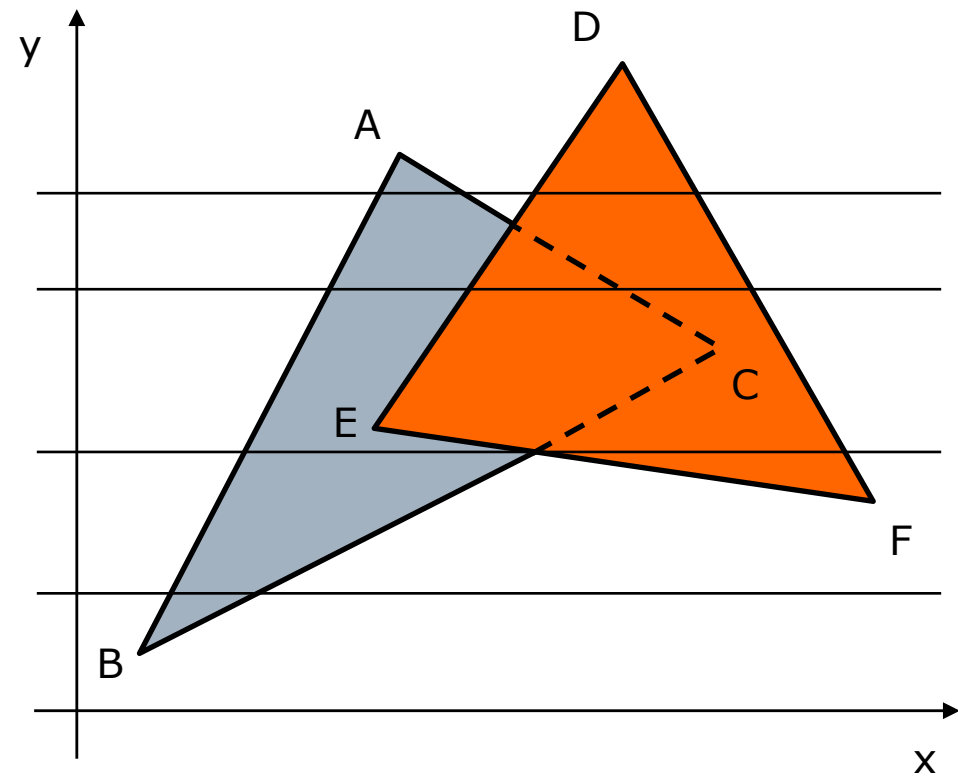


6. Scan-Line Algorithms

- ❑ Wylie et al. (1967), W. Bouknight (1970), G. Watkins (1970)
- ❑ Published in 1967, by Wylie, C., Romney, G.W., Evans, D.C., Erdahl, A.C., "Halftone Perspective Drawings by Computer", FJCC 67.
- ❑ Published in 1970, by Bouknight, W.J., "A Procedure for Generation of Three-Dimensional Half-Toned Computer Graphics Presentations", CACM, vol 13(9).
- ❑ Published in 1970, by Watkins, G.S., "A Real Time Visible Surface Algorithm", PhD Thesis, Computer Science Dept., Univ. of Utah.
- ❑ Extension of the Ordered Edge List Algorithm (see course "Scan-Conversion(3)")
- ❑ Uses the scan line coherence and edge coherence
- ❑ Works with all polygons that define an object rather than a single polygon as in the Ordered Edge List Algorithm

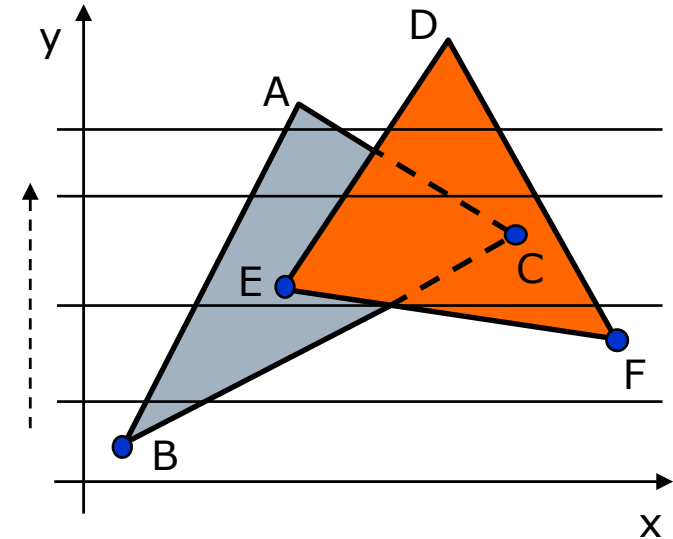
Algorithm steps

1. Create Edge Table (ET)
2. Create Polygon Table (PT)
3. Create Active-Edge Table (AET)



Create Edge Table (ET)

- All no horizontal edges
- Horizontal edges are ignored
- Edges sorted into buckets based on each edge's smaller y coordinate
- Within each bucket the edges are ordered by increasing x coordinate of their lower endpoint
- Each table entry contains:
 1. x coordinate of the end with the smaller y coordinate
 2. y coordinate of the edge's other end
 3. x increment Δx , is the step from one scan line to the next. Δx is the inverse slope of the edge ($\Delta y = m\Delta x$, $\Delta y = 1$, $\Delta x = 1/m$)
 4. Polygon identification number is the polygon to which the edge belongs

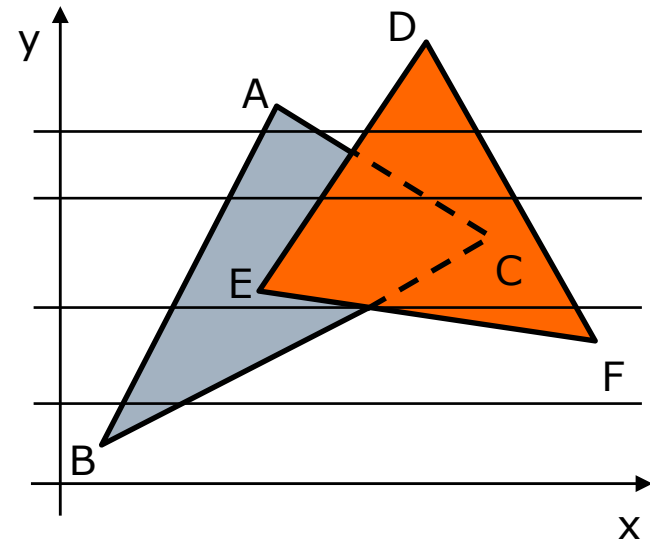


- ET:
- AC
 - DE
 - EF, DF
 - AB, BC

x	y _{max}	Δx	ID	next edge
---	------------------	------------	----	-----------

Create Polygon Table (PT)

- One entry for each polygon
- Each table entry contains the polygon ID
- Each polygon is described by at least the following information:
 1. Coefficients of the plane equation (i.e. A, B, C, D)
 2. Shading or color information
 3. An in-out Boolean flag, initialized to false



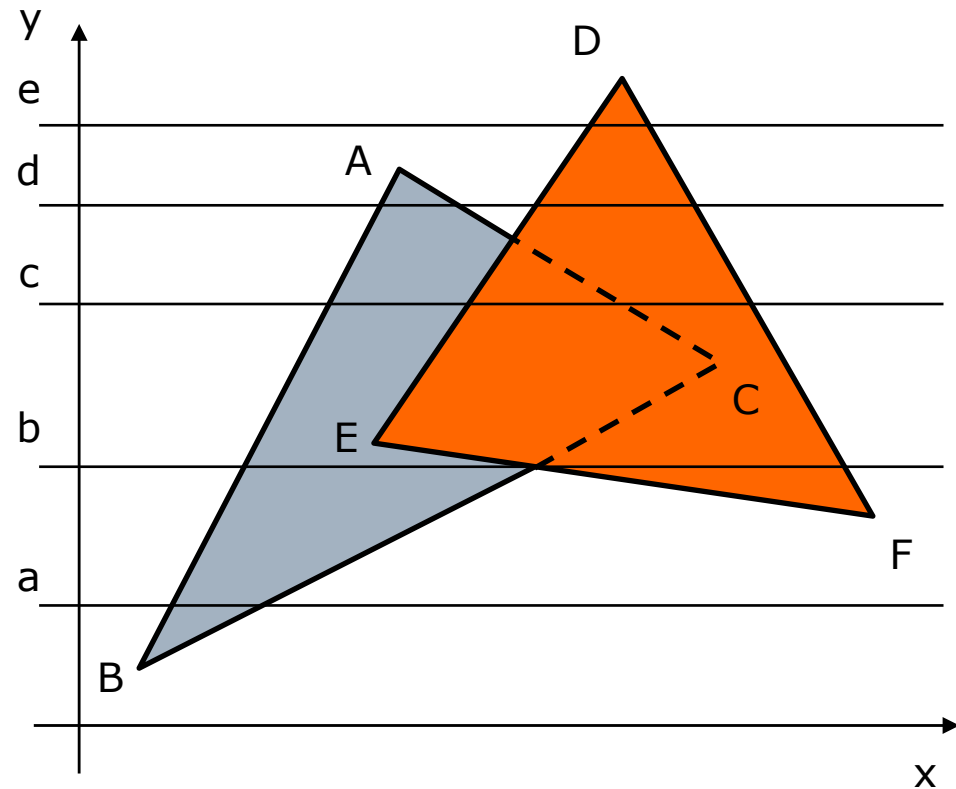
- PT:
 - ABC
 - DEF

ID	A,B,C,D	Shading info	in-out flag
----	---------	-----------------	----------------

Create Active-Edge Table (AET)

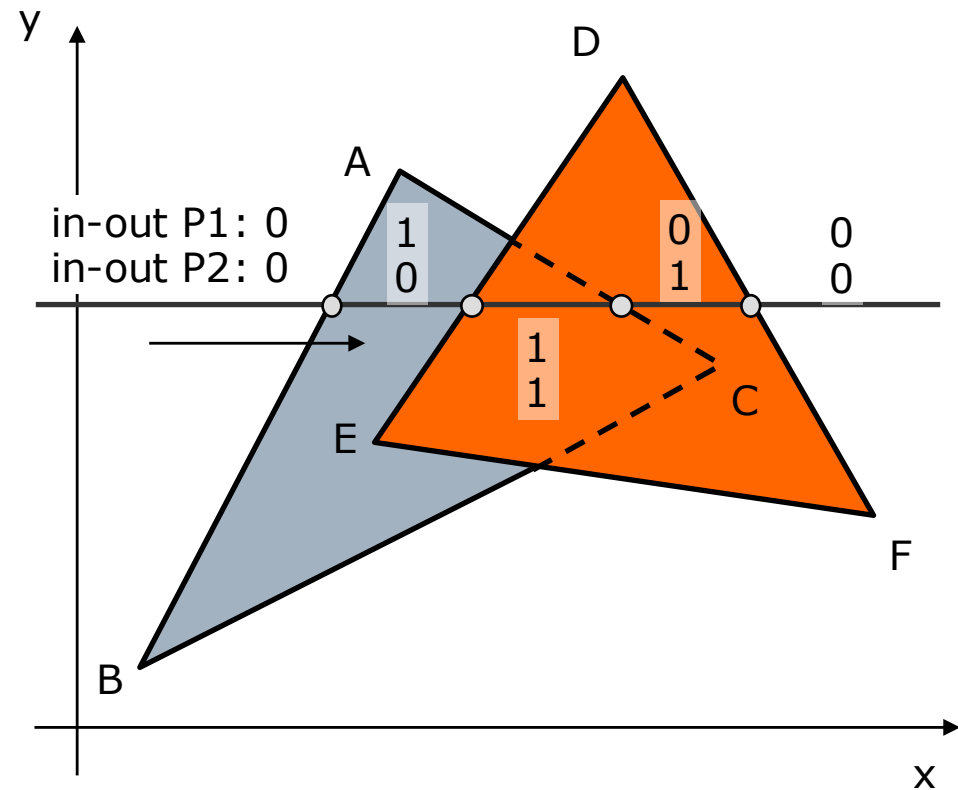
- Contains the edges intersected by the current scan line
- Edges are ordered by the x coordinate of the intersections between the active edges and the current scan line

- AET:
 - a: AB, BC
 - b: AB, EF, BC, DF
 - c: AB, DE, AC, DF
 - d: AB, AC, DE, DF
 - e: DE, DF



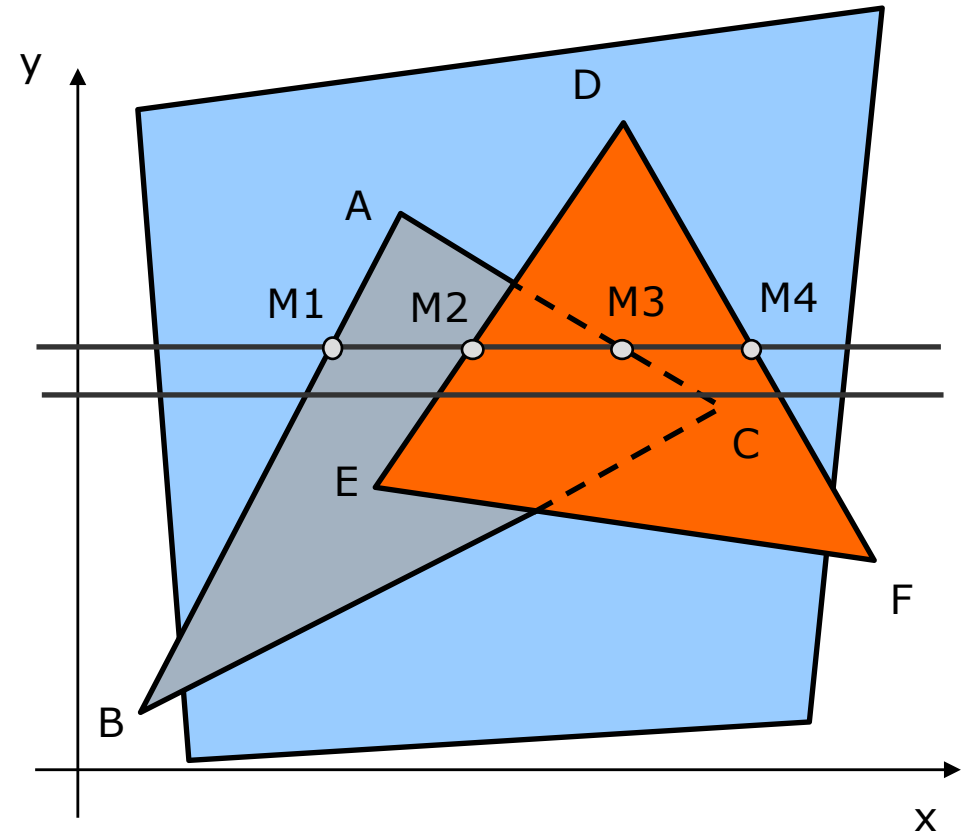
Scan line parsing

- Pixel to pixel, left to right
- Update in-out flags
- Compute the depth order at the intersections
 - Visible polygon
 - Graphics attributes (e.g. Pixel color, texture, filling pattern, etc)



Scan line algorithm optimization

- Scan line coherence:
 - If polygons do not intersect each other (in object space)
 - Then, on M3 the depth calculation can be avoided
 - Order in M2 remains the same until in M4
- Depth coherence
 - If polygons do not penetrate each other
 - If the same edges are in the AET on one scan line as are on the preceding scan line and in the same order
 - Then, no changes in depth relationships on any part of the scan line
 - No new depth computations are needed



Scan line algorithm extension

- ❑ Extend the Scan Line Algorithm to general surfaces (curved surfaces)
- ❑ ET replaced by ST (Surface Table)
- ❑ AET replaced by AST (Active Surface Table) sorted by the (x,y) extents of the surfaces
- ❑ When a surface is moved from ST to AST it is decomposed into a set of polygons
- ❑ When a surface leaves the AST all associated polygons are discarded

add surfaces to ST;

initialize AST;

for each scan line {

update AST;

for each pixel on scan line {

determine surfaces in AST that project to pixel;

find closest such surface;

determine the graphics attribute of the closest surface;

}

}

Scan line algorithm extension

□ Scan line z-buffer algorithm

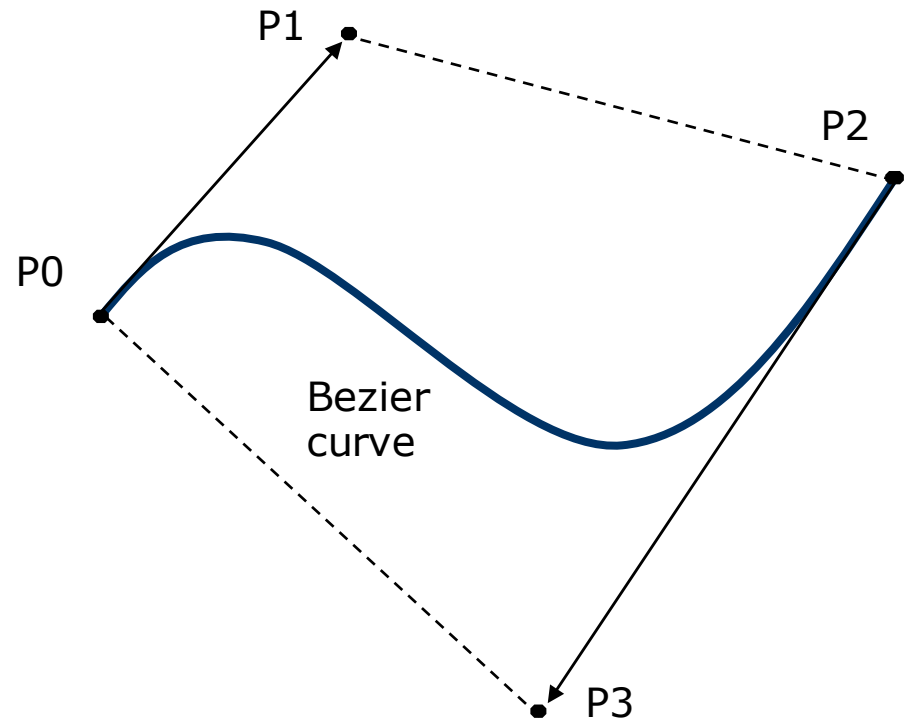
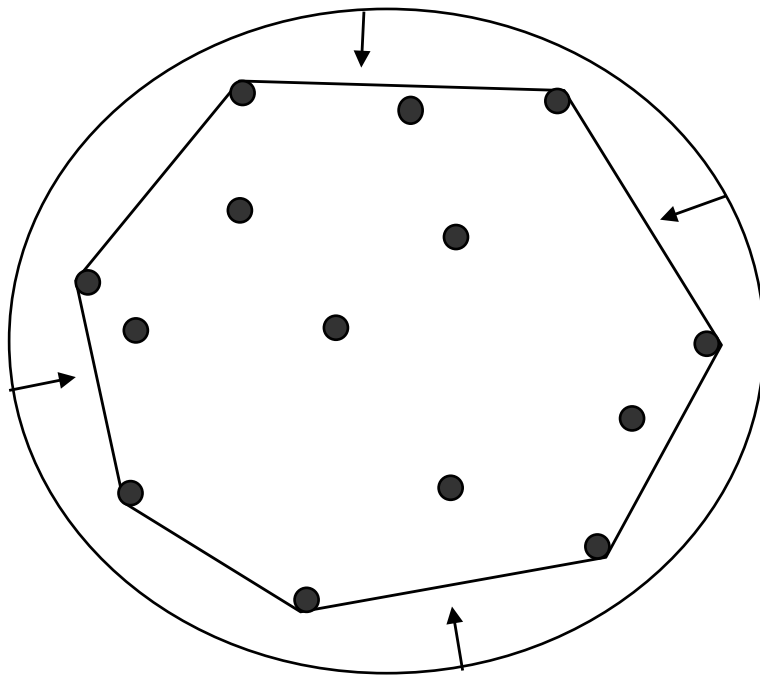
- Meyer, A.J. (1975), published by "An Efficient Visible Surface Program", Report to the National Science Foundation, Ohio State University.
- uses a z-buffer to resolve the visible surface problem
- A single scan line frame buffer and z-buffer is used for each scan line

□ Scan line algorithms for curved surfaces

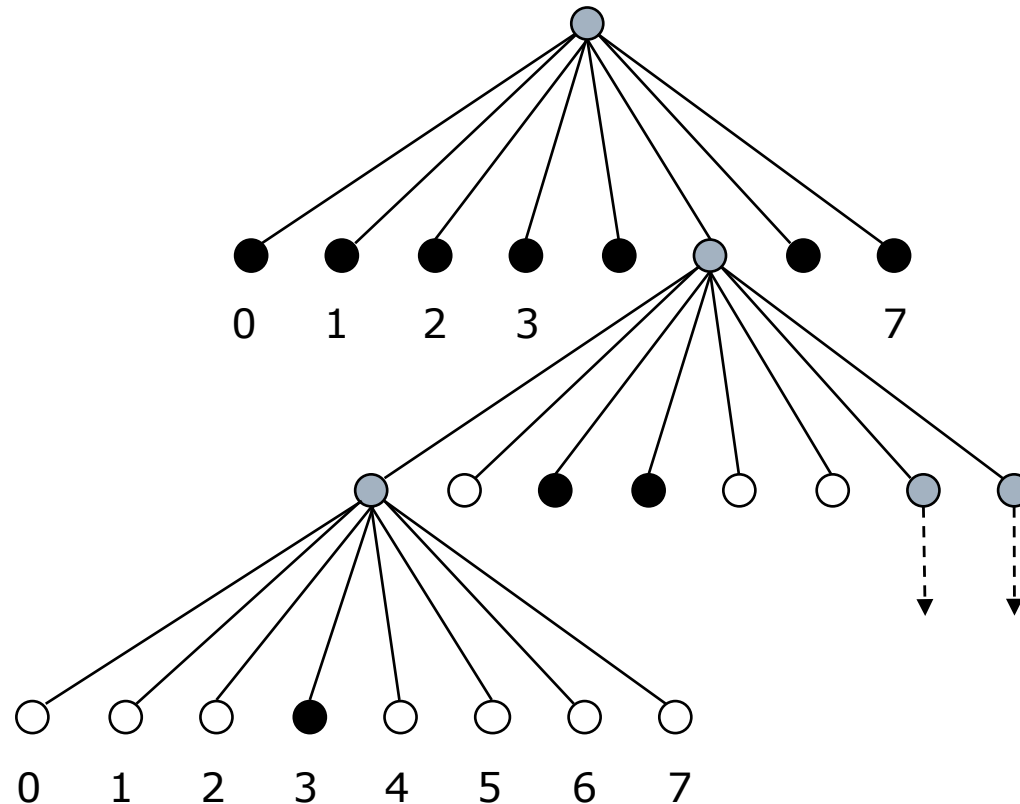
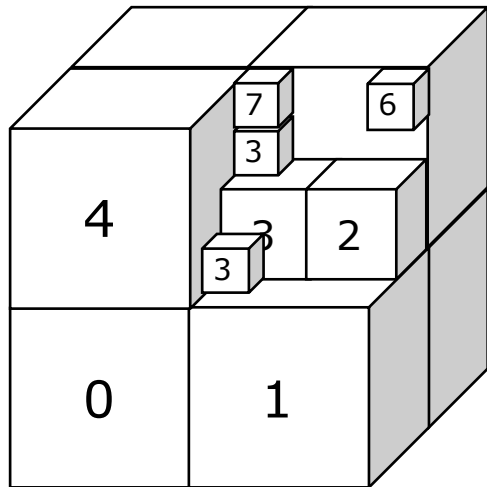
- Lane-Carpenter Algorithm
- Published in 1980, by Lane, J., Carpenter, L., Whitted, T, and Blinn, J. "Scan Line Methods for Displaying Parametrically Defined Surfaces", Communications of the ACM.
- Uses Patch Table (PT) and Active Patch Table (APT)
- Does the subdivision only as required when the scan line begins to intersect a patch, rather than in a preprocessing step
- The patch position is analyzed by the patch's control points that define its convex hull

Convex hull property

- Convex hull or convex envelope for a set of points X in a real vector space V is the minimal convex set containing X .
- A Bézier curve is circumscribed by the quadrilateral defined by its four control points.
- The Bézier curve as well as the Bézier surface could be approximated by the adjacent convex hull polygons, and rendered as a polygonal surface.

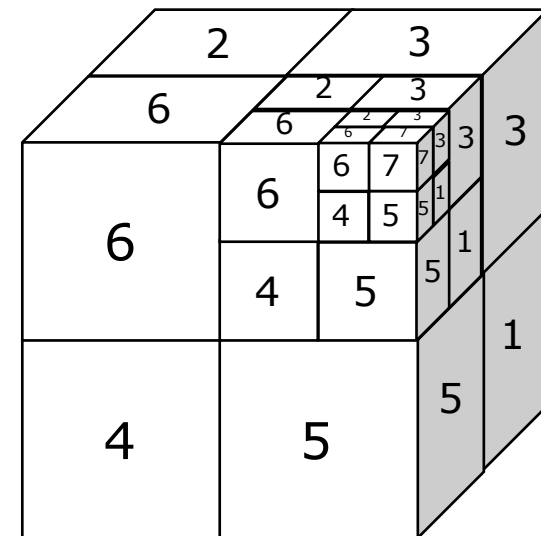
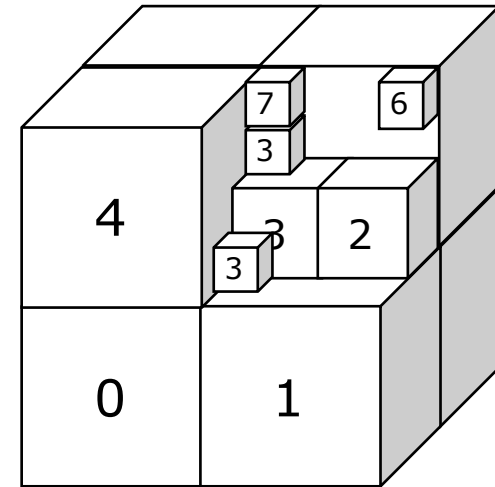


7. Octree Based Algorithms



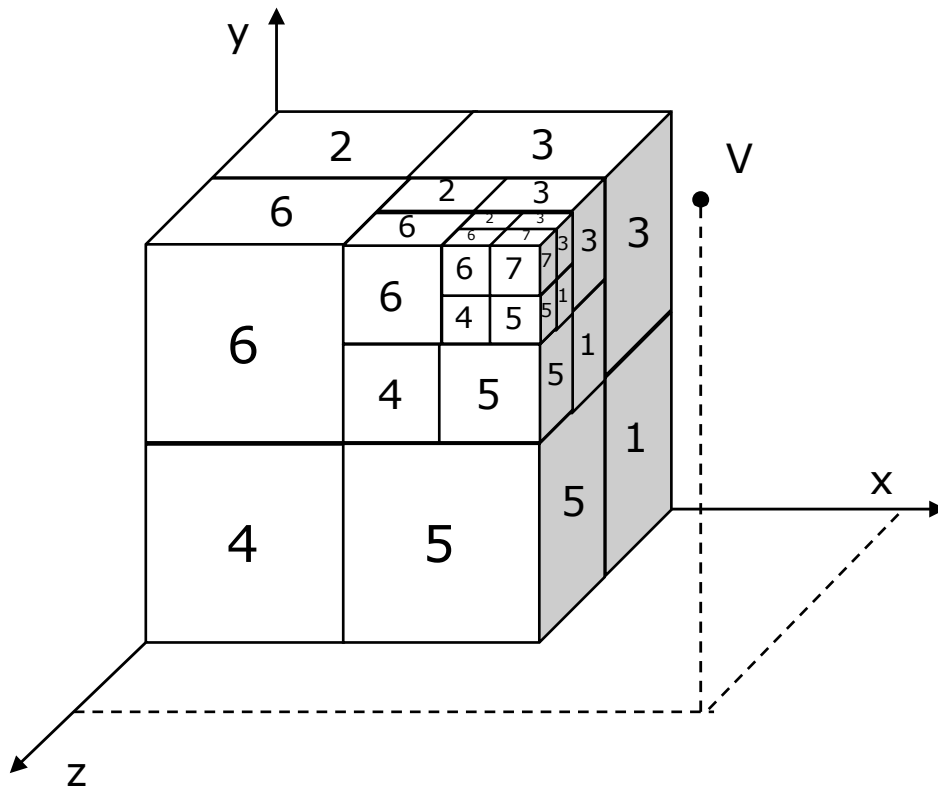
Octree encoded objects

- ❑ Octree regular structure gives nonintersecting cubes
- ❑ Octree is partially pre-sorted
- ❑ Octree enumeration could improve the operations (i.e. visibility computation):
 - Slice based enumeration
 - ❑ Iterate through the octants in slice perpendicular to one of the axes
 - ❑ Orthographic projection
 - ❑ E.g. variate z, then y, and then x
 - Back to front enumeration
 - ❑ The nodes are listed in order in which any node is guaranteed not to obscure any node listed after it
 - ❑ Orthographic projection
 - ❑ Display order: (1) the farthest octant, (2) three neighbors of the closest octant in any order, (3) closest octant



Octree enumeration for visibility

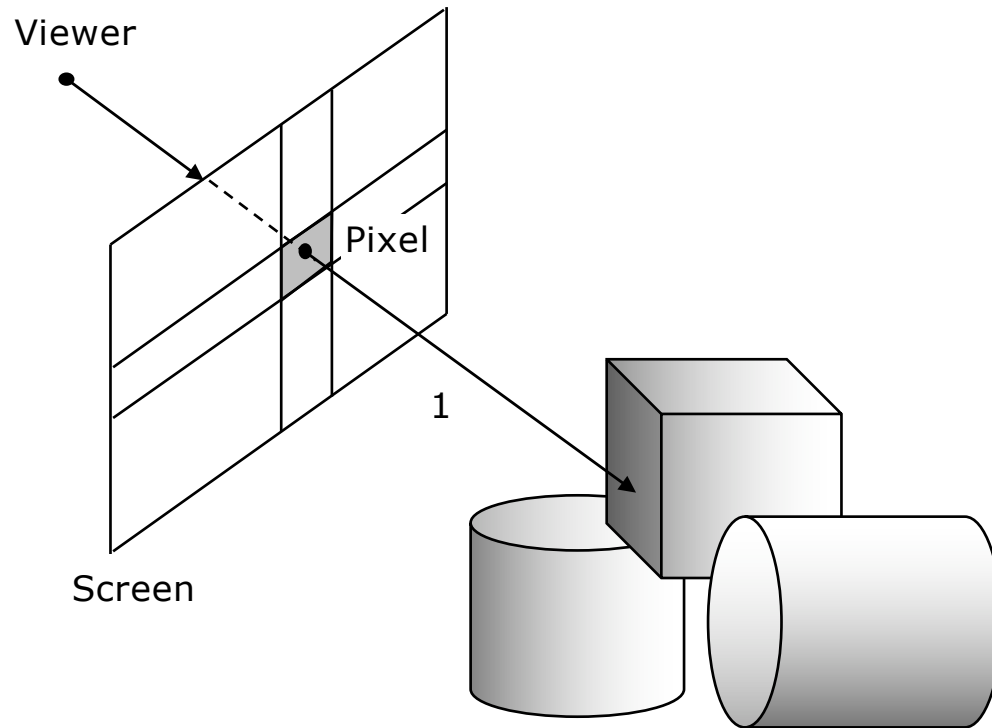
- Eight back to front orders given by the viewer's relative position
- Positive x coordinate means the right (R) face is visible, and negative x means the left (L) face is visible
- Similarly: y+/- gives up(U)/down(D) face, z+/- gives front(F)/back(B) face



Back to front enumeration and visible faces

z	y	x	Back to front order	Visible faces
-	-	-	7,6,5,3,4,2,1,0	B,D,L
-	-	+	6,7,4,2,5,3,0,1	B,D,R
-	+	-	5,4,7,1,6,0,3,2	B,U,L
-	+	+	4,5,6,0,7,1,2,3	B,U,R
+	-	-	3,2,1,7,0,6,5,4	F,D,L
+	-	+	2,3,0,6,1,7,4,5	F,D,R
+	+	-	1,0,3,5,2,4,7,6	F,U,L
+	+	+	0,1,2,4,3,5,6,7	F,U,R

8. Ray Casting Algorithms

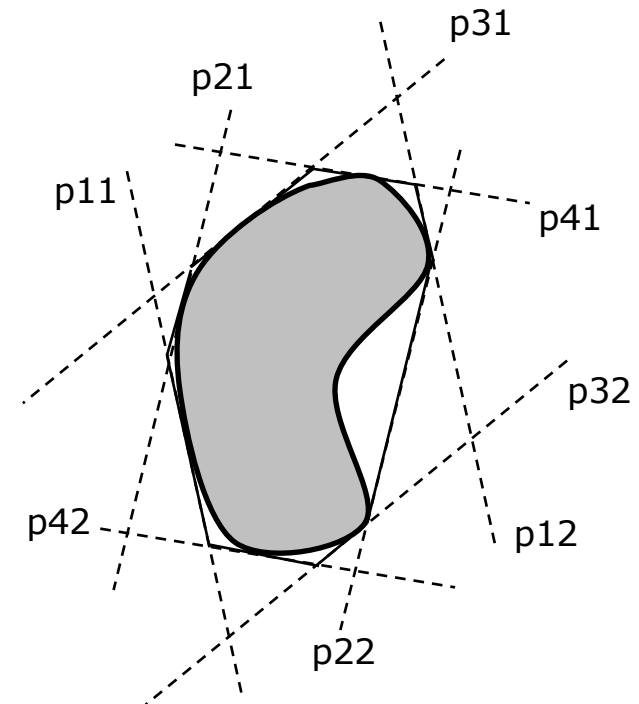
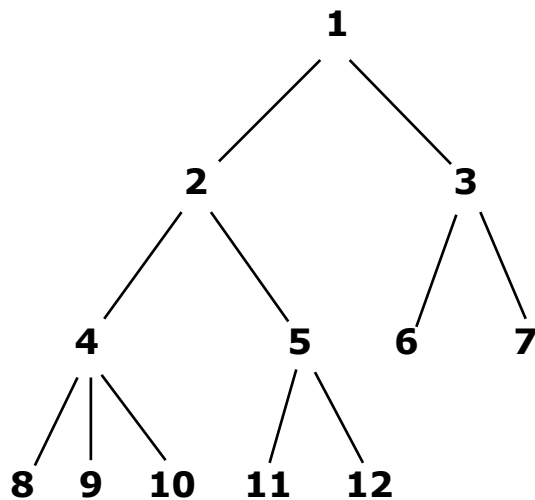
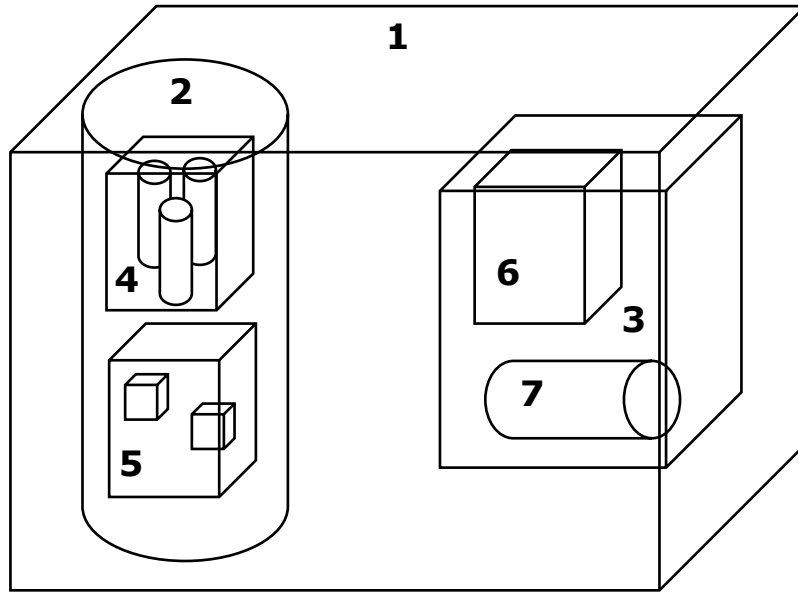


- Ray tracing, ray casting
- Developed by Appel (1968), Goldstein and Nagel (1968), Whitted (1980), and Kay (1979)
 - Appel: uses a sparse grid of rays to compute the shadows
 - Goldstein and Nagel: simulate the trajectories of ballistic projectiles and nuclear particles, and later apply it in graphics
 - Whitted and Kay: extended ray tracing to handle specular reflection and refraction

Ray casting issues

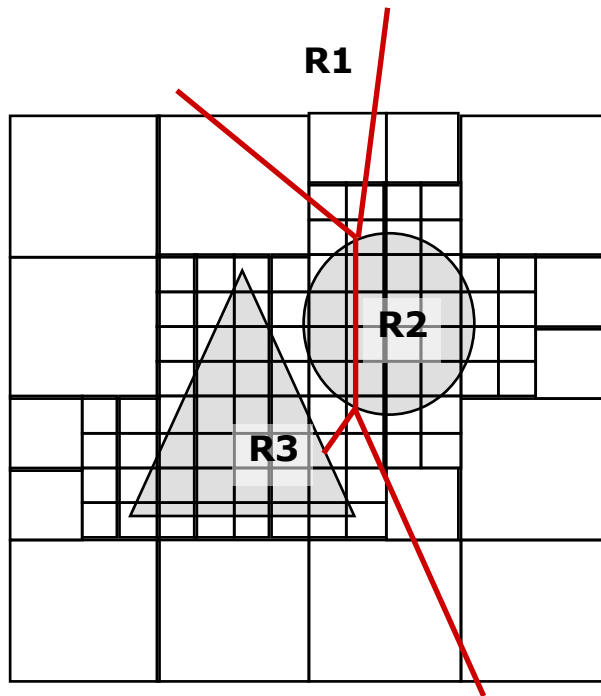
- Computing intersections
 - Ray (parametric line) and a polygon
 - Line and cylinder, cube, sphere, etc.
 - Line and quadrics (e.g. ellipsoid, paraboloid, etc.)
- Efficiency for visible surface ray tracing
 - Optimize the intersection computation
 - Avoid intersection computation
 - Hierarchical bounding volumes
 - Efficient hierarchy traversal
 - Automatically hierarchy generation
 - Spatial partitioning
- Computing Boolean set operations
 - Compute the visibility for 3D union, difference, and intersection of two solids (e.g. on CSG hierarchy)
- Antialiasing ray tracing
 - Point sampling on a regular grid produces aliasing images
 - Adaptive supersampling (additional samples by more rays), e.g. in corners of each pixel.

Ray casting issues

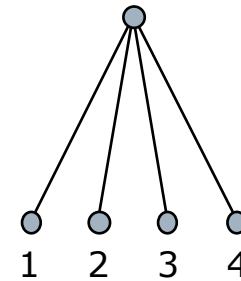


- Hierarchical bounding boxes
- Efficient extents

Ray casting issues

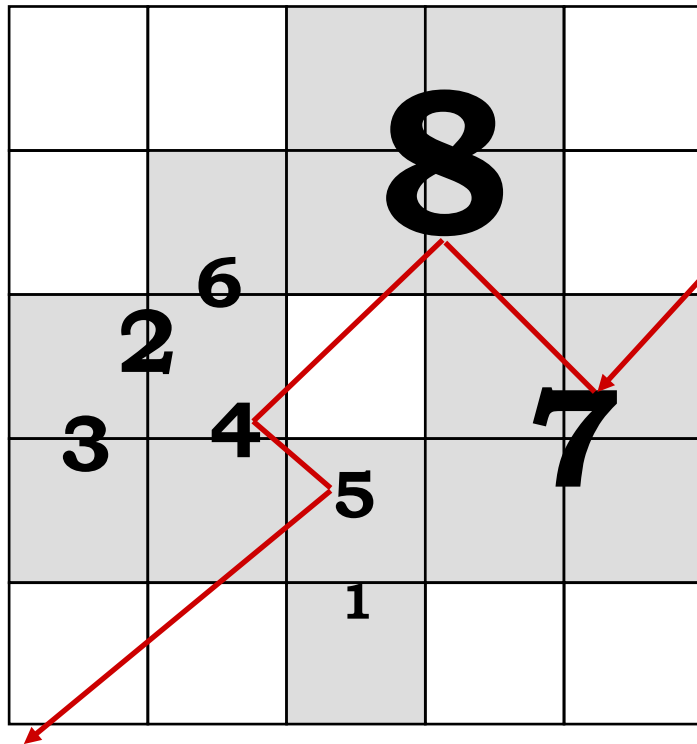


1	2
3	4



- ❑ Quadtree, octree
- ❑ Automatically hierarchy generation
- ❑ Spatial partitioning
- ❑ Efficient hierarchy traversal
- ❑ Optimize the intersection computation

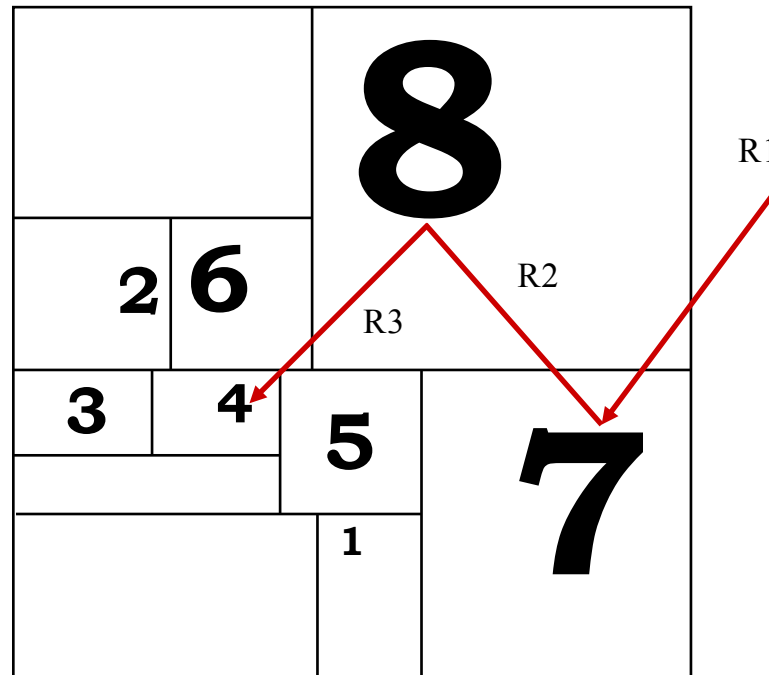
Ray casting issues



		8	8	
	6	8	8	
23	246		7	7
3	4	15	7	7
		1	8	

- ❑ Automatically hierarchy generation
- ❑ Spatial partitioning
- ❑ Efficient hierarchy traversal
- ❑ Optimize the intersection computation

Ray casting issues

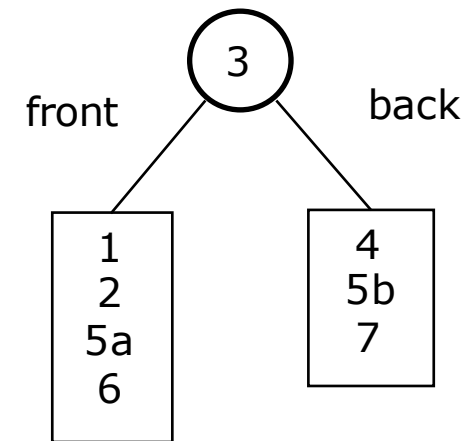
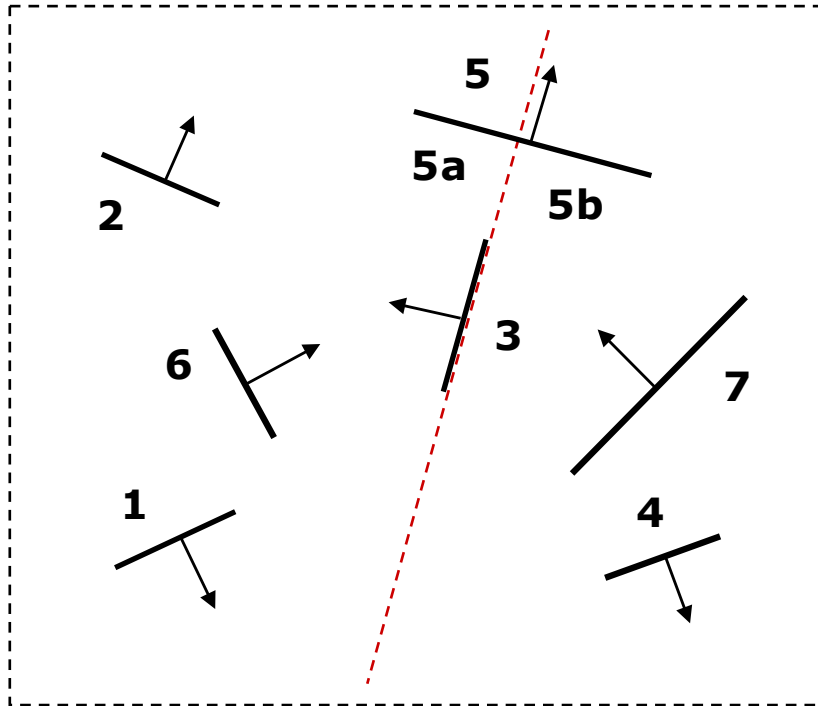


- ❑ Automatically hierarchy generation
- ❑ Spatial partitioning
- ❑ Efficient hierarchy traversal
- ❑ Optimize the intersection computation

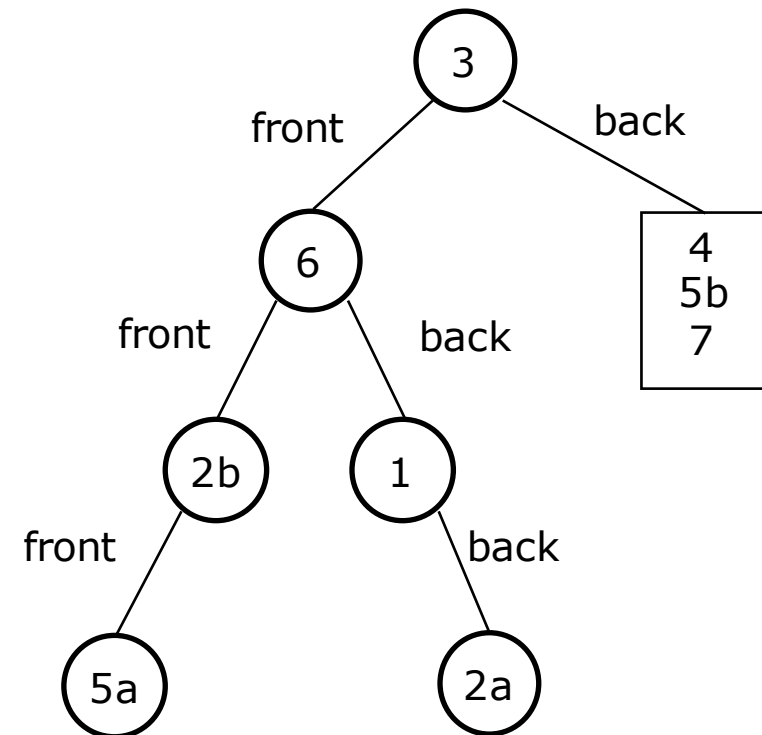
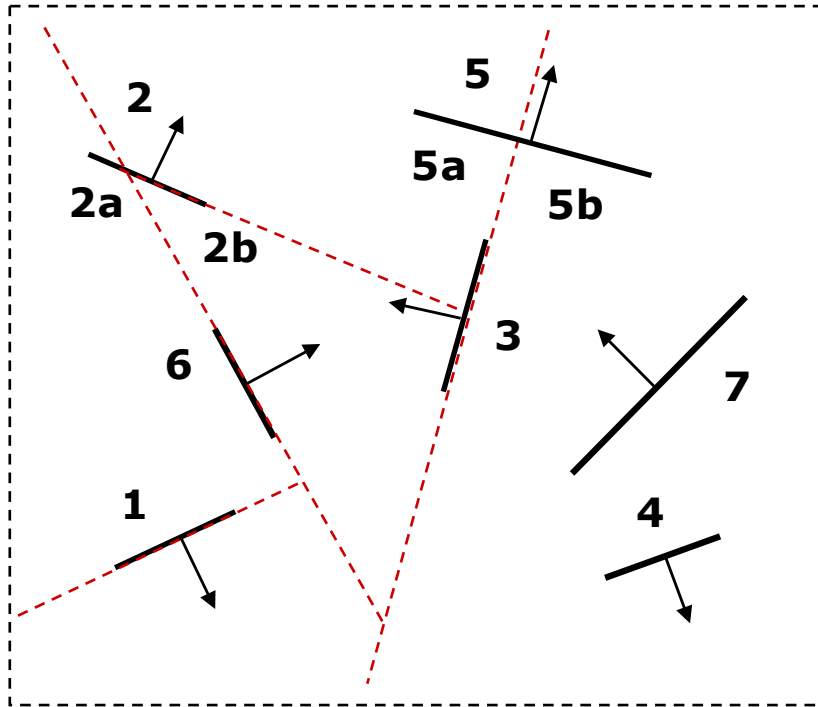
9. BSP Trees

- ❑ Binary Space Partitioning Trees (BSP Trees)
- ❑ Developed by Fuchs, Kedem and Naylor in 1980
 - Published in 1980, by Fuchs, H., Kedem, Z.M., Naylor, B.F., "ON Visible Surface Generation by A Priori Tree Structures", SIGGRAPH 80.
- ❑ The basic idea has been published in 1969 by Schumacker, who considered the space as a set of clusters separated by planes (faces):
 - Clusters that are on the same side of the plane as the eyepoint can obscure, but cannot be obscured by clusters on the other side
- ❑ Suited for applications in which the viewpoint changes, but the objects do not
- ❑ Steps:
 1. Build the BSP tree
 1. Compute polygon planes
 2. Determine the normal vector for each polygonal face
 3. Chose the root node
 4. Add nodes by analyzing the relative position of other faces relative to the current parent node
 2. Visualize the polygons in order infered from the BSP tree and the viewer's position

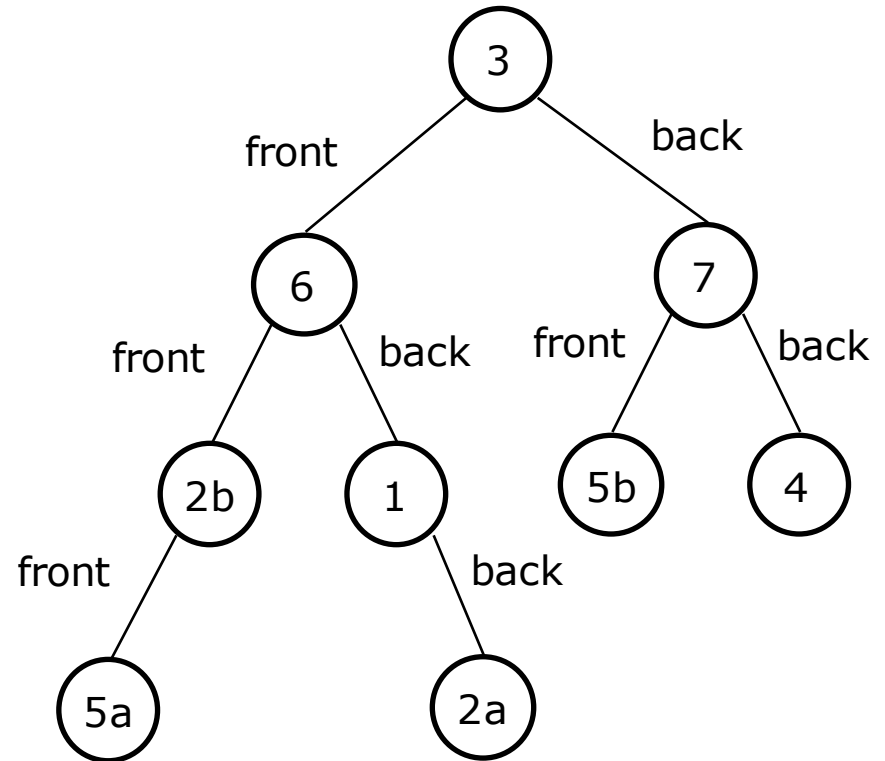
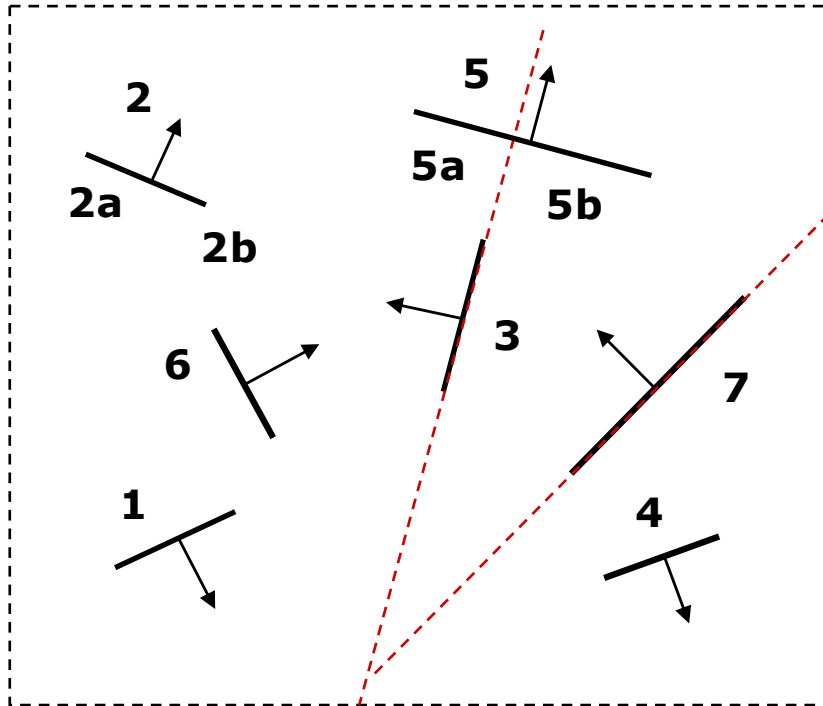
BSP Trees - Example



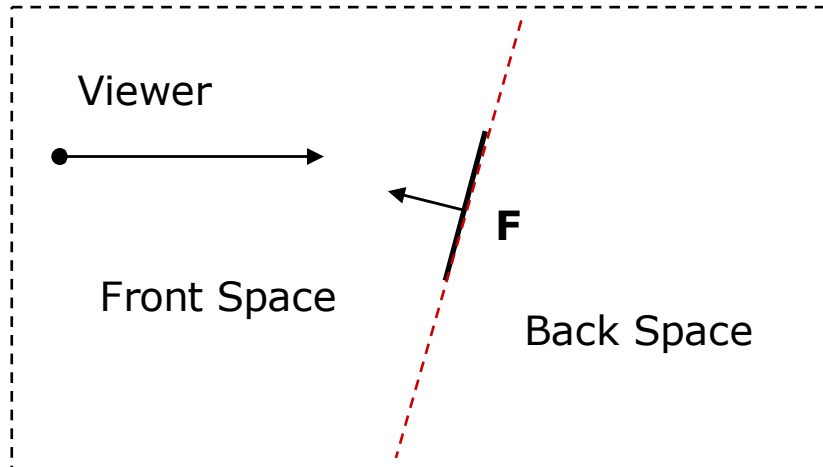
BSP Trees - Example



BSP Trees - Example

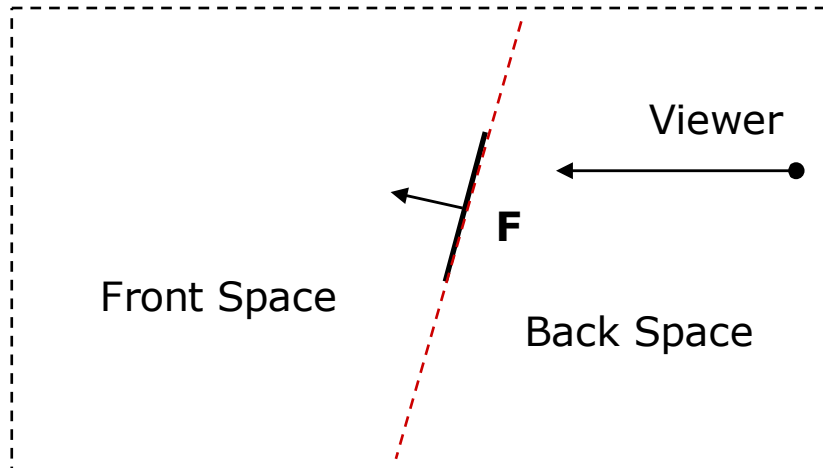


BSP Trees - Display



Display order:

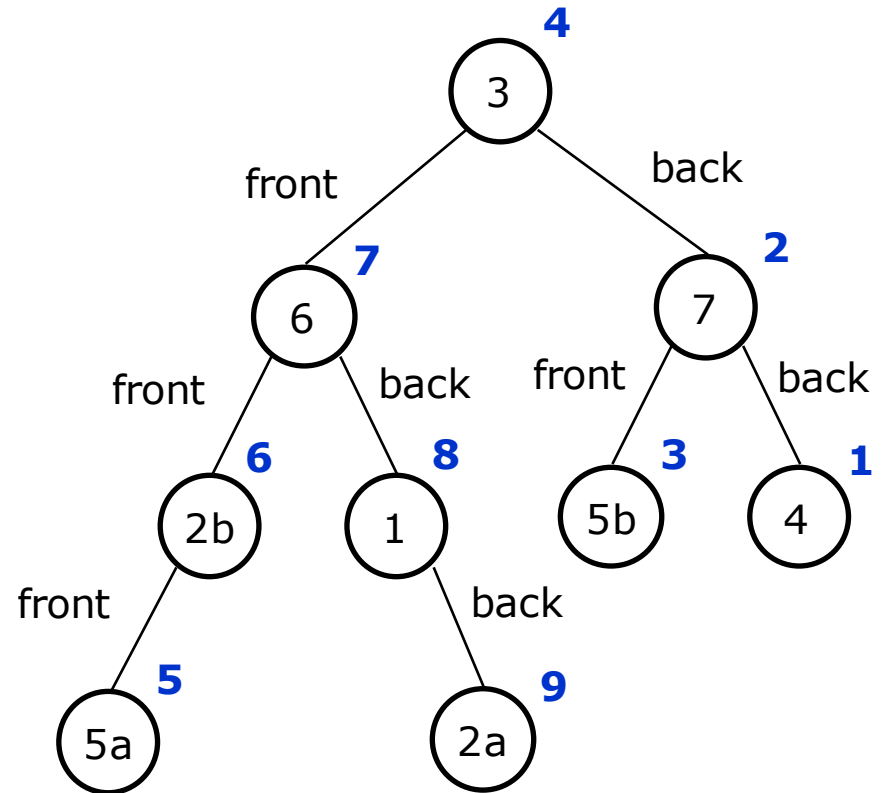
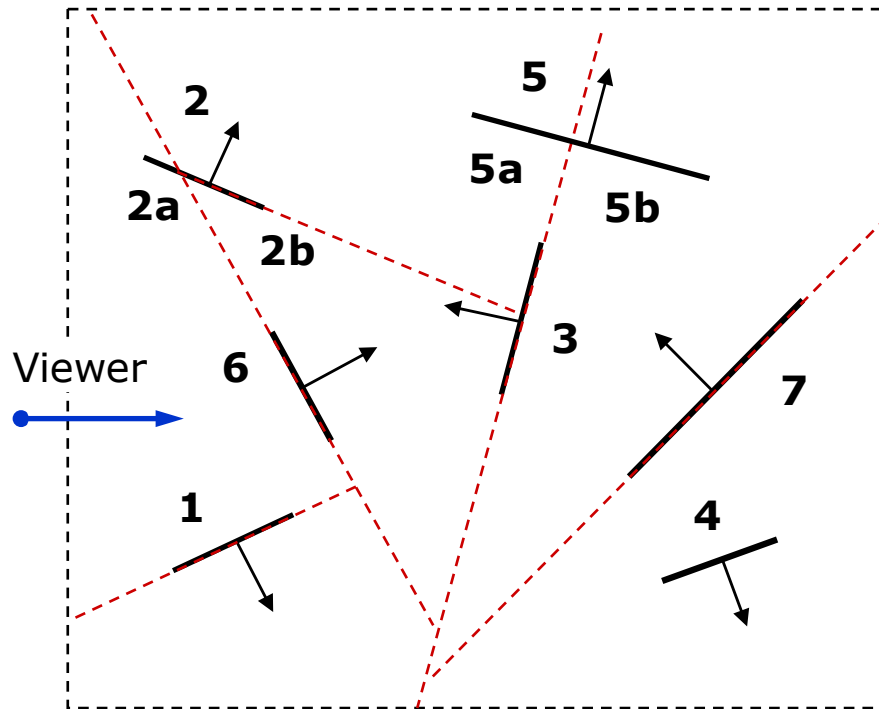
1. Back space polygons
2. Root face
3. Front space polygons



Display order:

1. Front space polygons
2. Root face
3. Back space polygons

BSP Trees - Display



Display order: 4, 7, 5b, 3, 5a, 2b, 6, 1, 2a

Questions and problems

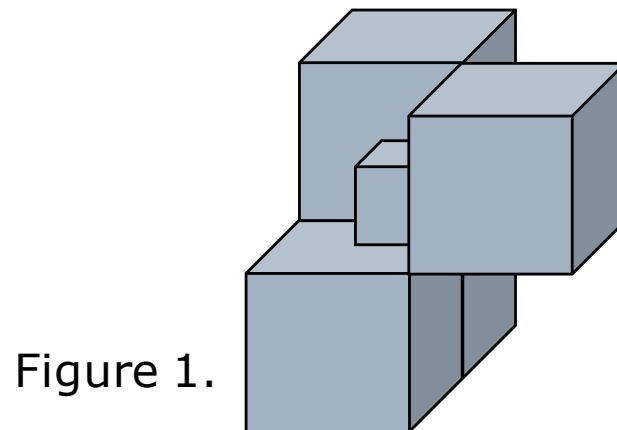
1. Let us consider the user's position P on axis z at infinity. Determine if the relative positions of two 3D polygons A and B are in one of the following cases:
 - a. Point P and polygon A are in the same half space of polygon B
 - b. Point P and polygon A are in opposite half space of polygon B
 - c. Polygon A intersects the plane of polygon B
 - d. Polygons A and B are within the same plane
2. Let us consider a cube C in the domain of positive z having one main diagonal parallel with axis z . The projection plane is xy , and the user is located on axis z at infinity.
 - a. Exemplify and explain the Depth Sorting algorithm for the cube C
 - b. Exemplify and explain the Scan Line algorithm for the cube C

Questions and problems

3. Why, in the Scan Line algorithm, the edges are sorted into buckets based on each edge's smaller y coordinate? In what case the bucket could be defined for the bigger y coordinate?
4. Why, in the Scan Line algorithm, within each bucket the edges are ordered by increasing x coordinate of their lower endpoint?
5. Explain how the concept of coherence is used in the Scan Line algorithm. What is the property that is propagated from one step to the next one?
6. Exemplify and explain how works the Scan Line algorithm in the case of more than three polygons.
7. What are the main issues of the Ray Casting algorithm working on the scene of: a) static objects; b) moving objects; c) deformable objects.

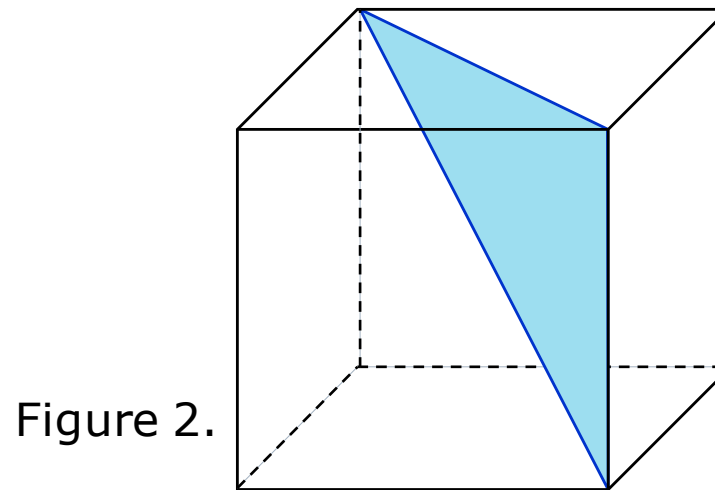
Questions and problems

8. Generate the octree for the complex object in the figure 1, composed of cubes.
9. Exemplify and explain the method "Octree enumeration for visibility" for the complex object in the figure 1, composed of cubes.



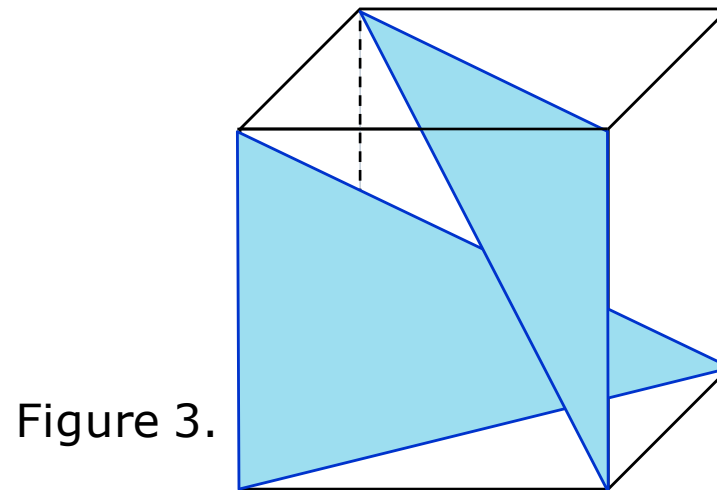
Questions and problems

10. Generate the octree for the object in the figure 2, composed of one triangle.
11. Exemplify and explain the method "Ray Casting" for the visualization of the object in the figure 2 (composed of just one triangle).
12. Exemplify and explain the method "Ray Casting based on the octree model" for the visualization of the object in the figure 2 (composed of just one triangle).



Questions and problems

13. Generate the octree for the object in the figure 3, composed of two triangles.
14. Exemplify and explain the method "Ray Casting based on the octree model" for the visualization of the object in the figure 3 (composed of two triangles).



Questions and problems

15. Exemplify and explain the method “Ray Casting based on the octree model” for the visualization of the object in the figure 4 (composed of six triangles).

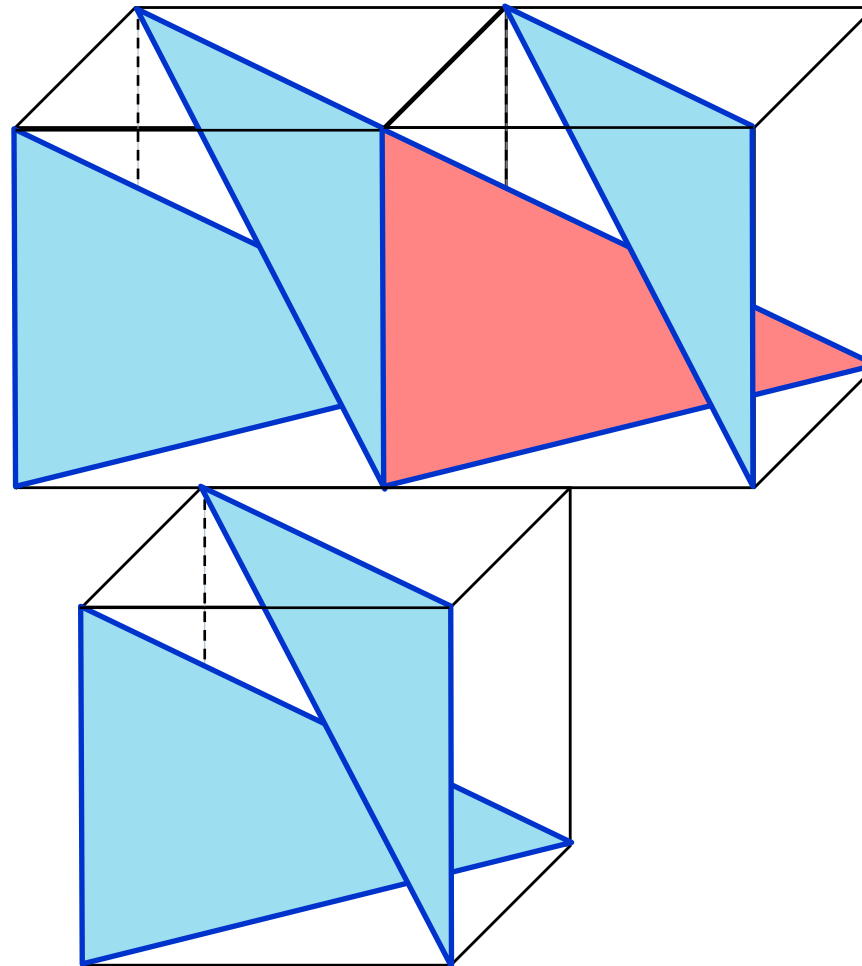


Figure 4.

Questions and problems

16. Exemplify the BSP Tree based visualization for the 3D object if the image in the orthogonal projection looks like in the figure.

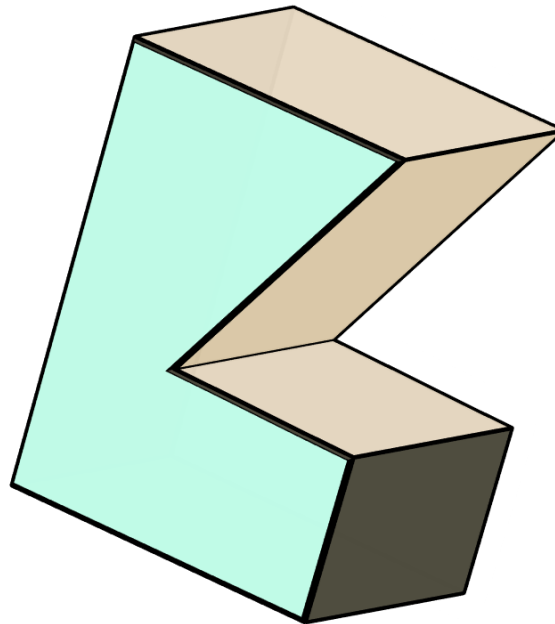


Figure 5.

Questions and problems

17. Exemplify the BSP Tree based visualization for the 3D object (composed of eight polygons), if the image in the orthogonal projection looks like in the figure.

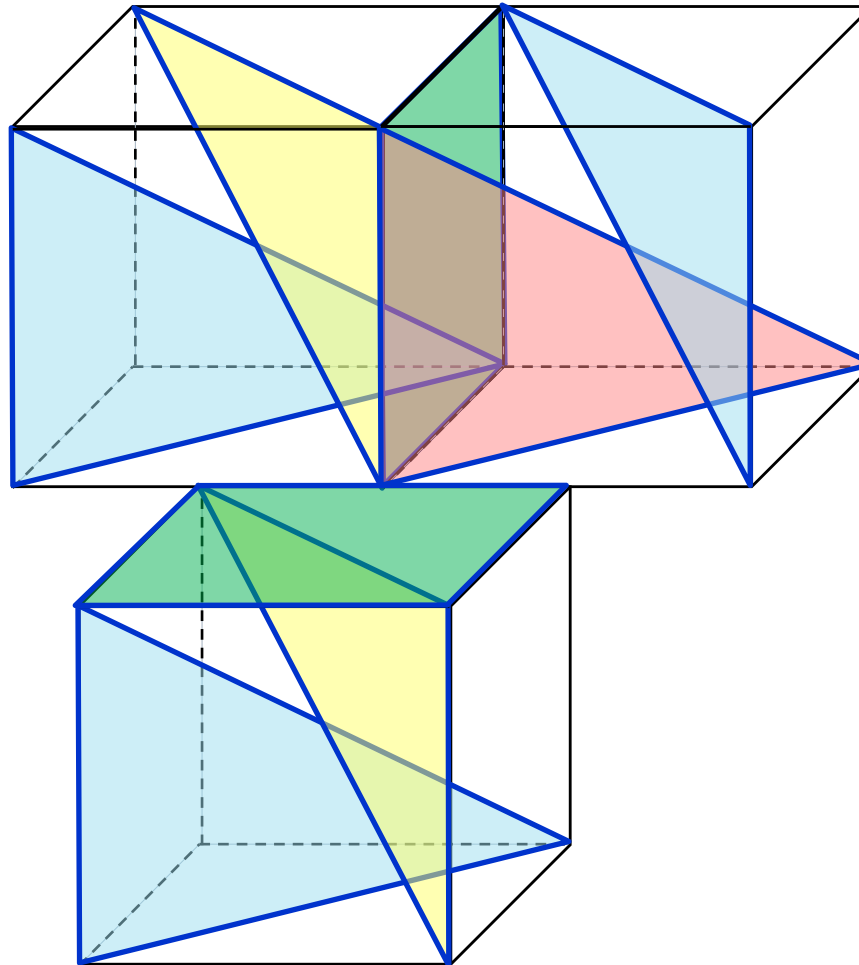


Figure 6.

Representation and Modeling of 3D Objects

Contents

- ❑ Implicit functions
- ❑ Polygonal representation
- ❑ Bicubic parametric patches
- ❑ Constructive solid geometry
- ❑ Space subdivision techniques
- ❑ Procedural modeling

Motivation

Object representation and modeling determine:

- ❑ Data structure
- ❑ Processing algorithm
- ❑ Design of software packages and hardware modules
- ❑ Resources required through 3D processing pipeline
- ❑ Final appearance of the object
- ❑ Approach for editing the shape of the object

Object representation

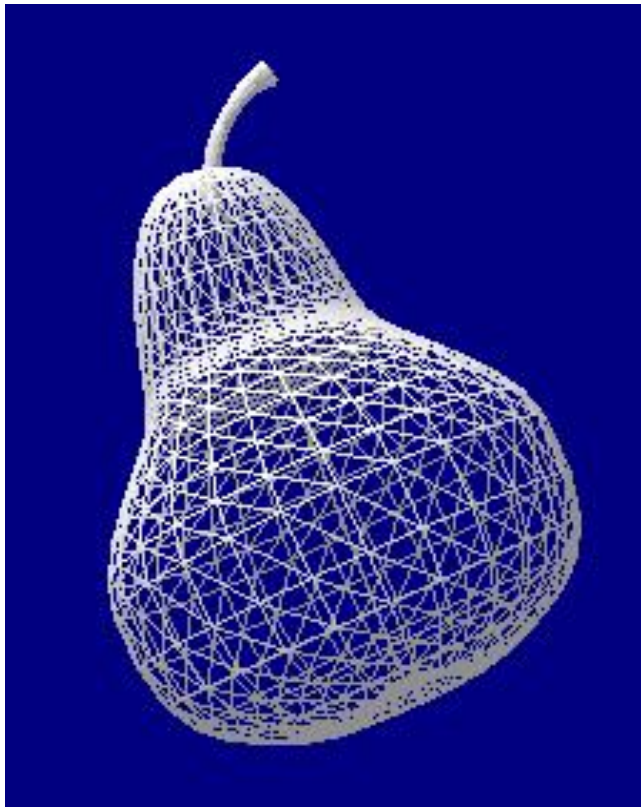
1. Implicit functions
 - object is an analytical definition, e.g. a sphere is defined as $x^2+y^2+z^2=r^2$
2. Polygonal
 - objects are approximated by a set of planar polygonal facets
3. Bicubic parametric patches
 - objects are represented as a set of patches. The sets of patches are polynomials in two parametric variables (e.g. Bézier, Hermite, B-spline, etc)
4. Constructive solid geometry (CSG)
 - object is a collection of elementary objects such as spheres, cylinders and boxes
5. Space subdivision techniques
 - object embedded in a space where the zones are labeled according to object occupancy
6. Procedural modeling

Implicit functions

- Some surfaces can be represented as points of functions (defined over 3D space)
 - Places where a function $f(x,y,z)=0$
- Some objects are easy represent this way
 - Spheres, ellipses, and similar
 - More generally, quadratic surfaces
$$F(x,y,z)=ax^2 + by^2 + cz^2 + 2dxy + 2eyz + 2fxz + 2gx + 2hy + 2jz + k = 0$$
 - Shapes depends on all the parameters $a,b,c,...,k$

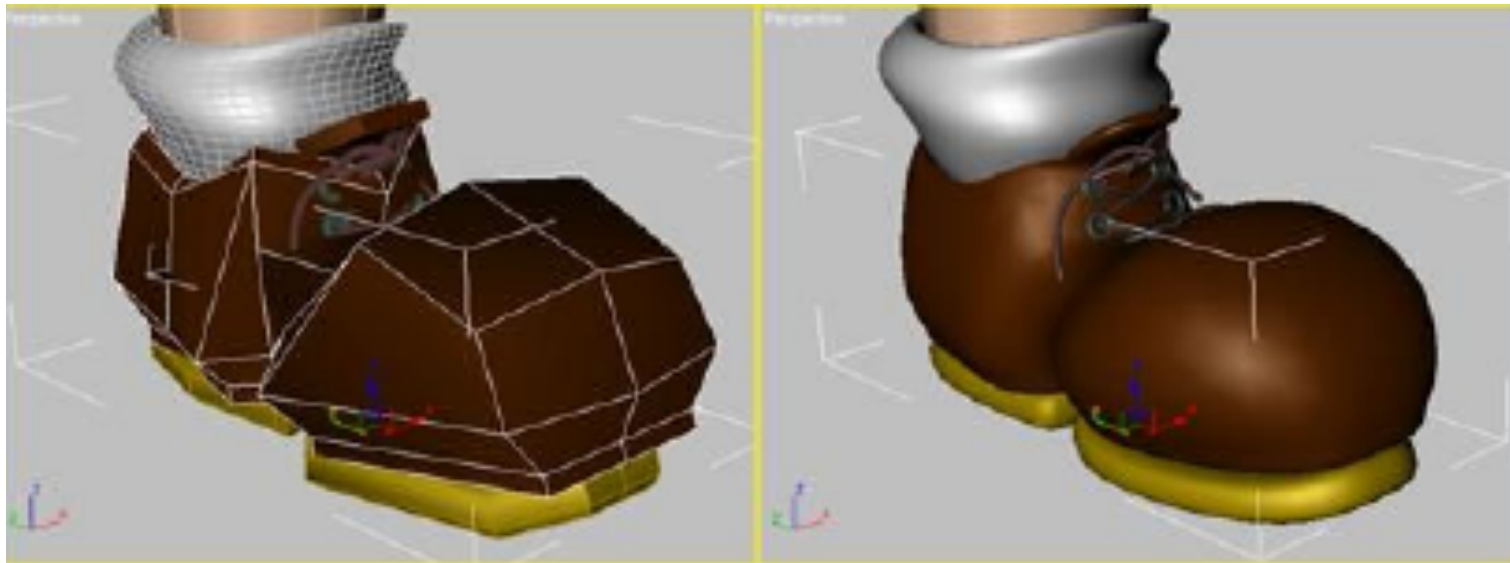
Polygonal representation

- **Polygonal mesh**, called **Boundary representation (Brep)**
Geometric and topological description of the boundary or surface of the object

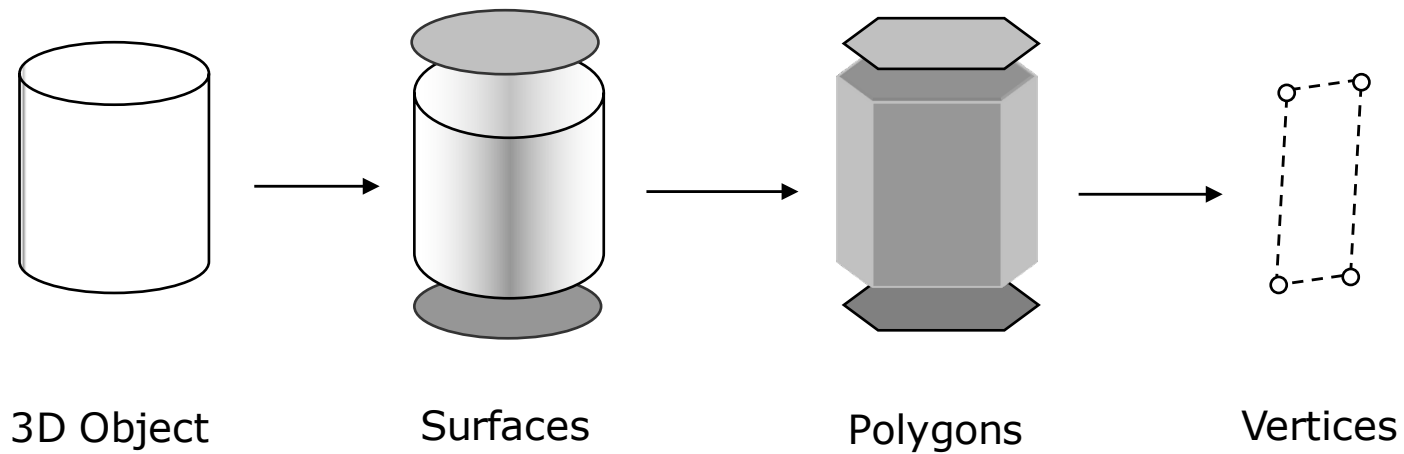


Polygon based modeling

Images taken from the 3D Studio Max tutorial of Modeling Clothing



Polygonal representation



Hierarchical polygonal decomposition of a 3D object

Polygonal representation

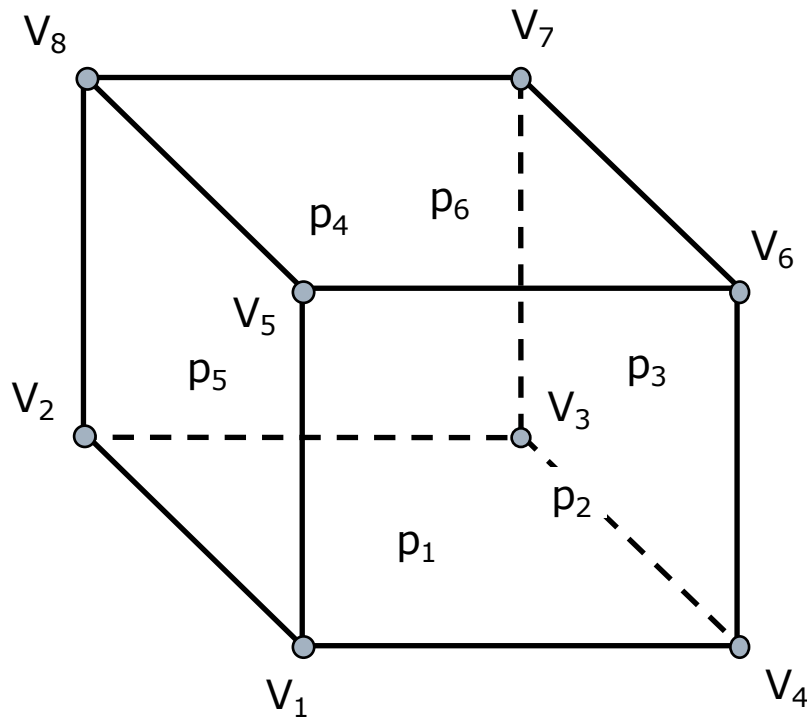
□ Advantages

- Ubiquitous in computer graphics
- Simple and straightforward
- Used as intermediate form for many other data structures

□ Issues

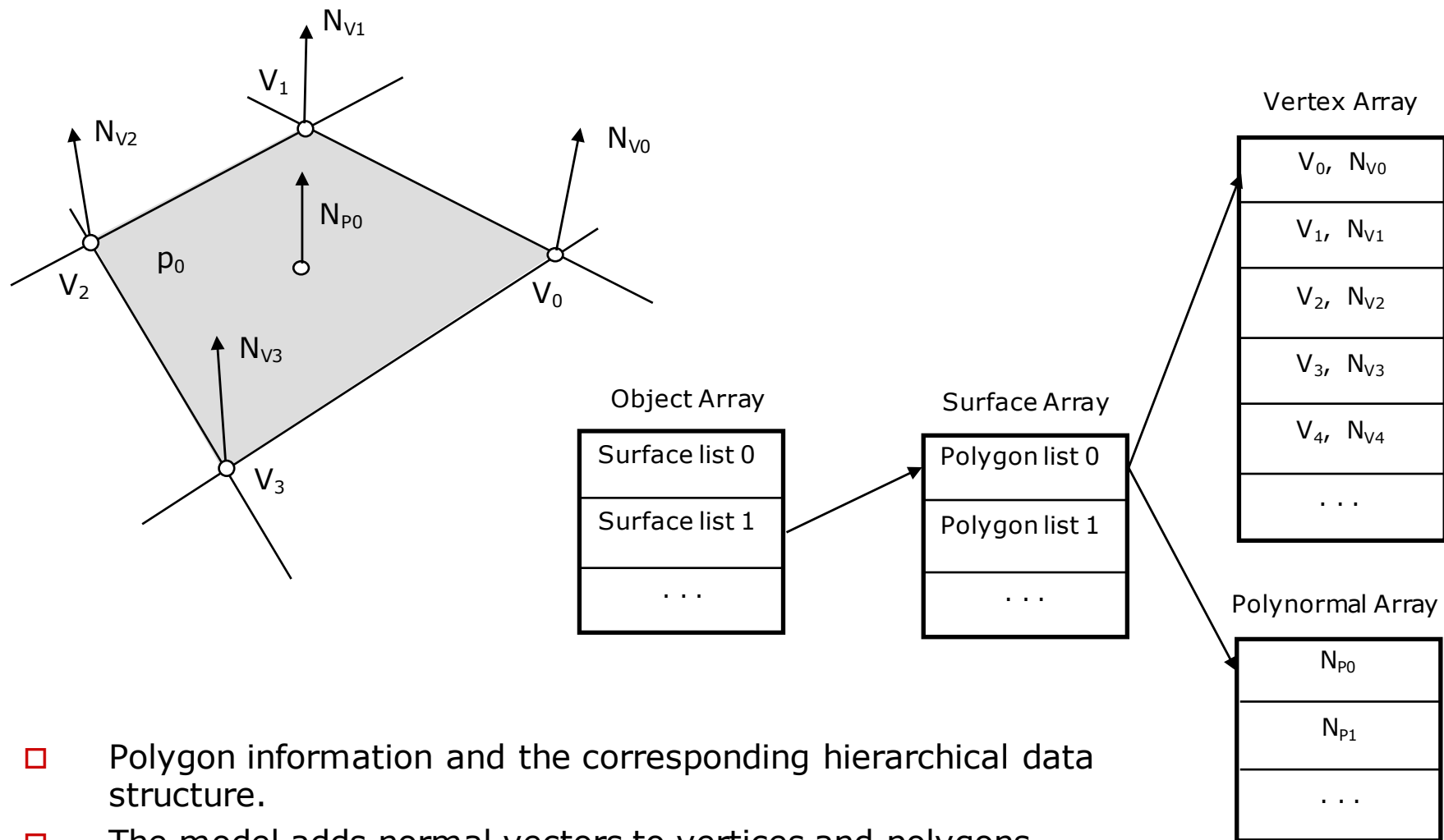
- Approximation of curved surfaces
- Accuracy depends extremely on the number of polygons
e.g. zoom, scale
- Texture mapping
- Smooth surfaces
- Shading algorithms

Brep – vertex based model



Polygon (face)	Vertices	Vertex	Coordinates
p1	V ₁ V ₂ V ₃ V ₄	V ₁	x ₁ y ₁ z ₁
p2	V ₆ V ₅ V ₁ V ₄	V ₂	x ₂ y ₂ z ₂
p3	V ₇ V ₆ V ₄ V ₃	V ₃	x ₃ y ₃ z ₃
p4	V ₈ V ₇ V ₃ V ₂	V ₄	x ₄ y ₄ z ₄
p5	V ₅ V ₈ V ₂ V ₁	V ₅	x ₅ y ₅ z ₅
p6	V ₈ V ₅ V ₆ V ₇	V ₆	x ₆ y ₆ z ₆
		V ₇	x ₇ y ₇ z ₇
		V ₈	x ₈ y ₈ z ₈

Brep - Polygon based model

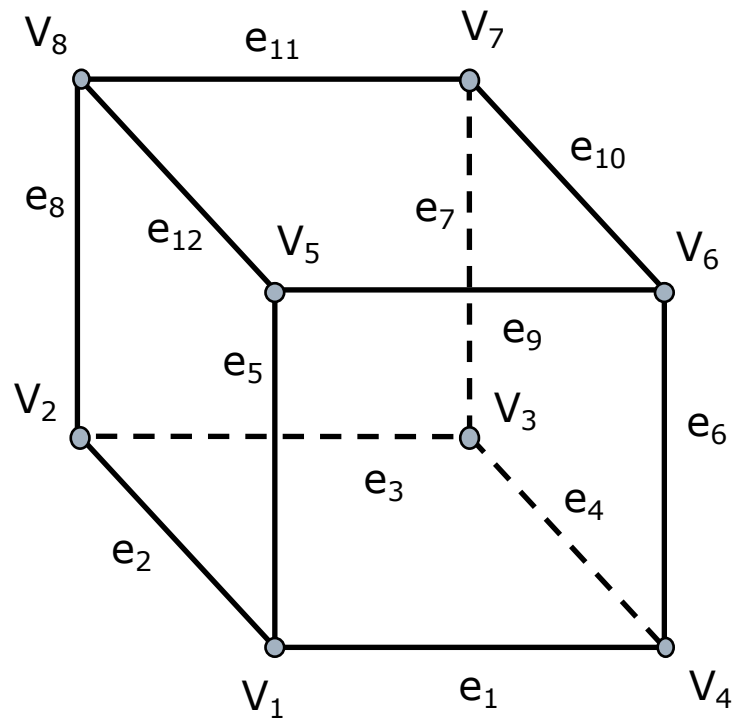


- ❑ Polygon information and the corresponding hierarchical data structure.
- ❑ The model adds normal vectors to vertices and polygons.

Polygonal model - Data structure

- The data structure consists of four arrays:
 1. *Object Array* - an array of all surfaces of the 3D object modelled through polygons. A position of the array is a list of indices of the polygonal components.
 2. *Surface Array* - it is the set of polygons that describes the surface. A position in the array describes a polygon as a list of indices of the polygon vertices and an index of the polygon normal.
 3. *Vertex Array* - describes the information that is associated to each vertex. Three coordinates and a normal vector describe a vertex. In fact the normal vector has three components: its projections onto the coordinate axes - N_x , N_y and N_z .
 4. *Polynormal Array* - a set of normal vectors of the polygons. Each vector has three vector components.
- Disadvantage:
 - Edges are shared between adjacent polygons. The edge is not explicitly contained in the data structure.
 - An edge is processed twice.

Brep – edge based model

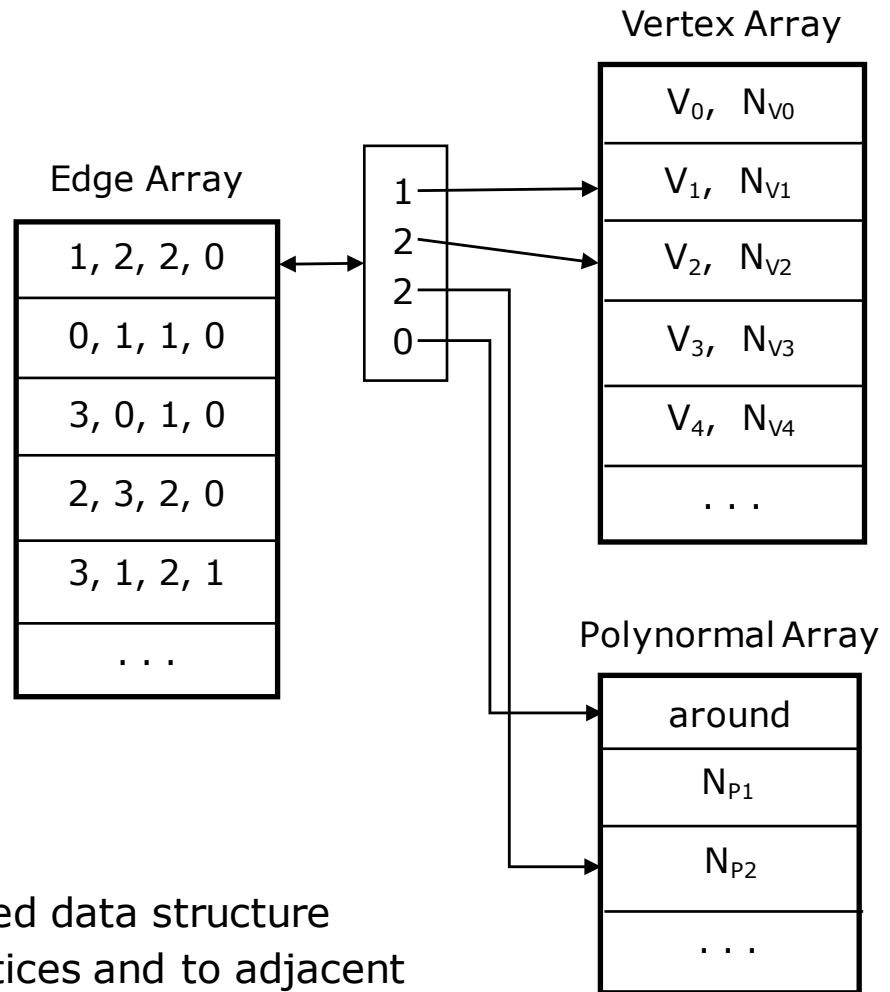
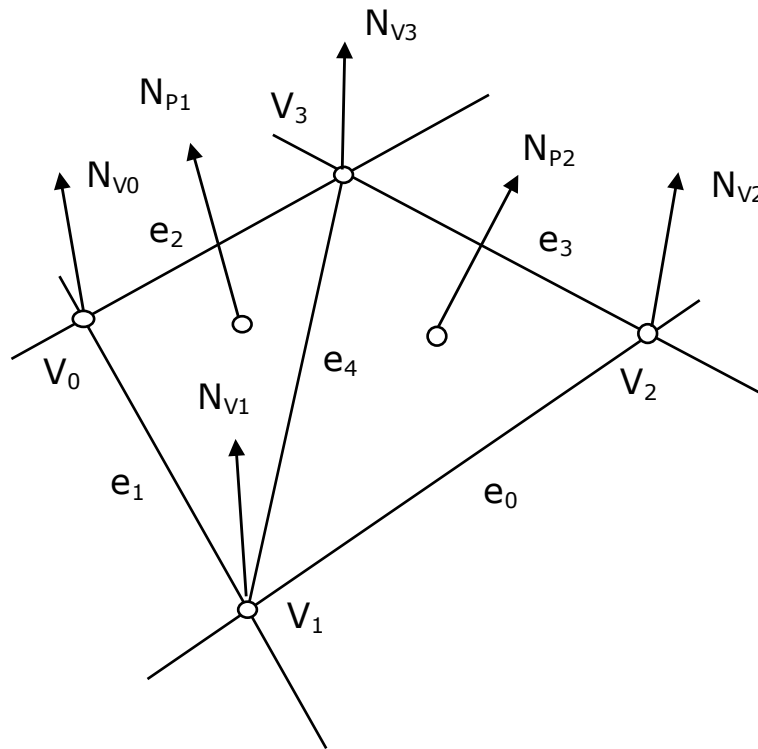


Polygon (face)	Edges
p ₁	e ₁ e ₂ e ₃ e ₄
p ₂	e ₁ e ₆ e ₉ e ₅
p ₃	e ₄ e ₇ e ₁₀ e ₆
p ₄	e ₅ e ₈ e ₁₁ e ₇
p ₅	e ₂ e ₅ e ₁₂ e ₈
p ₆	e ₉ e ₁₀ e ₁₁ e ₁₂

Edge	Vertices
e ₁	V ₄ V ₁
e ₂	V ₁ V ₂
e ₃	V ₂ V ₃
e ₄	V ₃ V ₄
e ₅	V ₅ V ₁
e ₆	V ₄ V ₆
e ₇	V ₃ V ₇
e ₈	V ₂ V ₈
e ₉	V ₅ V ₆
e ₁₀	V ₆ V ₇
e ₁₁	V ₇ V ₈
e ₁₂	V ₈ V ₅

Vertex	Coordinates
V ₁	x ₁ y ₁ z ₁
V ₂	x ₂ y ₂ z ₂
V ₃	x ₃ y ₃ z ₃
V ₄	x ₄ y ₄ z ₄
V ₅	x ₅ y ₅ z ₅
V ₆	x ₆ y ₆ z ₆
V ₇	x ₇ y ₇ z ₇
V ₈	x ₈ y ₈ z ₈

Improved edge based model



- ❑ Polygon information and the edge based data structure
- ❑ The model adds normal vectors to vertices and to adjacent polygons

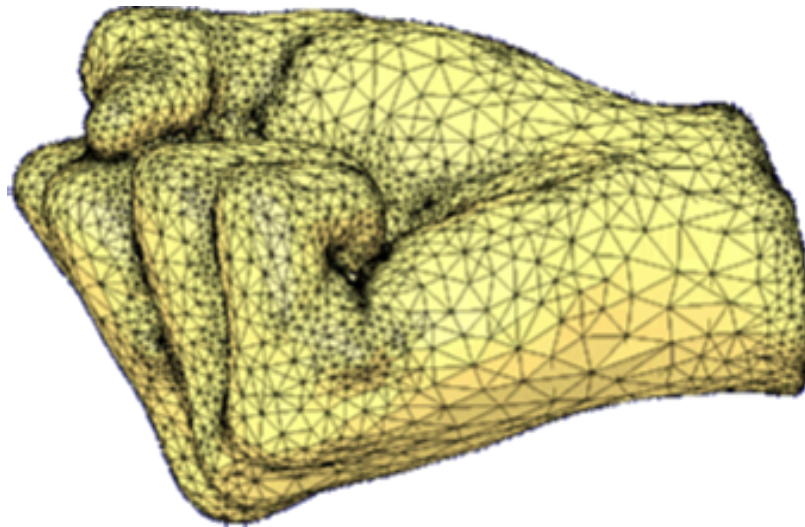
Creating the Surface Points

- Approaches:
 - Manual placement
 - Mathematical (geometrical) generation
 - Implicit functions
 - Swapping, extrusion
 - Scanning real objects
 - Laser scanning
 - Tomographic methods
 - Computer vision



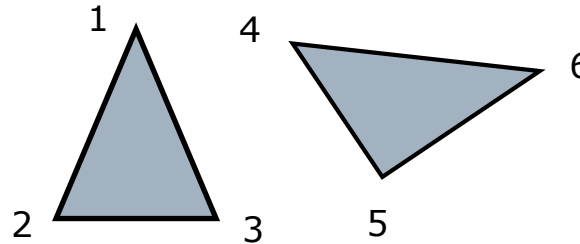
Triangulation

- Triangles are the first choice of polygons
 - Triangles are always planar
 - Triangles are always convex polygons -> simple and fast algorithms
 - Graphics hardware is often optimized for triangles
 - Ex. Nvidia processes nx10M triangles per second, and nx100M vertices per second



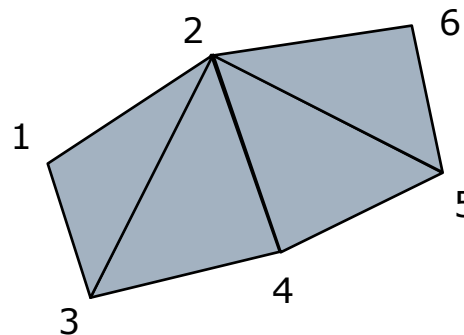
Triangle representation

□ Lists



3 vertices per
triangle

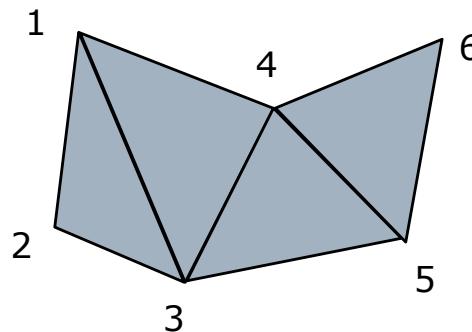
□ Fans



$6/4=1.5$ vertices
per triangle

□ Strips

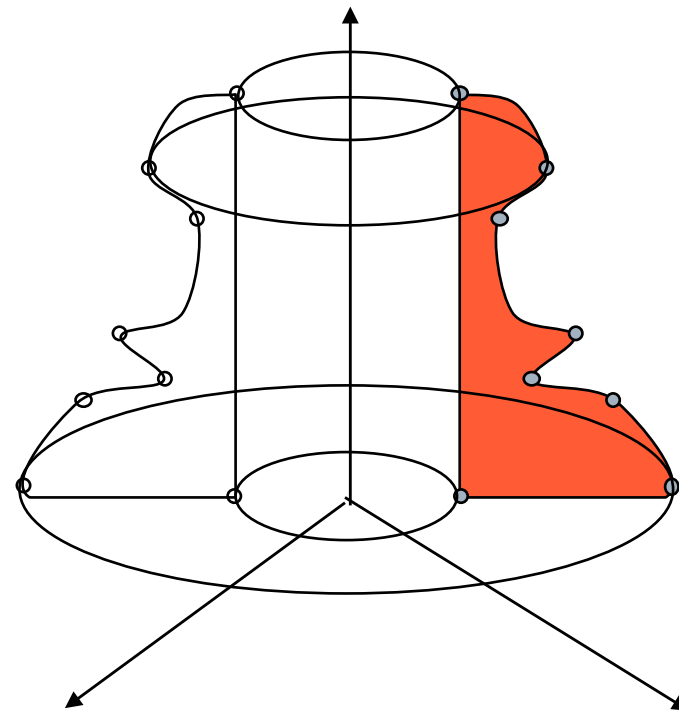
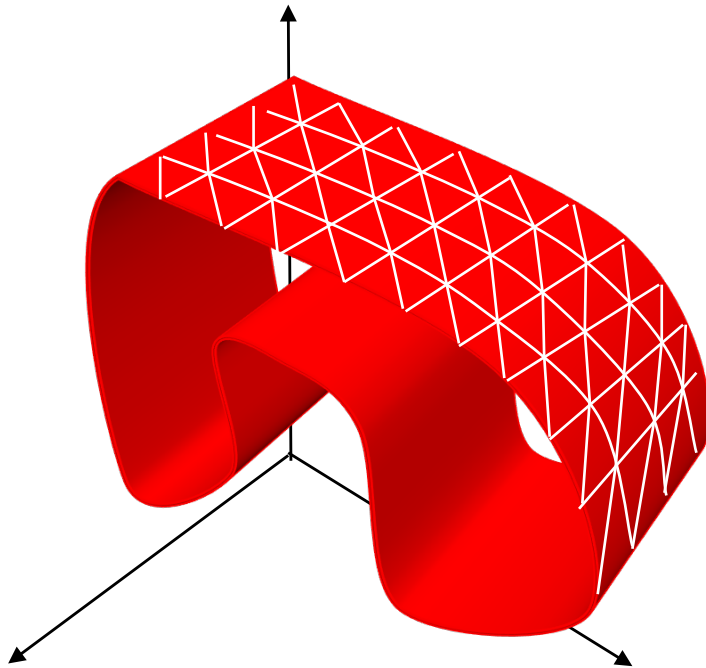
the most efficient
representation



$6/4=1.5$ vertices
per triangle

Polygonal objects by sweeping

- ❑ Sweep a cross section along a curve
- ❑ The cross section can be allowed to vary in shape as it is swept
- ❑ Define intervals along the curve at which the cross section is to be placed
- ❑ Join vertex to vertex with the previous emplacement of the cross section, with straight lines



Bicubic parametric patches

□ $P(u,v)=[x(u,v), y(u,v), z(u,v)], 0 \leq u \leq 1, 0 \leq v \leq 1$

□ Curves:

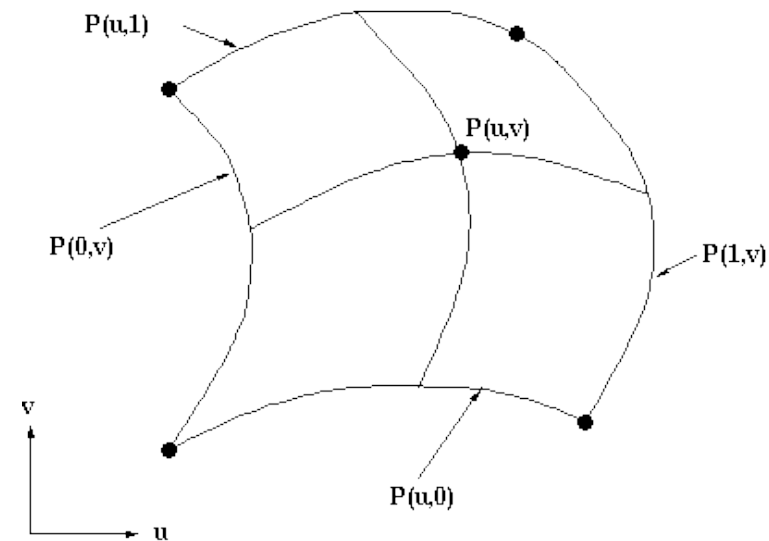
- Hermite
- Bézier
- Spline
- Uniform B-Spline

□ Surfaces

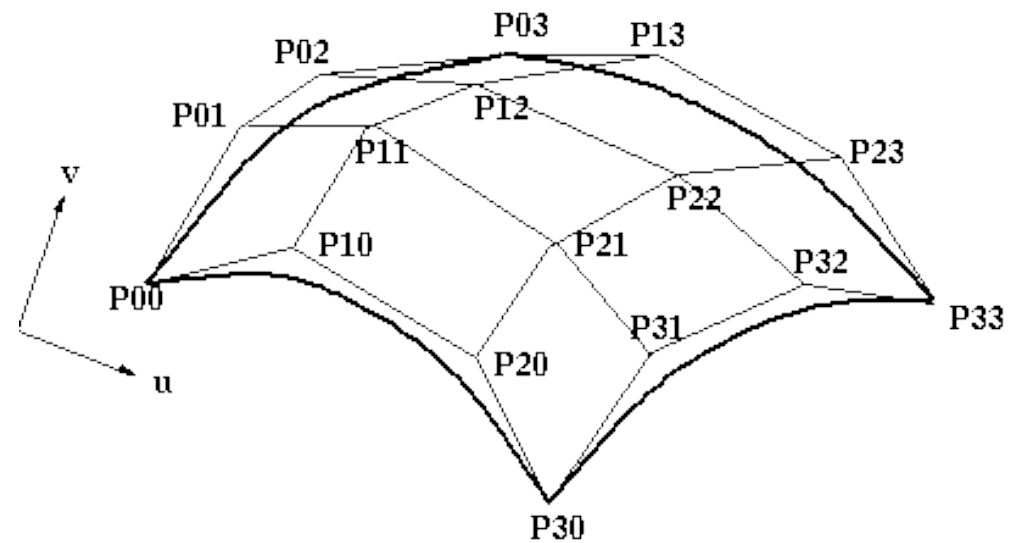
- Hermite
- Bézier
- B-Spline

□ Advantages:

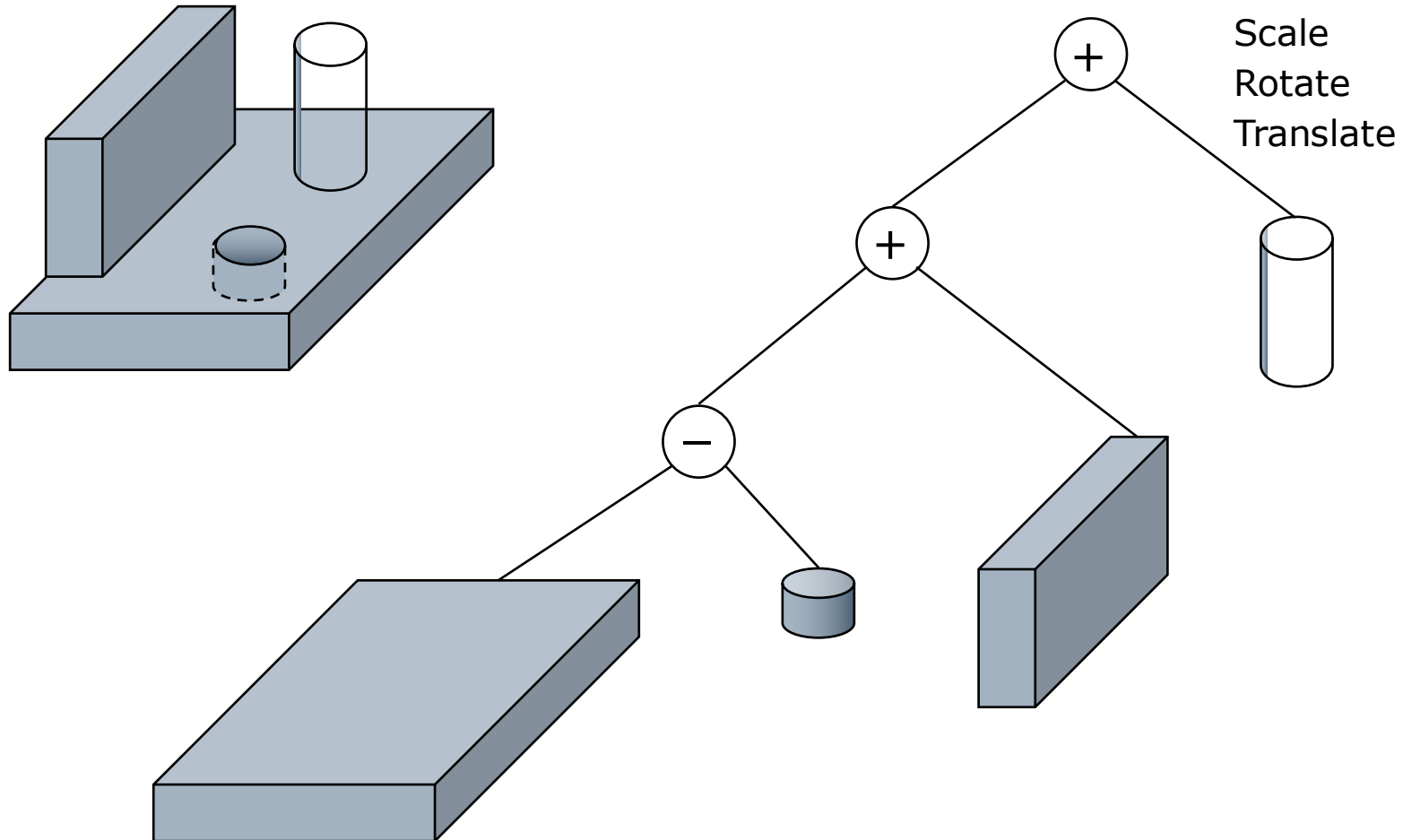
- Parametric representation is exact and economic since it is **analytical**. With a polygonal object, the shape accuracy can only be approximated at the expense of extra processing and database costs
- Deformations of parametric surfaces appear **smooth**. This is not generally the case with a polygonal object



Bicubic parametric patches



Constructive Solid Geometry (CSG)

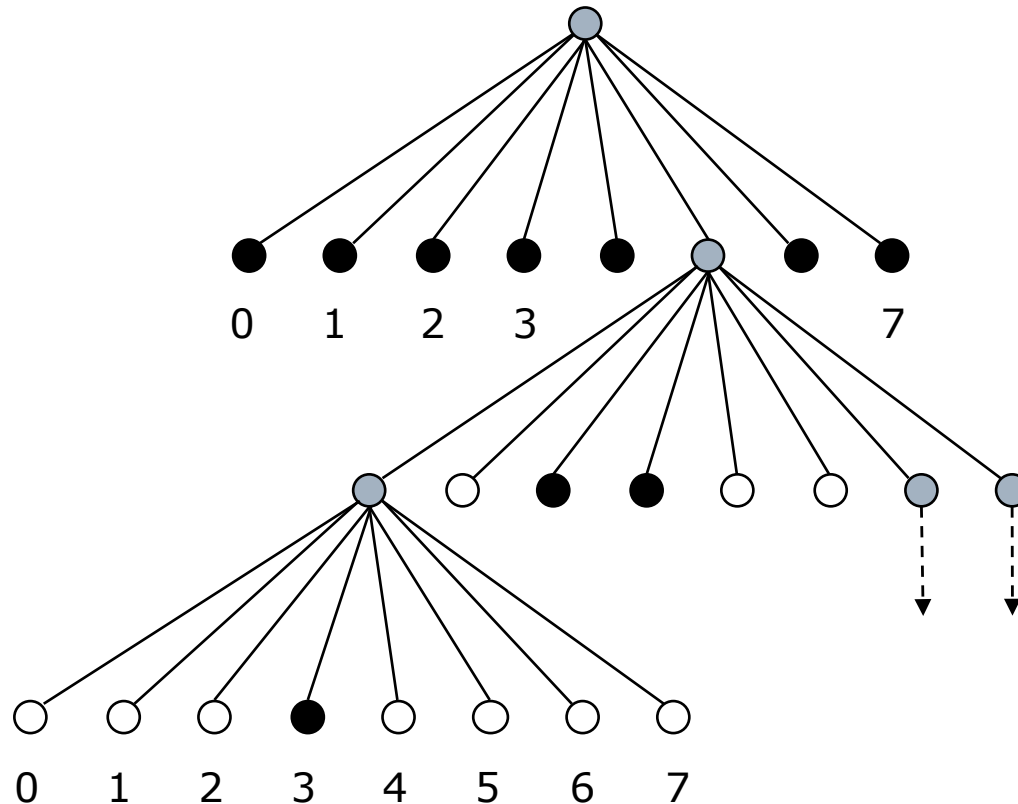
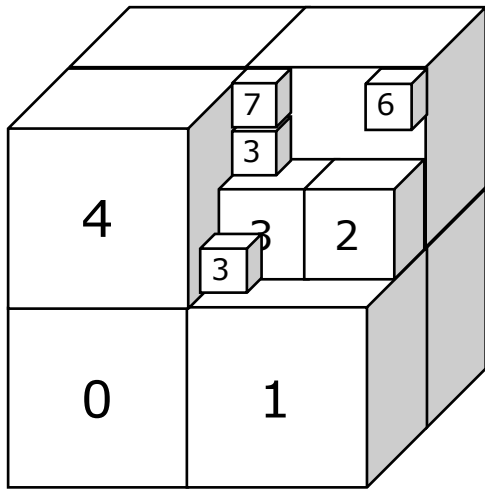


Constructive Solid Geometry (CSG)

- Hierarchical modeling based on tree structure
 - The internal nodes are set to operations: union, intersection or difference (sometimes complement)
 - The edges of the tree have transformations associated with them
Scale, rotate, translate
 - The leaves contain only geometry
- Allows complex shapes with only a few primitives
 - cylinders, cubes, etc, or quadric surfaces
- Motivated by computer aided design and manufacture
 - Boolean operators correspond to mechanical operations.
E.g. difference is like drilling or milling
 - A common format in CAD products

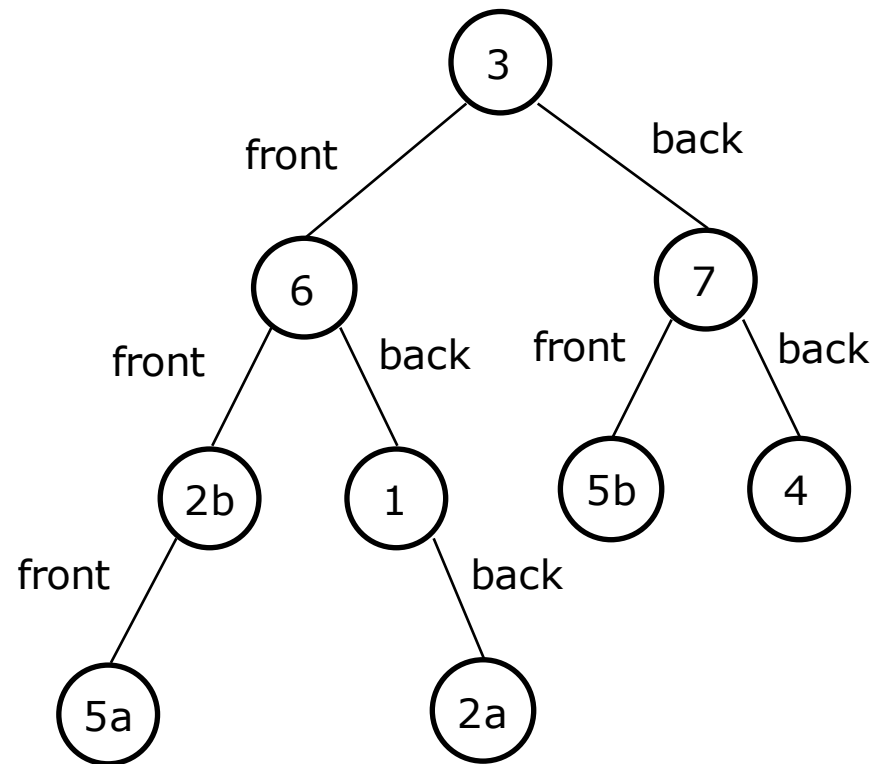
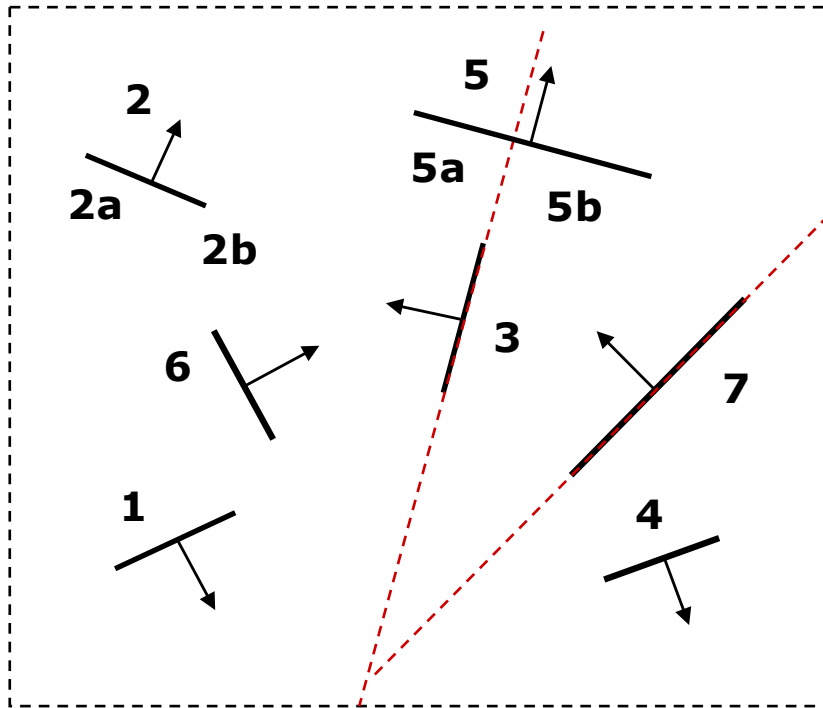
Space subdivision techniques

□ Octree representation



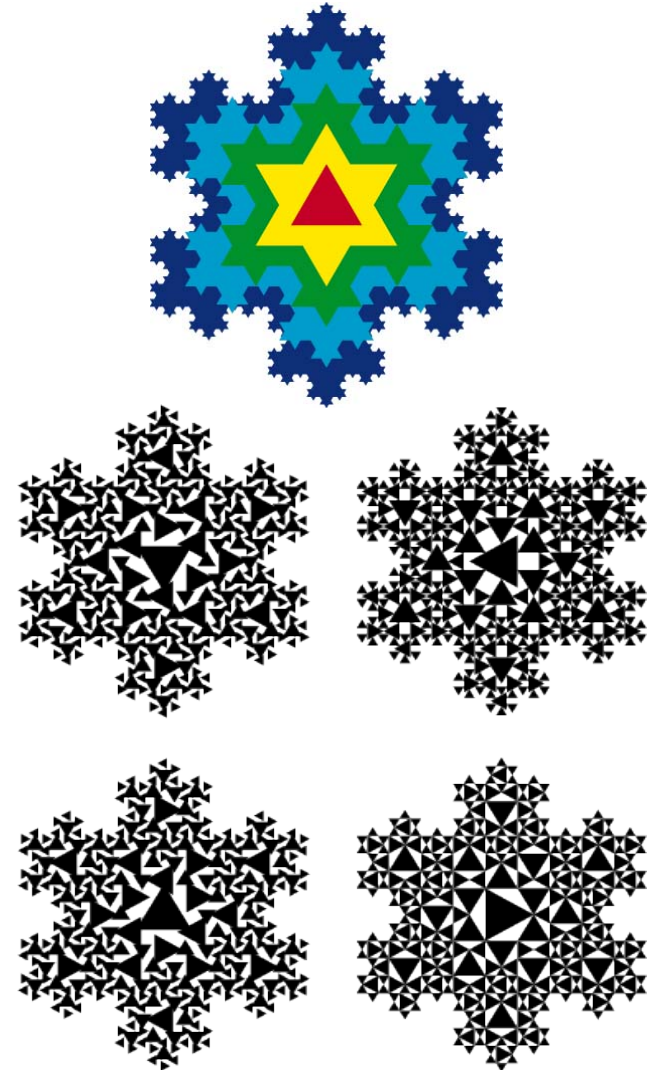
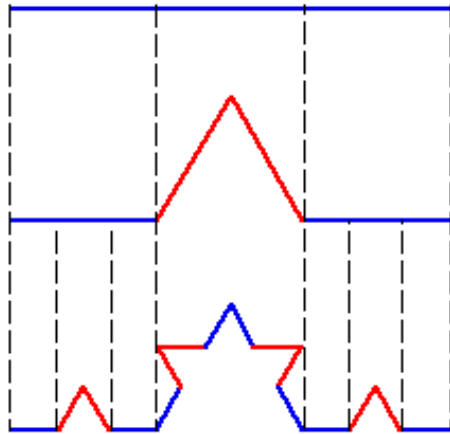
Space subdivision techniques

□ BSP Trees (Binary Space Partitioning Trees)

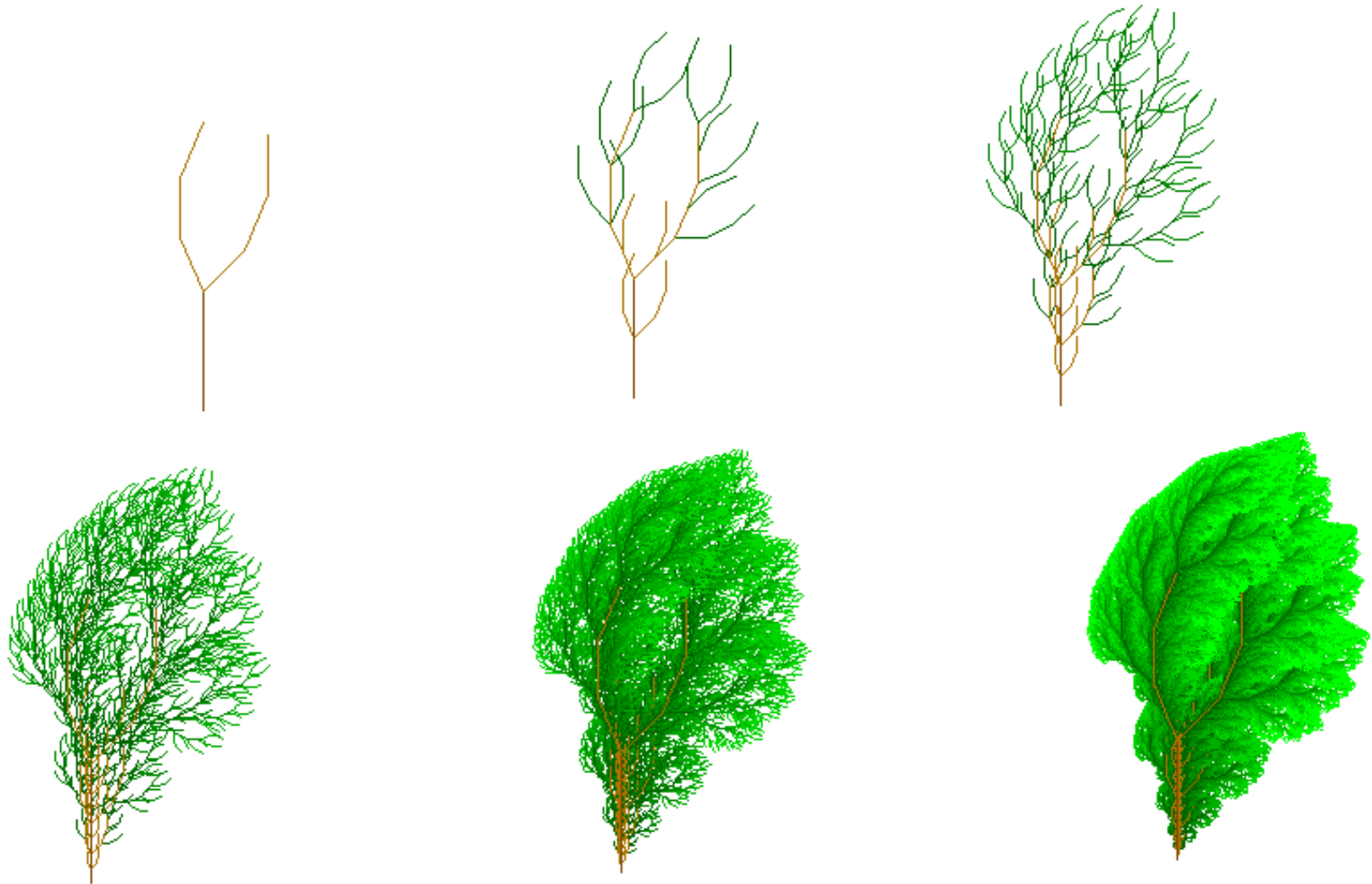


Procedural modeling

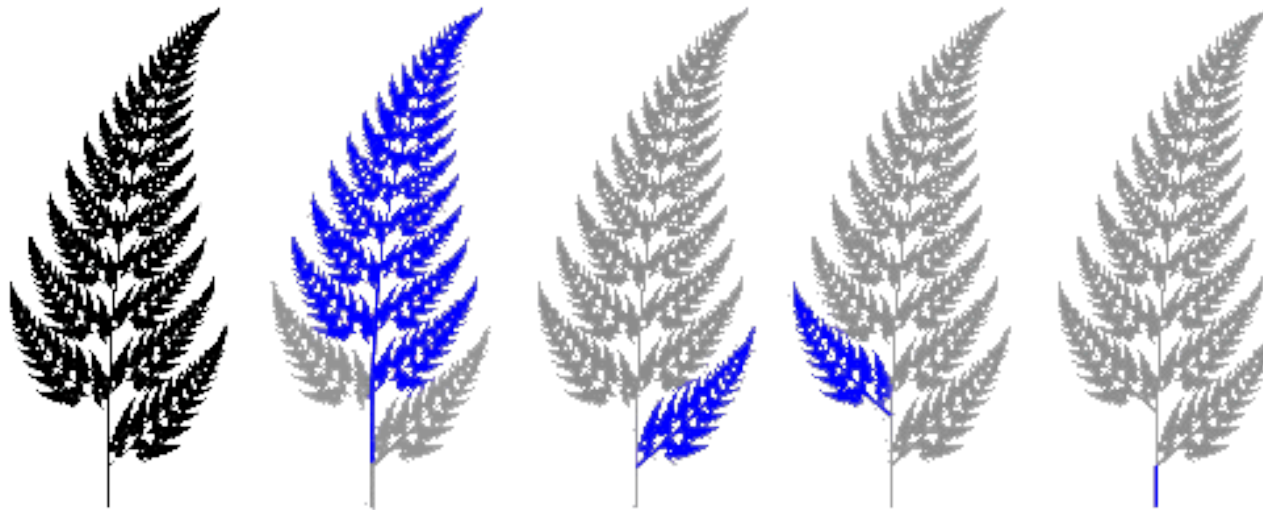
- ❑ Fractals, fractal dimension
- ❑ Apply algorithmic rules to generate shapes



Fractals

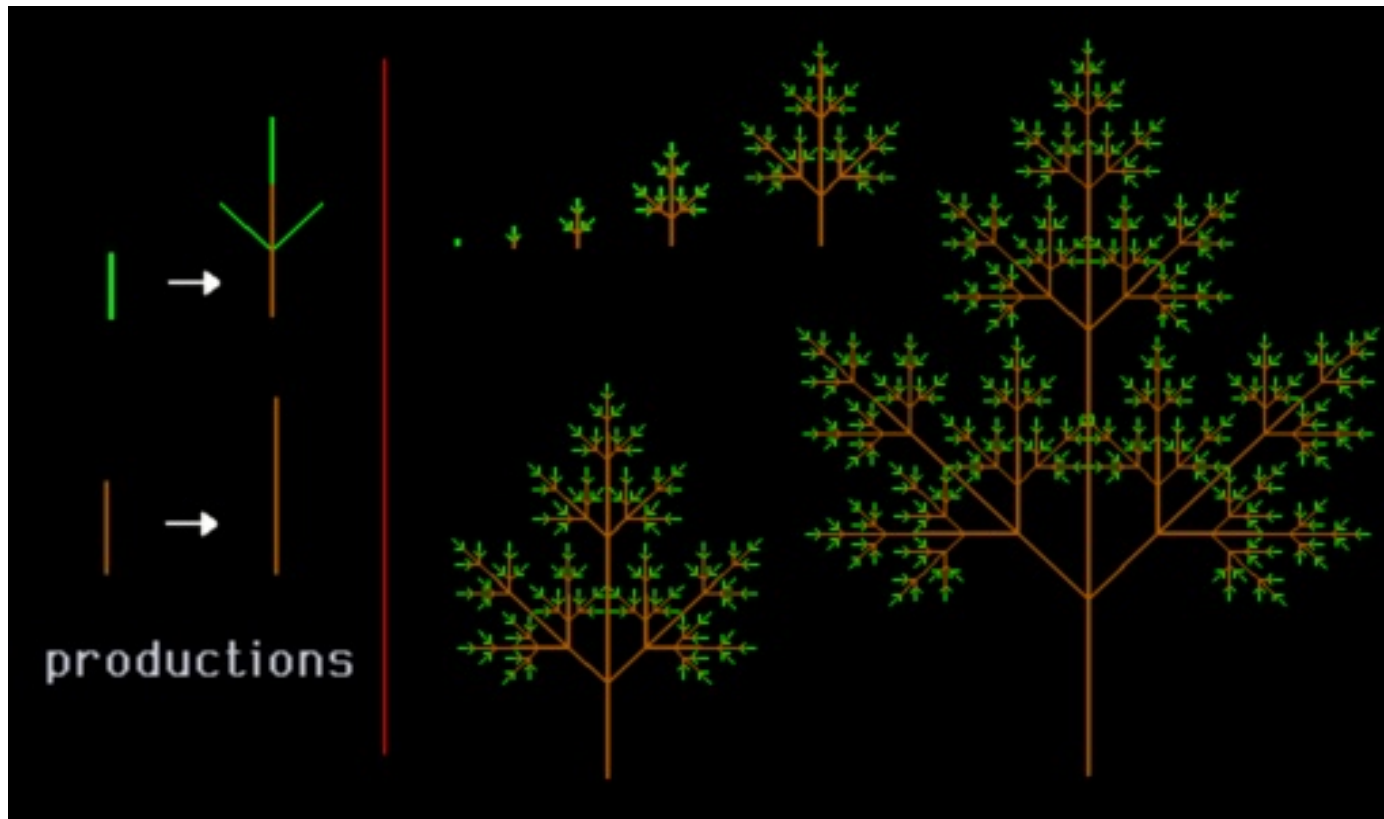


Fractals



Procedural modeling

- ❑ Lindenmayer-System or L-System
- ❑ Prusinkiewicz, P., Lindenmayer, A., Hanan, J.: "Development Models of Herbaceous Plants for Computer Imagery Purposes", SIGGRAPH 88, pp. 141-150
- ❑ Examples from Prusinkiewicz, Hammel, Mech. <http://www.cpsc.ucalgary.ca/projects>

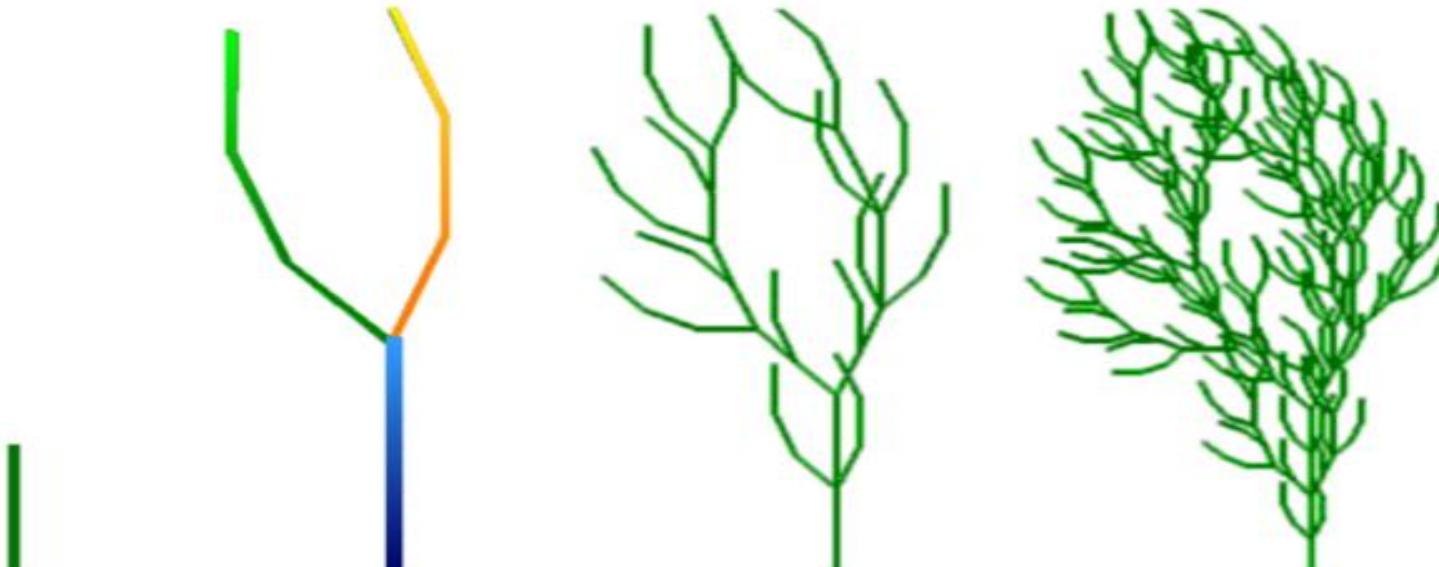


Lindenmayer-System or L-System

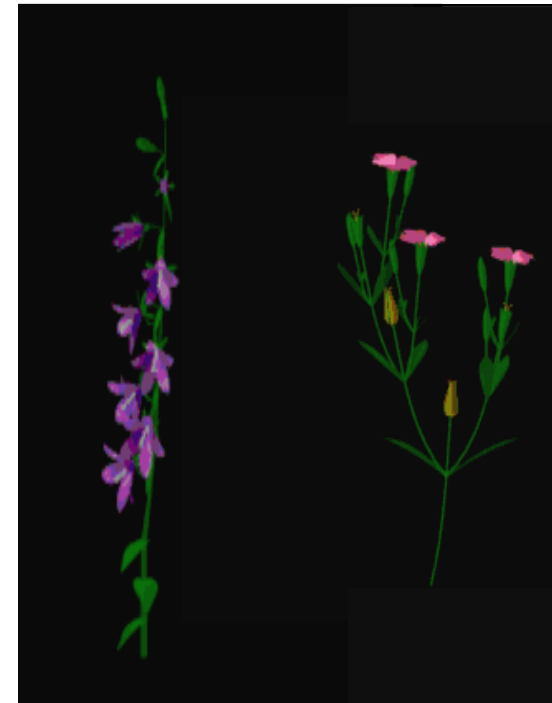
- Production rules:

W -> **F**

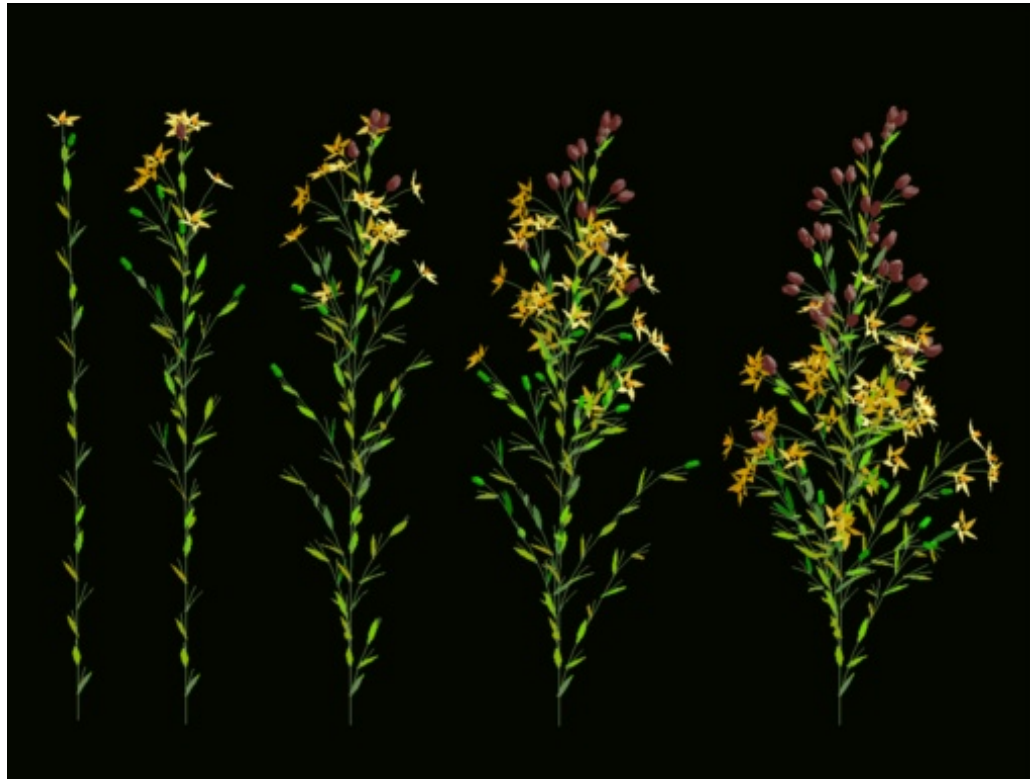
F -> **FF**+**[+F-F-F]** **-[-F+F+F]**



Lindenmayer-System or L-System



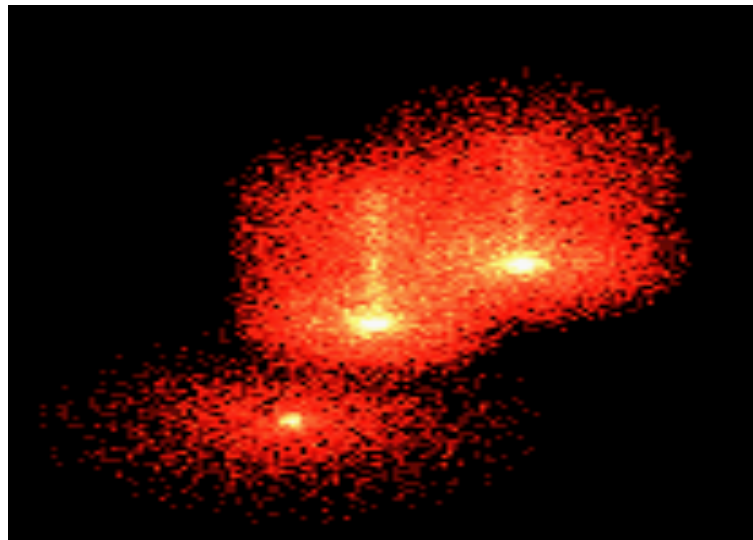
Lindenmayer-System or L-System



Procedural modeling

□ Particle based systems

- The term particle system is loosely defined in computer graphics. It has been used to describe modeling techniques, rendering techniques, and even types of animation. In fact, the definition of a particle system seems to depend on the application that it is being used for.
- Describe natural or irregular shaped objects which are flowing or fluid-like, particular objects which are changing over time
- Model objects like grass, waterfalls, fireworks, cloth, etc.

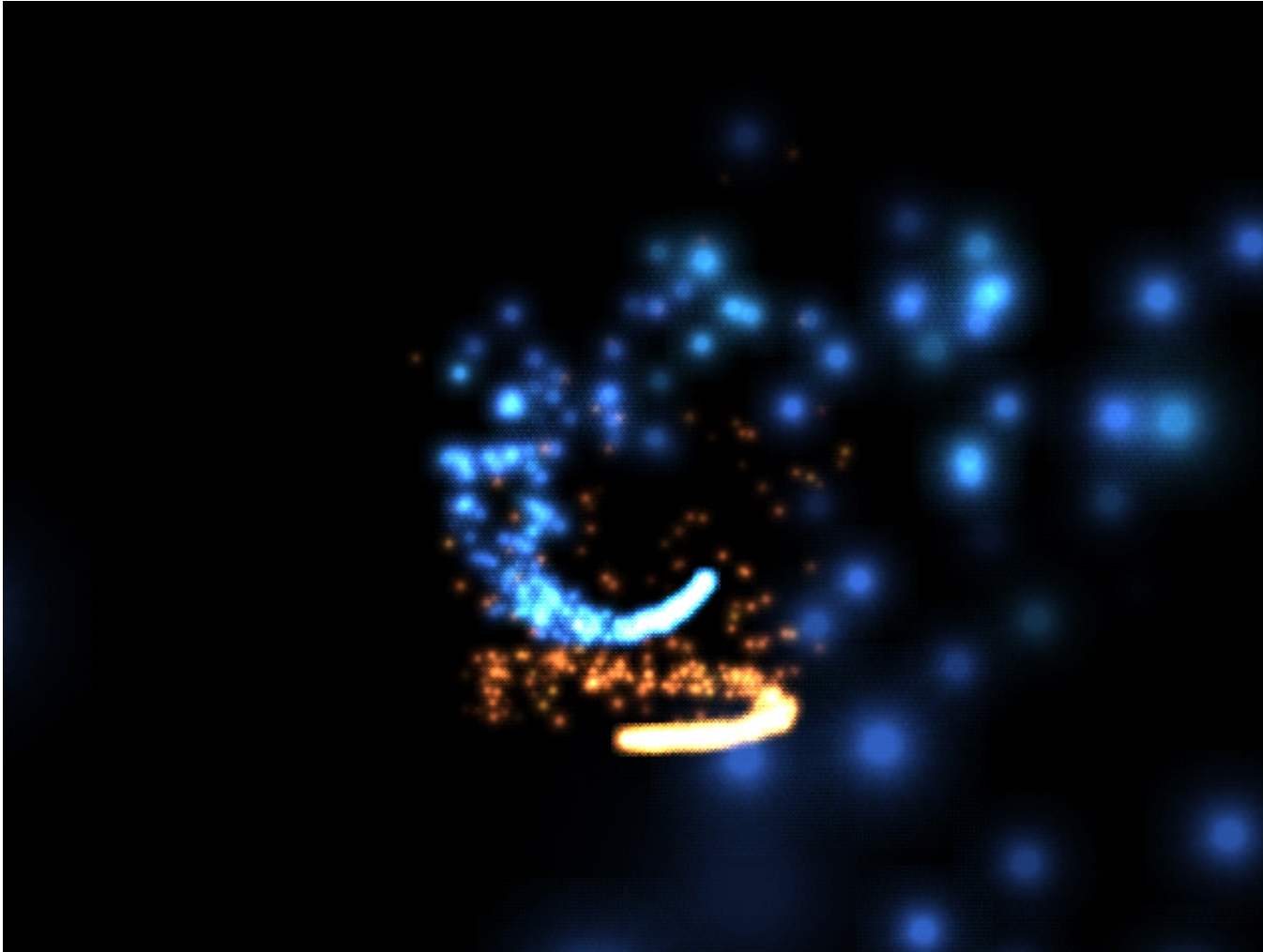


Particle based systems

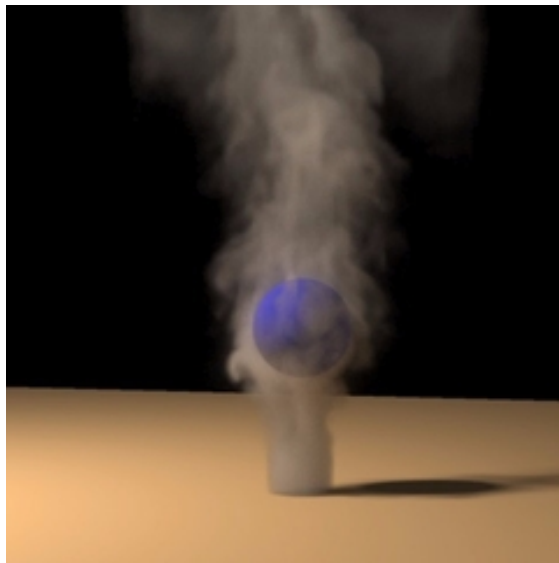
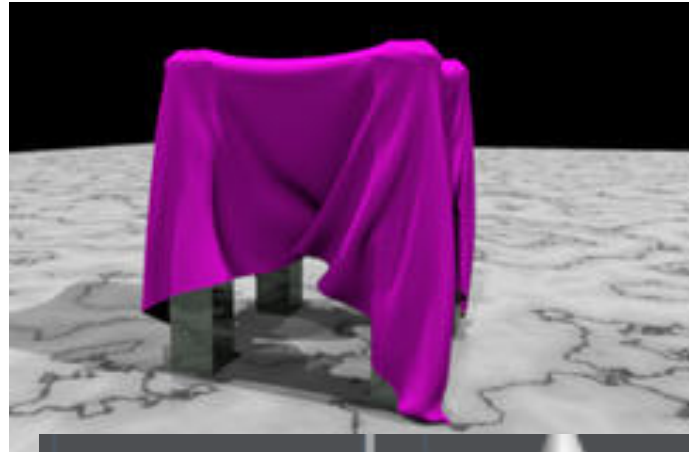
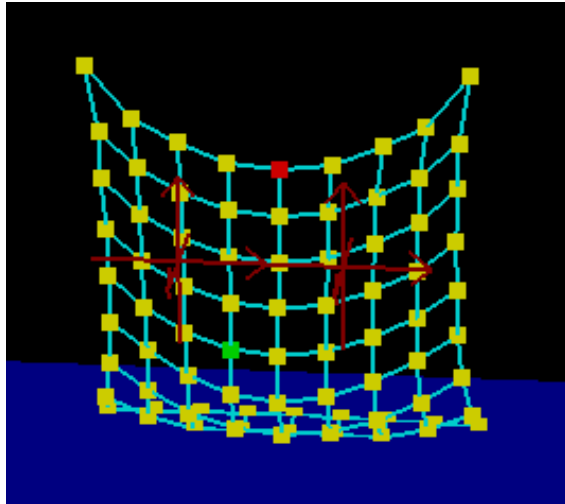
- ❑ Collection of particles - a particle system is composed of one or more individual particles.
- ❑ Stochastically defined attributes : position, velocity (speed and direction), color, lifetime, age, shape, size, transparency
- ❑ Particle Life Cycle
 1. Generation
 - particles in the system are generated randomly within a predetermined location of the fuzzy object
 2. Particle Dynamics
 - the attributes of each of the particles may vary over time. (particle position is going to be dependent on previous particle position and velocity as well as time)
 3. Extinction
 - each particle has two attributes dealing with length of existence: age and lifetime.
 4. Rendering
 - when rendering this system of thousands of particles, some assumptions have to be made to simplify the process.

Particle based modeling

ENERGiE - Particle Engine v2.26. Bua Manuel, <http://www.Planet-Source-Code.com>

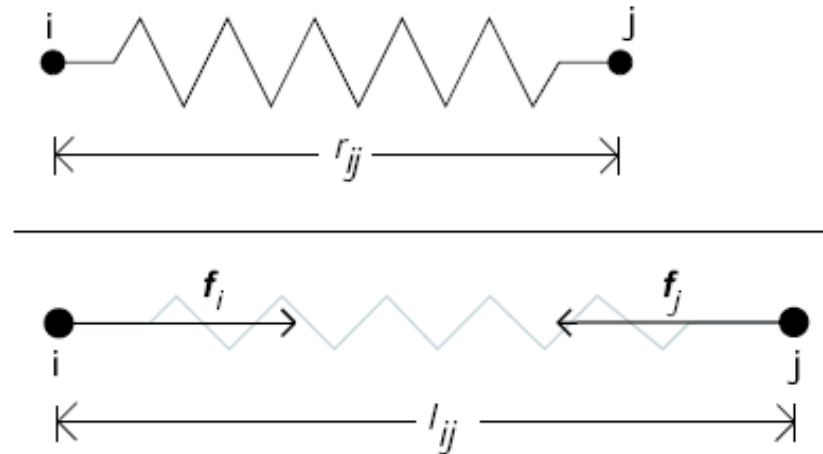


Particle based modeling

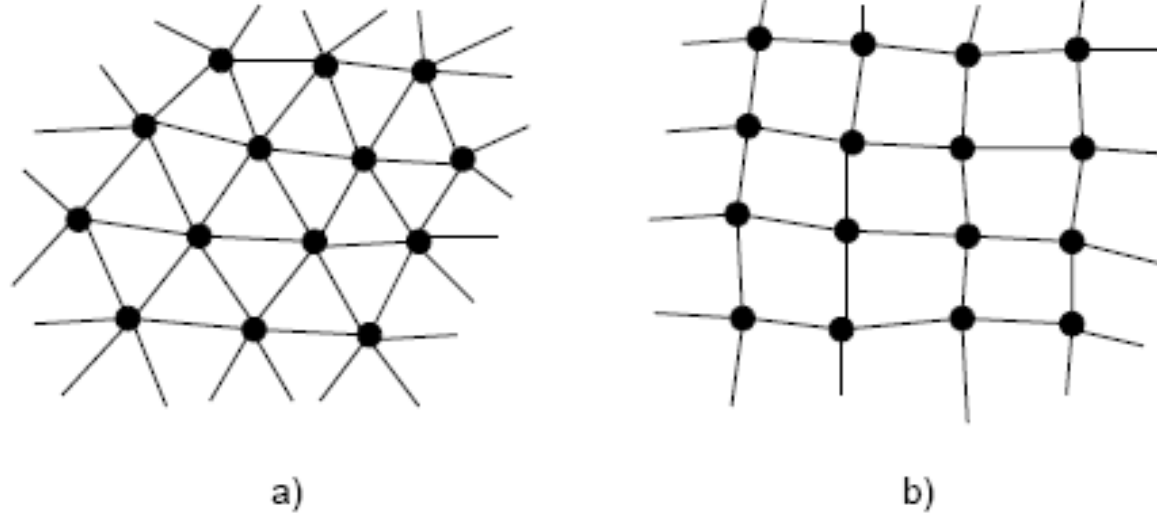


Mass-spring particle model

- ❑ Connected particle network
- ❑ The link between two nodes **i** and **j** is an arc
- ❑ $\Delta x = k x$



Mass-spring particle model



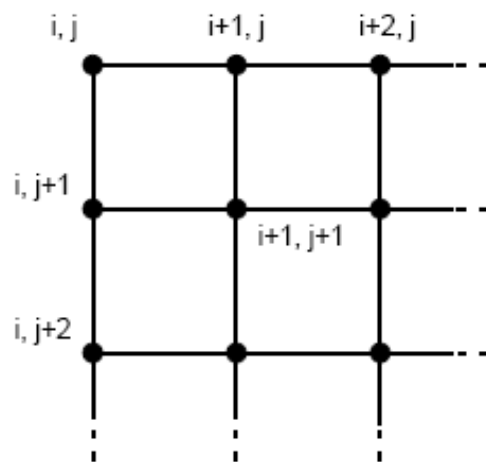
Particle network a) triangular topology b) rectangular topology.

Simulation:

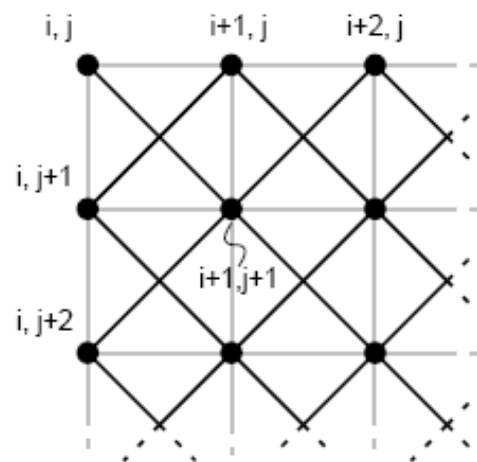
1. Force computation
2. Collision detection
3. Graphics rendering

Mass-spring particle model

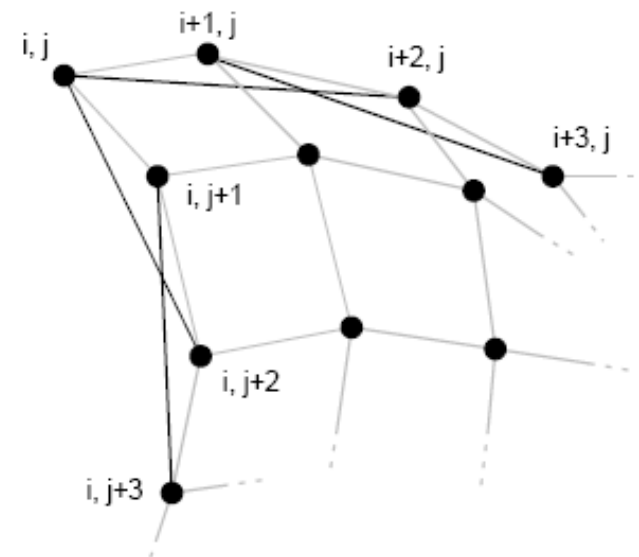
- Links between nodes (arcs):
 - Structural/Tensile springs (Strutturale)
 - Shear springs (Intindere)
 - Bend springs (Indoire)



a) Structural/Tensile springs



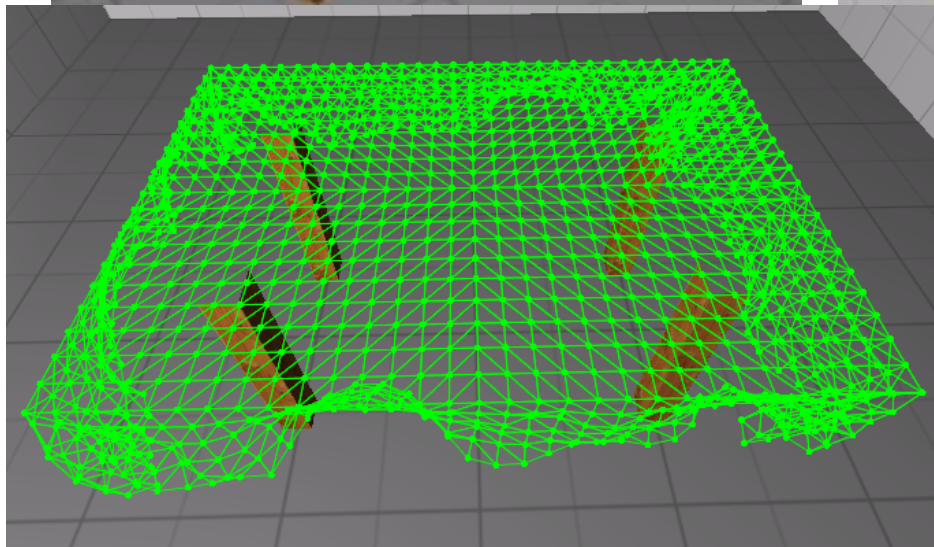
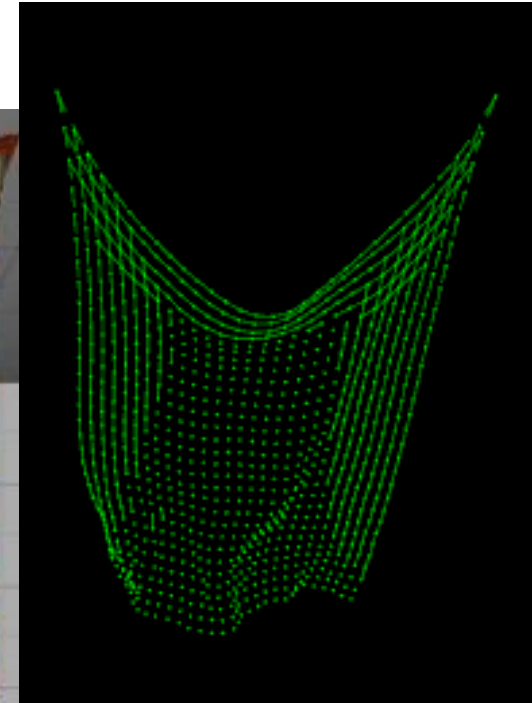
b) Shear springs



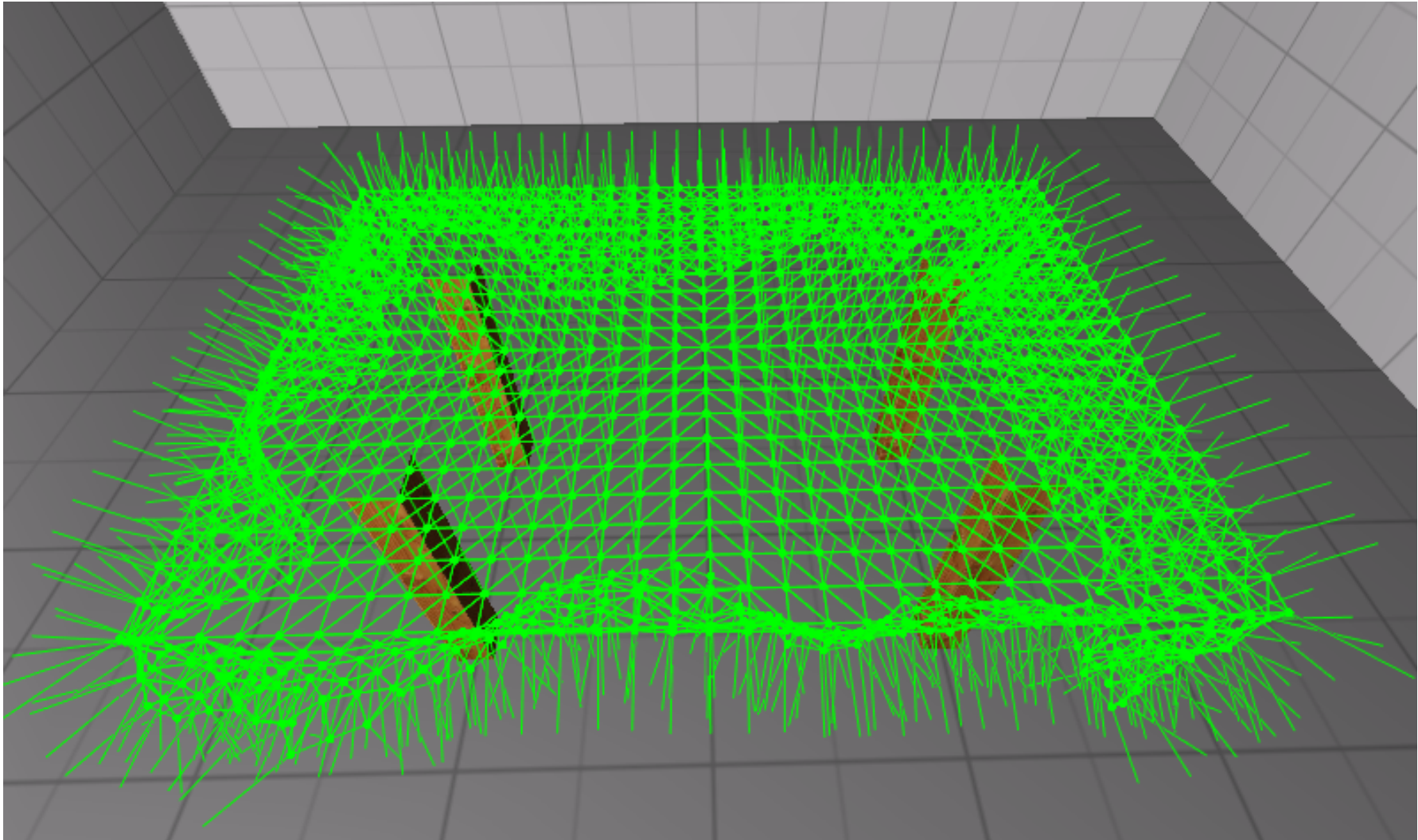
c) Bend springs
(not all connections shown)

Particle based modeling

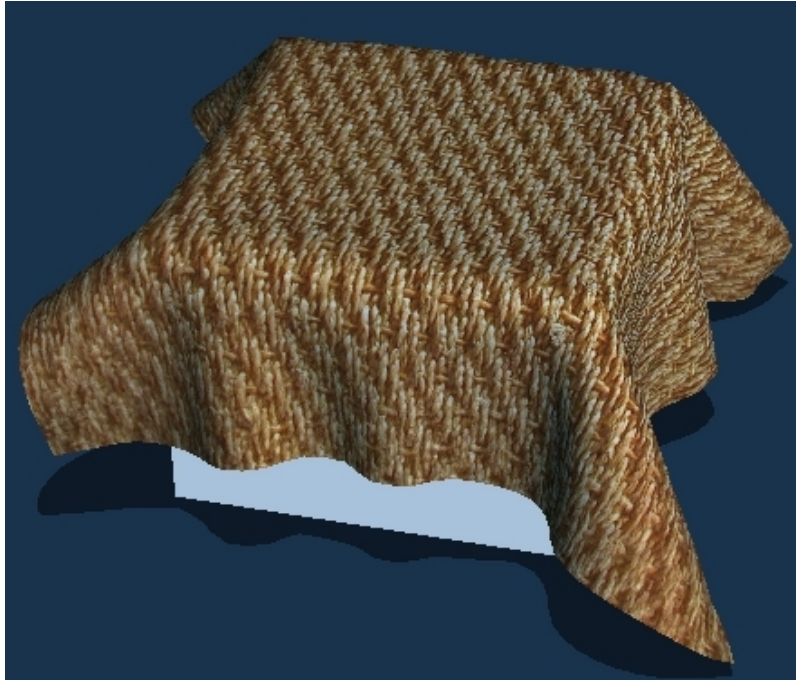
Cloth simulation: ClothSim, <http://www.benve.org/ClothSim>



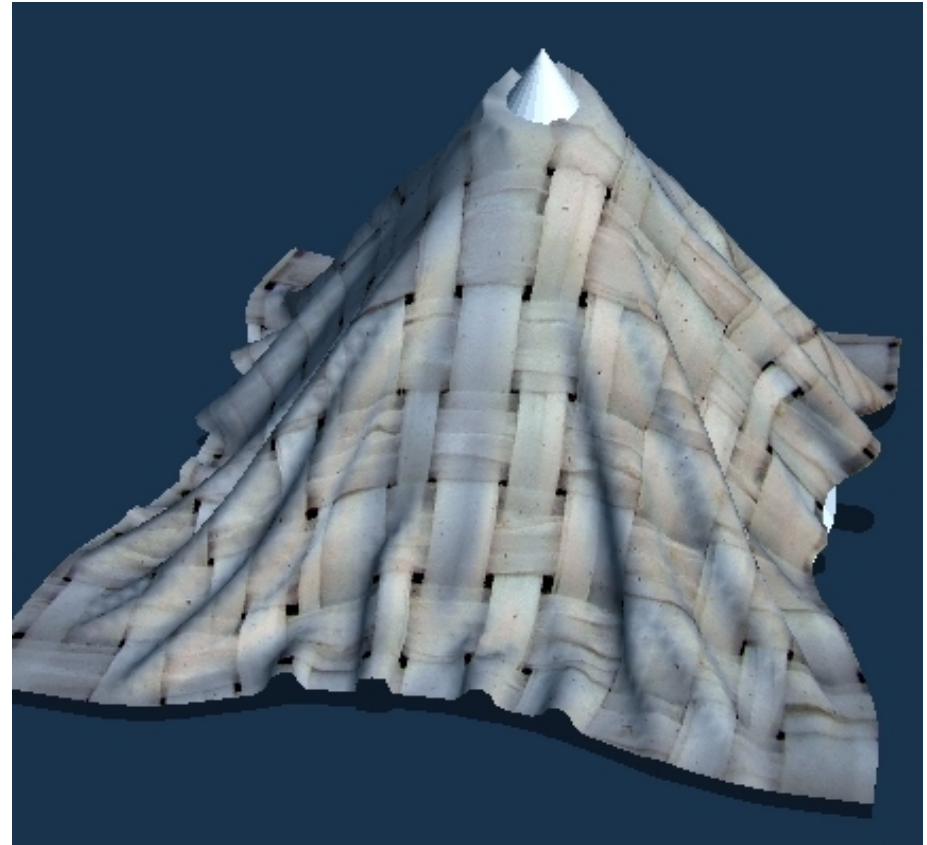
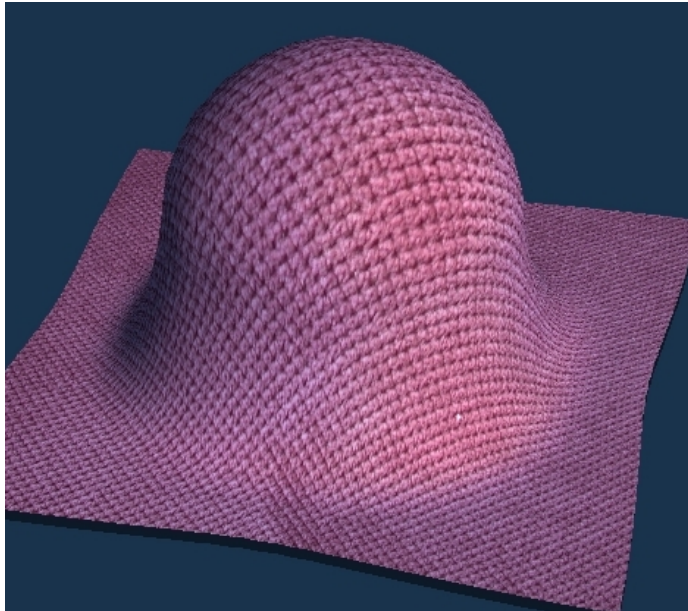
Particle based modeling - ClothSim



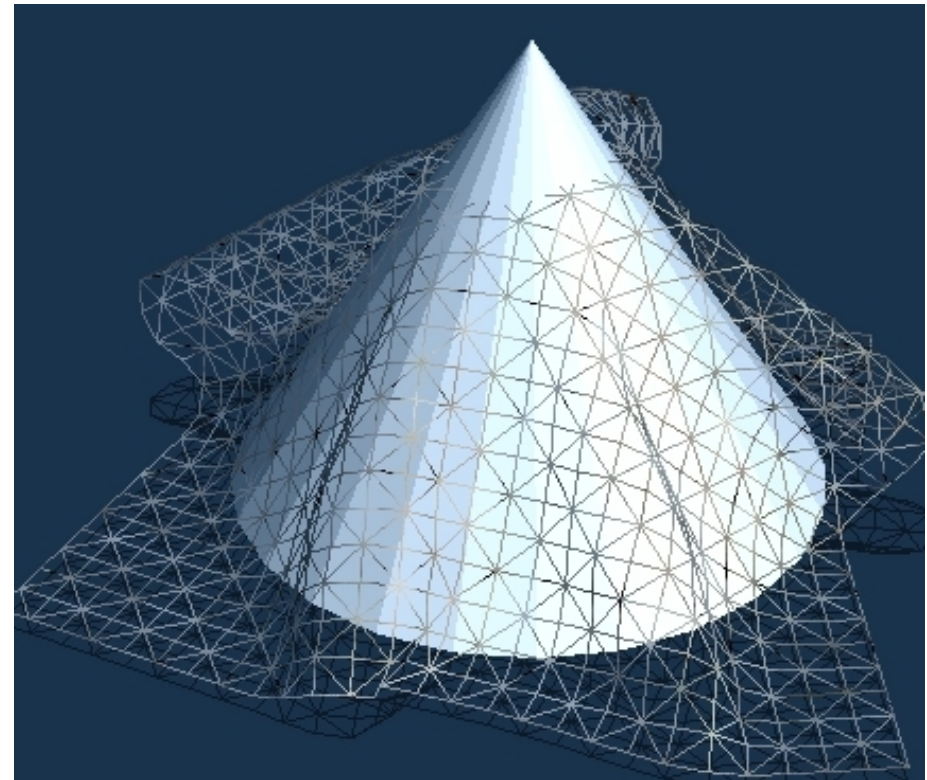
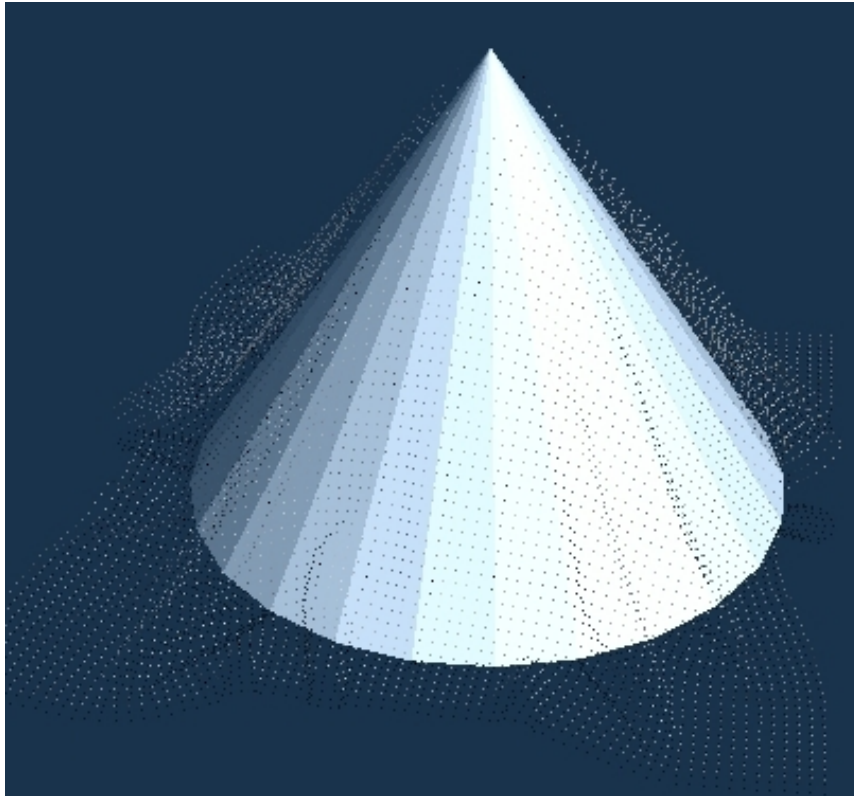
Particle based modeling



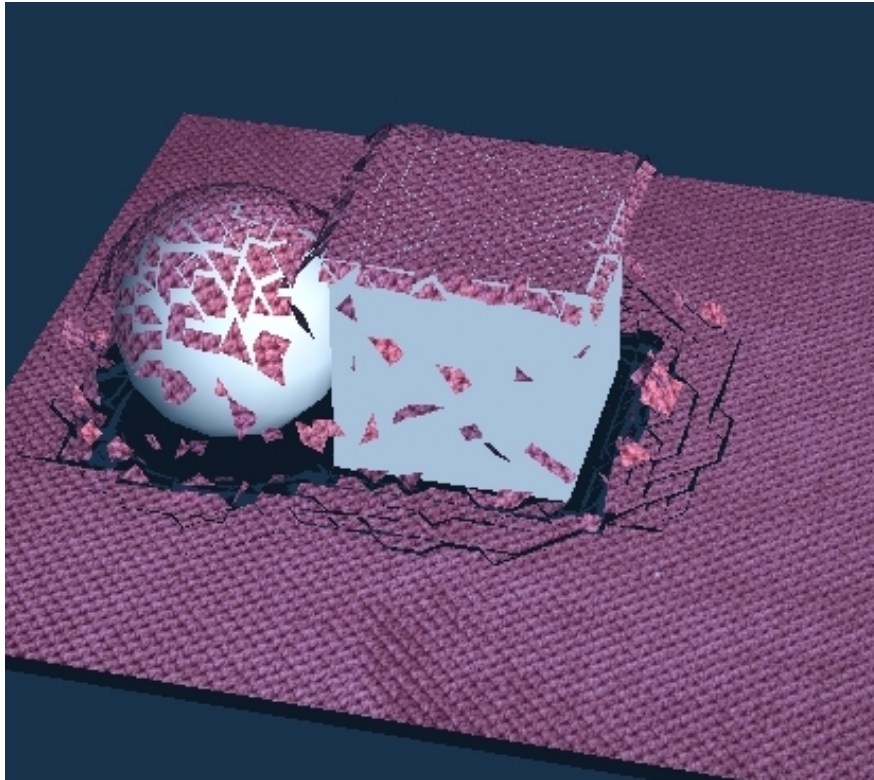
Particle based modeling



Particle based modeling



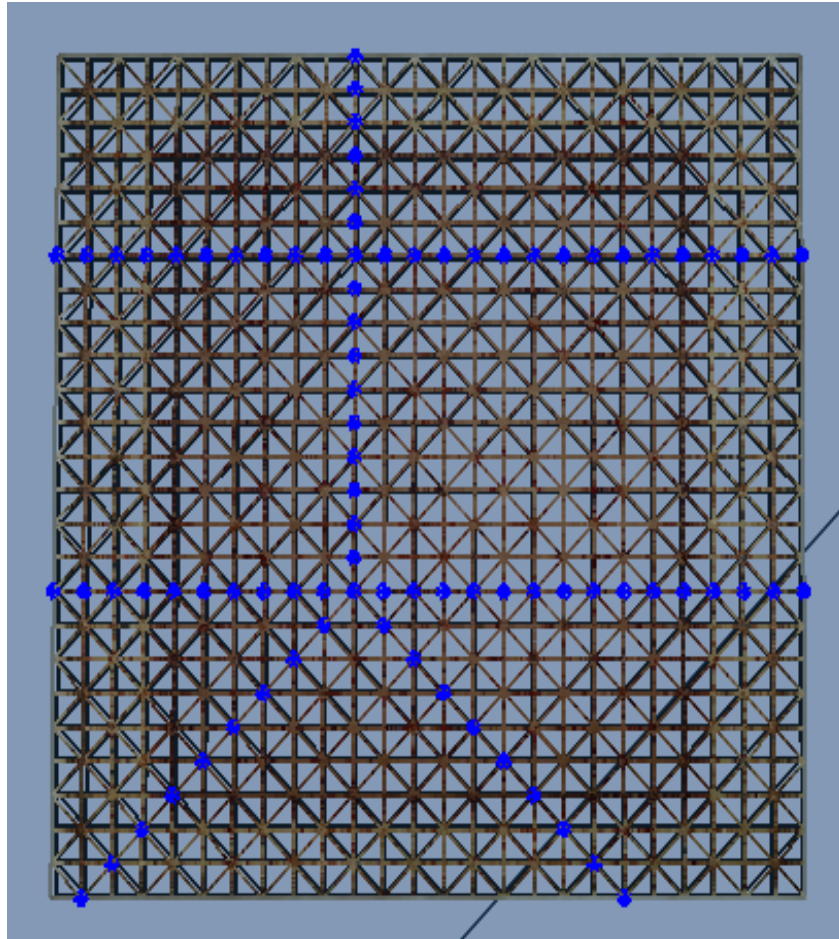
Particle based modeling



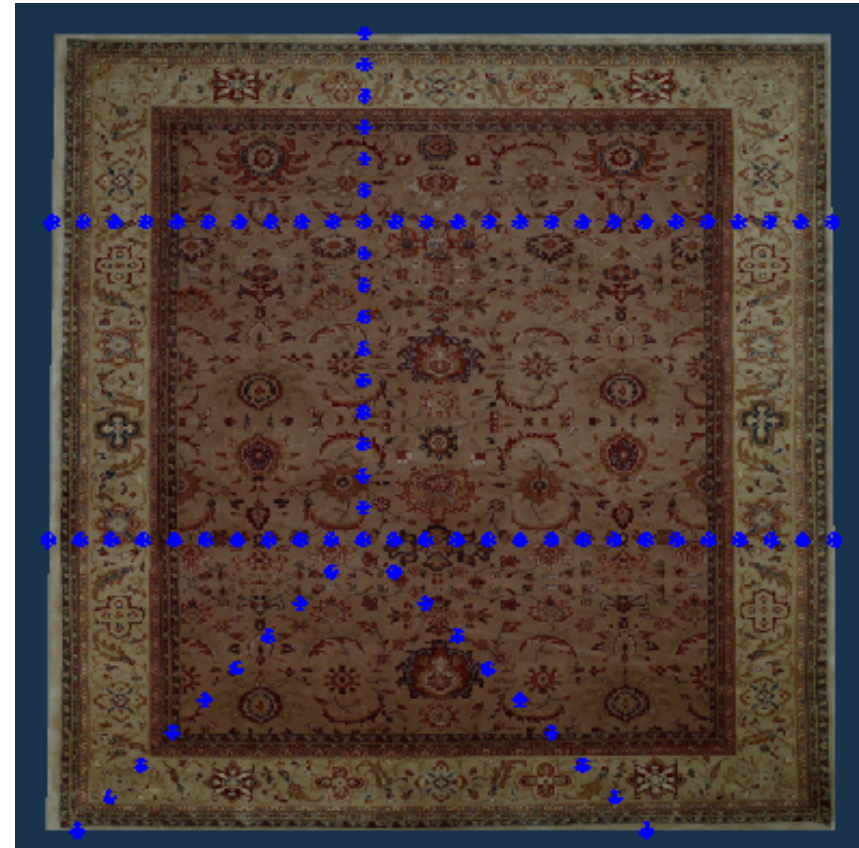
Particle based modeling



Particle based modeling - cutting

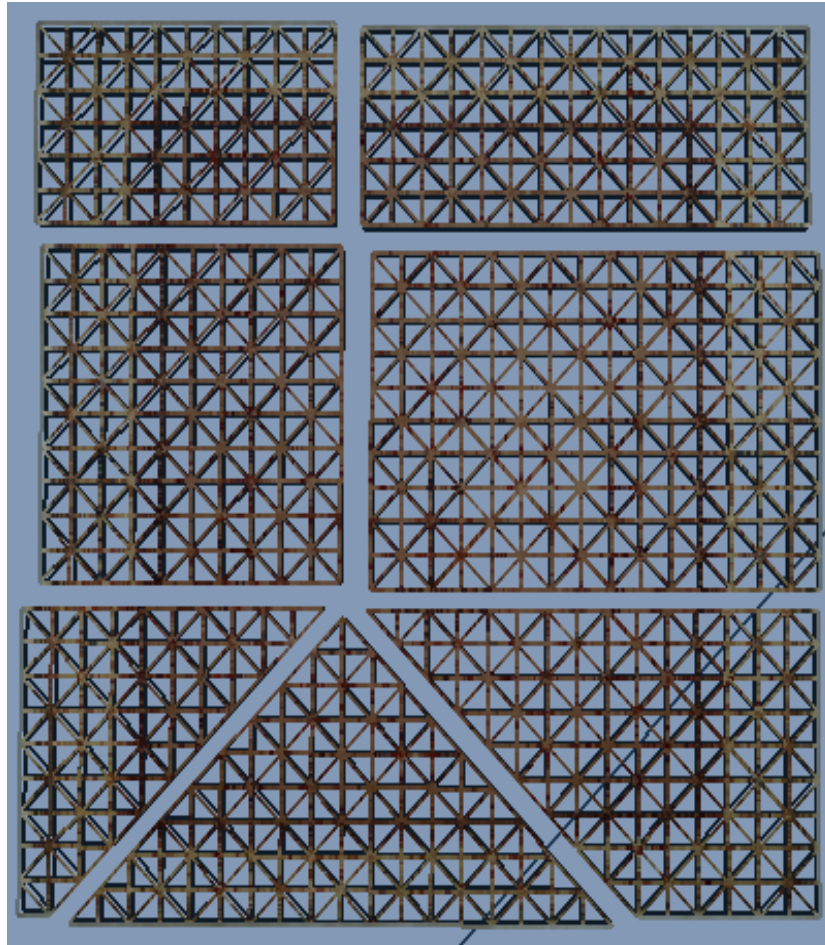


Selection of the cutting lines. Wireframe visualization.



Selection of the cutting lines. Texture mapping visualization.

Particle based modeling - cutting



Cutting operation. *Wireframe visualization.*

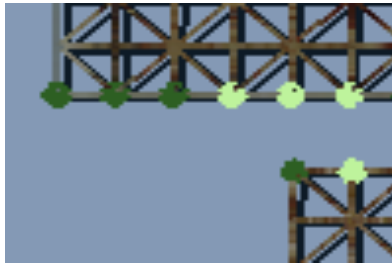


Cutting operation. *Texture mapping visualization.*

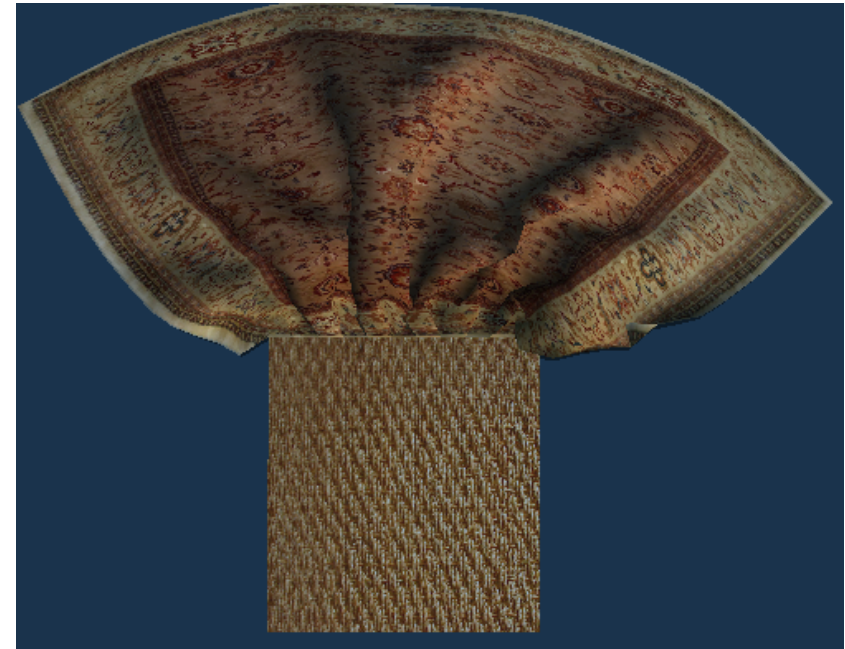
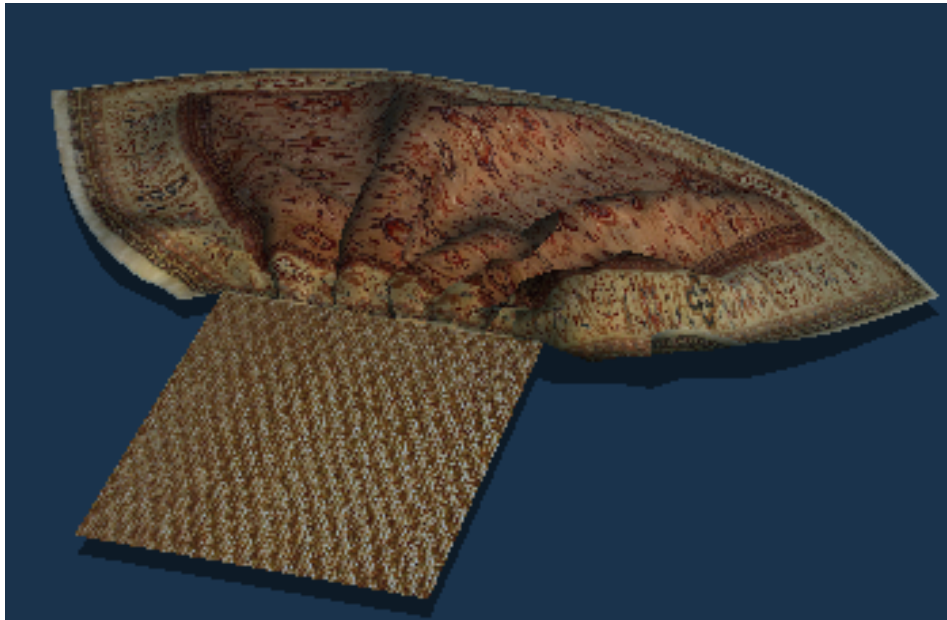


Particle based modeling - sewing

Sewing step equal three.



Preview of the cutting operation. Constant sewing step, equal three.



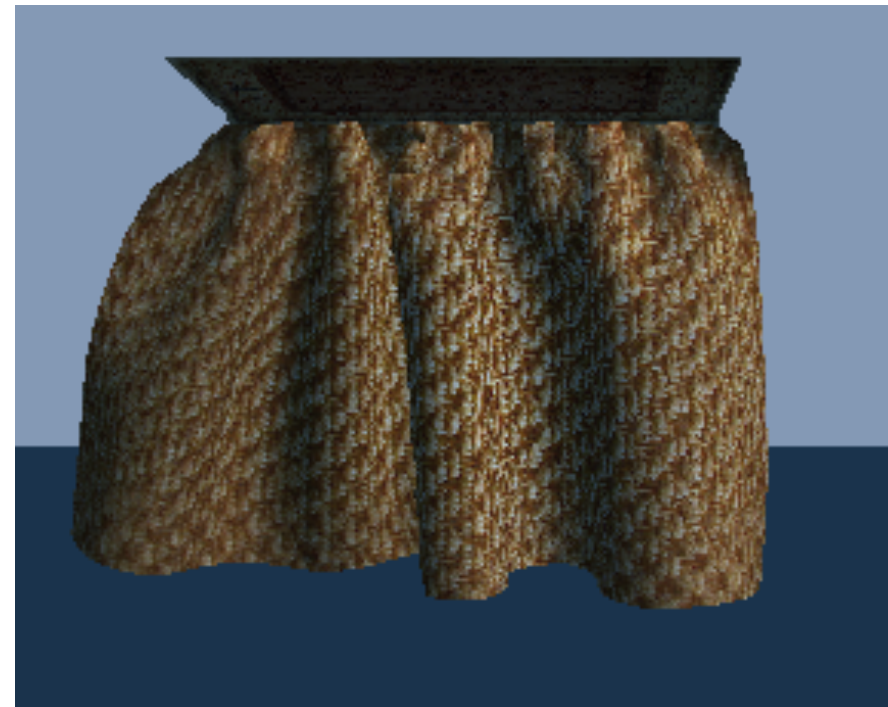
Sewing operation. Constant sewing step, equal three (Goga, 2009).

Particle based modeling - sewing

Variable sewing step.



Preview of the sewing operation. Variable sewing step, equal three and respectively two (Goga, 2009).



Sewing operation. Variable sewing step, equal three and respectively two (Goga, 2009).

Cloth on animated 3D objects



Baraff and Witkin, "Walking man and Dancer with long skirt", SIGGRAPH 98 Conference.

Case studies

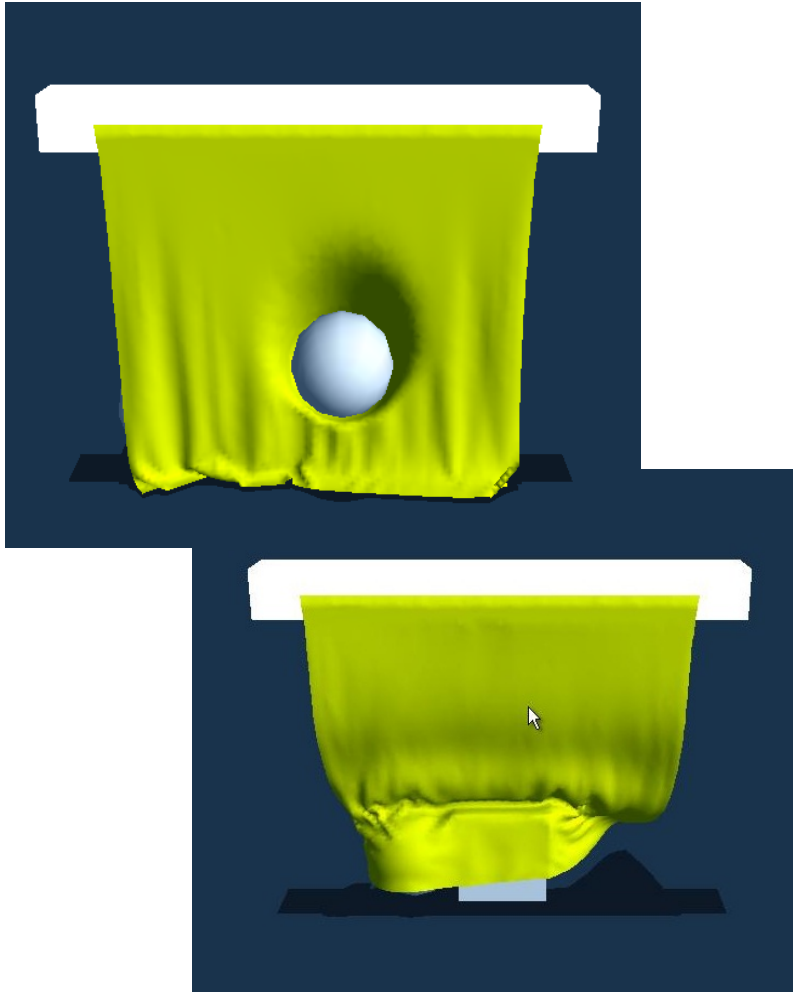
1. Distributed simulation and rendering
2. Parallel simulation
3. User interaction techniques

Use case 1 – Separated simulation and rendering

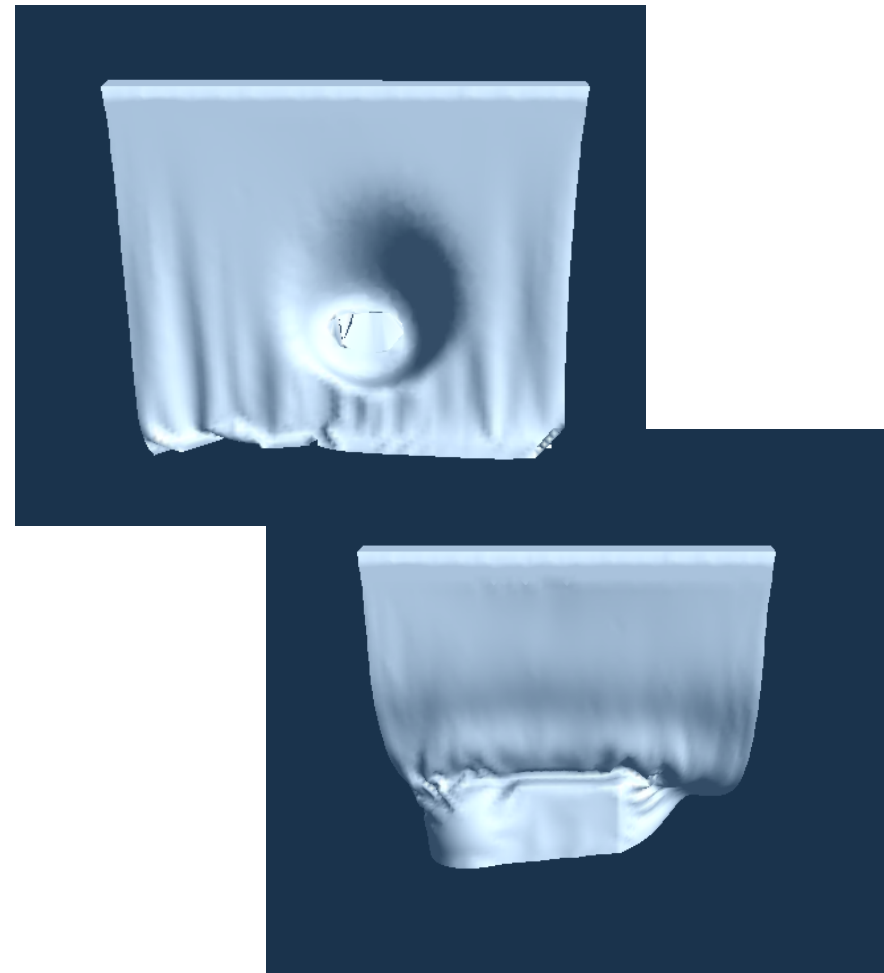


Use case 1 – Separated simulation and rendering

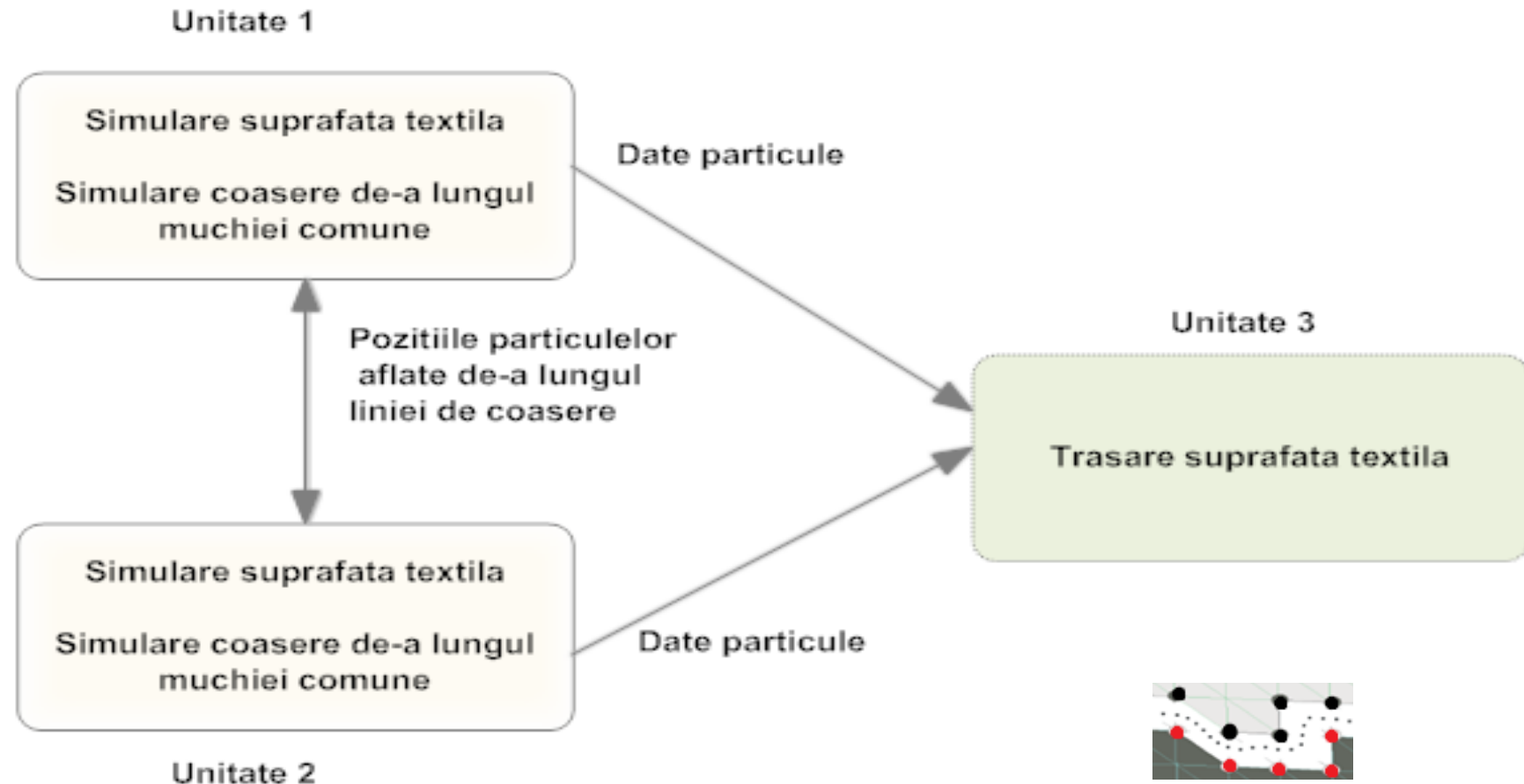
Textile surface on the simulation unit



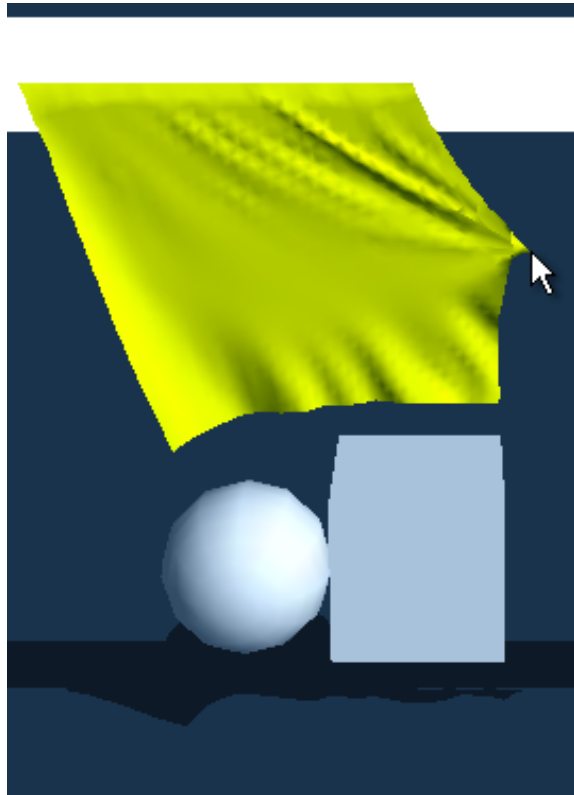
Textile surface on the rendering unit



Use case 2 – Parallel simulation



Use case 2 – Parallel simulation



Simulation unit 1

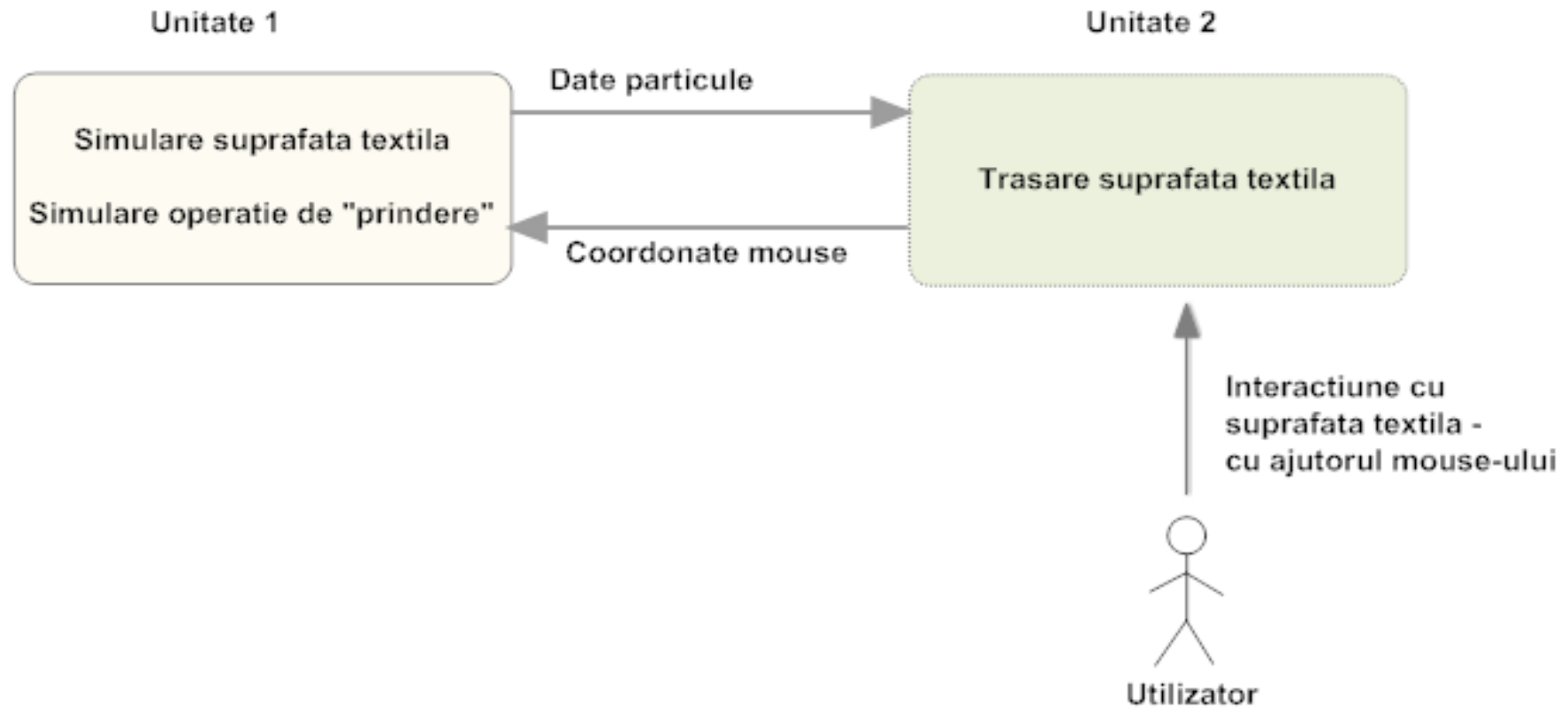


Simulation unit 2

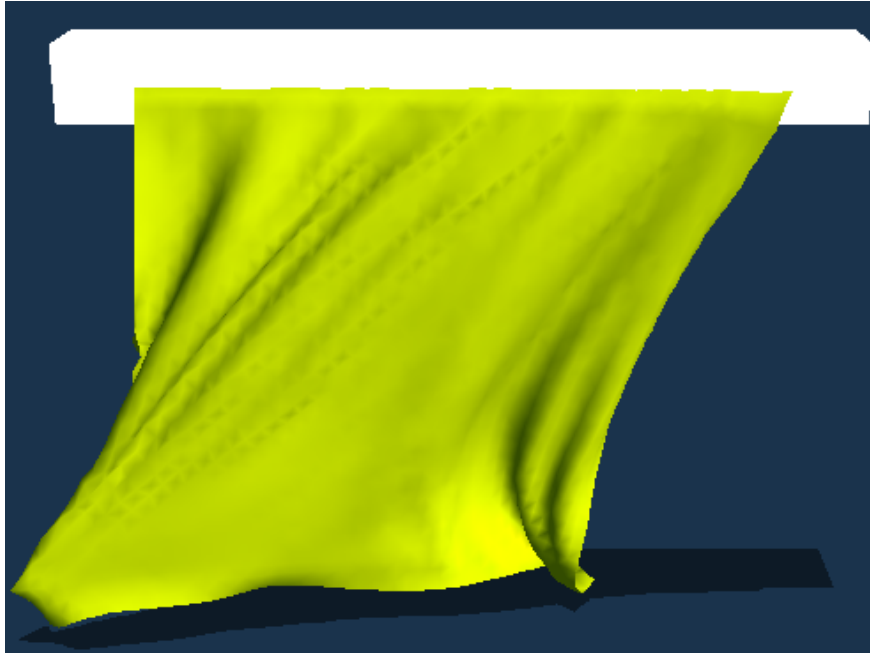


Rendering unit

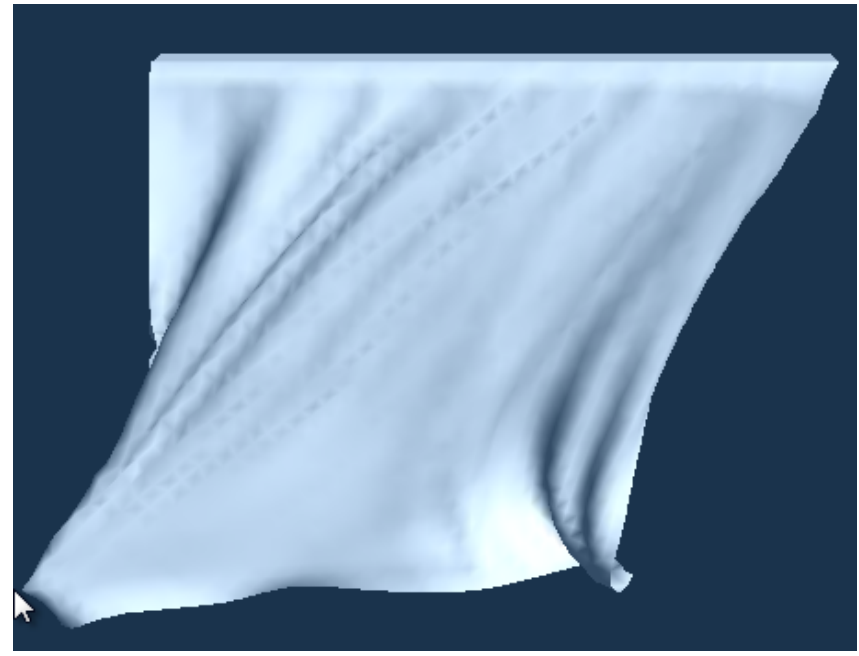
Use case 3 – User interaction



Use case 3 – User interaction



Simulation unit



User interaction at rendering unit

Questions and problems

1. Exemplify and explain the impact of the graphics processing objectives on the type and content of the graphics model.
2. Exemplify and explain the impact of the graphics model on the graphics processing pipeline resources (1) data structure; (2) processing algorithm; (3) design of software packages and hardware modules; (4) final appearance of the object; (5) techniques for editing the shape of the object.
3. Exemplify graphics model for a tetrahedron (regular triangular pyramid) in order to support the photorealistic presentation by BSP trees, shadows, and textures.
4. Why the triangle based polygonal model for the 3D object's surface is the most used?
5. Define and explain a fractal.
6. What is the fractal distance?

Questions and problems

7. What is the L-System grammar? Give examples of the grammar productions.
8. What features can be obtained by using the time based rules?
9. Explain and exemplify the particle based models.
10. How describes the mass-spring model the weight and the dimension of a piece of textile?
11. What issues arise the cutting and sewing operations on a mass spring model of a textile surface?

Representation and Modeling of 3D Objects - Examples

Examples

- Free-flowing Granular Materials with Two-way Solid Coupling ([video](#))
 - Continuum-based model
 - Efficient simulation of granular materials
 - Fully solves the internal pressure and frictional stresses in a granular material
 - Visually noticeable behaviors of granular materials
 - Freely dispersing splashes without cohesion, and a global coupling between friction and pressure
 - The full treatment of internal forces in the material also enables two-way interaction with solid bodies
 - Very small fraction of computational costs of the comparable particle-based models for granular flows

Examples

- ❑ RealFlow 2012 water simulation ([video](#))
- ❑ Stitch Meshes for Modeling Knitted Clothing with Yarn-level Detail - SIGGRAPH 2012 ([video](#))
- ❑ 3D Hair on Siggraph 2012 ([video](#))
- ❑ Realflow Flooded Corridor ([video](#))
 - Use standard fluids and emitters
 - Took 4 hours to simulate and mesh, 1 hour to render
 - Use an opaque material for the liquid
 - Expensive rendering of refractions

Rendering Strategies

Contents

- Rendering polygonal objects
- Rendering parametric patch net
- Rendering CSG description

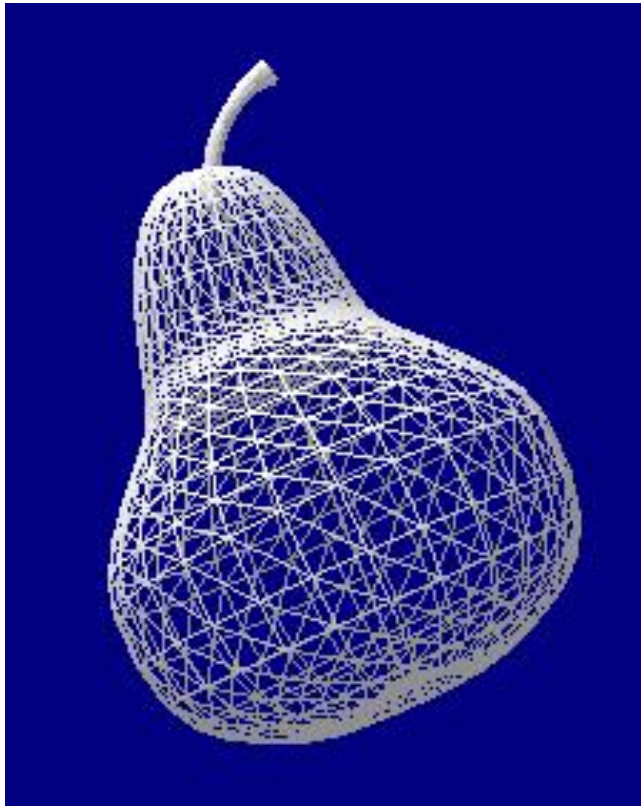
Rendering definition

- Collection of operations necessary to project a view of an object or a scene onto a view surface

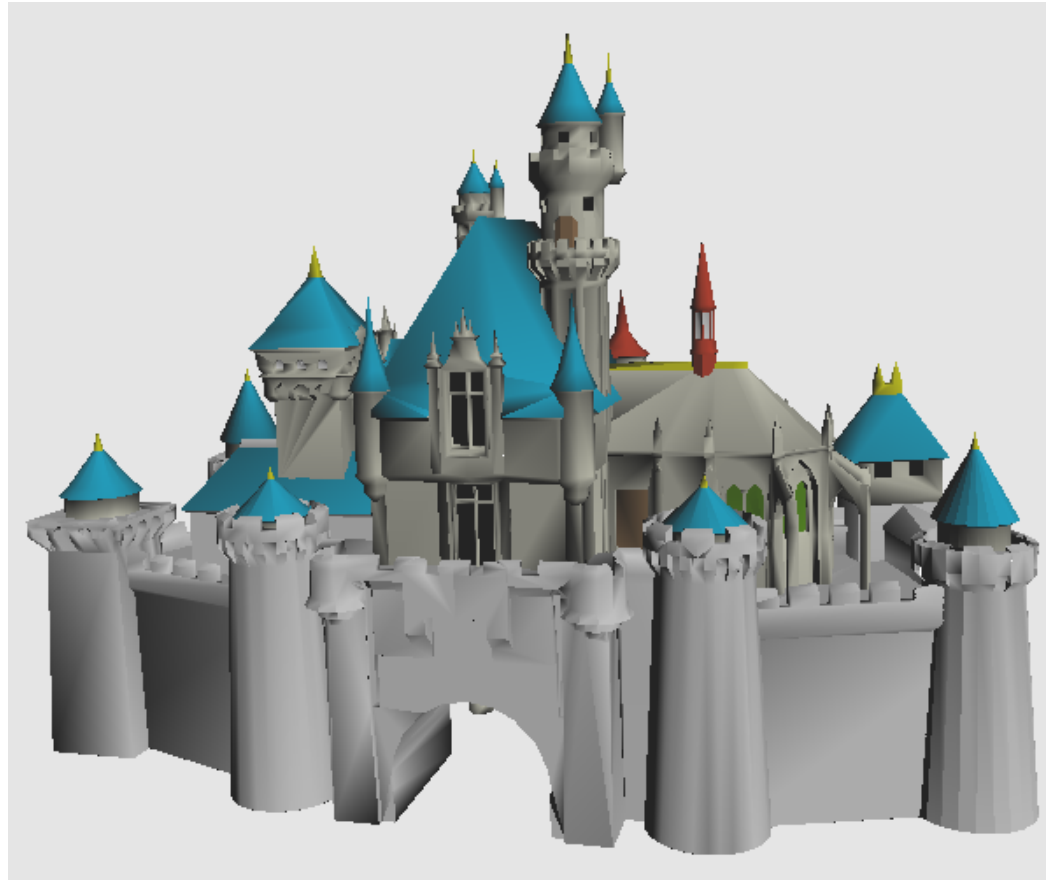
- Rendering strategies are determined by
 - Object representation
 - Algorithm options available for the internal processes

- Concerning on rendering:
 - Polygonal objects
 - Parametric patch net
 - CSG description

Rendering polygonal objects

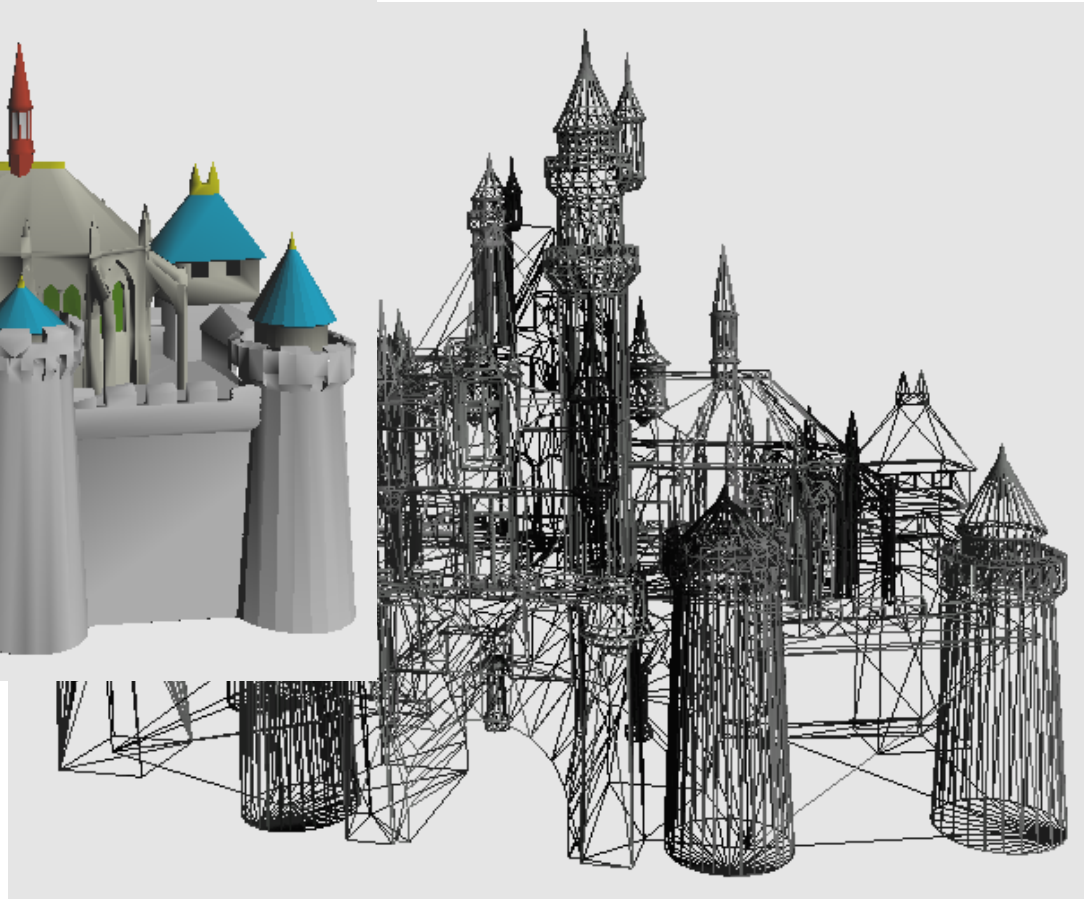


Rendering polygonal objects



Texture coordinates: 6620
Number of materials: 9
Number of groups: 17

Number of vertices: 6620
Number of triangles: 13114
Number of normals: 16710
Face normals: 13114



Rendering polygonal objects



Texture coordinates: 4028
Number of materials: 7
Number of groups: 8

Number of vertices: 4028
Number of triangles: 3360
Number of normals: 4050
Face normals: 3360

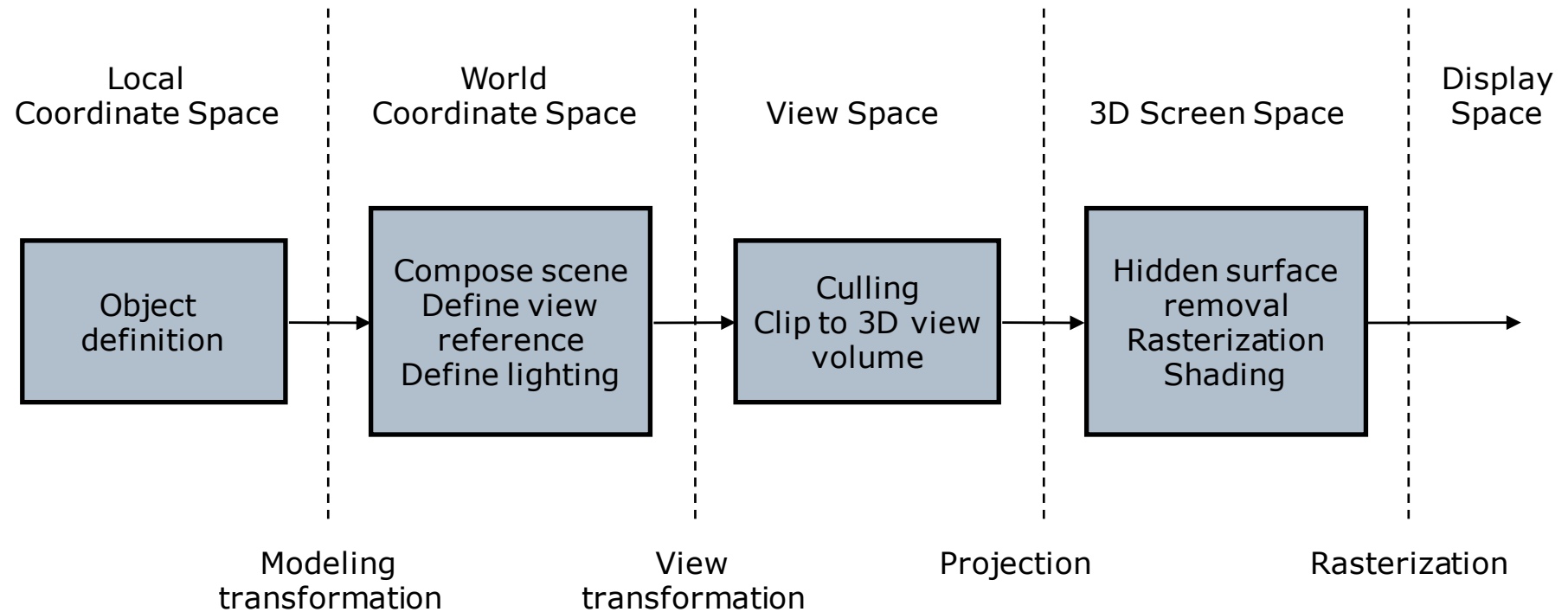


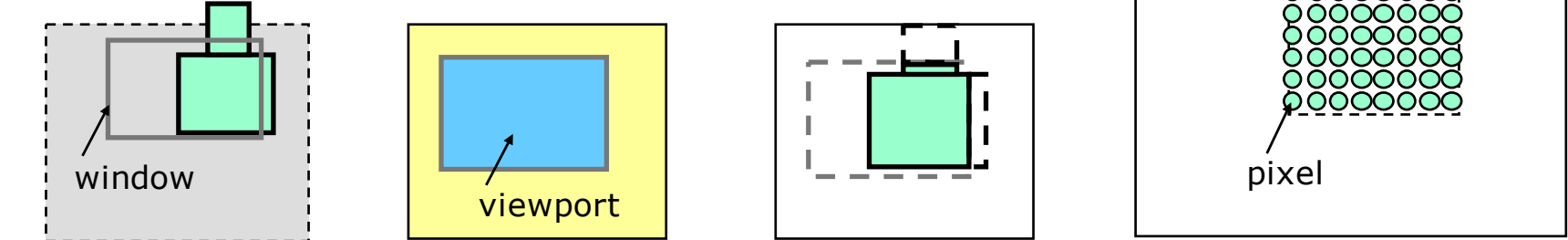
Polygonal renderer

- Polygonal objects are the most common representation

- Polygonal renderer
 - Software and hardware
 - Process a polygon as a single entity -> fast and simple processing
 - Input: list of polygons
 - Output: color for each pixel on the screen

3D Objects rendering pipeline





~~Scan conversion onto the raster screen~~

Rendering pipeline

1. Local coordinate space (modeling coordinate system)
 - Operations: Object definition
 - Object stored in a database using vertex coordinates
2. World coordinate space
 - Operations: 3D transformations, scene composition, view reference setting up, lights definition
3. View space (eye coordinate system)
 - Operations:
 - culling - removes entire polygons that cannot possibly be visible from the view point
 - 3D clipping – removes the parts of the scene outside of the view volume
4. 3D screen space
 - Simultaneously operations in the innermost pixel loop:
 - Hidden surface removal - removes invisible parts of polygons
 - Shading – compares the orientation of each polygon with the light source direction and assigns a shade to the interior of the polygon
 - Rasterization – allots a value to each pixel in the projection.
 - Anti-aliasing computes intermediate values for the overlapped projected parts of the polygons

Rendering pipeline operations

- Other operations
 - Texture mapping
 - Fixed mapping – local, world, view, and screen coordinate systems
 - Viewing dependent mapping – view and screen coordinate systems
 - Shadow shape computation
 - Depends just on relative positions of objects and light sources
 - Shadow volumes, shadow polygons
 - Local, world, and view coordinate systems
 - Move viewer's position
 - Changes view volume parameters (frustum parameters)
 - World coordinate systems
 - Object transformation
 - Of positions and dimensions in world and view coordinate systems
 - Of graphics attributes (e.g. colors, textures) in local, world, view, and screen coordinate systems

Rendering parametric patch net

- ❑ Preprocess the representation and convert the patches into planar polygons
- ❑ Original patch net provides higher accuracy representation
- ❑ Conversion is performed through a subdivision or splitting process
- ❑ Polygon size depends on the local curvature of the patch. In general the number of polygons of the converted model is greater than the number of patches in the parametric model
- ❑ Conversion provides a unification with polygon mesh rendering
- ❑ Steps of the conversion process:
 1. Basis conversion – convert the patch basis to Bézier form by matrix multiplication
 2. Patch splitting
 - ❑ subdivide by isoparametric curves ($u=0.5$, $v=0.5$) until a flatness criterion is satisfied (e.g. distance between a plane through the corner points of the patch and the patch surface)
 - ❑ De Casteljau algorithm

De Casteljau algorithm

$$R_0 = Q_0$$

$$R_1 = (Q_0 + Q_1)/2$$

$$R_2 = R_1/2 + (Q_1 + Q_2)/4$$

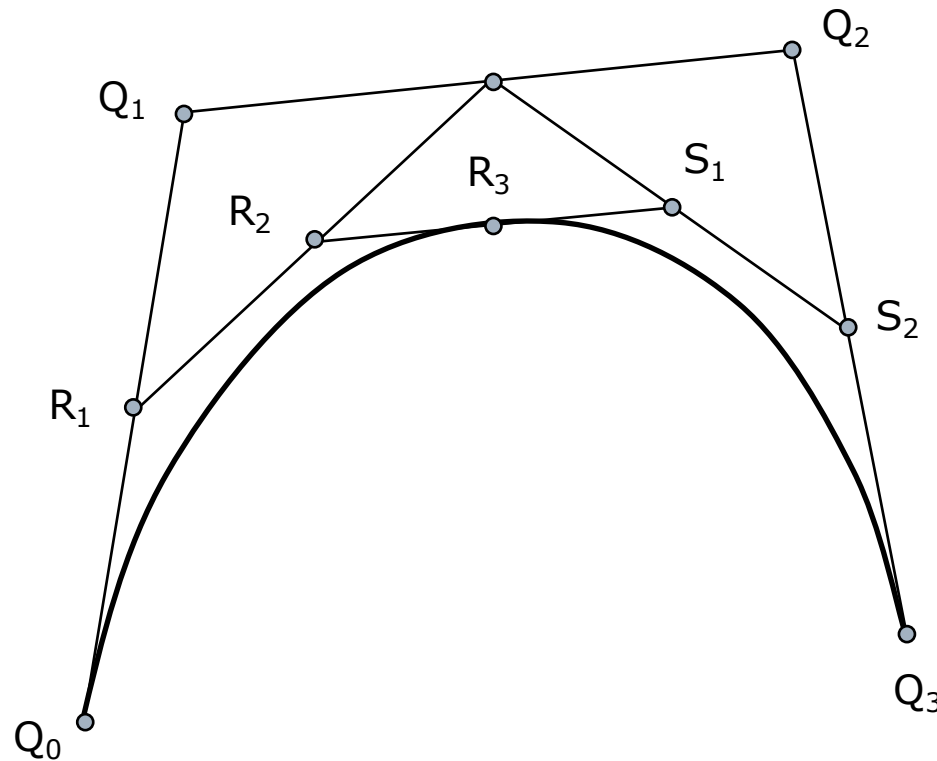
$$R_3 = (R_2 + S_1)/2$$

$$S_0 = R_3$$

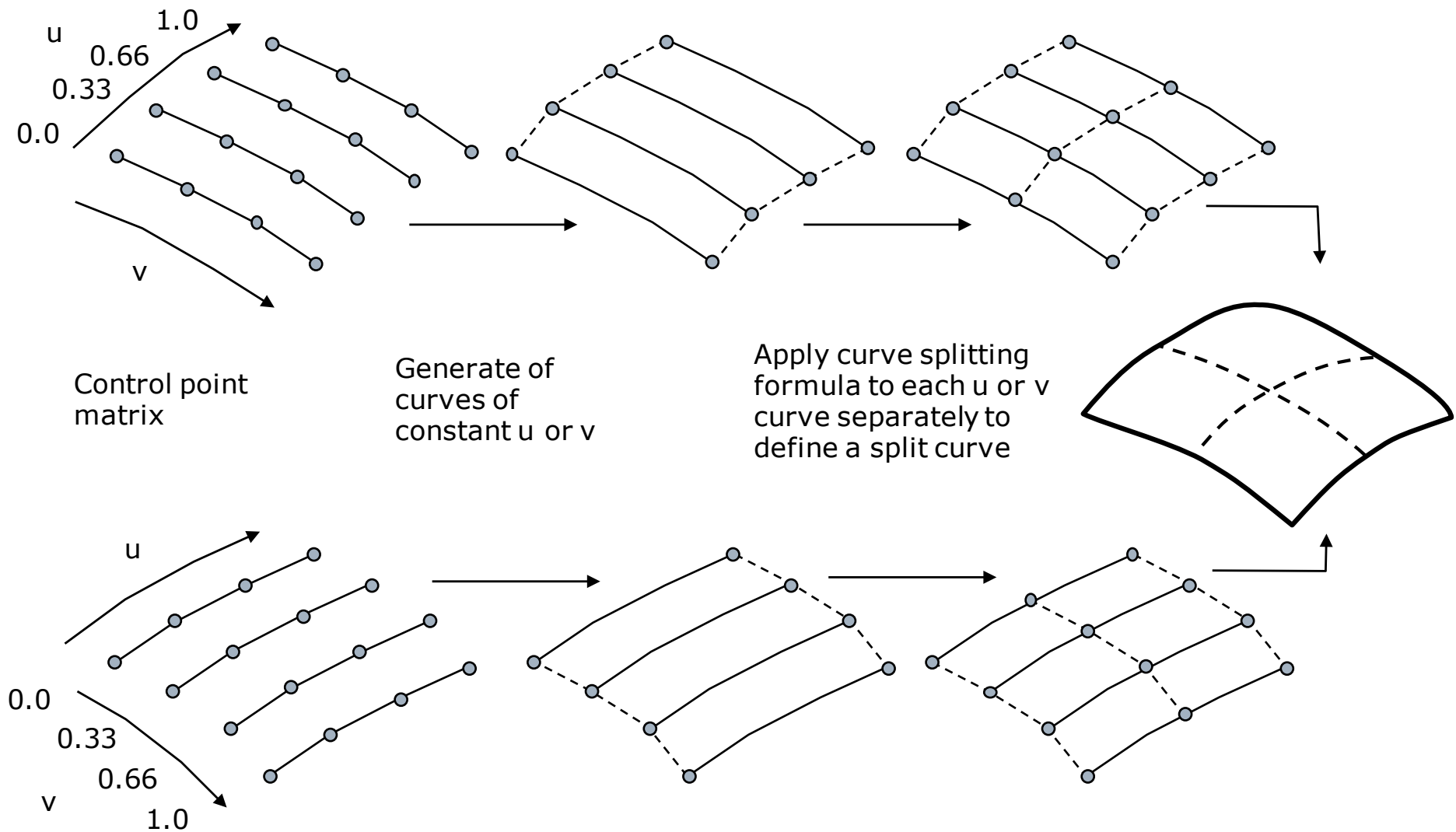
$$S_1 = (Q_1 + Q_2)/4 + S_2/2$$

$$S_2 = (Q_2 + Q_3)/2$$

$$S_3 = Q_3$$



Patch subdivision



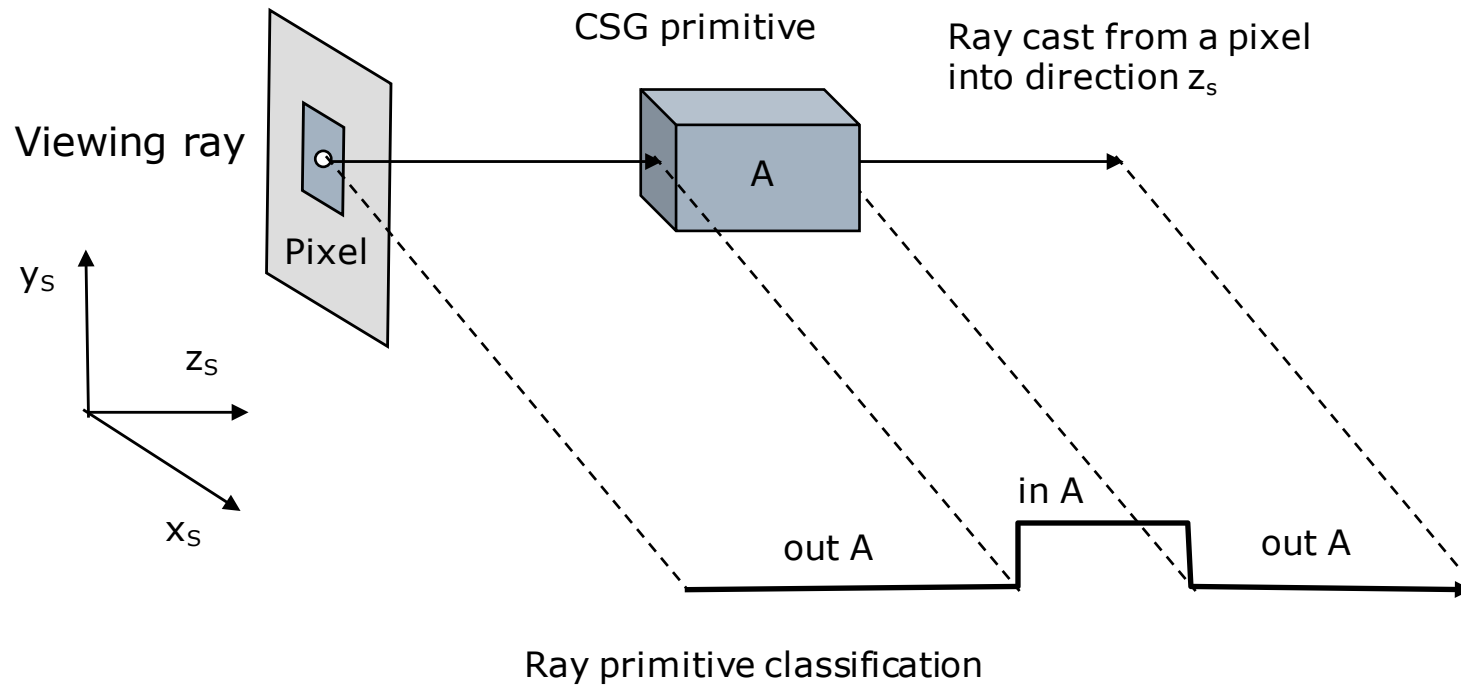
Rendering a CSG description

- ❑ Object database is a tree that relates a set of primitive objects to each other through boolean operations
- ❑ It is not a boundary representation (that divides the 3D space into internal and external regions of the object)
- ❑ There are 3 techniques to derive boundary representation from CSG:
 1. CSG ray tracing
 2. Conversion to a voxel representation followed by volume rendering
 3. Using a version of the Z-Buffer algorithm

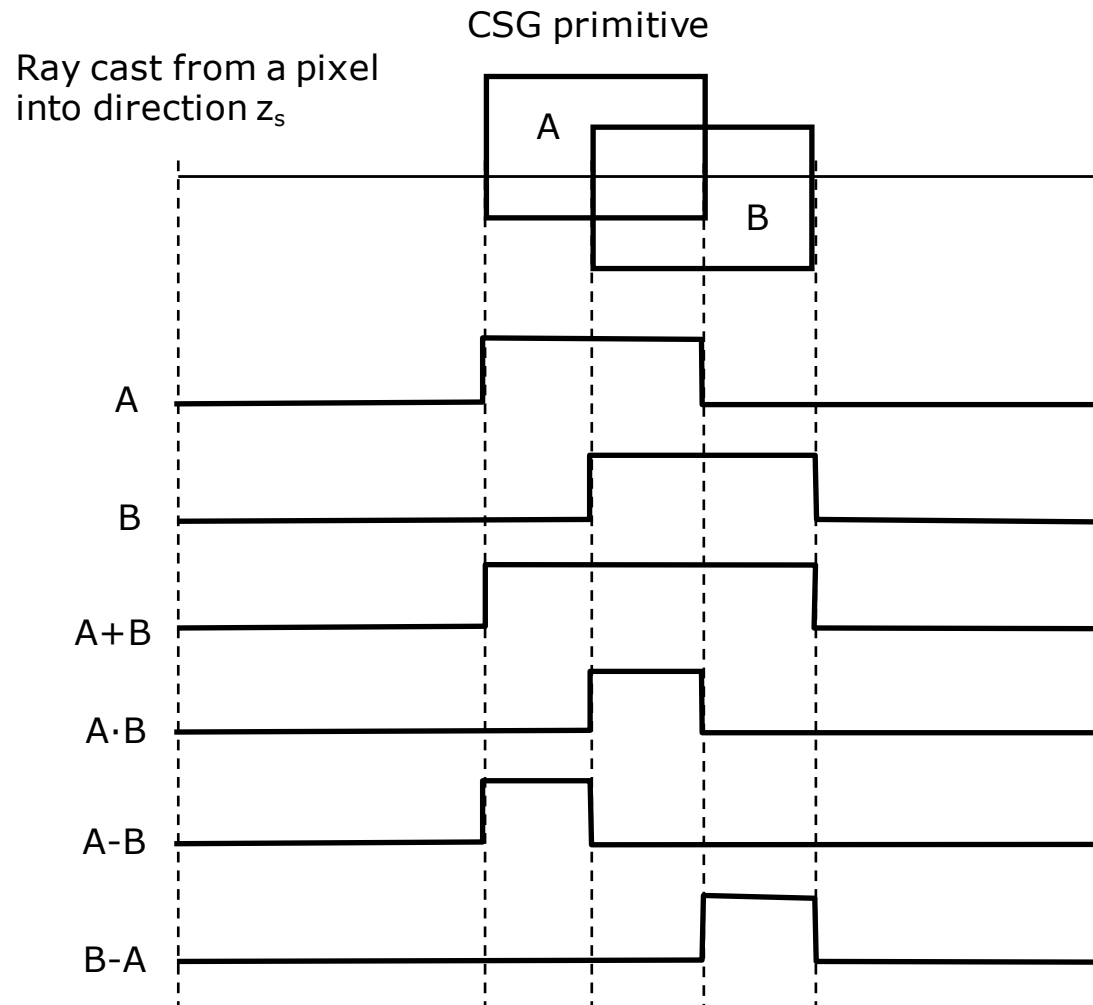
CSG ray tracing

- ❑ Reduce the problem to one dimension
- ❑ Cast a ray to each pixel in the view plane
- ❑ In the simplest case (parallel projection) explore the space of the object with a set of parallel rays
- ❑ Two stages of the process:
 1. Consider a single ray.
 - ❑ Every primitive instance is compared against this ray to see if it intersects the primitive
 - ❑ Any intersections are sorted in z depth. There is a ray/primitive classification for each ray
 - ❑ Analyze any boolean combination between the first two primitives encountered along the ray
 2. Allocate a shading value to the pixel by a simple reflection model applied at the first intersection along the ray
- ❑ All operations take place in screen space, in parallel (orthogonal) projection

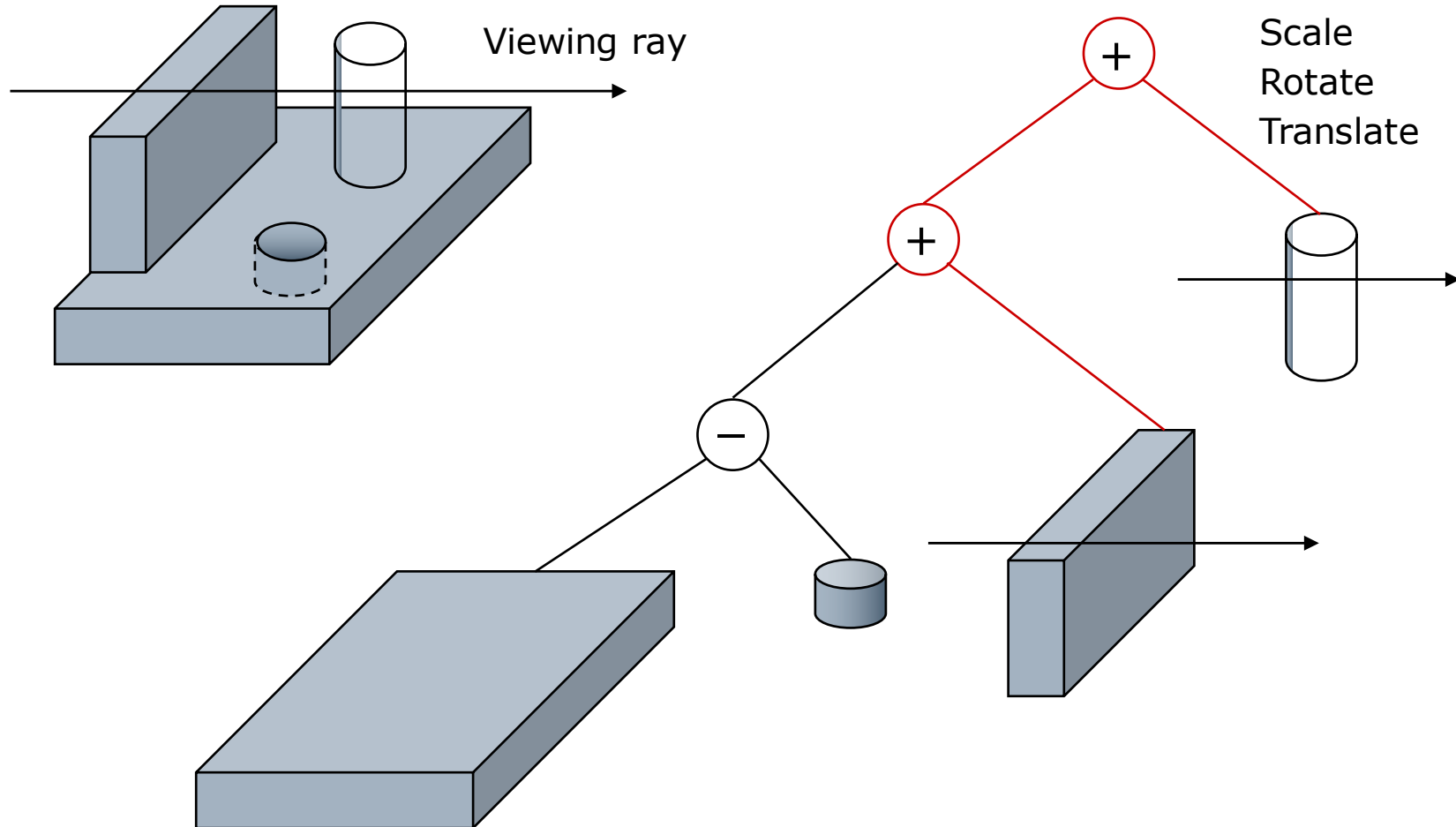
CSG ray tracing – primitive classification



CSG ray tracing – boolean operations



Constructive Solid Geometry (CSG)



Rendering a CSG description

- Ray tracing is the classical method that integrates into a single model:
 - a. Evaluation of a boundary representation from a CSG description
 - For each view ray
 - For all CSG primitives compute the ray primitive
 - Compute the global ray primitive from all the correspondent CSG ray primitives according with the CSG tree
 - b. Hidden surface removal – only intersection with the first primitive nearest to the viewer is considered
 - c. Shading

Questions and problems

1. Why the computation of light intensity and shadows can be completed either in World or View coordinate spaces?
2. Why the determination of the hidden lines and surfaces cannot be achieved in the World coordinate space?
3. Why the rendering of parametric patch needs splitting?
4. How do you compute in the CSG ray tracing approach the visible point of an elementary graphics object component (e.g. cube or cylinder)?
5. How do you compute in the CSG ray tracing approach the visible point of the complex graphics object (e.g. made of basic geometric components)?
6. Explain the composition of a complex 3D object by the CSG binary tree, using the information associated to nodes and arcs. What is this information?

Polygonal Objects Rendering (1)

Contents

- Basic renderer
- Rendering pipeline
- Local coordinates system
- World coordinates system
- View coordinates system
 - View transformation
 - View system specification in GUI
 - Back face culling
- Screen coordinate system
 - Perspective transformation
 - Orthogonal transformation
 - View volume

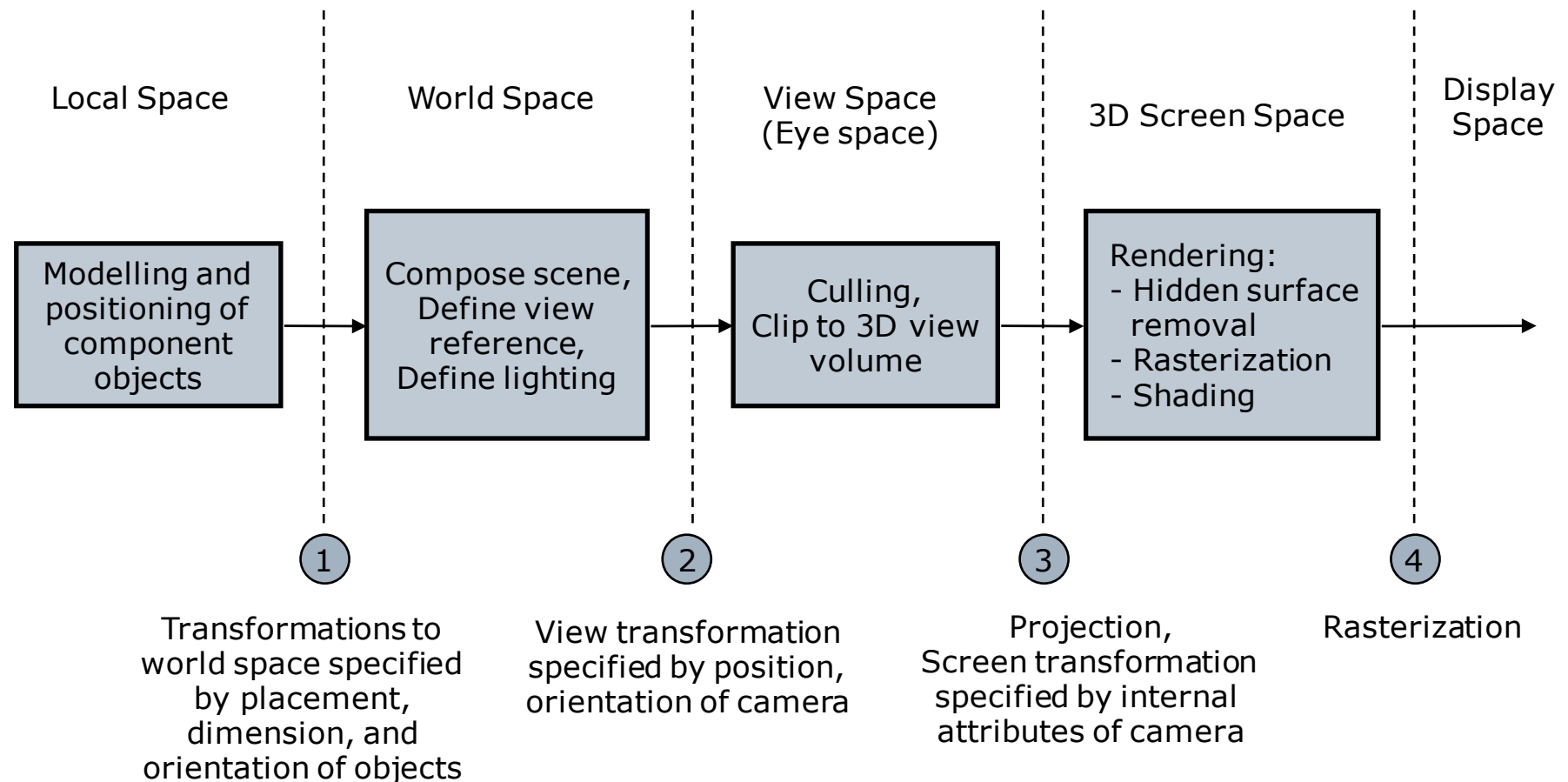
Rendering sequence

- **Basic renderer** - the traditional approach to render 3D objects.
- Polygonal renderer
 - Software and hardware
 - Process a polygon as a single entity -> fast and simple processing
 - Input: list of polygons
 - Output: color for each pixel on the screen
- The renderer incorporates a local reflection model (e.g. Phong model) into a Phong incremental shader.

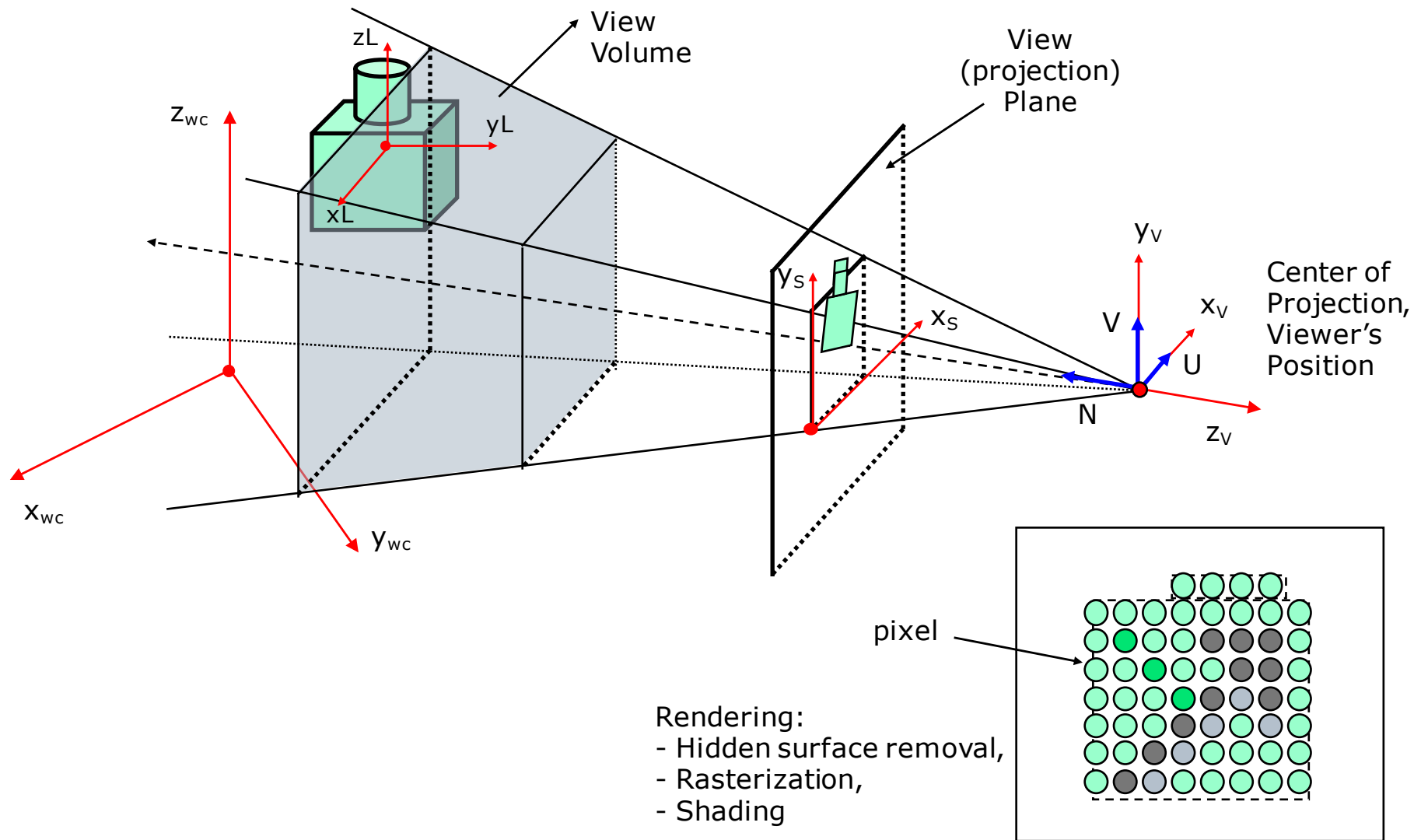
Polygonal modeling

- Advantages of the polygonal representation:
 - Modelling objects using polygons is straightforward, simple but tedious.
 - Geometric information is only stored at the polygon vertices.
 - A polygonal based renderer can be used to render objects that are made up of a set of bicubic patches.
- Disadvantages:
 - It is difficult to map textures from a 2D domain onto the surface of an object
 - Shadow algorithms that are based on polygonal objects generally have high coding complexity and produce hard-edge shadows.
 - Polygons are expensive. A complex object that is to be represented to a high level of detail can generate a very large number of polygons (i.e. 400 000 polygons).

3D Objects rendering pipeline

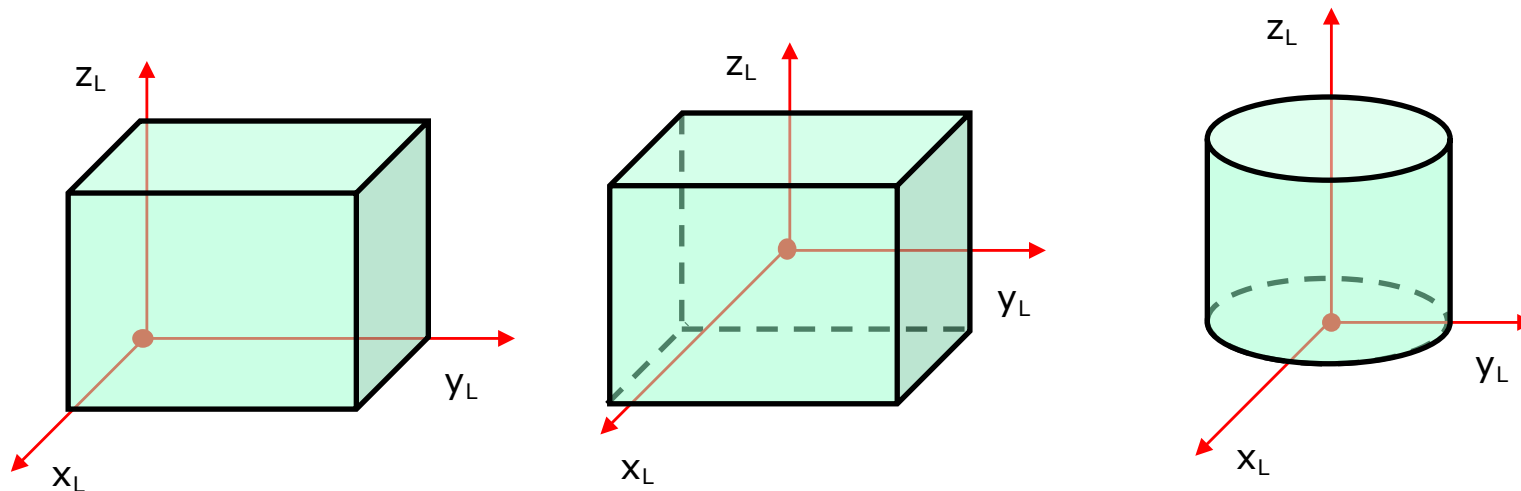


Conceptual model of 3D viewing process



Local coordinates system

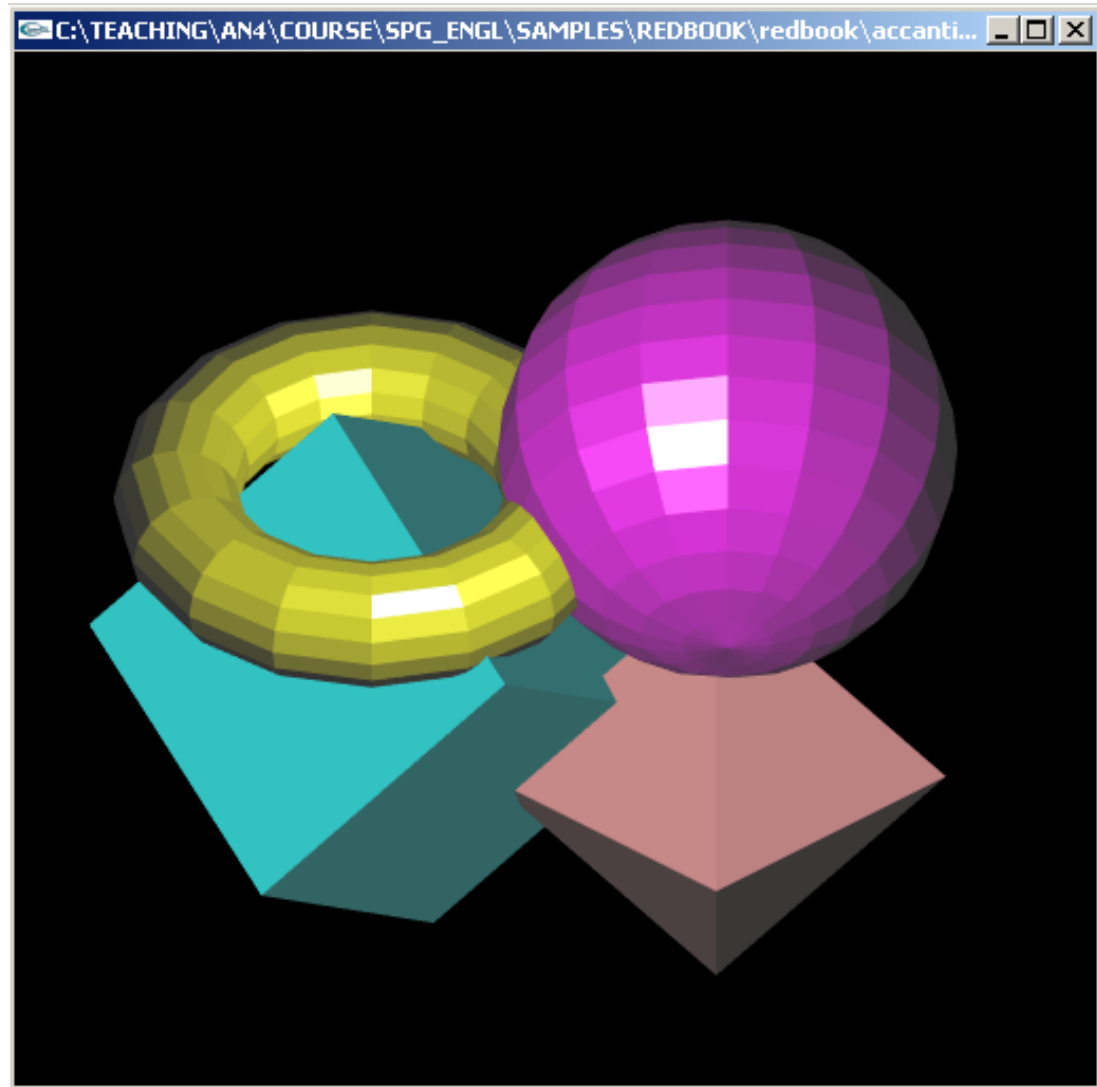
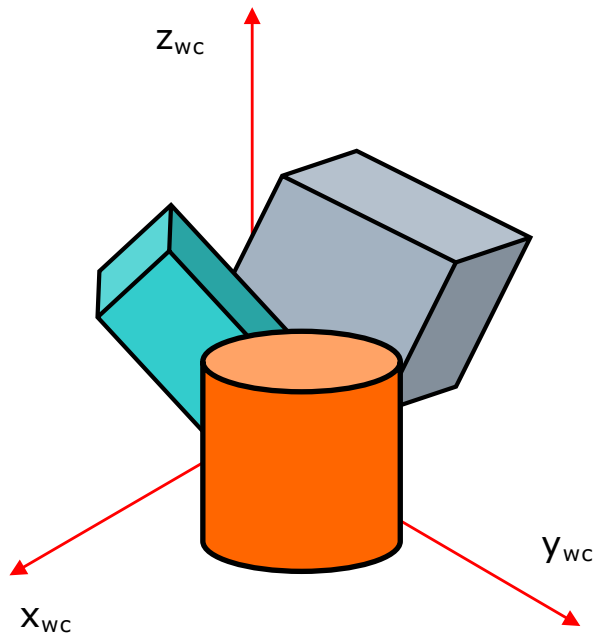
- The graphics model store the vertices of an object with respect to some point conveniently located in or near the object
- As well as storing the polygon vertices in a coordinate system that is local to the object, we would also store the polygon normals and the vertex normals.
- Example: in PHIGS the local coordinates system is called the **modelling coordinates system**



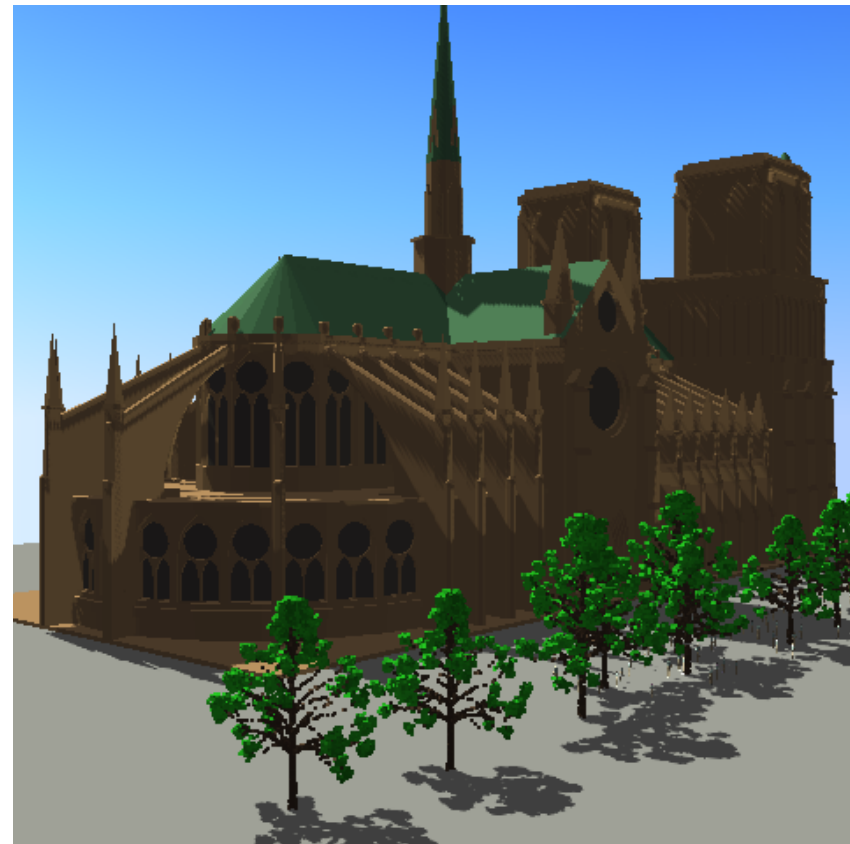
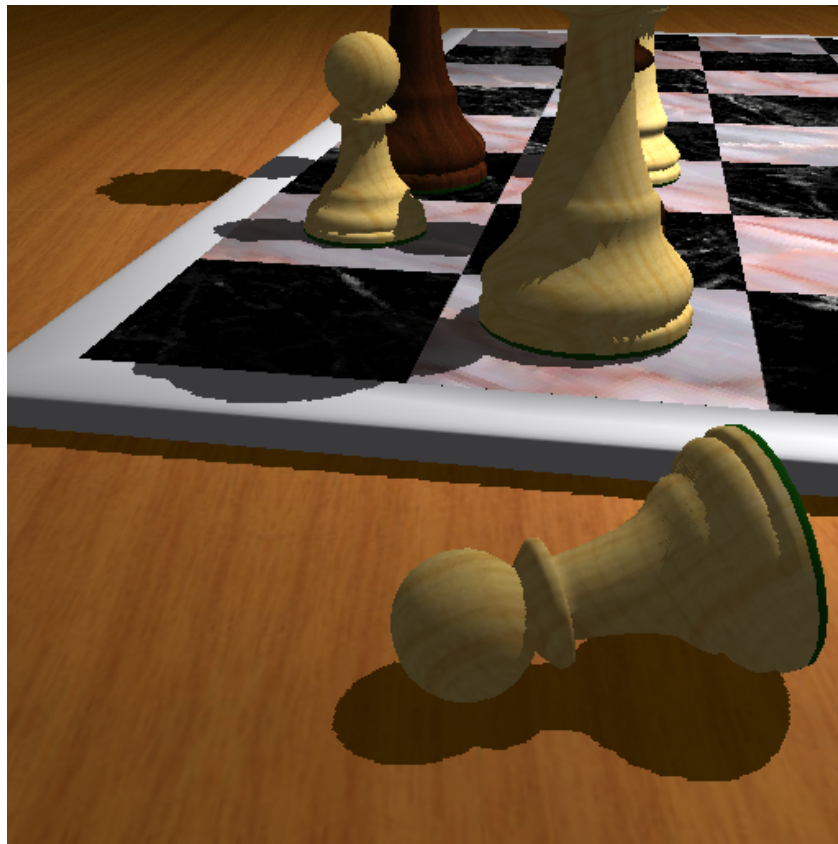
World coordinates system

- ❑ The objects are placed into a scene of objects
- ❑ The global coordinate system of the scene is known as the **world coordinate system**. The act placing an object in a scene defines the **transformation** required to take the object from local space to world space
- ❑ If this object is being animated then the system provides a time-varying transformation that takes the object into world space on a frame-by-frame basis
- ❑ **Light sources** are specified for the scene that is lit in world space.
- ❑ **Surface attributes** (i.e. texture, colour, etc) are specified in world space as well

World coordinates system



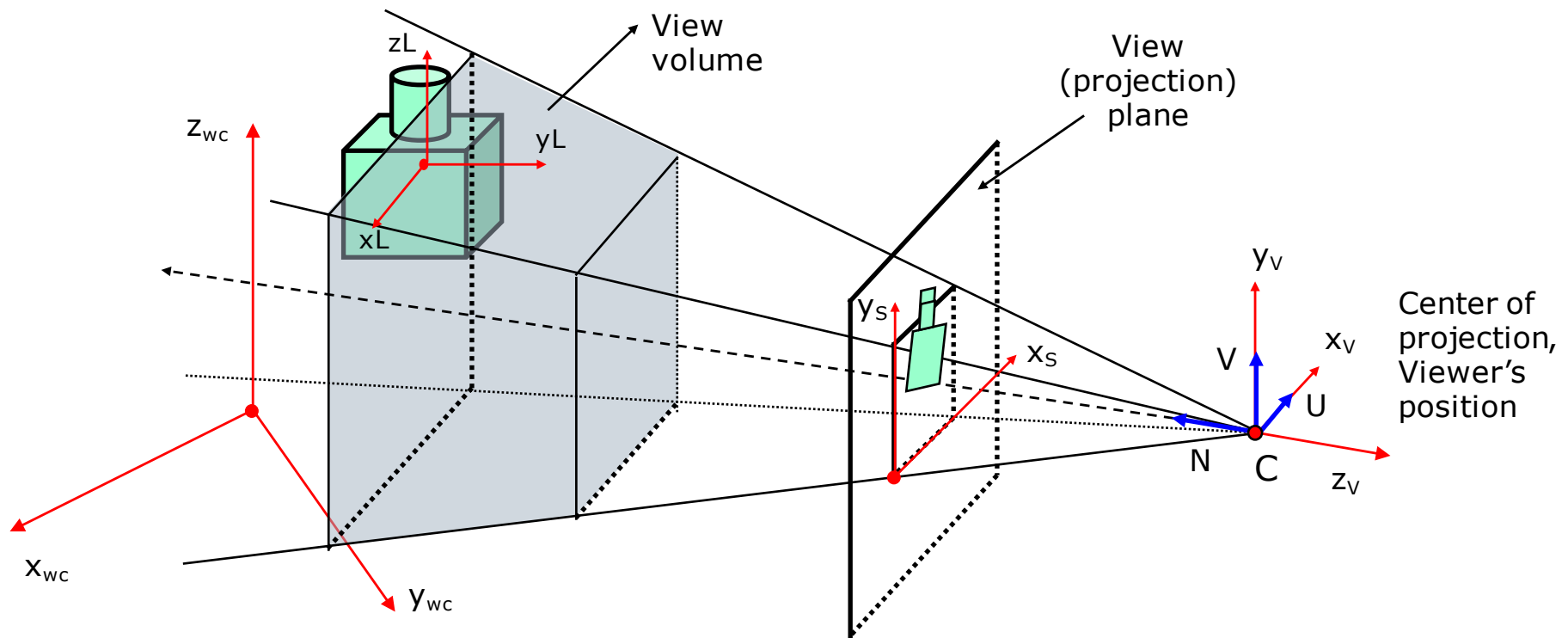
World coordinates system



View coordinates system

- Eye, view or camera coordinates system
- It is the space used to establish **viewing parameters** and a **view volume**. A **virtual camera** is often used as a conceptual aid in computer graphics
- The camera can be positioned anywhere in world space and point in any direction. The analogy of the film plane in computer graphics is the **view plane** - the plane in which the scene is projected
- Example: in PHIGS the viewing system is called "view reference coordinate system", or the **(U,V,N) system**

Conceptual model of 3D viewing process



View system definition

□ 3 vectors and a position:

1. **Camera position, C.** This is also the centre of projection.
2. **Viewing direction vector, N.** Along z axis, a vector normal to the view plane. In OpenGL the N vector gives the $-z$ axis.
3. **"Up" vector, V** - orients the camera about the view direction and establish with N the orientation of the view plane window within the view plane. The V vector gives the $+y$ axis.
4. **Optional vector, U** - denotes the direction of increasing x in the eye coordinate system. This establish a right ($Z = X \times Y$) or left ($-Z = X \times Y$, or $Z = Y \times X$) handed coordinate system.

In OpenGL:

- The view vector system is left handed $U = N \times V$, $N = V \times U$
- N vector gives the $-z$ direction, and is oriented toward the scene

Transformation from world to view system

- The transformation is given by the following equation:

$$\begin{bmatrix} x_v & y_v & z_v & 1 \end{bmatrix} = \begin{bmatrix} x_w & y_w & z_w & 1 \end{bmatrix} * H$$

where $H = T * R$, T is the translation to the C centred system, and R is a complex rotation which expresses the rotation into the view system

$$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -x_c & -y_c & -z_c & 1 \end{bmatrix}$$

R maps U , V and N to unit and orthogonal vectors:

$$U \rightarrow (1, 0, 0)$$

$$V \rightarrow (0, 1, 0)$$

$$N \rightarrow (0, 0, -1)$$

Transformation from world to view system

or U, V, and N in the matrix form are transformed to unit vector:

$$\begin{bmatrix} U_x & U_y & U_z & 1 \end{bmatrix} * R = \begin{bmatrix} 1 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} V_x & V_y & V_z & 1 \end{bmatrix} * R = \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} N_x & N_y & N_z & 1 \end{bmatrix} * R = \begin{bmatrix} 0 & 0 & -1 & 1 \end{bmatrix}$$

U, V and N are unit and orthogonal vectors:

$$U_x^2 + U_y^2 + U_z^2 = 1$$

$$U_x V_x + U_y V_y + U_z V_z = 0$$

$$V_x^2 + V_y^2 + V_z^2 = 1$$

$$N_x V_x + N_y V_y + N_z V_z = 0$$

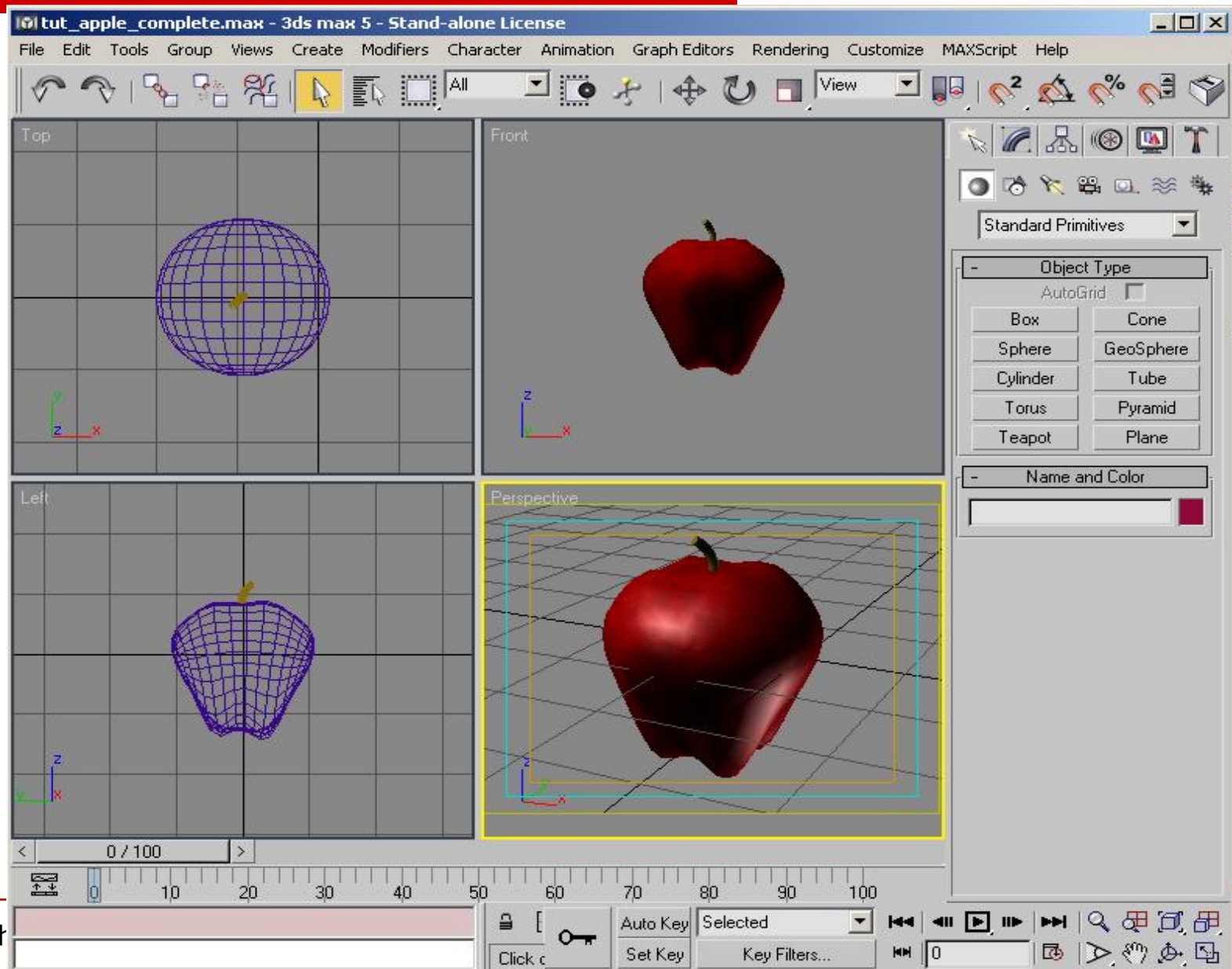
$$N_x^2 + N_y^2 + N_z^2 = 1$$

$$U_x N_x + U_y N_y + U_z N_z = 0$$

Therefore:

$$R = \begin{bmatrix} U_x & V_x & -N_x & 0 \\ U_y & V_y & -N_y & 0 \\ U_z & V_z & -N_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

View system specification in GUI



Graph

View system specification in GUI

- ❑ Objective: to control the viewer's position and viewing parameters in 3D applications.
- ❑ Viewing parameters C, U, V and N
- ❑ Position $C(x_C, y_C, z_C)$

x, y (2D screen space) $\rightarrow C, U, V, N$ (3D space)

- ❑ Vector N definition:

$$N_x = \sin \beta \cos \alpha$$

$$N_y = \sin \beta \sin \alpha$$

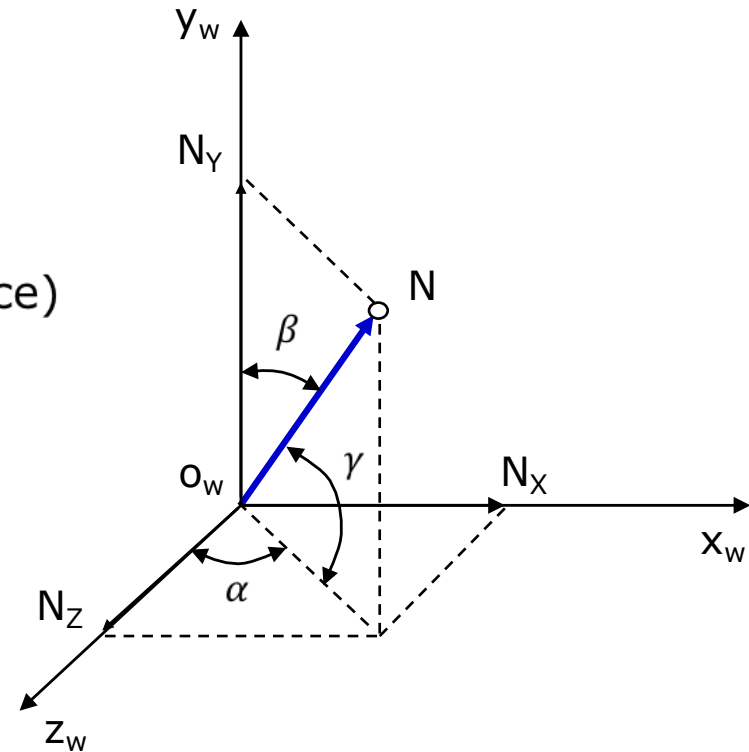
$$N_z = \cos \beta$$

- ❑ or as function of γ

$$N_x = \cos \gamma \cos \alpha$$

$$N_y = \cos \gamma \sin \alpha$$

$$N_z = \sin \gamma$$



View system specification in GUI

□ Vector V definition:

1. V must be perpendicular to N. We have to determine the intersection of the (N, V') plane and the viewing plane. The planes are perpendicular.
2. Draw the vector $V'' = V' \times N$. V'' is in the projection plane and perpendicular on the (N, V') plane.
3. Finally V is the normalized cross product of N and V'', $V^* = N \times V''$.

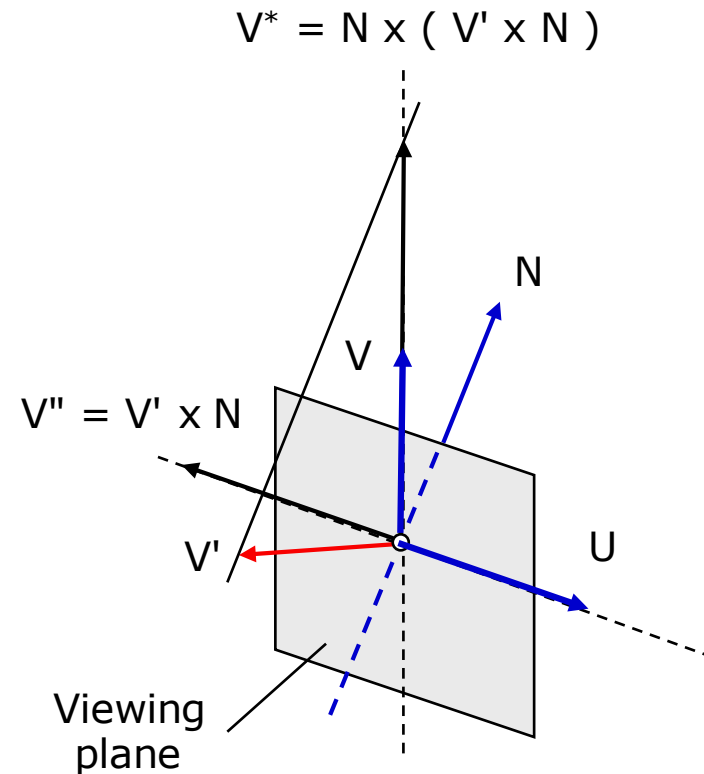
□ Conclusion:

V is the normalized vector

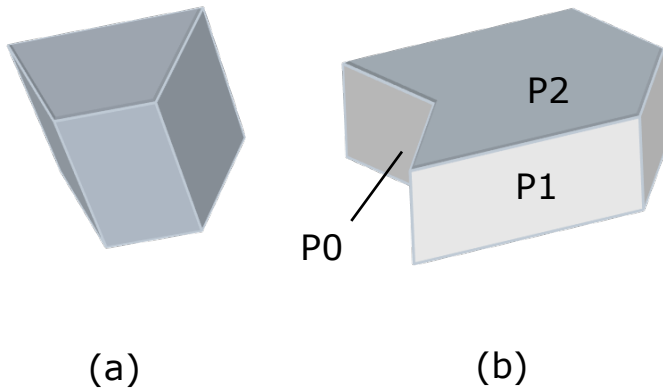
$$V^* = N \times (V' \times N)$$

□ The vector U is the cross product

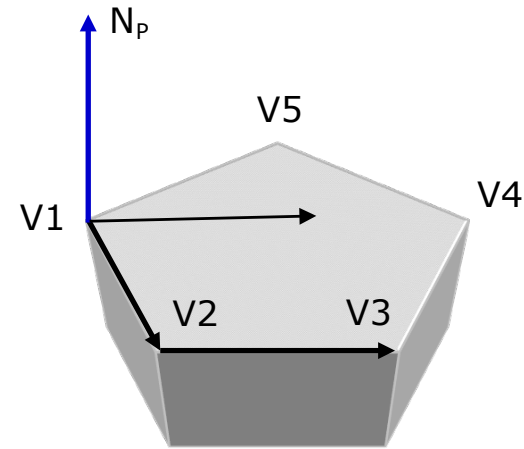
$$U = N \times V.$$



Back face culling



(a) Convex, and (b) concave objects



The computation of the polygon normal vector

Back face culling – visibility test

$$\mathbf{N_p} \cdot \mathbf{L} = |\mathbf{N_p}| \cdot |\mathbf{L}| \cos \varphi$$

There are 3 visibility cases:

(a) Visible polygon face

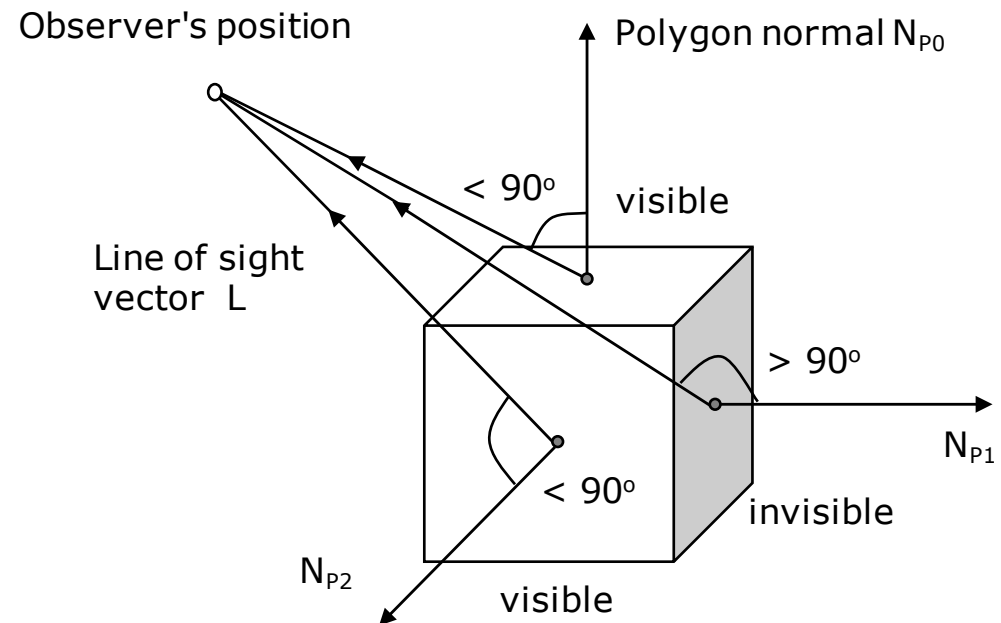
$$\varphi < 90^\circ, \quad \cos \varphi > 0 \rightarrow \mathbf{N_p} \cdot \mathbf{L} > 0$$

(b) Observer's line

$$\varphi = 90^\circ, \quad \cos \varphi = 0 \rightarrow \mathbf{N_p} \cdot \mathbf{L} = 0$$

(c) Invisible polygon face

$$\varphi > 90^\circ, \quad \cos \varphi < 0 \rightarrow \mathbf{N_p} \cdot \mathbf{L} < 0$$

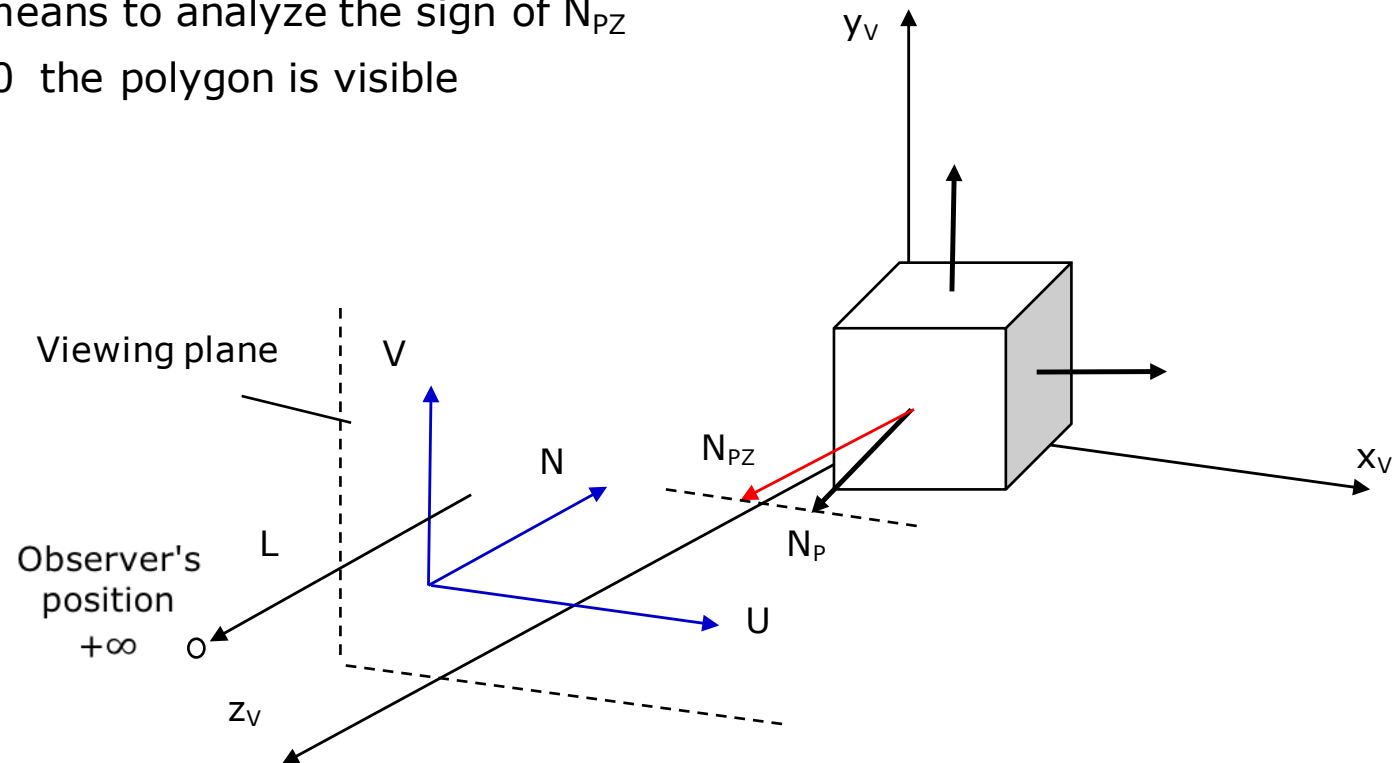


Computings:

1. Dot product: $\mathbf{N_p} \cdot \mathbf{L} = N_{pX} L_X + N_{pY} L_Y + N_{pZ} L_Z$
2. Line of sight vector L
3. Polygon normal vector: $\mathbf{N_p} = [V_1 \ V_2] \times [V_2 \ V_3]$

Accelerating the visibility test

- Perspective (view space) and orthogonal projection (screen space)
- Observer's position at infinity on the z_v axis
- The line of sight L is the opposite vector of N
 $N = [0 \ 0 \ -1]$, $V = [0 \ 1 \ 0]$, $U = [1 \ 0 \ 0]$
- Visibility test means to analyze the sign of N_{pz}
If $N_{pz} > 0$ the polygon is visible

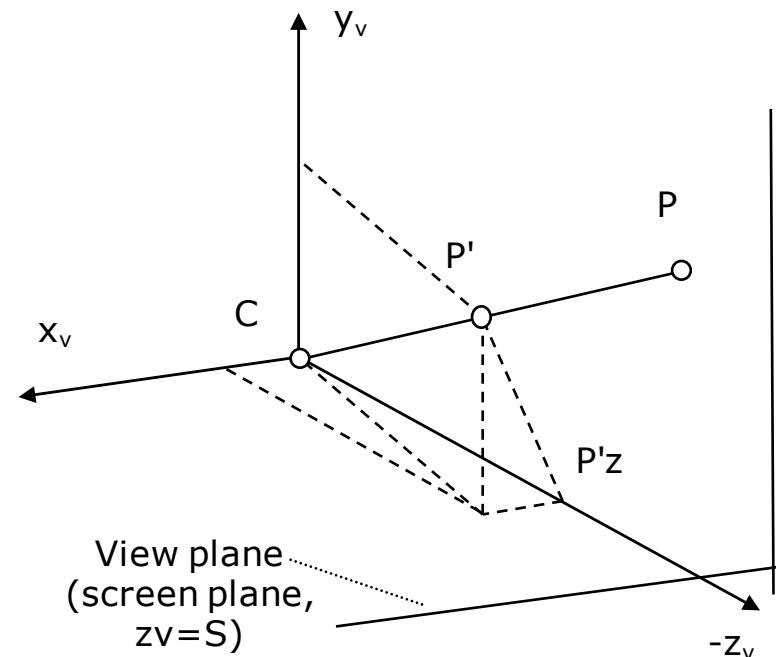
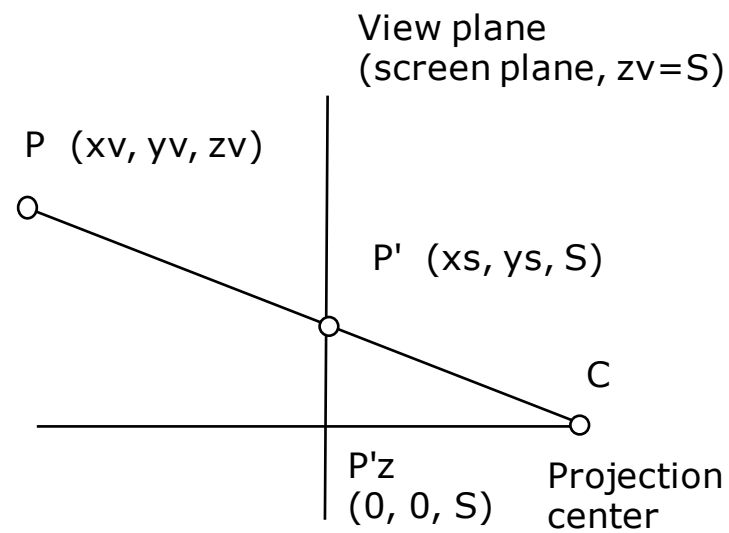


Screen space

- ❑ Screen space is a 3D space
- ❑ The value of x_s and y_s coordinates in the screen space are equal to the coordinate values in the view plane and not normalized screen coordinate values
- ❑ The z coordinate is not constant. It keeps the depth information to provide depth based computation in the screen space
- ❑ For simplicity, we use the expression of screen coordinate instead of coordinate in the screen space

Perspective projection to screen plane

- Screen coordinates: $x_s = S \cdot \frac{x_v}{z_v}$
- $y_s = S \cdot \frac{y_v}{z_v}$



Perspective transformation

To express the non linear transformation

$$xs = S \cdot \frac{xv}{zv}$$
$$ys = S \cdot \frac{yv}{zv}$$

as a 4x4 matrix consider the [homogeneous coordinates](#) and the transformation in two parts: a linear part (1°), followed by a non-linear part (2°):

1° [Linear transformation](#): $X=xv$, $Y=yv$, $Z=zv$, $w=zv/S$

The perspective transformation can be written as: $\begin{bmatrix} X & Y & Z & w \end{bmatrix} = \begin{bmatrix} xv & yv & zv & 1 \end{bmatrix} T_{\text{pers}}$

$$T_{\text{pers}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1/S \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

2° [Non-linear transformation](#): perspective division by w .

The final screen coordinates are: $xs=X/w$, $ys=Y/w$, $zs=Z/w$

Orthogonal transformation

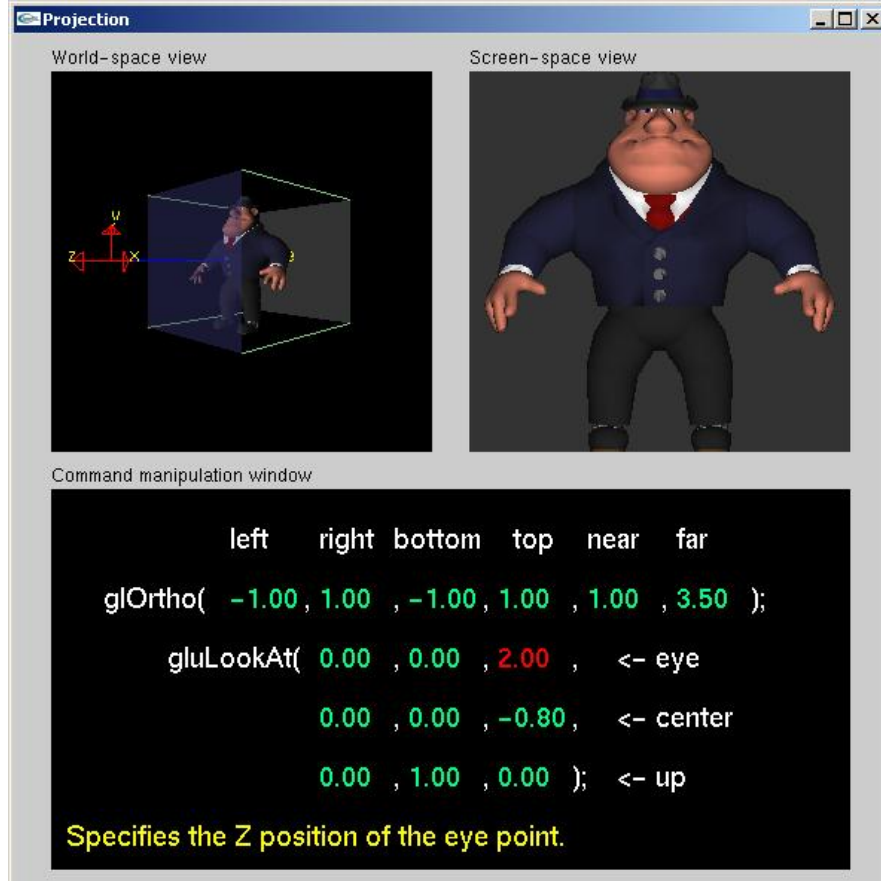
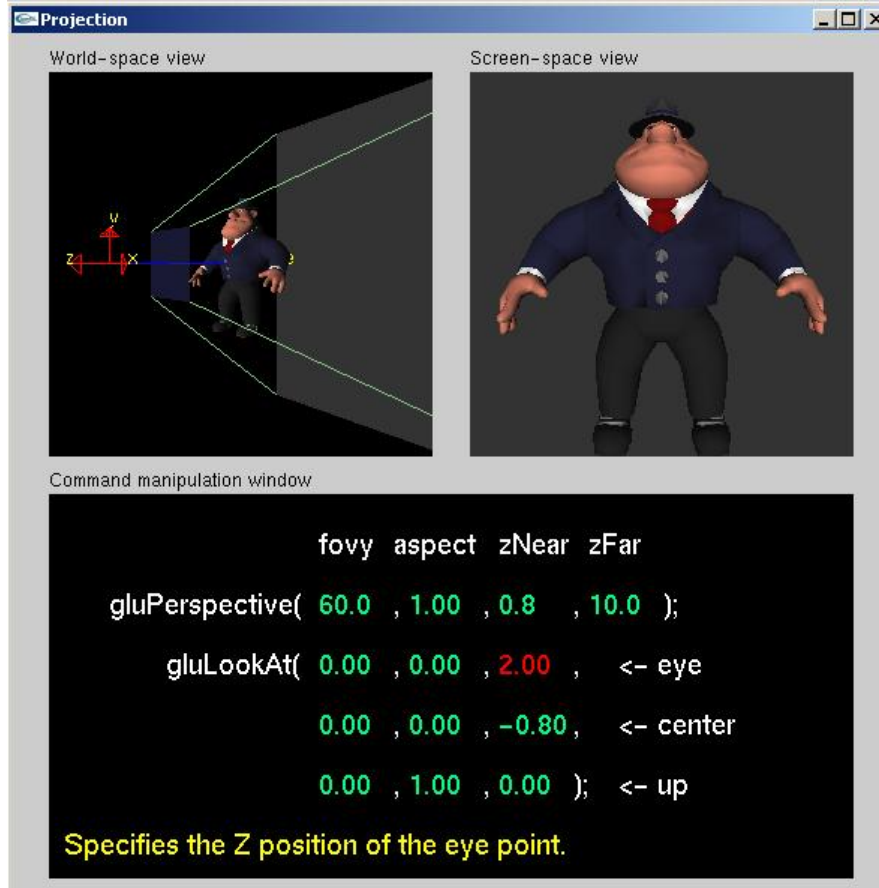
- Parallel projection
- The view plane is normal to the direction of projection

$$x_s = x_v, \quad y_s = y_v, \quad z_s = 0$$

and the orthogonal transformation is:

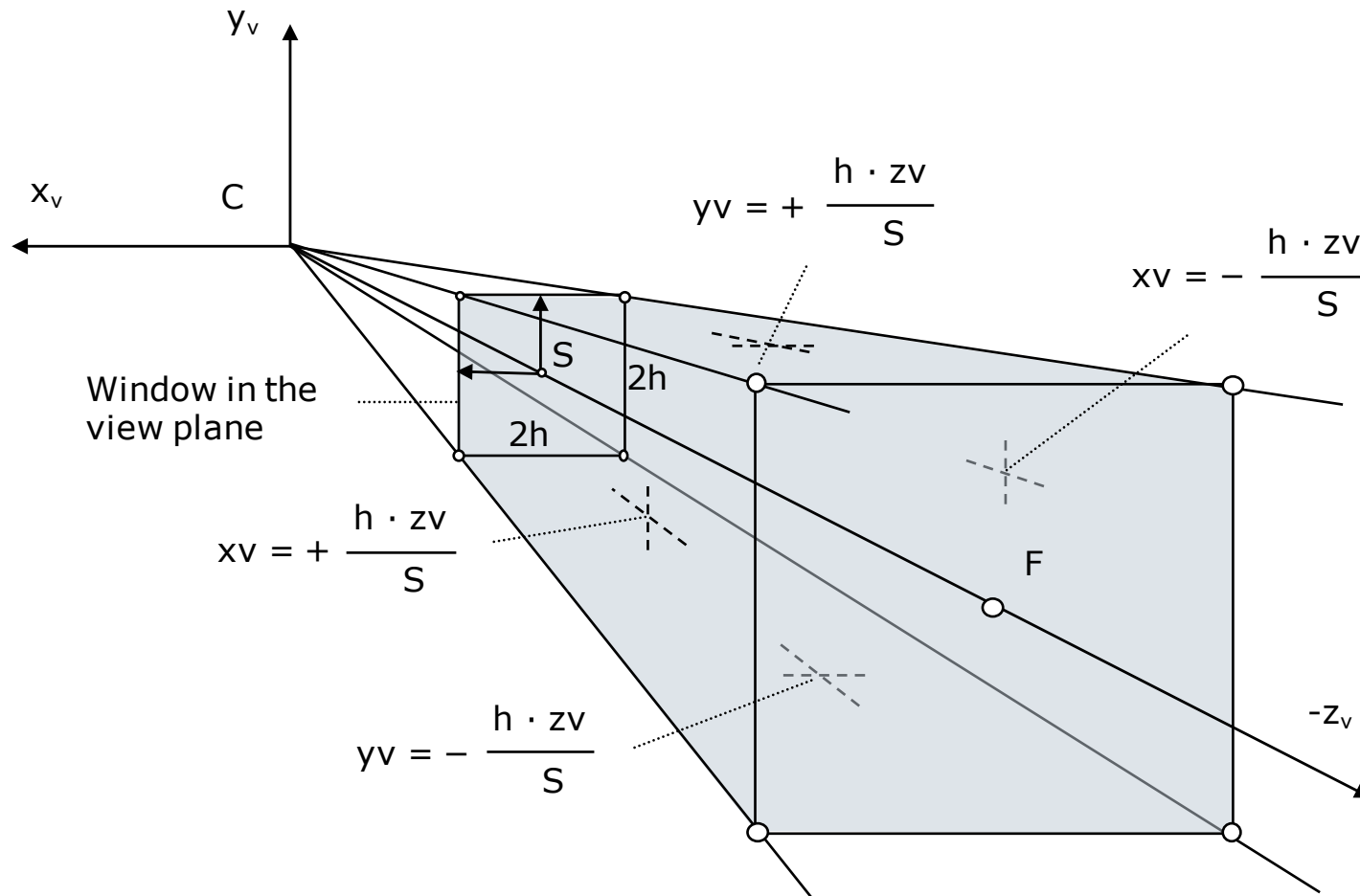
$$T_{\text{orth}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Perspective and orthogonal transformation



View volume

- Screen space is defined inside the view volume (viewing frustum)



View volume

- View volume definition: 6 planes

$$z_v = S$$

$$z_v = F$$

$$x_v = \mp \frac{h \cdot z_v}{S}$$

$$y_v = \mp \frac{h \cdot z_v}{S}$$

- $2h$ is the view plane window dimension
- S is the view plane distance, the virtual screen plane distance, and the near plane distance
- F is the far plane distance
- S and F are application based parameters

Questions and problems

1. What are the main advantages and disadvantages of the polygonal models?
2. What phases and coordinate system spaces, along the graphics pipeline, are able to support the following operation:
 1. Shadow computation;
 2. 3D clipping;
 3. Texture mapping;
 4. 2D translation;
 5. 3D rotation?
3. Exemplify and explain the transformation from world to view system.
4. Explain why is difficult to specify in GUI the viewing direction for the viewer.

Questions and problems

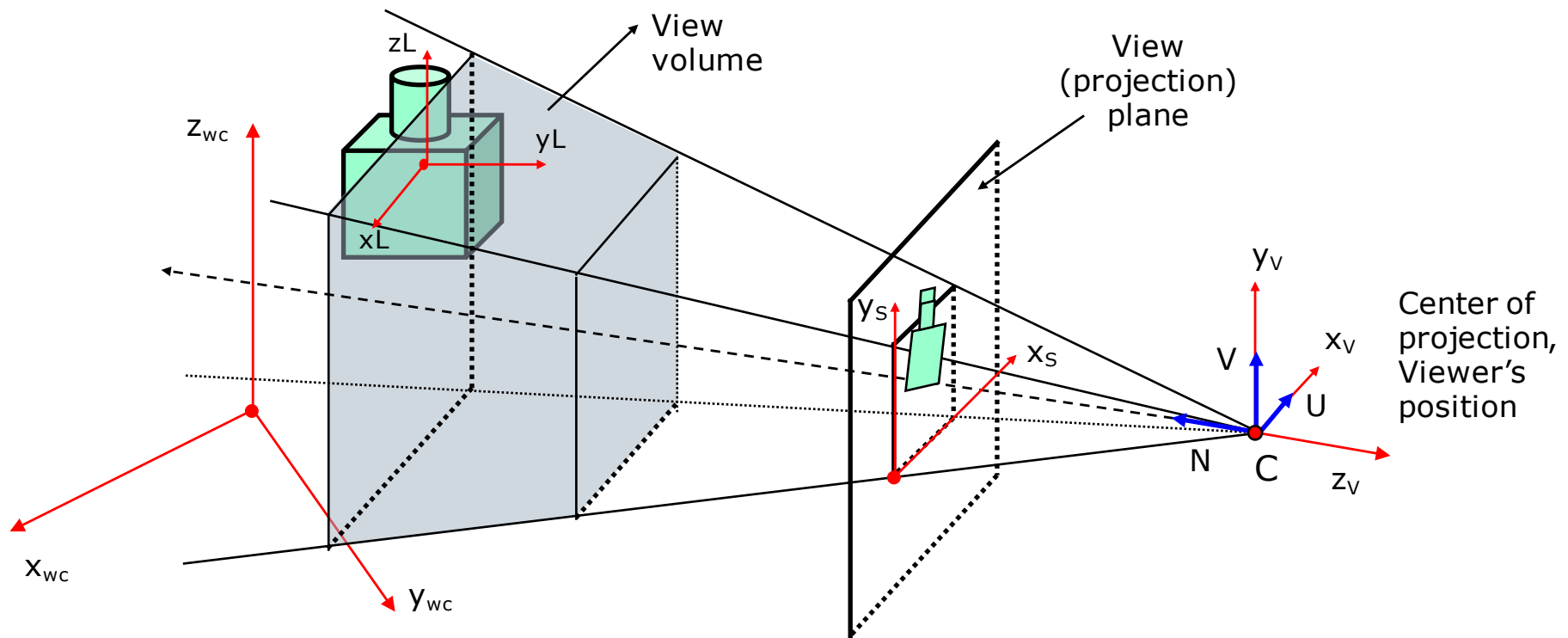
5. Explain why in orthogonal projection the visibility test for a polygonal phase is reduced to the analyzing of N_z (the z component of the normal vector)?

Polygonal Objects Rendering (2)

Contents

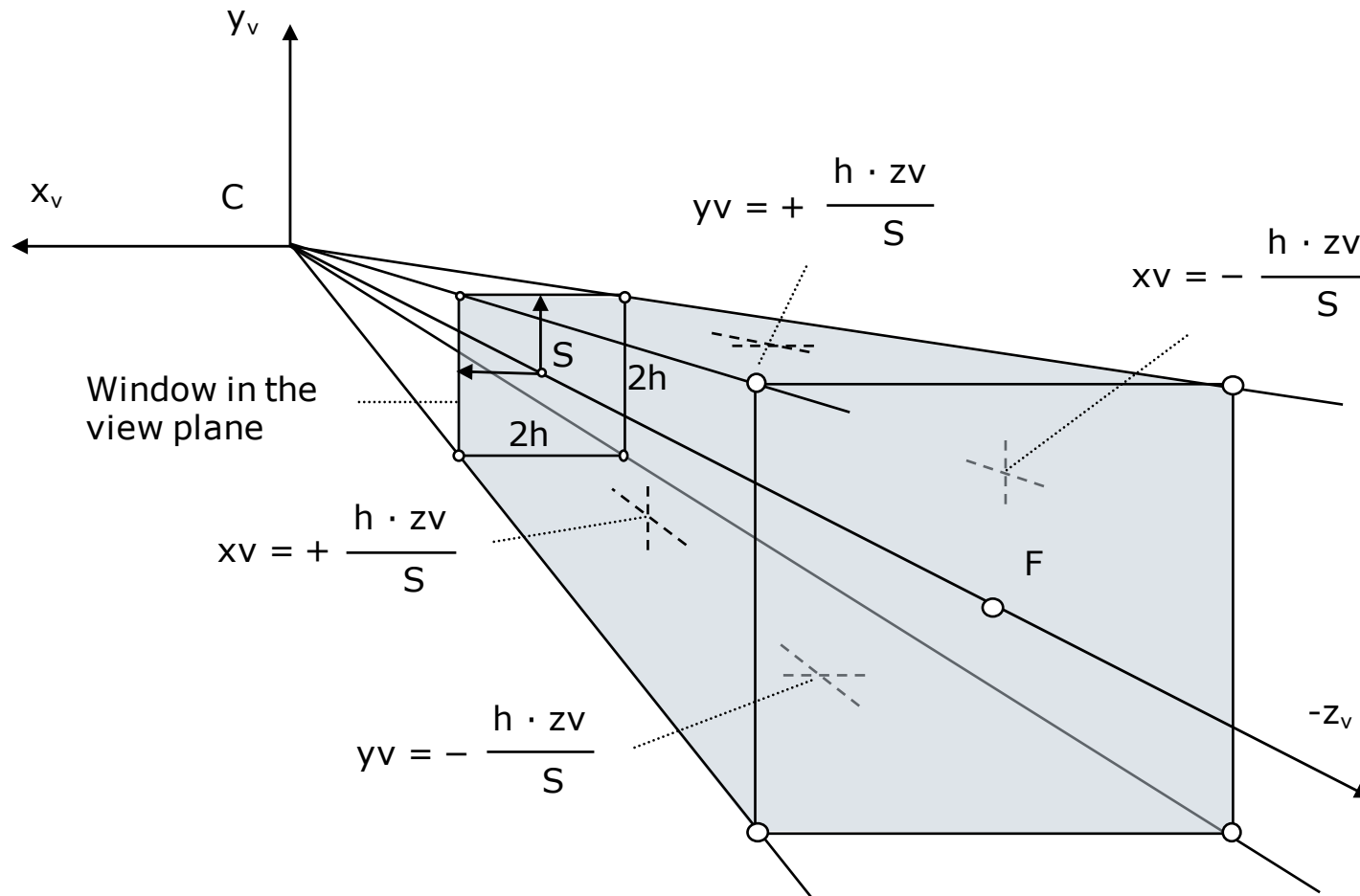
- Screen coordinate system
 - View volume
 - Transformation to screen space
 - 3D clipping against the view volume
 - Screen space rendering
 - Rasterization
 - Hidden surface removal
 - Incremental shading
 - Gouraud shading
 - Phong shading
 - Linear interpolator

Conceptual model of 3D viewing process



View volume

- Screen space is defined inside the view volume (viewing frustum)



View volume

- View volume definition: 6 planes

$$z_v = S$$

$$z_v = F$$

$$x_v = \pm \frac{h \cdot z_v}{S}$$

$$y_v = \pm \frac{h \cdot z_v}{S}$$

- $2h$ is the view plane window dimension
- S is the view plane distance, the virtual screen plane distance, and the near plane distance
- F is the far plane distance
- S and F are application based parameters $\pm$

Transformation to screen space

- ❑ Screen space is a 3D space
- ❑ Transformation conditions:
 1. z_s needs to be normalized so that precision is maximized
 2. Lines in view space need to transform into lines in screen space
 3. Planes in view space need to transform into planes in screen space
- ❑ To satisfy the conditions the transformation of z takes the form:

$$z_s = A + B/z_v$$

where A and B are constants with the following constraints:

1. Choosing $B < 0$ so that as z_v increases then so does z_s . This preserves the intuitive notion of depth. If one point is behind another, then it will have a larger z_v value. If $B > 0$ it will also have a larger z_s value
2. Normalizing the range of z_s values so that the range $z_v \in [S, F]$ maps into the range $z_s \in [0, -1]$.

Obs. The normalization into the negative range $[0, -1]$ is given by the localization of the scene of objects on the $-z_v$ axis, $S \leq 0, F \leq 0$

Transformation to screen space

Perspective transformation is given by the normalization relation:

$$\frac{h \cdot z_v}{S} \rightarrow 1 \quad \text{then} \quad x_v \rightarrow x_s$$

therefore the screen coordinate values are:

$$x_s = x_v \cdot \frac{S}{h \cdot z_v}$$

$$y_s = y_v \cdot \frac{S}{h \cdot z_v}$$

Conditions on the near and far planes of the frustum give:

$$A + B / S = 0$$

$$A + B / F = -1$$

and results $A = F / (S - F)$, $B = -F S / (S - F)$

Transformation to screen space

Finally the transformation is given by the relations:

$$x_s = S \cdot \frac{x_v}{h \cdot z_v}$$

$$y_s = S \cdot \frac{y_v}{h \cdot z_v}$$

$$z_s = F \cdot \frac{1 - S / z_v}{S - F}$$

the additional constant h ensures that x_s and y_s fall in the range $[-1,1]$.

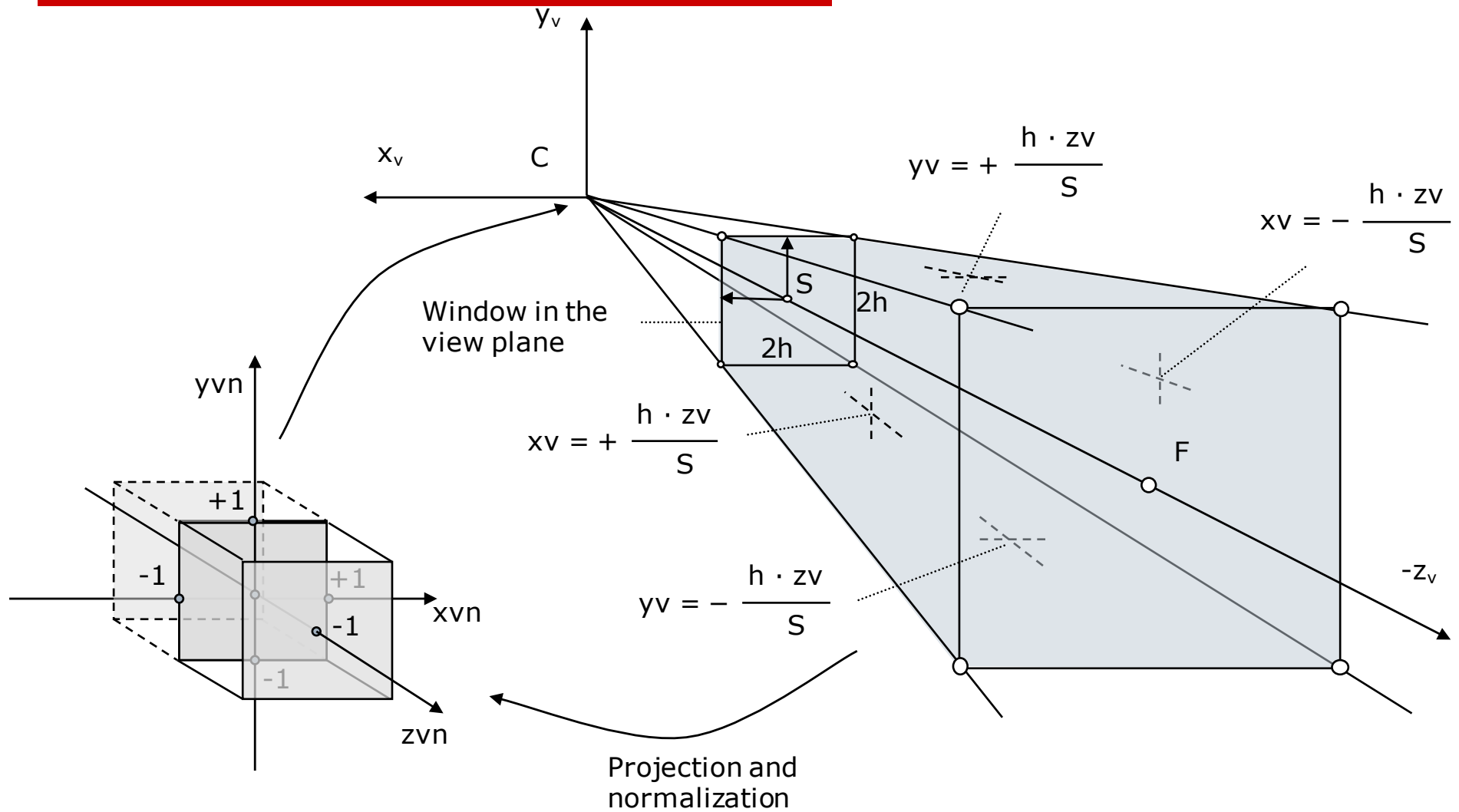
The transformation is not linear and cannot be expressed as a matrix. For that reason it is expressed in **homogeneous coordinates** and split in two parts: (1) linear part, and (2) non-linear part.

(1) **Linear transformation:** $X = x_v$ $Z = \frac{h \cdot F \cdot z_v}{S(S - F)} - \frac{h \cdot F}{S - F}$, and $w = \frac{h \cdot z_v}{S}$
 $Y = y_v$

(2) **Non-linear transformation:** perspective division by w . The final screen coordinates are:

$$x_s = X/w, y_s = Y/w, z_s = Z/w$$

Screen space normalization



Transformation to screen space

- The coordinates (X, Y, Z, w) are called homogeneous coordinates. The fourth coordinate w keeps the depth information, required for later graphics operations (e.g. hidden surface removal)
- The reason for that separation is to express the linear perspective transformation as a 4x4 matrix (i.e. T_{pers})
- The perspective transformation matrix can be concatenated with other previous transformation matrices in order to define a sequence of transformations called **rendering pipeline**

$$\begin{bmatrix} X & Y & Z & w \end{bmatrix} = \begin{bmatrix} xv & yv & zv & 1 \end{bmatrix} T_{\text{pers}}$$

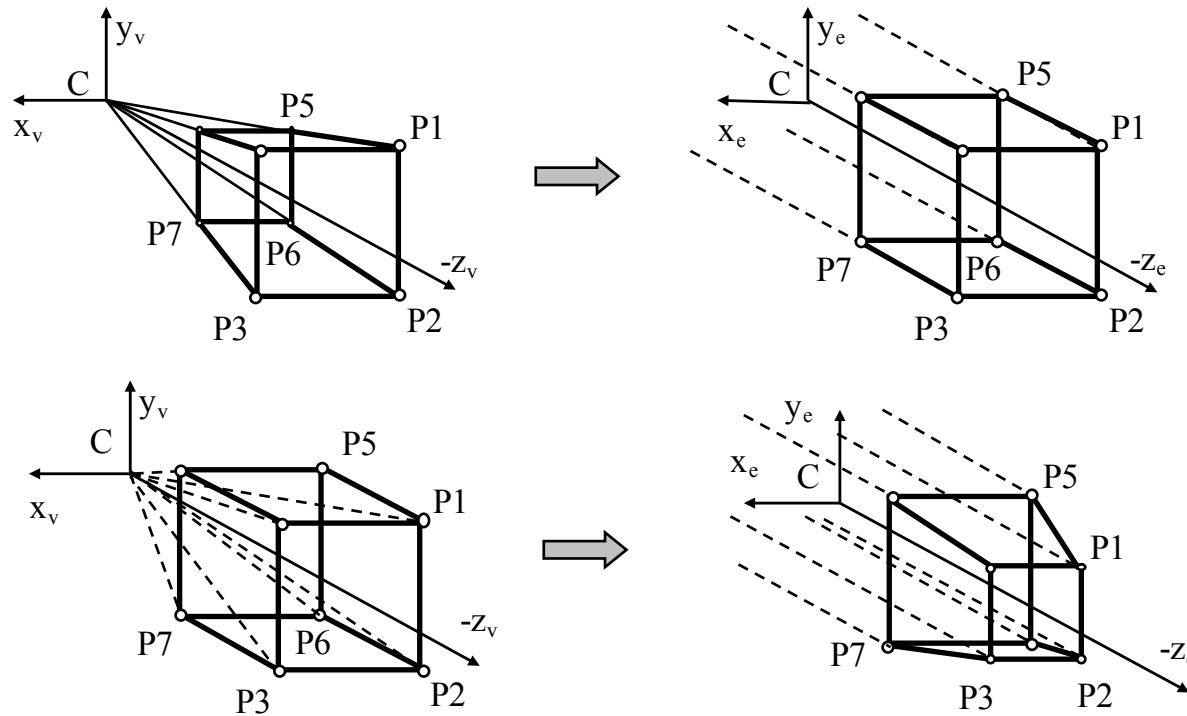
$$T_{\text{pers}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{h \cdot F}{S(S-F)} & \frac{h}{S} \\ 0 & 0 & -\frac{h \cdot F}{S-F} & 0 \end{bmatrix}$$

Transformation to screen space

- ❑ Perspective transformation is the unique transformation that really uses the w coordinate. The homogeneous representation of the Euclidian space is imposed by the need of the perspective in the rendering pipeline
- ❑ The standard transformation matrices M_{ij} such as scale, rotation and translation have the element M_{33} always equal 1. For these transformations w is redundant. W is always 1 and remains 1 even after the transformation.
- ❑ Through the transformation from the view space to the screen space the equal distances on Z_v are transformed into non-equal distances on Z_s . Anyway, both coordinates keep depth information, that is distance on Z .
- ❑ For non-equal distances the correspondent interpolation operation in view space and screen space give different results.
- ❑ The objects are deformed toward the back size of the frustum.
- ❑ The hidden surface removal algorithms are more performant in the screen space than in world or view coordinate systems.

Transformation to screen space

- In screen space the view rays are parallel with Z_s axis, because the projection center moves to infinity, along the Z_s axis (if $z_v = 0$, then $z_s \rightarrow -\infty$). Therefore the computation of the hidden points means to compare the coordinates x_s , and y_s respectively.
- A frustum in the view space becomes a cube in the screen space, and similarly a cube becomes a frustum in the screen space.



3D clipping against the view volume

- Motivation for clipping the objects against the view volume before transformation from the view space to the screen space:
 1. Application needs to deal only with objects that contribute to the final image (i.e. inside the viewing frustum)
 2. The screen space transformation is not defined outside the viewing frustum. There is an exception for $z_v=0$
- An object can have the following position relative to the viewing frustum:
 1. Completely outside the viewing frustum – it is removed from the model
 2. Completely inside the viewing frustum – it is transformed into the screen space and rendered
 3. Intersects the viewing frustum – it is clipped against the frustum and then transformed into the screen space

3D clipping against the view volume

- The clipping conditions are given by:

$$-1 \leq x_s = X/w \leq 1$$

$$-1 \leq y_s = Y/w \leq 1$$

$$-1 \leq z_s = Z/w \leq 0, \text{ where } w = (h \cdot z_v)/S \geq 0$$

or the clipping limits:

$$-w \leq X \leq w$$

$$-w \leq Y \leq w$$

$$-w \leq Z \leq 0$$

- The clipping operation can be performed on the homogeneous coordinates just before the perspective division

Screen space rendering

1. Rasterization

- assigns a value to each pixel in the projection.

2. Hidden surface removal

- removes invisible parts of polygons

3. Shading

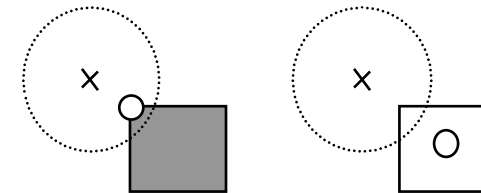
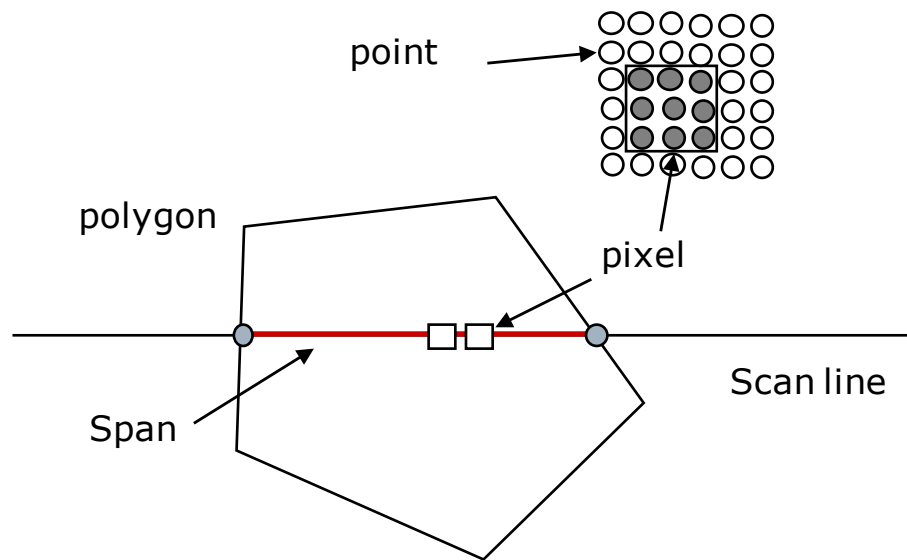
- compares the orientation of each polygon with the light source direction and assigns a shade to the interior of the polygon

4. Anti-aliasing

- computes intermediate values for the overlapped projected parts of the polygons

Rasterization

□ Scan line algorithm

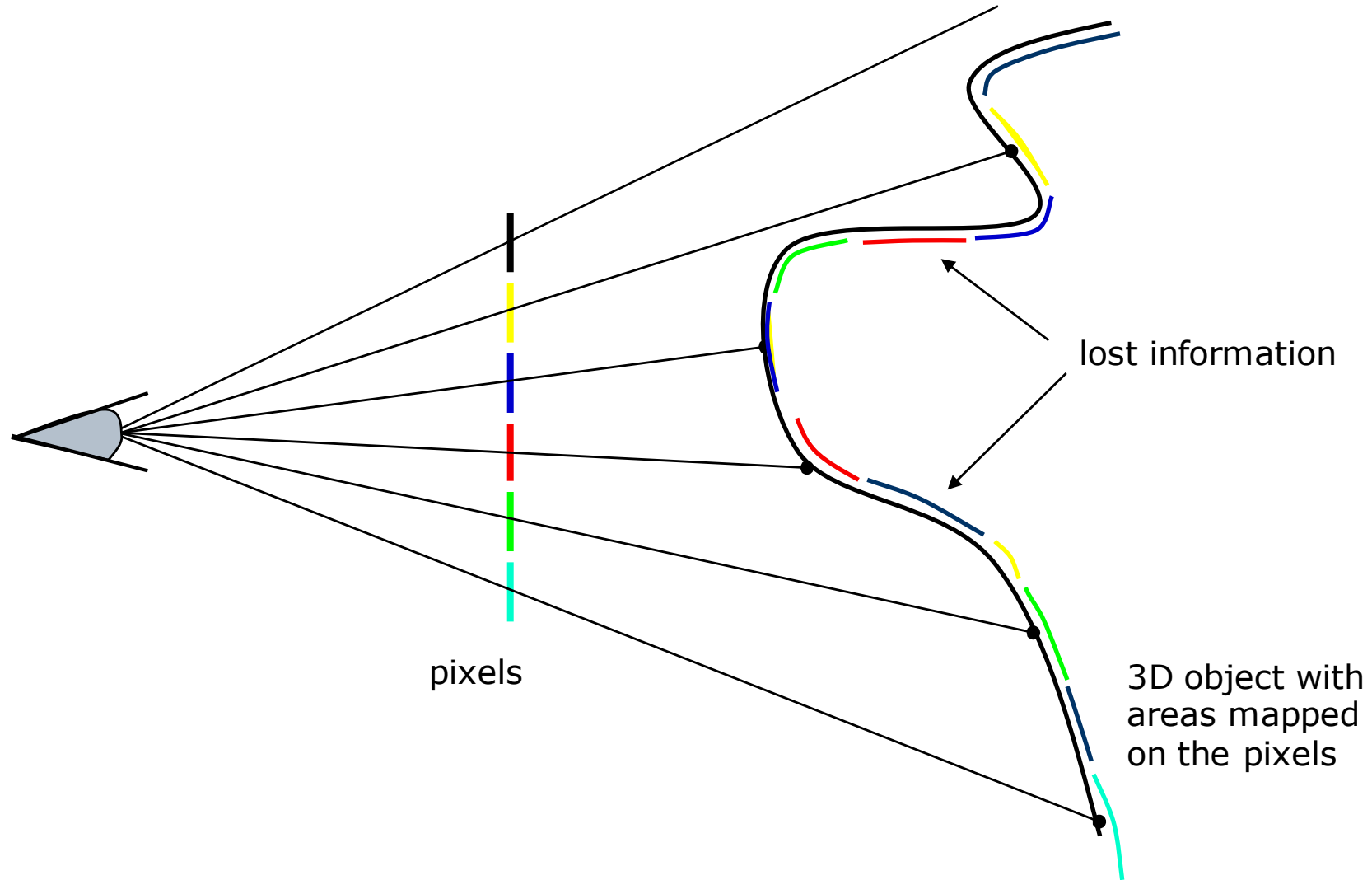


○ Pixel position

x Object point position

Relationship between pixel position and object point.

Mapping points into pixel



Hidden surface removal

- ❑ Z-Buffer algorithm (Catmull, 1975)

- ❑ Advantages of the Z-Buffer algorithm:
 - Simple implementation
 - Processes the polygon based models
 - Does not impose any pre-processing of polygons
 - Processes the polygons in any order they are fetched from the graphics model

- ❑ Disadvantages of the Z-Buffer algorithm:
 - Shading computation performed also for the pixels that are to be hidden by others
 - Needs large memory resources
 - Computation time increases in the case of the perspective projection
 - The size of the Z-Buffer depends on the accuracy to which the depth value of each point (x,y) is to be stored (e.g. integer vs floating point representation of the coordinates)

Incremental shading techniques

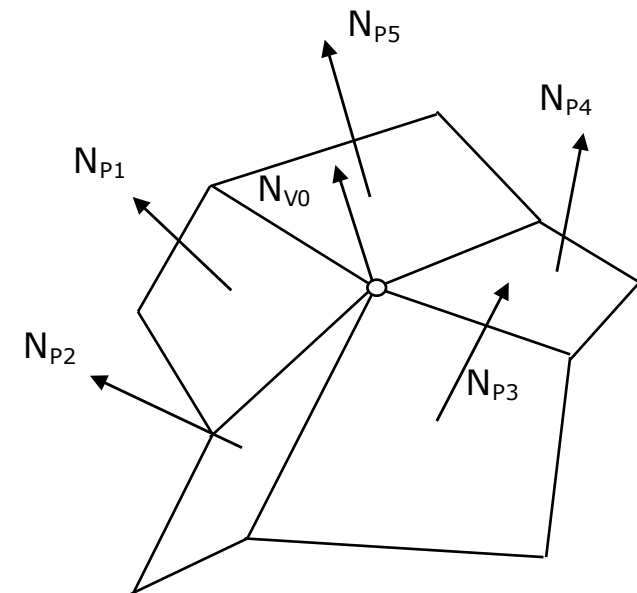
- Compute the light intensity and color over a polygon surface. There are two basic techniques:
 1. Gouraud interpolation (Gouraud, 1971)
 2. Phong interpolation (Phong, 1975)

- Characteristics:
 - Phong interpolation gives more accurate highlights and is generally preferred model
 - Gouraud method tends to be confined to applications where the diffuse component is sufficient, or to preview images prior to final rendering
 - Phong shading is more expensive than Gouraud shading. Phong calculation can exceed more than 50% of the total rendering time

- Issues
 - Final polygon visibility – polygon boundaries should be invisible in the final shaded. There two functions of a shading scheme for polygon meshes:
 1. Use some interpolative method for interior points
 2. Diminish the visibility of the polygon mesh approximation

Gouraud shading

- Gouraud interpolation (Henri Gouraud, 1971)
 - Gouraud, H.: "Continuous Shading of Curved Surfaces", IEEE Trans. On Computers, C-20(6), June 1971, pp.623-629.
- Computes the light intensity for points inside a polygon
- Consider the light source at infinity
- Each vertex has associated a normal vector that is supposed to be orthogonal to the initial curved surface
- The vertex normal could be given or computed from the normals of the neighbor polygons

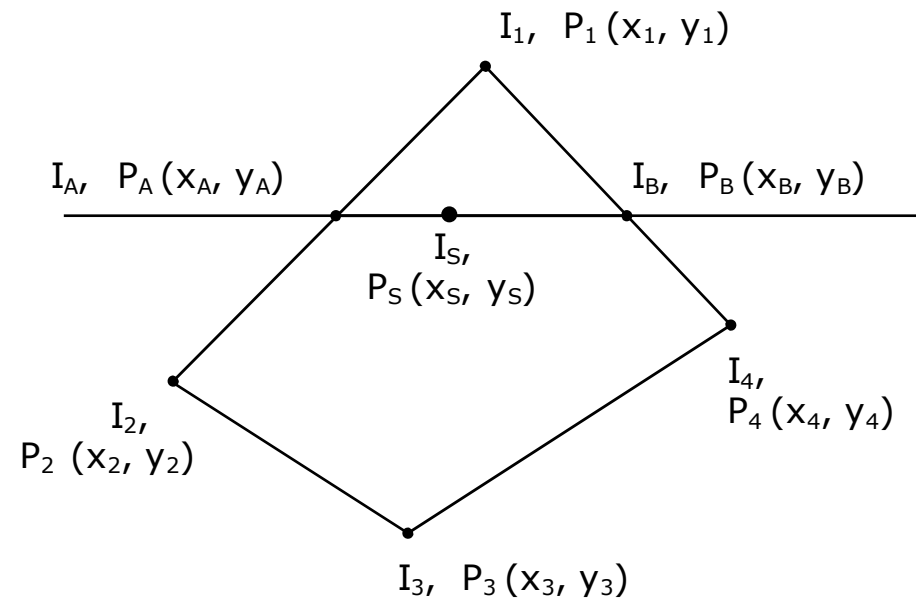


$$N_{v0} = 1/n (\alpha_1 N_{p1} + \alpha_2 N_{p2} + \dots + \alpha_n N_{pn})$$

Gouraud shading

□ Algorithm steps:

1. Compute light intensity I_v for each vertex. Light intensity is a function of normal vector in the vertex and its relative position to the light source and the viewer's position
2. For each scan line position compute by interpolation the light intensities I_A and I_B on the span end points A and B. The span is given by the intersection between the scan line and two edges of the polygon



Gouraud shading

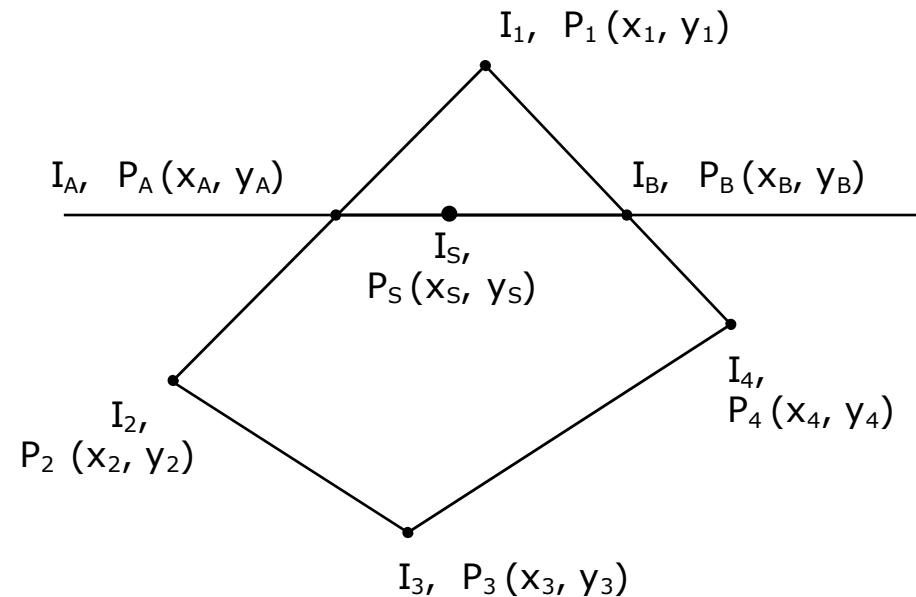
□ Algorithm steps (continuation):

3. For each point position S on the current span compute by interpolation the light intensity I_S from the end point intensities I_A and I_B

$$I_A = I_1 + (I_2 - I_1) / (y_2 - y_1) \cdot (y_A - y_1)$$

$$I_B = I_1 + (I_4 - I_1) / (y_4 - y_1) \cdot (y_B - y_1)$$

$$I_S = I_A + (I_B - I_A) / (x_B - x_A) \cdot (x_S - x_A)$$



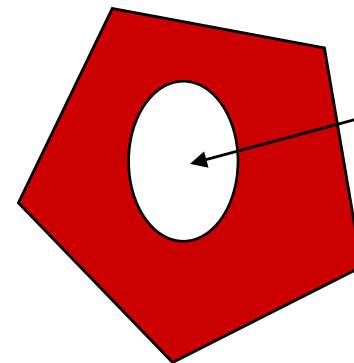
Gouraud shading

- The highlight is not visible by the Gouraud shading.

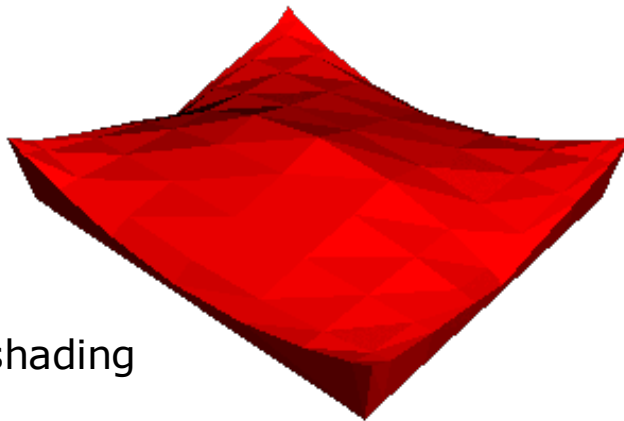
Some examples from the course Computer Graphics, by Pascal Vuylsteker, AN Univ., 2003.



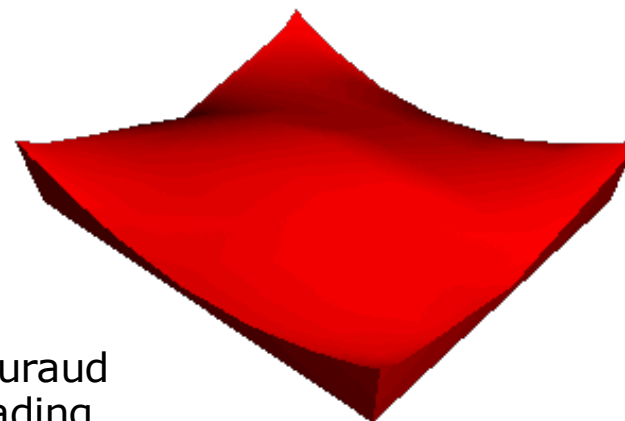
Flat shading



Light patch non-visible by the Gouraud shading



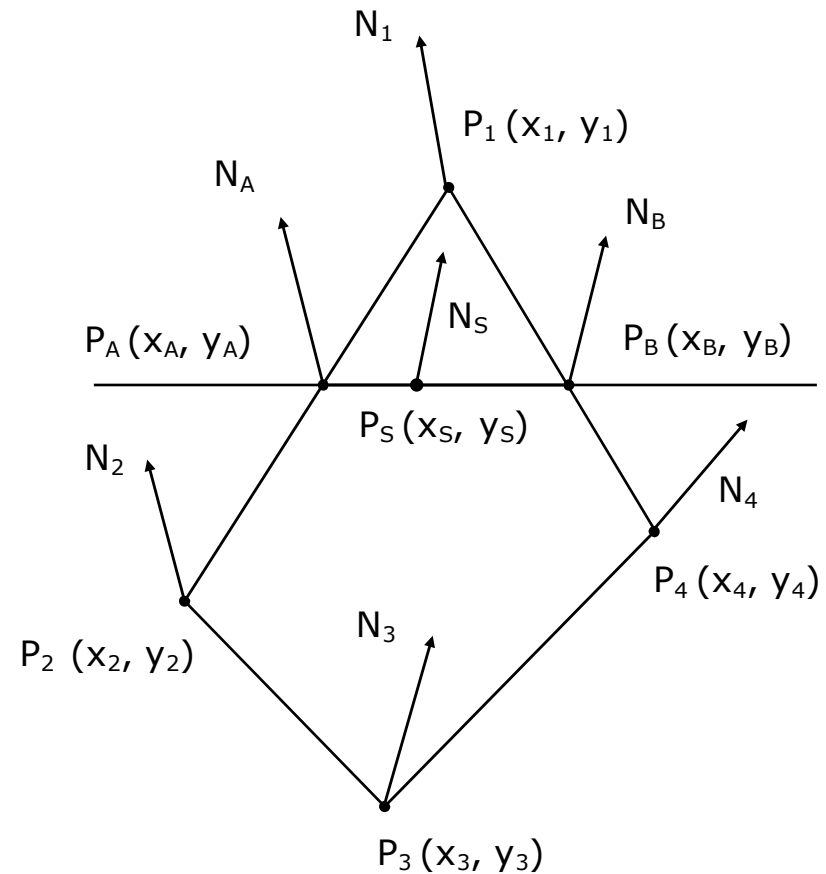
Flat shading



Gouraud shading

Phong shading

- Phong interpolation (Bui-Tuong Phong, 1975)
 - Phong, B. : "Illumination for computer-generated pictures", Comm. ACM, 18 (6), pp.311-317, 1975.
- Consider the light source at infinity
- Computes the light intensity for points inside a polygon
- The attributes interpolated are the vertex normals, rather than vertex intensities
- Each vertex has associated a normal vector that is supposed to be orthogonal to the initial curved surface
- The vertex normal could be given or computed from the normals of the neighbor polygons



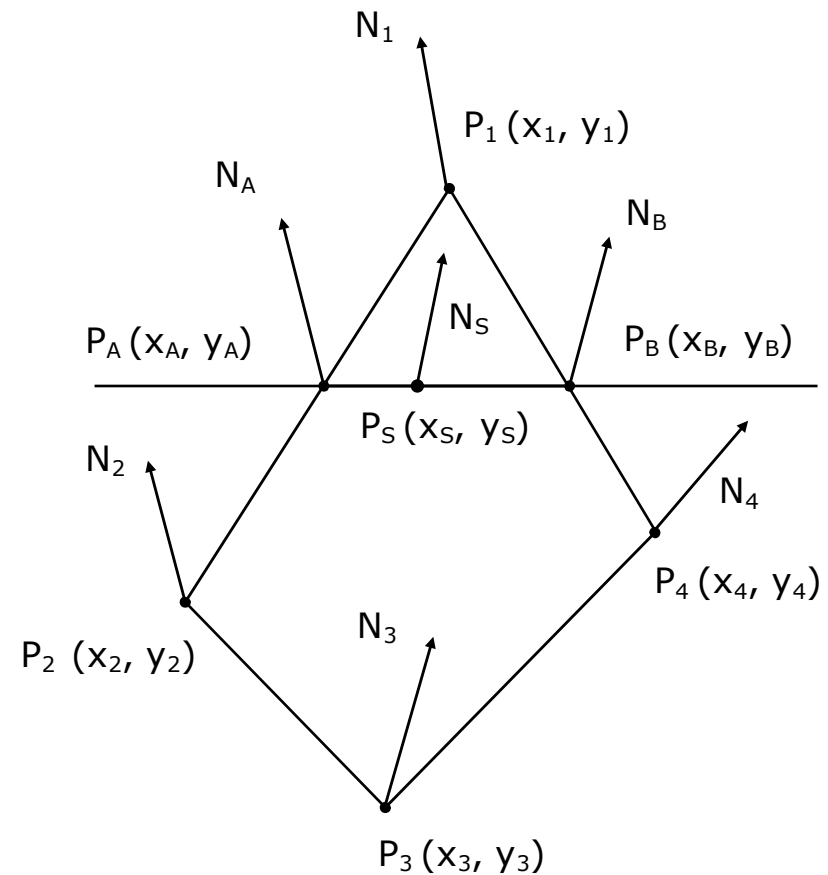
Phong shading

□ Algorithm steps:

1. Compute the normal vector N_v for each vertex.
2. For each scan line position compute by interpolation the normal vectors N_A and N_B at the span end points A and B. The span is given by the intersection between the scan line and two edges of the polygon
3. N_A and N_B are used to interpolate N_S . By this approach a normal vector is computed for each point or pixel on the polygon:

$$N_{Sx}, N_{Sy}, N_{Sz}$$

1. Compute the light intensity using the N_S normal vector



Phong shading

$$N_A = N_1 + (N_2 - N_1) \frac{y_A - y_1}{y_2 - y_1}$$

$$N_B = N_1 + (N_4 - N_1) \frac{y_B - y_1}{y_4 - y_1}$$

$$N_S = N_A + (N_B - N_A) \frac{x_S - x_A}{x_B - x_A}$$

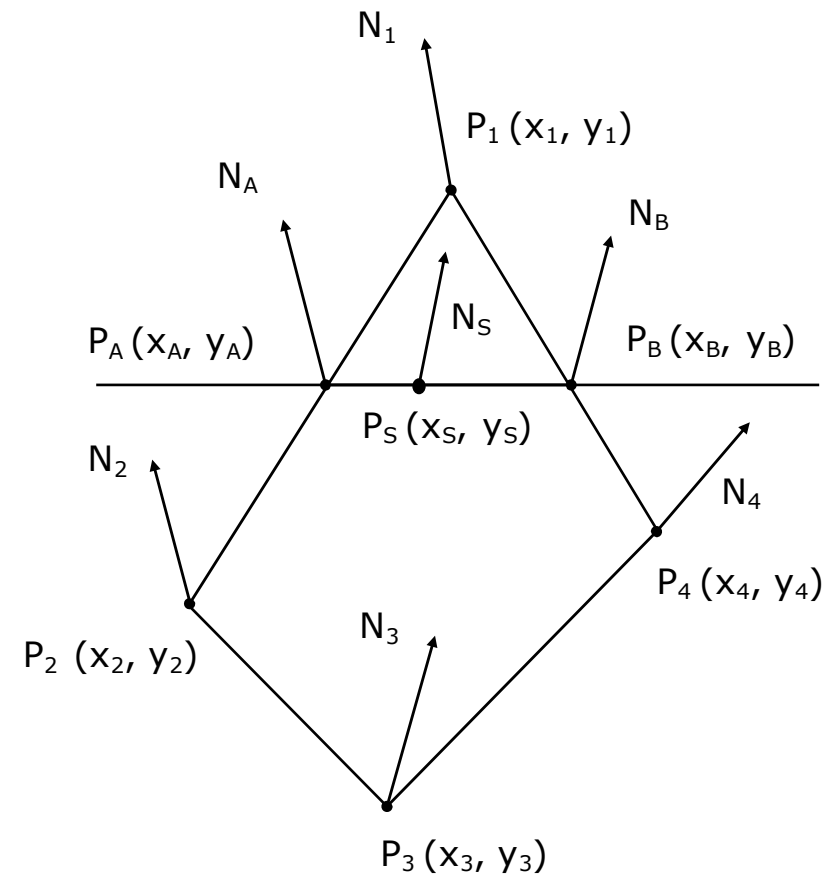
Incremental computation:

$$N_{Si} - N_A = (N_B - N_A) \frac{x_{Si} - x_A}{x_B - x_A}$$

$$N_{Si-1} - N_A = (N_B - N_A) \frac{x_{Si-1} - x_A}{x_B - x_A}$$

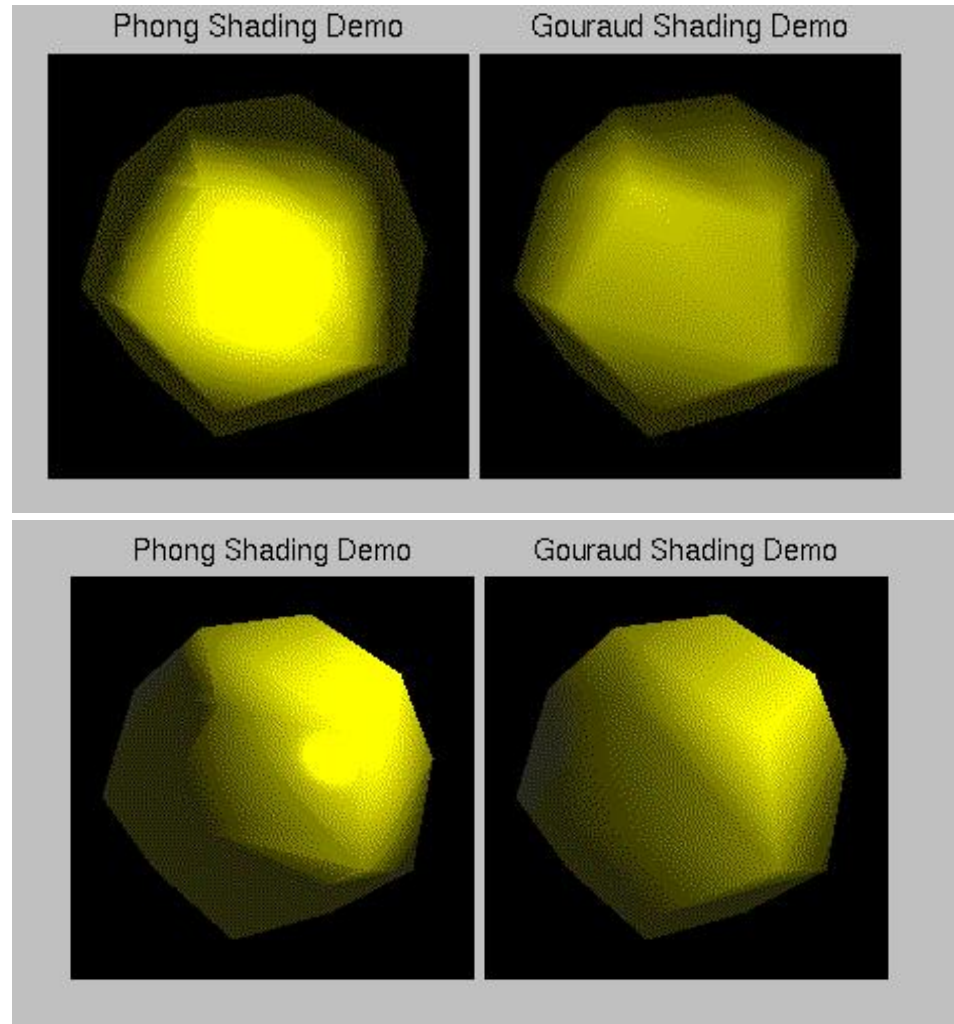
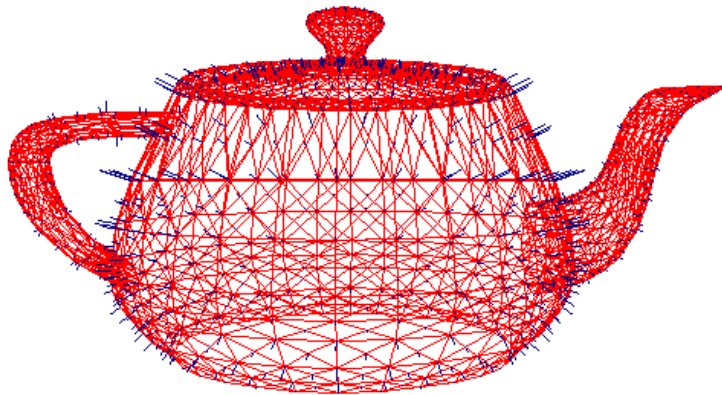
$$\Delta N_{Si} = \Delta x_{Si} \frac{\Delta N_{AB}}{\Delta x_{AB}}$$

$$\text{if } \Delta x_{Si} = 1, \text{ then } N_{Si} = N_{Si-1} + \frac{\Delta N_{AB}}{\Delta x_{AB}}$$



Phong shading

Some examples from the course Computer Graphics,
by Pascal Vuylsteker, AN Univ., 2003.



Gouraud and Phong shading

Examples taken from the course Computer Graphics, by Pascal Vuylsteker, AN Univ., 2003.



Flat shading



Gouraud shading



Phong shading

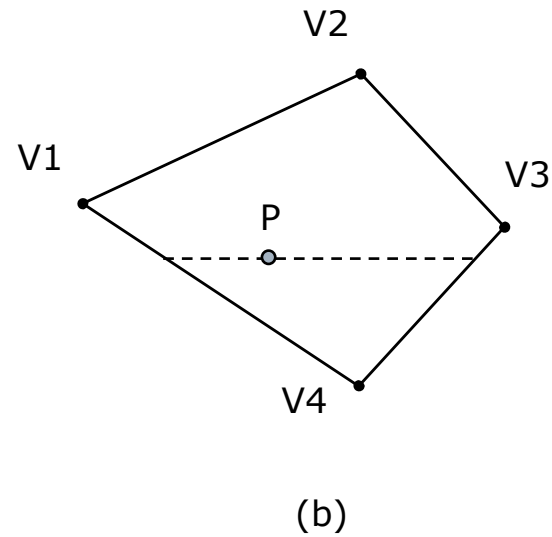
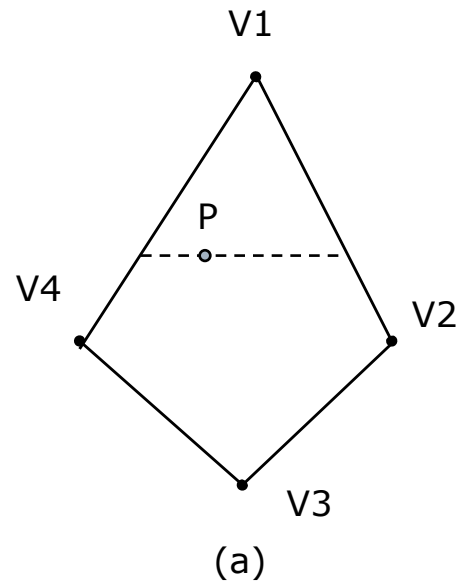
Gouraud and Phong shading

□ Issues:

1. Phong shading is three times as expensive as Gouraud interpolation, N_{Sx} , N_{Sy} , N_{Sz} against I_s
2. Phong shading applies the local reflection model (e.g. Phong model) at each point, against the Gouraud shading that computes the light intensity just on the vertices
3. Hard edges, high quality for great number of polygons
4. Anomalies for rotation, figure (a) and (b)
5. Light intensity computation depends on the underlying polygons, figure (c)
6. Could give identical vertex normals from non-identical surface normals, figure (d)
7. Anomalies because the intensity interpolation is carried out in screen coordinates from vertex normals computed in world coordinates, figure (e) and (f)

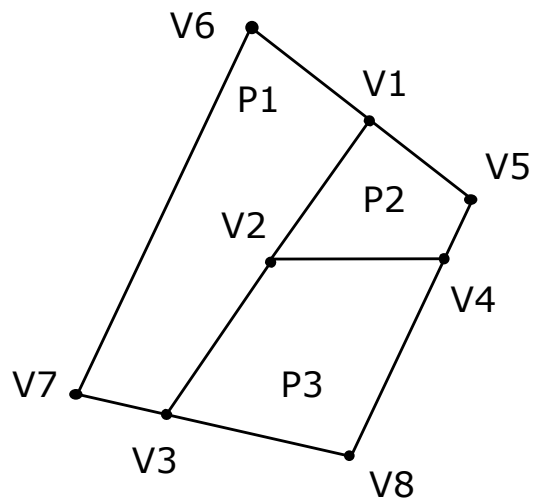
Issues in Gouraud and Phong shading

Visualization depends on object orientation.

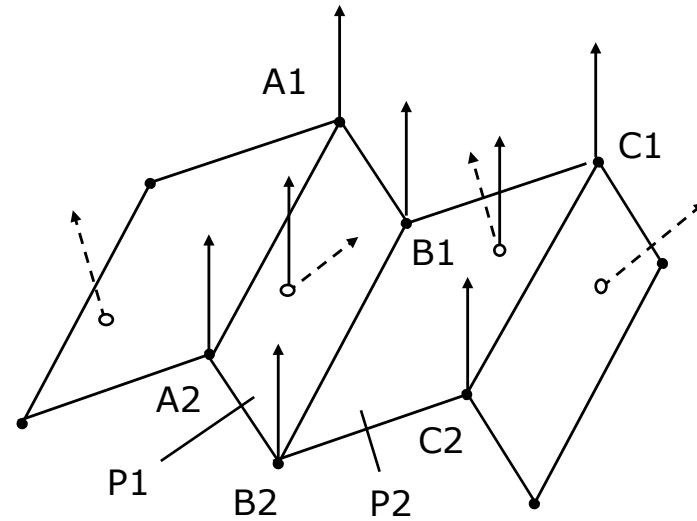


Issues in Gouraud and Phong shading

Visualization depends on the polygonal model (c). Initial vertex normal vectors could give unreal shading (d).



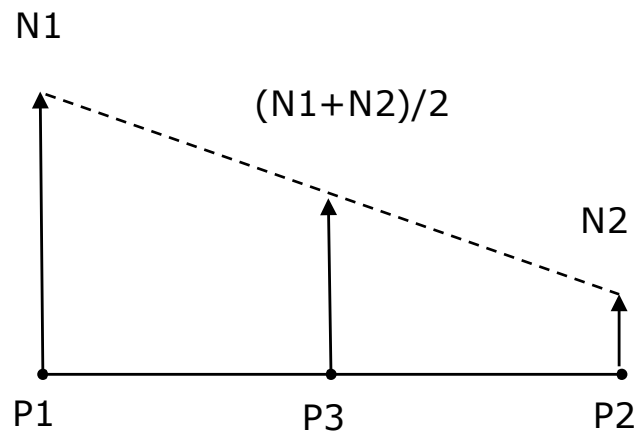
(c)



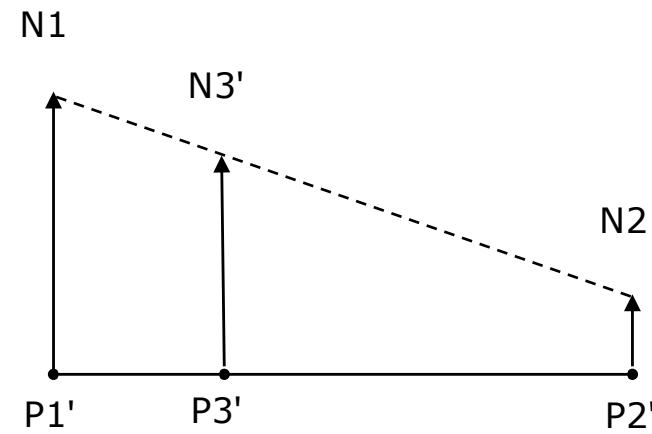
(d)

Issues in Gouraud and Phong shading

The computations in view space and screen space give different light intensity for corresponding points (P3 and P3'), for the distortion of distances and object shapes.



(e)



(f)

Speeding up Phong shading

□ Geometric techniques

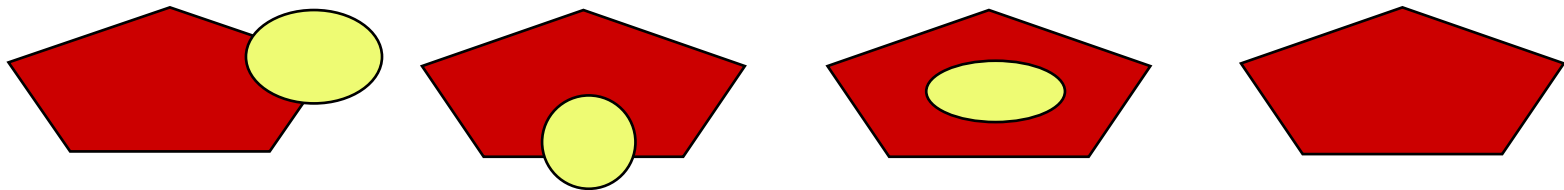
1. Align the light source (if it is unique) along the z axis
2. Compute Phong shading for n^{th} point or pixel (e.g. $n=2$ or 4), and Gouraud shading (linear interpolation for light intensities) for the intermediate points
3. Use the Phong shading just for polygons that involves highlights, and the Gouraud shading for all others:

Highlight detection $N \cdot H \geq T$, where

N is the normal vector at vertex or on the polygon edge;

$H = (L+V)/2$, L – light direction vector, V – viewing vector (see the course on Illumination Models);

T is a given light threshold



□ Numerical techniques

1. Evaluate the ambient, diffuse and specular term for each pixel with additions and memory access for pixel

Linear interpolator

- ▣ Compute intermediate values along a span, for given end point values
- ▣ Applied for
 - Intersections between current scan line and polygon edges
 - Z distance in hidden line and surface removal algorithms
 - Light intensity through Gouraud shading
 - Normal through Phong shading

- ▣ Unique interpolator

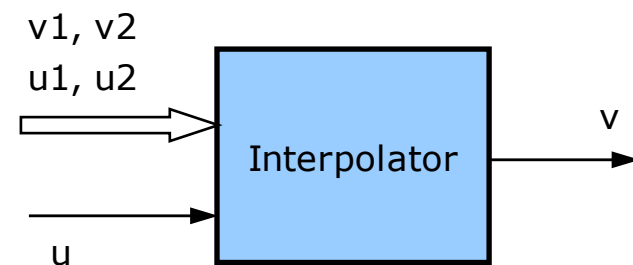
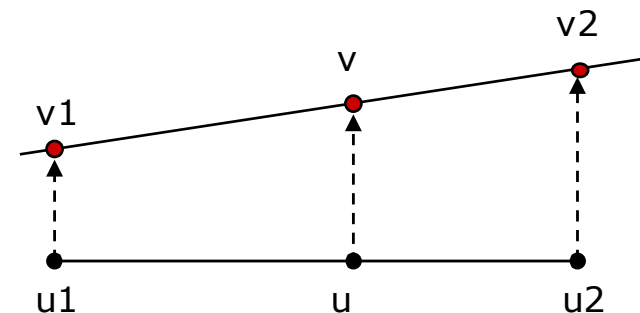
$$\Delta v = \frac{v_2 - v_1}{u_2 - u_1} \Delta u$$

where

$$\Delta v = v - v_1, \Delta u = u - u_1$$

Inputs are v_1, v_2, u_2, u_1 și u

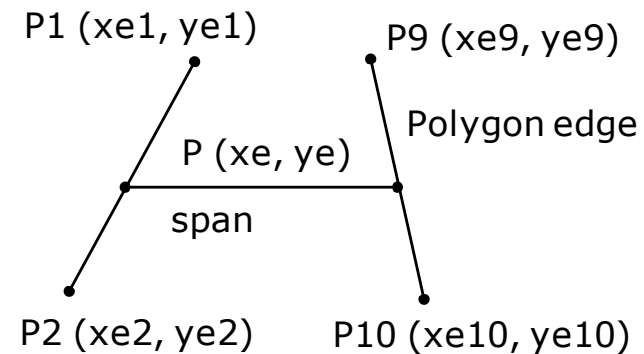
Output is v



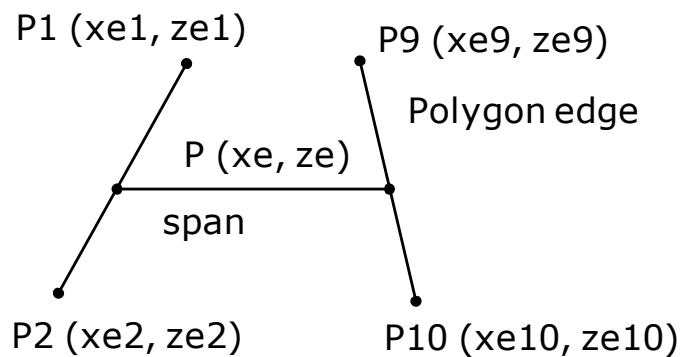
Linear interpolator

□ Various inputs

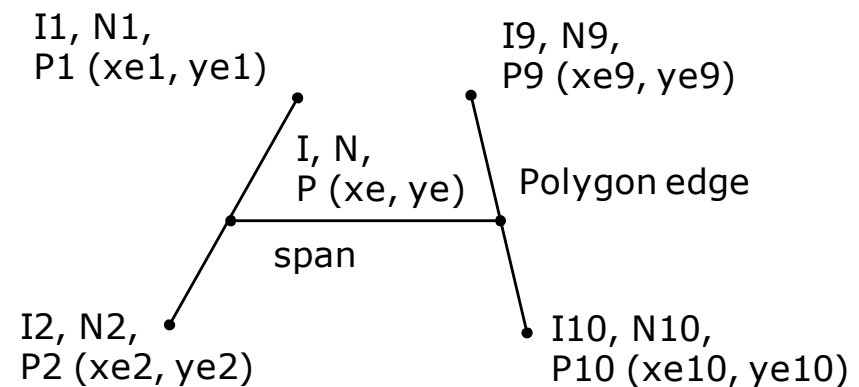
- Vertex coordinates (fig. a)
- Vertex depth (fig. b)
- Vertex light intensity (fig. c)
- Vertex normal vector



(a). $v = x, u = y$



(b). $v = z, u = x, \text{ or } u = y$



(c). $v = I$ (Gouraud shading), or $v = N$ (Phong shading), and $u = x, \text{ or } u = y$

Questions and problems

1. Explain where is located the projection plane within the viewing frustum?
2. Why the graphics model is defined in screen space rather than in screen plane?
3. Explain why the transformation from view space to screen space is performed through two phases?
4. What are the main advantages of the homogeneous coordinates?
5. Explain why the normalized screen space is a semi-cube?
6. Explain why the perspective projection from the view space is transformed into orthogonal projection in the screen space?
7. Exemplify the deformation of a cube, cylinder and sphere by transformation from the view space to the screen space.

Questions and problems

8. Exemplify the deformation of a rectangle, square, circle and triangle by transformation from the view space to the screen space.
9. Why the definition of the reference point for a pixel is important? How can be improved the graphics quality in such a case?
10. Why the z-Buffer algorithm can be applied in view and screen spaces but not in world coordinate space?
11. What is the main goal of the Gouraud and Phong shading algorithms?
12. Explain why and how the Gouraud and Phong shading algorithms works in screen space and screen plane? What are the differences than working similarly in view space and projection plane?

Questions and problems

13. Why the the Gouraud shading algorithm is not able to present light patches inside the polygon?
14. Explain why the Gouraud shading is is less performant but faster than the Phong shading?

Lighting Models

Contents

- Object illumination
- Light interaction with objects
- Energy and light intensity
- Reflection models in graphics
- Phong reflection model
 - Ambient illumination
 - Diffuse illumination
 - Specular illumination
 - Implementation
 - Color computation
- Conclusions

Introduction

□ **Reflection model**

- Describes the interaction of light with a surface, in terms of the properties of the surface and the nature of the incident light
- Computes the light intensity at a object point. Works in world coordinate space
- Basic approach for a photorealistic presentation of a 3D object
- Local reflection model – considers only the interaction between a point on the surface being shaded and the light source
- Global reflection model – considers the indirect illumination (e.g. radiosity and ray tracing)

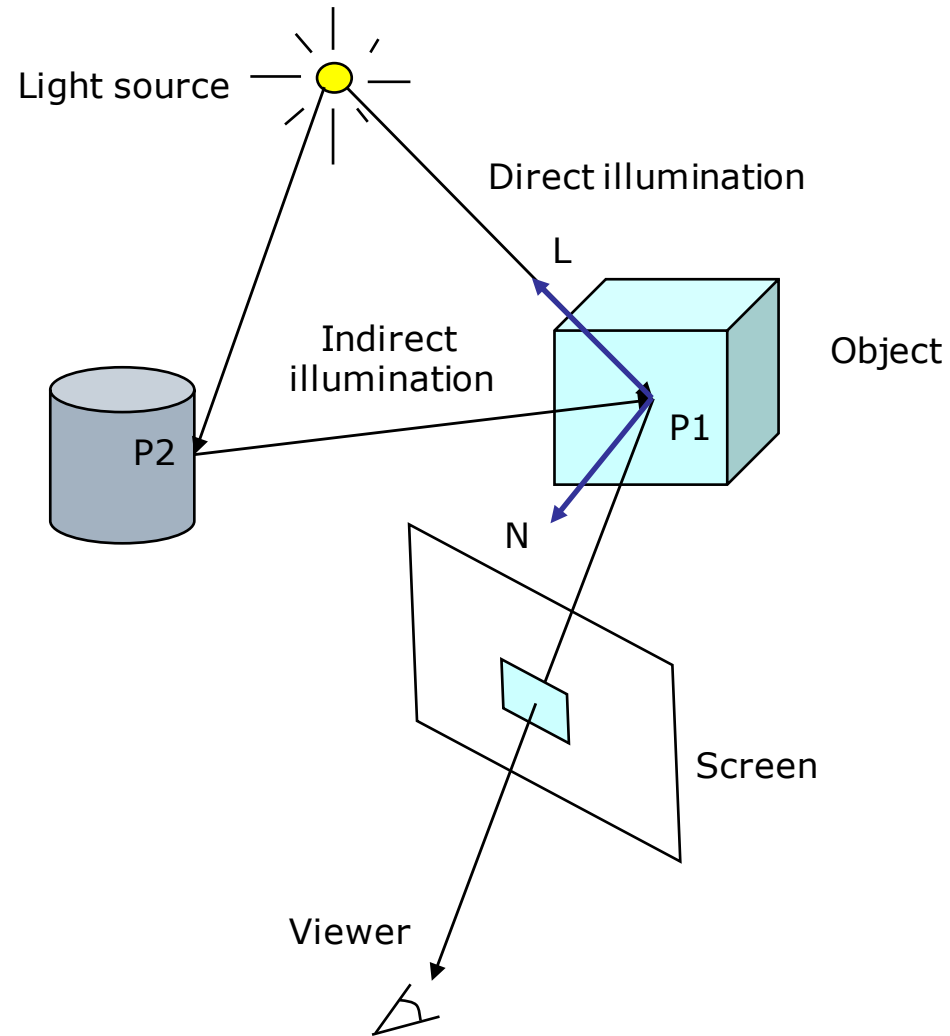
□ **Illumination model**

- Defines the nature of a light emanating from a source (e.g. color, geometry of the intensity distribution, etc)

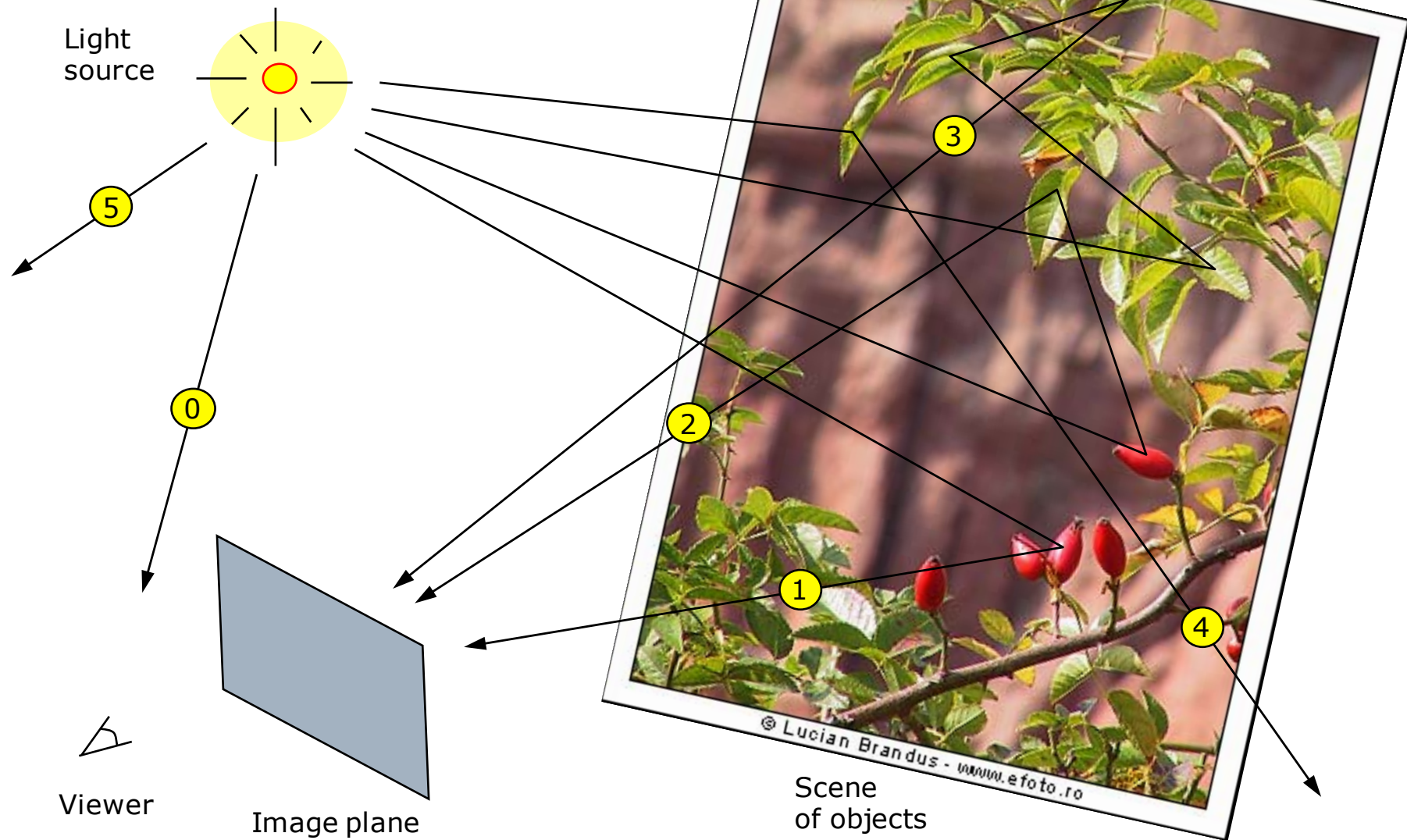
□ **Incremental shading technique**

- Computes the light intensity in a pixel projection of a surface area
- Works in screen space

Object illumination



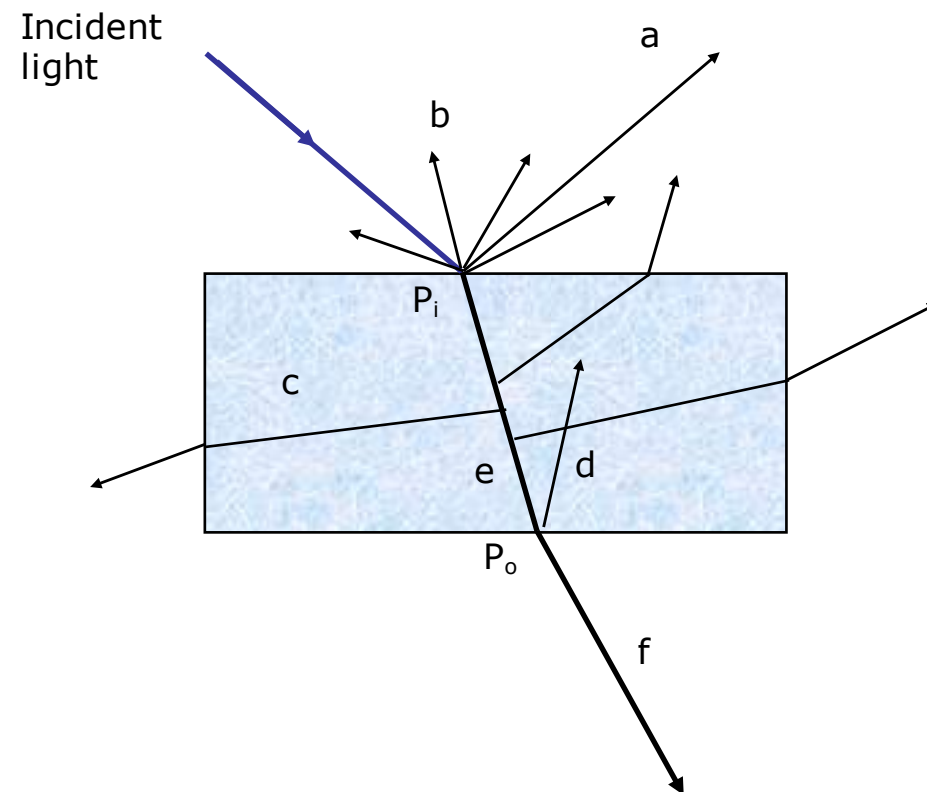
Object illumination



Light interaction with a 3D object

- a. Specular reflection
- b. Diffuse reflection
- c. Fluorescence emission
- d. Internal reflection
- e. Absorption
- f. Transmitted light (refraction)

Incident light =
reflected light +
scattered light +
absorbed light +
transmitted light



Energy and light intensity

- Bidirectional Reflectivity Function (BDRF)
 - Describes qualitatively the reflection phenomenon
 - Assess the limitations of computer graphics models
 - Definition: $R_{bd}(\lambda, \theta_i, \phi_i, \theta_v, \phi_v)$
 - (θ_i, ϕ_i) – incoming light direction
 - (θ_v, ϕ_v) – outgoing light direction toward the viewing direction.

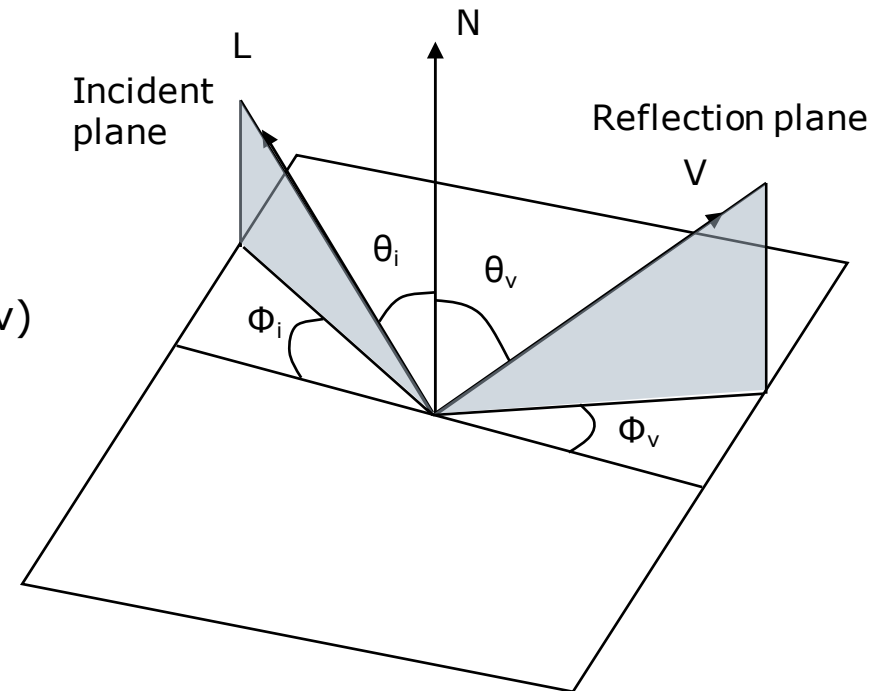
- Reflection model vs Radiation method:

$$I(\theta_v, \phi_v) = I_{\text{emitted}}(\theta_v, \phi_v) + I_{\text{reflected}}(\theta_v, \phi_v)$$

$$R_{bd}(\lambda, \theta_i, \phi_i, \theta_v, \phi_v) = \frac{I_v(\theta_i, \phi_i, \theta_v, \phi_v)}{E_i(\theta_i, \phi_i)}$$

$E_i(\theta_i, \phi_i)$ – incoming light energy

$I_v(\theta_i, \phi_i, \theta_v, \phi_v)$ – outgoing light intensity



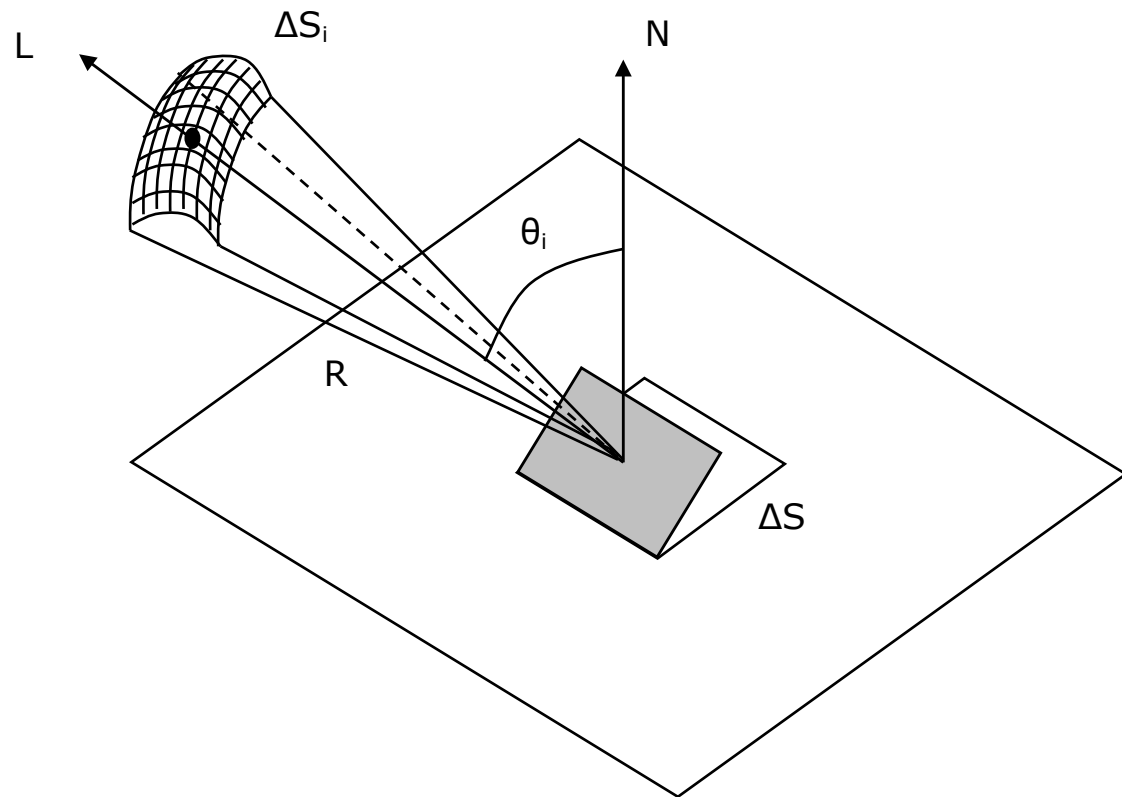
Energy and light intensity

- Incoming energy:

$$E_i(\theta_i, \Phi_i) = I_i(\theta_i, \Phi_i) d\omega_i \cos \theta_i$$

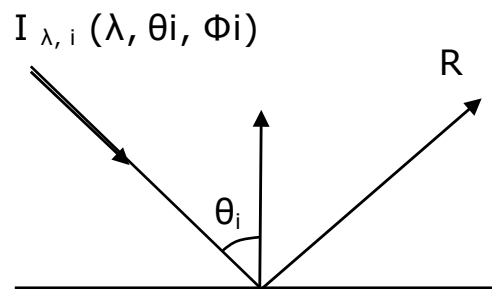
where ω_i is the solid angle

$$d\omega_i = \frac{\Delta S_i}{R^2}$$

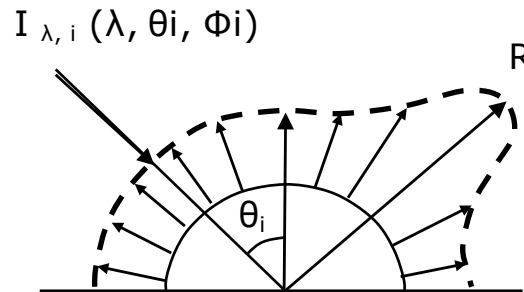


Energy and light intensity

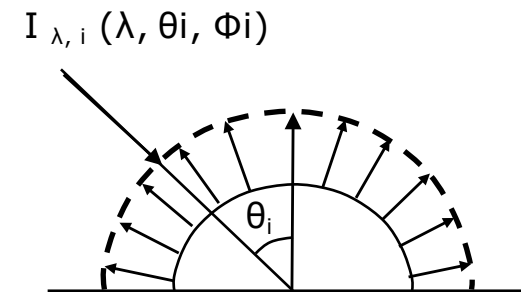
- BDRF is divided in three components:
 1. Specular contribution
 2. Directional diffuse contribution
 3. Ideal diffuse contribution
- Specular and directional diffuse components are due to first order reflections
- Ideal diffuse component is due to multiple reflections and subsurface intersections (microscopic variations in the surface)



(a) Specular reflection



(b) Directional diffuse reflection



(c) Ideal diffuse reflection

Reflection models in graphics

- Torrance and Sparrow (1967) introduced a surface model.
- Warnock (1967) computed the monochrome light intensity as a value attenuated by the distance between the light source and the object.
- Bouknight (1970) computed the light intensity as a function of the orientation of the polygonal surface against the light source. He introduced the diffusion Lambertian ($\cos\theta$), to evaluate the constant light intensity of a polygonal area

$$I_{\lambda, r} = I_{\lambda, \text{ambient}} k_{\text{ambient}}(\lambda) + I_{\lambda, i} k_{\text{diffuse}}(\lambda) (\mathbf{N} \cdot \mathbf{L})$$

- Phong reflection model, proposed by Phong Bui-Tuong in 1975
 - Considers just the first order reflection (i.e. direct illumination)
 - Has become a standard reflection model in the computer graphics community
- Blinn (1977) introduced an empirical specular term.
- Cook and Torrance (1982) concerned on accurate rendering of different metallic and non-metallic surfaces.

Phong reflection model

- Phong model = linear combination of three components: ambient, diffuse, and specular

Ambient light source

- Ambient component is an approximation to global illumination.
- In reality such surfaces would be illuminated by indirect illumination, such as multiple reflections from walls.
- In local reflection model the ambient component is modelled as a constant:

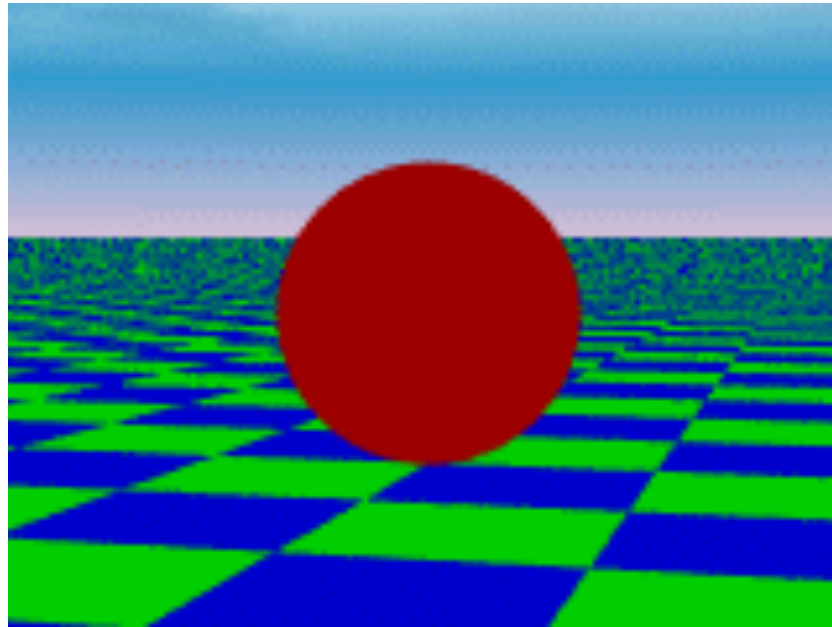
$$I_{\text{reflected}} = I_a k_a(\lambda).$$

- Ambient Light source has no spatial or directional characteristics.
- Ambient light incident on each object is a constant for all the scene.
- Ambient light that is reflected by an object is independent of the object's position and orientation.
- Surface properties are used to determine how much ambient light is reflected. For RGB model they are k_{ar} , k_{ag} , k_{ab} .

Usually $k_{\text{ambient}} = k_{\text{diffuse}}$

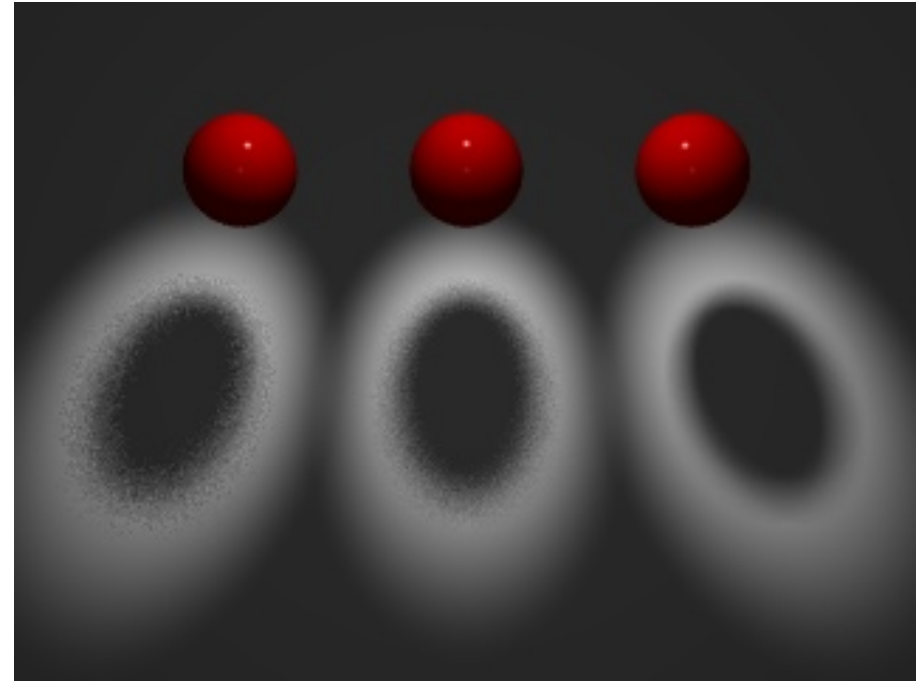
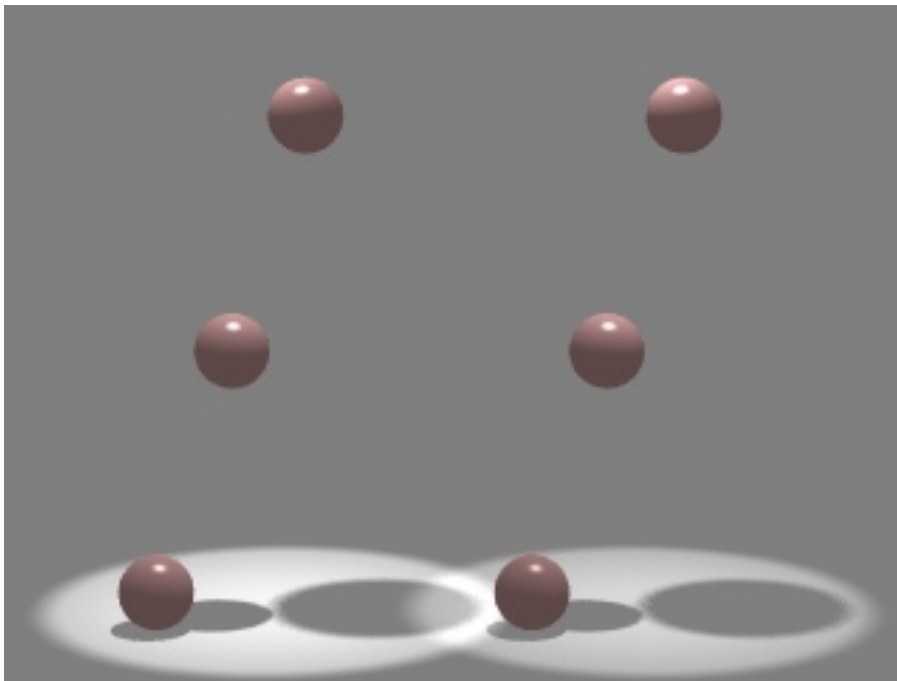
Ambient light source

Examples from the course Computer Graphics, by Pascal Vuylstekker, AN Univ., 2003.



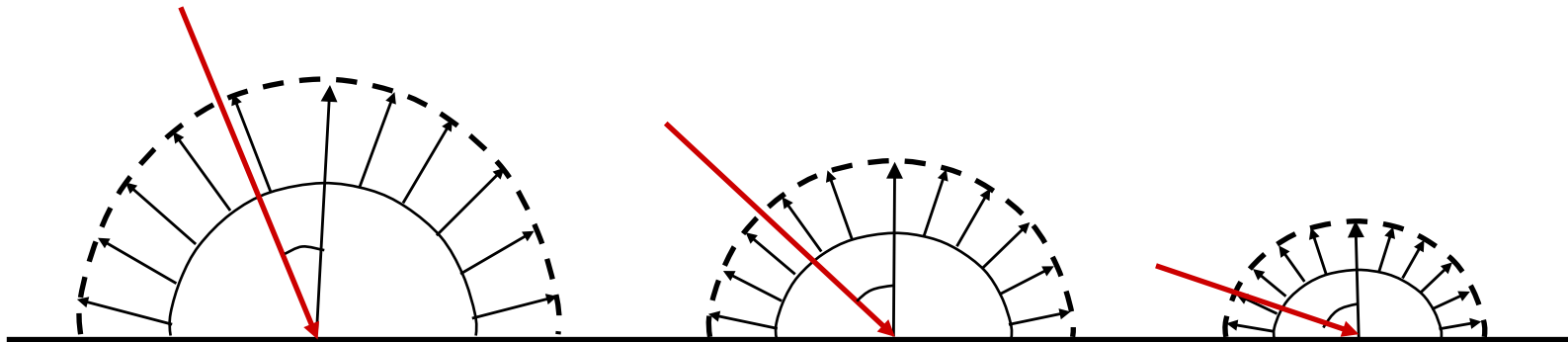
Directional light source

- Consider the direction of the light source to compute both the diffuse and specular components of illumination.
- With a directional light source the illumination direction is a constant.
- Point light source, spotlight, area light source.



Ideal diffuse reflection

- Lambert's Cosine law:
 - reflected energy from a small surface area in a particular direction is proportional to the cosine of the angle between that direction and the surface normal
- Ideal diffuse reflectors = Lambertian reflectors
 - Ideal diffuse reflectors reflect light according to Lambert's cosine law (Lambertian reflectors).
 - Energy that is reflected in any one direction is constant in Phong model. Reflected intensity is independent of the viewing direction.
 - Intensity depends on the light source's orientation relative to the surface.



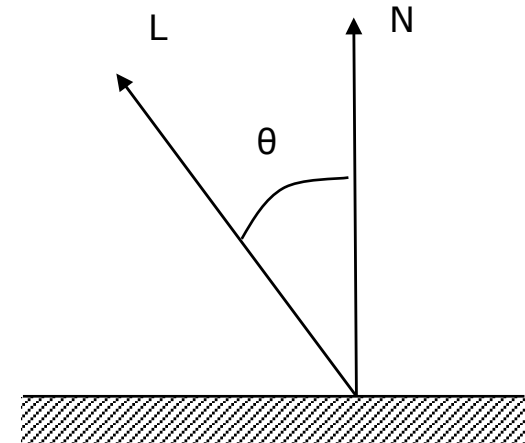
Diffuse component

$$I_{\text{diffuse}} = I_i (k_d(\lambda) \cos \theta)$$

$$I_{\text{diffuse}} = I_i (k_d(\lambda) (L \cdot N))$$

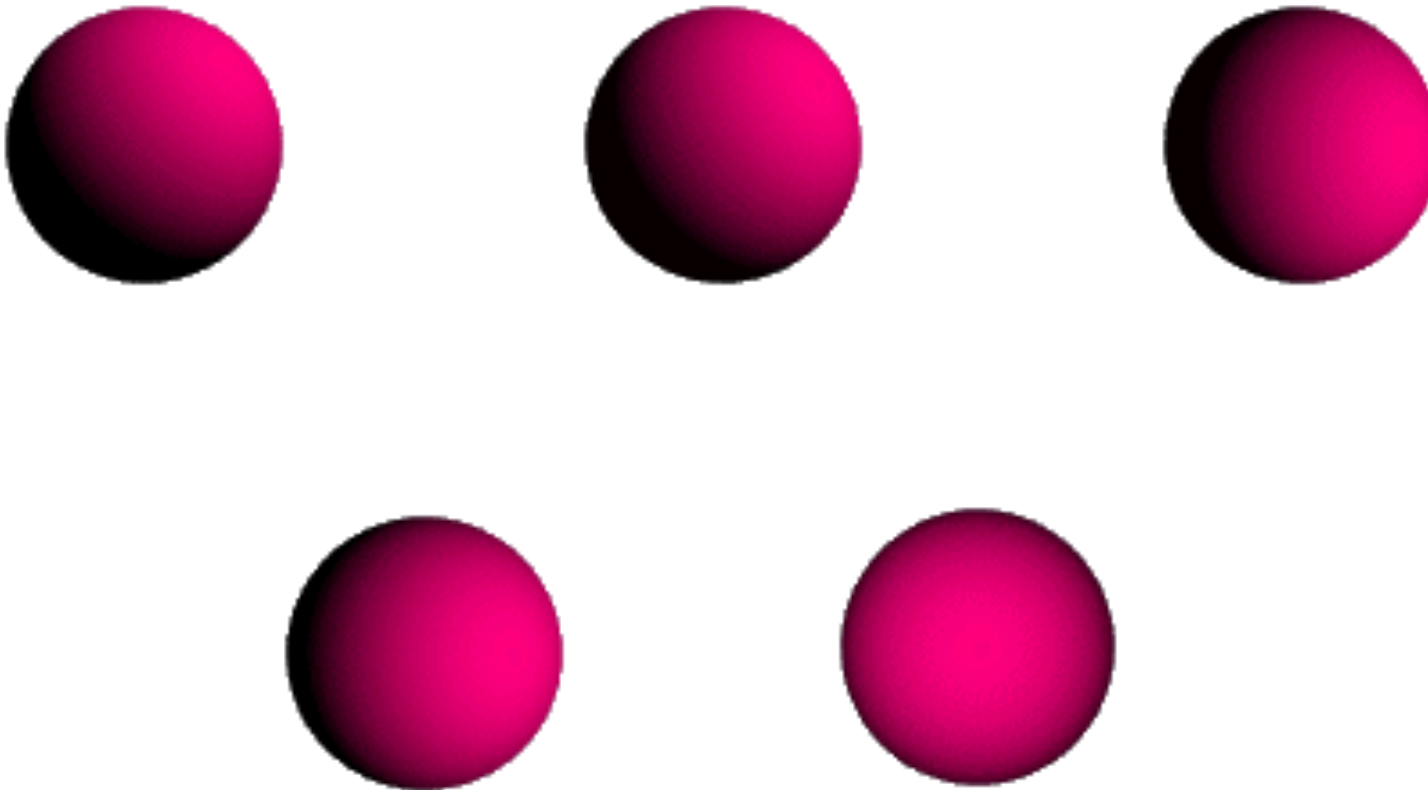
for m light sources:

$$I_{\text{diffuse}} = \sum_{j=1}^m I_{i,j} (k_d(\lambda) (L_j \cdot N))$$



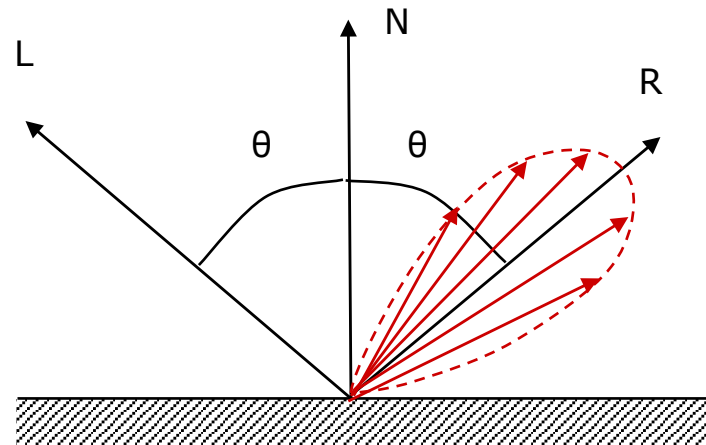
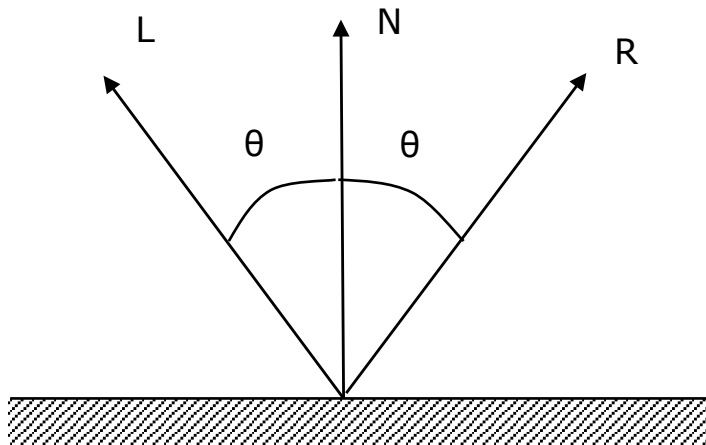
Diffuse lighting examples

Examples from the course Computer Graphics, by Pascal Vuylsteker, AN Univ., 2003.



Specular reflection

- Specular reflector.
- Shiny surface, very smooth surface (e.g. polished metal).
- The specular reflector is an ideal mirror.



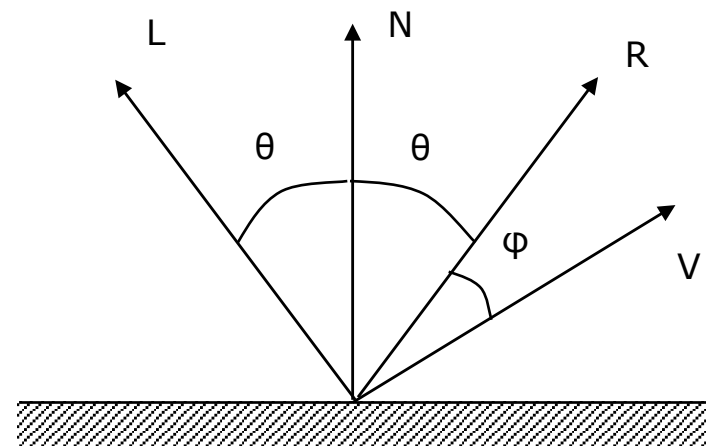
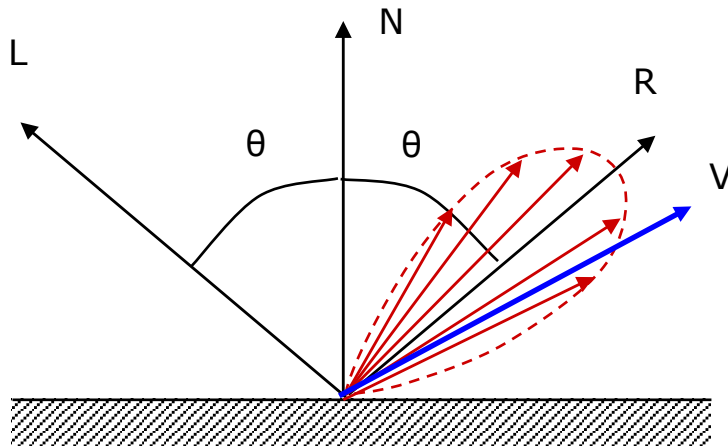
Phong illumination model

- Specular reflection toward the viewing direction.
- No physical basis, but is the most used illumination model

$$I_{\text{specular}} = I_i k_s (\cos \phi)^n$$

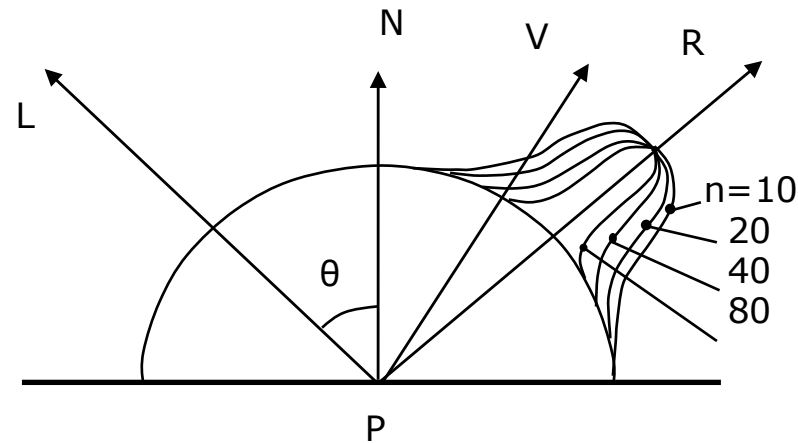
$$I_{\text{specular}} = I_i k_s (R \cdot V)^n$$

n controls the surface shininess, the spread of the light rays around the R direction.



Phong illumination model

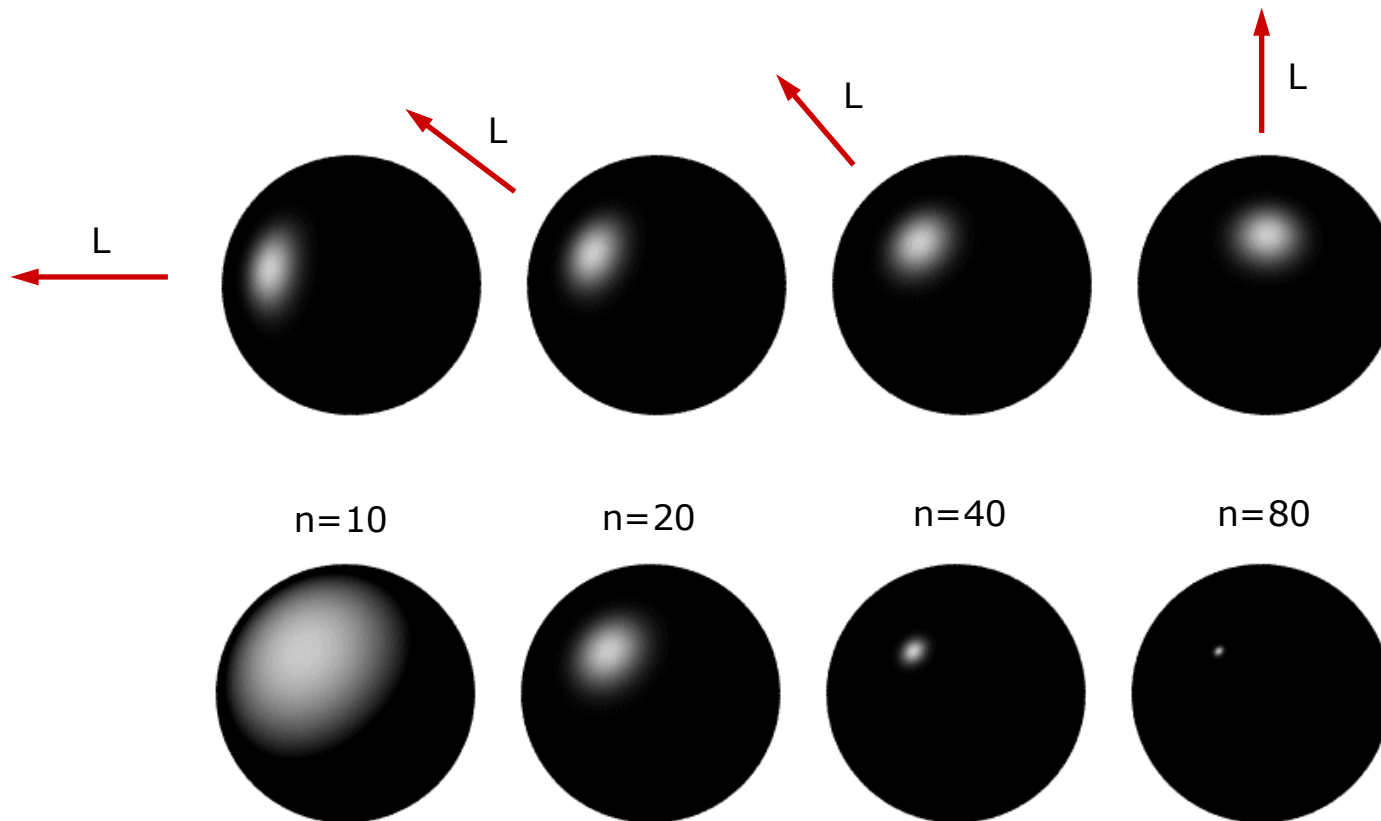
n controls the surface shininess, the spread of the light rays around the R direction



Phong illumination examples

Examples from the course Computer Graphics, by Pascal Vuylsteker, AN Univ., 2003.

- The following spheres illustrate specular reflections as the direction of the light source (L) and the coefficient of shininess (n) is changed



Phong reflection model

Consider all three components together: ambient diffuse and specular

$$I_v(\lambda) = I_a k_a(\lambda) + I_i k_d(\lambda) (L \cdot N) + I_i k_s (R \cdot V)^n$$

or as trigonometrical form:

$$I_v(\lambda) = I_a k_a(\lambda) + I_i k_d(\lambda) \cos \theta + I_i k_s (\cos \varphi)^n$$

for m light sources

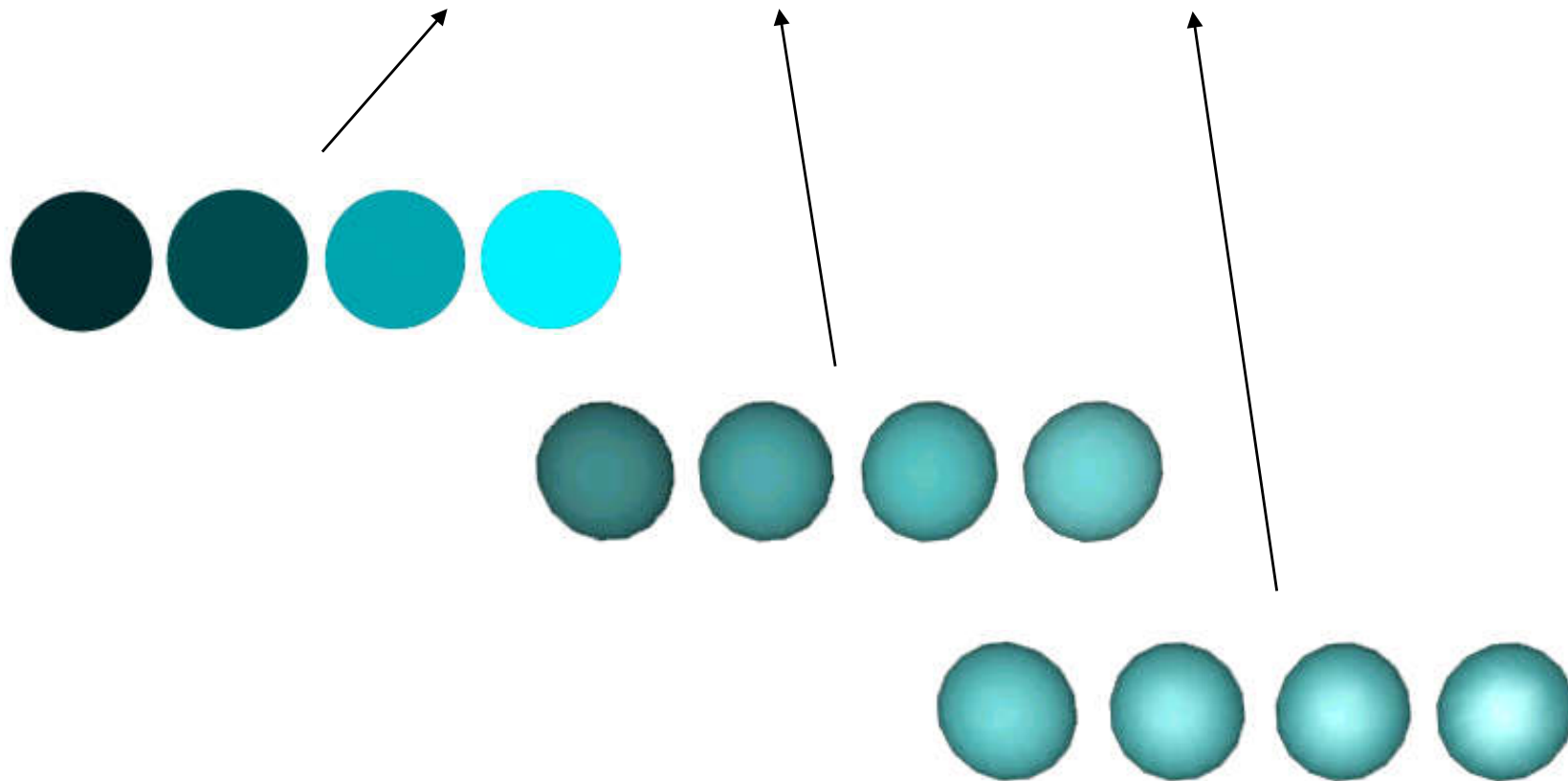
$$I_v(\lambda) = I_a k_a(\lambda) + \sum_{j=1}^m I_{i,j} [k_d(\lambda) (L_j \cdot N) + k_s (R_j \cdot V)^n]$$

$$I_v(\lambda) = I_a k_a(\lambda) + \sum_{j=1}^m I_{i,j} [k_d(\lambda) (\cos \theta_j) + k_s (\cos \varphi_j)^n]$$

Phong reflection model - examples

Consider all three components together: ambient, diffuse and specular

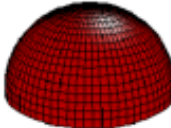
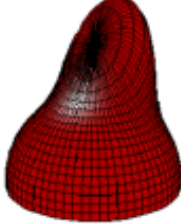
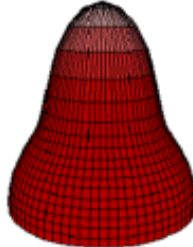
$$I_v(\lambda) = I_a k_a(\lambda) + I_i k_d(\lambda) (L \cdot N) + I_i k_s (R \cdot V)^n$$



Phong reflection model - examples

Study taken from the course Computer Graphics, by Pascal Vuylsteker, AN Univ., 2003.

$$I_v(\lambda) = I_a k_a(\lambda) + I_i k_d(\lambda) \cos \Phi + I_i k_s (\cos \varphi)^n$$

Phong	ρ_{ambient}	ρ_{diffuse}	ρ_{specular}	ρ_{total}
$\phi_i = 60^\circ$				
$\phi_i = 25^\circ$				
$\phi_i = 0^\circ$				

Phong model implementation

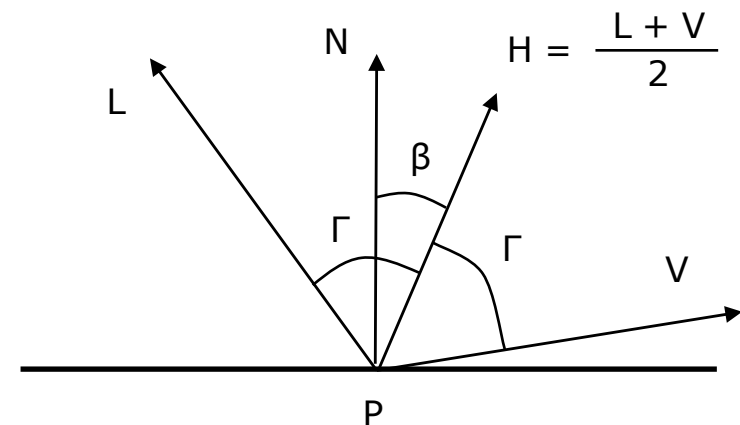
□ Assumptions for simplicity and computation time:

1. Light source and the view point are at infinity, then the L and V vectors are constant over the domain of the scene
2. Introduce a vector H, and the dot product ($N \cdot H$) instead of ($R \cdot V$). H is the unit normal vector to a hypothetical surface that is oriented in a direction halfway between the light direction vector L and the viewing vector V

$$H = (L+V)/2$$

The angle between R and V is
twice the angle between N and H:

$$\varphi = 2\beta$$



Phong model implementation

Consider all three components together: ambient diffuse and specular

$$I(\lambda) = I_a k_a(\lambda) + I_i k_d(\lambda) (L \cdot N) + I_i k_s (N \cdot H)^n$$

or as trigonometrical form:

$$I(\lambda) = I_a k_a(\lambda) + I_i k_d(\lambda) \cos \theta + I_i k_s (\cos \beta)^n$$

for m light sources

$$I(\lambda) = I_a k_a(\lambda) + \sum_{j=1}^m I_{i,j} [k_d(\lambda) (L_j \cdot N) + k_s (N \cdot H_j)^n]$$

$$I(\lambda) = I_a k_a(\lambda) + \sum_{j=1}^m I_{i,j} [k_d(\lambda) (\cos \theta_j) + k_s (\cos \beta_j)^n]$$

Colors in Phong model

- Color as light intensity is a function of wavelength

$$I(\lambda) = I_a k_a(\lambda) + I_i k_d(\lambda) (L \cdot N) + I_i k_s (N \cdot H)^n$$

- In the RGB system the color is: $I = \alpha_r I_r + \alpha_g I_g + \alpha_b I_b$
- Consider the specular highlights as white
- Control the color of the objects by appropriate setting of the diffuse coefficients, and the scene by the ambient coefficients
- It should be computed three intensity equations:

$$I_r = I_a k_{ar} + I_i [k_{dr} (L \cdot N) + k_s (N \cdot H)^n]$$

$$I_g = I_a k_{ag} + I_i [k_{dg} (L \cdot N) + k_s (N \cdot H)^n]$$

$$I_b = I_a k_{ab} + I_i [k_{db} (L \cdot N) + k_s (N \cdot H)^n]$$

Conclusion:

$$I(r,g,b) = I_a k_a(r,g,b) + I_i [k_d(r,g,b)(L \cdot N) + k_s (N \cdot H)^n]$$

Conclusions on Phong model

- Phong local reflection model:

$$I(\lambda) = I_a k_a(\lambda) + I_i k_d(\lambda) (L \cdot N) + I_i k_s (N \cdot H)^n$$

or
$$I(\lambda) = I_a k_a(\lambda) + I_i k_d(\lambda) \cos \theta + I_i k_s (\cos \beta)^n$$

- Light sources are point sources
- Light sources and the viewer are located at infinity
- The diffuse and specular terms are modeled as local components
- An empirical model (shininess n) is used to simulate the decrease of the specular term around the reflection vector modeling the glossiness of the surface
- The color of the specular reflection is assumed to be that of the light source. It means the highlights are rendered white regardless of the material
- The global term that is the ambient contribution is a constant

Questions and problems

1. What is the surface model for the object illumination?
2. How the Phong model describes the multiple reflections of the light rays?
3. What is the location of the light source for the ambient component?
4. What means the Lambertian surface?
5. Why the diffuse illumination infers the light source position?
How the light source position is described within the Phong model?
6. What is the difference between R and V directions of the specular reflection in the Phong model?
7. Why the specular illumination infers the light source position?
How the light source position is described within the Phong model?

Questions and problems

8. Why the specular light is white and the ambient and diffuse components are colored?
9. How is computed the illumination by three light sources by Phong model, if one is red, one is green, and another is blue?
10. How is computed the illumination by three light sources by Phong model, if one is red, one is magenta, yellow, and another is cyan?

Texture Mapping (1)

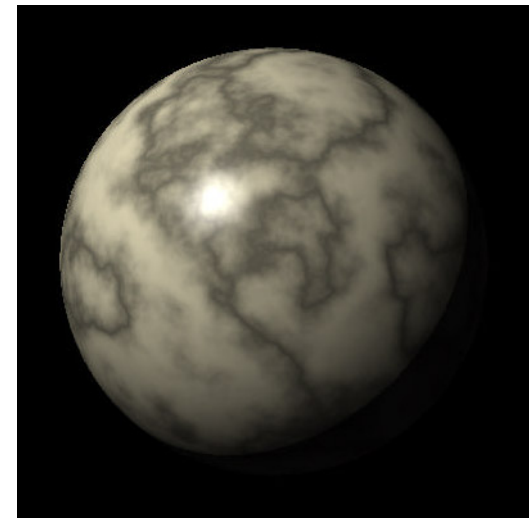
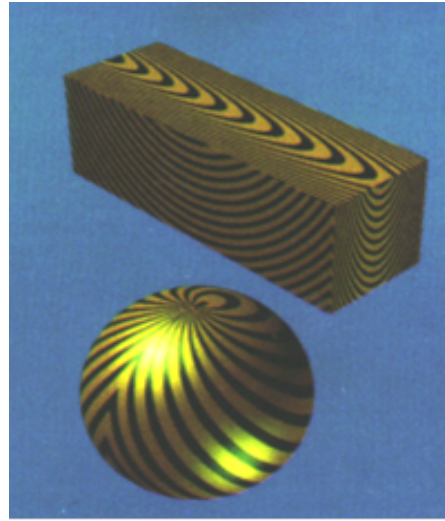
Contents

- Introduction
- 2D texture mapping
- Surface parameterization
- Polygon mapping
- Cylindrical and spherical mapping
- Two phase mapping
 - S-mapping
 - O-mapping
 - Issues
 - Examples
- Mapping through modeling
- Mapping onto bicubic parametric patches
- Mipmapping

Introduction

- ❑ Texture – graphics pattern based on matrix of pixel
- ❑ Texture mapping – transposition of graphics pattern onto the surface of an object
- ❑ Through texture are presented more attributes of the object surface, rather than simply the color by the Phong reflection model
- ❑ There are three major considerations in the texture mapping:
 1. What attribute or parameter of the model or object is to be modulated to produce the desired textural effect?
 2. Texture mapping method. Define the function of correspondence between the texture domain and the world space of the object
 3. Anti-aliasing processing

Introduction



Attributes and parameters

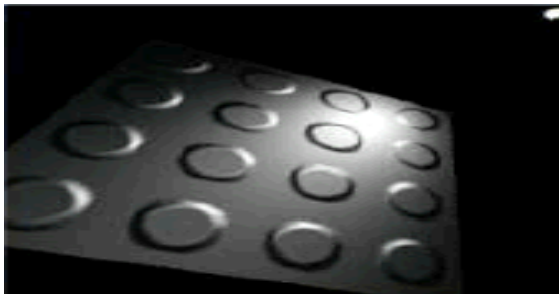
- In 1986 Heckbert classified the parameters that should be modulated to provide the textural impression:

(Heckbert P.S. "Survey of texture mapping", IEEE Computer Graphics and Applications, 6(11), pp. 56-67)

1. *Surface color* (diffuse reflection coefficients) – the most commonly used parameter for texture mapping. Proposed by Catmull in 1974
 2. *Specular and diffuse reflection* (environment mapping) – known as environment mapping. Developed by Blinn and Newell in 1976
 3. *Normal vector perturbation* – it is a modification of the reflection model, proposed by Blinn in 1978, that gives the appearance of a perturbed surface
 4. *Specularity* – modulate a function of the surface roughness. The approach was proposed by Blinn in 1978
 5. *Transparency* – proposed by Gardener in 1985. It is in fact a method of generating a complete object (e.g. a cloud, chemically etched glass)
- All texture mapping methods, excepting the environment mapping, bind the texture onto the object, and the texture then becomes a permanent part of the object, invariable under object transformations and viewing conditions

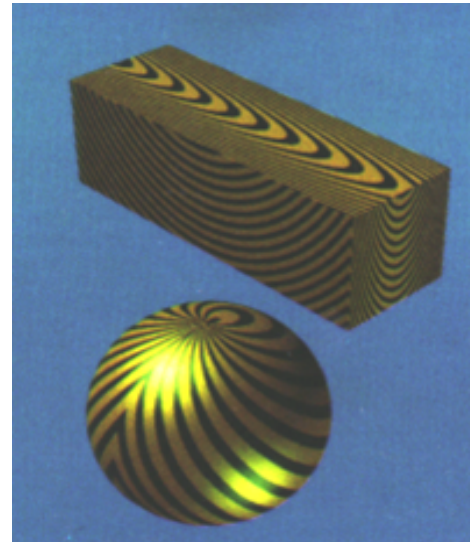
Mapping approach

1. *Bound texture* – a 2D texture pattern is mapped onto the surface of the object. Then the texture remains attached to the surface
2. *View dependent mapping* – the texture pattern watched by the viewer depends on the viewing direction. For instance, chrome mapping, environment mapping, reflection and refraction mapping
3. *Bump mapping* – the effect of surface deformation by the modification of the normal vector. Computing the light intensity by the local reflection model produces a textural effect



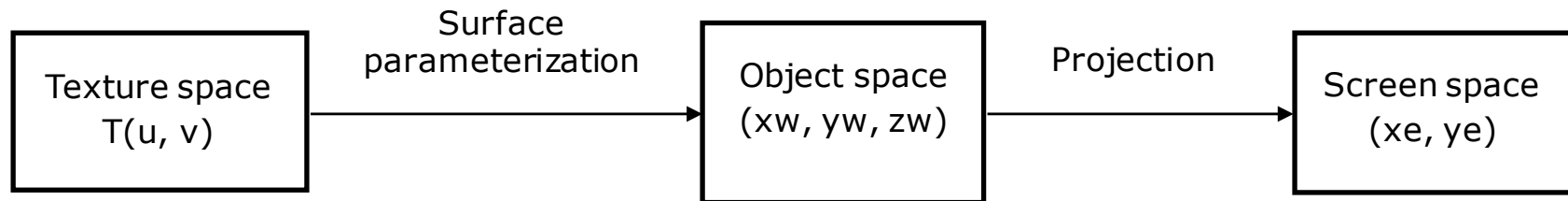
Texture space

1. 2D texture (u,v)
2. 3D texture (u,v,w)



2D texture mapping

□ 2D texture domain



□ Main issues:

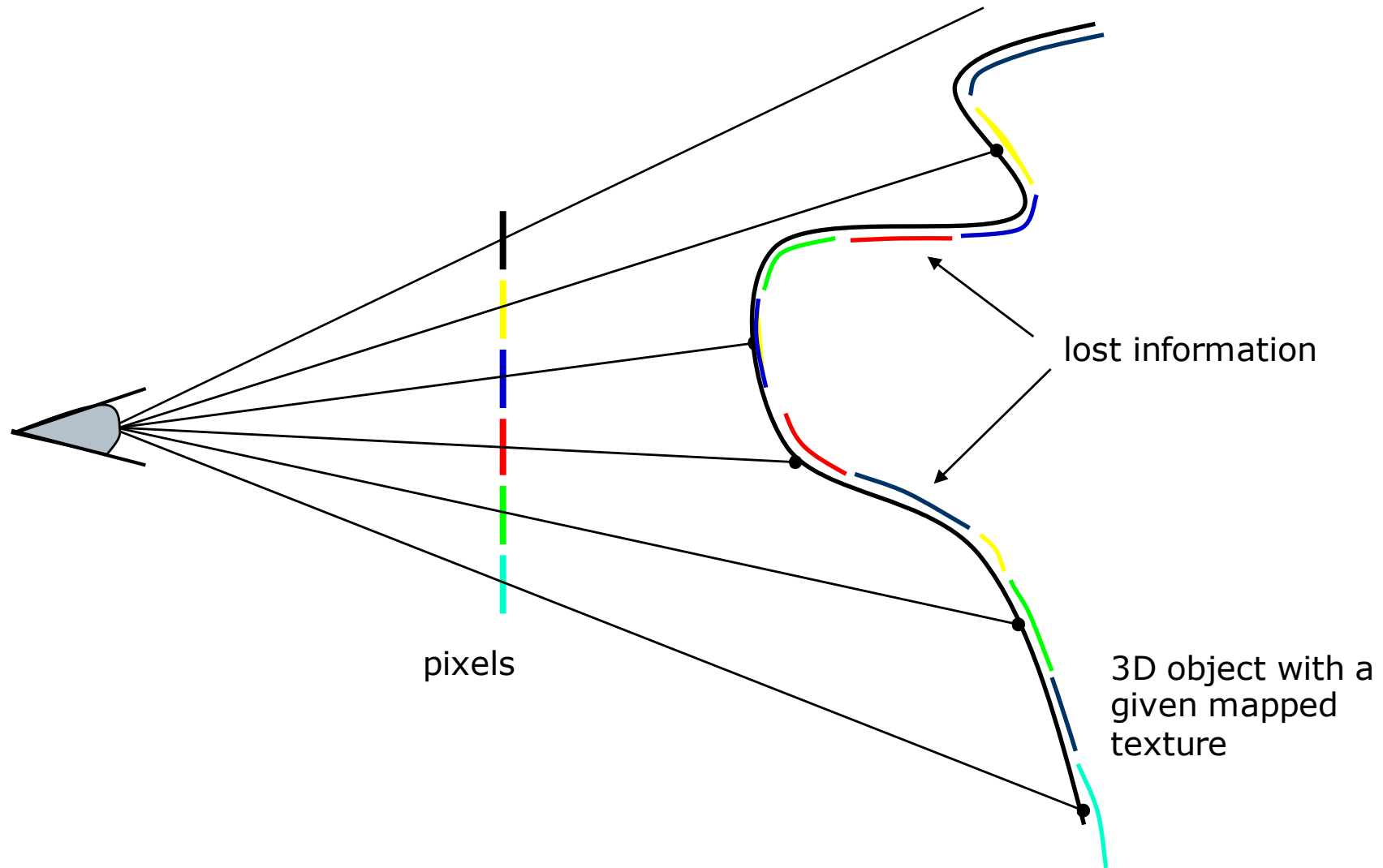
1. Surface parameterization

- Need to stick a texture pattern onto a polygon mesh object
- Difficult approximation for discontinuous real objects
- Simple for cubic parametric surfaces

2. Anti-aliasing

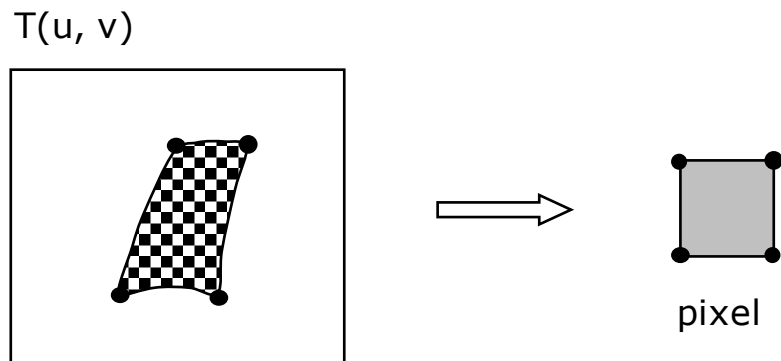
- Mapping a texture onto adjacent polygons
- Projection of an area into a pixel -> lost information
- Keep graphical quality even by lost information projection

Texture mapping - antialiasing

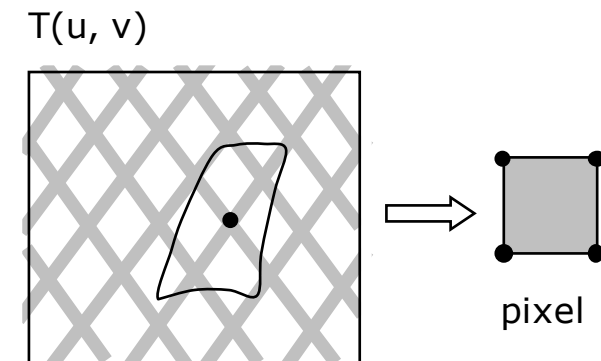
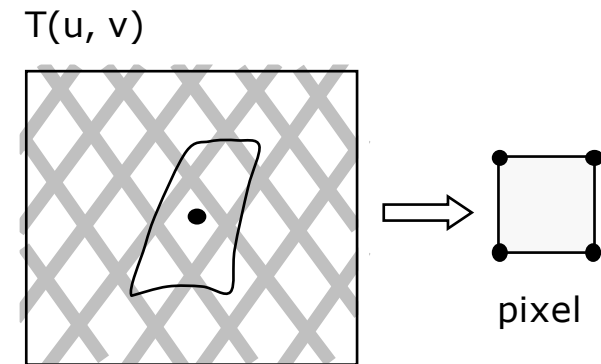


2D texture mapping

- Anti-aliasing
- Computing the pixel value:
 - a) Intensity is a function of the filling factor.
 - b) Pixel value depends on the mapping of the pixel center.



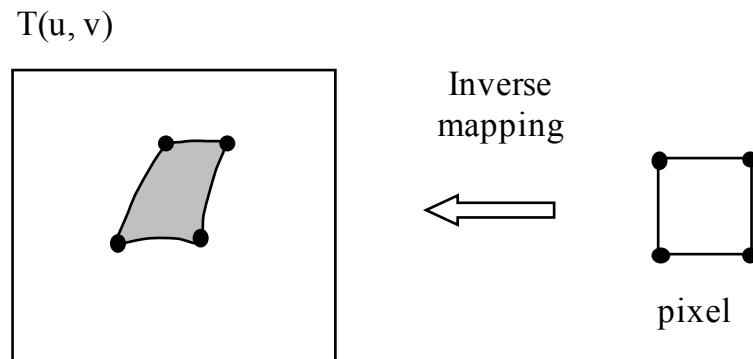
(a)



(b)

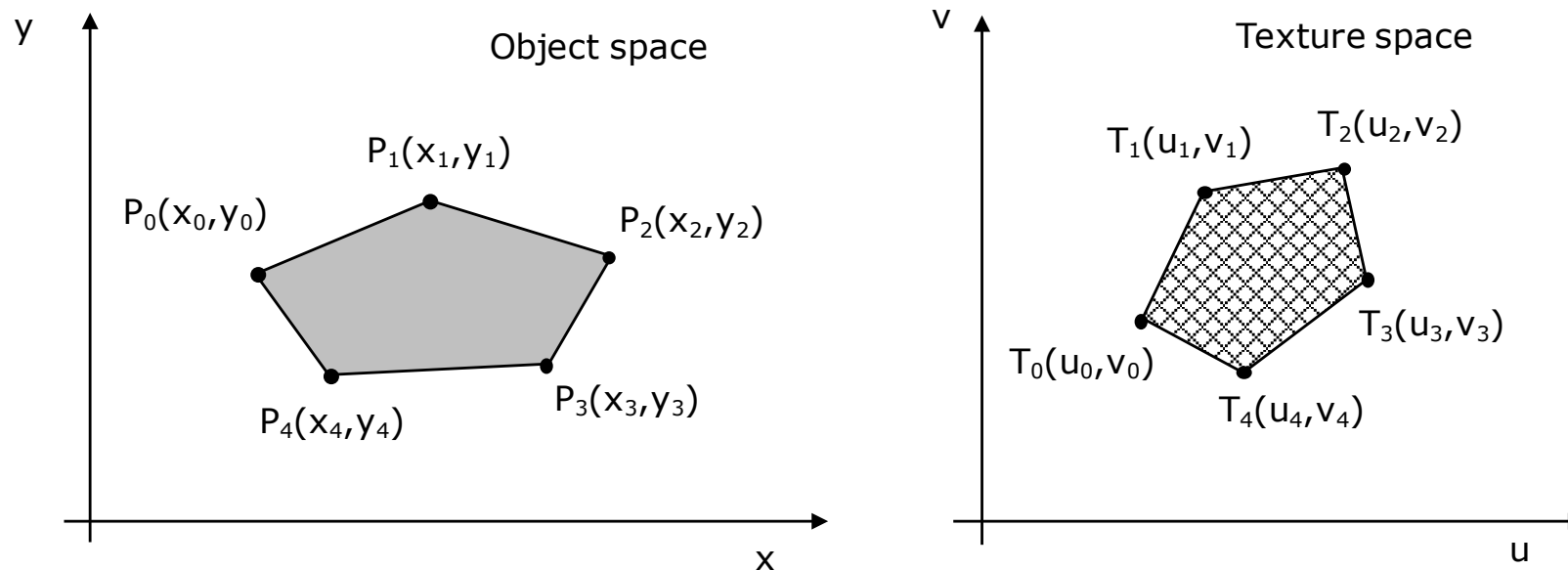
2D texture mapping

- Inverse mapping
 - Find the pre-image of the current pixel in the texture domain
 - Non-linear transformation
 - Pixel maps into an area in texture space that in general is a curvilinear quadrilateral
 - Steps:
 1. Consider four pixel corner points
 2. Invert the object to screen space
 3. Invert the surface parameterization



Surface parameterisation

- ❑ Associates a point on the object (x_w, y_w, z_w) with a point (u, v) in texture space
- ❑ Polygon mapping
 1. Local problem: mapping each interior point in the polygon into a point in $T(u, v)$ space
 2. Global problem: mapping each polygon, for an entire object, from a number of connected polygons



Surface parameterisation

□ Problem:

- knowing the mapping of the object vertices into the texture space let us find the mapping of each point in the polygon for operations like: Simple rendering, clipping, scale, and deformation
- Case1: the mapping function for each point is already known – map the clipped polygon as the initial polygon by the rendering of each point
- Case 2: the mapping function is unknown – should determine the analytical function of correspondence
 1. Linear toward some direction
 2. Non-linear: divide the initial polygon into a set of linear variation areas

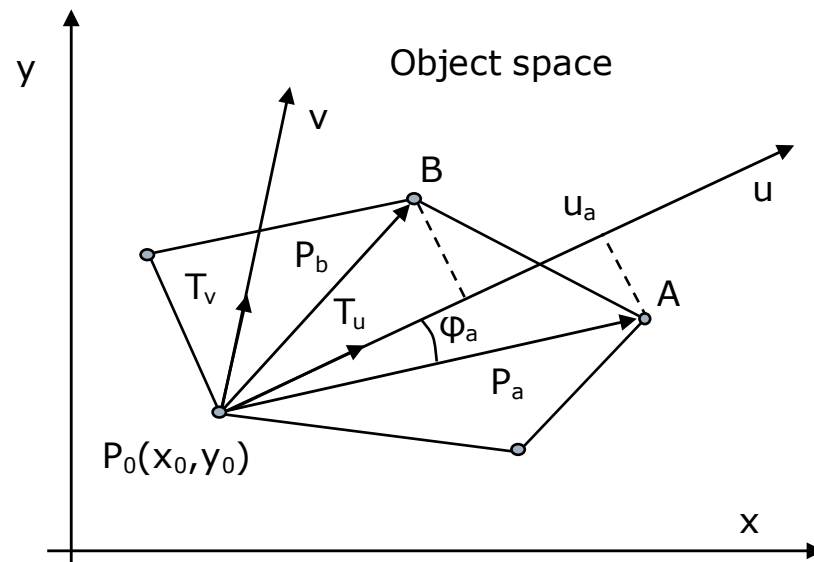
Polygon mapping – linear mapping

Linear variation of u by x and v by y , and (x_0, y_0) maps onto (u_0, v_0) :

$$u = T_u \cdot (x - x_0) + u_0$$

$$v = T_v \cdot (y - y_0) + v_0$$

where T_u and T_v are the directions for linear variations of u and v in the object space. Let us compute the T_u and T_v vectors.



Polygon mapping

Considering the vertex A, let us compute the u_a coordinate in texture space as the projection of vector P_a onto u axis. T_u is a normalized vector:

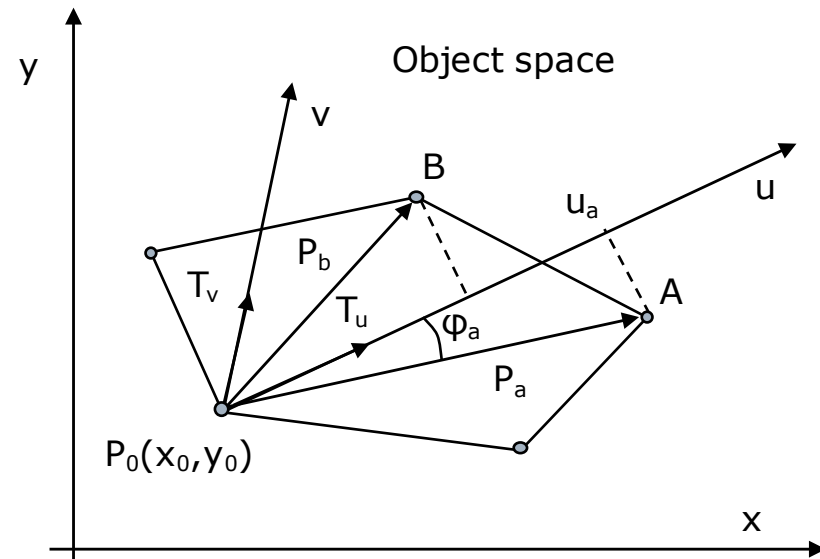
$$u_a = T_u \cdot P_a + u_0 = |T_u| \cdot |P_a| \cos \phi_a + u_0$$

or
$$u_a = t_{ux} (x_a - x_0) + t_{uy} (y_a - y_0) + u_0$$

To compute $T_u(t_{ux}, t_{uy})$ we need to know the texture coordinates at least for two positions A and B in object space. Let be A and B two polygon vertices:

$$u_a = t_{ux} (x_a - x_0) + t_{uy} (y_a - y_0) + u_0$$

$$u_b = t_{ux} (x_b - x_0) + t_{uy} (y_b - y_0) + u_0$$



Polygon mapping

Resolving the two equations for t_{ux} and t_{uy} :

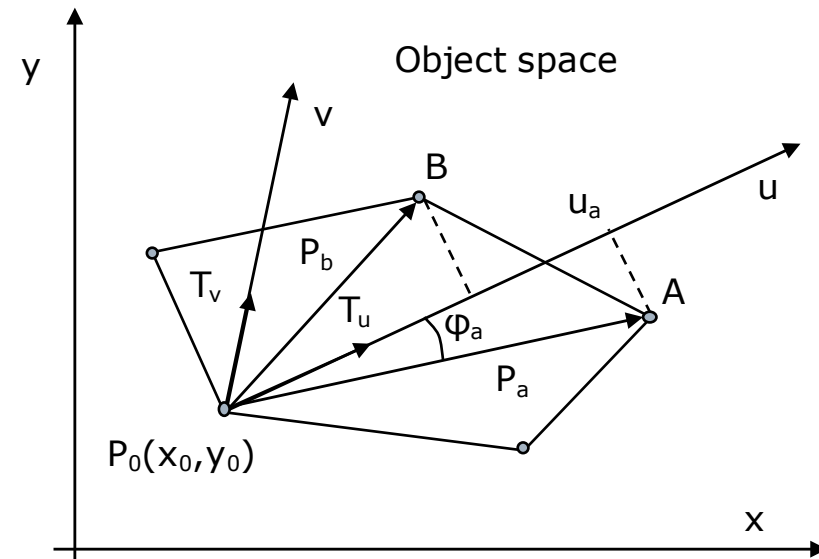
$$t_{ux} = \frac{\frac{u_a - u_0}{y_a - y_0} - \frac{u_b - u_0}{y_b - y_0}}{\frac{x_a - x_0}{y_a - y_0} - \frac{x_b - x_0}{y_b - y_0}} \quad t_{uy} = \frac{\frac{u_a - u_0}{x_a - x_0} - \frac{u_b - u_0}{x_b - x_0}}{\frac{x_a - x_0}{y_a - y_0} - \frac{x_b - x_0}{y_b - y_0}}$$

Similarly compute $T_v(t_{vx}, t_{vy})$.

Therefore, a point $P(x, y)$ in the object space has the texture coordinates:

$$u = t_{ux}(x - x_0) + t_{uy}(y - y_0) + u_0$$

$$v = t_{vx}(x - x_0) + t_{vy}(y - y_0) + v_0$$



Cylindrical and spherical mappings

□ Cylindrical mapping

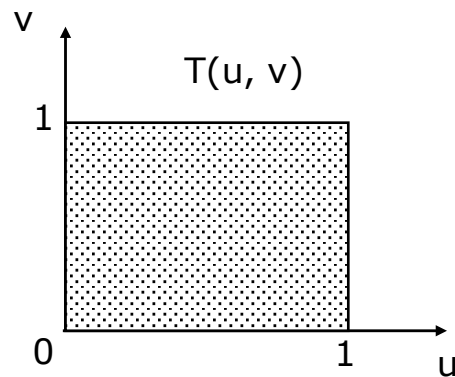
A point $P(x, y, z)$ on the cylinder surface is represented as:

$$(r \cos\theta, r \sin\theta, hs)$$

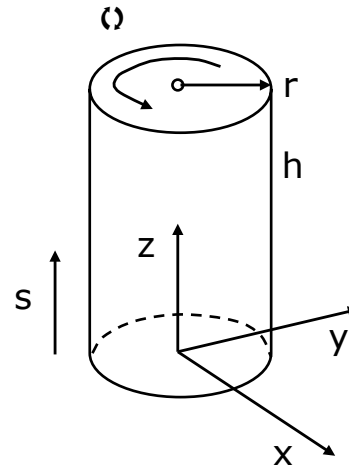
where r radius, h height, $0 < \theta \leq 2\pi$ and $0 < s \leq 1$

The inverse mapping is:

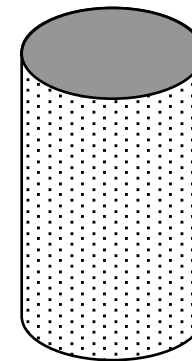
$$(u, v) = (\theta/2\pi, s), \quad u, v \in [0, 1]$$



(a) Texture domain



(b) Cylinder object



(c) Texture mapped on the cylindrical surface

Cylindrical and spherical mappings

■ Spherical mapping

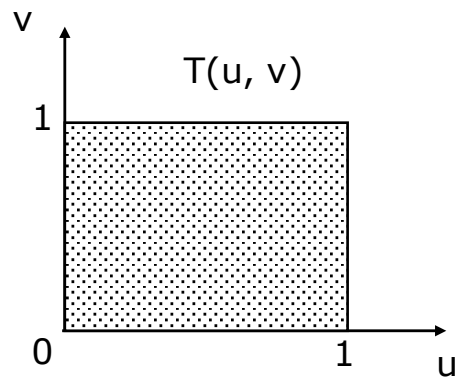
A point $P(x, y, z)$ on the spherical surface is represented as:

$$(r \cos\theta \sin\phi, r \sin\theta \sin\phi, r \cos\phi)$$

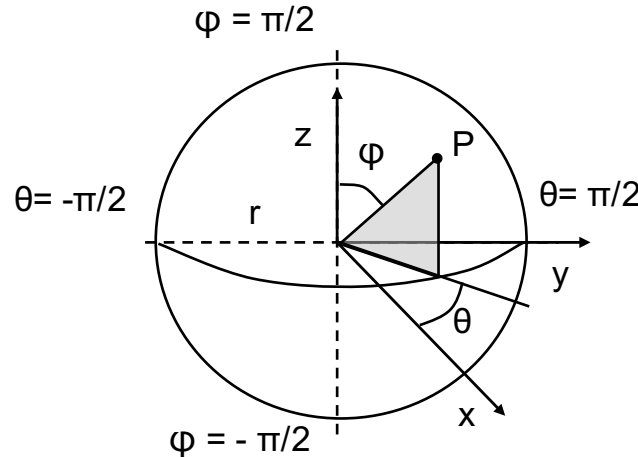
where r radius, $-\pi < \theta \leq \pi$, $-\pi/2 < \phi \leq \pi/2$

The inverse mapping is:

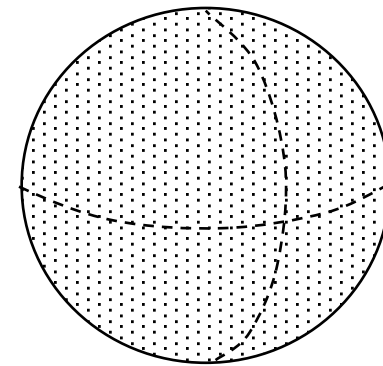
$$(u, v) = ((\theta + \pi)/2\pi, (\phi + \pi/2)/\pi), \quad u, v \in [0, 1]$$



(a) Texture domain



(b) Sphere object



(c) Texture mapped on the spherical surface

Cylindrical and spherical mappings

□ Proposed exercises:

1. Compute the inverse mapping of a cylinder surface for
 $\theta_0 < \theta \leq \theta_{\max}$ and $s_0 < s \leq s_{\max}$
2. Compute the inverse mapping of a spherical surface for
 $0 < \theta \leq \pi/2$ and $\pi/4 < \varphi \leq \pi/2$
3. Compute the inverse mapping of a spherical surface for
 $\theta_0 < \theta \leq \theta_{\max}$ and $\varphi_0 < \varphi \leq \varphi_{\max}$

Two phase mapping

- Proposed in 1986, by E.A. Bier and K.R. Sloan
- Overcomes the global mapping problem
 - mapping each polygon, for an entire object, from a number of connected polygons
- Unifies texture mapping and environment mapping
- Two phases:

1. S mapping

Maps the 2D texture space to a simple 3D intermediate surface

$$T(u,v) \rightarrow T'(x_i, y_i, z_i)$$

2. O mapping

Maps the 3D texture pattern onto the object surface

$$T'(x_i, y_i, z_i) \rightarrow O(x_w, y_w, z_w)$$

Two phase mapping – S mapping

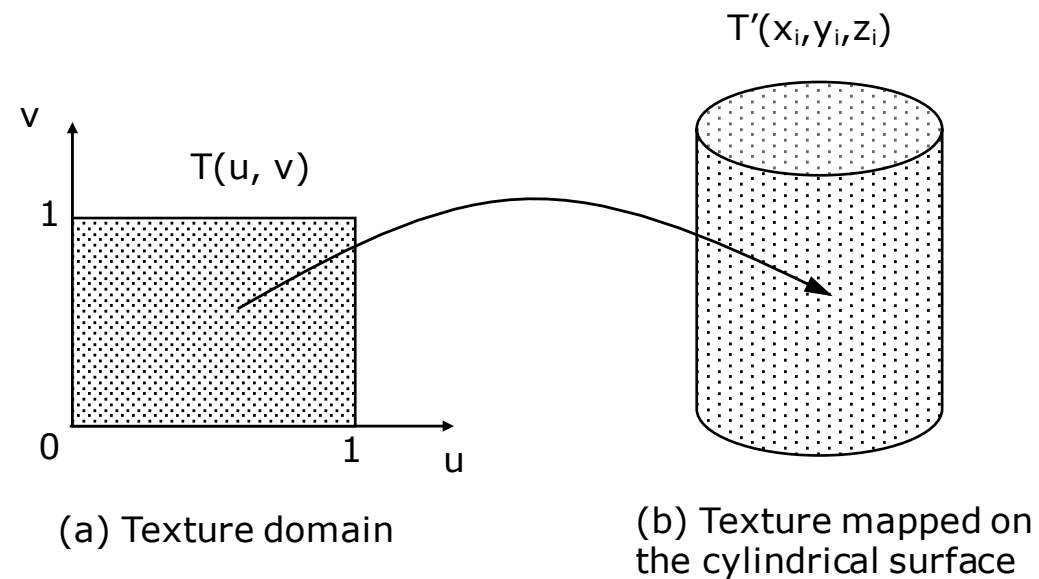
□ S mapping

Maps the 2D texture space to a simple 3D intermediate surface

$$T(u,v) \rightarrow T'(x_i, y_i, z_i)$$

□ Intermediate surfaces (Bier):

1. Plane at any orientation
2. Curved surface of a cylinder
3. Faces of a cube
4. Surface of a sphere

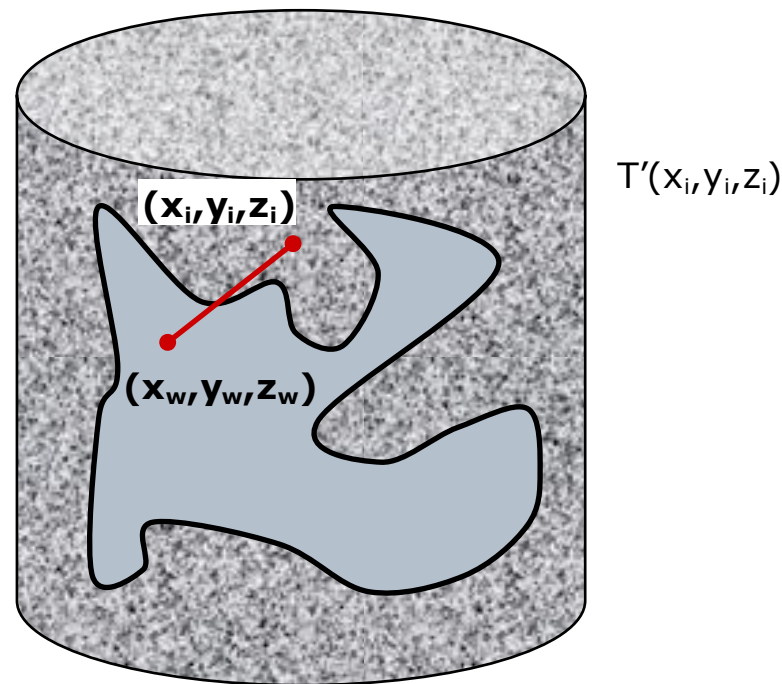


Two phase mapping – O mapping

□ O mapping

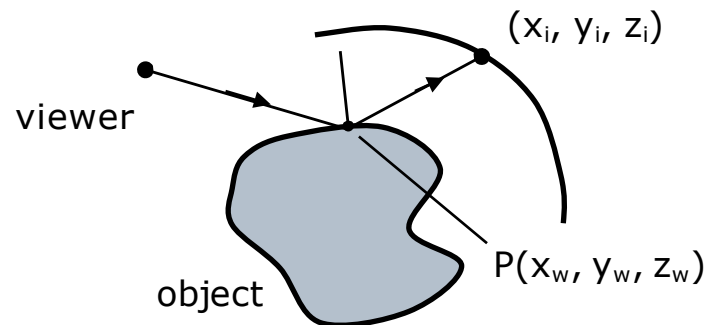
Maps the 3D texture pattern onto the object surface

$$T'(x_i, y_i, z_i) \rightarrow O(x_w, y_w, z_w)$$

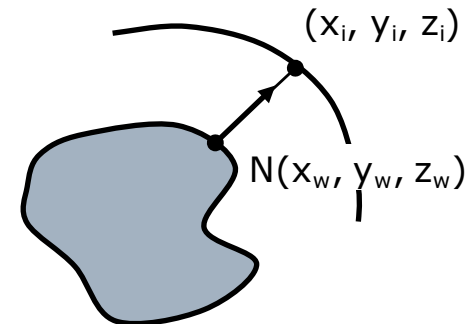


Two phase mapping – O mapping

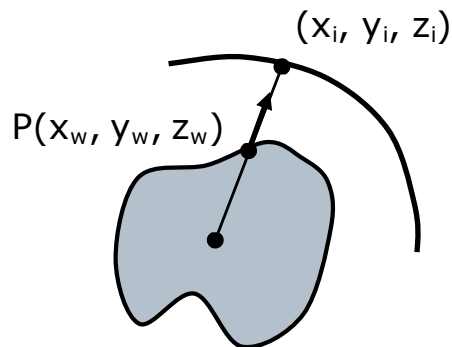
Intermediate surface T'



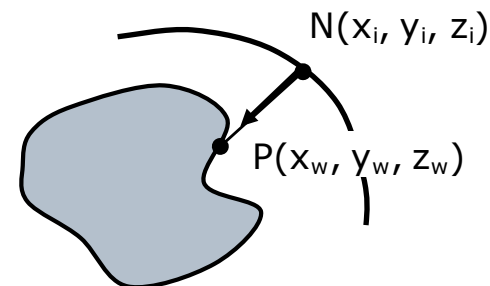
(a) Intersection of the reflected view ray with the intermediate surface T' (environment mapping)



(b) Intersection of the surface normal with T'



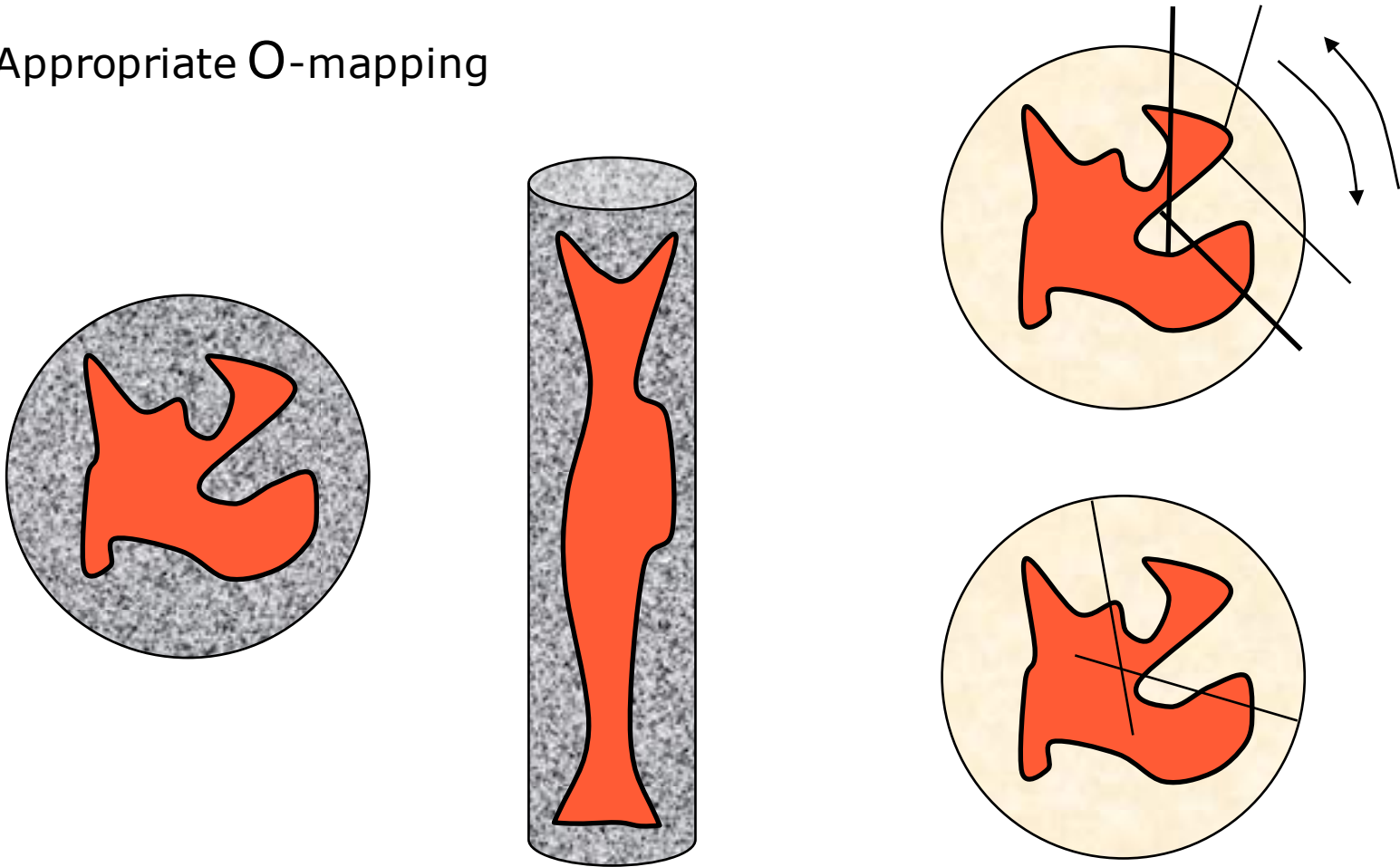
(c) Intersection of the line through object point and the object centroid with T'



(d) Intersection of the line from object point to T' whose orientation is given by the intermediate surface normal

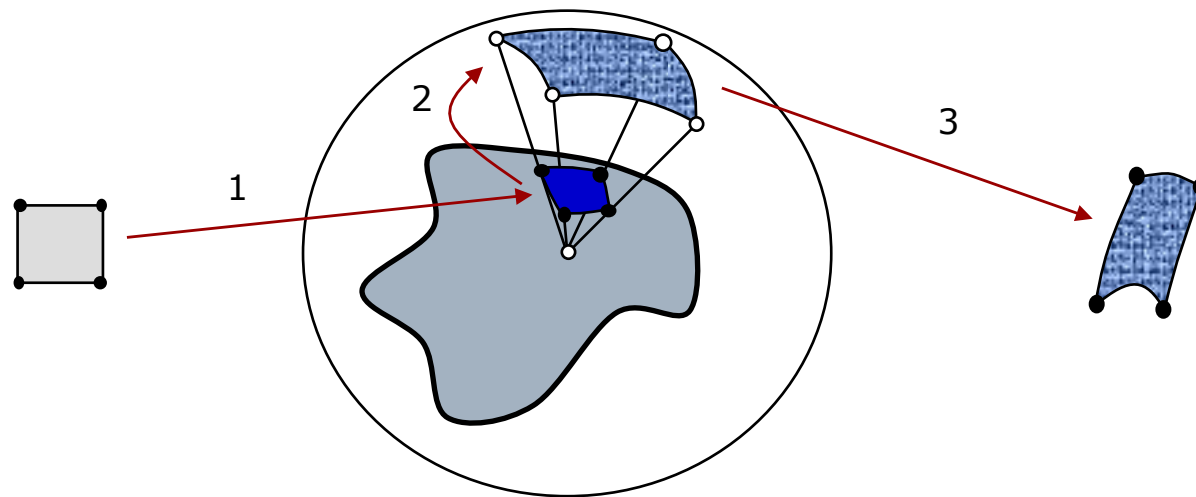
Two phase mapping – issues

- Appropriate intermediate surface
- Appropriate O-mapping



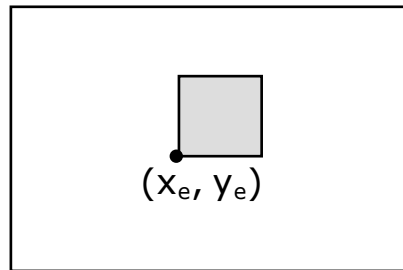
Two phase inverse mapping – example

- ❑ Intermediate surface: sphere
- ❑ O-mapping: intermediate surface normal
- ❑ Inverse mapping:
 1. Inverse mapping for all 4 pixel vertices onto 4 object points (x_w, y_w, z_w) , $a \rightarrow b$
 2. O-mapping to compute the point (θ, ϕ) onto the sphere surface, $b \rightarrow c$
 3. S-mapping to determine the correspondent position (u, v) in the texture domain for the (θ, ϕ) point, $c \rightarrow d$

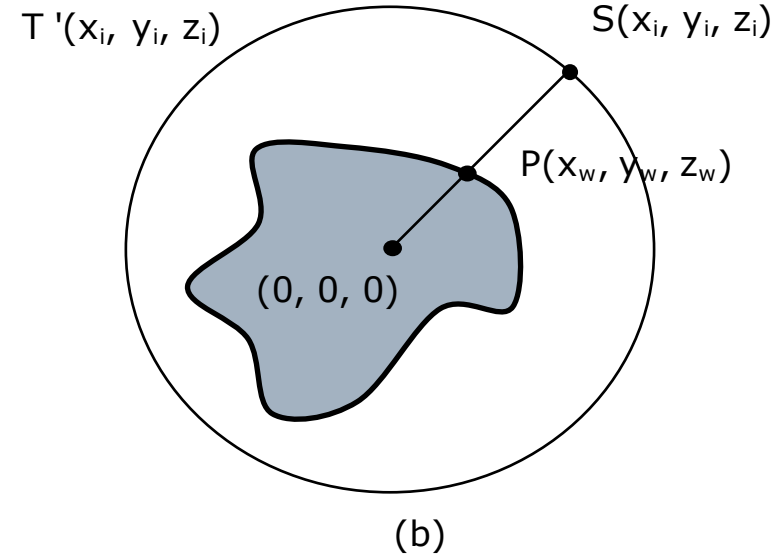


Two phase inverse mapping – example

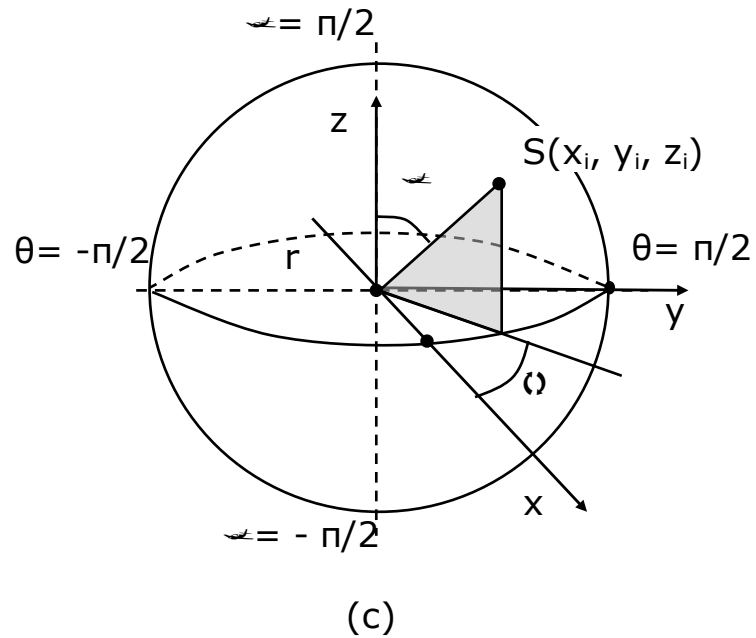
Screen space



(a)

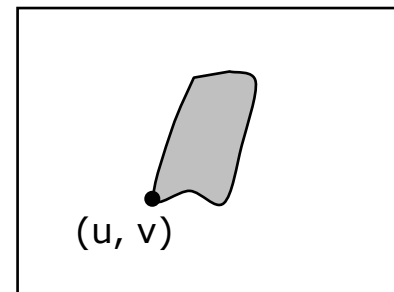


(b)



(c)

$T(u, v)$



(d)

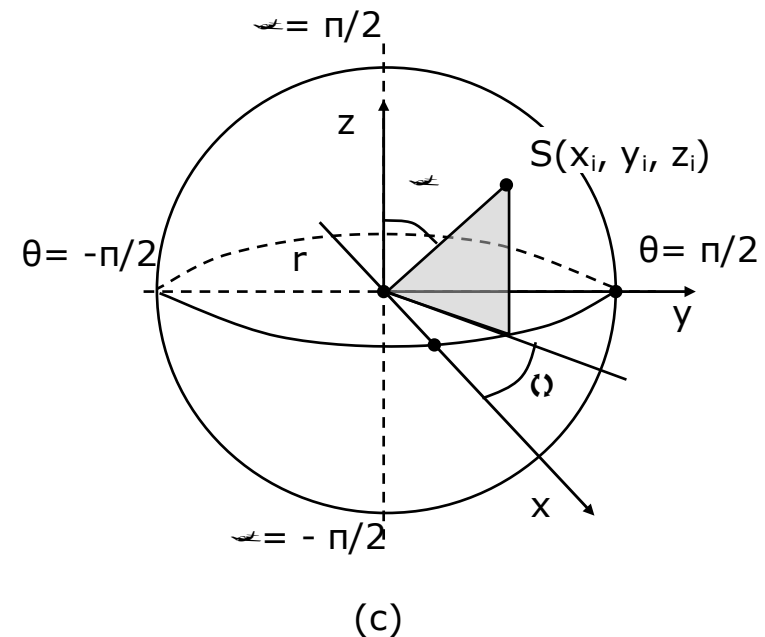
Two phase inverse mapping – example

$$\frac{x_w}{x_i} = \frac{y_w}{y_i} = \frac{z_w}{z_i} = \frac{\sqrt{x_w^2 + y_w^2 + z_w^2}}{r}$$

$$x_i = \frac{r x_w}{\sqrt{x_w^2 + y_w^2 + z_w^2}}$$

$$y_i = \frac{r y_w}{\sqrt{x_w^2 + y_w^2 + z_w^2}}$$

$$z_i = \frac{r z_w}{\sqrt{x_w^2 + y_w^2 + z_w^2}}$$



Compute the spherical coordinates θ and φ :

$\text{tg } \theta = y_i / x_i = y_w / x_w$, and then $\theta = \text{arctg}(y_w / x_w)$

$\text{tg } \varphi = [(x_w^2 + y_w^2)^{1/2} / z_w]$, and then $\varphi = \text{arctg}[(x_w^2 + y_w^2)^{1/2} / z_w]$

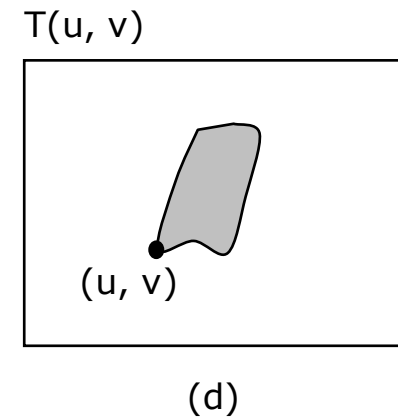
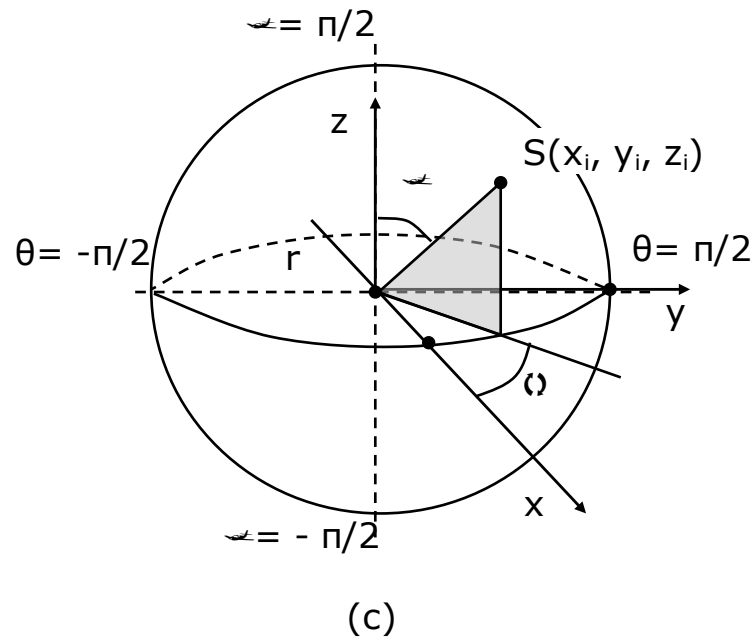
Two phase inverse mapping – example

Compute (u,v) by inverse S-mapping (θ,φ) :

$$(\theta, \varphi) \rightarrow (u, v)$$

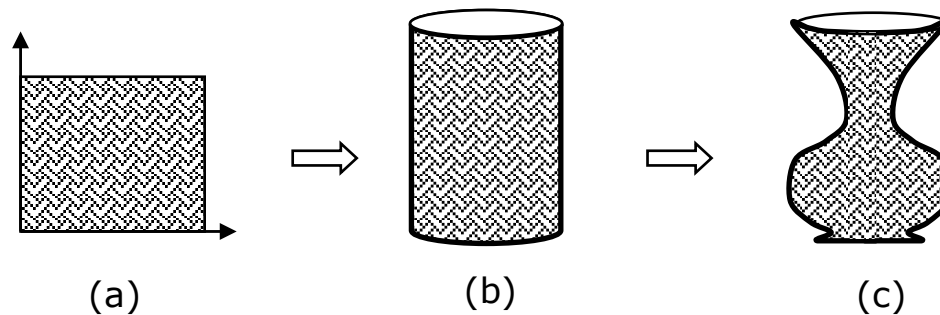
$$u = \theta / 2\pi = 1/2\pi \arctg (y_w / x_w)$$

$$v = (\varphi + \pi/2) / \pi = 1/\pi \arctg [(x_w^2 + y_w^2)^{1/2} / z_w] + 1/2$$



Mapping through modeling

- Appropriate for objects that can be shaped from a deformed cylinder, sphere or another geometry
- Processing steps:
 1. Model the object as a set of cylinders and spheres
 2. Map the textures onto the object components – cylinders and spheres. The texture becomes fixed onto the object surface. The graphics model consists of bounded textures and objects
 3. Deformed by matrix based operations the objects and therefore the bounded textures

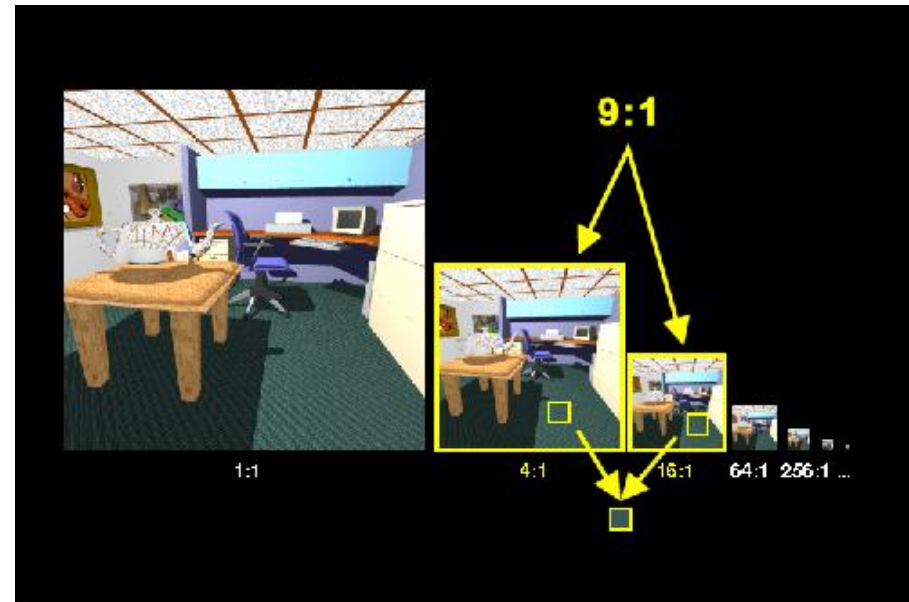
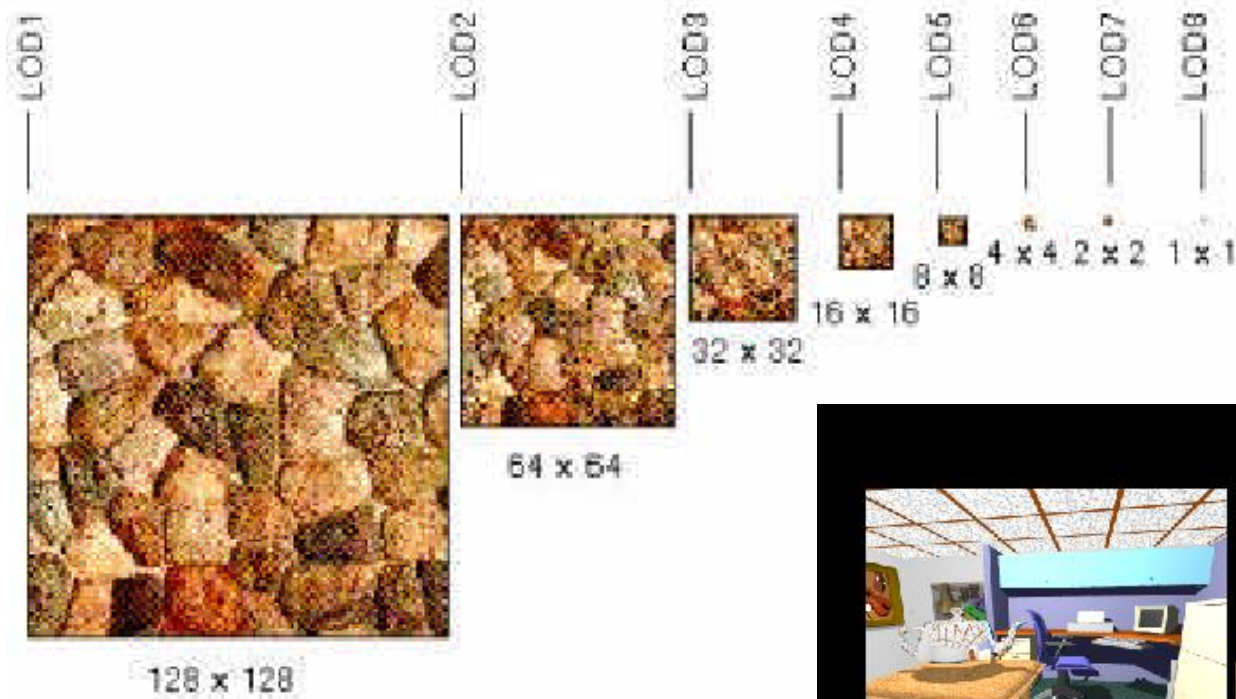


Mapping onto bicubic parametric patches

- ❑ Developed by Catmull in 1974
- ❑ Object is a bicubic parametric patch
- ❑ Texture mapping is trivial since a parametric patch definition is by u and v
- ❑ Mapping approach:
 1. Subdivide a surface patch in object space
 2. At the same time executes a corresponding subdivision in texture space
 3. Patch subdivision proceeds until it covers a single pixel
 4. The required texture value is obtained from the area enclosed by the current level of subdivision in the texture domain
- ❑ Another approach developed by Cook, Carpenter and Catmull in 1987
 - The object surfaces are subdivided into micro polygons and flat shaded with values from a corresponding subdivision in texture space

Mipmapping

- Texture depends on the LOD (Level of Details)



Questions and problems

1. Explain and exemplify the terms “texturing” and “texture mapping”.
2. What means the terms “fix texture” and “view dependent mapping”?
3. Explain the terms “surface parameterization” and “projection” of the direct texture mapping process.
4. Explain the antialiasing process. Why the rasterisation process loose information?
5. Explain the reason for the process of inverse mapping.
6. Explain the texture mapping onto a surface if the function of correspondence is: (a) linear toward two directions; (b) non-linear.
7. Compute the inverse mapping of a spherical surface for

$$0 < \theta \leq \pi/2 \text{ and } \pi/4 < \varphi \leq \pi/2$$

Questions and problems

8. Propose a rule based O mapping in order to resolve the main issues for the two phase inverse mapping onto concave 3D objects.
9. Why the two phase inverse mapping considers a pixel as a polygon?
10. Explain how the texture mapping could depends on LOD (Level Of Details)?

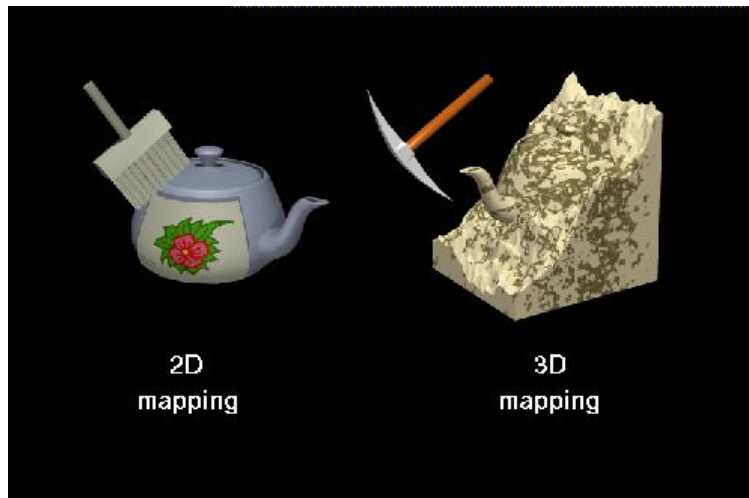
Texture Mapping (2)

Contents

- 3D texture mapping
- Hypertextures
- Noise in 3D texture definition
- View dependent texture mapping
 - Reflection mapping (chrome)
 - Refraction mapping
 - Environment mapping
- Bump mapping

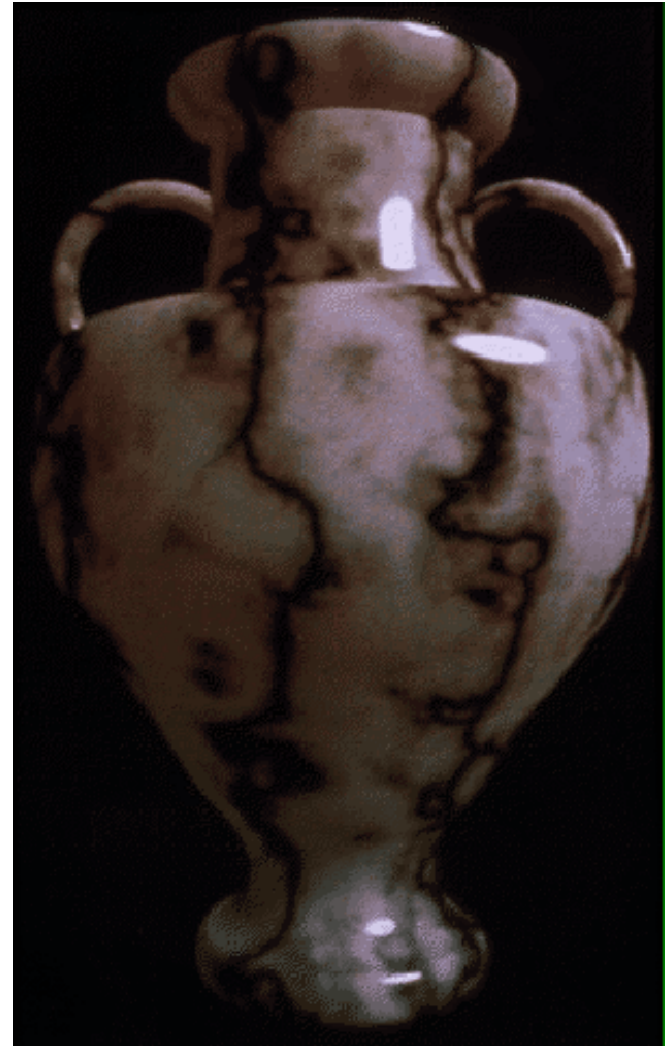
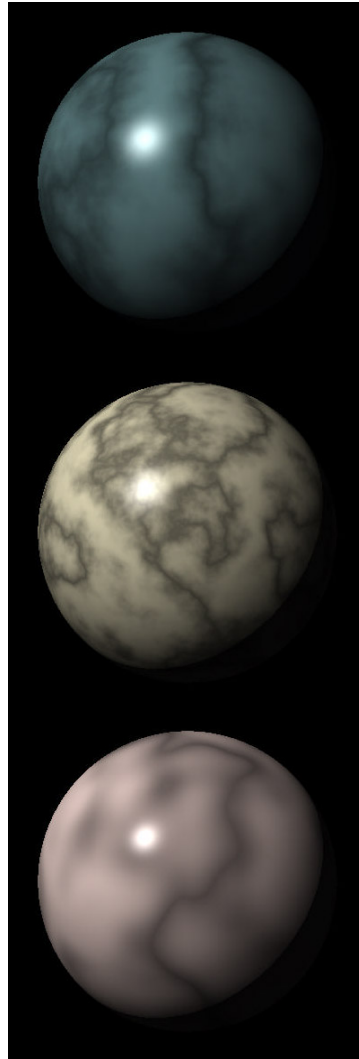
3D texture mapping

- ❑ Developed by D.R. Peachey in 1983 and by K. Perlin in 1985
- ❑ Texture – texture value exists everywhere in the object domain
- ❑ Given a point on the surface of an object (x_w, y_w, z_w) the texture value is given by the identity mapping $T(x_w, y_w, z_w)$
- ❑ Equivalent with sculpting an object out of a block of material
- ❑ The color of the object is determined by the intersection of its surface with the predefined 3D texture field
- ❑ Usually the 3D texture space is obtained by procedural generation



Examples from the course Computer Graphics, by Pascal Vuylstekker, AN Univ., 2003.

3D texture mapping - examples



3D texture mapping – example

Wood grain simulation

1. A cylinder:

$$r_1 = (u^2 + w^2)^{1/2}$$

2. Perturb r with a sinusoidal function

$$r_1 = r_1 + 2 \sin(a\theta)$$

3. Twist along the axis of the cylinder

$$r_1 = r_1 + 2 \sin(a\theta + v/b)$$

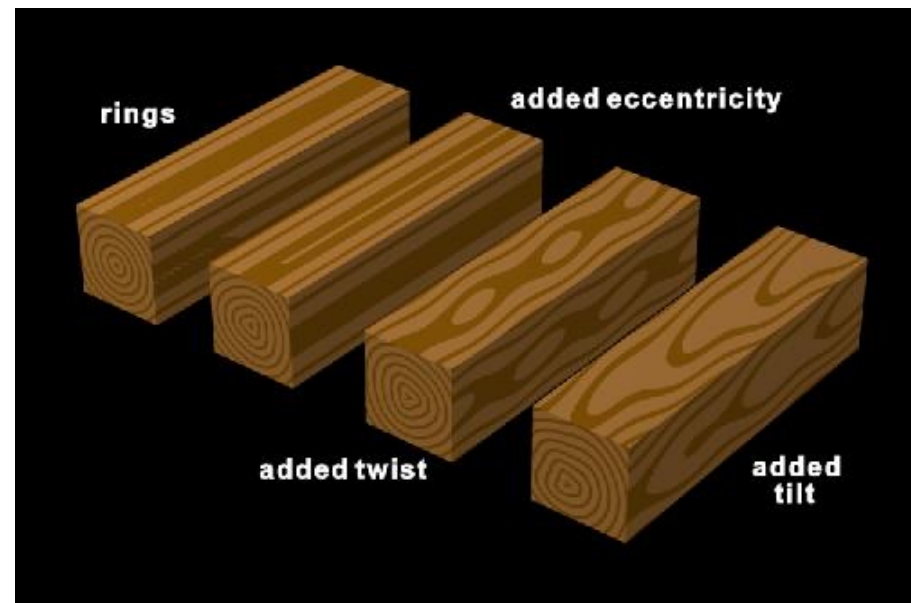
Where a , b are constants

$$\theta = \tan^{-1}(u/w)$$

4. Index the texture

$$(x,y,z) = \text{tilt}(u,v,w)$$

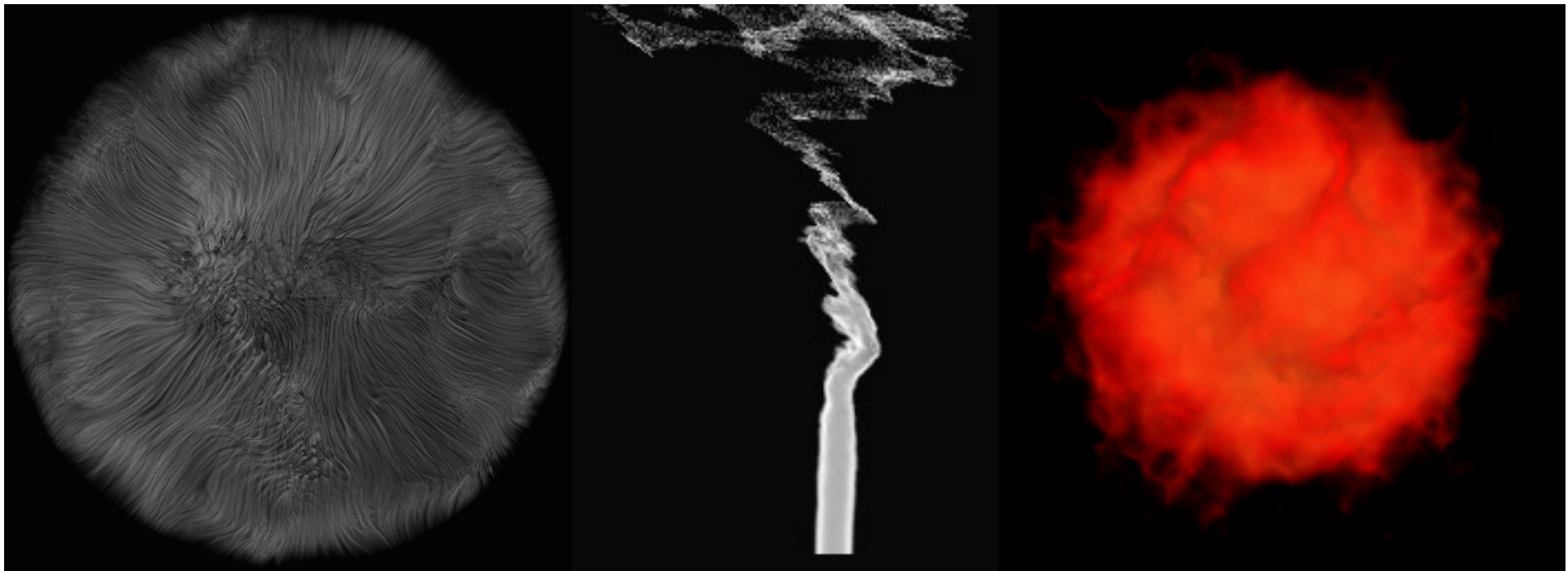
Where “tilt” is a function implementing any rotation between the two coordinate systems



Examples from the course Computer Graphics, by Pascal Vuylstekker, AN Univ., 2003.

Hypertextures

- ❑ Hypertextures – 3D textures introduced by K. Perlin in 1989
- ❑ Objects do not have surface
- ❑ The object surface is defined by a density function and the texture is evaluated within the whole object space
- ❑ Example of modeling: hair, fur, fire, glass, flowing fluids, etc.



Reference: Textures produced by Ken Perlin, NYU Media Research Lab

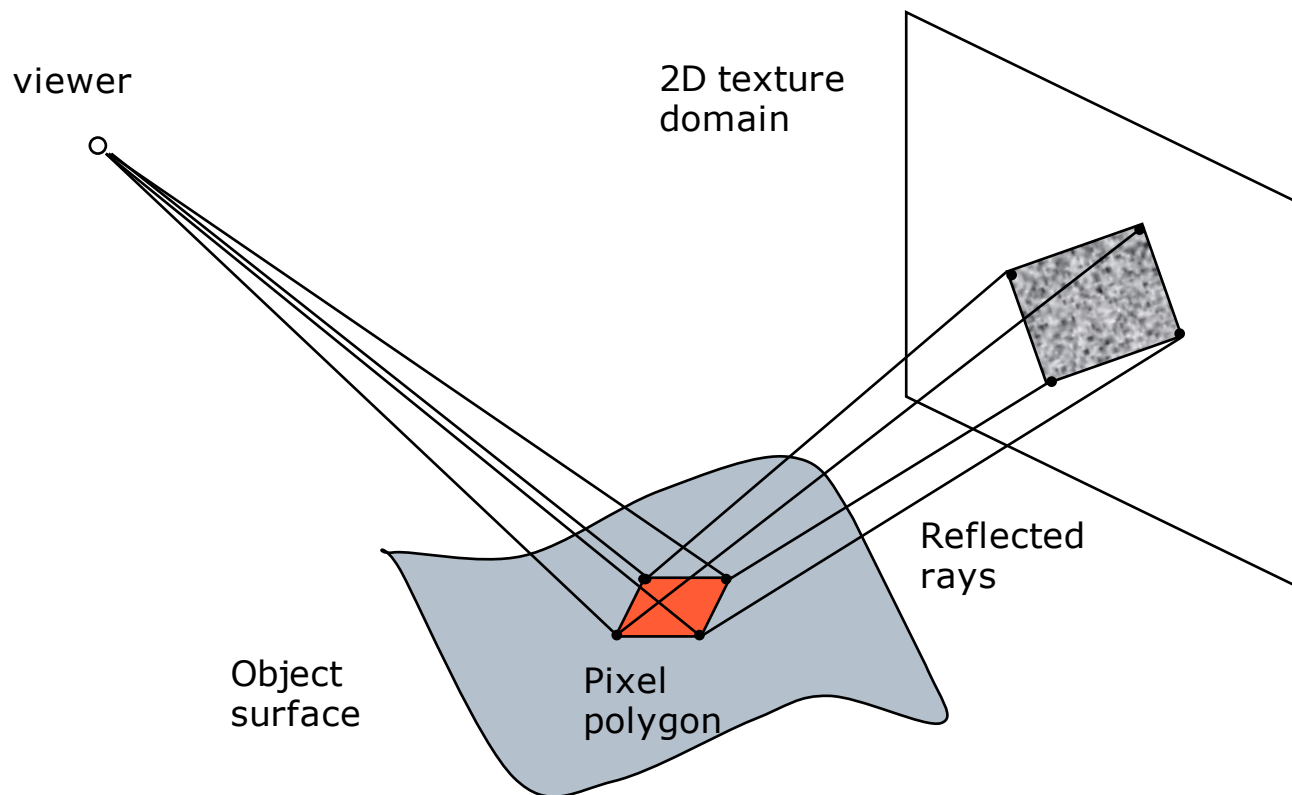
3D texture and noise

Examples from the course Computer Graphics, by Pascal Vuylsteker, AN Univ., 2003.



View dependent texture mapping

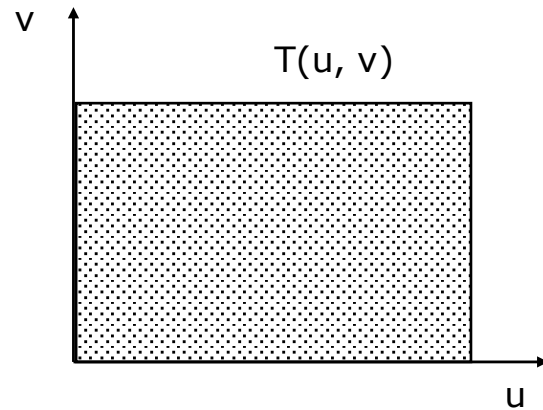
- Texture mapping by reflection (chrome mapping) and refraction of viewing ray
- Ex. Environment mapping



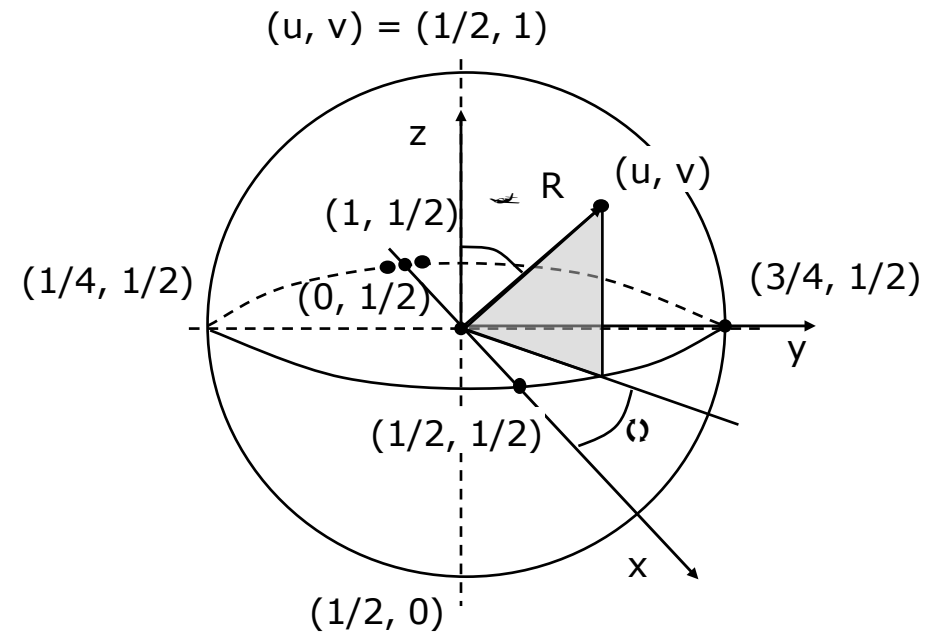
View dependent mapping - examples



Chrome mapping - example

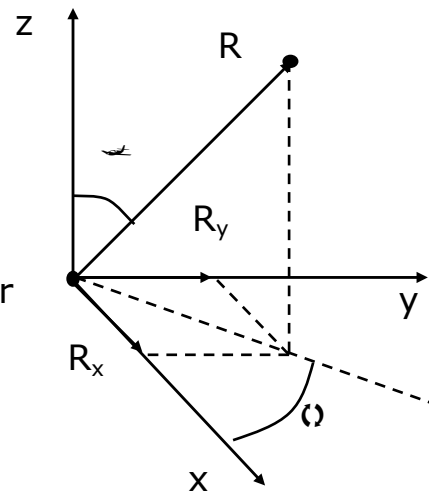


(a) Texture domain



(b) Intermediate surface mapping

(c) Specular reflection vector in spherical coordinates



Chrome mapping - example

Chrome mapping through an intermediate surface as a sphere

Inverse mapping:

$$(u, v) = ((\theta + \pi)/2\pi, (\varphi + \pi/2)/\pi), \quad u, v \in [0, 1]$$

where:

- $\pi < \theta \leq \pi$ azimuth angle
- $-\pi/2 < \varphi \leq \pi/2$ vertical angle

Considering the unitary R vector, it can be written:

- $-\pi < \theta = \arctg(Ry / Rx) \leq \pi$
- $-1 < Rz \leq 1$

Then the inverse mapping based on the reflection vector:

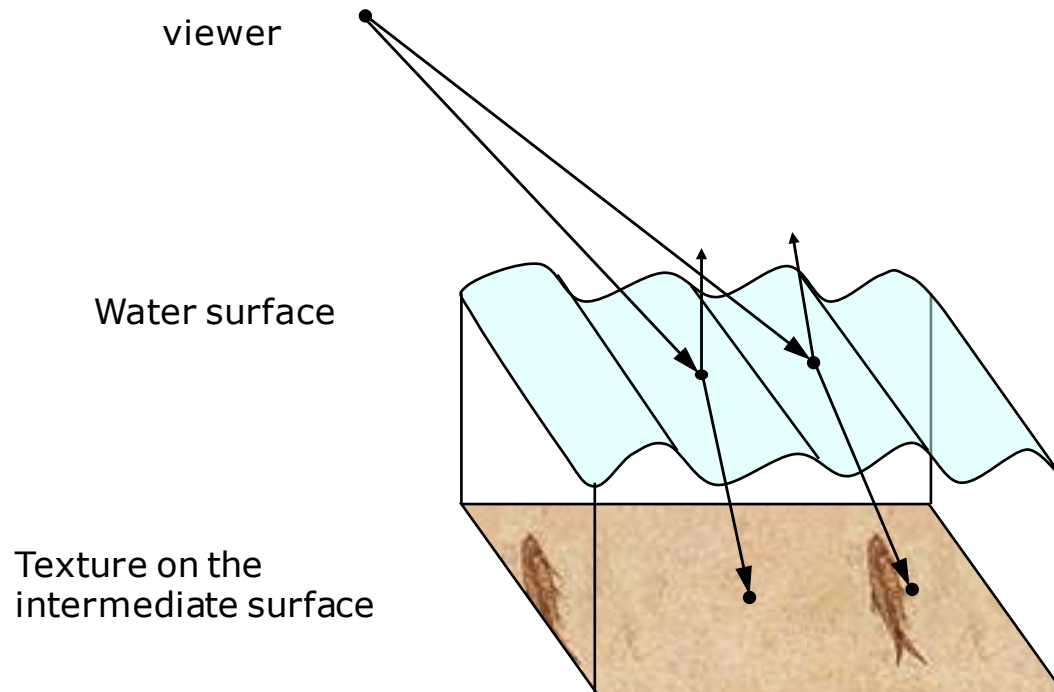
$$u = 1/2 (1 + 1/\pi \arctg(Ry / Rx))$$
$$v = 1/2 (1 + Rz)$$

Chrome mapping approach:

1. Compute the normal vector N for the P point on the object
2. Compute the specular reflection vector R as a function of N and viewer's position
3. Compute the u and v coordinates in the texture domain

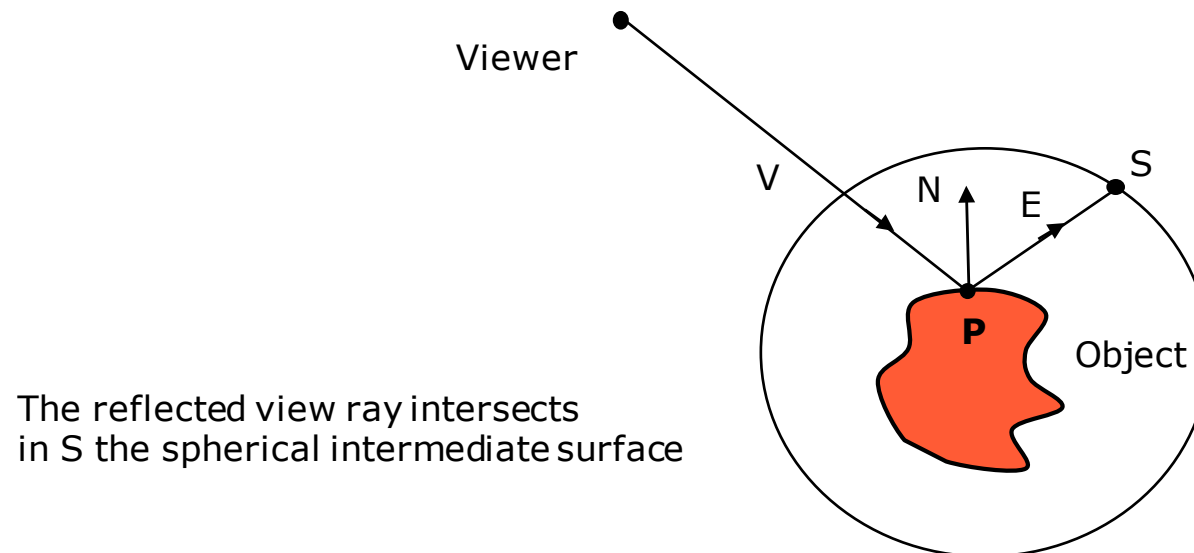
Refraction mapping

- ❑ Compute the refracted rays
- ❑ Intermediate surface is a plane hit by the refracted ray



Environment mapping

- ❑ Introduced by J.F. Blinn and M.E. Newell in 1976. The intermediate surface is the interior surface of a sphere
- ❑ In 1984, G.S. Miller and C.R. Hoffman, and in 1986, N. Greene and P.S. Heckbert used as intermediate surface a cube
- ❑ A simplification of the ray tracing method



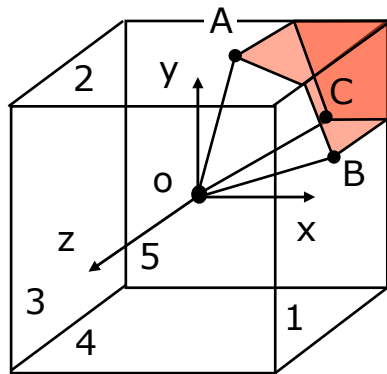
Environment mapping - example



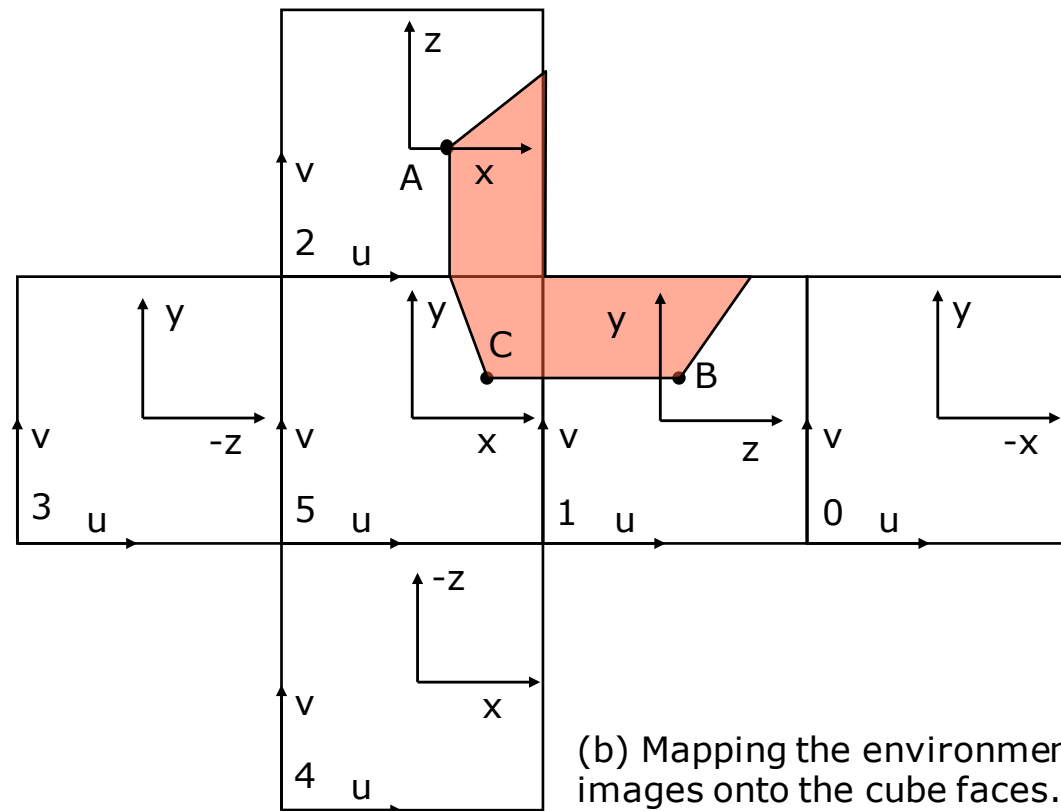
Environment mapping by a cube

- Transformation of the point $(x, -0.5, z)$ on the normalized cube into texture domain uv :

$$u = x + 0.5, v = -z + 0.5.$$

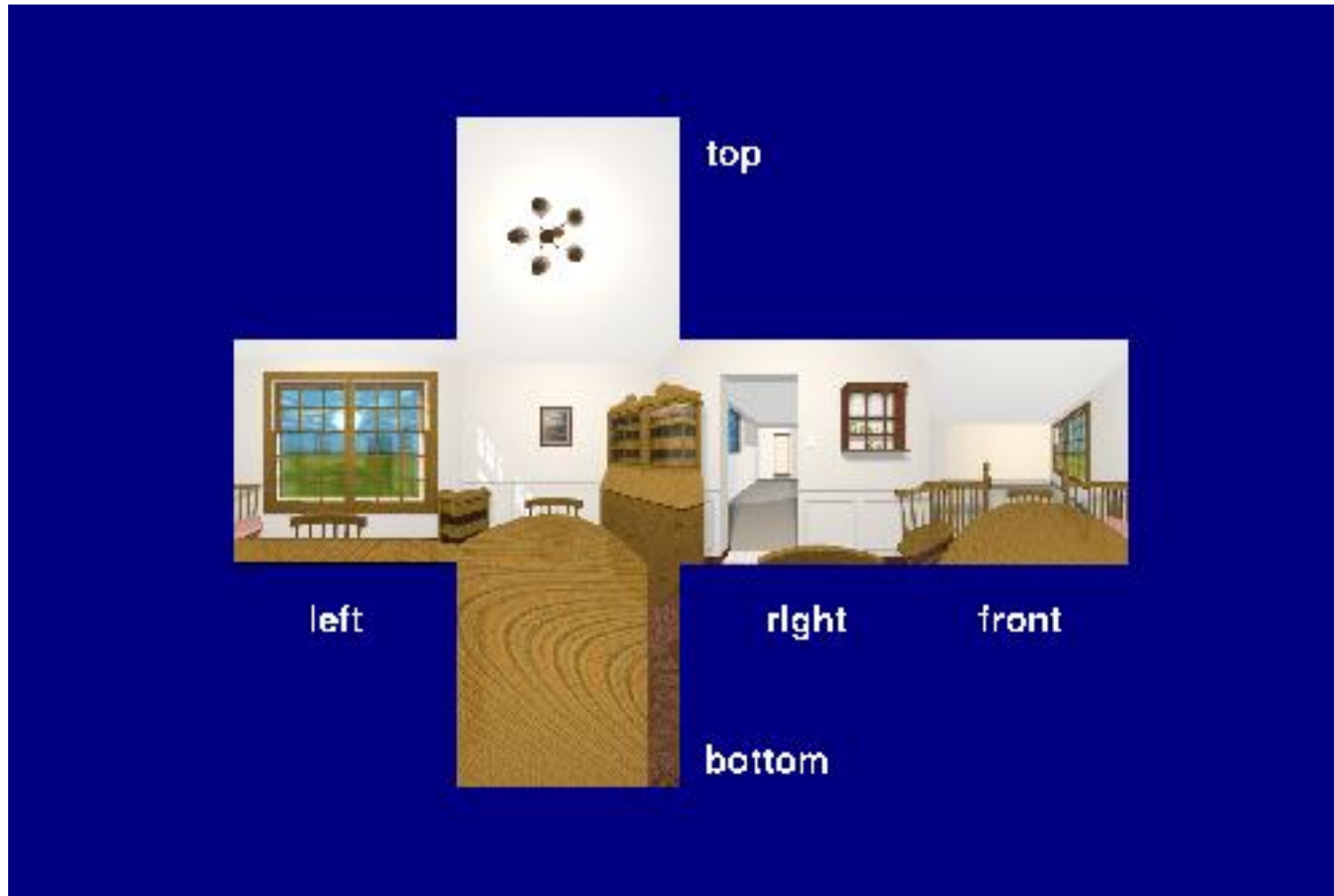


(a) View rays intersect the cube surface



(b) Mapping the environment images onto the cube faces.

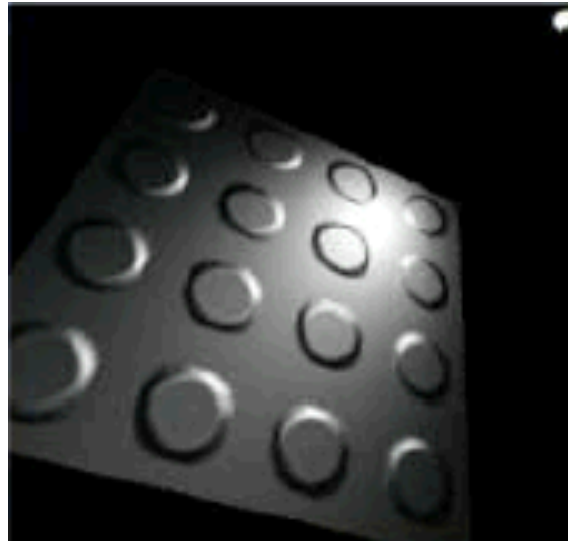
Env mapping by a cube - example



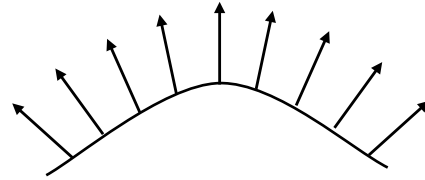
Bump mapping

- ❑ Developed in 1978 by J.F. Blinn
- ❑ Very efficient and simple method to present deformed surfaces (e.g.)
- ❑ Method:
 1. Angular modification of the normal vector according with a 2D model of deformation
 2. Apply the local reflection model to compute the light intensity
- ❑ Requirements:
 - Vector modification should be independent of object orientation and position
 - The texture should change (animate) as the object changes
 - The normal vector deformation should be defined in local coordinates relative to the surface point

Bump mapping - examples



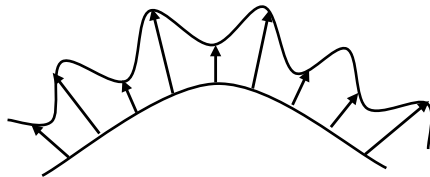
Bump mapping - phases



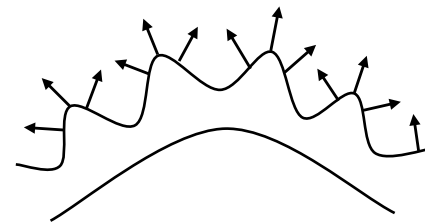
$O(u)$ Initial surface



$B(u)$ Deformation scheme



$O'(u)$ - Modification of $O(u)$
by $B(u)$



$N'(u)$ - Normal vectors of
the deformed surface

Bump mapping - definitions

- $O(u, v)$ – parametrical definition of the object surface. O is a point on the surface.
- Normal vector is: $N = O_u \times O_v$
- The deformation function is defined relative to two vectors A and B , tangent at the surface:

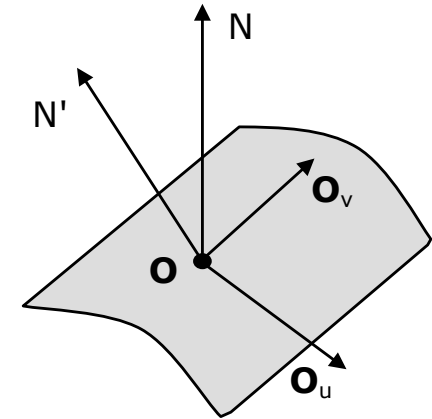
$$A = N \times O_v \quad B = N \times O_u$$

- Deformation function modifies the normal N to N' :

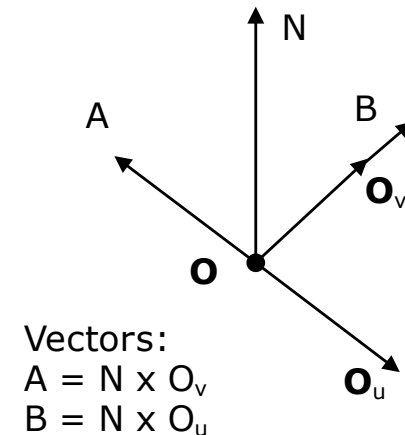
$$D = B_u A - B_v B$$

$$N' = N + D$$

Where B_u and B_v are the partial derivatives of the perturbation function $B(u, v)$

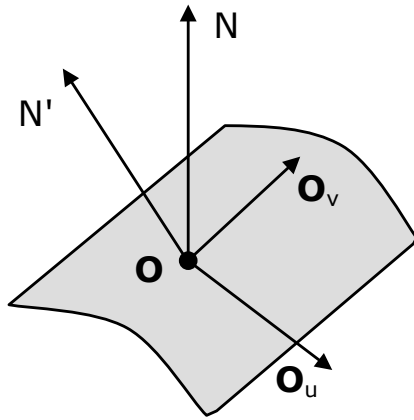


Normal vector at O is
 $N = O_u \times O_v$

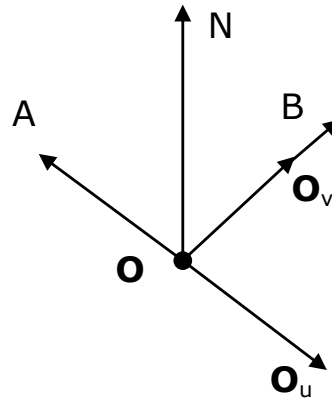


Vectors:
 $A = N \times O_v$
 $B = N \times O_u$

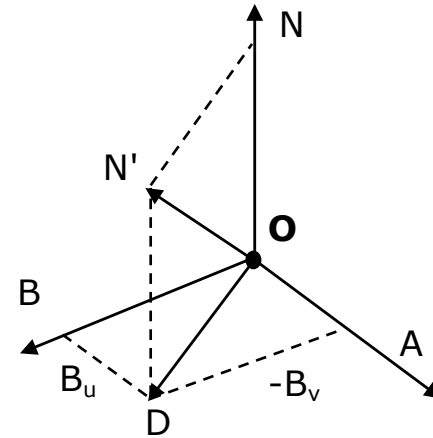
Bump mapping - vectors



Normal vector at O is
 $N = O_u \times O_v$



Vectors:
 $A = N \times O_v$
 $B = N \times O_u$



Vectors:
 $D = B_u A - B_v B$
 $N' = N + D$

Bump mapping - arguments

- $O'(u, v)$ is the deformed surface by the perturbation function and normal vector:

$$O'(u, v) = O(u, v) + B(u, v) \frac{N}{|N|}$$

- The derivatives of the object surface at point O are:

$$O'_u = O_u + B_u \frac{N}{|N|} + B \left(\frac{N}{|N|} \right)_u$$

$$O'_v = O_v + B_v \frac{N}{|N|} + B \left(\frac{N}{|N|} \right)_v$$

- The function $B(u, v)$ is quite small regarding the spatial extension, and the last term can be neglected

$$N'(u, v) = O_u \times O_v + B_u \left(\frac{N}{|N|} \times O_v \right) + B_v \left(O_u \times \frac{N}{|N|} \right) + B_u B_v \left(\frac{N \times N}{|N|^2} \right)$$

Bump mapping - arguments

The first term is the normal vector to the surface, and the last is zero. Then:

$$D = B_u (N \times O_v) - B_v (N \times O_u)$$

where, $N/|N|$ is considered as N , and

$$D = B_u A - B_v B$$

D is a vector coplanar with A and B , having the components B_u and $-B_v$.

N' is given by:

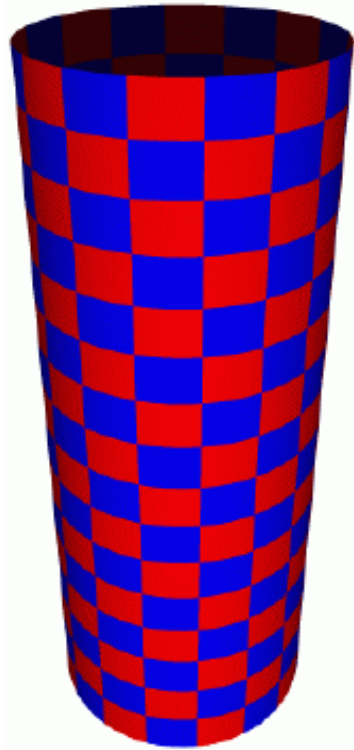
$$N' = N + D$$

That is the estimated result for the perturbed vector

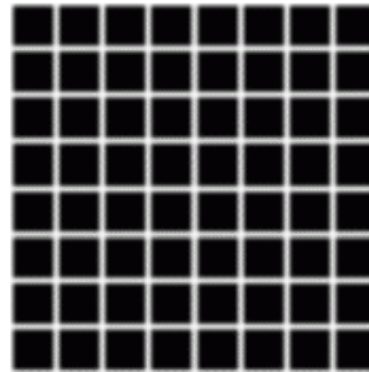
Bump mapping - issues

- ❑ Bump mapping depends on the scaling factor on the object
- ❑ For instance, if the scale factor is 2, then the N vector increases by 4, but the length of D increases by 2
- ❑ The perturbation function B is not scaled at all. Therefore, the bumps become smooth, as well as the object surface stretches out

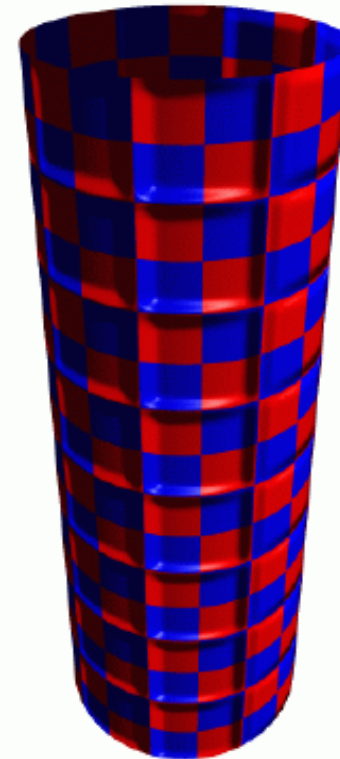
Bump mapping - examples



Cylinder w/Diffuse Texture Map

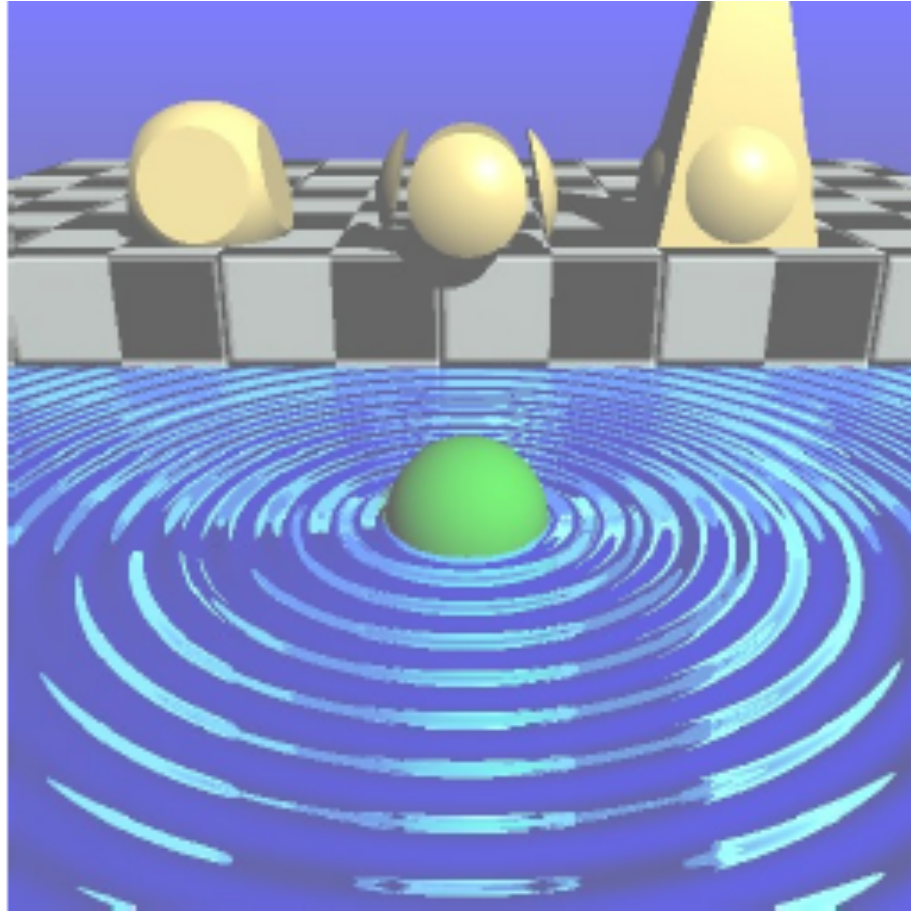


Bump Map



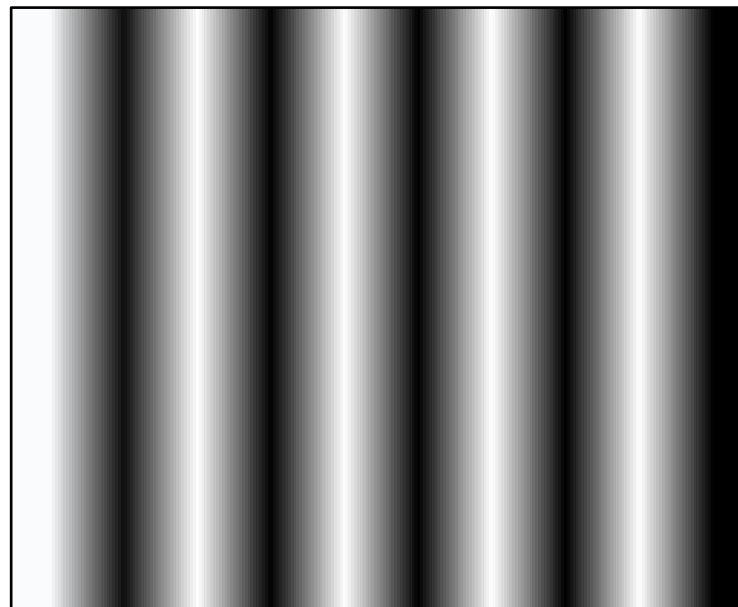
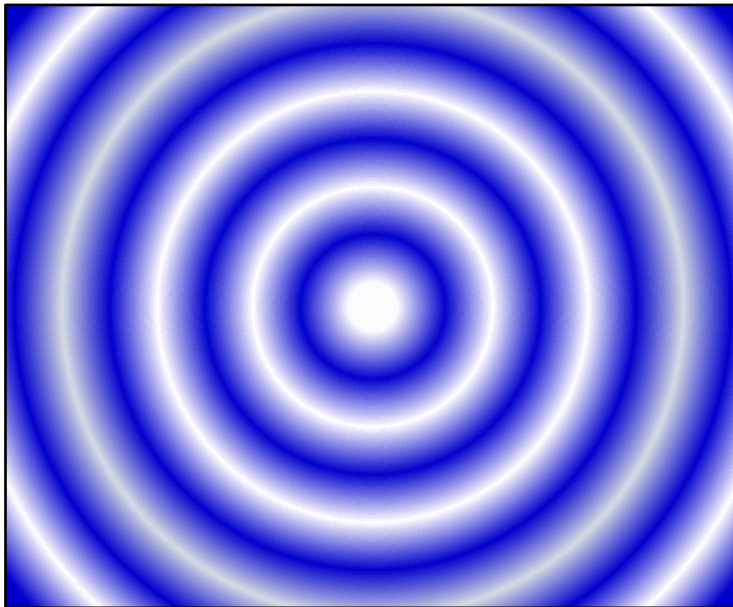
Cylinder w/Texture Map & Bump Map

Bump mapping - examples



Questions and problems

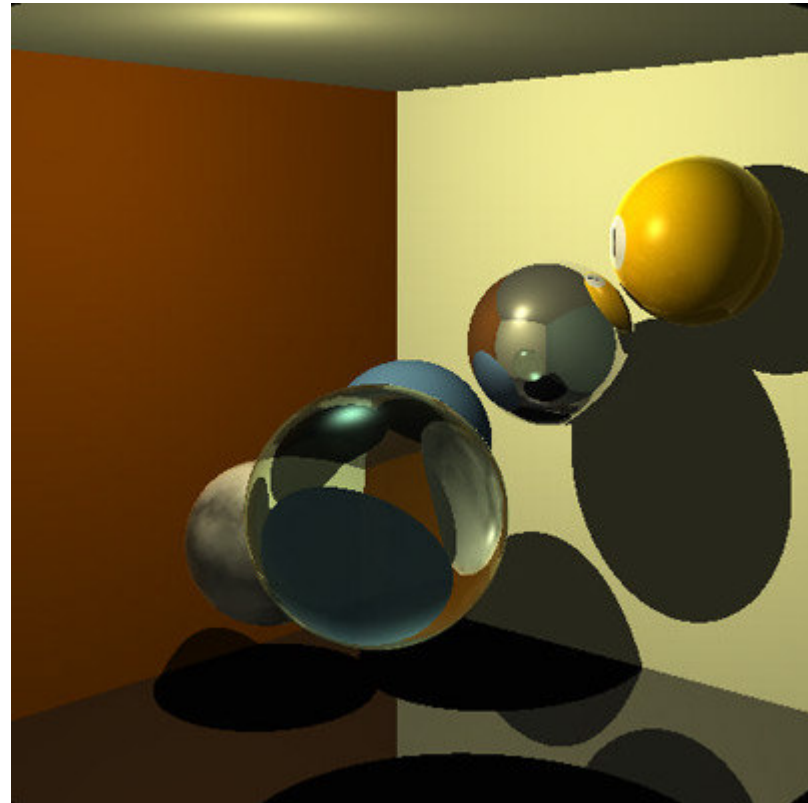
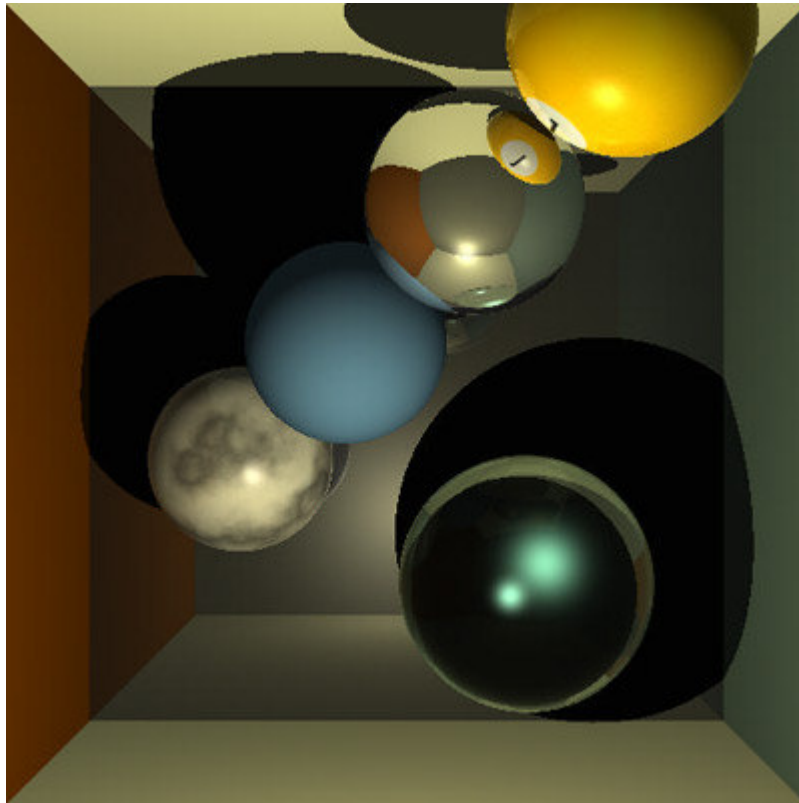
1. Exemplify and explain how is computed the value of a point on the 3D object by mapping of a 3D normalized texture.
2. Explain the mechanism of bump mapping to obtain the image of: (a) water surface with circular waves; (b) cylindrical bumps.



Raytracing - Examples

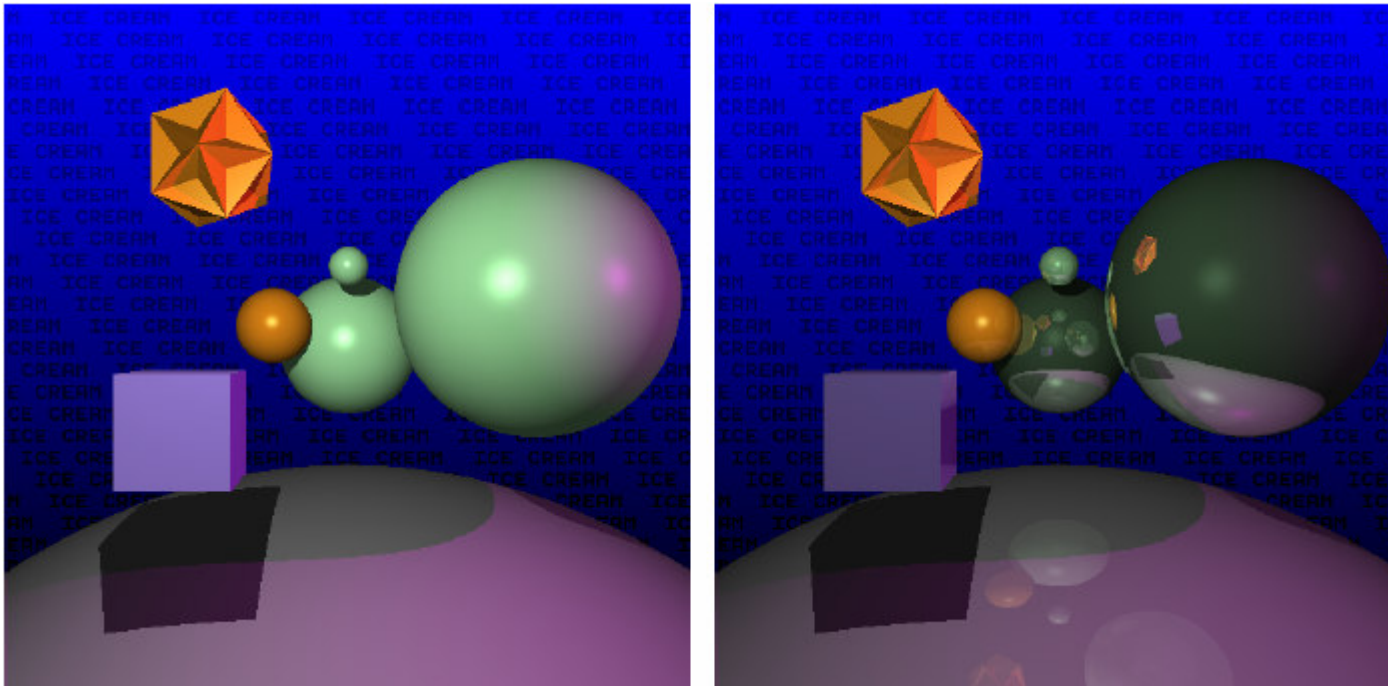
Raytracing

Fig1. Box above and angle.



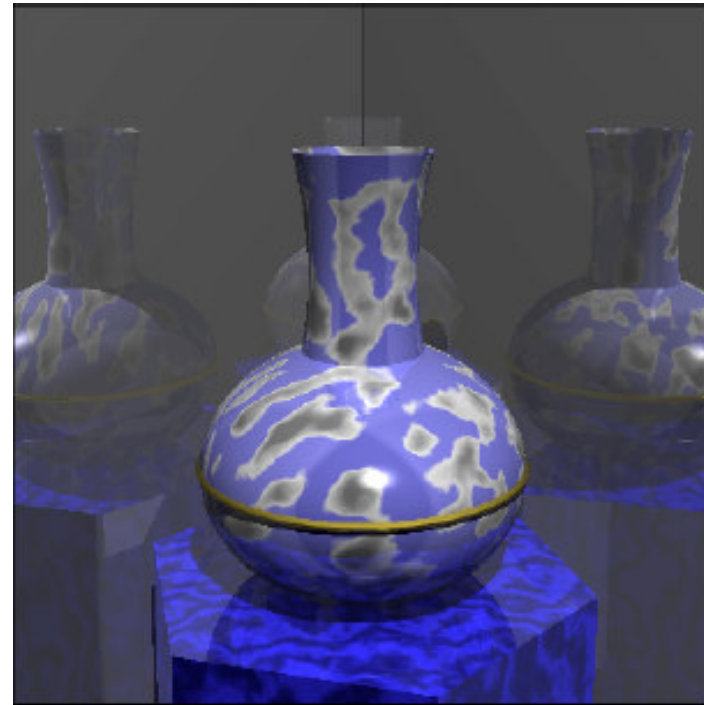
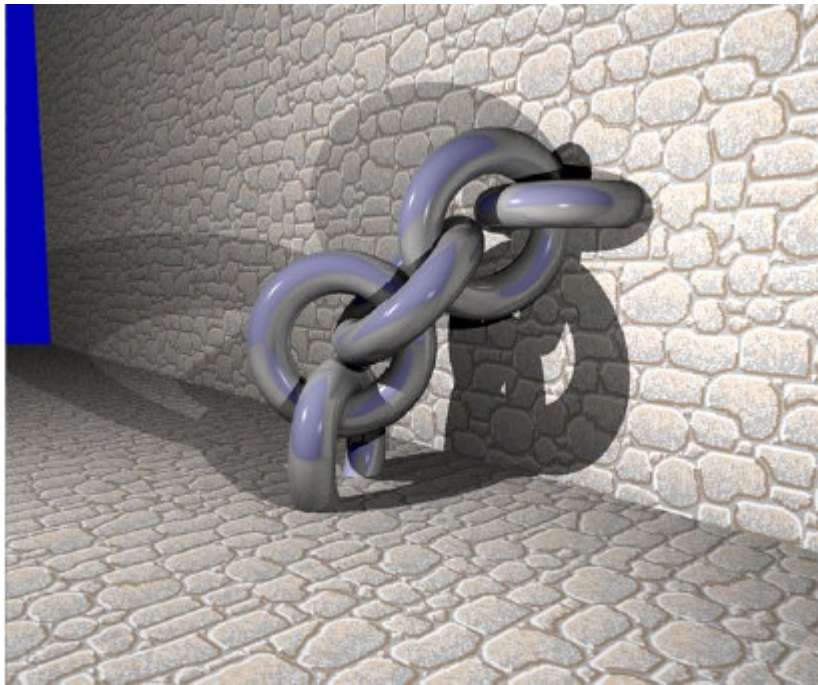
Raytracing

Fig2. Different level of reflection (recursivity).



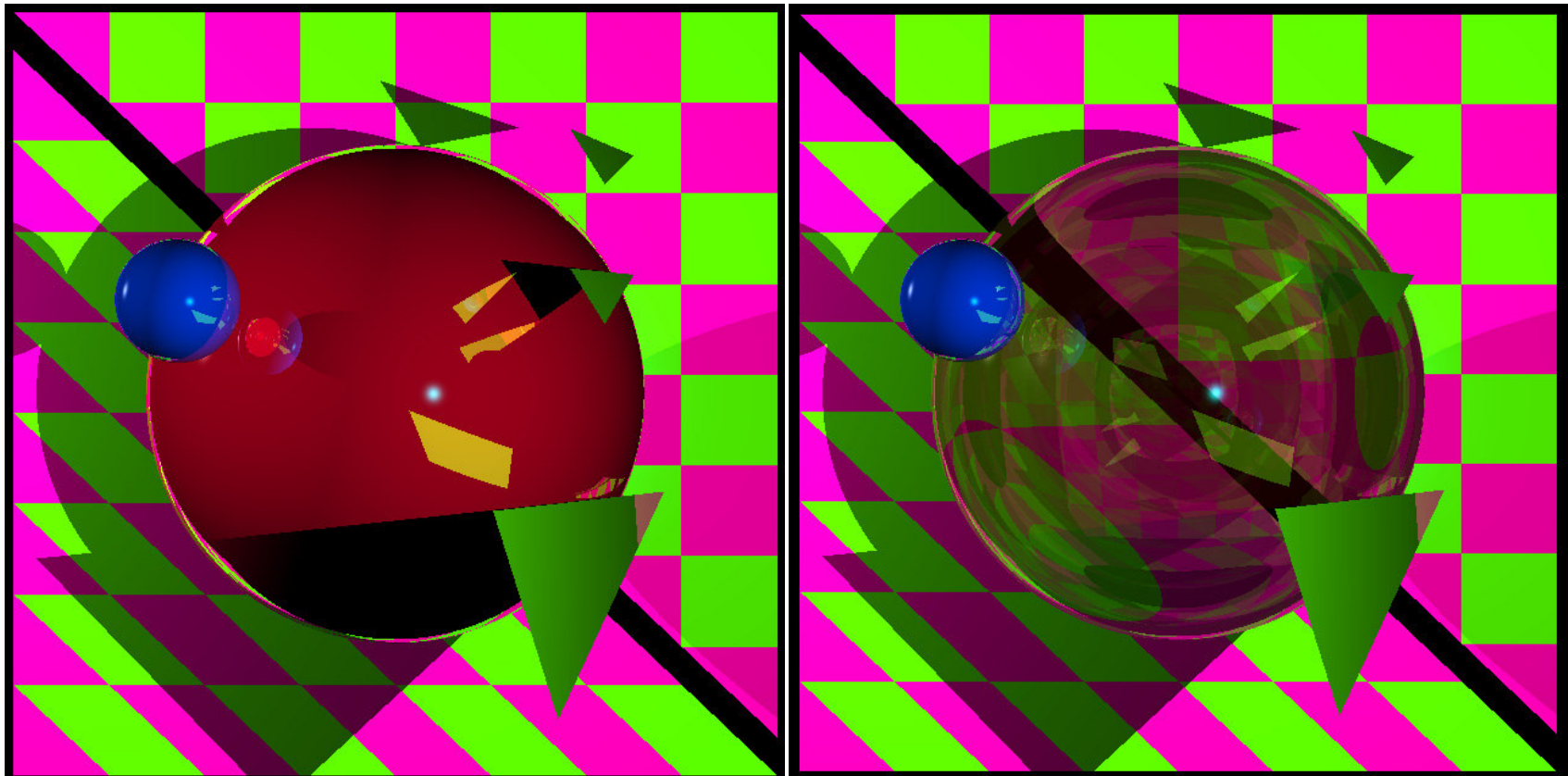
Raytracing

Fig3. Ray tracing with texture mapping and shadows.



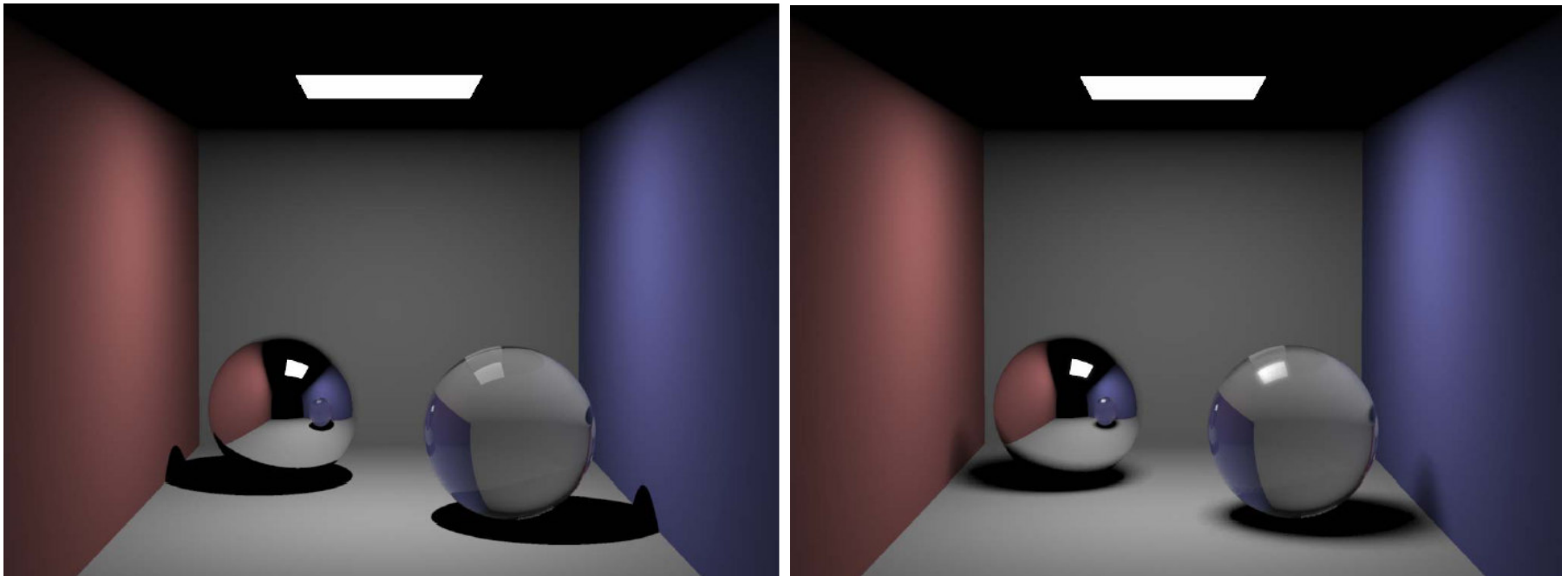
Raytracing

Fig4. Ray tracing with texture mapping and shadows.

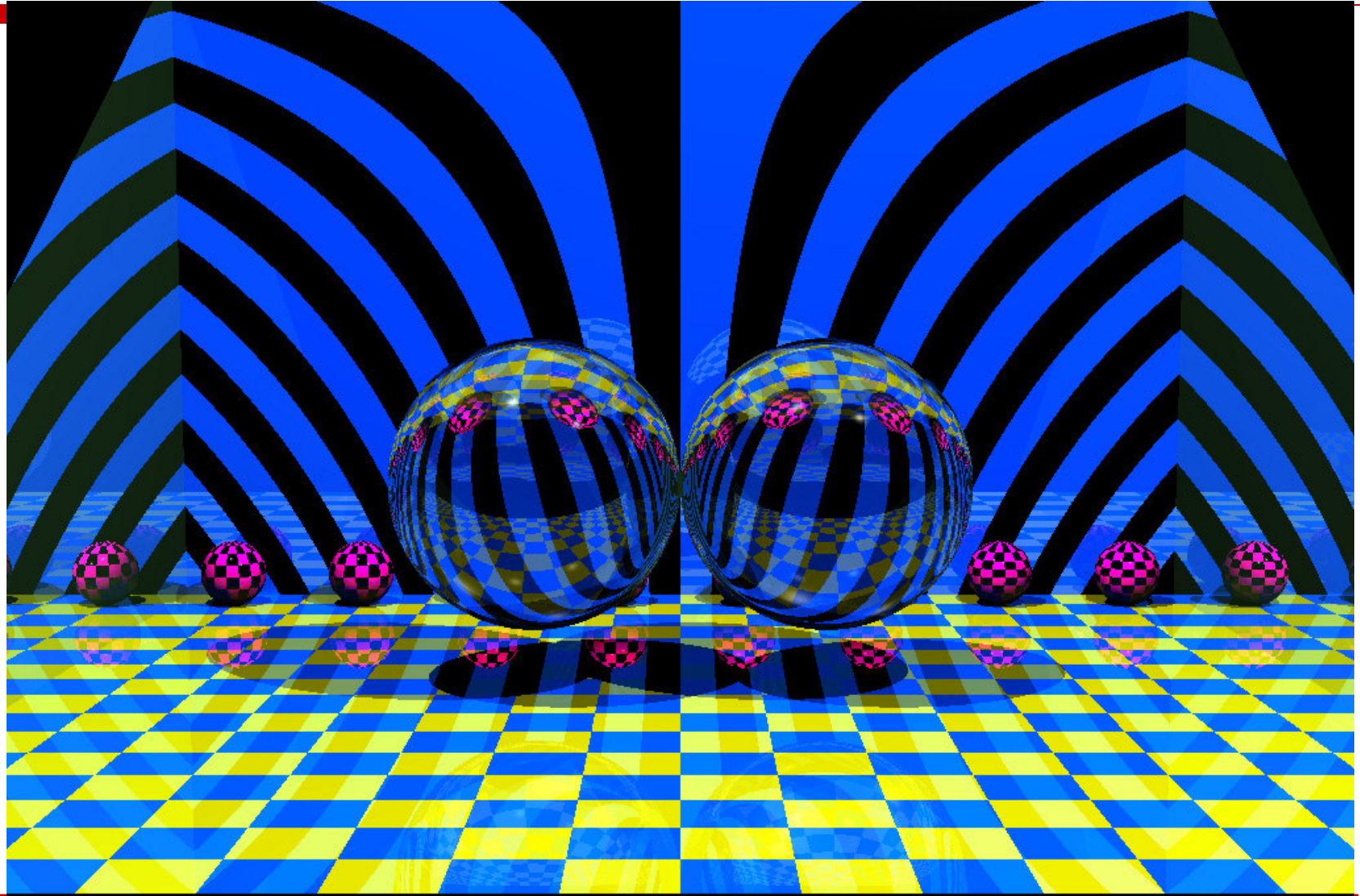


Raytracing

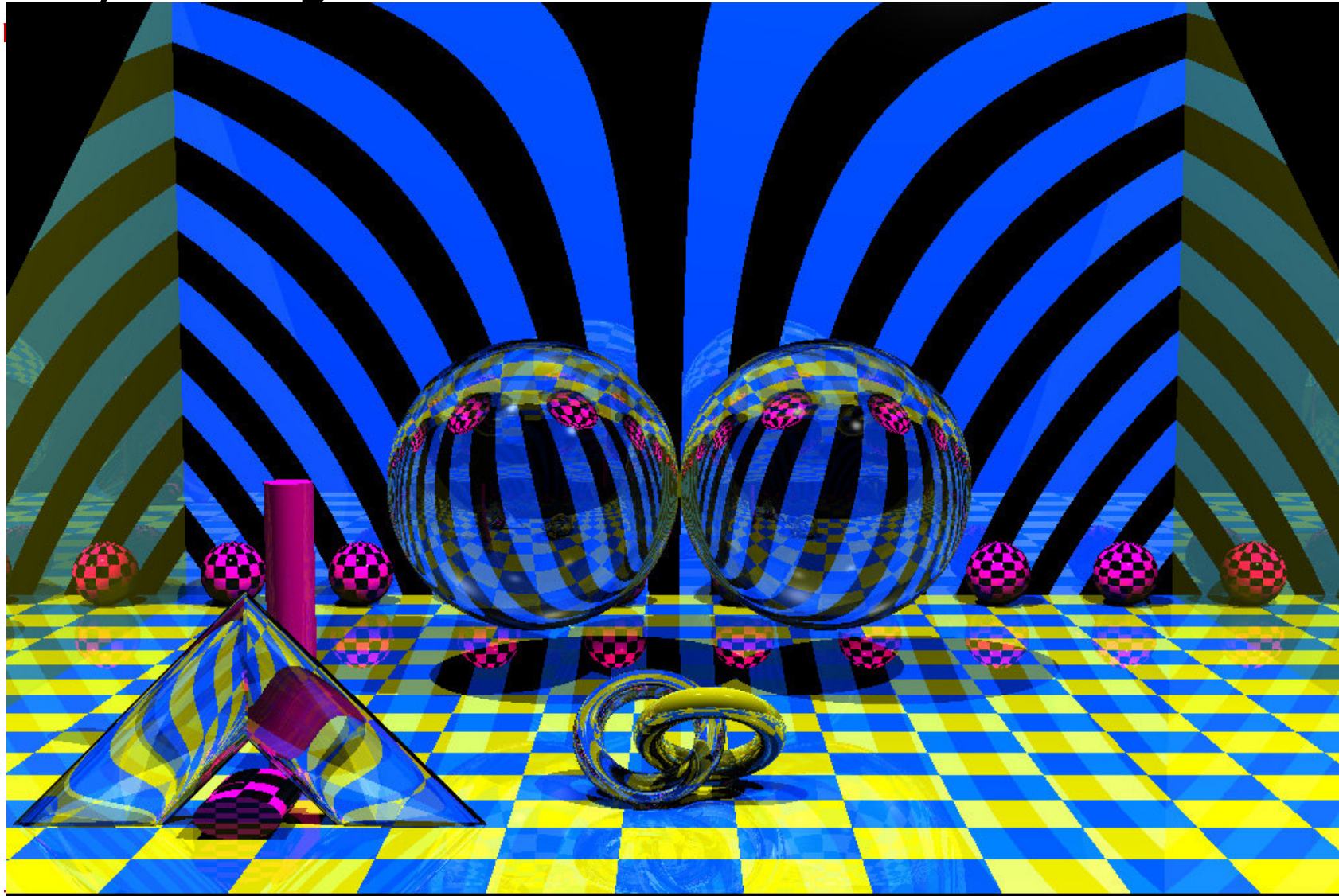
Fig5. Ray tracing with hard and soft edge shadows.



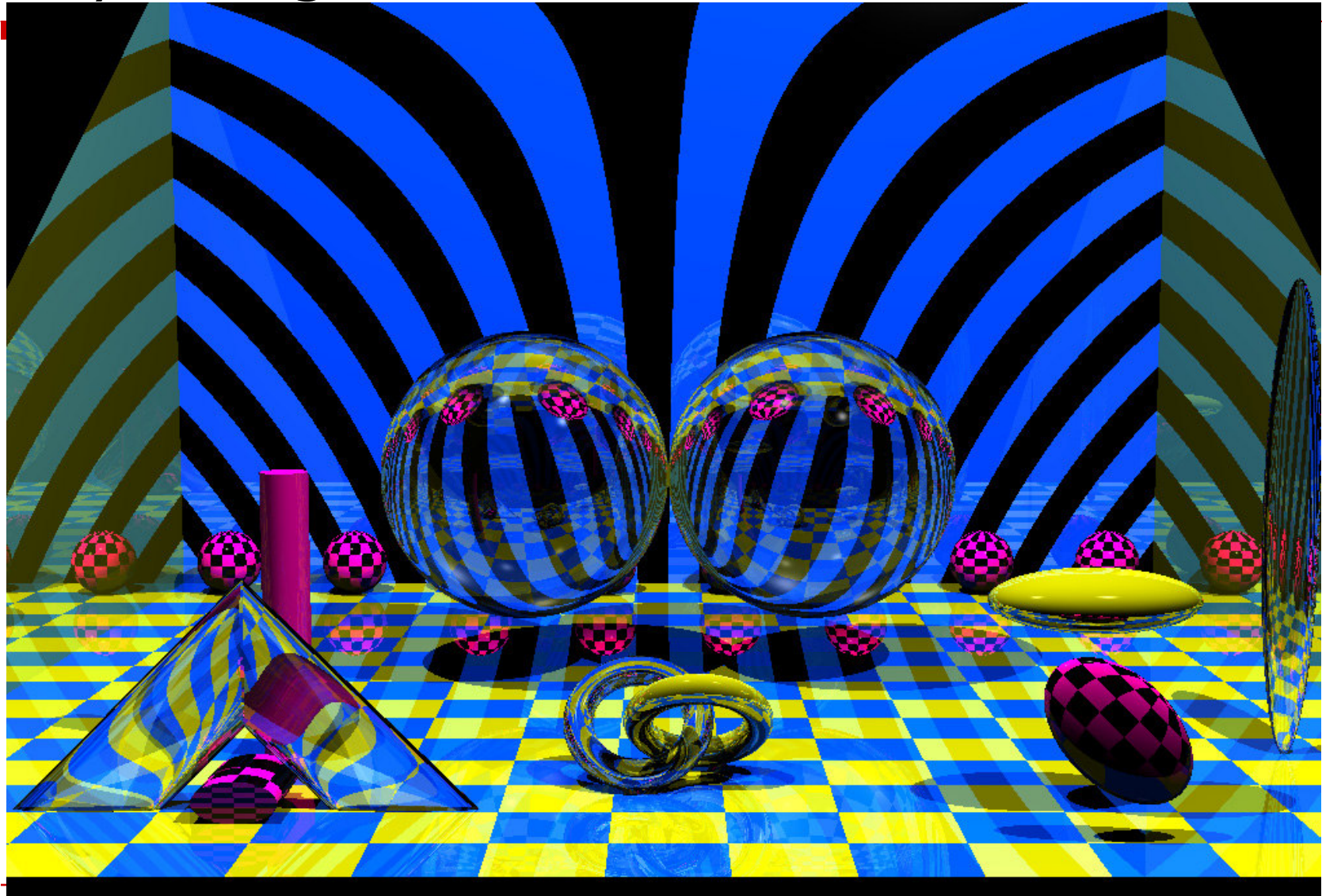
Raytracing



Raytracing



Raytracing



Ray Tracing

Contents

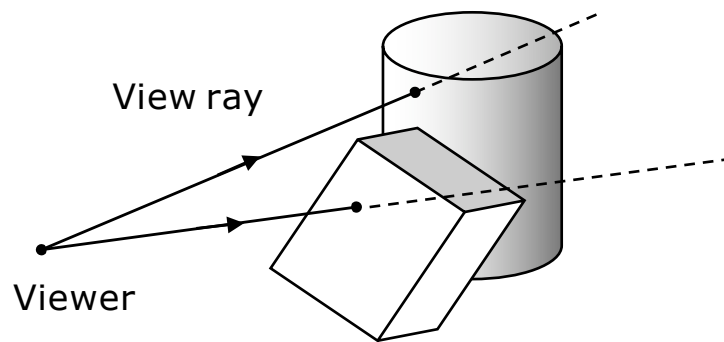
- Introduction
- Basic algorithm
- Recursive ray tracing
- Intersections
 - Ray-sphere
 - Ray-convex polyhedron
 - Ray-box
 - Ray-quadrics
 - Ray-any object
- Reflection and refraction
- New local reflection model
- Global reflection model
- Recursive computation
- Shadow computation
- Implementation

Introduction

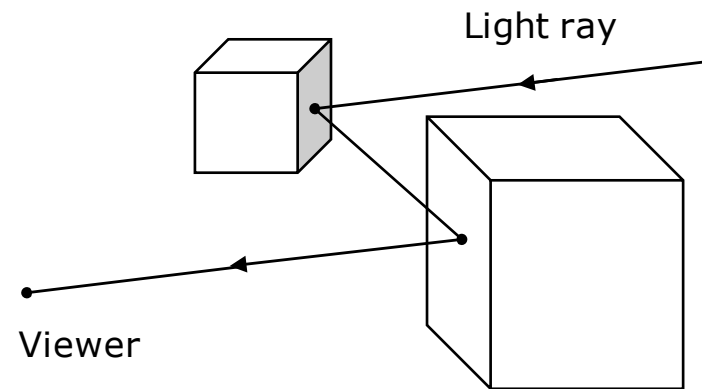
- ❑ Ray tracing was originally used in computer graphics by Appel in 1968
- ❑ In 1980, Turner Whitted has developed the first ray tracing model that integrates reflection, refraction, hidden surface removal, and shadow computation
- ❑ Partial solution to the global illumination problem
- ❑ (-) High processing time

Basic algorithm

- ❑ Works completely in object space
- ❑ Color and intensity at a point are obtained by tracing a ray backwards from the eye through the point into the scene
- ❑ Illumination is a contribution of the direct illumination, reflected and transmitted rays

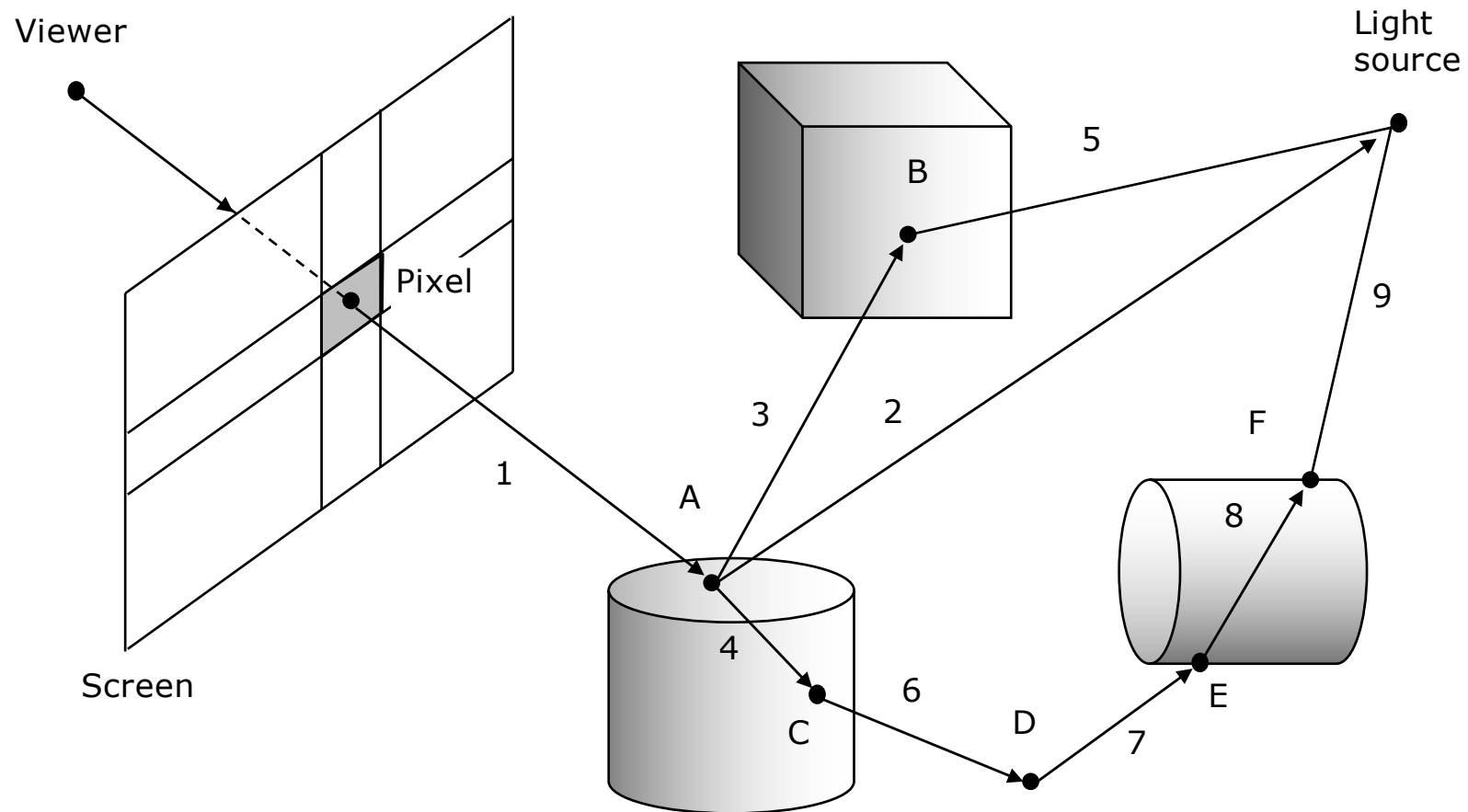


(a)



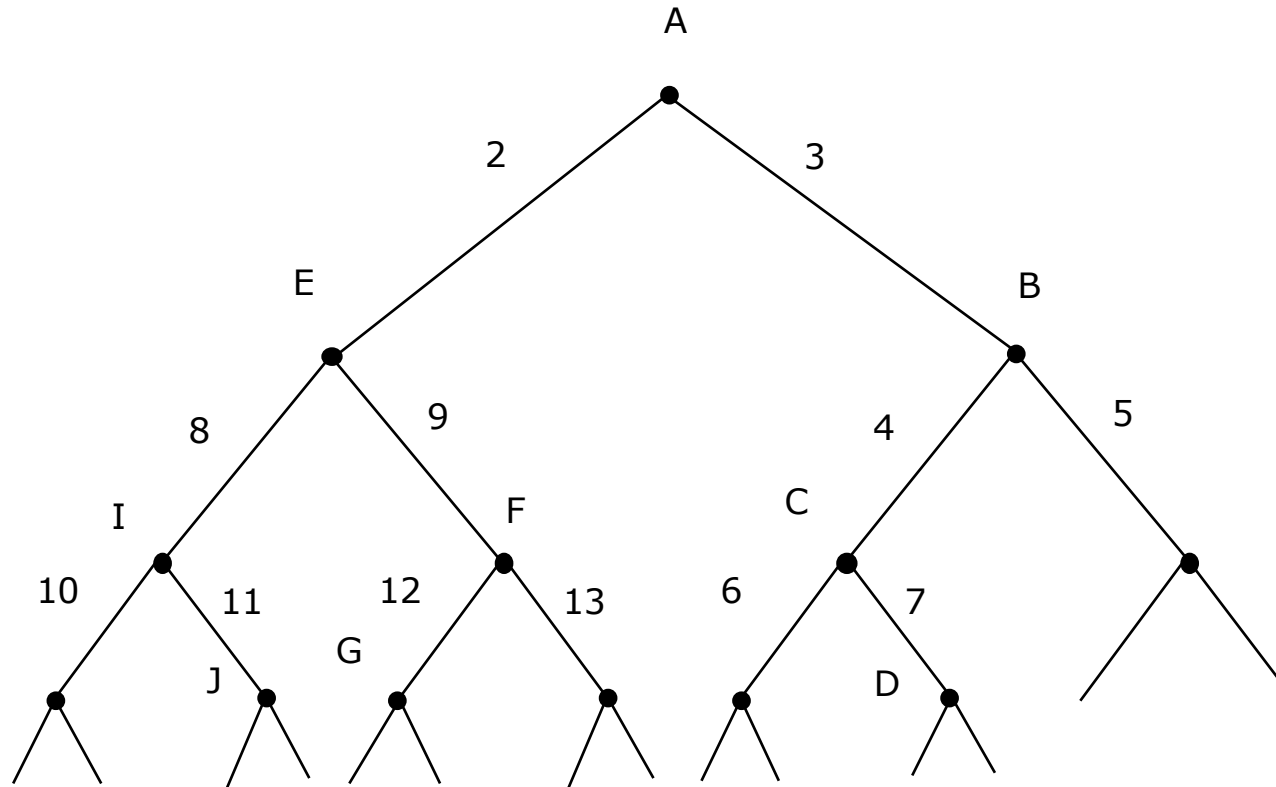
(b)

Basic algorithm





Recursive ray tracing – viewing ray tree



Simple recursive ray tracing procedure

TraseazaRaza (vectori start, vectori directie, int nivel, culori culoare)

{

vectori punct_intersectie, directie_reflexie, directie_refractie;

culori culoare_locala, culoare_reflectata, culoare_refractata;

Intersectează raza cu toate obiectele și determină cel mai apropiat punct de intersecție, dacă există, fata de începutul razei.

if (nu există intersecție)

culoare = culoarea de fond;

else {

culoare_locala = contribuția modelului local de culoare la punctul de intersecție;

if (nivel = nivel_maxim)

culoare_reflectata = black;

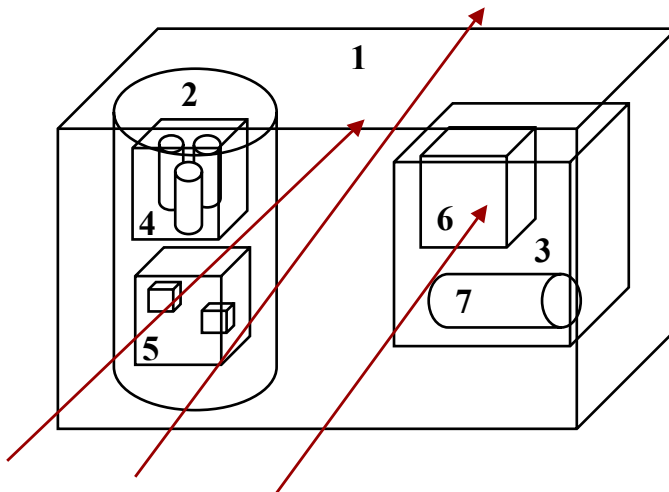
culoare_refractie = black;

Simple recursive ray tracing procedure

```
    else {
        Calculează direcția razei reflectate;
        TraseazaRaza(punct_intersectie,
                    directie_reflexie,
                    nivel+1,
                    culoare_reflectata);
        Calculează direcția razei refractate;
        TraseazaRaza(punct_intersectie,
                    directie_refractie,
                    nivel+1,
                    culoare_refractata);
    }
    Combina( culoare, culoare_locala,
            ponderea_locala_suprafata,
            culoare_reflectata,
            coeficient_reflexie_al_suprafetei,
            culoare_refractata,
            coeficient_refractie_al_suprafetei);
} // TraseazaRaza
```

Ray tracing geometry - Intersections

- ❑ Intersection computation – time consuming
- ❑ 95% of processing time (Whitted, 1980)
- ❑ Speeding up the intersection computation:
 - bounding volumes (Whitted 1980, Clark 1976)
 - Avoid extensive computation of the intersection outside of bounding boxes
 - Bounding boxes: sphere, cylinder, cube, parallelepiped, quadrics (i.e. ellipsoid, paraboloid, hyperboloid, etc)



Intersection: ray-sphere

View ray: $P1(x1, y1, z1)$ and $P2(x2, y2, z2)$

Parametrical equation of the view ray:

$$x = x1 + (x2 - x1) t = x1 + it$$

$$y = y1 + (y2 - y1) t = y1 + jt$$

$$z = z1 + (z2 - z1) t = z1 + kt$$

A sphere at centre (l, m, n) and radius r :

$$(x - l)^2 + (y - m)^2 + (z - n)^2 = r^2$$

Substituting for x, y and z :

$$at^2 + bt + c = 0$$

where:

$$a = i^2 + j^2 + k^2$$

$$b = 2i(x1 - l) + 2j(y1 - m) + 2k(z1 - n)$$

$$c = l^2 + m^2 + n^2 + x1^2 + y1^2 + z1^2 + 2(-lx1 - my1 - nz1) - r^2$$

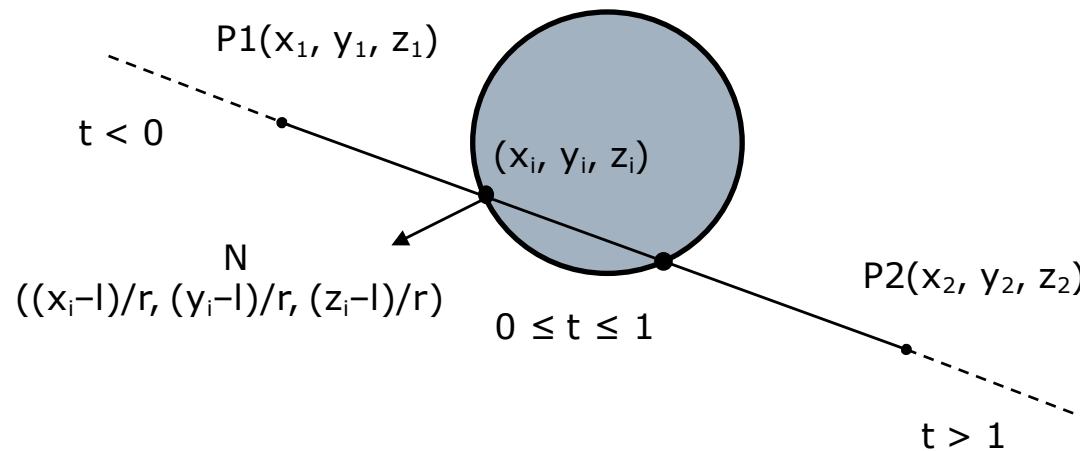
Intersection: ray-sphere

If $\Delta < 0$, the line does not intersect the sphere

$\Delta = 0$, the line is tangential to the sphere

$\Delta > 0$, there is a front and back intersections

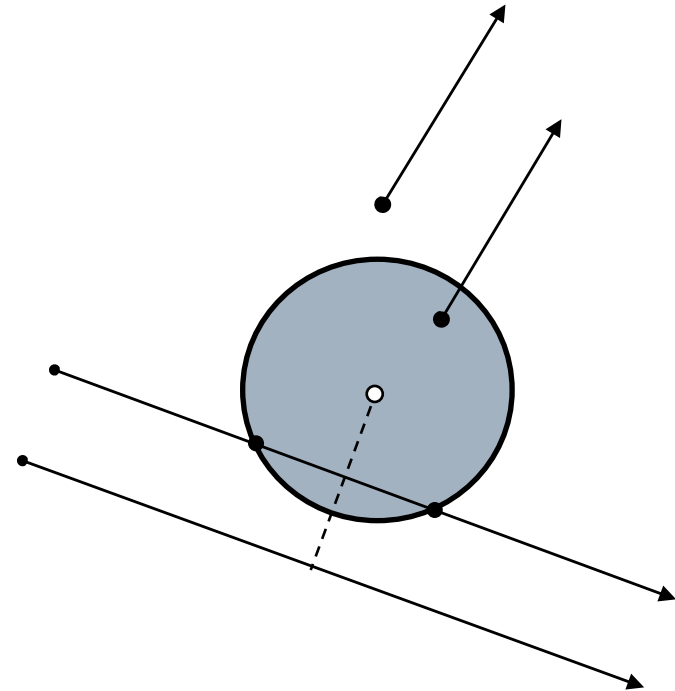
Value of t gives the position of the intersections relative to $P1$ and $P2$:



Intersection: ray-sphere

Speeding up the decision on the intersection:

1. Find if the ray's origin is outside the sphere
2. Find the closest approach of the ray to the sphere's centre
3. If the ray is outside and points away from the sphere, the ray must miss the sphere
4. Else, find the distance from the closest approach to the sphere surface
5. If the value is negative, the ray misses the sphere
6. Else, from above, find the ray-surface distance
7. Calculate the intersection coordinates
8. Calculate the normal at the intersection point



Intersection: ray-convex polyhedron

- ❑ Object – convex polygonal model
- ❑ Test the ray individually against each polygon:
 1. Determine the equation of the polygon plane
 2. Check for an intersection between this plane and the ray
 3. Check the intersection is inside the polygon

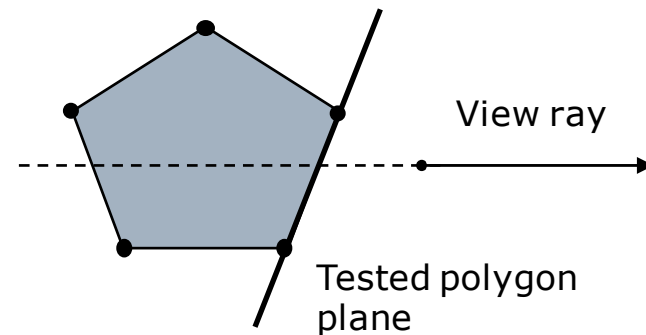
If the polygon plane equation is:

$$ax + by + cz + d = 0$$

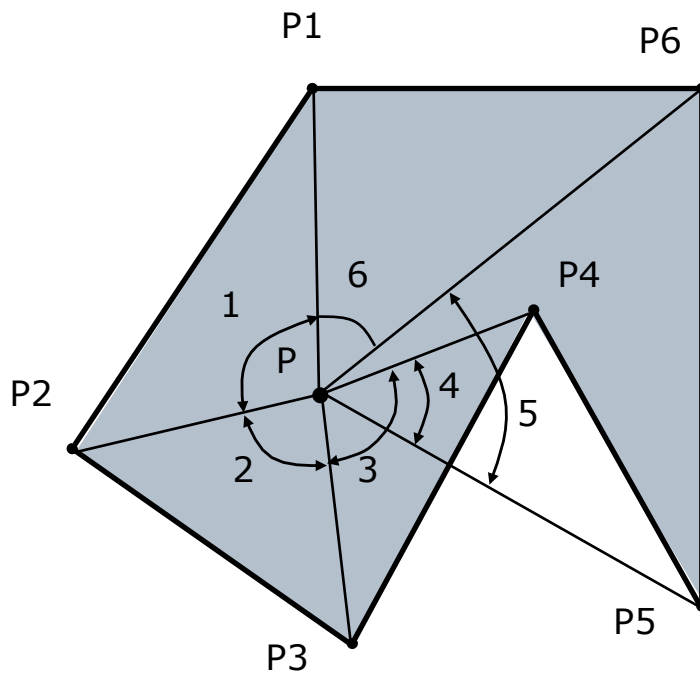
Then the intersection with a parametric line is given by t:

$$t = - (a x_1 + b y_1 + c z_1) / (a i + b j + c k)$$

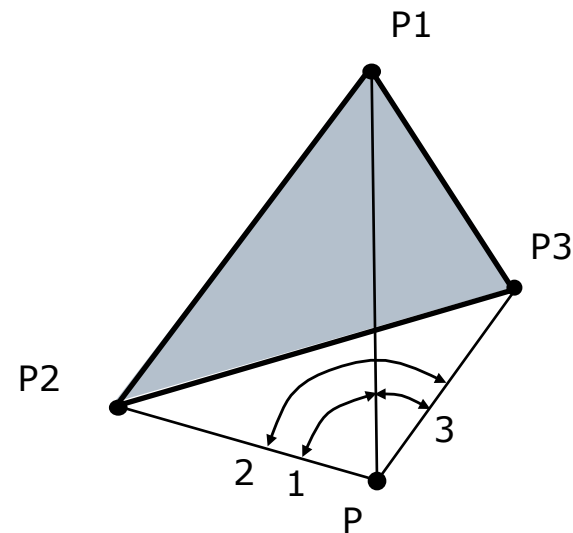
Exit the test if $t < 0$,
the intersection is on back side of the viewer.



Intersection: ray-convex polyhedron

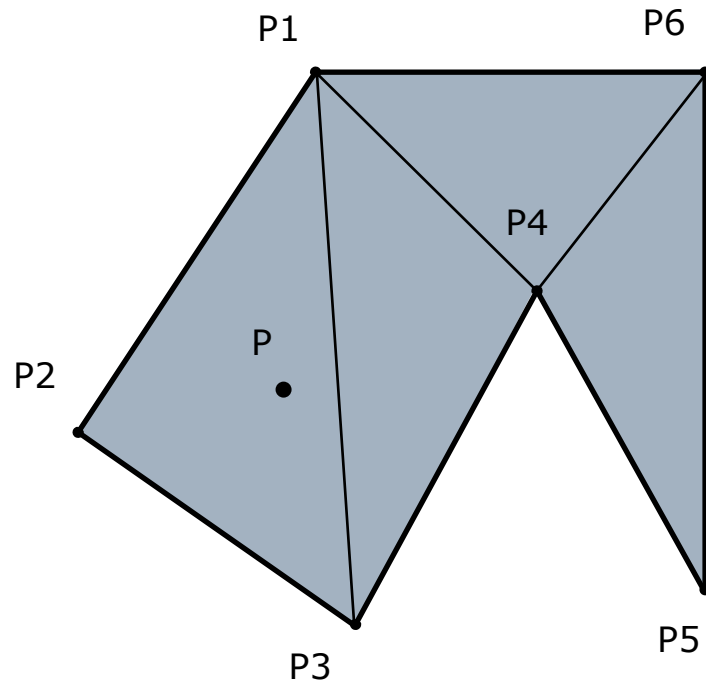


(a) $\sum (1 + 2 + 3 - 4 + 5 + 6) = 360^\circ$
 P inside of polygon ($P1, \dots, P6$)



(b) $\sum (1 - 2 + 3) = 0^\circ \neq 360^\circ$
 P outside of polygon ($P1, P2, P3$)

Intersection: ray-convex polyhedron



Triangulation:

$$P_1, \dots, P_6 \rightarrow \Delta P_1P_2P_3$$

$$\Delta P_1P_3P_4$$

$$\Delta P_4P_5P_6$$

$$\Delta P_1P_4P_6$$

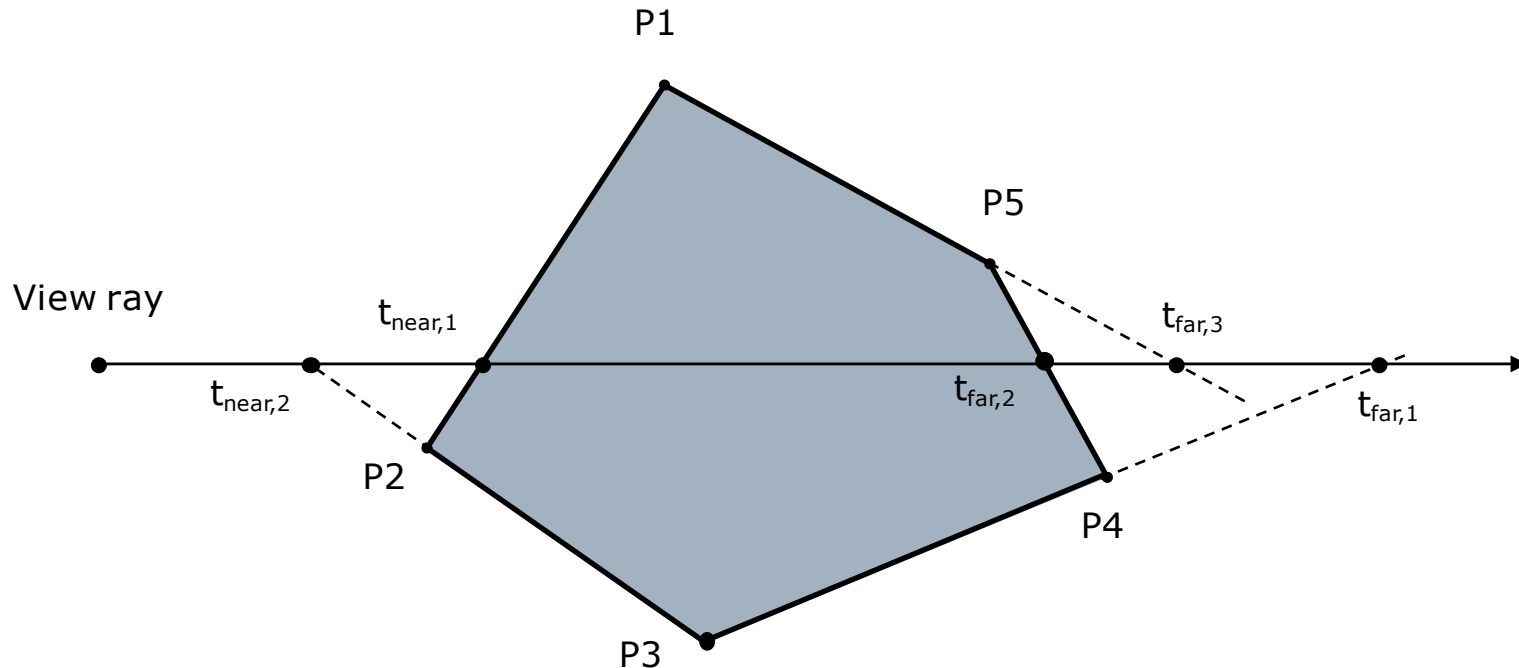
Check the position of P against a triangle:

$$P \notin \Delta P_1P_2P_3$$

Conclusion:

$$P \notin P_1, \dots, P_6$$

Intersection: ray-convex polyhedron



Cyrus-Beck algorithm:

Initialize t_{near} to large negative value, t_{far} to large positive value

If (plane is back-facing) and ($t < t_{far}$) then $t_{far} = t$

If (plane is front-facing) and ($t > t_{near}$) then $t_{near} = t$

If ($t_{near} > t_{far}$) then (exit – ray misses)

Intersection: ray-box

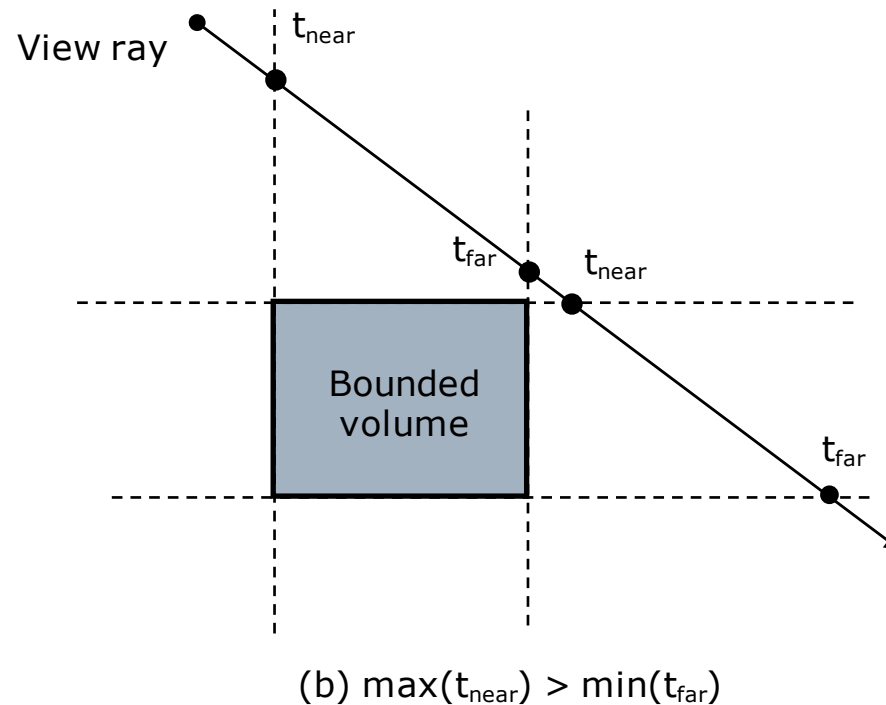
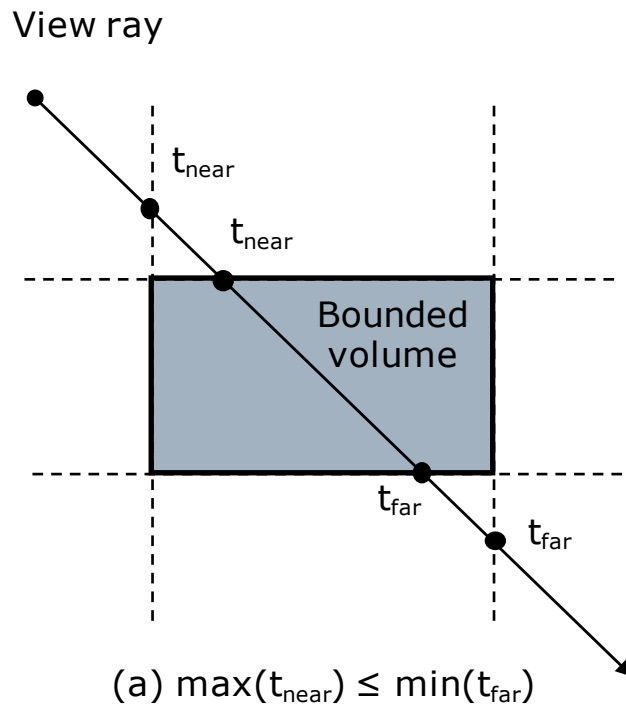
Bounded volume: (x_{e1}, y_{e1}, z_{e1}) and (x_{e2}, y_{e2}, z_{e2})

Intersections with the x axis:

$$t_{1x} = (x_{e1} - x_1) / (x_2 - x_1)$$

$$t_{2x} = (x_{e2} - x_1) / (x_2 - x_1)$$

Where x_1 and x_2 are the coordinates of the view ray end points



Intersection: ray-quadrics

- ❑ Quadrics: cylinder, ellipsoid, paraboloid, and hyperboloid
- ❑ General equation for a quadric:

$$Ax^2 + Ey^2 + Hz^2 + 2Bxy + 2Fyz + 2Cxz + 2Dx + 2Gy + 2Iz + J = 0$$

That is the following matrix based form:

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} A & B & C & D \\ B & E & F & G \\ C & F & H & I \\ D & G & I & J \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = 0$$

Using the same substitution as for the sphere the a, b and c coefficients are:

$$a = Ax_d^2 + Ey_d^2 + Hz_d^2 + 2Bx_dy_d + 2Cx_dz_d + 2Fy_dz_d$$

$$b = 2(Ax_1x_d + Ey_1y_d + Hz_1z_d + B(x_1y_d + x_dy_1) + C(x_1z_d + x_dz_1) + F(y_1z_d + y_dz_1) + Dx_d + Gy_d + Iz_d)$$

$$c = Ax_1^2 + Ey_1^2 + Hz_1^2 + 2Bx_1y_1 + 2Cx_1z_1 + 2Fy_1z_1 + 2Dx_1 + 2Gy_1 + 2Iz_1 + J$$

where (x_d, y_d, z_d) is the normalized vector of the viewing ray direction

Intersection: ray-quadrics

- Sphere

$$(x - l)^2 + (y - m)^2 + (z - n)^2 = r^2$$

where (l, m, n) is the centre of the sphere

- Infinite cylinder

$$(x - l)^2 + (y - m)^2 = r^2$$

- Ellipsoid

$$(x - l)^2 / \alpha^2 + (y - m)^2 / \beta^2 + (z - n)^2 / \gamma^2 - 1 = 0$$

where α , β and γ are the semi-axes

- Paraboloid

$$(x - l)^2 / \alpha^2 + (y - m)^2 / \beta^2 - z + n = 0$$

- Hyperboloid

$$(x - l)^2 / \alpha^2 + (y - m)^2 / \beta^2 - (z - n)^2 / \gamma^2 - 1 = 0$$

Intersection: ray-any object

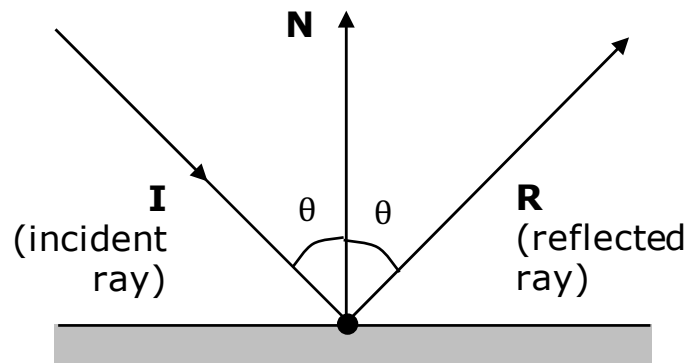
- Procedural modeling of the object – intersection depends on the type of the definition
- General approach:
 1. Use for each object a spherical bounded volume
 2. If the view ray intersects the sphere then the objects is subdivided in a set of object components. Each component has a spherical bounded volume
 3. The subdivision continues until the spherical bounded volume reach a minimum radius or the view ray does not intersect any more the spherical extension

Reflection and refraction

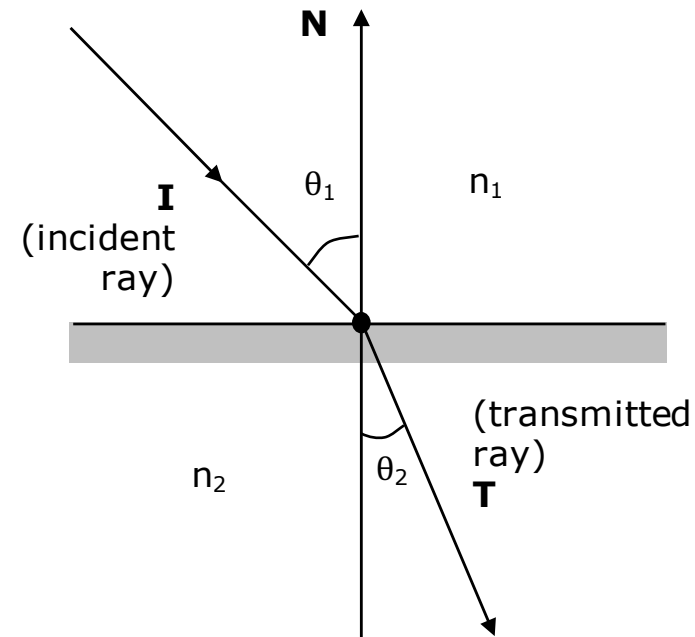
Snell's law: $\frac{\sin \theta_1}{\sin \theta_2} = \frac{n_2}{n_1}$ $n = \frac{n_2}{n_1}$

In 1984 Heckbert and Hanrahan determined the formula for the refracted (transmitted) ray:

$$\cos \theta_2 = n \left[1 - \frac{1}{n^2} (1 - \cos^2 \theta_1) \right]^{1/2}$$



(a) $R = I + 2N \cos \theta$

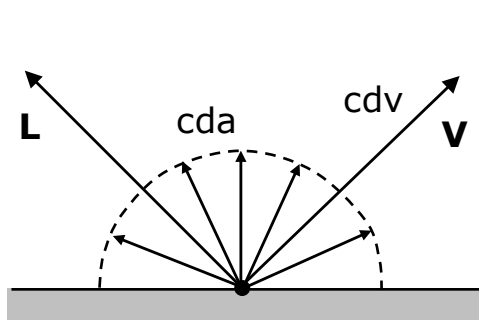


(b) $T = \frac{1}{n} I - (\cos \theta_2 - \frac{1}{n} \cos \theta_1) N$

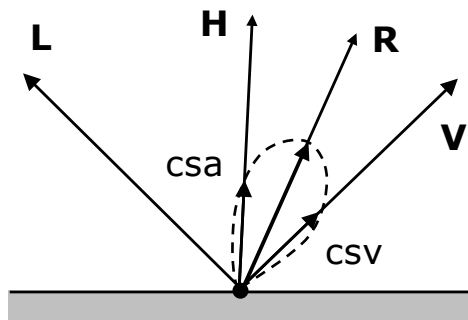
Illumination and reflection model

- Global illumination – direct, reflected and transmitted rays
- Direct illumination described by Phong model:

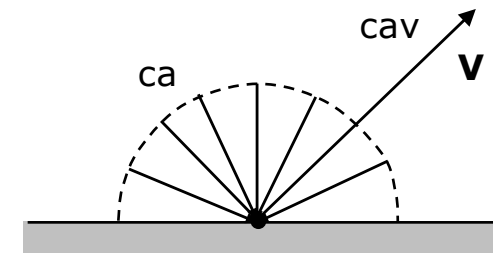
$$I_{\text{Phong}} = \text{ambient illumination} + \text{diffuse illumination} + \text{specular illumination}$$



(a) cdv – diffuse component in direction V ; cda – diffuse components in other directions.



(b) csv – specular component in direction V ; csa – specular components in other directions.



(c) cav – ambient component in direction V ; ca – ambient components from all directions

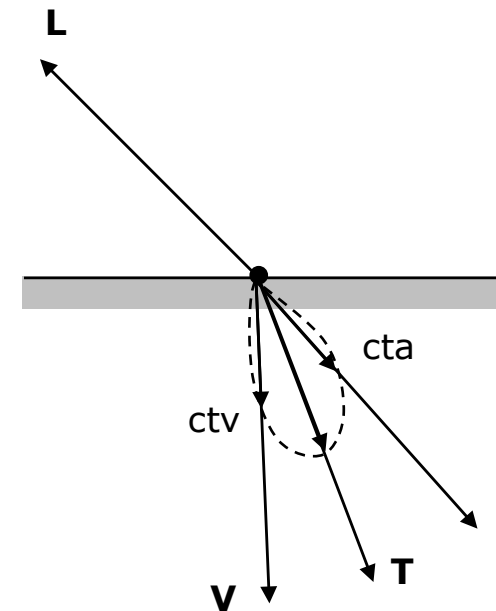
Illumination and reflection model

- Illumination for transparent objects
- Light that is seen through a transparent object due to a light source located at the other side of the object
- Generalizes the direct illumination component of the model

$$I_{\text{direct transmitted}} = k_t (N \cdot H')^n$$

where H' is the normal vector to a surface that refract the light to the V direction:

$$H' = (-L - nV) / (n - 1)$$



ctv – transmitted component in direction V ;
cta – transmitted components in other directions.

New local reflection model

$$I_{\text{local}} = I_a k_a + I_i [k_d (N \cdot L) + k_s (N \cdot H)^{ns} + k_t (N \cdot H')^{nt}]$$

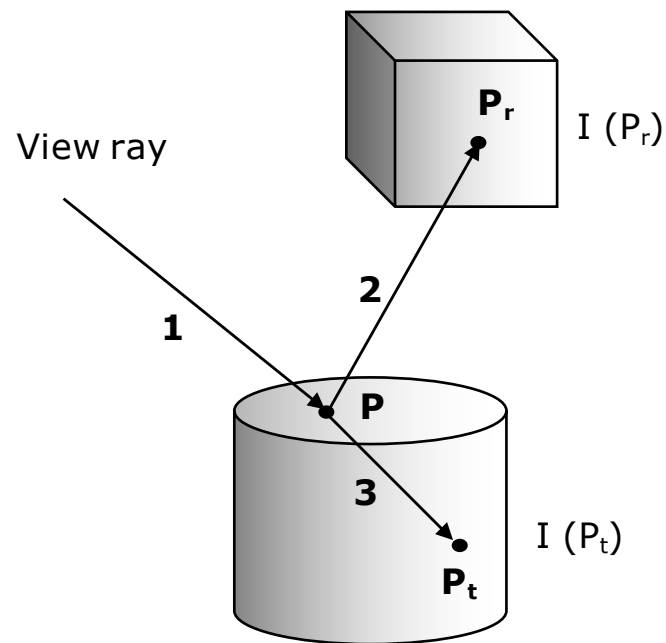
Global reflection model

$$I = I_{\text{local}} + k_{\text{rg}} I_{\text{reflection}} + k_{\text{tg}} I_{\text{transmitted}}$$

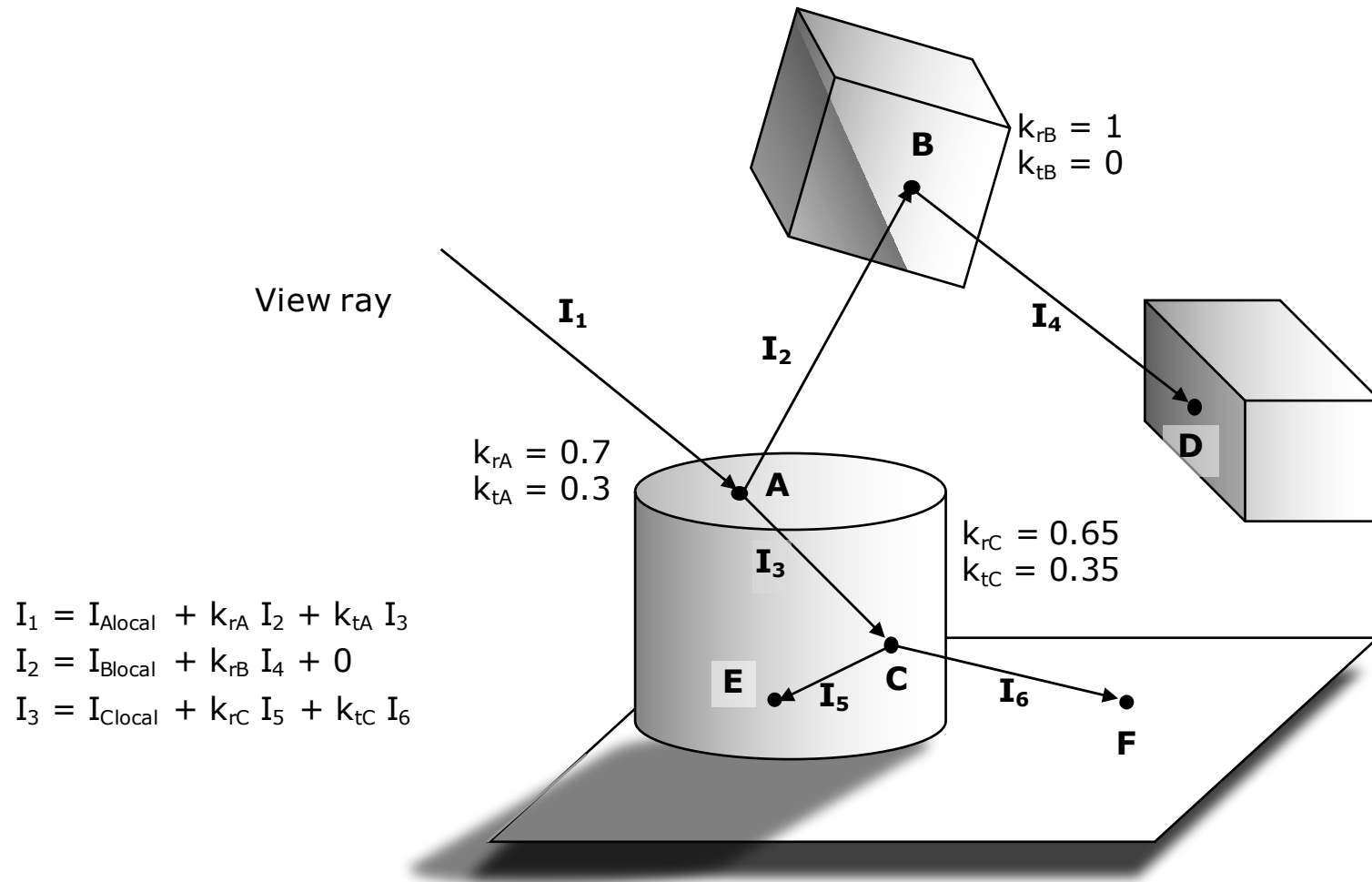
$$\begin{aligned} I &= I_{\text{local}} + I_{\text{global}} = \\ &= I_a k_a + I_i [k_d (N \cdot L) + k_s (N \cdot H)^n + k_t (N \cdot H')^n] + k_{\text{rg}} I_r + k_{\text{tg}} I_t \end{aligned}$$

Ray tracing - recursive computation

$$I(P) = I_{\text{local}}(P) + k_{rg} I(P_r) + k_{tg} I(P_t)$$



Ray tracing - recursive computation



Ray tracing – shadow computation

- ❑ Global reflection model incorporates the shadow computation
- ❑ At each point computes the local illumination I_{local}
- ❑ If the point is not direct illuminated it is in shadow
- ❑ Shadow feeler – view ray from the point to the light source, whether another object lies in the path of L direction and obstructs the direct illumination of the point
- ❑ If the obstruction object is opaque the I_{local} reduces to ambient component
- ❑ In simple ray tracing approach the transmitted shadow feeder is not considered at all

Implementation

- Adaptive depth control
- Bounding volume extensions
- Precomputing the first intersections
- Spatial coherence
- Auxiliary data models
 - Octree
 - SEADS
 - BSP

Adaptive recursion level

- In 1983, Hall and Greenberg proposed to adapt the recursion level according with the properties of the materials the light interacts with
- Rays are attenuated in various ways as they pass through the scene
 - The reflected ray is attenuated by the global specular reflection coefficient (k_{rg})
 - The refracted ray is attenuated by the global transmission coefficient (k_{tg})
 - The ray through several intersections is attenuated by the product of these coefficients (e.g. $k_{rD} k_{rB} k_{rA}$)
 - If the product of the coefficients is below some threshold the contribution of the ray is below a significant value -> there will be no point in ray tracing back further than that level (e.g. ray 4, further than D)
- For RGB color system there are three color contributions

Adaptive recursion level - algorithm

TraseazaRaza (vectori start, vectori directie, int nivel, culori culoare
float pondere_cumulata)

```
{
    vectori    punct_intersectie, directie_reflexie, directie_refractie;
    culori     culoare_locala, culoare_reflectata, culoare_refractata;

    if (pondere_cumulata < prag_pondere)
        culoare = black;

    else
        if ( nivel > nivel_maxim )
            culoare = black;
        else {
            Intersectează raza cu toate obiectele și determină cel
            mai apropiat punct de intersecție, dacă există, fata de începutul razei.

            if (nu există intersecție)
                culoare = culoarea de fond;
            else {
                culoare_locala = contribuția modelului local de culoare la
                                punctul de intersecție;
            }
        }
}
```

Adaptive recursion level - algorithm

```
    Calculează direcția razei reflectate;  
    TraseazaRaza(punct_intersectie,  
                directie_reflexie,  
                nivel+1,  
                culoare_reflectata,  
                pondere_cumulata * coeficient_reflexie_al_suprafetei);  
    Calculează direcția razei refractate;  
    TraseazaRaza(punct_intersectie,  
                directie_refractie,  
                nivel+1,  
                culoare_refractata  
                pondere_cumulata * coeficient_refractie_al_suprafetei);  
    Combina(culoare, culoare_locala,  
            ponderea_locala_suprafata,  
            culoare_reflectata,  
            coeficient_reflexie_al_suprafetei,  
            culoare_refractata,  
            coeficient_refractie_al_suprafetei);  
    }  
}  
} // TraseazaRaza
```

Bounding volume extensions

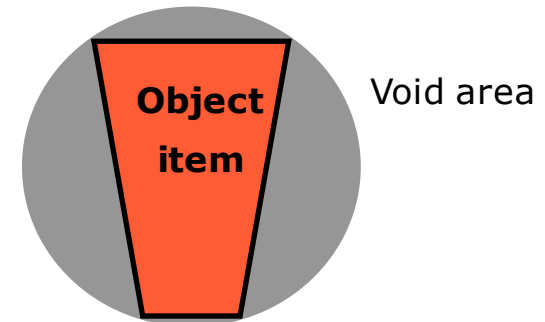
- Bounding volumes – intersection tests
- In 1984, Weghorst, Hooper și Greenberg
 - Defined a “void” area of a bounding volume – difference in area between the orthogonal projections of the object and bounding volume onto a plane perpendicular to the ray
 - Defined a cost function for the intersection test

Cost function:

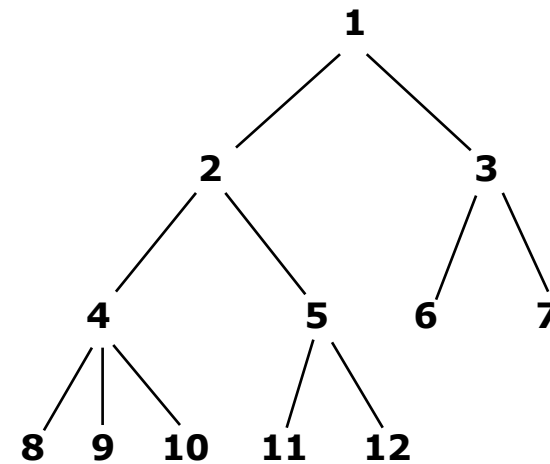
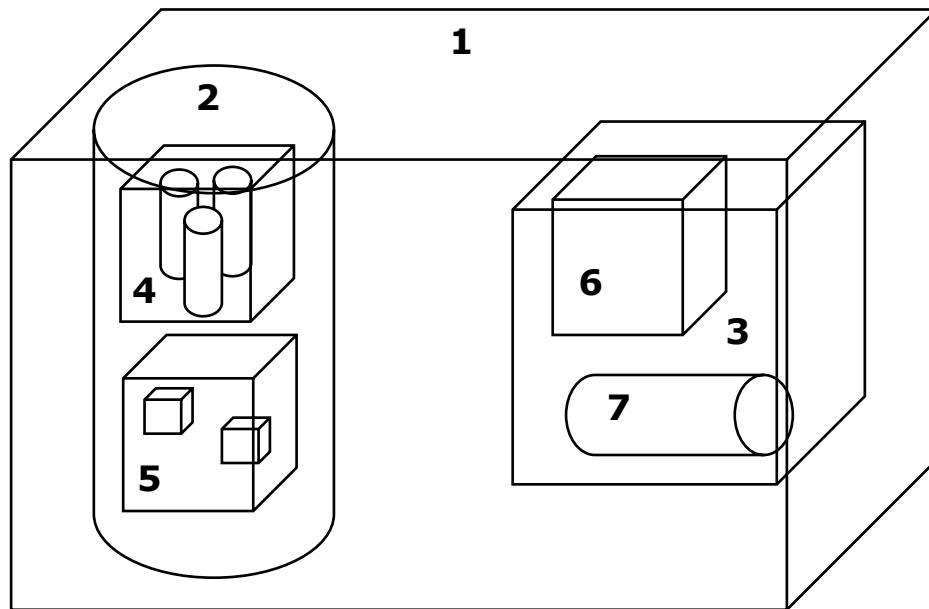
$$T = bB + iI$$

where:

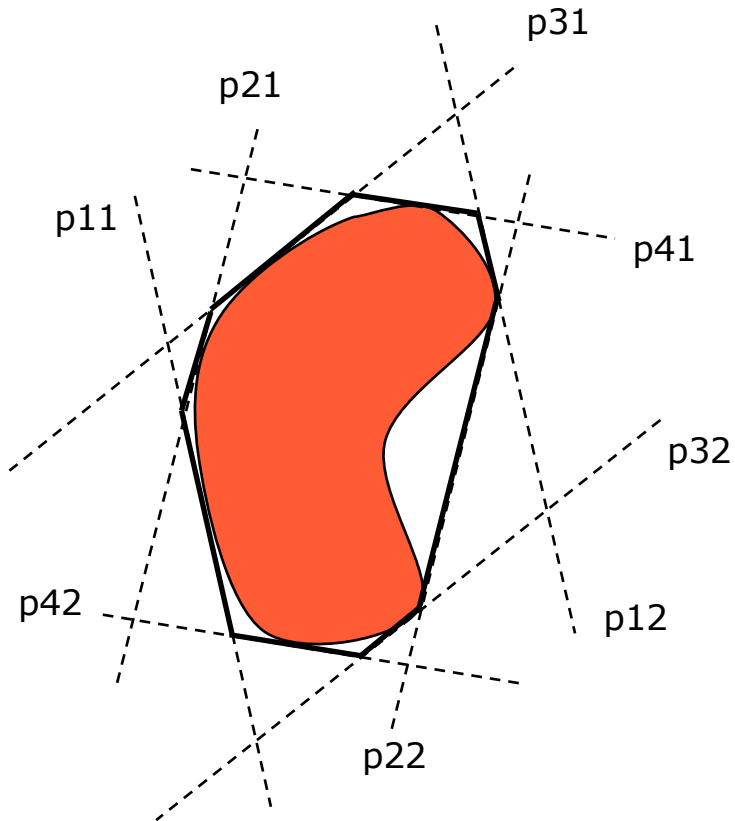
- T - total cost function
- b - number of intersection tests for the bounding volume
- B - cost of an intersection test for the bounding volume
- i - number of times the included item is tested for intersection ($i \leq b$)
- I - cost of the intersection test for an included item



Bounding volume - Hierarchical structure



Bounding volume – Convex hull



- Bounding volumes made from polyedra consisting of pairs of parallel planes:
 $(p_{11}, p_{12}),$
 $(p_{21}, p_{22}),$
 $(p_{31}, p_{32}),$
 (p_{41}, p_{42})
- Simple computation of intersections

Bounding volume extensions

❑ Disadvantages:

1. Ray tracing efficiency depends on how well the objects fill the space of the bounding volumes
2. Each ray must be tested against the bounding extent of every object and the search time is a function of the scene complexity
3. Computation of an appropriate hierarchical structure of bounding volumes is difficult

Precomputing the first intersections

- Even for highly reflective scenes the average depth of ray tracing is 1 and 2
- In 1984, Weghorst, Hooper and Greenberg developed a precomputed ray tracer:
 - Intersection of the initial ray is evaluated in a preprocessing phase
 - Use for the first intersection computation a hidden surface algorithm (more efficient than the ray tracing computation)
 - A modified Z-Buffer algorithm computes for each pixel in the image plane, a pointer to the object visible at that pixel

Spatial coherence

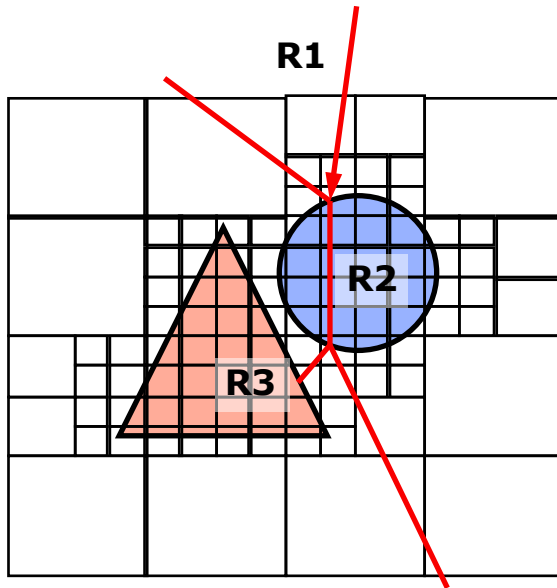
- ❑ Glassner (1984), Kaplan (1985), and Fujimoto, Tanaka and Iwata (1986)
- ❑ Space occupied by the scene is subdivided into regions
- ❑ Rather than checking a ray against all objects or sets of bounded objects, test just the region occupied by any objects
- ❑ Spatial coherence, spatial subdivision, space tracing
- ❑ Approach:
 1. Preprocess the space to set up an auxiliary data structure that contains information about the object occupancy of the space
 2. Rays are then traced using this auxiliary data structure to enter the object data structure
- ❑ Main advantages against the bounding volume extension:
 - Makes the rendering time constant (for a given resolution)
 - Eliminates the dependency on the scene complexity
- ❑ Octree representation, BSP, and SEADS

Ray tracer - Kaplan's requirements

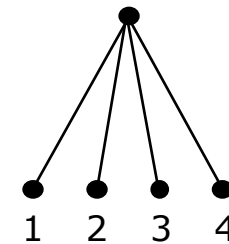
1. Computation time should be relatively independent of scene complexity
2. Per ray time should be relatively constant, and not depend on the origin or direction of the ray
3. Computation time should be in a acceptable range (e.g. one hour, few minutes)
4. The algorithm should not require the user to supply hierarchical object description or object clustering information
5. The algorithm should deal with a wide variety of primitive geometric types and easily extensible to new types
6. The use of coherence should not reduce the applicability to parallel processing or other advanced architectures

Octree representation in ray tracing

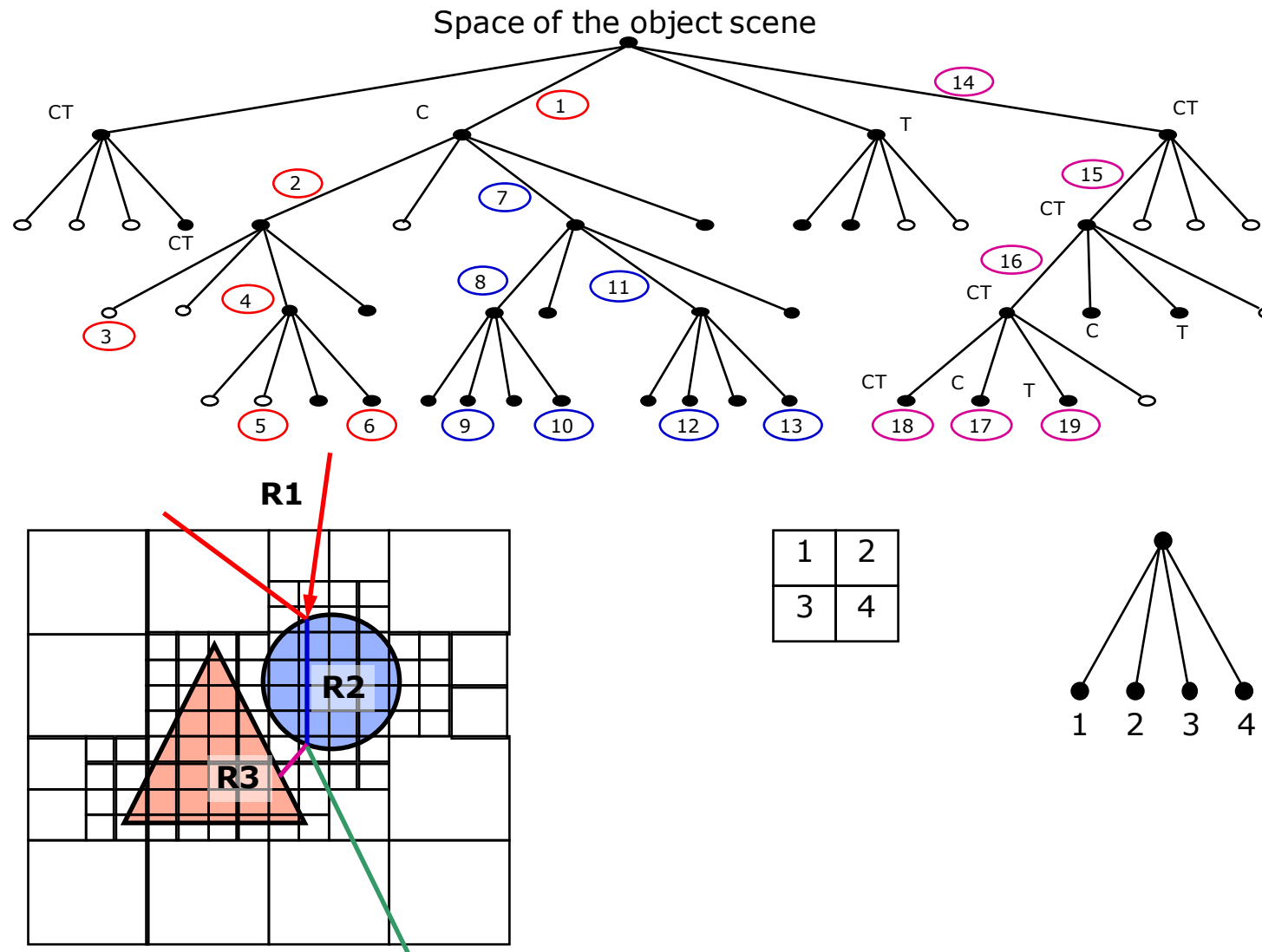
- ❑ Use spatial coherence
- ❑ Instead of doing intersection computation between the ray and every object in the scene, the algorithm traces the ray from sub-region to sub-region in the subdivision of occupied space
- ❑ Rapidly find the node in the octree that corresponds to a sub-region that a ray is passing through, and immediately accesses the objects that are on, or close to, the path of the ray



1	2
3	4

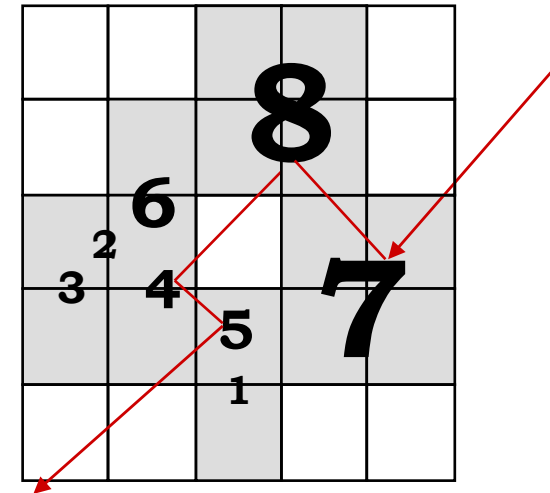


Octree representation in ray tracing



SEADS in ray tracing

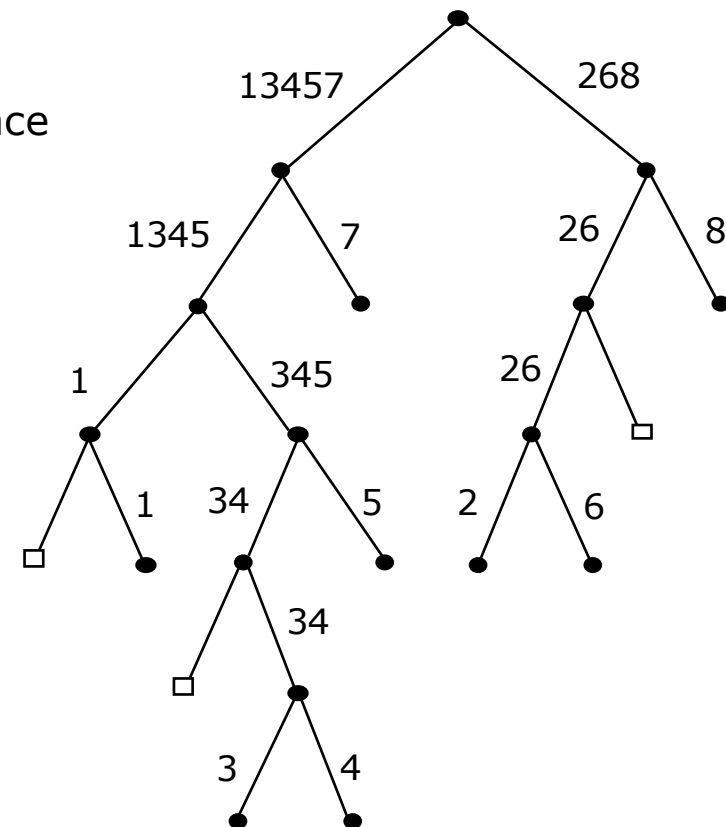
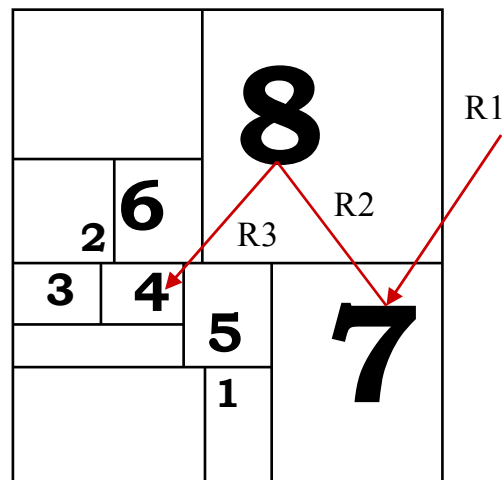
- ❑ SEADS (Spatially Enumerated Auxiliary Data Structure)
- ❑ Proposed by Fujimoto, Tanaka and Iwata (1986)
- ❑ Divide all the occupied space into equally sized voxels regardless of occupancy by objects
- ❑ Enables very fast tracking of rays from region to region
- ❑ Tracking algorithm: extension to 3D of the DDA (Digital Differential Analyzer) algorithm – used in 2D graphics to select a subset of the pixels passed through by a line
- ❑ 3D-DDA does not involve floating point multiplication and division, just addition, subtraction and comparison (integer based)
- ❑ The practice combines SEADS with Octree representation



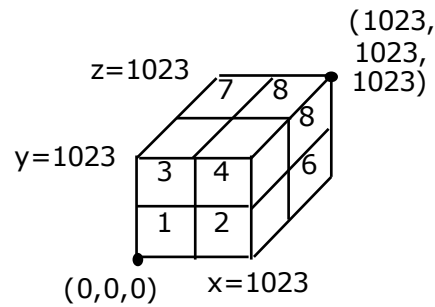
		8	8	
	6	8	8	
23	246		7	7
3	4	15	7	7
		1	8	

BSP in ray tracing

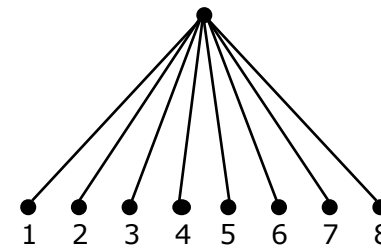
- ❑ BSP (Binary Space Partitioning)
- ❑ Proposed by Fuchs (1980), and used by Kaplan (1985)
- ❑ Divide the object space in two subspaces (e.g. parallel with coordinate system planes)
- ❑ Division plane: front-face space, back-face space
- ❑ Binary tree



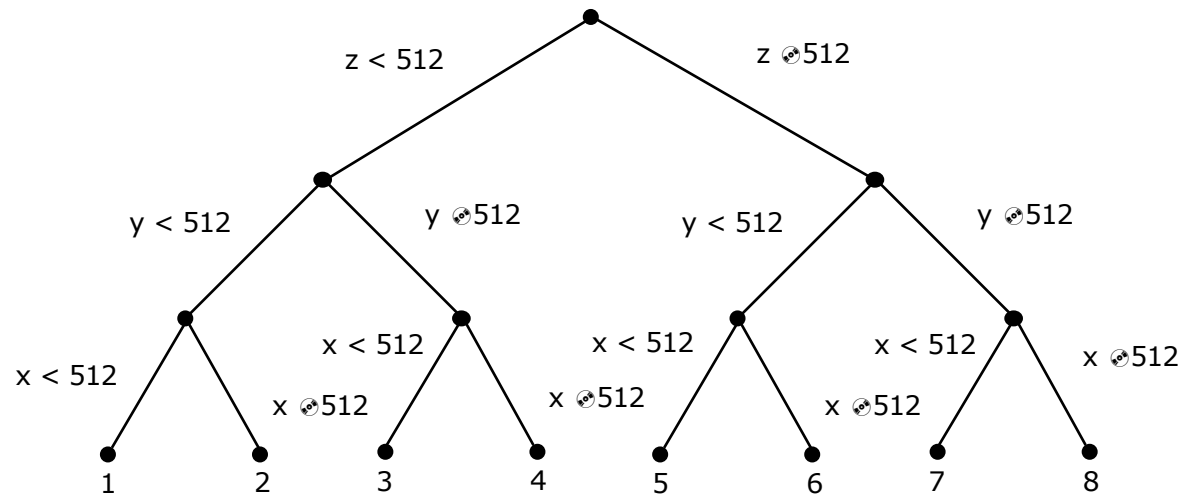
BSP in ray tracing - example



(a) BSP in 3D



(b) Octree



(c) BSP tree

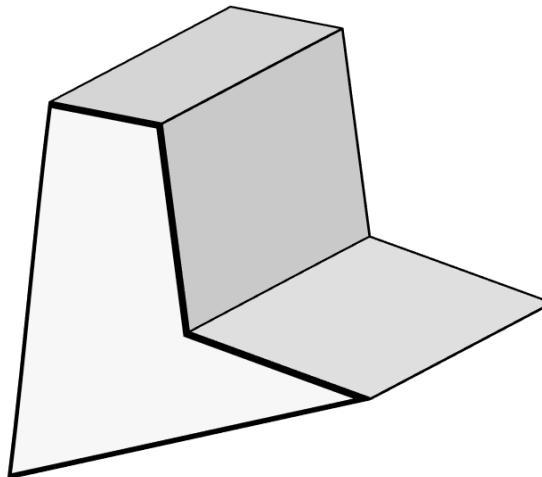
Questions and problems

1. What is the reason to stop the reflection and the refraction of the viewing ray?
2. Why a point in the scene is illuminated even it is not visible by the light source (direct illuminated)?
3. What parameters describes in the ray tracing process the properties of the following objects: a) cube of glass; b) cube of chalk; c) cube of shiny steel; d) cube of varnished wood?
4. Why the ray tracing simulates the illumination within the real world, even it uses the abstract concept of the viewing ray?
5. How the ray tracing method can be parallelized?
6. Explain the possibility of combining in ray tracing the computations of illumination and the texture mapping.

Questions and problems

7. Explain why in computation of intersections the classification of cases speeds up the rendering? Exemplify the case of a sphere.
8. Exemplify and explain the ray tracing algorithm for a scene of two cubes considering the following conditions: 2 point light sources; 4x4 pixels screen resolution; one cube is of transparent glass; one cube is of shiny steel.
9. Exemplify and explain the computation of intersection by the Cyrus-Beck algorithm for the object in the figure 1.

Figure 1



Questions and problems

10. Why in the new local reflection model at least one of the specular and transmitted components could be not null?
11. Exemplify and explain the using of an efficient bounding box system for the object in figure 1. Is a single box efficient enough?
12. Exemplify and explain the use of the SEADS structure for ray tracing of the object in figure 1.

Graphics Processing Units (Part 1)

Contents

- ❑ GPU definition
- ❑ Motivation and examples
- ❑ GPU History
- ❑ GPU companies
- ❑ Processing capability of GPU vs. CPU
- ❑ Graphics pipeline
- ❑ Graphics pipeline bottlenecks
- ❑ Haptics rendering pipeline
- ❑ Classic GPU pipeline
- ❑ GPU graphics pipeline
- ❑ Classic vs. unified shader architecture
- ❑ Unified shader architecture
- ❑ NVIDIA GeForce 8800
- ❑ Shader languages
- ❑ Software technologies
- ❑ Distributed visualization
- ❑ Sort-first pipeline
- ❑ Sort-middle pipeline
- ❑ Sort-last pipeline
- ❑ Load Balancing
- ❑ References

Graphics Processing Unit

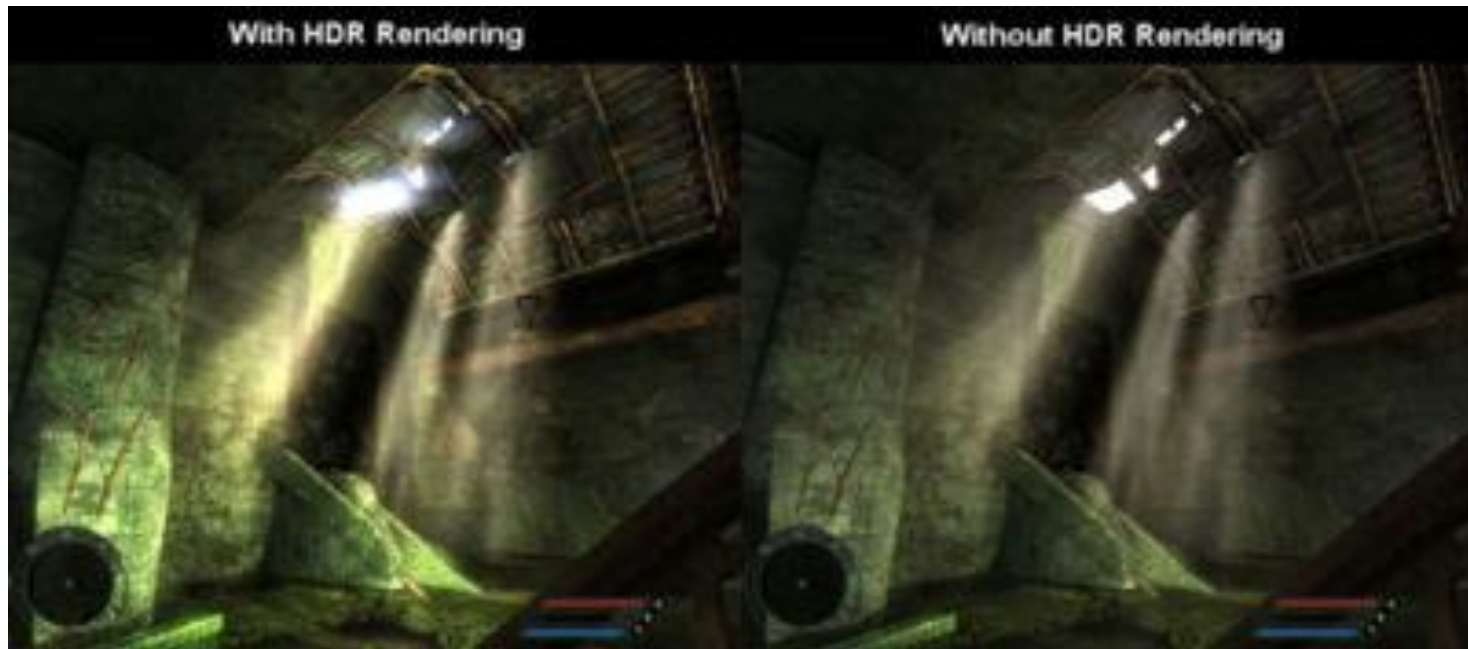
- ❑ August 31, 1999 - introduction of the Graphics Processing Unit (GPU) for the PC industry.
- ❑ Technical definition of a GPU: "a single chip processor with integrated transform, lighting, triangle setup/clipping, and rendering engines that is capable of processing a minimum of 10 million polygons per second." (NVIDIA site: <http://www.nvidia.com/object/gpu.html>).

GPU - Motivation

- ❑ In classic applications the CPU performs the most graphics operations (e.g. transformations for coordinate spaces, projections, clipping, rasterization, light computation, texture mapping etc).
- ❑ Necessity to offload graphics operations from CPU to specialized hardware for vector and matrix based processing.
- ❑ Application control over the stages in the graphics pipeline.
- ❑ GPU provides a general pipeline solution instead of specialized hardware for each graphics algorithm.
- ❑ Programmability and scalability.
- ❑ GPUs are used in:
 - embedded systems
 - mobile phones
 - personal computers
 - Workstations
 - game consoles

GPU based rendering

- High Dynamic Range Imaging (HDRI): Preservation of detail in large contrast differences



A comparison of HDR and LDR in the game *Far Cry*.

Wikipedia, "High Dynamic Range Rendering", Wikipedia , January 2008,
<http://en.wikipedia.org/wiki/High_dynamic_range_rendering>

GPU based rendering



From *Half-Life 2: Lost Coast*, comparing HDR to LDR on how
(a) light is reflected.

(b) light passes through transparent materials.

Wikipedia, "High Dynamic Range Rendering", Wikipedia, January 2008,
<http://en.wikipedia.org/wiki/High_dynamic_range_rendering>



GPU based rendering



From *Serious Sam SE* , comparing
Anisotropic Filtering (AF)

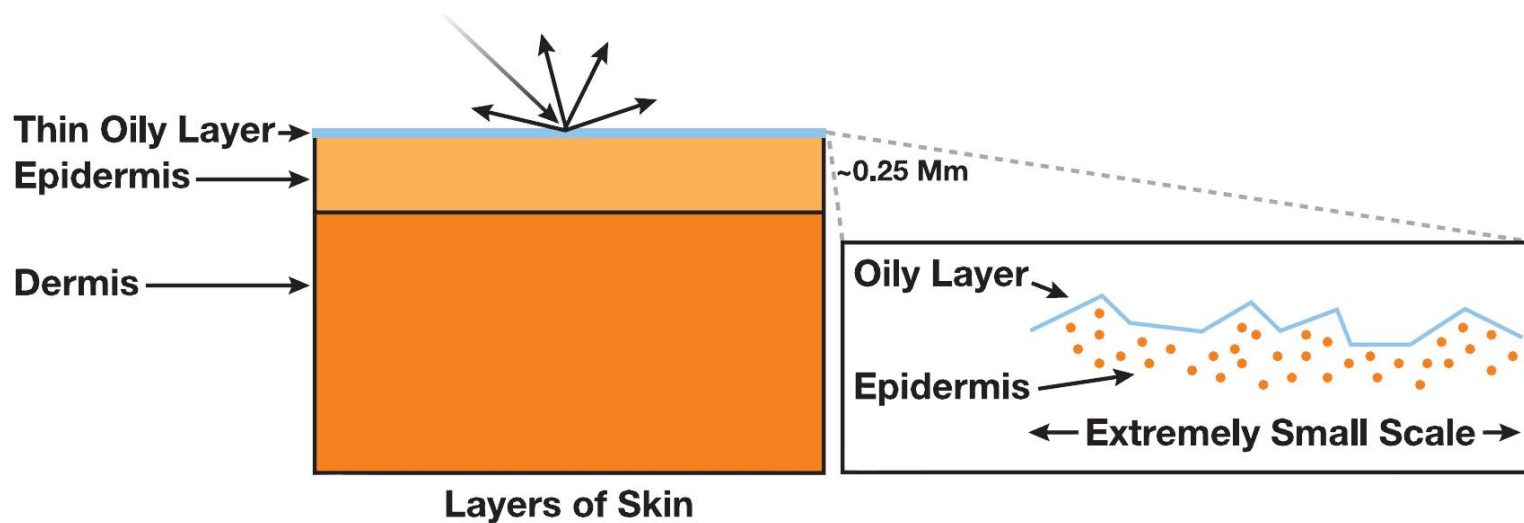
(a) with AF.

(b) without AF.

The Naked Truth About Anisotropic
Filtering, January 2008,
[http://www.extremetech.com/article2/
0,1697,1157434,00.asp](http://www.extremetech.com/article2/0,1697,1157434,00.asp)

GPU based simulation and rendering

- The Appearance of Skin as materials of two components:
 1. Surface reflectance (specular)
 2. Subsurface reflectance (diffuse)
- Use Physically based models
 1. Specular BRDF
 2. Efficient subsurface scattering approximations



GPU based simulation and rendering

- ❑ Skin subsurface scattering
 - Gives skin soft appearance and color
 - Expensive to compute
 - Essential for realistic appearance



Eugene d'Eon, GPU Gems 3:Advanced Skin Rendering, SIGGRAPH 2007

GPU based rendering



4:1

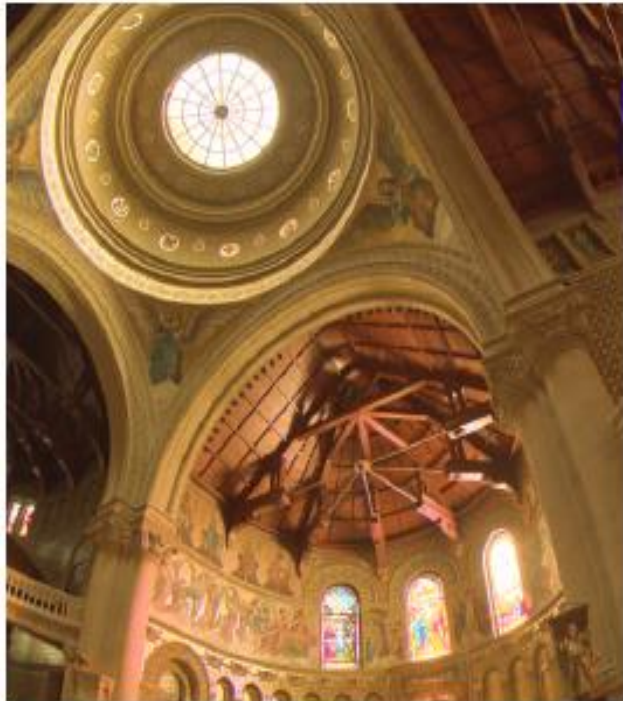


53:1

A series of 512x512 HDR images that have been tone mapped on the GPU. All images were generated at nearly 30 Hz.

Modern graphics architectures have replaced stages of the graphics pipeline with fully programmable modules. Therefore, it is now possible to perform fairly general computation on each vertex or fragment in a scene. In addition, the nature of the graphics pipeline makes substantial computational power available if the programs have a suitable structure.

GPU based rendering



56:1



140:1

It allows an interactive application to achieve higher levels of realism by rendering with physically based, unclamped lighting values and high dynamic range texture maps. The tone mapping operator can be extended to include a time-dependent model, which is important for interactive behavior.

GPU based rendering



621 : 1



905 : 1

Ref: Nolan Goodnight, Rui Wang, Cliff Woolley, and Greg Humphreys, Interactive Time-Dependent Tone Mapping Using Programmable Graphics Hardware, Proceedings of the 14th Eurographics Workshop on Rendering, Per Christensen and Daniel Cohen-Or (Editors) 2003, pp.180-192.

Graphics Processing Units (Part 2)

Contents

- ❑ GPU definition
- ❑ Motivation and examples
- ❑ GPU History
- ❑ GPU companies
- ❑ Processing capability of GPU vs. CPU
- ❑ Graphics pipeline
- ❑ Graphics pipeline bottlenecks
- ❑ Haptics rendering pipeline
- ❑ Classic GPU pipeline
- ❑ GPU graphics pipeline
- ❑ Classic vs. unified shader architecture
- ❑ Unified shader architecture
- ❑ NVIDIA GeForce 8800
- ❑ Shader languages
- ❑ Software technologies
- ❑ Distributed visualization
- ❑ Sort-first pipeline
- ❑ Sort-middle pipeline
- ❑ Sort-last pipeline
- ❑ Load Balancing
- ❑ References

GPU History

1. Before 1998)

- Ex. NVIDIA RIVA TNT, ATI Rage, Matrox G400, and 3dfx Voodoo3.
- Triangle rasterization with textures.
- No vertex transformation.
- Limited set of math operations to combine textures for pixel's color.

2. 1999-2000

- NVIDIA GeForce 256 and GeForce2, ATI Radeon 7500, and S3 Savage3D
- Offload vertex transformations and lighting from the CPU to the GPU
- Increase the number of fixed pixel-pipelines
- Configurable, but not yet programmable

GPU History

1. 2001

- NVIDIA GeForce3 and GeForce4 Ti, Microsoft Xbox, and ATI Radeon 8500
- Vertex programmability (the first programmable GPU)
- Pixel level partial configuration

2. 2002 - 2006

- NVIDIA GeForce FX family (i.e. 6800, 7800), ATI Radeon 9700
- Vertex and fragment programmability
- NVIDIA 7950 and ATI Radeon X1950
- NVIDIA creates CUDA architecture

3. 2007

- Unified Shader Architecture (vertex, geometry, and fragment processors)
- GPGPU, NVIDIA GeForce 8800. Tesla is the first dedicated General Purpose GPU

GPU History

6. After 2006

- Unified Shader Architecture (vertex, geometry, and fragment processors)
- GPGPU, NVIDIA GeForce 8, 9, 200, and 400 Series (e.g. 8800), support DirectX 10, CUDA - first C programming language API for GPGPU
- Tegra mobile processor, Tesla based first supercomputer, Apple adopts GeForce 9400M for MacBook, PhysX – physics engine (2008)
- GeForce 400 Series: April 2010, Fermi Chip, 512 stream processors, in 16 groups of 32 processors, support OpenGL 4.0 and Direct3D 11.
- GeForce 400 Series (November 2010), GTX 580 (20% faster than GTX 480)
- Kepler architecture, GeForce 600, Tegra 3 on tablets, virtualized GPU onto Cloud (2012)
- Tegra 4 quad-core mobile processor (2013)

GPU History

Last product:

- GeForce GTX TITAN Z is a dual GPU gaming engine, 5760 cores and 12 GB of memory.

Memory interface 768 bits (384 bits/ GPU processor), Memory bandwidth 672 GB/sec.

Maximum digital resolution 4096x2160.

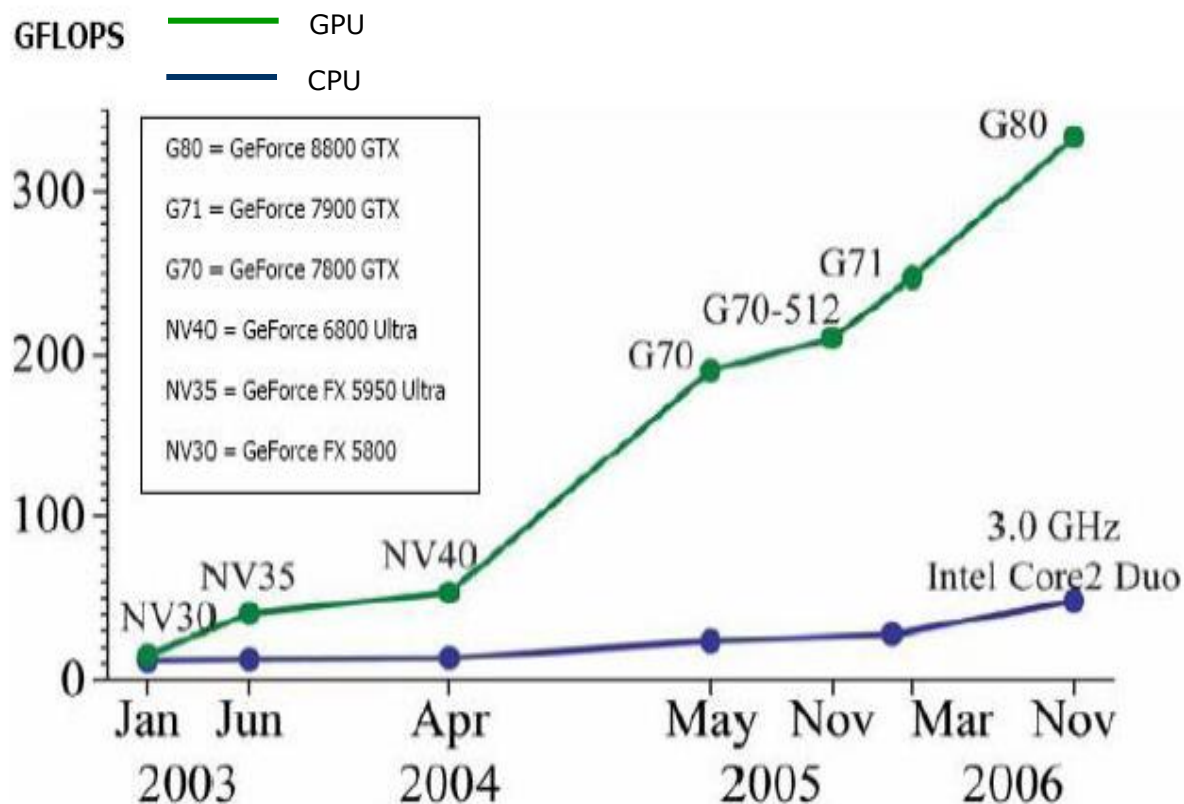
Tendency:

- Unify the GPU and CPU functionality within the same chip (e.g. join NVIDIA GPU and ARM RISC architecture)
- Graphics cluster of GPUs
- GPGPU cluster in heterogeneous architectures (e.g. Grid, Cloud)

GPU companies

- ❑ **AMD** (producer of the ATI Radeon graphics chip line)
AMD (Advanced Micro Devices) takes control over ATI in 2006 and produces the Radeon graphics chipsets under the AMD brand: AMD 580X CrossFire, AMD 690G, <http://www.amd.com/>
- ❑ **NVIDIA** (producer of the NVIDIA Geforce graphics chip line), founded in 1993. <http://www.nvidia.com/>
- ❑ **Intel** also produces GPUs that are built into their motherboards, such as the 915 and 945. These chips are less optimum for playing 3D games. <http://www.intel.com>
- ❑ **S3 Graphics** (purchased by HTC in 2011), founded in 1989. Produces ViRGE, Savage 3D, SavageXP, Chrome series, <http://www.s3graphics.com/>
- ❑ **Matrox**, founded in 1978, <http://www.matrox.com/>. Produces graphics card Matrox Mystique, Matrox G400, Matrox Parhelia.

Processing capability of GPU vs. CPU



GPU definition

- In 1999 NVIDIA invents the graphics processing unit, GeForce 256, defined as:

“a single-chip processor with integrated transform, lighting, triangle setup/clipping, and rendering engines that is capable of processing a minimum of 10 million polygons per second”.

The modern GPUs process over 2×10^9 polygons per second.

Performance GeForce 6, 7 and 8 series

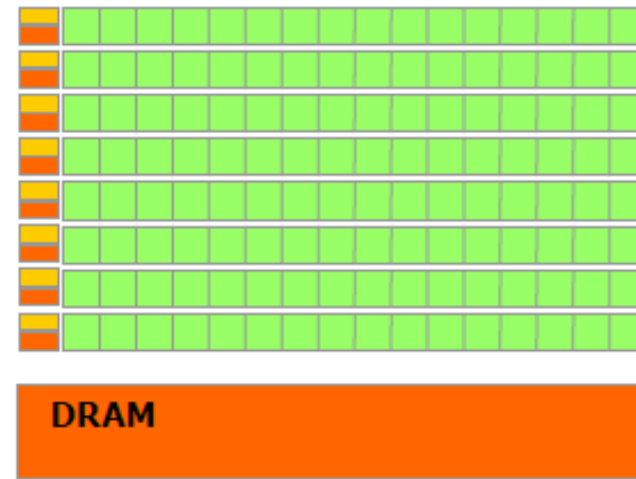
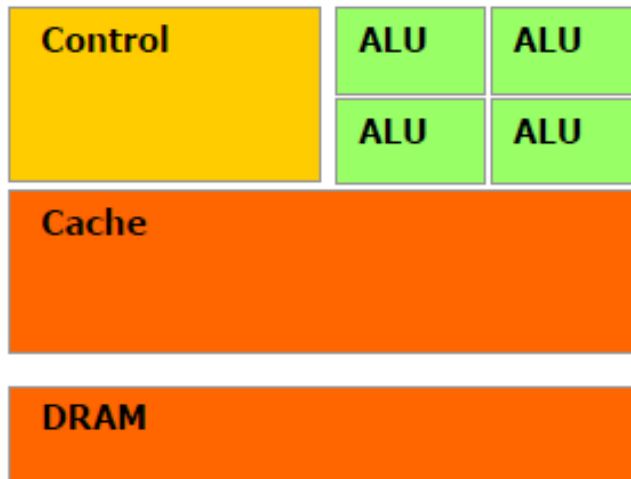
	Geforce 6800 Ultra	Geforce 7800	Geforce 8800 Ultra
Stream processors			128
Memory bandwidth	35.2 GB/sec	54.4 GB/sec	103.7 GB/sec
Fill rate (texels/sec.)	6.4 billion	13.2 billion	39.2 billion
Vertices per second	600 million	1100 million	
Pixel per clock (peak)	16	24	48
Performance	53 Gigaflops	200 Gigaflops	330 Gigaflops

GPU vs CPU

- ❑ GPU is designed for high parallel operations while CPU executes programs serially
- ❑ GPU has many parallel execution units and higher transistor counts, while CPU has a few execution units and higher clockspeeds
- ❑ GPU has much deeper pipelines (several thousand stages vs 10-20 for CPU)
- ❑ GPU has significant faster and more advanced memory interfaces as they need to shift around a lot more data than CPU

GPU vs CPU Architecture

- ❑ GPU is specialized for compute-intensive, highly parallel computation, and graphics rendering
- ❑ Designed such that more transistors are devoted to data processing rather than data caching and flow control



NVIDIA GeForce 8800 GTX

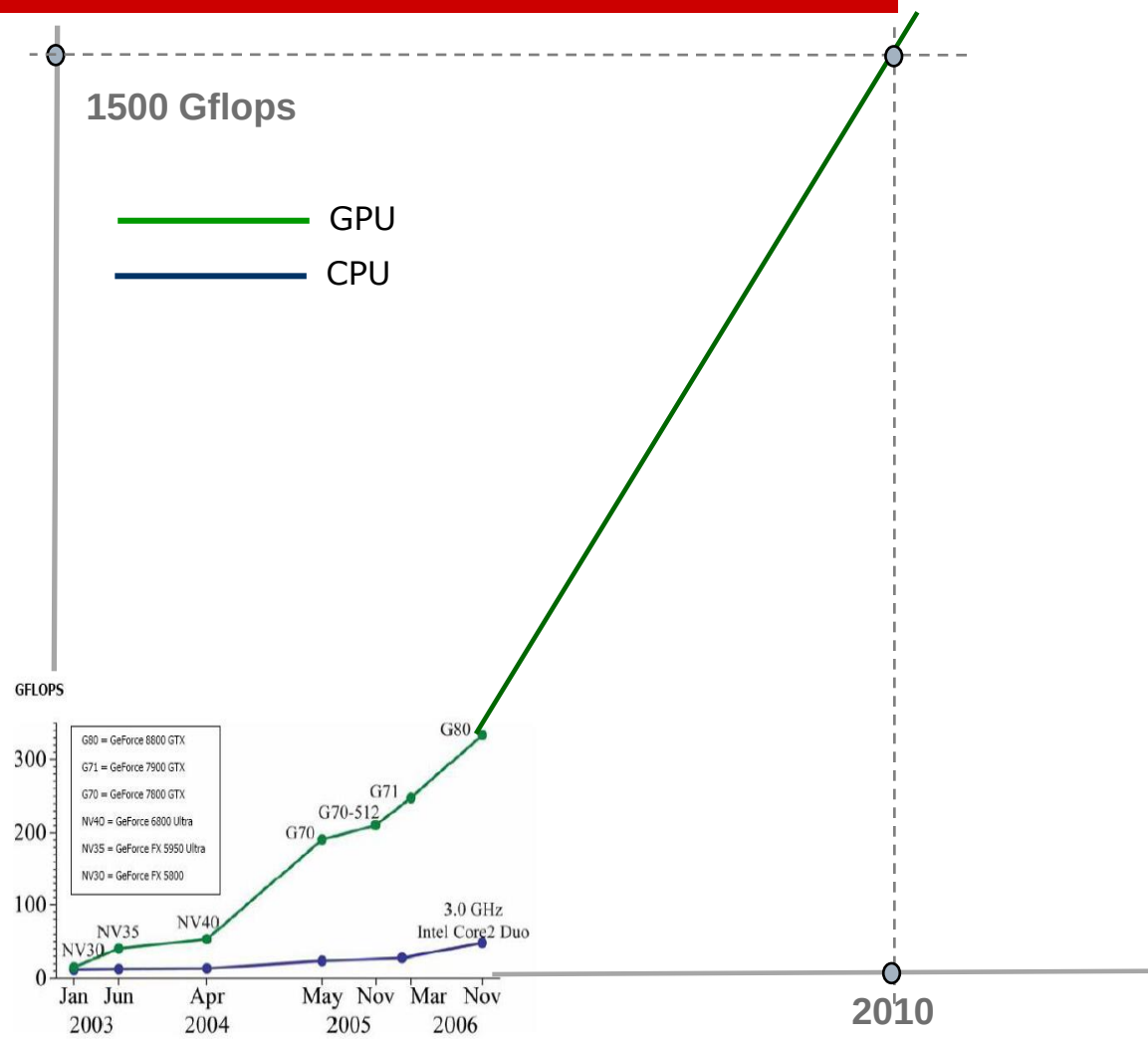


Picture taken from Wikipedia, the free encyclopedia.

Performance GeForce GTX performance

	Geforge 9800 GTX	GeForce GTX 285	GeForce GTX 480	GeForce GTX 580	GeForce GTX 590
Stream (Shader) processors	128	240	480	512	1024
Core Clock (MHz)	675	648	700	772	1215
Memory bandwidth	70.4 GB/s	159 GB/s	177 GB/s	192. GB/s	327.7 GB/s
Memory amount	512 MB	1024 MB	1536 MB	1536 MB	3072 MB
Memory bus width	256 bits	512 bits	384 bits	384 bits	768 bits
Texture fill rate (texels/sec.)	43.2 GT/s	51.8 GT/s	42 GT/s	49.4 GT/s	77.7 GT/s
Performance (Gflops)	648	1063	1345	1581	2480

Processing capability of GPU vs. CPU



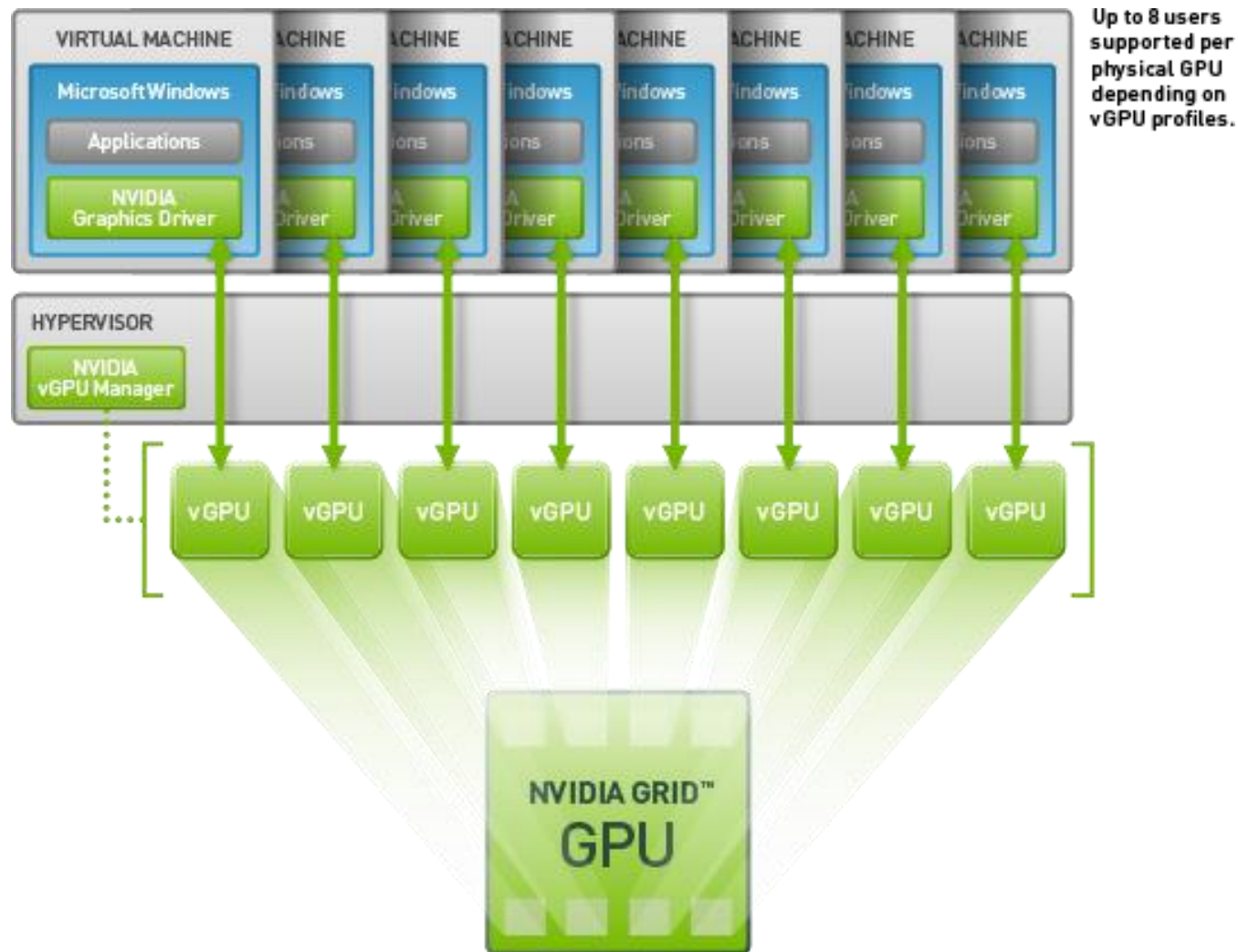
NVIDIA Grid Virtual GPU Technology

- ❑ Multiple users can share a single GPU, providing true PC performance and compatibility.
- ❑ Secure and graphics-intensive applications to designers and engineers wherever they are.
- ❑ Users access and use all their office productivity and design applications, virtually, from nearly any device—even on their own notebooks and portable devices—just as they would at their desks.
- ❑ For institutions in higher education, NVIDIA GRID vGPU enables scalable lab environments that deliver tools to students wherever they learn best.

Grid vGPU key features

	GRID K1	GRID K2
Number of GPUs	4 x KG107 GPUs	2 x GK104GPUs
Total NVIDIA CUDA cores	768 (192 cores/ GPU)	3072 (1536 cores/ GPU)
Total memory size	16 GB DDR3 (4 GB/ GPU)	8 GB GDDR5 (4 GB/ GPU)
Compatibiliy	DirectX 11, Shader 5.0, OpenGL 4.3	

Virtual GPU Technology

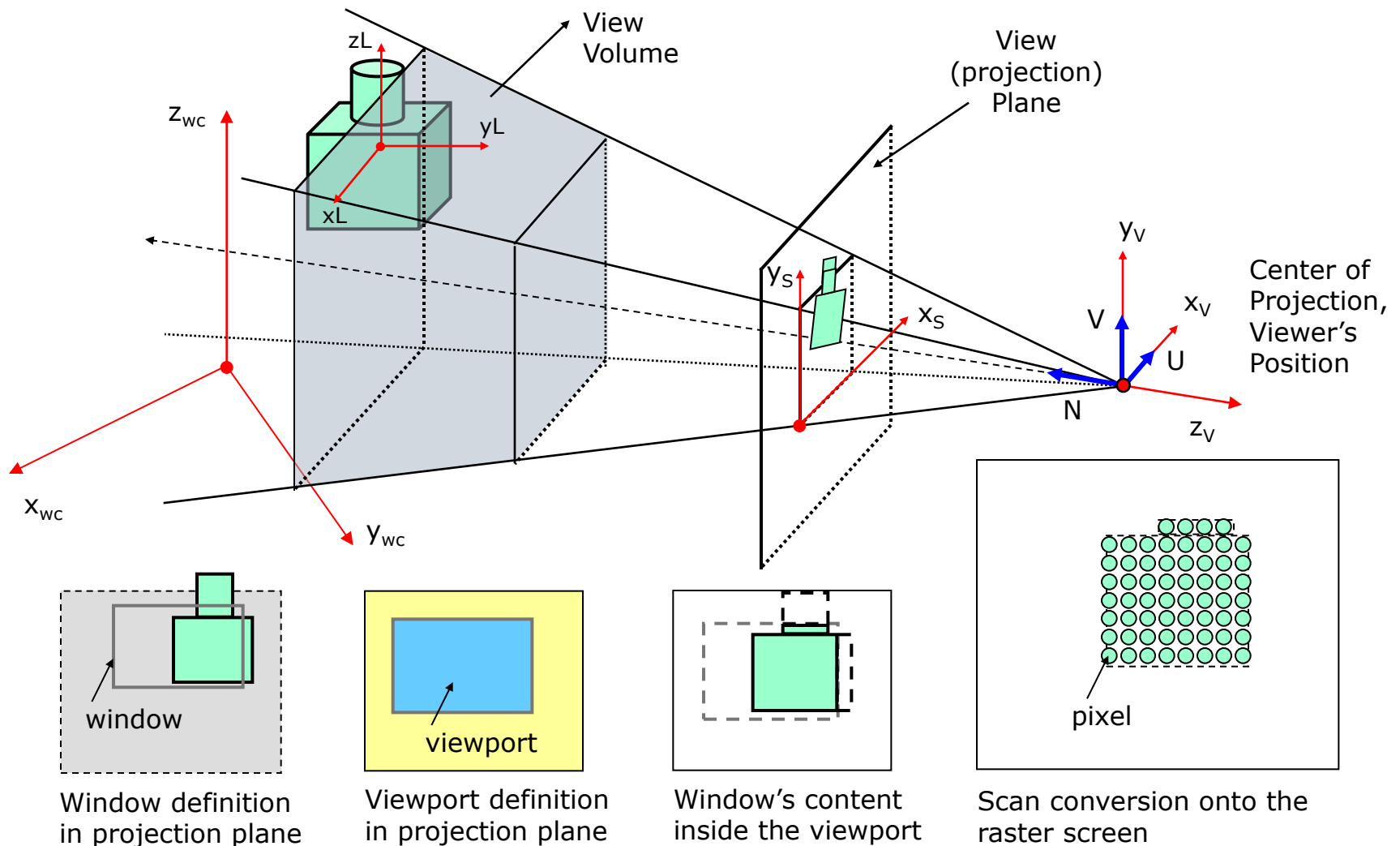


NVIDIA GeForce GTX 580

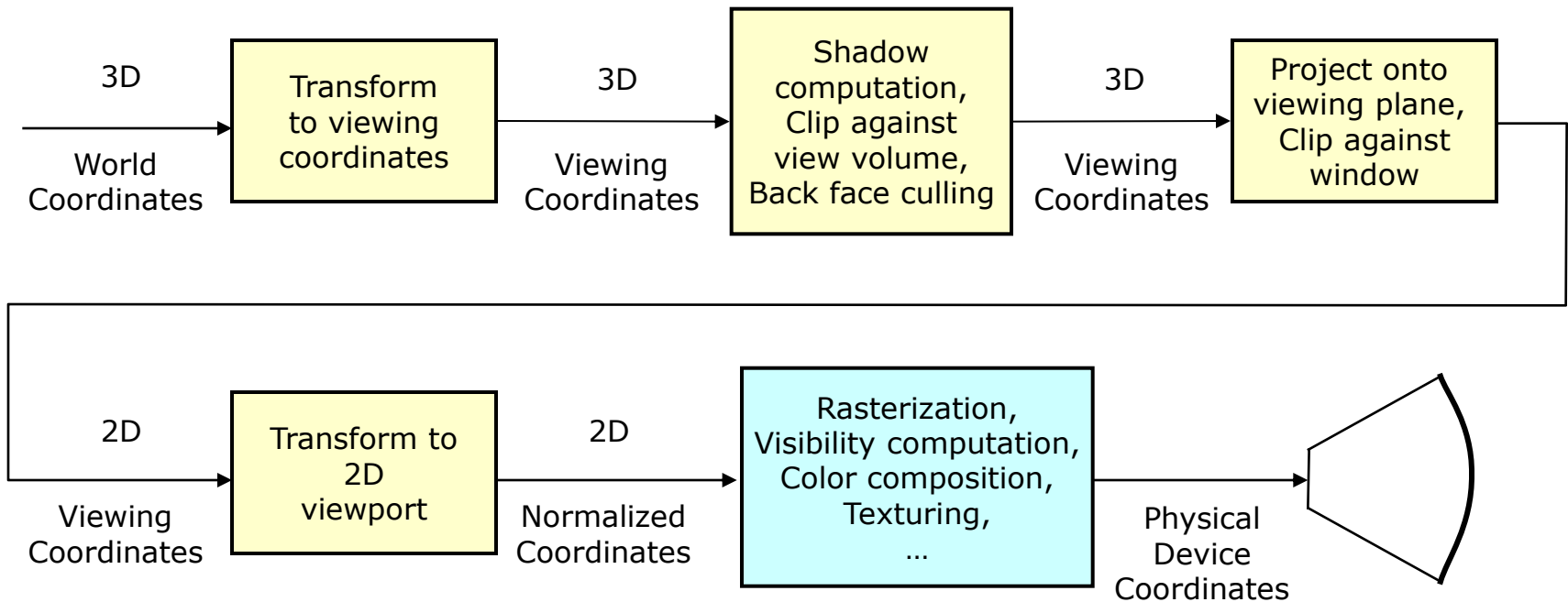


Picture taken from Wikipedia, the free encyclopedia.

Conceptual model of 3D viewing process



Viewing transformations pipeline



GPU pipeline

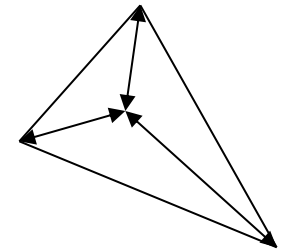
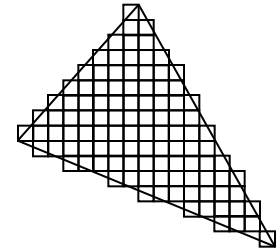
- ❑ GPU receives geometry information from the CPU as input and provides a picture as output
- ❑ Host interface:
 - Communication bridge between CPU and GPU.
 - Receives commands from the CPU and pulls geometry information from system memory.
 - Outputs a stream of vertices in object space with all their associated information (e.g. normals, texture coordinates, vertex color, ...)
- ❑ Vertex processing:
 - Receives vertices from the host interface in object space and outputs them in screen space.



GPU pipeline

□ Triangle setup:

- In this stage the triangles that are hidden or are outside the viewing frustum are removed
- Geometry information becomes raster information (screen space geometry is the input, pixels are the output)
- A fragment (virtual pixel) is generated if its center is inside the triangle
- Every fragment has its attributes computed to be the perspective correct interpolation of the three vertices that define the triangle (e.g. distance, z coordinate)



GPU pipeline

□ Fragment processing:

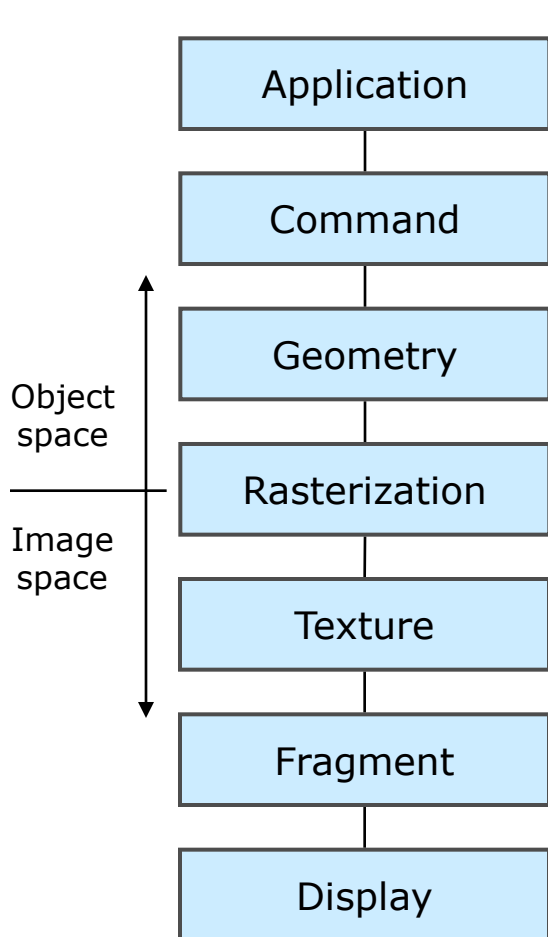
- Each fragment is a set of attributes (i.e. position, normal, texcoord, etc), which are processed to give the final color of the pixel
- Processing consists of texture mapping and mathematical operations
- !!! It is the bottleneck in graphics applications (e.g. games, animation, etc)

□ Memory interface

- Final fragment colors are written to the framebuffer
- A part of fragments are rejected by the zbuffer, stencil and alpha tests



Canonical graphics pipeline



□ Application:

- Simulation
- Input event handlers
- Modify data structures
- Database traversal
- Primitive generation
- Utility functions

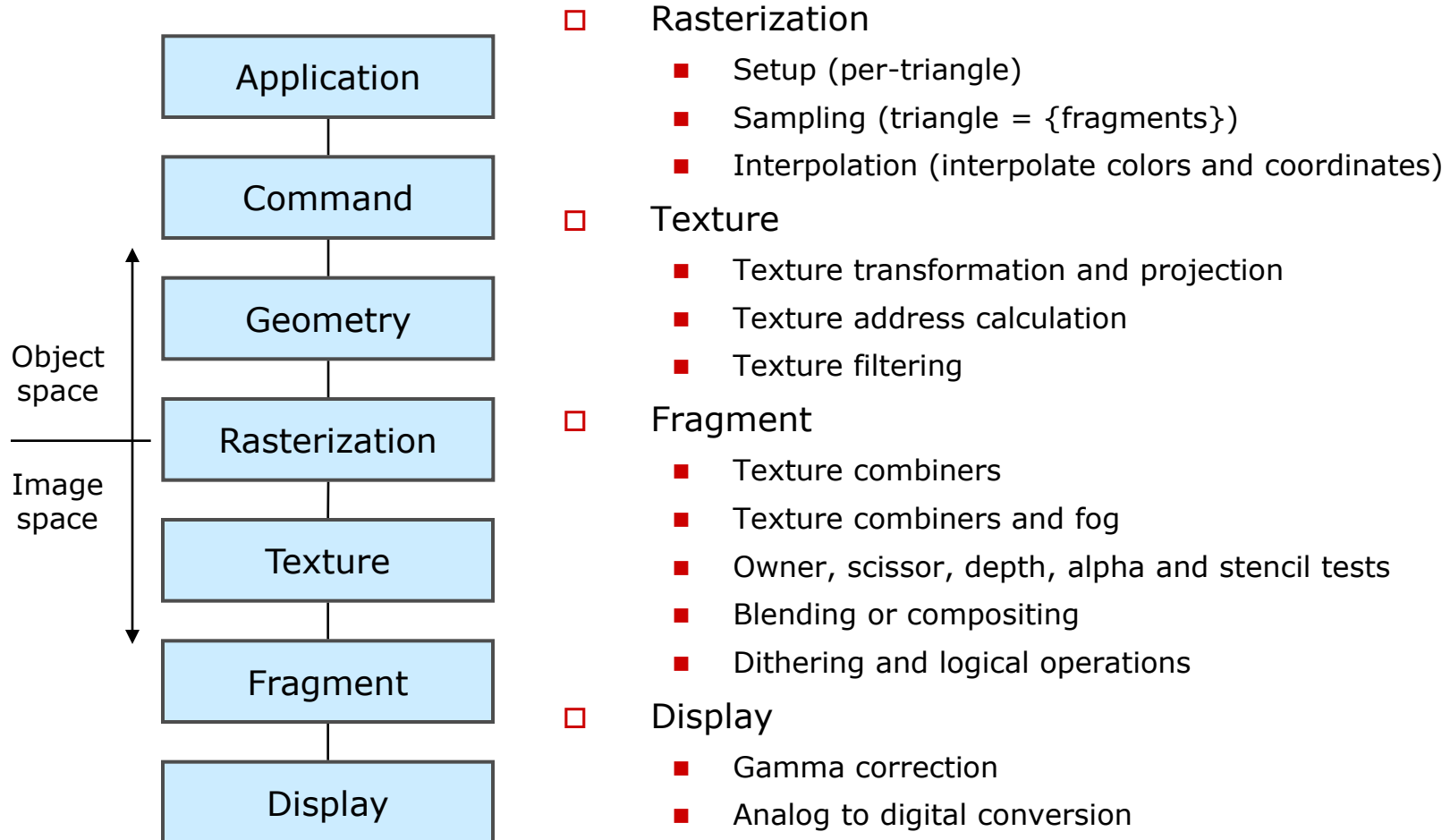
□ Command:

- Command buffering
- Command interpretation
- Unpack and perform format conversion
- Maintain graphics state

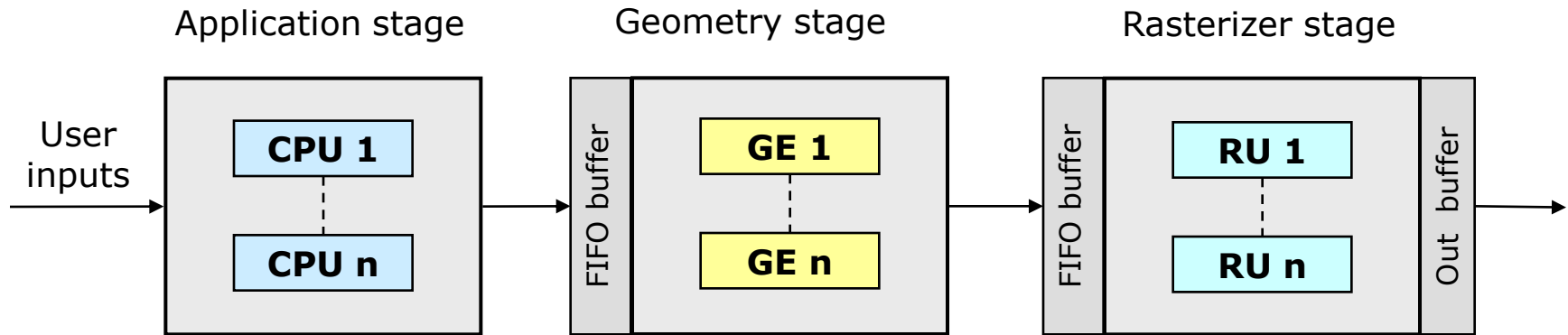
□ Geometry

- Transform and projection
- Evaluation of polynomials for curved surfaces
- Transform and projection (object → image space)
- Clipping, culling and primitive assembly
- Lighting (light sources and surface reflection)
- Texture coordinate generation

Canonical graphics pipeline



Graphics rendering pipeline



- Done in software.
- Reads: world geometry state and user inputs.
- May change the view to the simulation or change orientation of virtual objects.

- Done in software or hardware.
- Model transformations: translation, rotation, scaling;
- Lighting computations.
- Scene projection.
- Slipping.
- Mapping.

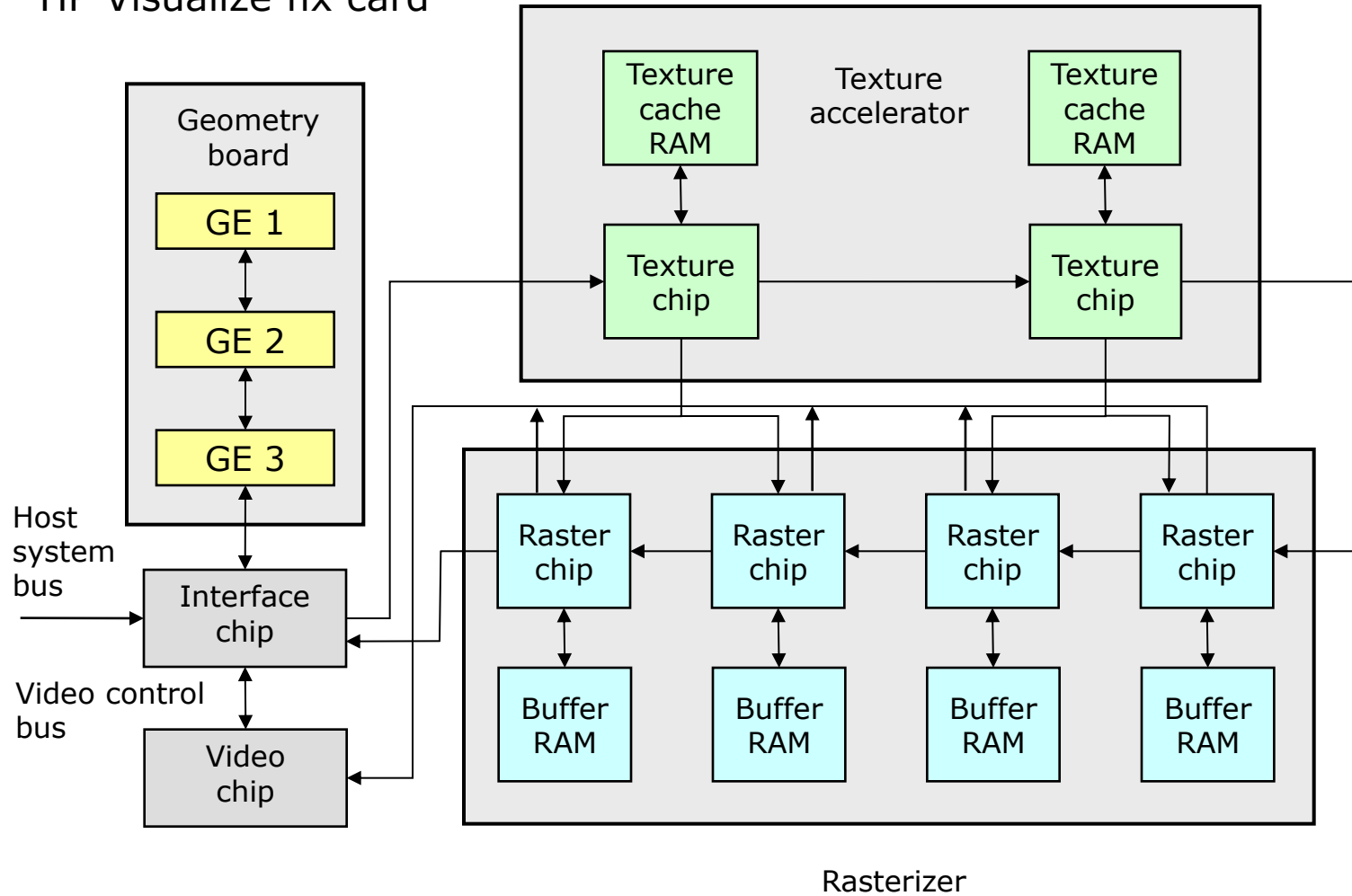
- Done in hardware.
- Converts vertex information (color and texture) into pixel information.
- Performs antialiasing.

Graphics rendering pipeline

- ❑ Parallel architectures for performance speed-up.
- ❑ Application stage – several CPUs.
- ❑ Geometric Stage – several GEs.
 - ❑ Each processing a portion of the scene geometry.
- ❑ Use FIFO buffers to store intermediate results.
- ❑ Each RU renders a patch of frame pixels.
- ❑ Frame buffer:
 - ❑ Pixel color info – color buffer.
 - ❑ Depth or z value – Z buffer.
- ❑ Modern architectures use double buffering:
 - ❑ Front buffer – stores the image to be displayed.
 - ❑ Back buffer – stores the new scene.
 - ❑ Then back buffer is displayed and the new image is rendered in the front buffer – flicker is reduced.

Rendering pipeline example

HP Visualize fix card



Graphics pipeline bottlenecks

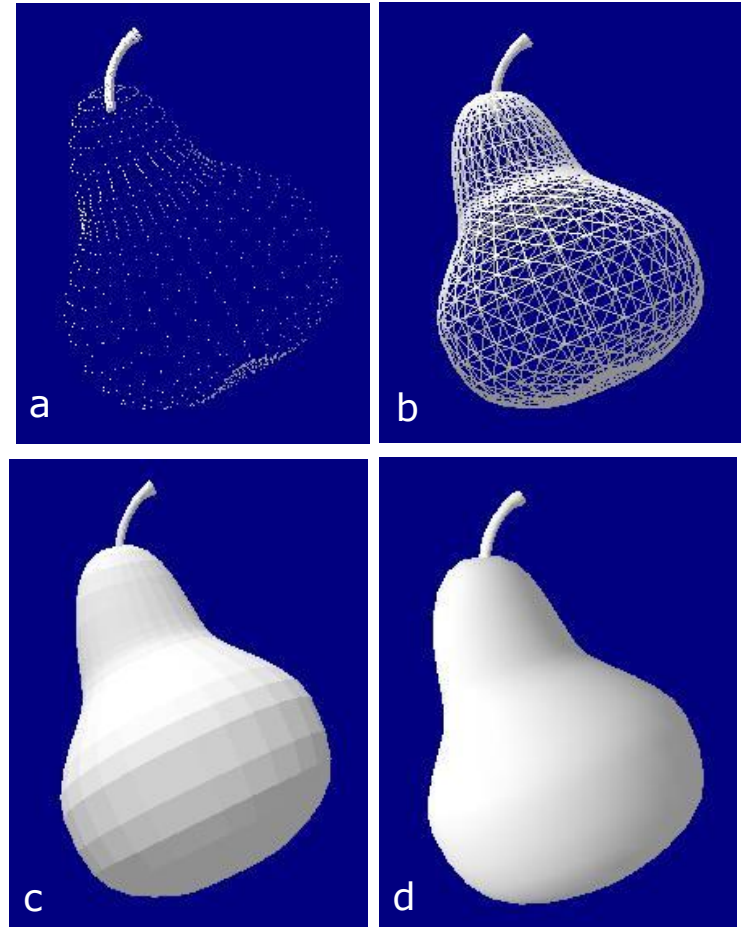
- ❑ One of the 3 stages will be the slowest
 - ❑ Operates at 100%
 - ❑ Set the overall pipeline processing time
- ❑ Bottleneck appears in the
 - ❑ Application Stage (most cases)
 - ❑ Geometry stage
 - Ex. if the frame refresh rate increases when the number of light sources is reduced. Transform-limited pipeline
 - ❑ Rasterizing stage
 - Ex. if pipeline output increases when the size or the resolution of the display window is reduced. Fill-limited pipeline

Optimizations in Application stage

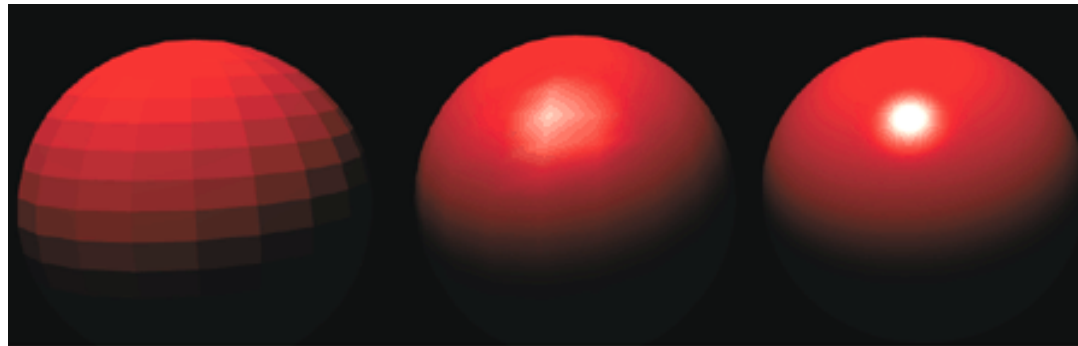
- ❑ Replace CPU with a faster one.
- ❑ Add another CPU.
- ❑ Reduce the load of the CPU
 - ❑ reduce scene complexity
 - ❑ 3D models with lower polygonal number
- ❑ Optimize the simulation software.
- ❑ Use lower precision arithmetic instead of double precision.
- ❑ Minimize the number of divisions in the code.

Optimizations in Geometry stage

- ❑ Polygon shading modes
 - (a) dot modeling
 - (b) wireframe.
 - (c) faceted or flat shading.
Color in polygon is constant.
 - (d) Gouraud shading.
Intensity interpolation.
- ❑ Phong shading
- ❑ Texturing
- ❑ Computational load increases.
Frame refresh rate decreases.
- ❑ Optimize the pipeline by using the type of polygon for which its rendering hardware is optimized.



Optimizations in Geometry stage



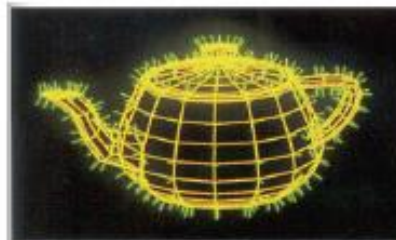
Flat

Gouraud

Phong

Wireframe

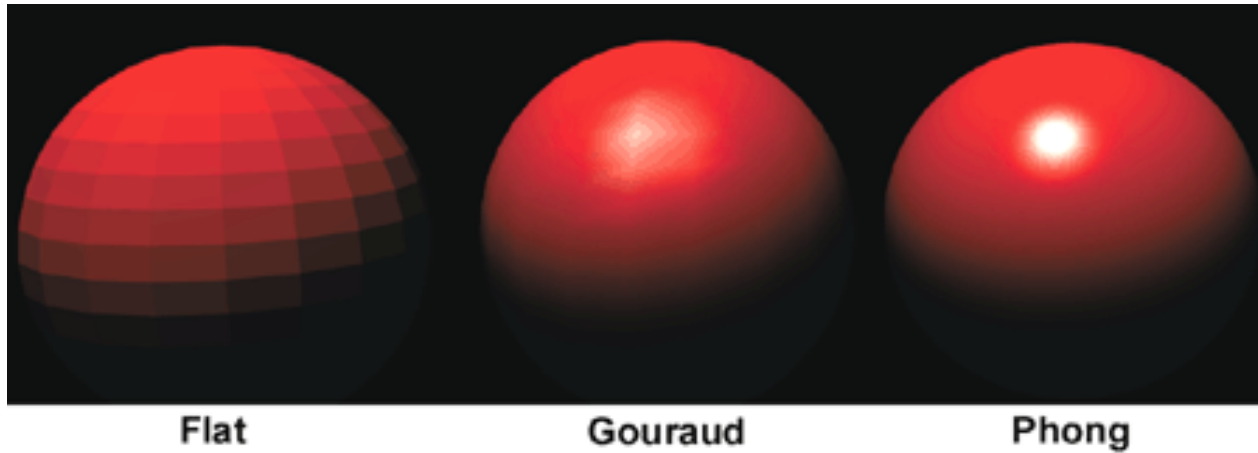
Flat



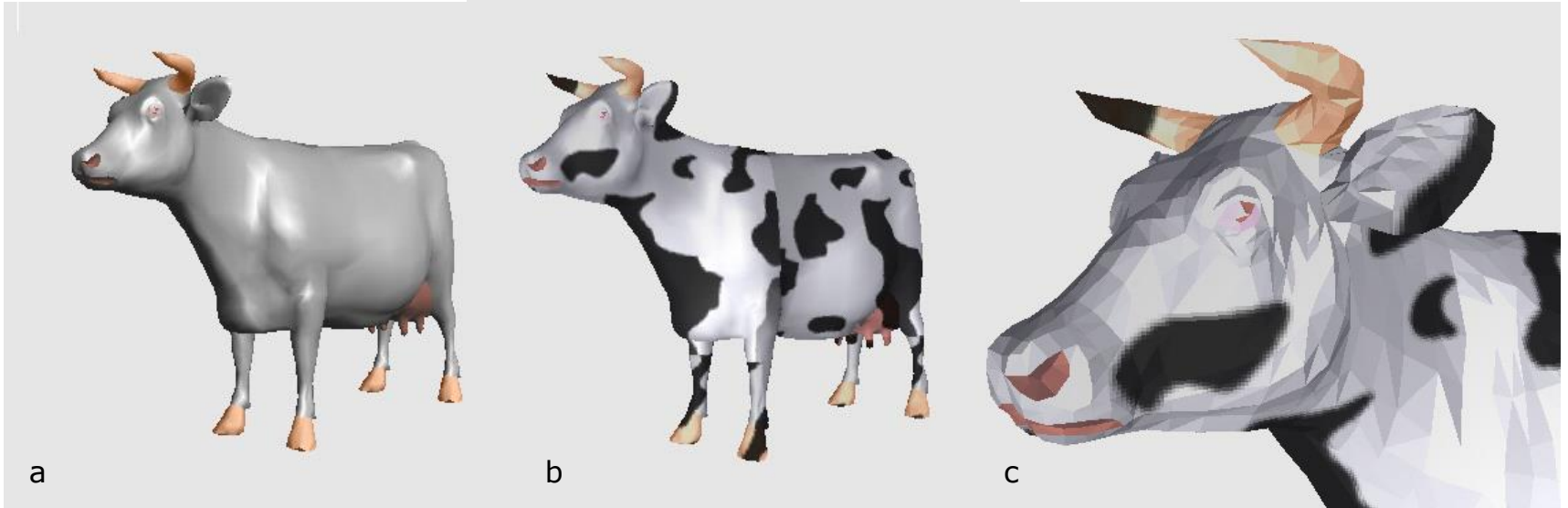
Gouraud

Phong

Optimizations in Geometry stage



Optimizations in Geometry stage

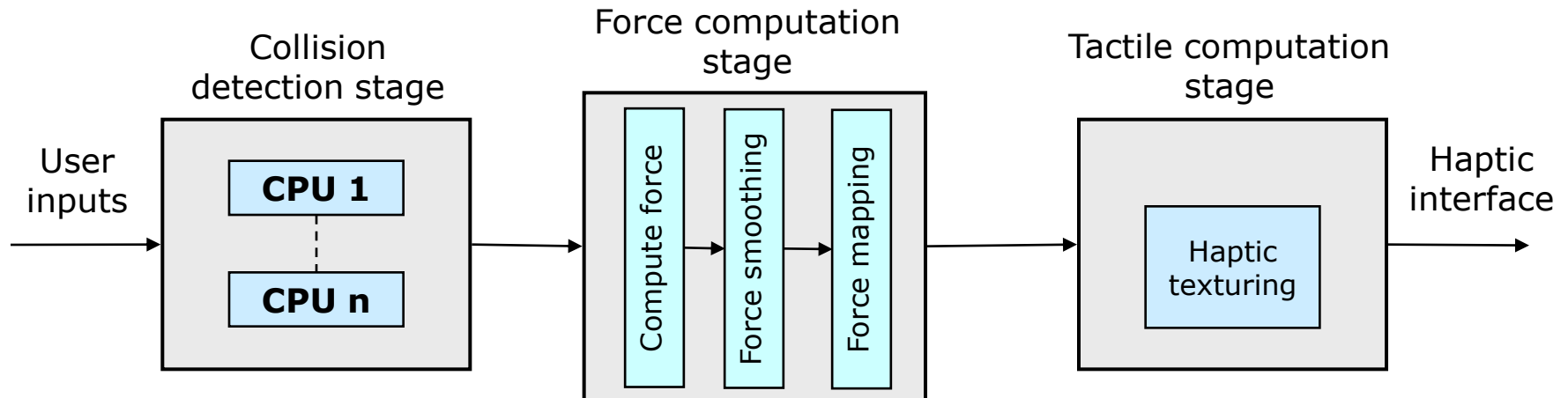


(a) Phong shading. (b) Phong shading and texture. (c) Flat shading and texture.

Optimizations in Rasterizing stage

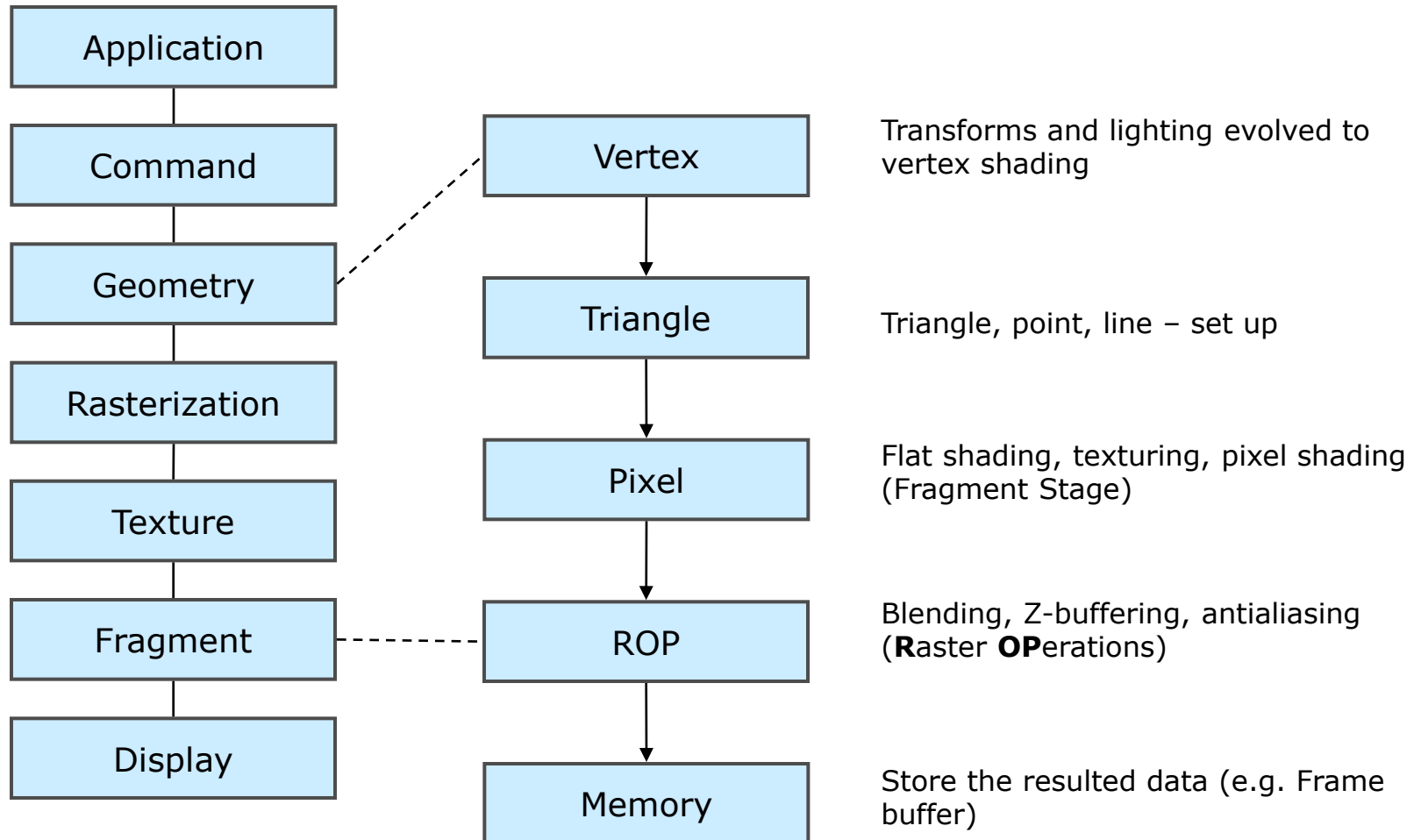
- ❑ Reduce the size of the display window.
- ❑ Reduce the resolution
 - ❑ Less work for the RUs.
- ❑ Assume that RU works at 100% and the other stages work at 80%
 - ❑ Increase the model complexity - in order to balance the load.
- ❑ If GEs operate at 100%
 - ❑ Increase display resolution or size to balance load.

Haptics rendering pipeline

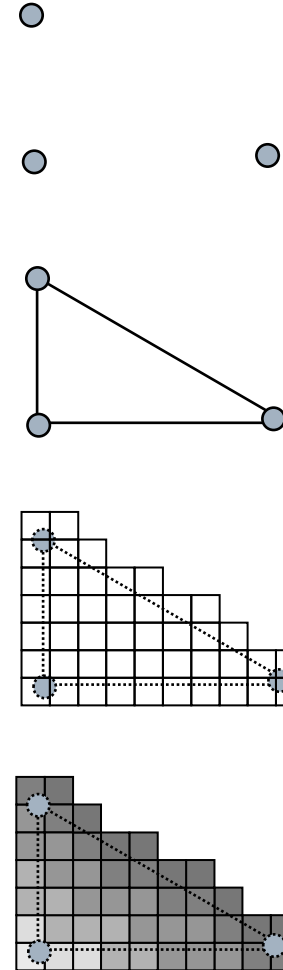
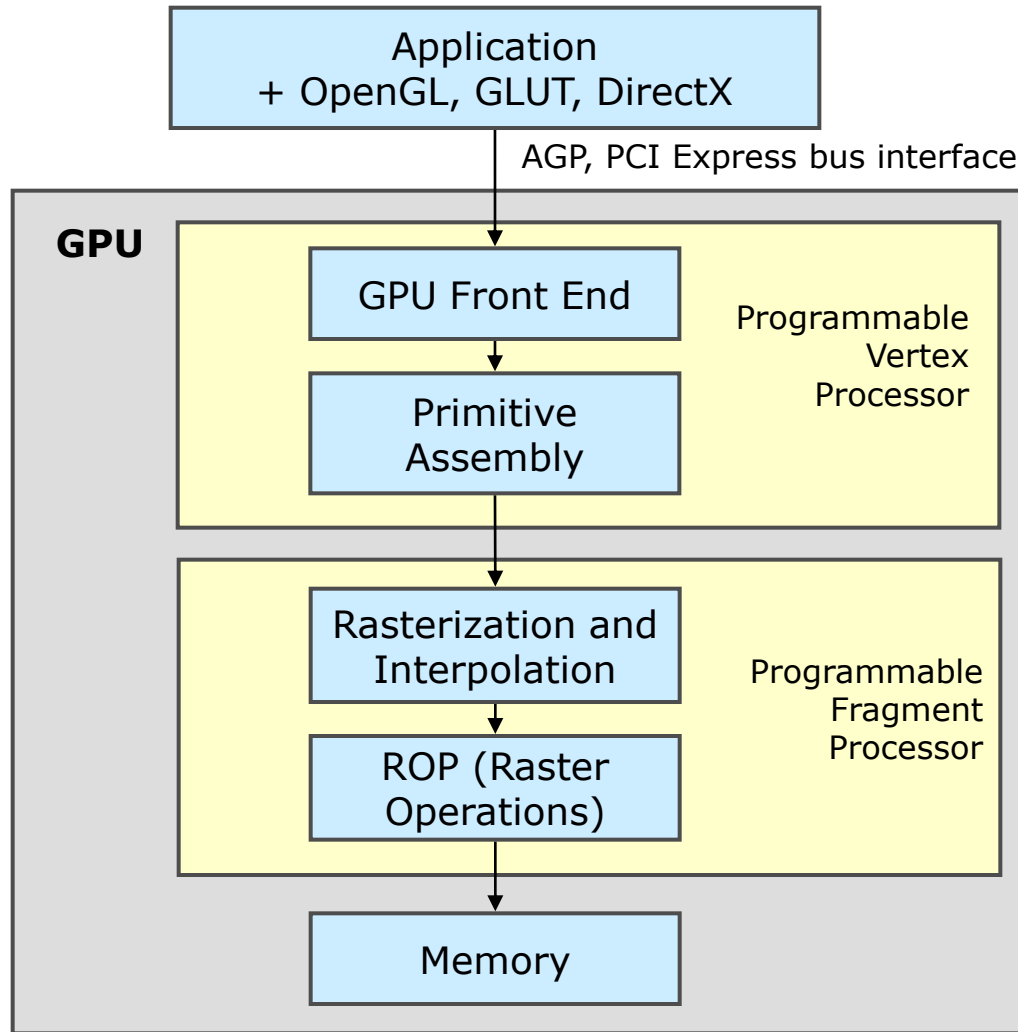


- ❑ Collision detection to determine which virtual objects collide.
- ❑ Only the colliding objects in the scene are passed to next pipeline stages.
- ❑ The haptics rendering pipeline has a less standardized architecture, i.e. it reflects the evolving state of the haptic technology.
- ❑ Compute friction forces (Hooke's law), the contact force has a spring like dependence on the degree of surface deformation. Other models – damping and friction forces.
- ❑ Simpler force models optimize the pipeline.
- ❑ Force smoothing – adjust the direction of the force vector, in order to avoid sharp transitions between polygonal surfaces. Force mapping projects the force to the characteristic of the haptic display system.
- ❑ Renders the touch feedback component of the simulation.
- ❑ Computed effects: surface temperature, vibrations – added to the force vector and sent to the haptic output display.

Classic GPU pipeline (Processor/function)



GPU pipeline (Function blocks)



GPU graphics pipeline

- GPU has evolved from *hardwired* implementation of the graphics pipeline to a *programmable* computational structure.

- Fixed-function units for transforming vertices and texturing pixels have been subsumed by a *unified grid of processors*, or *shaders*.

Ex. NVIDIA GeForce 3, launched in February 2001, introduced *programmable vertex shaders*. GeForce 3 introduced the reconfigurability in pixel processing, exposing the texturing hardware's functionality as a set of *register combiners* that could achieve novel visual effects.

- The ATI Radeon 9700 (July 2002) and NVIDIA GeForce FX (January 2003) replaced the register combiners with fully *programmable pixel shaders*. NVIDIA GeForce 8800 (November 2006), adds programmability to the primitive assembly stage, allowing developers to control how they construct triangles from transformed vertices.
- *Color*. The traditional graphics pipeline provided only 8-bit integers per color channel (range 0 to 255). The ATI Radeon 9700 increased the representable range of color to 24-bit floating point, and NVIDIA GeForce FX followed with both 16-bit and 32-bit floating point.

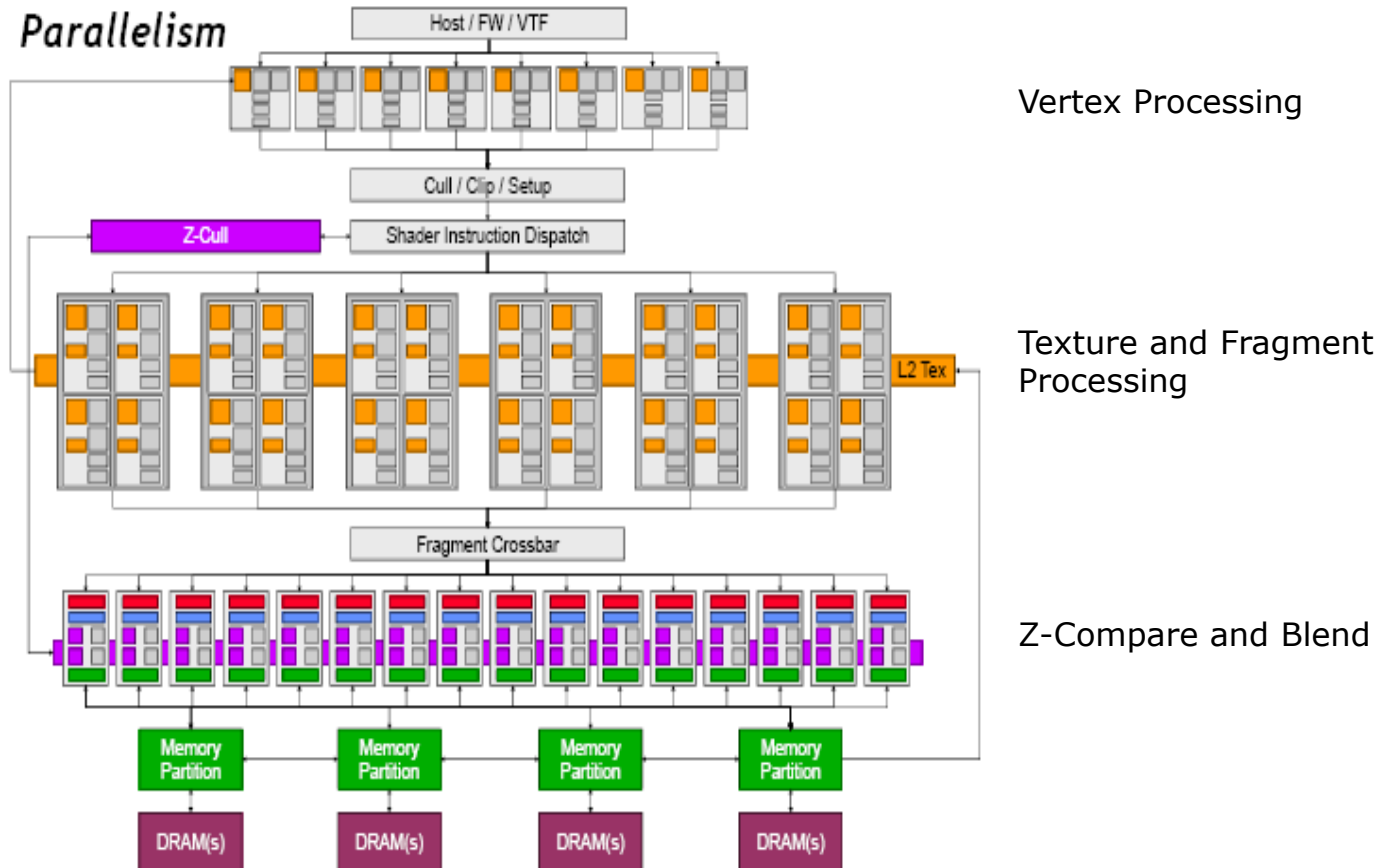
GPU graphics pipeline

- The number of the four-wide vector processors executing in parallel has increased from only four on GeForce FX to 16 on GeForce 6800 (April 2004) to 24 on GeForce 7800 (May 2005). The GeForce 8800 includes 128 scalar *shader processors (Streaming Processors)*.
- The NVIDIA GeForce 8800 GPU replaces the traditional graphics pipeline with a *unified shader architecture* in which vertices, triangles, and pixels recirculate through a set of programmable processors. The flexibility and computational power of these processors support the general-purpose computing tasks. A single GeForce 8800 chip achieves a sustained 330 billion floating-point operations per second (Gflops) on simple benchmarks

NVIDIA GeForce 7800

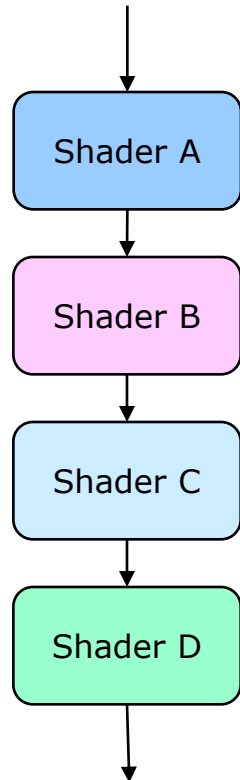
GeForce 7800 Parallelism

Figure courtesy of NVIDIA

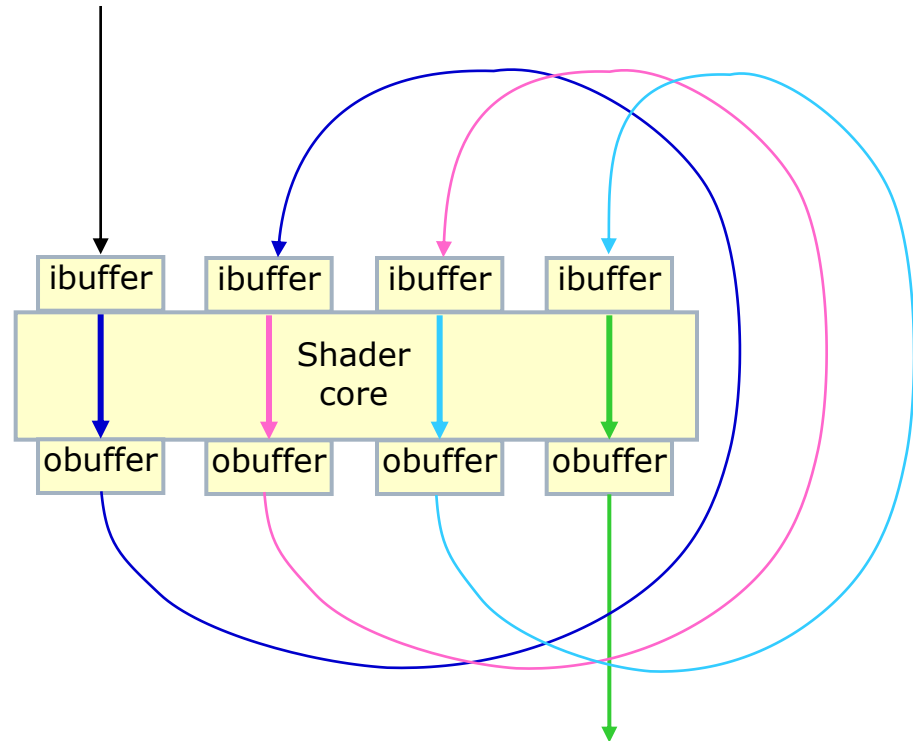


Classic vs. unified shader architecture

Discrete Design

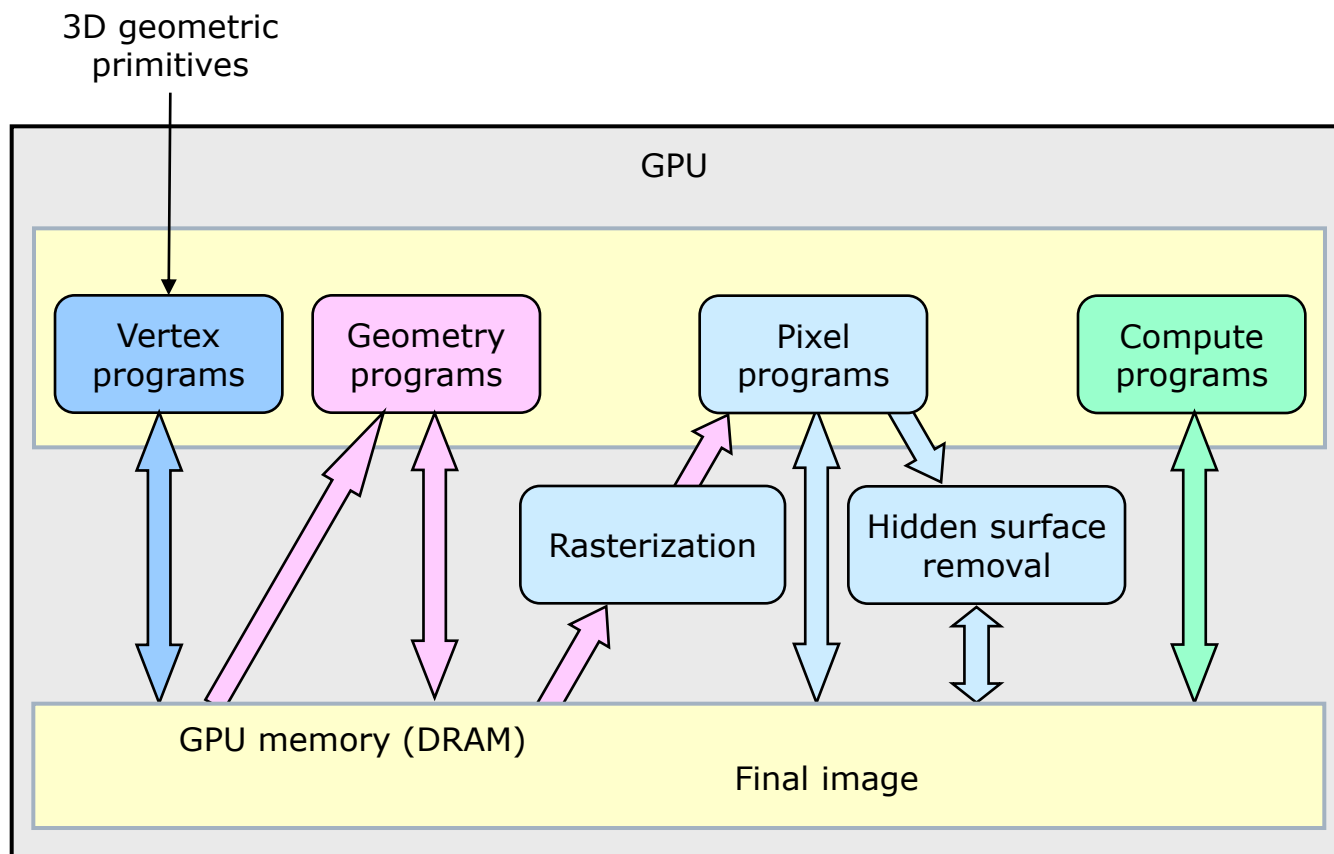


Unified Design



NVIDIA GeForce 8800 GPU Architecture Overview, November 2006 TB-02787-001_v01, <http://www.nvidia.com>

Unified shader architecture



Ref: David Luebke, NVIDIA Research, Greg Humphreys, University of Virginia, "How GPUs Work"

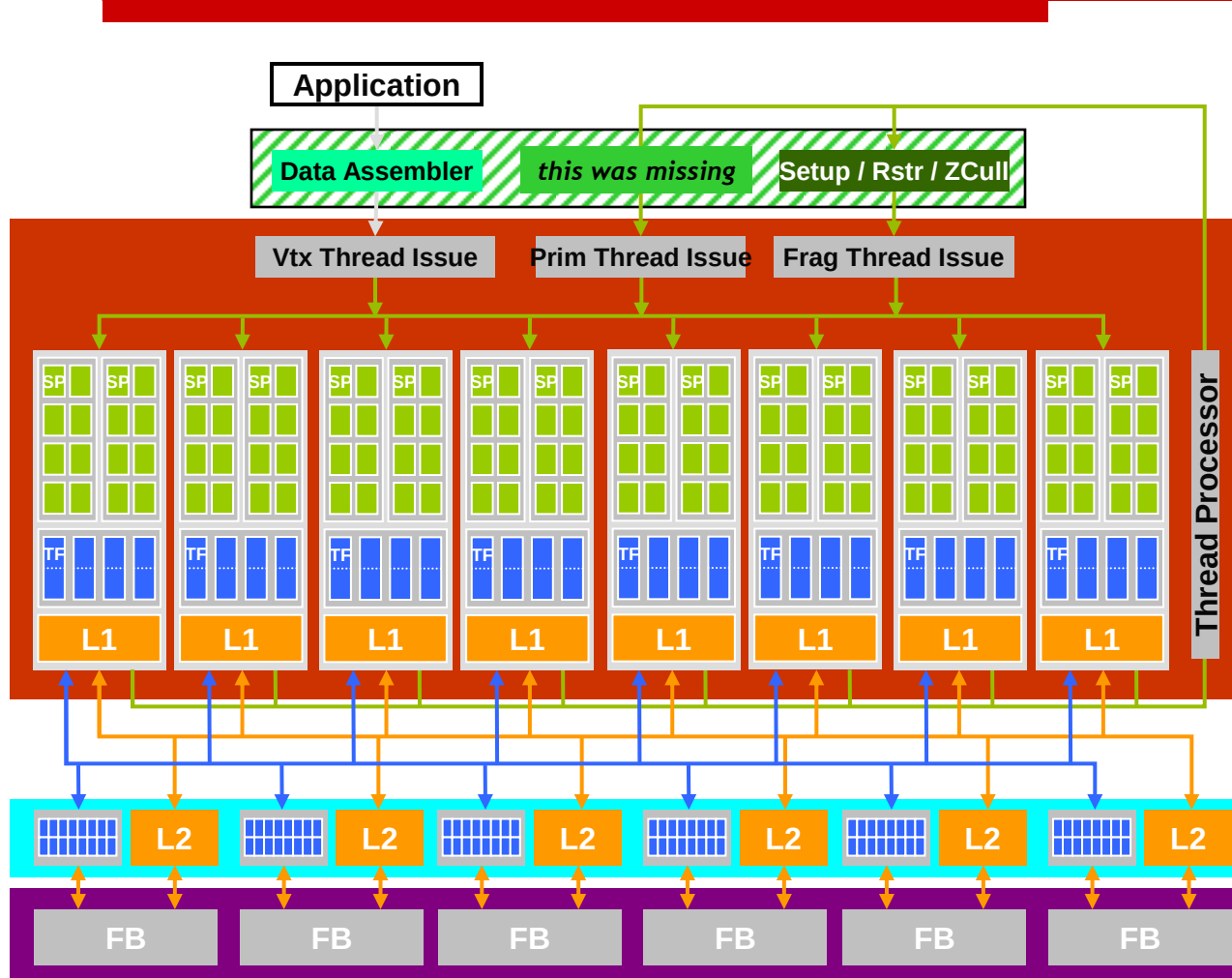
NVIDIA GeForce 8800

GeForce 8800 block diagram (NVIDIA Corporation 2006)

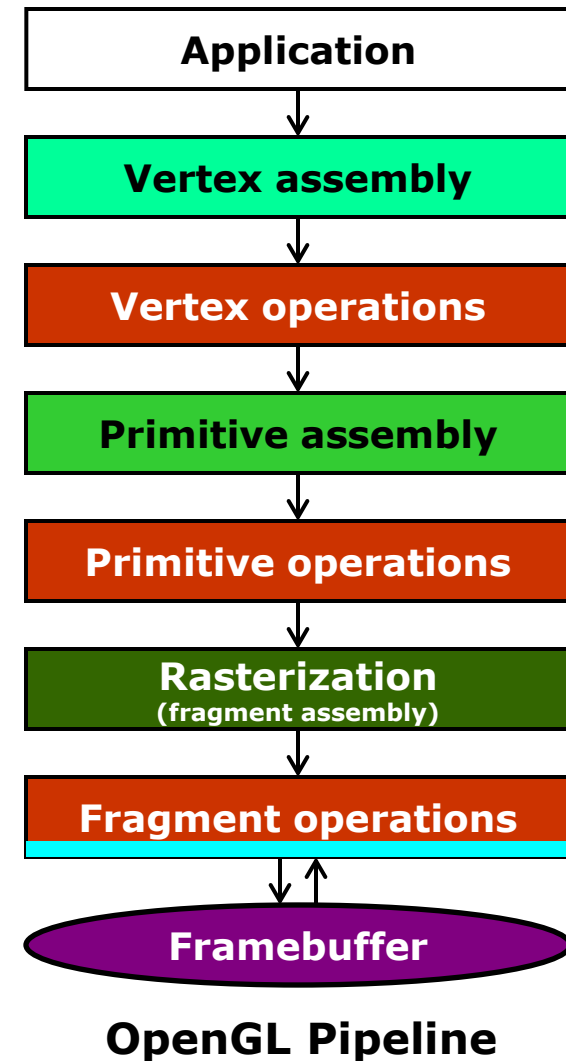


GeForce 8800	Core Clock (MHz)	Shader Clock (MHz)	Memory Clock (MHz)	Memory Amount	Memory Interface	Memory Bandwidth (GB/sec)	Texture Fill Rate (billion/sec)
Ultra	612	1500	1080	768 MB	384 bit	103.7	39.2
GTX	575	1350	900	768 MB	384 bit	86.4	36.8
GTS	500	1200	800	640 MB	320 bit	64	24

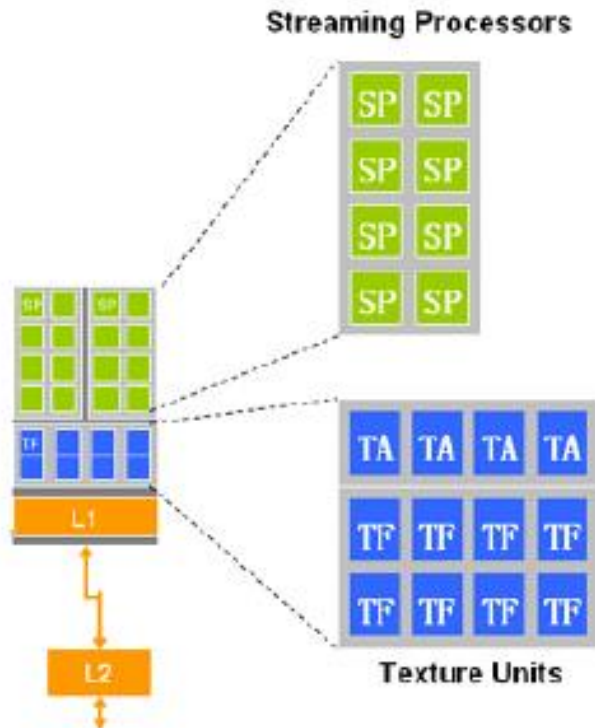
NVIDIA GeForce 8800



NVIDIA GeForce 8800



Streaming processors, texture units, and caches



SP – Stream Processors

TF – Texture Filtering Unit

TA – Texture Address Unit

L1/L2 – Caches

- Stream processors perform calculations on an input stream and produce an output stream that can be used by other stream processors. SPs can be grouped in close proximity, and in large numbers, to provide immense parallel processing power.
- SPs are a critical component of NVIDIA GigaThread technology, where thousands of threads can be in flight within a GeForce 8800 GPU at any given instant. SPs schedule and dispatch various types of threads for execution (e.g. pixel, vertex, geometry, and physics).

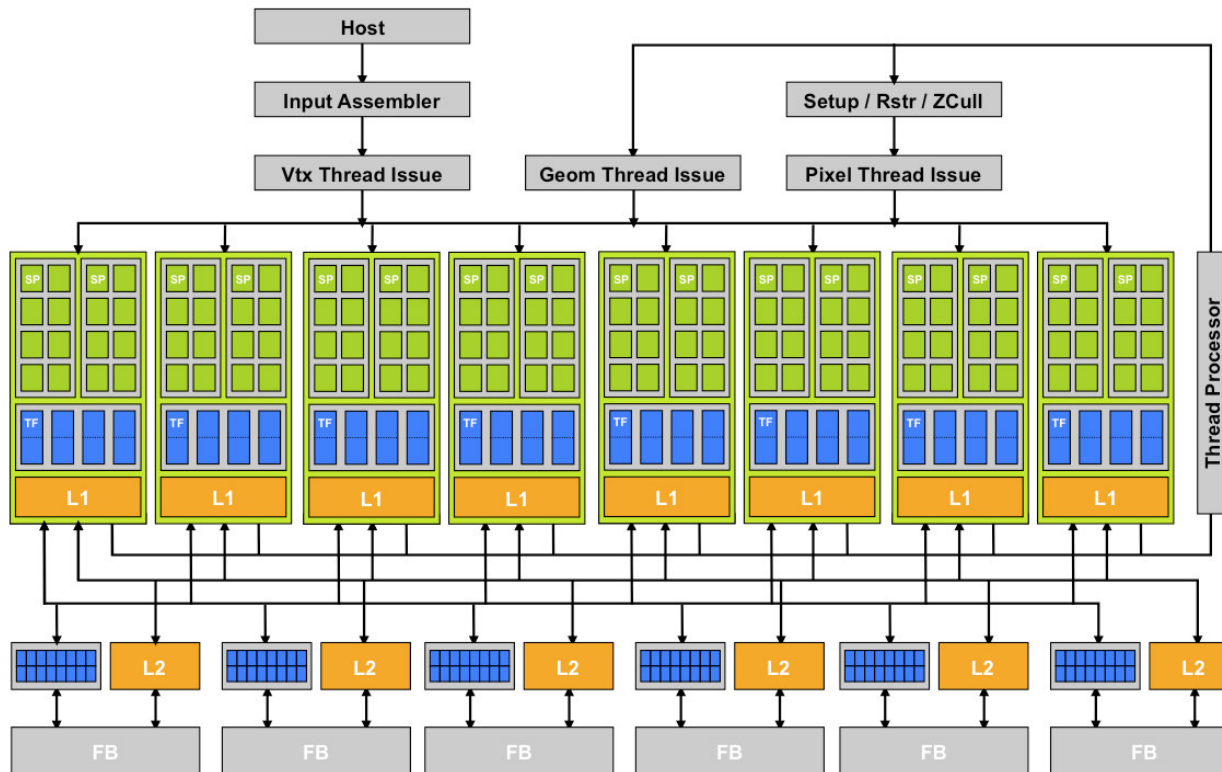
NVIDIA GeForce 8800 GTX



Picture taken from Wikipedia, the free encyclopedia.

GPGPU – General Purpose GPU

□ Modern architecture – NVIDIA G80 – Unified Shader



http://www.nvidia.com/content/PDF/fermi_white_papers/P.Glaskowsky_NVIDIAFermi-TheFirstCompleteGPUComputingArchitecture.pdf

□ GPGPU – Specialized languages (CUDA, OpenCL)

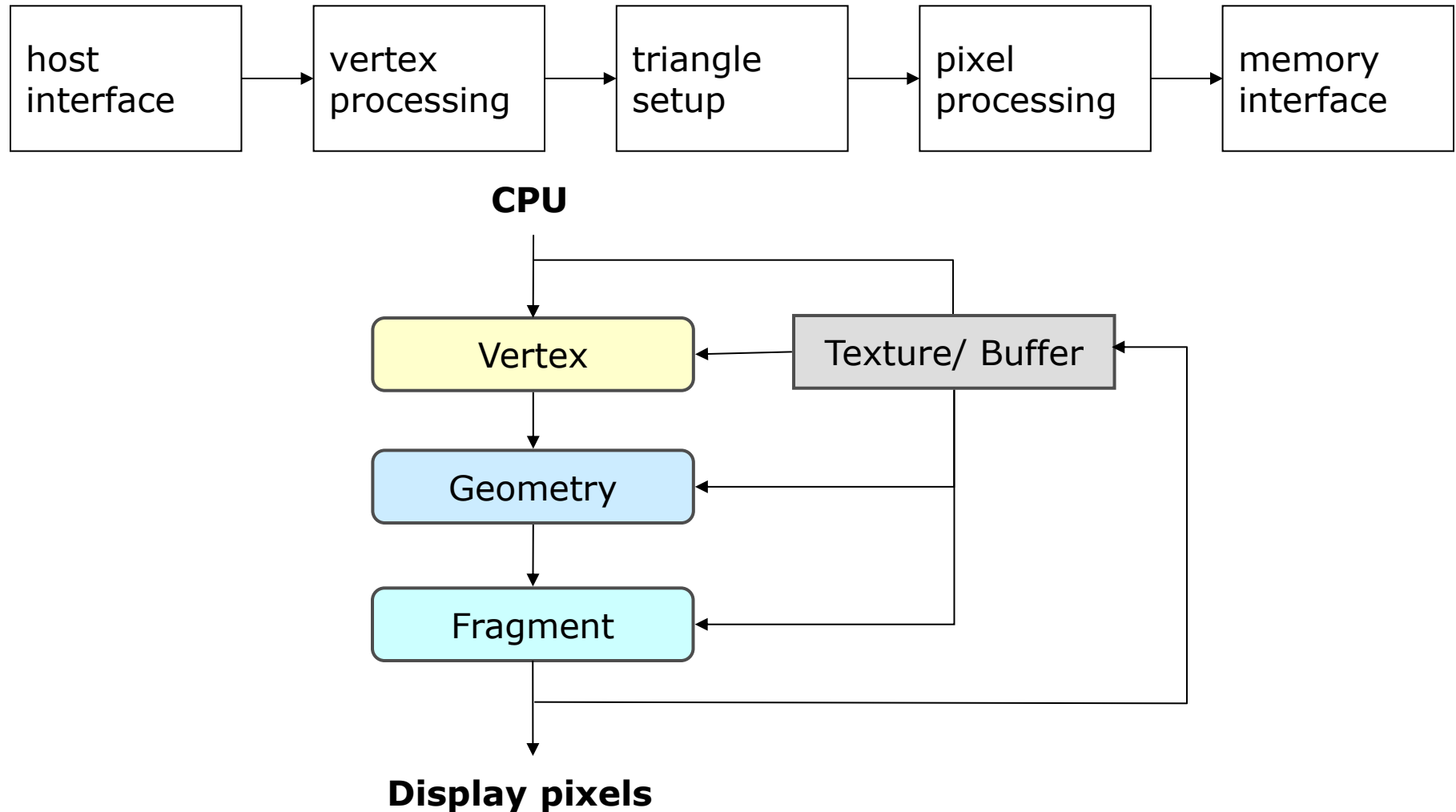
GPGPU

- GPGPU - General Purpose Graphics Processing Unit
- Application domains:
 - General computation
 - Linear algebra
 - Differential equations
 - Image processing and Computer vision
 - Pattern recognition
 - Classification

Shader languages

- ❑ Assembler
- ❑ Cg (NVIDIA)
C for Graphics
- ❑ GLSL (OpenGL)
OpenGL Shading Language
- ❑ HLSL (Microsoft/Direct3D)
High Level Shading Language

GPU computation pipeline (programmer's view)



Cg Language

- ❑ C for Graphics (Cg) is a high-level shading language developed by NVIDIA's open-source high-level shading language.
- ❑ Developed in collaboration with Microsoft for *programming vertex and pixel shaders*.
- ❑ Cg is based on the C programming language. New data types were added to make Cg more suitable for programming graphics processing units.
- ❑ Cg code is portable to a wide range of hardware and platforms
- ❑ Cg can be used with two APIs: OpenGL or DirectX. Each has its own set of Cg functions to communicate with the Cg program
- ❑ Cg compiler outputs DirectX or OpenGL shader programs.

Ref: Cg Homepage, http://developer.nvidia.com/page/cg_main.html

GLSL – OpenGL Shading Language

- ❑ GLSL (OpenGL Shading Language), also known as GLSLang, is a high level shading language based on the C programming language. It was created by the OpenGL ARB to give developers more direct control of the graphics pipeline without having to use assembly language or hardware-specific languages.
- ❑ The OpenGL Shading Language is actually two closely related languages. These languages are used to create shaders for the programmable processors contained in the OpenGL processing pipeline.
- ❑ The specific languages are referred by the name of the processor they target: vertex or fragment.
- ❑ The *vertex processor* is a programmable unit that operates on incoming vertices and their associated data. Compilation units written in the OpenGL Shading Language to run on this processor are called *vertex shaders*.
- ❑ The *fragment processor* is a programmable unit that operates on fragment values and their associated data. Compilation units written in the OpenGL Shading Language to run on this processor are called *fragment shaders*.

Ref: GLSL specification, <http://www.opengl.org/documentation/glsl/>

HLSL - High Level Shading Language

- High Level Shading Language (HLSL) is a proprietary shading language developed by Microsoft for use with the Microsoft Direct3D API.

It is analogous to the GLSL shading language used with the OpenGL standard. It is very similar to the NVIDIA Cg shading language, as it was developed alongside it.

- There are three HLSL program forms: *vertex shaders*, *geometry shaders*, and *pixel (or fragment) shaders*.
- A *vertex shader* can be used for:
 - Transform vertices to clip-space
 - Pass normal, texture coordinates or other vertex data to Pixel Shader
 - Transforming vectors to other spaces (e.g. texture space)
 - Calculate vertex lighting
 - Distort geometry (e.g. waves, fish-eye camera)
- The Pixel Shader can be used for:
 - Texturing objects
 - Pixel lighting
 - Pixel shadow
 - Environment mapping
 - Normal mapping (every pixel has its own normal)

Ref: High Level Shading Language for DirectX, [http://msdn2.microsoft.com/en-us/library/bb509561\(VS.85\).aspx](http://msdn2.microsoft.com/en-us/library/bb509561(VS.85).aspx)

Software technologies

- ❑ OS: Windows, Linux, Mac
- ❑ API: OpenGL, DirectX, CUDA C/C++, OpenCL , DirectCompute, CUDA Fortran, and others
- ❑ Language: Cg, GLSL, HLSL
- ❑ Compiler: Cg, OpenGL, DirectX, ASHLI
- ❑ Runtime: CgFX, OSG, ASHLI, other codes

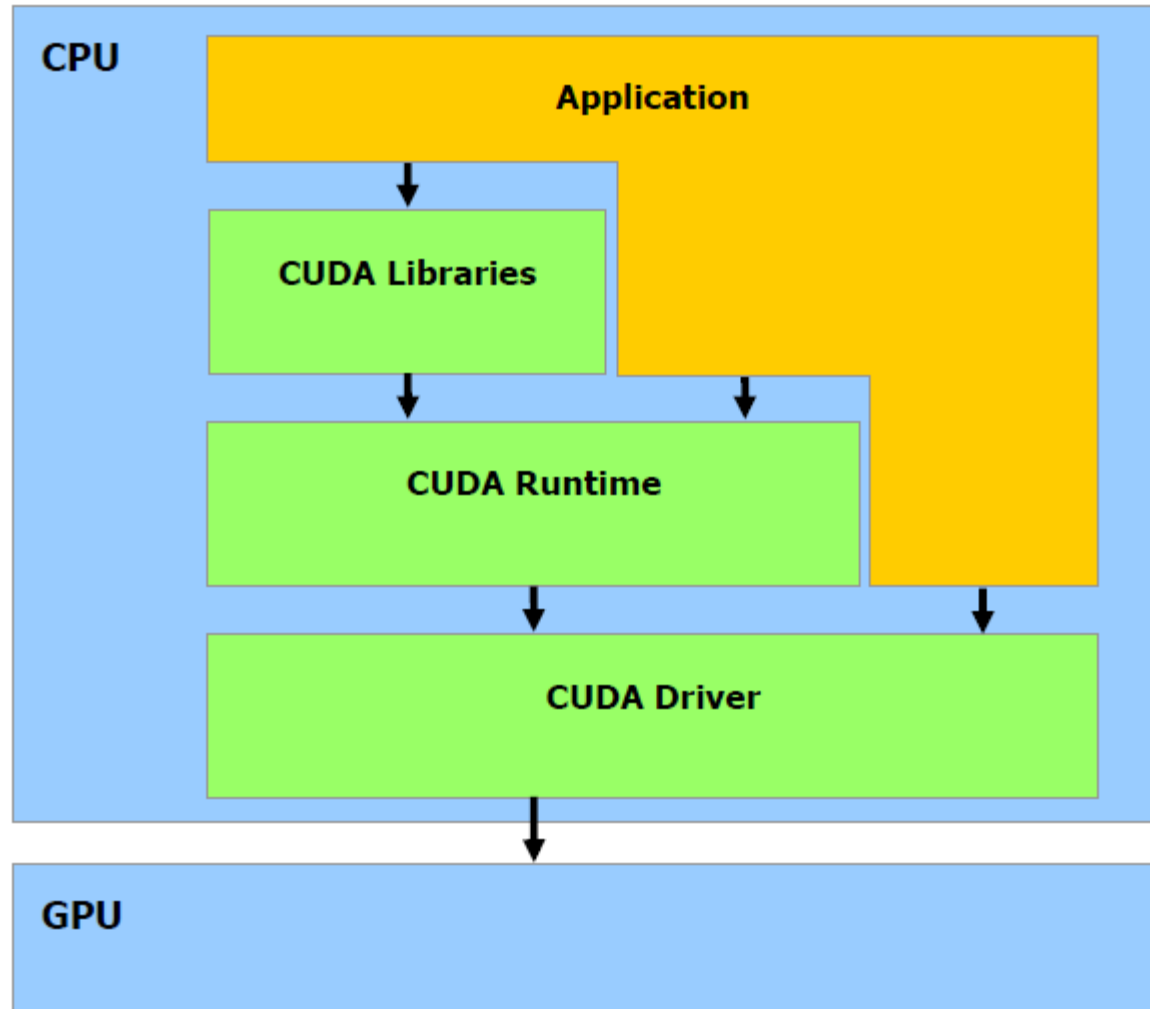
CUDA

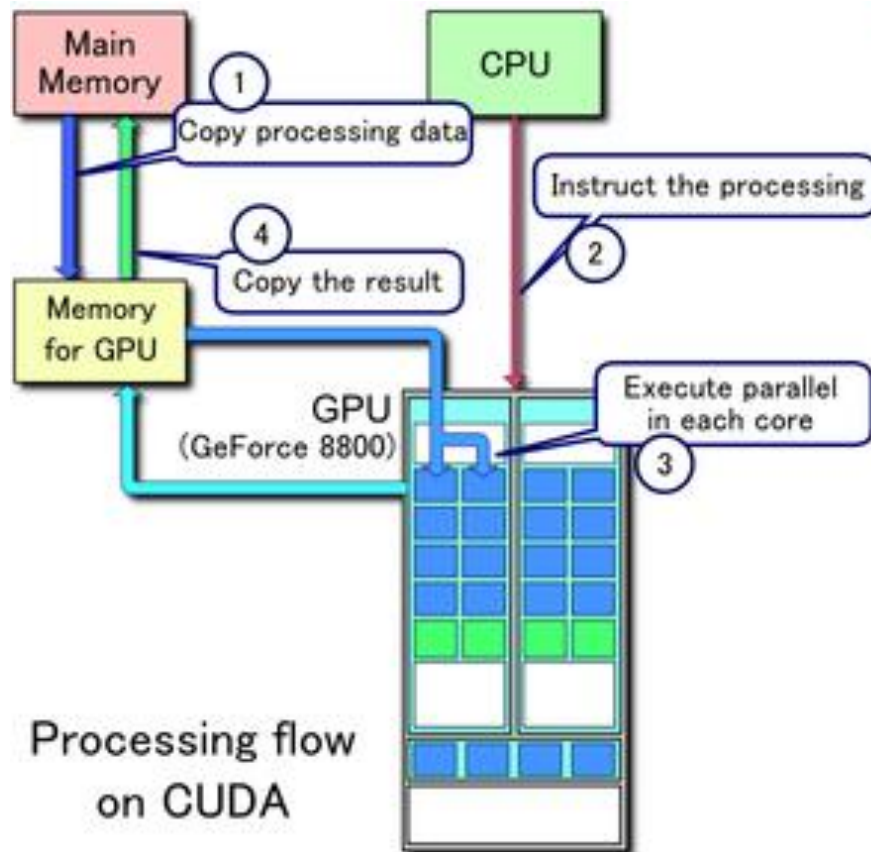
- ❑ CUDA - Compute Unified Device Architecture
- ❑ NVidia GPU cards support an API extension to the C programming language CUDA
- ❑ CUDA allows specified functions from a normal C program to run on the GPU's stream processors
- ❑ C programs become capable of taking advantage of the GPU's ability to operate on large matrices in parallel, while still making use of the CPU when appropriate
- ❑ CUDA is the first API to allow CPU-based applications to access directly the resources of a GPU for more general purpose computing without the limitations of using a graphics API

Reference:

1. CUDA site, http://www.nvidia.com/object/cuda_home.html
2. CUDA on Wikipedia, <http://en.wikipedia.org/wiki/CUDA>

CUDA Software Stack





Example of CUDA processing flow [Ref 2]:

1. Copy data from the main to the GPU memory
2. CPU instructs the process to GPU
3. GPU execute parallel in each core
4. Copy the result from GPU to the main memory

OpenCL

- ❑ OpenC - Open Computing Language
- ❑ Cross platform environment for general purpose computing on GPU
- ❑ Low-level API for heterogeneous computing that runs on CUDA architecture GPUs
- ❑ C-like programming language to work with massive parallel computing power of NVIDIA GPU's to create compelling computing applications
- ❑ OpenCL supports processors from other vendors
- ❑ First conformant GPU implementation for both Windows and Linux in 2009

Reference:

- <http://developer.nvidia.com/ocl>

OpenCL and CUDA

- ❑ OpenCL is portable between different graphics hardware
- ❑ CUDA is NVIDIA only
- ❑ CUDA is an independent language and is much more powerful and has a set of good tools. CUDA coding environment is almost like coding in C
- ❑ OpenCL is easier to use out of the box
- ❑ Both have extensive documentation and examples
- ❑ CUDA allows for greater control, hence can be better fine-tuned for higher performance

Reference:

- http://www.cse.scitech.ac.uk/disco/mew20/presentations/GPU_MattHarvey.pdf

Distributed visualization

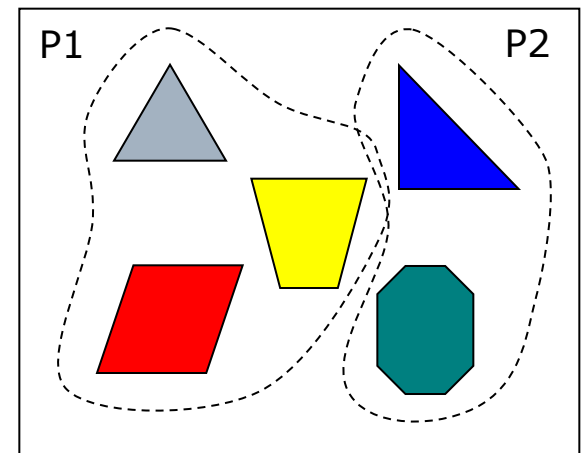
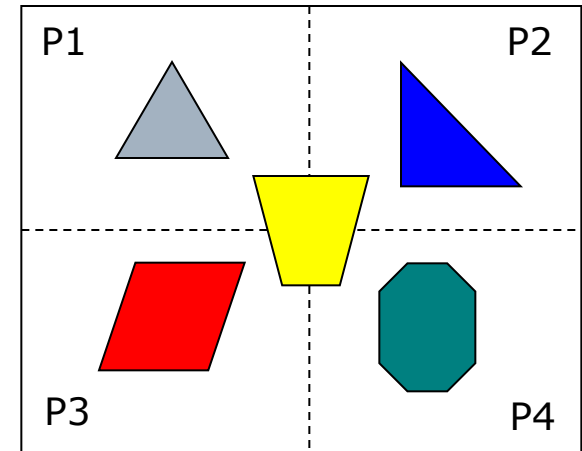
- Parallel and distributed computation in visualization works in:

1. Image space

- decomposition of a visualization task into a collection of sub-tasks, each responsible for a small portion of pixels in the visualization image to be synthesized.
- *sort-first* algorithms

2. Object space

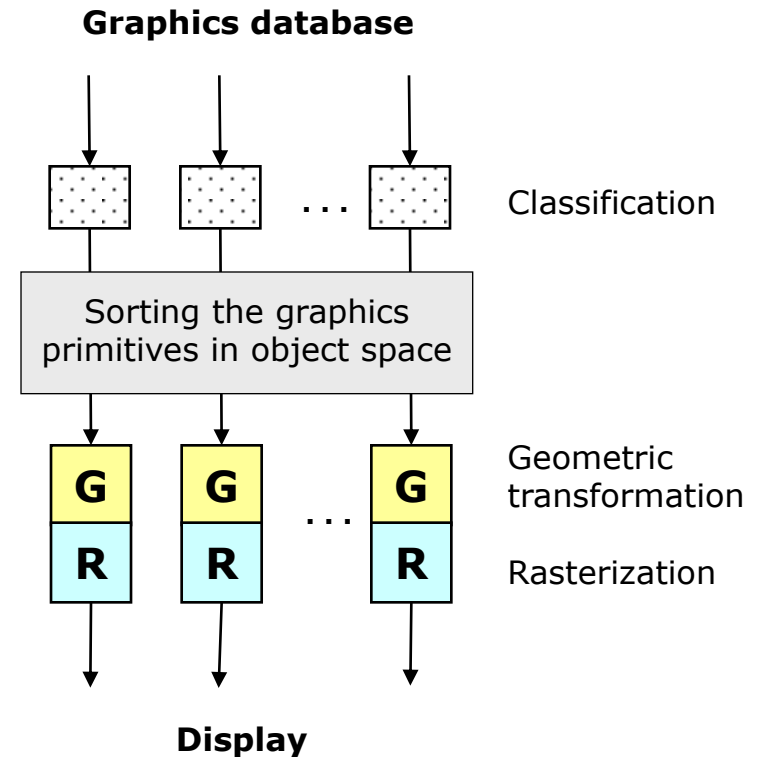
- divide input data into a collection of smaller components, each being processed by a computation node
- *sort-last* algorithms



Sort-first pipeline

□ Sort-first

- each processor is assigned a portion of the screen to render.
- classify the primitives according to their positions on the screen.
- during classification, the processors redistribute the primitives such that they all receive all of the primitives that fall in their respective portions of the screen.
- each processor performs the remaining transformation and rasterization steps for all of its resulting primitives.
- finished pixels are sent to one or more frame buffers to be displayed.

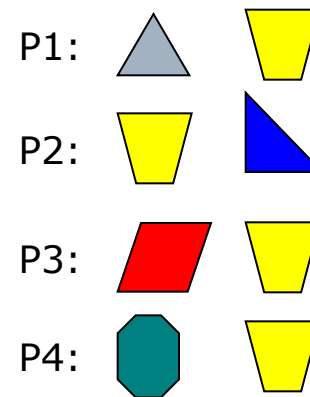
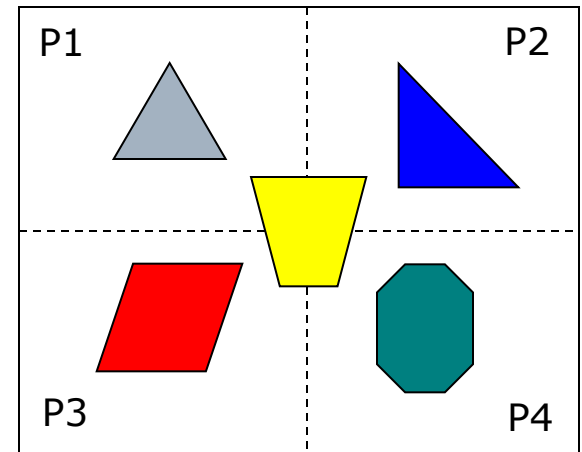


Sort first

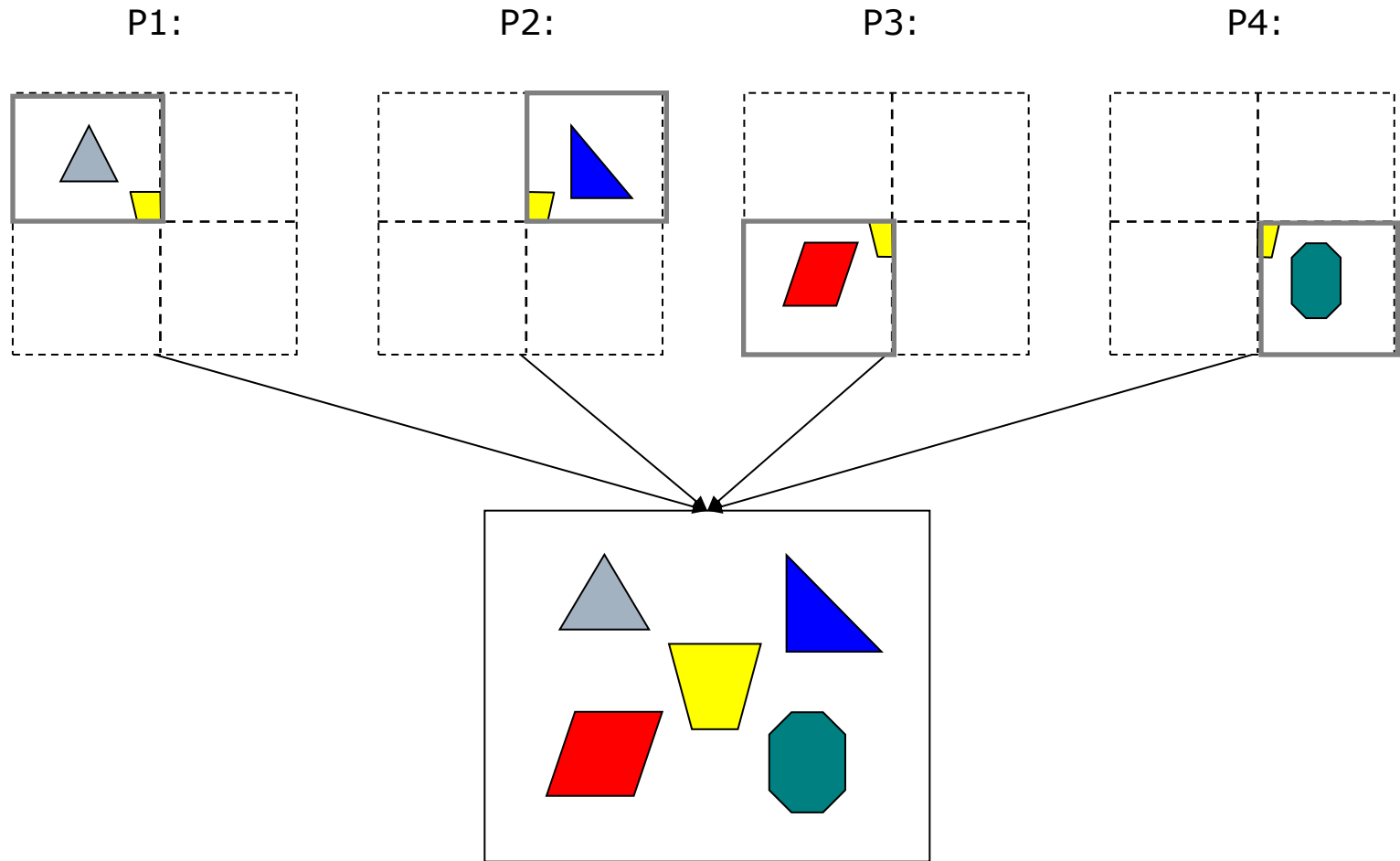
Distribute the primitives early in the rendering pipeline

Algorithm:

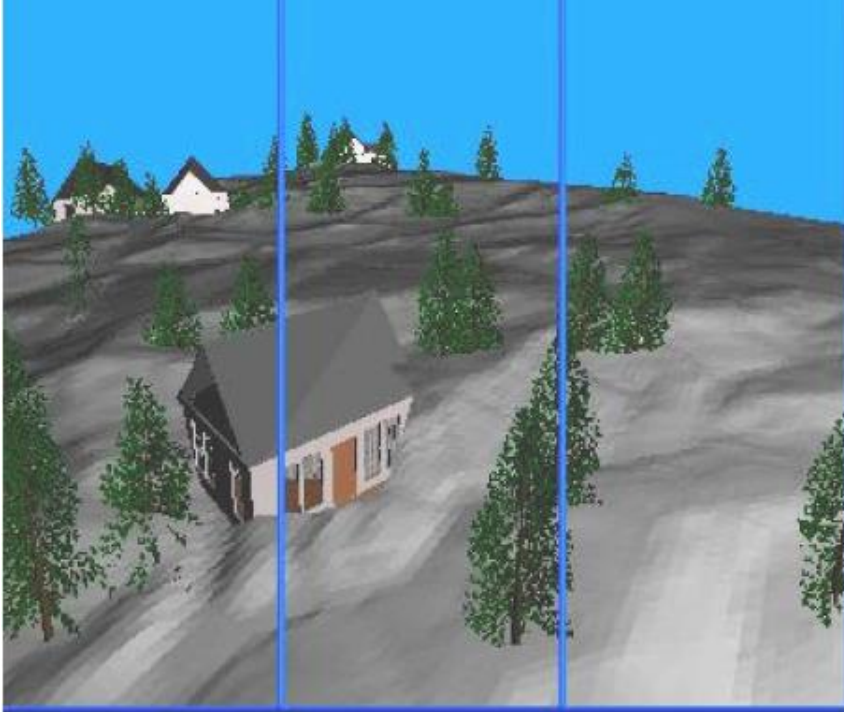
1. Subdivide the screen
2. Transform the primitives
3. Distribute the primitives
4. Each processor renders its own primitives
5. Does not require communication



Sort first



Sort first visualization

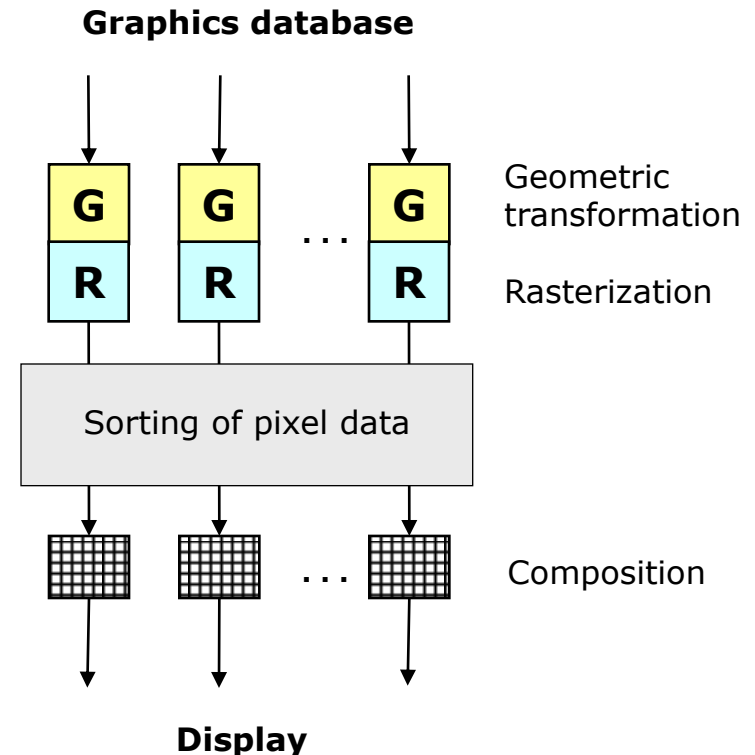


Bacu V., Muresan L., Gorgan D., Cluster Based Modeling and Visualization of Virtual Geographical Space. Workshop on Grid Computing Applications Development (GridCAD). Proceedings of the SYNASC 2008, IEEE Computer Press, pp. 416-421 (2008).

Sort-last pipeline

□ Sort-last

- each processor has a complete rendering pipeline and produces an incomplete full-area image by transforming and rasterizing its fraction of the primitives.
- these partial images are composited together, typically by depth sorting each pixel, in order to yield a complete image for the frame buffer.
- the composition step requires that pixel information (at least color and depth values) from each processor be sent across a network and sorted along the way to the frame buffer.

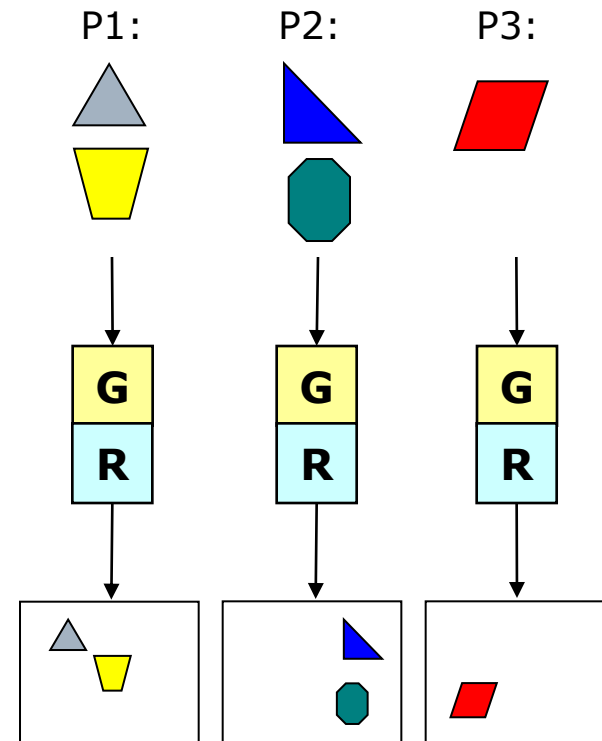
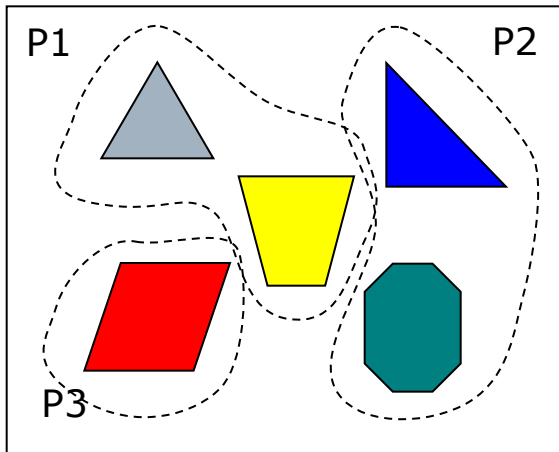


Sort last

Sort the primitives in the end of the pipeline

Algorithm:

1. Arbitrary assignment
2. Geometry processing
3. Rasterization

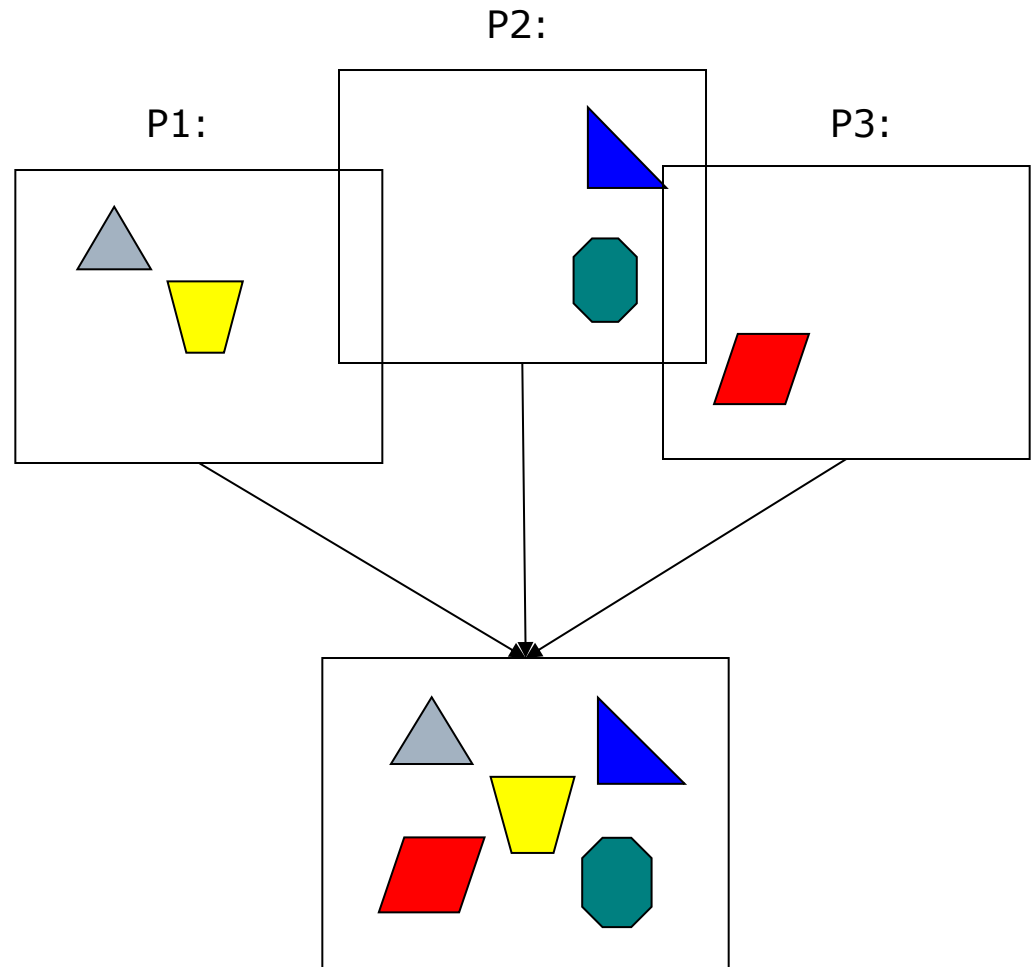
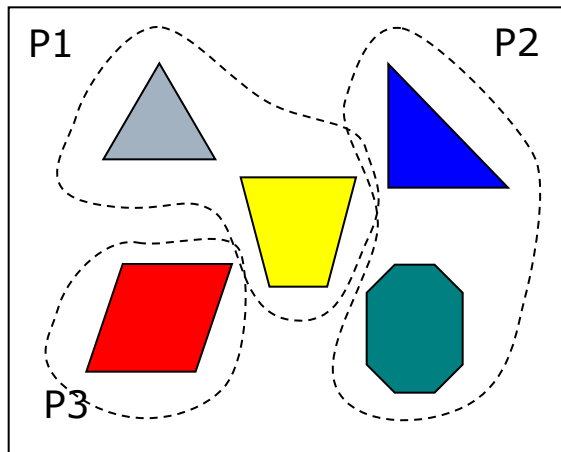


Sort last

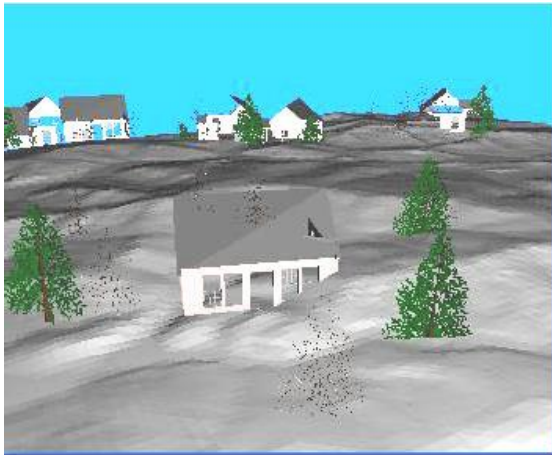
Sort the primitives in the end of the pipeline

Algorithm (cont):

4. Sort images by z
5. Composition/z-buffer



Sort-last visualization

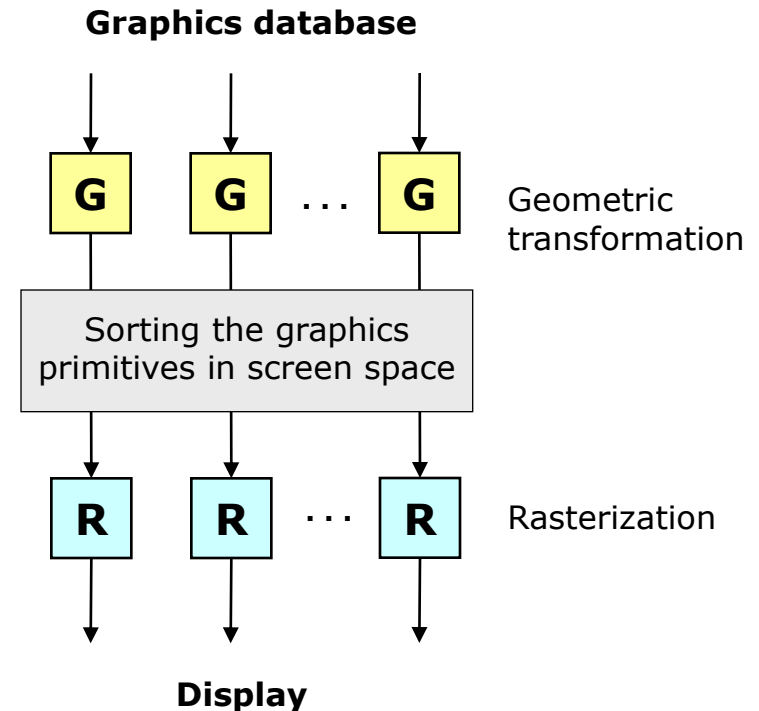


Bacu V., Muresan L., Gorgan D., Cluster Based Modeling and Visualization of Virtual Geographical Space. Workshop on Grid Computing Applications Development (GridCAD). Proceedings of the SYNASC 2008, IEEE Computer Press, pp. 416-421 (2008).

Sort-middle pipeline

□ Sort-middle

- there is a set of transformation processors and a set of rasterization processors.
- each rasterization processor is assigned a portion of the screen.
- to produce an image, each transformation processor completely transforms its portion of the primitives.
- the resulting primitive information is again classified by screen location and sent to the correct set of rasterization processors.
- after rasterization, finished pixels go to the frame buffer(s).



Conclusions

□ **Sort-first**

■ Frame-to-frame coherence

It takes advantage of frame-to-frame coherence. In an interactive application, the viewpoint changes very little from frame to frame, and thus the on-screen distribution of primitives does not change appreciably.

■ Reduced processor communication

Since primitives in a sort-first system are only transferred when they cross from one processor's screen region to another's, only a fraction of them will have to be communicated each frame. communication that does occur is typically fairly local; usually only "neighboring" processors will need to talk with each other.

■ Good scalability

It has good scalability in terms of the number of primitives it can handle.

■ Flexible rendering

in sort-first, once a processor has the correct set of primitives to render, only that processor is responsible for computing the final image for its portion of the screen. This allows great flexibility in terms of the rendering algorithms which may be used.

■ Very high resolution

since only finished pixels need to be sent to the frame-buffer, sort-first can easily handle very-high-resolution displays

■ Loadbalancing

Problem: loadbalancing. Because the on-screen distribution of primitives may be highly variable, also, managing a set of migrating primitives is a complex task

Conclusions

□ **Sort last**

- **Good scalability**

It offers excellent scalability in terms of the number of primitives it can handle.

- **Limited resolution**

Its pixel budget is limited by the bandwidth available at the composition stage.

- **Anti-aliasing**

It is a major problem for sort-last

- **Limited rendering algorithms**

Since visibility is not decided until after the composition stage, sort-last places limitations on the kinds of rendering algorithms which may be used. E.g. rendering transparent polygons

□ **Sort middle**

- **Limited scalability**

Any transformation processor must be able to talk to any rasterization processor → its scalability is limited.

- **Communications network**

Increasing the number of processors geometrically increases the demands on the communications network between them.

- **Load-balancing**

Load-balancing problems when the on-screen distribution of primitives is uneven. This will result in rasterization processors becoming unevenly loaded, and this in turn may degrade system performance

Load Balancing

- Strategy for mapping screen regions to processors
 - A simple strategy is to divide the screen into as many equal rectangles as there are processors and assigning them one-to-one can result in severe load-balancing problems. If the greatest concentration of primitives happens to fall into a single region, the parallel advantage of the system will be lost as the non-busy processors wait for the overloaded one to do its job.

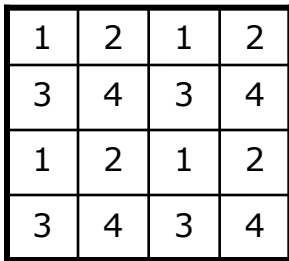


Fig 1. Static region assignment.

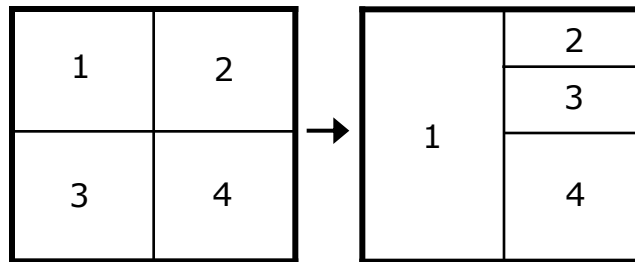


Fig 2. Roble's adaptive method. Regions 1 and 3 are combined. Processor 3 helps with the original region 2.

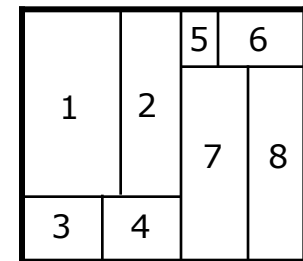


Fig 3. Whelan's adaptive method. The screen is recursively subdivided according to primitive centroids.

References

1. Burdea G.C., Coiffet P.: "Virtual Reality Technology", J.Wiley & Sons, Second Ed., 2003.
2. CS445: Intro Graphics, greg Humphreys, University of Virginia, Fall 2004.
3. Carl Mueller, The Sort-First Rendering Architecture for High-Performance Graphics. Department of Computer Science, University of North Carolina at Chapel Hill
4. R. Anderson, M. Carlsen, J. Postman, S. Rollins, GPU Technology, ECE 477 Spring 2007.
5. NVIDIA GeForce 8800 GPU Architecture Overview, November 2006 TB-02787-001_v01, <http://www.nvidia.com/page/geforce8.html>
6. GPUs, in ACM Queue, vol6, No.2, Murch/April 2008, <http://mags.acm.org/queue/20080304/>
7. Bacu V., Muresan L., Gorgan D., Cluster Based Modeling and Visualization of Virtual Geographical Space. Workshop on Grid Computing Applications Development (GridCAD). Proceedings of the SYNASC 2008, IEEE Computer Press, pp. 416-421 (2008).
8. Sabou, A., Gorgan, D., Parallel implicit time integration for particle-based models on graphics clusters. In Information and Communication Technology, Electronics and Microelectronics (MIPRO), IEEE 37th International Convention on, pp. 336-341 (2014).